

**Д.А. Короченцев, Л.В. Черкесова, Е.А. Ревякина,
Н.В. Болдырихин, О.А. Сафарьян**

Импортозамещающие технологии обеспечения информационной безопасности и защиты данных



**Ростов–на–Дону
2021**

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

КАФЕДРА «КИБЕРБЕЗОПАСНОСТЬ ИНФОРМАЦИОННЫХ СИСТЕМ»

**ИМПОРТОЗАМЕЩАЮЩИЕ ТЕХНОЛОГИИ
ОБЕСПЕЧЕНИЯ ИНФОРМАЦИОННОЙ
БЕЗОПАСНОСТИ И ЗАЩИТЫ ДАННЫХ**

УЧЕБНОЕ ПОСОБИЕ

**Ростов–на–Дону
ДГТУ
2021**

УДК 681.3
ББК
К47???

Рецензенты:

доктор технических наук, профессор, заслуженный деятель науки РФ,
заместитель директора по научной работе Ростовского филиала
Российской таможенной академии *Д.А. Безуглов*

доктор физико–математических наук, профессор,
заместитель директора по науке Ростовского научно–исследовательского
института радиосвязи *Д.Д. Габриэльян*

Авторы:

Д.А. Короченцев, Л.В. Черкесова, Е.А. Ревякина,
Н.В. Болдырихин, О.А. Сафарьян

К47? **Импортзамещающие технологии обеспечения информационной безопасности и защиты данных:** учеб. пособие / Д.А. Короченцев, Л.В. Черкесова, Е.А. Ревякина, Н.В. Болдырихин, О.А. Сафарьян. Донской гос. техн. ун–т.– Ростов–на–Дону: ДГТУ, 2021. – 400 с.

ISBN

В учебном пособии представлены различные направления информационной безопасности, для каждого из которых разработана импортзамещающая технология.

Предназначено для студентов укрупнённой группы специальностей 10.00.00 – «Информационная безопасность» и укрупнённой группы направлений подготовки 09.00.00 – «Информатика и вычислительная техника». Учебное пособие может быть полезно выпускникам – дипломникам или магистрантам, для написания выпускной квалификационной работы или для подготовки магистерской диссертации, а также для всех читателей, интересующихся технологиями импортозамещения в сфере кибербезопасности информационных систем.

УДК 681.3

Печатается по решению редакционно–издательского совета
Донского государственного технического университета

Научный редактор – доктор физико–математических наук, профессор
кафедры «Кибербезопасность информационных систем» ДГТУ Л.В. Черкесова

© Короченцев Д.А., Черкесова Л.В.,
Ревякина Е.А. и др., 2021

© Издательский центр ДГТУ, 2021

ISBN

ВВЕДЕНИЕ

"Внешнее вторжение в электронные системы в сфере обороны и государственного управления, жизнеобеспечивающей инфраструктуры и финансов, утечка электронных документов могут обернуться самыми тяжёлыми последствиями для страны".

"Очевидно, что устойчивая работа информационных систем, средств коммуникации и связи, их защищённость имеют для страны стратегическое значение. Это важный фактор обеспечения суверенитета, обороноспособности, безопасности государства, эффективного развития экономики, социальной сферы, государственного управления на базе передовых, в том числе цифровых, технологий".

*В.В. Путин на расширенном заседании
Совета Безопасности Российской Федерации.*

Национальная безопасность России во многом зависит от обеспечения защищённости информационной сферы общества в рамках государства. При этом роль обеспечения информационной безопасности, с учётом технического прогресса и глобализации, которая не только открывает новые возможности, но и создает дополнительные риски, будет только возрастать. В то время как сложившаяся геополитическая ситуация выявила значительную зависимость российских потребителей от информационных продуктов, производимых за рубежом, кибертерроризм, кибершпионаж, таргетированные атаки на инфраструктуру компаний и госструктур повлияли на принятие на законодательном уровне нормативных актов, касающихся импортозамещения в информационной сфере.

Концептуальным документом, определяющим государственную политику в сфере информационной безопасности, является Доктрина информационной безопасности Российской Федерации, утвержденная Президентом Российской Федерации 05.12.2016 г. № 646, согласно которой под информационной безопасностью Российской Федерации понимается «состояние защищённости личности, общества и государства от внутренних и внешних информационных угроз, при котором обеспечиваются реализация конституционных прав и свобод человека и гражданина, достойные качество и уровень жизни граждан, суверенитет, территориальная целостность и устойчивое социально–экономическое развитие Российской Федерации, оборона и безопасность государства».

Другим нормативным актом, указывающим на необходимость импортозамещения, является Стратегия развития отрасли информационных технологий в Российской Федерации на 2014 – 2020 годы и на перспективу до 2025 года, утвержденной Распоряжением Правительства РФ от 01.11.2013 г. № 2036–р, в которой предусмотрена разработка и запуск специальной программы импортозамещения продукции сферы информационных технологий для решения задач отдельных государственных структур и организаций (в том числе оборонно–промышленного комплекса), а также ряд иных мер, направленных на обеспечение информационной безопасности, таких как: стимулирование циркуляции данных «облачных» сервисов внутри страны, обеспечение стабильности развития отечественной отрасли информационных технологий и её суверенности в долгосрочной перспективе, за счёт создания условий для развития в стране глобальных отечественных лидеров этой области.

Ограничение закупок иностранного программного и аппаратного обеспечения государственными и муниципальными заказчиками было введено Постановлением Правительства Российской Федерации от 16.11.2015 г. №1236 «Об установлении запрета на допуск программного обеспечения, происходящего из иностранных государств, для целей осуществления закупок для обеспечения государственных и муниципальных нужд».

С момента появления термина «импортозамещение» рынок российских технологий значительно видоизменился. Реестр отечественного ПО с каждым годом увеличивается. Современный рынок наших ИБ–продуктов уже может предложить решения, позволяющие обеспечить защиту и государственных интересов и обезопасить бизнес коммерческих компаний. Раньше заказчики с высоким уровнем зрелости служб ИТ / ИБ не рассматривали в принципе российских вендоров, а сейчас они всё чаще соглашаются протестировать отечественные решения, и нередко в конечном итоге выбор падает на них. При этом предлагаемые отечественные решения перестали уступать по функционалу и уровню сервиса зарубежным аналогам, и в сочетании с рублевым прайс–листом выглядят для российских заказчиков привлекательно.

Еще одно преимущество отечественных решений – они создаются с учётом российской специфики. Наиболее конкурентоспособны решения отечественных производителей в части антивирусной защиты (AV), защиты Web–приложений (WAF), защиты от утечек конфиденциальной информации (DLP), защиты SCADA–систем, где понимание отечественных условий работы особо важно.

Рынок SIEM–систем, межсетевых экранов, систем обнаружения / предотвращения вторжений (IPS/IDS), систем управления учётными данными (IDM), систем управления мобильностью предприятия (EMM), защиты от целенаправленных атак, систем контроля привилегированных пользователей (PAM) и ряд других систем остается за зарубежными производителями, но доля присутствия отечественных решений неуклонно растёт.

Наиболее активный интерес к теме информационной безопасности сегодня проявляют государственные организации и предприятия промышленного комплекса. И политика импортозамещения, безусловно, сыграла в этом не последнюю роль. Кроме того, нужно понимать, что наиболее широкий выбор российских продуктов сегодня характерен для тех средств обеспечения информационной безопасности, которые давно отмечены регуляторами в качестве обязательных и рекомендуемых. Это означает, что, рынок ИБ оказался более подготовленным к импортозамещению, чем рынок ИТ в целом. Наиболее широкий выбор российских продуктов сейчас наблюдается в области антивирусной защиты, DLP–систем, средств защиты от несанкционированного доступа, анализа защищенности, средств криптографической защиты информации и др.

Безусловно, за последние несколько лет значительно увеличился спрос на средства защиты информации отечественного производства. Такая тенденция характерна как для коммерческих, так и государственных предприятий. Необходимо также отметить, что на фоне увеличения спроса, на рынке стали появляться новые российские продукты и решения, которые в некоторых областях вполне могут конкурировать с западными аналогами.

ГЛАВА 1

ИССЛЕДОВАНИЕ СОВРЕМЕННЫХ КРИПТОГРАФИЧЕСКИХ ПРОТОКОЛОВ И ВОЗМОЖНОСТЕЙ ИХ ИМПОРТОЗАМЕЩЕНИЯ

ОСНОВНЫЕ ПОЛОЖЕНИЯ

На сегодняшний день вопрос организации защиты и безопасности данных стоит имеет первостепенное значение. Прежде всего это связано с резкой активностью киберпреступников и ростом киберугроз. Согласно статистике «Лаборатории Касперского» больше всего заражений и атак выявлены в России, Германии, Бразилии, Китае и США [1].

В связи с этим, создаются более усовершенствованные криптографические протоколы и средства защиты данных, что вызывает интерес у злоумышленников и конкурентов, побуждая тем самым их создавать и осуществлять более мощные и совершенные кибератаки для дестабилизации компьютерных систем государственных учреждений, банковских, деловых и финансовых центров.

Разработка квантовых компьютеров, которые основаны на законах квантовой механики, привело появлению и развитию квантовых и постквантовых криптографических алгоритмов. На данный момент ведущие страны, университеты, компании и корпорации активно занимаются квантовыми разработками. Так, например, в январе 2019 г. компания «Ростелеком» успешно провела второй этап испытаний отечественных оборудования и решений для квантовой защиты передачи данных [2]. В январе 2019 г. корпорация IBM представила компактный модульный квантовый компьютер Q System One, разработанный для научного и коммерческого применения. Работают над квантовыми технологиями такие крупные корпорации, как NASA, Microsoft, Google, IBM и другие ведущие компании и университеты всего мира [3].

Российские разработчики в области квантовых технологий, такие как ООО «С–Терра СиЭсПи», университет ИТМО в Санкт–Петербурге, Национальный исследовательский технологический университет МИСиС, Российский Квантовый Центр и др. утверждают, что квантовое шифрование – актуальнейшая задача, которая позволит России соблюдать свои национальные интересы и сохранять суверенитет в сфере кибербезопасности в эпоху кибернетических войн.

Угроза суверенитета России в сфере кибербезопасности стала более ощутимой после введения западными странами санкций против России в 2014 г. В связи с этим, Президентом Российской Федерации была поставлена задача к 1 октября 2014 г. разработать собственную национальную стратегию в области импортозамещения и, в частности, в сфере высоких технологий [4].

Согласно оценке Минпромторга России, на 2014 г. доля используемого в России зарубежного ПО составляла 67%, а аппаратной части – 90%.

Вследствие ввода технологических санкций против России, в сфере информационных технологий возникли следующие угрозы:

- отключение критически важных данных;
- сложности с оборудованием связи и микроэлектроникой для специализированных устройств в ближайшие 3 – 5 лет;
- возврат к практике широкого использования нелегального ПО; использование механизмов «жучков» и «закладок», способных по сигналу извне парализовать ПО на стратегических объектах.

Предложенное решение Минкомсвязи по данному спектру проблем – переход на сертифицированное отечественное аппаратное обеспечение и использование нейтрального ПО такого как Oracle, IBM, VMware, Microsoft и Symantec.

Поскольку информационное воздействие касается всех сфер жизни общества, то защита её от посягательств извне обретает особую важность. В этом случае информационная безопасность, сохранение секретности встает по важности наряду с такими вопросами как обеспечение безопасности государства в целом. На основании этого можно с уверенностью говорить о том, данная проблематика очень актуальна и требует глубокого исследования.

Цель исследования: исследование современных криптографических протоколов, обоснование возможности их импортозамещения, и реализация нового разработанного криптографического протокола, совместимого с доквантовыми, квантовыми и постквантовыми протоколами.

Объект исследования: современные отечественные и зарубежные доквантовые, квантовые и постквантовые криптографические протоколы.

Предмет исследования: исследование недостатков протоколов и разработка альтернативного защищенного протокола.

Для достижения поставленной цели *решаются следующие задачи:*

- изучение различных криптографических протоколов;
- исследование алгоритмов работы протоколов и области их применения согласно каждой классификации;
- проведение криптоанализа криптографических протоколов;
- разработка программного средства, реализующего защищенный протокол;
- криптоанализ и надежность работы разработанного протокола.

1. НАЗНАЧЕНИЕ КРИПТОГРАФИЧЕСКОГО ПРОТОКОЛА

В современной криптографии большое внимание уделяется не только созданию и исследованию шифров, но и разработке новых криптографических протоколов, как доквантовых, так и квантовых и постквантовых.

Протоколом является последовательность действий, исполняемых двумя или более сторонами, спроектированная для решения какой-либо задачи.

Криптографическим протоколом называется такой протокол, в котором применяются криптографические методы или шифры, предназначенные для защиты информации. В основе протокола лежит набор правил, регламентирующих применение криптографических преобразований и алгоритмов в информационных процессах. Любой криптографический протокол специализирован для решения конкретной задачи. Каждый протокол имеет следующие свойства [5]:

- непротиворечивость;
- упорядоченность (каждое действие должно выполняться строго в свою очередь, и только по окончании предыдущего действия);
- полнота (для каждой ситуации, которая может произойти в процессе работы протокола, должно быть предусмотрено соответствующее действие).

Члены протокола обязаны знать его содержание и осуществлять целиком все без исключения его правила. Участниками криптографического протокола, как правило, считаются абоненты определенной системы связи. Члены протокола имеют права не доверять друг другу. По этой причине криптографические протоколы обязаны защищать их участников не только лишь от внешнего противника, но и от нечестных действий партнеров.

Рассмотрим назначение некоторых видов протоколов:

1.1 Протоколы конфиденциальной передачи сообщений. Эта задача состоит в следующем: имеются два участника протокола, какие считаются абонентами сети связи. Члены этой сети объединены некоторой линией связи, согласно которой возможно посылать сообщения в обе стороны. Линию связи способен контролировать противник. У одного из абонентов хранится секретное сообщение m , и цель состоит в том, чтобы это сообщение секретным способом предоставить второму абоненту. Протоколы данного типа, вероятно, возникли раньше других криптографических протоколов потому, что задача секретной или конфиденциальной передачи информации – это исторически первая проблема, которая решалась криптографическими методами.

1.2 *Протоколы аутентификации и идентификации.* Они предусмотрены с целью предотвращения допуска к определенной информации лиц, не являющихся её пользователями, а также предотвращения доступа пользователей к ресурсам, на которые у них отсутствуют полномочия. Общая область применения – организация доступа пользователей к ресурсам большой информационной системы.

1.3 *Протоколы распределения ключей* нужны с целью предоставления секретных ключей участникам сеанса обмена зашифрованными сообщениями.

1.4 *Протоколы электронной цифровой подписи* дают возможность устанавливать под электронными документами подпись, схожую с обыкновенной подписью на бумажных документах. Вследствие выполнения протокола электронной цифровой подписи, к передаваемым данным добавляется уникальное числовое дополнение, позволяющее проверить её авторство.

1.5 *Протоколы обеспечения неотслеживаемости («Электронные деньги»).* Под электронными средствами в криптографии подразумевают электронные платежные средства, обеспечивающие неотслеживаемость, то есть невозможность для злоумышленника отследить ресурс передачи данных.

1.1 Требования к безопасности протокола

Чтобы передаваемая информация не попала в руки злоумышленника, необходимо строго соблюдать требования к безопасности используемых протоколов. Криптографическая система может обеспечить различные функции безопасности, для реализации которых применяются разнообразные криптопротоколы.

Свойств, определяющих безопасность криптографического протокола, также достаточно много. Как правило, свойства протоколов, определяющих их стойкость к различным кибератакам, выражают как цели или требования к протоколам. Наиболее полное и современное толкование этих целей даётся в документах международной организации Internet Engineering Task Force (IETF).

Под свойствами (целями и требованиями) безопасности в документах IETF на данный момент понимаю следующие 20 целей, сгруппированные в 10 групп, которые представлены в таблице на рисунке 1.1.

Представленные группы характеризуются следующим образом [6]:

1. *Аутентификация (нешироковещательная)* – как проверка идентичности, объявленной участником либо субъектом системы, в качестве которого способна выступить одна из сторон коммуникации, либо ресурс определенных сведений. Аутентификация обычно является односторонней. Взаимная аутентификация должна осуществляться в обоих направлениях.

№	Код	Название
1	G1	Аутентификация субъекта
	G2	Аутентификация сообщения
	G3	Защита от повтора
2	G4	Неявная (скрытая) аутентификация получателя
	G5	Аутентификация источника
3	G6	Авторизация (доверенной третьей стороной)
4	G7	Аутентификация ключа
	G8	Подтверждение правильности ключа
	G9	Защищенность от чтения назад
	G10	Формирование новых ключей
	G11	Защищенная возможность договориться о параметрах безопасности
5	G12	Конфиденциальность
6	G13	Обеспечение анонимности при прослушивании (несвязываемость)
	G14	Обеспечение анонимности при работе с другими участниками
7	G15	Ограниченная защищенность от атак типа отказ в обслуживании
8	G16	Неизменность отправителя
9	G17	Подотчетность
	G18	Доказательство отправки
	G19	Доказательство получения
10	G20	Безопасное временное свойство

Рисунок 1.1 – Свойства безопасности протокола

2. *Аутентификация при рассылке по многим адресам или при подключении к службе подписки / уведомления* – это условие аутентификации для групп участников с одним источником и большим количеством потенциальных получателей (широковещательная передача) или ресурс и служба, которые отправляют сведения подключенным, а также авторизованным пользователям.

3. *Авторизация (доверенной третьей стороной)* – уполномоченная третья сторона представляет одного субъекта *В* иному субъекту *А*. В следствии этого данный субъект *А* приобретает гарантии того, что субъект *В* удостоверен с помощью уполномоченной третьей стороны, а также авторизован (наделен правами) в необходимом для протокола значении.

4. *Свойства совместной генерации ключа.*

5. *Конфиденциальность.*

6. *Анонимность.* Большая часть протоколов не предоставляют свойства анонимности, поскольку каждая взаимодействующая сторона желает знать, с кем она взаимодействует при формировании ключа.

7. *Ограниченная защищенность DOS-атак.*

8. *Инвариантность отправителя* – участник приобретает гарантии того, что ресурс сообщений никак не менялся с начала выполнения сеанса, хотя установление идентичности данного источника для получателя не важно.

9. *Невозможность отказа от ранее совершенных действий* – устранение той вероятности, что реальный участник сеанса может категорически отказаться от реально совершенного им действия.

10. *Безопасное временное свойство* – применяя два оператора временной логики (линейной временной логики), непосредственно операторы «всегда» и «когда-то в прошлом», возможно формализовать качества вида: «Для каждого достижимого состояния, обладающее свойством p , ранее было состояние со свойством q ».

Теперь рассмотрим свойства каждой подгруппы.

G1 (аутентификация субъекта) – это проверка с доказательством подлинности одной из сторон наличия либо возможности идентичности второй стороны, участвующей в выполнении протокола, а также того, что она на самом деле берет на себя выполнение протокола и что она действительно принимает участие в выполнении текущего сеанса протокола.

G2 (аутентификация сообщения) – состоит в обеспечении аутентификации источника данных, а также целостности передаваемого сообщения. Аутентификация источника данных означает, что протокол обязан гарантировать, что полученные данные или сообщение были созданы определенным участником в определенный момент времени, предшествующий получению сообщения, и что эта информация не была искажена или подделана, но без предоставления гарантий однозначности и своевременности.

G3 (защита от повтора) – это обеспечение гарантий одним из участников того, что аутентифицированные данные не являются старыми (были сгенерированы в данном сеансе протокола, либо в течении известного промежутка времени, либо сообщение не было принято ранее).

G4 (неявная, скрытая аутентификация получателя) – протокол обязан обеспечивать средства гарантии того, что отправленные данные будут прочитаны только теми сторонами, которым они направлялись, т.е. только лишь легитимные участники приобретут доступ к текущей информации, широковещательному сообщению либо групповой коммуникации (передаче). Это относится и к группам с очень динамичным составом участников.

G5 (аутентификация источника) – легитимные категории участников обязаны аутентифицировать ресурс, а также содержание информации либо коллективной коммуникации. Это относится к тем случаям, когда группы участников не доверяют друг другу.

G6 (авторизация доверенной третьей стороной) – состоит в том, что в определенных протоколах уполномоченная третья сторона представляет одного субъекта *B* иному субъекту *A*. Вследствие этого субъект *A* получает гарантии того, что субъект *B* подтвержден с помощью уполномоченной третьей стороны и авторизован в требуемом для протокола значении. Когда протокол выполняется тремя участниками *A*, *B*, уполномоченная третья сторона, то субъект *A*, возможно, не сможет иметь доступ к ключевому списку или иным механизмам для авторизации *B* (т.к. имя *B* неведомо субъекту *A* или способно быть вымышленным), однако субъект *A* получает гарантии того, что субъект *B* авторизован доверенной третьей стороной.

G7 (аутентификация ключа) – данное свойство подразумевает, что один из участников приобретает доказательство того, что практически никакой иной участник помимо заранее конкретного второго участника не способен приобрести доступ ни к одному секретному ключу.

G8 (подтверждение правильности ключа) – состоит в том, что один из участников приобретает доказательство того, что второй участник на самом деле является владельцем определенного секретного ключа.

G9 (защищенность от чтения назад/совершенная секретность в будущем) – протокол обладает этим свойством, в случае если дискредитация долгосрочных ключей не приводит к компрометации прежних сеансовых ключей.

G10 (формирование новых ключей) – протокол применяет динамическое распределение ключей в целях получения новых ключей.

G11 (защищенная возможность договориться о параметрах безопасности) – если протокол открытого распределения ключей даёт возможность сторонам прийти к соглашению о параметрах защищенности, то данное свойство необходимо для доказательства того, что объявленные свойства и характеристики, о которых договорились участники протоколов, не были подменены злоумышленником.

G12 (конфиденциальность) – заключается в том, что своеобразный набор сведений не будет доступным либо раскрытым для неавторизованных субъектов или процессов, а остается неизвестным злоумышленнику.

G13 (защита идентификаторов от прослушивания) – атакующий, осуществляющий перехват сообщений, не должен иметь возможность связать сообщения одного из участников с самим участником.

G14 (защита идентификаторов от других участников) – в ходе взаимодействия иные участники не обязаны иметь возможности определить сообщения конкретного участника с самим участником, а только лишь с несвязанным с ним псевдонимом либо частным идентификатором.

G15 (ограниченная защищенность от DDoS-атак) – протокол может быть объектом DoS (DDoS) – атак по различным причинам. Одна из них – протокол потребляет слишком много ресурсов (памяти, вычислительной мощности), перед тем, как стороны смогут аутентифицировать друг друга.

G16 (неизменность отправителя) – данное свойство системы предоставляет гарантию для одной из сторон, что источник сообщения остался таким же, как тот, который начал общение, хотя идентификация источника не важна для получателя.

G17 (подотчётность) – свойство системы (включая все системные ресурсы), состоит в том, что даются гарантии того, что действия системных субъектов могут быть однозначно прослежены теми субъектами, кто отвечает за эти действия.

G18 (доказательство источника сообщения) – это неоспоримое доказательство того, что информация была отправлена из источника сообщения.

G19 (доказательство доставки и получения) – это неопровержимое доказательство того факта, что информация была получена.

G20 (безопасное временное свойство) – применяя два оператора временной логики (линейной временной логики), а именно оператора «всегда» и «когда-то в прошлом», возможно формализовать качества вида: «Для каждого достижимого состояния, обладающего свойством p , ранее было состояние со свойством q ».

Гарантия возможности доказать, что факт нахождения системы в одном из состояний означает, что некогда в прошлом система хотя бы раз находилась в некотором другом состоянии. Например, что получение субъектом доступа к ресурсу означает, что некогда в прошлом субъект успешно оплатил данный доступ. Свойствами такого типа, которые непосредственно не видны из стандартных свойств безопасности, являются:

- в случае, если абонент или пользователь обязан платить за ресурс, то в таком случае он обязан обладать правом доступа к ресурсу;
- если хост приобретает огромное число пакетов в режиме потока, то он должен их предварительно запросить.

В таблице (рисунок 1.2) указаны свойства безопасности ряда протоколов:

Протокол \ Цель G	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
EAP-IKEv2	×	×	×			×	×			×					×
EKE	×	×										×			
IKE	×	×	×				×		×	×	×		×	×	×
IKEv2	×	×	×				×		×	×	×				×
DHCP-IPSec-tunnel	×	×										×			
kerberos	×	×	×			×	×			×					
SSH	×	×	×				×			×	×				
TLS	×	×	×				×			×	×		×		
TLS-v1.1	×	×	×				×			×	×		×		
TLS-SRP	×	×	×				×			×	×		×		
tls-sharedkeys	×	×	×				×			×	×		×		
SET	×	×	×										×		

Рисунок 1.2 – Примеры свойств безопасности, характеризующих протоколы

1.2 Атаки на протокол безопасности

Под *атакой на протокол* понимается попытка проведения исследования сообщений протокола и/или исполнения не предусмотренных протоколом действий в целях нарушения работы протокола и/или получения данных, составляющей тайну (конфиденциальную информацию) его участников.

Атака считается успешной, если нарушено хотя бы одно из заявленных свойств, характеризующих безопасность криптографического протокола.

Атаки на криптопротоколы можно разделить на два класса [7]:

- 1) При *пассивной атаке* злоумышленник не участвует в протоколе, он только следит за ним и пытается раздобыть ценные данные на основании шифротекста.
- 2) При *активной атаке* злоумышленник пытается изменить протокол в свою пользу. Для этого активный взломщик может выдавать себя за другого человека, повторять сообщения, заменять сообщения, разбивать строки, изменять данные.

Эти классы можно объединить в разновидности атак на протоколы:

- атаки, нацеленные против криптографических алгоритмов;
- атаки против криптографических методов реализации протоколов;
- атаки против самих протоколов (активные или пассивные).

Известны следующие атаки на криптографические протоколы:

1. *Подмена* – попытка подменить одного пользователя другим. Нарушитель, выступая от имени одной из сторон и полностью имитируя её действия, получает в ответ сообщения определенного формата, необходимые для подделки отдельных шагов протокола. Методы противодействия такой атаке состоят в:

- сохранении в тайне от противника секретной информации, определяющей работу алгоритма идентификации пользователя;
- в необходимости использования различных форматов сообщений, передаваемых на различных шагах протокола;
- в применении вставок в передаваемые сообщения идентификационных меток и специальных номеров сообщений.

В протоколах с внедрением третьей стороны вероятны атаки, базирующиеся на подмене доверенного сервера. Такая атака может быть успешной для протоколов, в которых аутентификация при доступе к серверу базируется лишь только на идентификаторах сторон и случайных числах, генерируемых при каждом взаимодействии. Для защиты от этих атак используются средства привязки ключей не к одной, а к обоим взаимодействующим сторонам методом передачи обоих идентификаторов в закодированном виде.

2. *Повторное навязывание сообщения* — повторное применение ранее переданного в текущем либо предыдущем сеансе сообщения или какой-либо его части в текущем сеансе протокола. В протоколах передачи ключей представленная атака нередко применяется для использования уже использованного раньше сеансового ключа — атака на основе новизны.

Методы противодействия состоят в обеспечении целостности сеанса и невозможности вставки в него лишних сообщений. Для этого применяется:

- техника типа «запрос – ответ»;
- вставка в передаваемые сообщения временных меток, случайных чисел или возрастающих последовательностей чисел.

3. Еще один тип подобных атак связан с обратной передачей адресату ранее переданных им сообщений и получил название *атака отражением*. Атаки такого типа относят к классу атак с применением повторного навязывания сообщения. Для защиты от таких атак протоколы специально делают асимметричными, включая в зашифрованные сообщения идентификаторы сторон, или изменяя процедуры таким образом, чтобы стороны обязаны были выполнять разные действия, вводят в протокол идентификационную информацию, применяя различные ключи для приема и передачи сообщений.

4. *Задержка передачи сообщения* – перехват злоумышленником сообщения и навязывание его в более поздний момент времени. Это тоже разновидность атаки с повторным навязыванием сообщения. Способы противодействия данной атаке включают применение случайных чисел вместе с лимитированием временного промежутка для ответа, использование временных меток.

5. *Комбинированная атака* – замена или же иной способ обмана, применяющий комбинацию данных из ранее выполненных протоколов, в том числе протоколов, ранее навязанных противником. Метод противодействия данной атаке состоит в обеспечении целостности сеансов протоколов и отдельных сообщений. Существует и специальный частный случай этой атаки, в котором противник специально открывает одновременно несколько параллельных сеансов с целью использования сообщений из одного сеанса.

Выводы

Раздел посвящен теоретическим материалам. Рассмотрены и кратко описаны основные положения, цели и требования к безопасности протоколов. Приведены свойства безопасности некоторых протоколов. Проанализированы активные и пассивные атаки на протоколы, а также методы противодействия этим атакам.

2. СОВРЕМЕННЫЕ КРИПТОГРАФИЧЕСКИЕ ПРОТОКОЛЫ

На сегодняшний день все существующие криптографические протоколы можно разделить на три категории: доквантовые, квантовые и постквантовые. К доквантовым криптографическим протоколам относят: PAP, CHAP, Kerberos, X.509, PPTP, L2F, L2TP, SSL, TLS, PGP, Ирес и др.; к квантовым криптопротоколам относят: BB84, B92, протоколы Гольденберга–Вайдмана, Коаши–Имото и др.; к постквантовым – NTRUEncrypt, McEliece, GGH, NTRUSign и др.

Рассмотрим подробнее эти алгоритмы, согласно данной классификации.

2.1 Доквантовые протоколы

2.1.1 *Transport layer security (TLS)*

Протокол защиты транспортного уровня – TLS (Transport Layer Security) – базируется на *протоколе степени безопасности сокетов* – SSL (Secure Sockets Layer), первоначально разработанном в компании Netscape с целью увеличения защищенности электронной коммерции в Интернете. Протокол SSL был выполнен на уровне приложений, непосредственно над TCP (Transmission Control Protocol), что даёт возможность более высокоуровневым протоколам (таким, как HTTP или протоколу электронной почты) функционировать без изменений. В случае, если SSL сконфигурирован верно, то злоумышленник способен выяснить всего лишь параметры соединения (к примеру, тип используемого шифрования), а также частоту пересылки и примерное количество данных, однако никак не способен прочитывать и изменять их. Определенное положение TLS (SSL) в стеке протоколов сети Интернета представлено на рисунке 2.1:

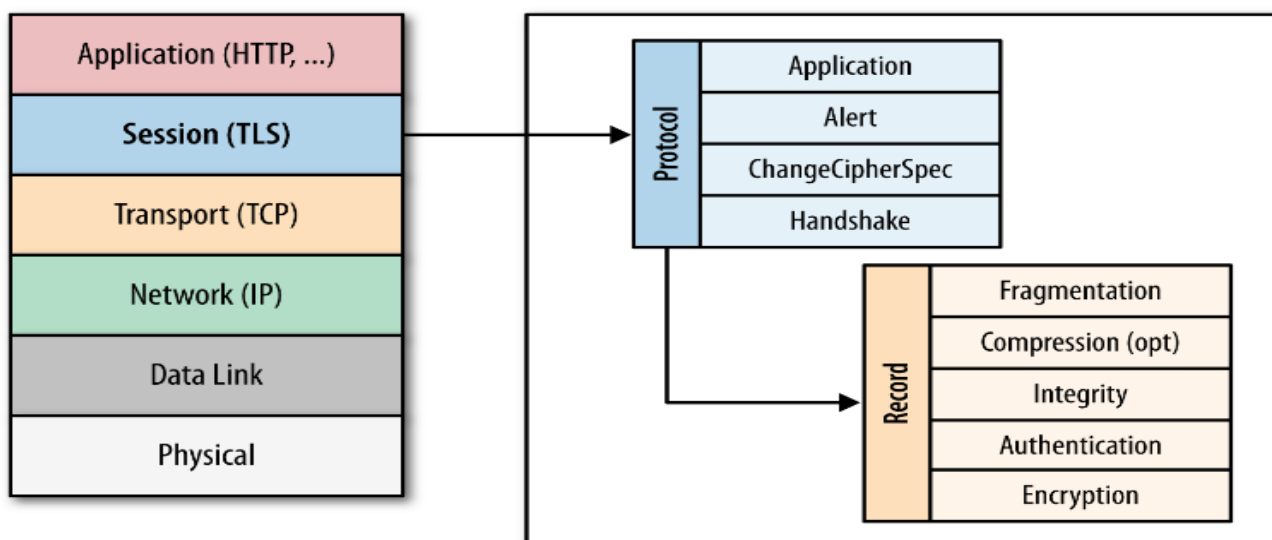


Рисунок 2.1 – TSL протокол

Протокол TLS специализирован с целью предоставления трёх услуг абсолютно всем приложениям, работающим над ним, а непосредственно: кодирование, проведение аутентификации и проверки целостности.

Технически, могут применяться не все три услуги, но на практике, для обеспечения безопасности, как правило, применяются все три:

- шифрование – сокрытие от неавторизованных лиц конфиденциальной информации, передаваемой от одного компьютера к другому;
- аутентификация – проверка авторства передаваемой информации;
- целостность – обнаружение подмены информации подделкой.

С целью того чтобы установить криптографически безопасный канал данных, узлы соединения обязаны согласовать используемые методы кодирования и ключи. Протокол TLS конкретно устанавливает данную процедуру. Стоит отметить, что TLS применяет криптографию с открытым ключом, которая даёт возможность соединениям установить совместный секретный ключ шифрования без каких-либо предварительной информации друг о друге.

В рамках процедуры TLS Handshake существует возможность определить истинность личности и клиента, и сервера. К примеру, клиент может быть уверен, что сервер, предоставляющий ему информацию о банковском счёте, действительно является банковским сервером. И наоборот: сервер компании может быть уверен, что клиент, подключившийся к нему – это действительно сотрудник компании, а не постороннее неавторизованное лицо, выдающее себя за сотрудника банка.

Кроме того, TLS гарантирует отправку каждого сообщения с кодом MAC (Message Authentication Code), алгоритм формирования которого – односторонняя криптографическая функция хеширования (фактически – контрольная сумма), ключи которой известны обоим участникам сеанса. Каждый раз при отправке сообщения, формируется его MAC-значение, которое может сгенерировать и приёмник, это обеспечивает целостность информации и защиту от её подмены.

Перед тем, как приступить к взаимнообмену данными через TLS, клиент и сервер обязаны утвердить между собой параметры соединения, а именно: версия применяемого протокола, метод кодирования информации, в том числе проверить сертификаты при необходимости. Схема начала соединения TLS Handshake и продемонстрирована на рисунке 2.2.

Рассмотрим детальнее каждый этап данной процедуры:

- 1) Так как протокол TLS работает над протоколом TCP, для начала между клиентом и сервером устанавливается TCP-соединение.

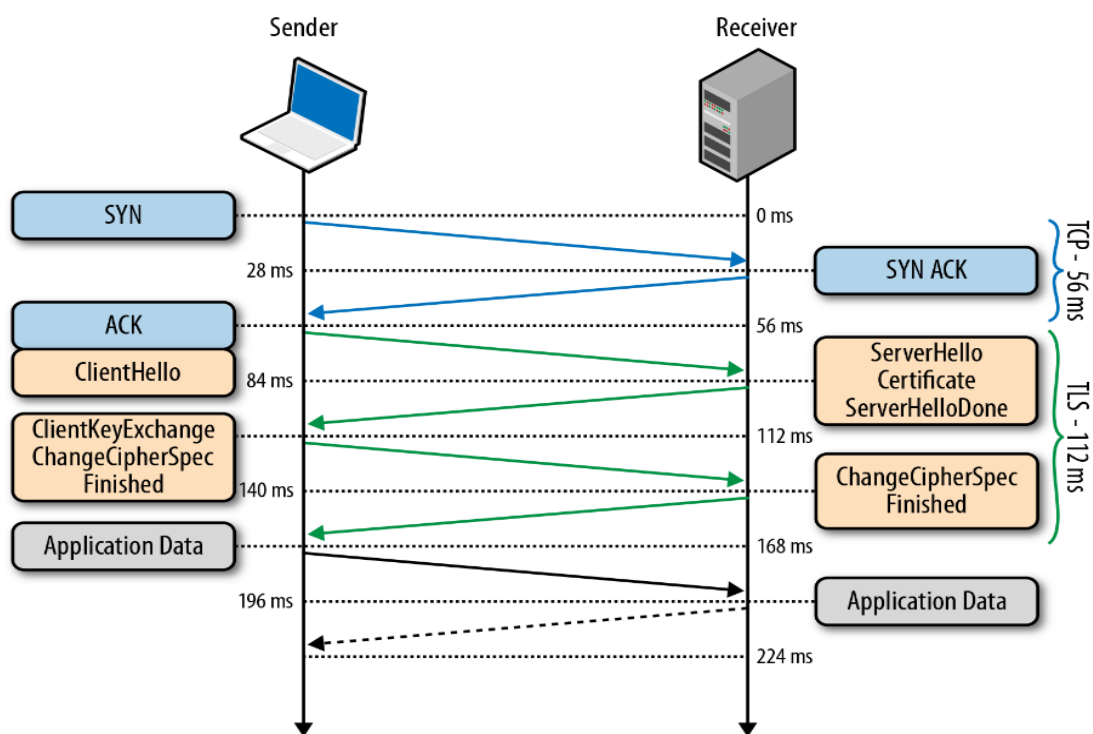


Рисунок 2.2 – Алгоритм Handshake

2) После установки TCP, клиент отправляет на сервер спецификацию в виде обычного текста (а именно версию протокола, которую он хочет использовать, поддерживаемые методы шифрования, etc).

3) Сервер утверждает версию используемого протокола, выбирает способ кодирования из предоставленного списка, прикрепляет свой сертификат и отправляет ответ клиенту. Сервер может запросить и клиентский сертификат.

4) Версия протокола и способ шифрования на данном этапе считаются утверждёнными, клиент проверяет присланный сертификат и инициирует или RSA, или обмен ключами по Диффи–Хеллману, в зависимости от устанавливаемых параметров.

5) Сервер обрабатывает присланное клиентом сообщение, проверяет MAC, и отправляет клиенту окончательное ('Finished') сообщение в закодированном виде.

6) Клиент расшифровывает полученное сообщение, сверяет MAC, и если всё хорошо, то соединение считается установленным и начинается обмен данными приложений.

Обмен ключами используется по алгоритму RSA или Диффи–Хеллмана. Но, т.к. RSA считается более уязвимым, чем Диффи–Хеллмана, то предпочитают использовать последний. О криптоанализе RSA можно почитать в источнике.

Актуальной версией протокола TLS на текущий момент является версия TLS 1.2. В российском криптографическом протоколе транспортного уровня TLS 1.2 применяются алгоритмы блочного шифрования – ГОСТ Р 34.12–2015 «Кузнечик» и ГОСТ 28147–89 «Магма», которые на данный момент считаются надежными и не подвергаются криптоанализу. Подробнее о применении этих алгоритмов в данном протоколе можно почитать в источнике [7].

2.1.2 Challenge Handshake Authentication Protocol (CHAP)

CHAP — протокол аутентификации с косвенным согласованием. Является алгоритмом проверки подлинности и предусматривает передачу не самого пароля пользователя, а косвенных сведений о нём. Аутентификация узла выполняется путём трехэтапной процедуры согласования. Протокол CHAP широко используется различными поставщиками серверов и клиентов сетевого доступа. В протоколе CHAP применяется трёхразовая проверка подлинности при регистрации клиента на хосте. Формат пакета протокола CHAP представлен на рисунке 2.3:

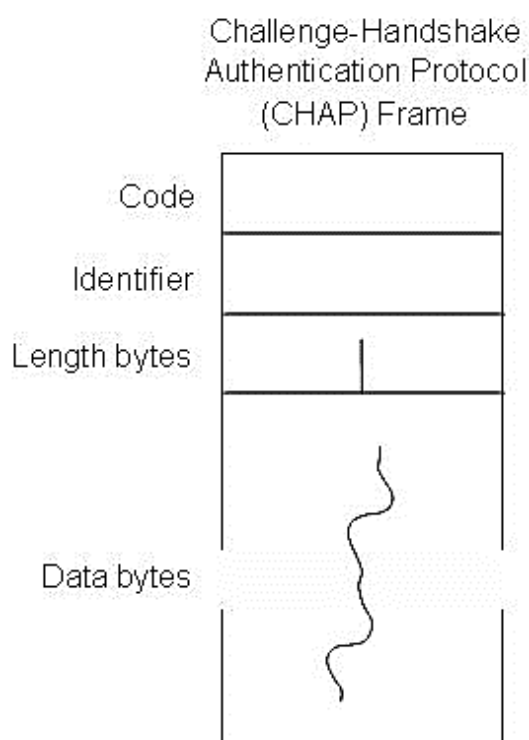


Рисунок 2.3 – Формат пакета CHAP

Поле **Код** имеет в длину один октет и идентифицирует тип пакета CHAP. Значения поля могут быть: вызов, ответ, ответ успешная аутентификация, отказ.

Поле **Идентификатор** имеет в длину один октет. С помощью этого поля происходит идентификация пакета CHAP. Затем клиент и хост по этому идентификатору могут согласовывать свои запросы и ответы на них.

Поле **Длина** составляет два октета и используется для задания длины пакета CHAP. Длина пакета считается с учётом полей Код, Идентификатор, Длина и Данные.

Поле **Данные** может быть нулевой длины или несколько октетов в длину. В него включаются данные, которые заданы в поле Код пакета CHAP.

На рисунке 2.4 показан протокол, подтверждающий установление соединения, который применяется в CHAP для проверки подлинности клиента.

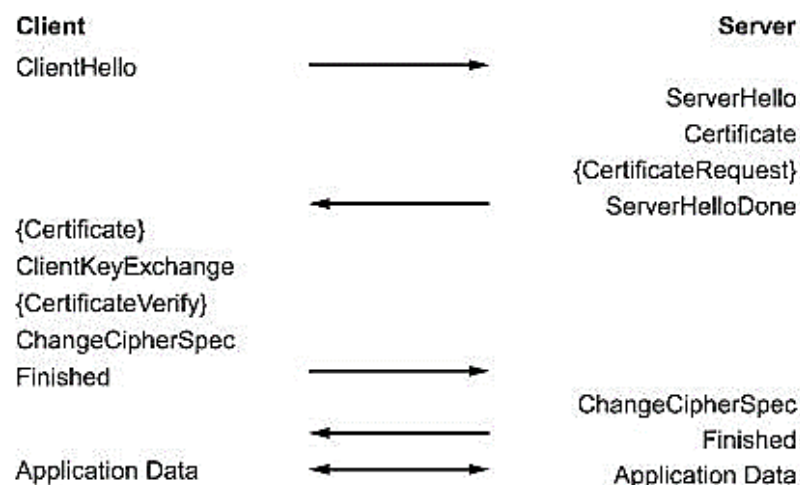


Рисунок 2.4 – Работа протокола Handshake

Подтверждение установления соединения в CHAP проходит в три фазы:

- 1) Хост посылает пакет запроса CHAP–клиенту со случайным значением.
- 2) Клиент отвечает пакетом ответа CHAP, в который включено значение, вычисленное с помощью наложения на значение вызова секретного ключа. Ключ выводится с помощью односторонней функции хеширования.

3) Далее хост сверяет значение в ответе клиента, с полученным им самим таким же способом (наложение значения вызова на ключ). Если они совпадают, то хост высылает пакет **"Success"** (Успешная аутентификация), и сеанс PPP продолжается. В противном случае хост посылает клиенту пакет **"Failure"** (Отказ), сеанс PPP прекращается и формируется LCP–пакет типа **Terminate–Request**.

Данный протокол также применяется в РФ и описывается в документе «Р 1323565.1.020–2018 Информационная технология».

2.1.3 Secure Shell (SSH)

Протокол «безопасная оболочка» или SSH — сетевой протокол прикладного уровня, позволяющий производить удалённое управление операционной системой и туннелирование TCP–соединений, для передачи файлов. Схож по функциональности с протоколами Telnet и Rlogin, но, в отличие от них, шифрует весь трафик, включая и передаваемые пароли. SSH допускает выбор различных алгоритмов шифрования. SSH–клиенты и SSH–серверы доступны для большинства сетевых операционных систем. SSH позволяет безопасно передавать практически любой другой сетевой протокол в незащищённой среде. Таким образом, можно не только удалённо работать на компьютере через командную оболочку, но и передавать по зашифрованному каналу звуковой поток или видео (например, с Web–камеры). Также SSH может использовать сжатие передаваемых данных для последующего их шифрования [8].

Вспомогательными инструментами проверки подлинности SSH являются средства управления ключами и связанные с ними агенты. Если настроена аутентификация с использованием открытого ключа, то ключ подтверждает подлинность пользователя удаленному узлу SSH. Подлинность на основе SSH определяется на основе двух компонентов: открытого и закрытого ключей. Закрытый ключ SSH подтверждает подлинность пользователя при исходящем SSH-подключении и должен храниться в секрете. Когда пользователь инициирует SSH- или SCP-подключение к удаленному компьютеру или серверу, он является SSH-клиентом (рисунок 2.5). При помощи математических алгоритмов закрытый ключ становится вашей персональной электронной картой доступа, а открытый ключ – замком, к которому подходит ваша электронная карта.

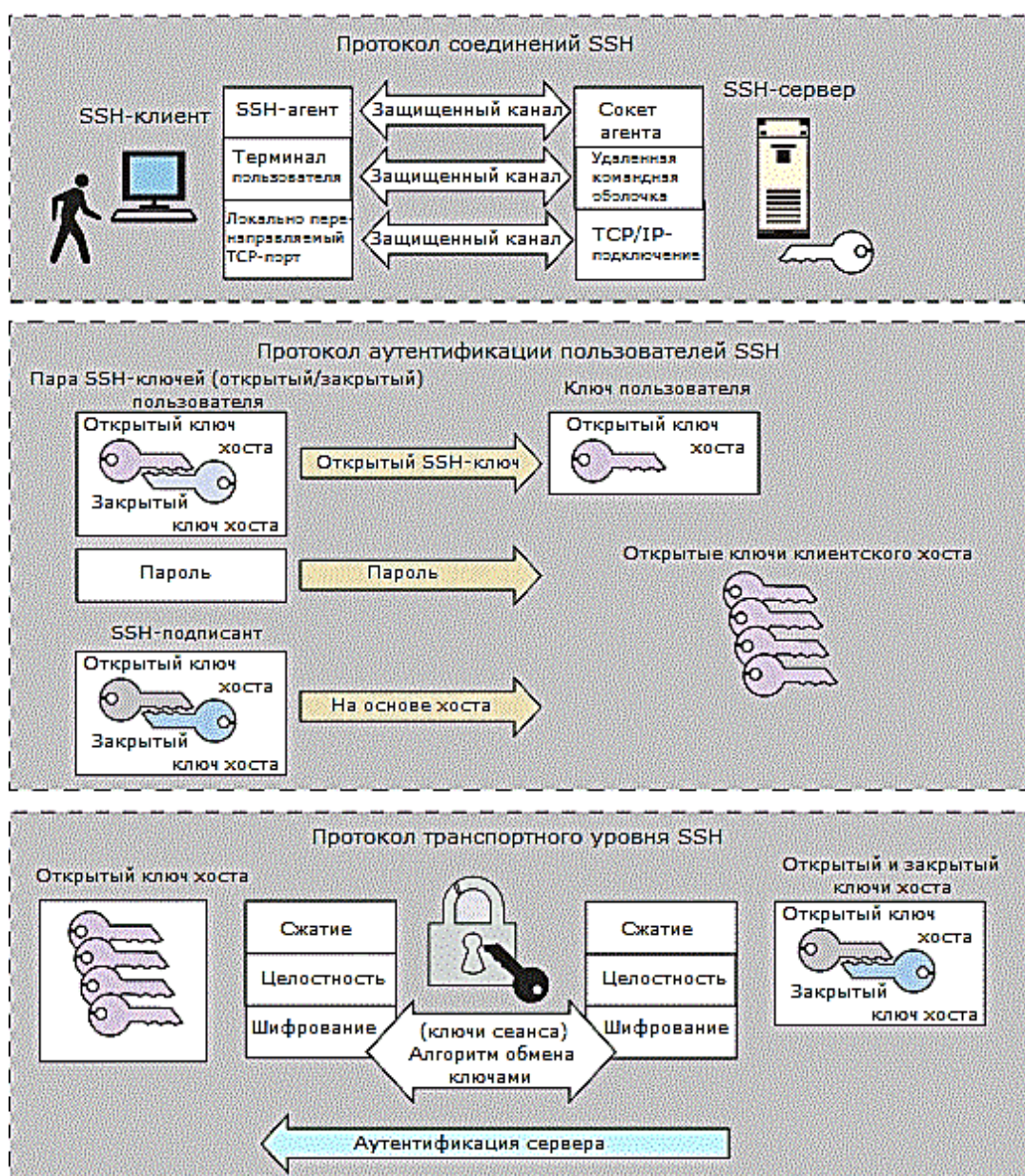


Рисунок 2.5 – Диаграмма взаимодействия открытых и закрытых SSH-ключей согласно архитектуре SSH

Открытый ключ определяет, кому разрешено открывать замок и получать право на вход. Открытые ключи не нужно держать в секрете, поскольку их невозможно использовать для компрометации системы или для получения несанкционированного доступа. В операционных системах Linux и UNIX открытые и закрытые ключи хранятся в виде текстовых ASCII-файлов; в системах под управлением Windows пары ключей могут храниться как в текстовых файлах, так и в реестре Windows. Подробнее о работе протокола SSH можно прочитать в [8].

2.2 Квантовые протоколы

2.2.1 Протокол BB84

BB84 — первый протокол *квантового распределения ключа*, который был предложен в 1984 г. Чарльзом Беннетом и Жилем Brassаром. Протокол использует для кодирования информации четыре квантовых состояния двухуровневой системы, формирующие два сопряжённых базиса. Носителями информации являются кубиты — квантовые биты.

Протокол использует 4 квантовых состояния, образующих 2 базиса, например, поляризационные состояния света. Состояния внутри одного базиса ортогональны, но состояния из разных базисов — попарно неортогональны. Эта особенность протокола позволяет определить возможные попытки нелегитимного съёма информации. Носителями информации в протоколе являются фотоны, поляризованные под углами 0° , 45° , 90° , 135° . С помощью измерения можно различить только два ортогональных состояния [9]:

1. Фотон поляризован вертикально или горизонтально;
2. Фотон поляризован под углами 45° или 135° .

Достоверно отличить за одно измерение горизонтальный фотон от фотона, поляризованного под углом 135° , не представляется возможным. Реализация этого протокола показана на рисунке 2.6.

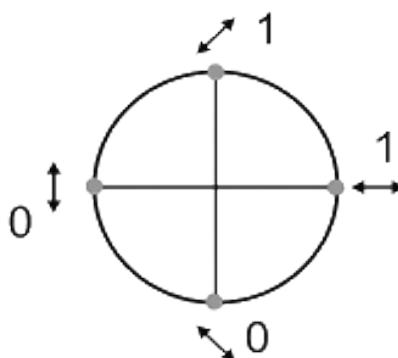


Рисунок 2.6 – Реализация протокола BB84

Этапы формирования ключей происходят следующим образом:

1. Алиса случайным образом выбирает один из базисов. Затем внутри базиса она случайно выбирает одно из состояний, соответствующее 0 или 1, и посылает фотоны, как показано на рисунке 2.7:

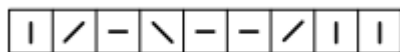


Рисунок 2.7 – Фотоны с различной поляризацией

Боб случайно и независимо от Алисы выбирает для каждого поступающего фотона прямолинейный (+) или диагональный (×) базис (рисунок 2.8):

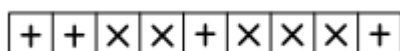


Рисунок 2.8 – Выбранный тип измерений

Затем Боб сохраняет результаты измерений, как показано на рисунке 2.9:

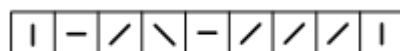


Рисунок 2.9 – Результаты измерений

Боб по открытому общедоступному каналу связи сообщает, какой тип измерений был использован для каждого фотона, то есть какой был выбран базис, но результаты измерений остаются в секрете;

Алиса сообщает Бобу по открытому общедоступному каналу связи, какие измерения выбраны в соответствии с исходным базисом Алисы (рисунок 2.10).

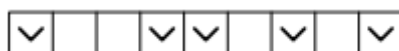


Рисунок 2.10 – Случаи правильных замеров

Далее пользователи оставляют только те случаи, в которых выбранные базисы совпали. Эти случаи переводят в биты (0 и 1), и, таким образом, получают ключ, как показано на рисунке 2.11.

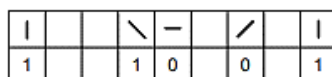


Рисунок 2.11 – Получение ключевой последовательности по результатам правильных замеров

Ключи, согласно протоколу BB84, формируются согласно таблице, показанной на рисунке 2.12.

Двоичный сигнал Алисы	0	1	0	1
Поляризационный код Алисы	$\leftrightarrow$	$\updownarrow$	$\nwarrow$	$\nearrow$
Детектирование Бобом	$\oplus$	$\oplus$	$\oplus$	$\oplus$
Двоичный сигнал Боба	0	1	?	?

Рисунок 2.12 – Формирование квантового ключа по протоколу BB84

Более подробно об этом алгоритме можно прочитать в источнике [10].

Возможные атаки на данный протокол:

- для случая однофотонных сигналов (некогерентные, когерентные);
- разделения числа фотонов и др.

2.2.2 Протокол B92

Протокол B92 — один из первых протоколов квантового распределения ключа, который был предложен в 1992 году Чарльзом Беннетом (англ. Charles H. Bennett). Протокол B92 основан на принципе неопределённости в отличие от таких протоколов, как E91. Носителями информации являются 2-х уровневые системы – кубиты (квантовые биты). Важной особенностью протокола является использование двух неортогональных квантовых состояний.

Схема кодирования квантовых состояний в протоколе B92 схожа со схемой кодирования протокола BB84, но использует только два неортогональных из четырёх состояний BB84, а качестве базиса применяют различные поляризации:

- линейную (+) – горизонтальную и вертикальную;
- круговую (O) – правую круговую и левую круговую.

В этом протоколе классический бит кодируется двумя неортогональными состояниями, которые представлены на рисунке 2.13:

Поляризация	Горизонтальная ($\leftrightarrow$)	Вертикальная ($\updownarrow$)	Правая круговая ($\curvearrowright$)	Левая круговая ($\curvearrowleft$)
Бит	0	1	1	0

Рисунок 2.13 – Неортогональные состояния протокола B92

Поскольку в соответствии с принципом неопределённости Гейзенберга при измерении нельзя отличить два неортогональных состояния друг от друга, поэтому невозможно достоверно определить значение бита. Более того, любые попытки узнать состояние кубита непредсказуемым образом изменяют его. Это и есть идея, лежащая в основе квантового протокола разделения ключей B92 [11].

Так как схема кодирования протокола использует только лишь два состояния, то иногда она проще в реализации, нежели другие схемы. Однако, получение достаточной надёжности такого протокола в некоторых экспериментах является трудной задачей, и часто оказывается, что протокол является совсем не безопасным. Алгоритм работы протокола B92 показан на рисунке 2.14:

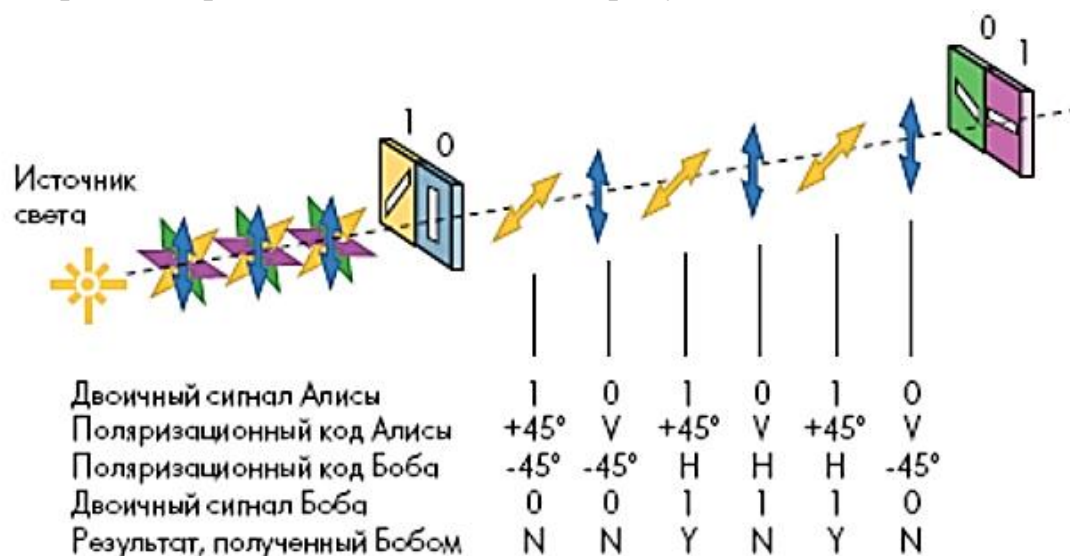


Рисунок 2.14 – Алгоритм работы протокола B92

Формирование квантового ключа по протоколу B92 показан на рисунке 2.15:

Двоичный сигнал станции Алиса	1	0	1	0
Поляризационный код станции Алиса	↗	↕	↗	↕
Поляризационный код станции Боб	↘	↘	↔	↔
Двоичный сигнал станции Боб	0	0	1	1
Результат, полученный станцией Боб	-	-	+	-

Рисунок 2.15 – Формирование квантового ключа по протоколу B92

Более подробно о работе и формировании ключей квантового протокола B92 можно прочитать в источниках [9 – 11].

Криптостойкость протокола B92 использует тот факт, что при попытке измерить Евой состояние фотона, вносится ошибка в другое неортогональное состояние данному. Также существует еще идея PNS-атаки, которая основана на том, что протокол может быть реализован на не единичных фотонах. В таком случае появляется возможность отобрать часть фотонов из квантового канала связи, достаточную для измерения поляризации данного пучка Евой.

2.3 Постквантовые протоколы

2.3.1 Протокол McEliece

McEliece – криптосистема с открытыми ключами, основанная на базе теории алгебраического кодирования, и разработанная в 1978 году Робертом Мак–Элисом. Это была первая схема, использующая рандомизацию в процессе шифрования. Работу этой криптосистемы можно разбить на три основных этапа [12]:

- случайная генерация ключа, дающая на выходе открытый и закрытый ключи;
- случайное шифрование, дающее на выходе шифротекст сообщения;
- детерминированное расшифровывание, дающее на выходе исходный открытый текст первоначального сообщения.

Открытый ключ получается путем перемножения производящей матрицы G на невырожденные случайные матрицы S и P . В качестве закрытого ключа выступает код исправления ошибок, количество которых равно t , а также применяется эффективный алгоритм декодирования. Обычно в алгоритме оперируют двоичные коды Гоппа. McEliece с использованием кодов Гоппа до сих пор не поддается криптоанализу. Алгоритм формирования ключа представлен на рисунке 2.16.



Рисунок 2.16 – Блок–схема алгоритма генерации ключей криптосистемы Мак–Элиса

Блок–схемы алгоритма расшифровывания и дешифрования представлены на рисунке 2.17. Подробнее об этом алгоритме можно прочитать в источнике [12].

Алгоритм не получил широкого признания, но в то же время является кандидатом для постквантовой криптографии, так как устойчив к атаке с помощью квантового алгоритма факторизации Питера Шора. Алгоритм основан на сложности декодирования полных линейных кодов (общая задача декодирования является NP–сложной) и использует двоичные коды Гоппа, которые легко декодируются благодаря алгоритму параллельного программирования для взаимного исключения потоков Гарри Петерсона.

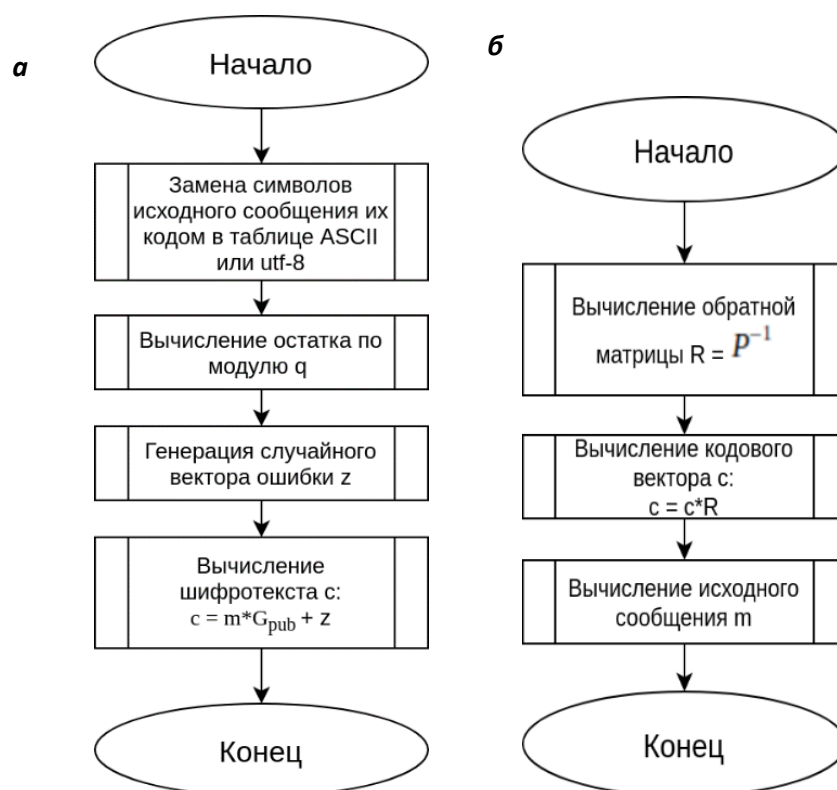


Рисунок 2.17 – Блок–схемы криптосистемы Мак–Элиса:

а) алгоритм шифрования сообщения; б) алгоритм расшифрования сообщения

Открытый ключ получается при помощи маскировки выбранного кода как полного линейного. Самые известные атаки на данную криптосистему [13]:

- Атака Финиаз – Сендреир;
- Атака Бернштейн – Ланг – Петерс;
- Атака Кантеаут – Шабант;
- Атака Штера;
- Атака Л. Миндера и А. Шокроллахи;
- Атака на основе кодов Гоппы;
- Атака на основе кодов Рида–Маллера.

2.3.2 Протокол NTRUSign

NTRUSign — специальная версия GGH, отличающейся меньшим ключом и размером подписи и являющейся более эффективной. С другой стороны, она использует только решетки подмножества множества всех решеток, связанных с некоторыми полиномиальными кольцами. NTRUSign была выдвинута на рассмотрение IEEE-standard P1363.1. Стойкость алгоритма обеспечивается трудностью поиска кратчайшего вектора решётки, которая более стойкая к атакам, реализуемым с помощью квантового компьютера.

Работа криптосистемы включает в себя следующие алгоритмы [14]:

- генерацию ключей;
- шифрование сообщения;
- расшифровывание сообщения.

Подробное описание алгоритма можно почитать в источнике.

На данный момент известны следующие атаки на эту криптосистему:

- Атака полного перебора;
- Атака «встреча посередине»;
- Атака на основе множественной передачи сообщения;
- Атака на основе решётки;
- Атака на основе подобранного шифротекста.

2.3.3 Протокол NTRUEncrypt

NTRUEncrypt – это криптографическая система с открытым ключом, ранее называвшаяся NTRU. Криптосистема NTRUEncrypt, основанная на решётчатой криптосистеме, создана в качестве альтернативы RSA и криптосистемам на эллиптических кривых (ECC). Стойкость алгоритма обеспечивается трудностью поиска кратчайшего вектора решётки. Эта криптографическая система является более стойкой к кибератакам, осуществляемым с квантовых компьютеров.

Чтобы обеспечить высокую стойкость алгоритма к различным атакам, рекомендуется использование параметров, показанных на рисунке 2.18.

Обозначение	N	q	p	df	dg	dr	Гарантированная стойкость
NTRU167:3	167	128	3	61	20	18	Умеренный уровень стойкости
NTRU251:3	251	128	3	50	24	16	Стандартный уровень стойкости
NTRU503:3	503	256	3	216	72	55	Высочайший уровень стойкости
NTRU167:2	167	127	2	45	35	18	Умеренный уровень стойкости
NTRU251:2	251	127	2	35	35	22	Стандартный уровень стойкости
NTRU503:2	503	253	2	155	100	65	Высочайший уровень стойкости

Рисунок 2.18 – Рекомендованные параметры

Описание этого алгоритма можно прочитать во многих источниках.

Существуют следующие атаки на данную криптографическую систему:

- атака полного перебора;
- атака «встреча посередине»;
- атака на основе множественной передачи сообщения;
- атака на основе решётки;
- атака на основе подобранного шифротекста.

Выводы

Данный раздел посвящен теоретическим данным. Были рассмотрены и кратко описаны современные криптографические протоколы. Условно они разделены на три группы: доквантовые, квантовые и постквантовые. Из доквантовых были рассмотрены протоколы TLS, CHAP и SSH. Из квантовых – протоколы BB94 и B92. Из постквантовых – протоколы McEliece, NTRUSign, NTRUEncrypt. У каждой группы выявлены свои недостатки и уязвимости, описаны возможные атаки на данные шифры.

На основании этих теоретических исследований в разделе 4 производится разработка нового криптографического протокола IAS–2019.

3. ОБОСНОВАНИЕ НЕОБХОДИМОСТИ ИМПОРТОЗАМЕЩЕНИЯ

Основной проблемой России в ИТ–сфере является доминирующее наличие зарубежных технологий и разработок. Тем самым, мы не только зависим от иностранных технологий, но также уязвимы ко вторжению со стороны наших партнеров и вероятных противников. Надеяться на надёжность и безопасность импортного оборудования и технологий не стоит, т.к. все уязвимости и атаки, которые нам не известны, зарубежными спецслужбами уже разработаны и ждут своё «время X». От этого зависит жизнеобеспечение и безопасность российского общества и государства в целом. Следовательно, России необходимо иметь собственный национальный суверенитет в ИТ–сфере.

Сильным толчком для становления на рельсы независимости послужили санкции, введённые в 2014 году иностранными государствами. Данные санкции затронули все сферы государства, большого и малого бизнеса. В связи с этим возникает необходимость найти замену импортным решениям из российских аналогов. Эта необходимость важна по двум причинам:

- в современных условиях никто не гарантирует доступность технологий;
- встает вопрос информационной безопасности на национальном уровне.

С другой стороны, существуют определенные сдерживающие факторы для перехода на отечественное программное обеспечение:

1. Распорядительные документы, содержащие ограничения отечественного ПО, в том числе при предоставлении государственных услуг населению своих функций.
2. Недостаточная компьютерная грамотность пользователей и низкая компетентность системных администраторов.
3. Привязка прикладного программного обеспечения государственных и ведомственных информационных систем к Windows–технологиям.
4. Несовместимость имеющегося программно–аппаратного обеспечения и оборудования с отечественными операционными системами.
5. Отторжение проектов пользователями.

Для преодоления этих объективных факторов нужно создание серьезной нормативно–правовой базы. Рассмотрим такую базу в Российской Федерации.

3.1 Нормативная база в РФ

Указы Президента Российской Федерации:

1. От 05.12.2016 № 646 «Об утверждении Доктрины информационной безопасности Российской Федерации».

2. От 09.05.2017 № 203 «О Стратегии развития информационного общества в Российской Федерации на 2017 – 2030 годы».

Постановления Правительства Российской Федерации:

1. От 4 августа 2015 г. № 785, постановление «О создании Правительственной комиссии по вопросам импортозамещения».

2. От 16.11.2015 г. №1236 «Об установлении запрета на допуск программного обеспечения, происходящего из иностранных государств, для целей осуществления закупок для обеспечения государственных и муниципальных нужд».

3. От 20.12.2017 №1594, постановление расширило применение Постановления Правительства № 1236 при закупках (аренда, услуги, поддержка).

4. От 23.03.2017 г. № 325 «Об утверждении дополнительных требований к программам для электронных вычислительных машин и базам данных, сведения о которых включены в реестр российского программного обеспечения, и внесении изменений в Правила формирования и ведения единого реестра российских программ для электронных вычислительных машин и баз данных».

Также было принято о создании Реестра российского ПО, который действует с 1 января 2016 г. в соответствии с требованиями ФЗ–188, и постановления правительства № 1236 от 16 ноября 2015 г., и создан, в частности, в целях оказания мер государственной поддержки российским разработчикам программного обеспечения. На сегодняшний день Реестр содержит 2602 наименования отечественных программных продуктов.

Стоит упомянуть, что 26 июля 2016 г. вышло в свет распоряжение Правительства Российской Федерации № 1588–р, которым утвержден «План перехода федеральных органов исполнительной власти и государственных внебюджетных фондов на использование отечественного офисного ПО».

Опубликованы следующие Приказы Министерства цифрового развития, связи и массовых коммуникаций Российской Федерации:

1. От 1.04. 2015 г. № 96 «Об утверждении плана импортозамещения ПО».

2. От 29.06.2017 г. №334 «Об утверждении методологических рекомендаций по переходу федеральных органов исполнительной власти и государственных внебюджетных фондов на использование отечественного офисного ПО, в том числе ранее закупленного офисного программного обеспечения».

3. От 04.07.2018 г. №335 «Об утверждении методических рекомендаций по переходу органов исполнительной власти субъектов Российской Федерации и органов местного самоуправления муниципальных образований Российской Федерации на использование отечественного офисного программного обеспечения, в том числе ранее закупленного офисного программного обеспечения».

Благодаря данной нормативно–правовой базе, доля отечественного ПО в органах власти и местного самоуправления, а также в госкомпаниях на 2019 г. существенно выросла. Согласно утвержденному плану Минкомсвязи России, максимальная доля импорта ПО к 2025 г. должна составлять 50% [15].

3.2 Нормативная база иностранных государств

3.2.1 Соединенные Штаты Америки

В США импортозамещение регулируется следующими нормативно–правовыми актами: Trade Agreements Act (ТАА), Buy American Act (БАА), Federal Acquisition Regulation (FAR). Преференции для продукции национального производства устанавливаются в БАА и Recovery Act «Buy America».

Характеристика импортозамещения по БАА устанавливает обязанность государственных органов закупать товары, произведенные только в США (41 United States Code § 10a–10d и FAR Part 25), если только руководитель такого органа не докажет несоответствие закупки публичным интересам, несоизмеримость цены, отсутствие внутреннего аналога и т. п. Таким образом, БАА не устанавливает полного запрета на закупки иностранных продуктов, но предоставляет преференции национальным производителям / разработчикам (FAR Subpart 25.1).

Характеристика импортозамещения по ТАА предусматривает исключения из правил, установленных БАА. В соответствии с международными договорами производителям некоторых стран предоставляется в США национальный режим, т.е. конечные продукты могут участвовать в процессе государственных закупок наравне с отечественными (FAR 25.003). При этом продукты, которые произведены в третьих странах, запрещено закупать для государственных нужд. Предусматриваются те же исключения (слишком высокая цена, отсутствие аналогов и т. д.), что установлены и в БАА, но применяются они редко.

Кроме того, предусматривается возможность признания продуктов отечественными при условии, что они соответствуют тесту достаточной переработки (FAR 25.003), произведенной в любой стране из установленного списка. Продукт может признаваться отечественным в случаях: а) если он произведен в одной из стран – участниц международного договора; или б) если он существенно переработан в одной из этих стран.

Стоит отметить, что в США 13 августа 2018г. принят закон H.R. 5515, в котором присутствует статья SEC. 1655. «Mitigation of risks to national security posed by providers of information technology products and services who have obligation to foreign governments».

Суть данной статьи состоит в том, что: «...министерство обороны США не должно использовать продукты ИТ–продукты / услуги, пока поставщик не предоставит определенную информацию, в том числе о предоставлении в иностранные государства исходного кода своих продуктов».

Что означает данная статья для компаний, чей исходный код инспектировался во время сертификации по требованиям безопасности в любой стране, которая установила такие требования (например, Китай, Германия или Россия)? В самом лучшем варианте – существенное усложнение процедуры продажи таких решений министерству обороны США, поскольку в положения закона уже заложена возможность отказа. В худшем сценарии возможен полный отказ МО США от использования и приобретения таких решений, что может обернуться огромными потерями для ИТ / ОТ / ИБ–компаний, измеряемыми десятками и даже сотнями миллионов долларов.

Возможные последствия для России: уход иностранных производителей из российского рынка сертифицируемых программных продуктов. Это Check Point, Cisco, Dell, Forcepoint, Fortinet, HP, IBM, McAfee, Microfocus (Arcsight), Microsoft, Palo Alto, Rockwell, SAP, Trend Micro, VMware и др.

3.2.2 Китайская Народная Республика

Импортозамещение в КНР, в частности, регулируется Законом о государственных закупках 2002 г. (Government Procurement Law, далее – GPL) и Законом о тендерах 1999 г. Преференции для продукции национального производства устанавливаются в Проекте Регламента о применении Закона о государственных закупках 2002 г. (Draft Implementation Regulations, далее — DIR).

Характеристика импортозамещения в Китае. Согласно GPL, под государственными закупками понимается процесс получения товаров, работ и услуг за счёт государственных средств в случае (ст. 2 GPL): а) если такие товары, работы и услуги содержатся в специальном реестре; или б) стоимость закупок превышает установленные лимиты. GPL не устанавливает полного запрета на закупки иностранных товаров, в том числе в сфере информационных технологий, однако содержит ограничения — такой товар должен быть включен в специальный реестр. GPL содержит преференции для товаров отечественного производства при государственных закупках, содержанием которых является обязанность китайских публичных субъектов при госзакупках ориентироваться исключительно на отечественный рынок за исключением следующих случаев:

- отсутствие отечественных аналогов требуемых товаров, работ или услуг, а также их необоснованно высокая стоимость;
- и / или закупаемые товары предназначены для использования за рубежом;
- и / или иное предусмотрено нормативно–правовыми актами.

Фактически, в Китае государственные закупки товаров и услуг, в том числе в сфере информационных технологий, ограничены реестром, который содержит наименования «разрешенных» продуктов. Критерии для внесения продукта в список, устанавливаются на подзаконном уровне уполномоченными органами.

3.2.3 Европейский союз

В настоящее время сфера импортозамещения в ЕС регулируется Директивой № 2014/24/ЕС Европейского парламента и Совета ЕС «О государственных закупках и отмене Директивы 2004/18/ЕС», Директивой № 2004 / 17 / ЕС Европейского парламента и Совета ЕС «О координации процедуры закупок на предприятиях, работающих в сфере водоснабжения, энергетическом, транспортном и почтовом секторах», а также иными специализированными директивами и регламентами.

Характеристика импортозамещения в ЕС на данном этапе правового регулирования преференции для отечественных производителей программного обеспечения законодательством не предусмотрены.

В целом механизм доступа иностранных продуктов на рынок ЕС является еще не сформированным. Из защитных мер можно отметить только статью 58 Директивы № 2004/17/ЕС, согласно которой «все заявки, поданные для заключения договора поставки, могут быть отклонены, если соотношение продукции, происходящей из третьих стран... превышает 50% общей стоимости продукции, составляющей предмет торгов». Именно по причине отсутствия единого подхода некоторые государства – участники ЕС принялись разрабатывать собственное законодательство, а в 2012 г. Комиссия ЕС опубликовала Проект регламента, которым планируется ограничить доступ компаний третьих стран к процессу государственных закупок в странах – участницах Европейского Союза.

Выводы

Данный раздел посвящен нормативно–правовой базе, применяемой в отношении информационных технологий в РФ, а также в иностранных государствах. Были рассмотрены и кратко описаны нормативно–правовые документы США, Китайской Народной Республики и Европейского союза, а также основные угрозы и последствия применения иностранного ПО для России.

4. ПРОГРАММНОЕ ПРОЕКТИРОВАНИЕ

Исходя из выше изученного материала, можно сделать вывод о том, что абсолютно любой протокол, доквантовый, квантовый, постквантовый, имеет свои уязвимости. Они связаны с невозможностью обеспечения всех свойств безопасности протокола, описанных выше. Стоит отметить, что, по большей части, уязвимости протоколов связаны с криптографическими алгоритмами, которые передают открытый ключ. Основная проблема при передаче ключа состоит в том, чтобы получатель смог идентифицировать этот ключ, т.е. определить его истинность. На основе этого вывода разрабатывается протокол, позволяющий защитить открытый ключ. Этот протокол способен работать совместно с любыми другими протоколами, тем самым обеспечив защиту ключа.

4.1 Разработка протокола

Прежде чем приступить к разработке протокола, необходимо определиться с требованиями, которыми должен обладать протокол:

- протокол должен быть четко и однозначно определен;
- протокол должен быть полным, действия его участников должны быть определены для любых возможных ситуаций;
- каждый участник должен заранее знать весь протокол, а также последовательность действий при выявлении его нарушений;
- каждый участник протокола обязан ему следовать;
- нарушение протокола участниками или вмешательство злоумышленника не должно снижать стойкость криптосистемы в целом, если ключи не раскрываются их владельцами.

В разрабатываемом протоколе должны выполняться следующие требования безопасности: G1, G2, G3, G5, G7, G8, G9, G10, G11, G12, G13, G18, G19. О данных требования можно почитать в пункте 1.1 данной работы.

Опишем работу протокола:

1. Отправитель формирует ключ аутентификации и отправляет его Получателю. Далее конвертирует его по согласованному алгоритму.
2. Получатель, получив ключ от Отправителя, переделывает его по согласованному алгоритму и отправляет его Отправителю.
3. Если сконвертированный и полученный ключ совпадает, то Отправитель действует согласно шагу 5.

4. Если сконвертированный и полученный ключ не совпадает, то Отправитель экстренно завершает сеанс, чтобы избежать утечку данных.

5. Отправитель формирует данные и шифрует их по второму алгоритму. После формирования отправляет Получателю.

6. Получатель декодирует данные их и проверяет их подлинность. В случае обнаружения атаки на данные отправляет уведомление Отправителю и завершает сеанс. В ином случае ждет других сообщений от Отправителя.

7. Если Отправитель решил завершить сеанс, он отправляет уведомление Получателю и ждет подтверждение о получении сообщения и готовности завершить сеанс. При любых обстоятельствах сеанс завершается.

Работа данного протокола представлена на рисунке 4.1.

Опишем механизм формирования данных или ключа, который будет отправляться по незащищенному каналу связи. Измененный передаваемый открытый ключ должен быть визуально для злоумышленника похожим на истинный, но при этом иметь в себе информацию, об исходном ключе и об изменениях, сделанных третьим лицом, чтобы идентифицировать саму подмену ключа.

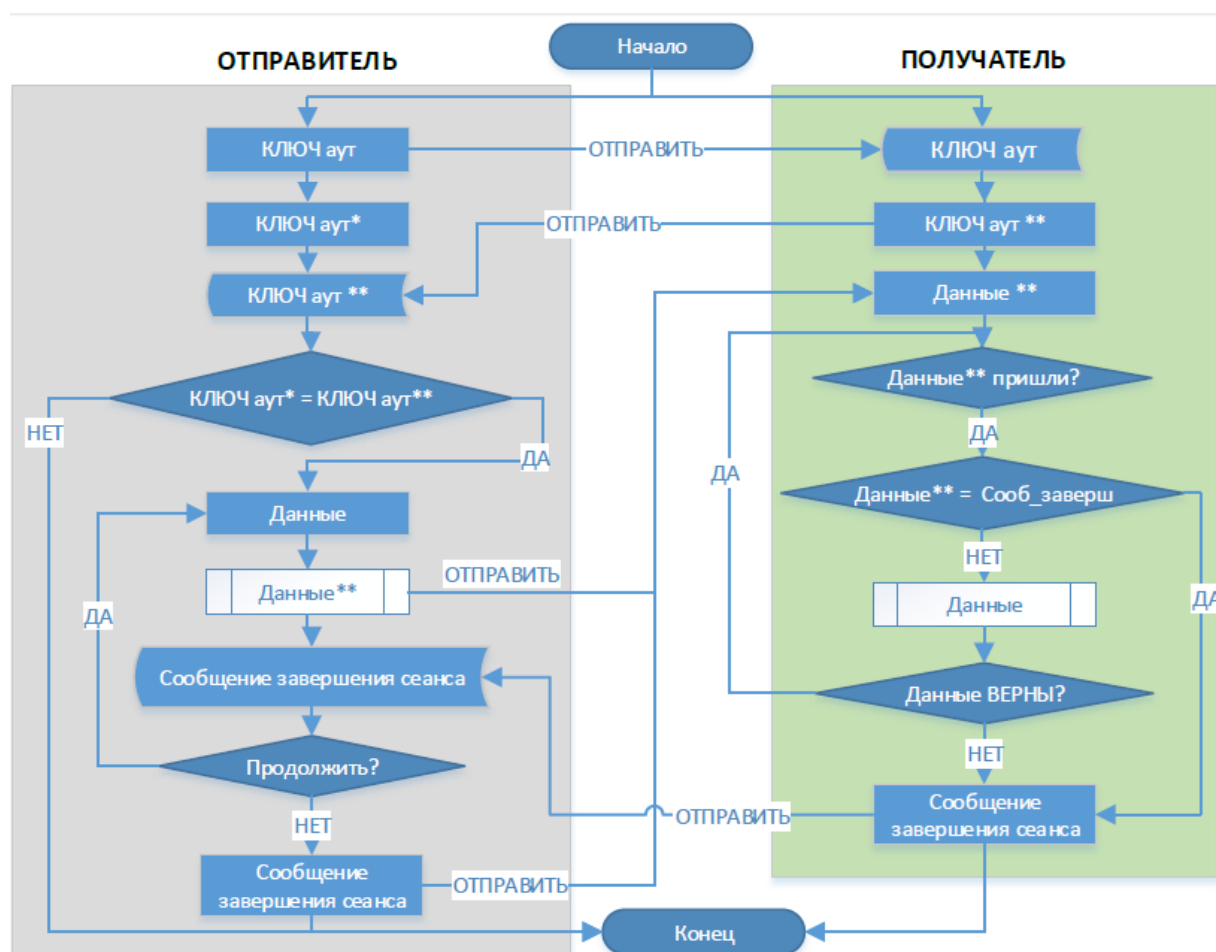


Рисунок 4.1 – Схема работы протокола

Для создания протокола защиты открытого ключа необходимо определить, какую информацию должен о себе содержать измененный ключ. В данной разработке измененный ключ содержит о себе следующую информацию:

- о теле измененного ключа;
- о размерность первоначального ключа;
- о правильности первоначального ключа;
- об алгоритме изменения ключа;
- о правильности зашифрованного ключа.

Именно такой информацией должен обладать легальный участник протокола, чтобы в дальнейшем мог получить истинный открытый ключ. Зашифрованный ключ представлен на рисунке 4.2:

ППК	ПЗК	ТИК	АИК	РПК
-----	-----	-----	-----	-----

Рисунок 4.2 – Визуальное представление зашифрованного ключа

Обозначения на рисунке:

1. *ППК* – правильность первоначального ключа.
2. *ПЗК* – правильность зашифрованного ключа.
3. *ТИК* – тело измененного ключа.
4. *АИК* – алгоритм изменения ключа.
5. *РПК* – размерность первоначального ключа.

В итоге мы получаем сразу три проверочных метки, по которым сможем проверить истинность нашего ключа – *ППК*, *ПЗК*, *РПК*. Если хотя бы одна из этих меток окажется неверной, то с уверенностью можно будет сказать, что данный ключ был изменен злоумышленником.

Опишем алгоритм формирования зашифрованного ключа:

1. Определим размерность первоначального ключа:

$$РПК = String(Ключ).Length()$$

2. Определим алгоритм изменения ключа:

$$АИК = Ключ[РПК - 1]$$

3. Определим правильность первоначального ключа:

$$ППК = \sum_{i=0}^N Ключ[i],$$

где N – это размерность ключа, т.е. равно *РПК*.

4. Убрать из ключа *АИК* и изменить ключ согласно алгоритму *АИК*.
5. Вернуть в измененный ключ *АИК*. Получим *ТИК*.
6. Создаем метку правильности зашифрованного ключа:

$$ПЗК = ТИК \bmod РПК$$

7. Формируем зашифрованный ключ:

$$ЗК = ППК + ПЗК + ТИК + РПК$$

Данный алгоритм предоставлен на рисунке 4.3:

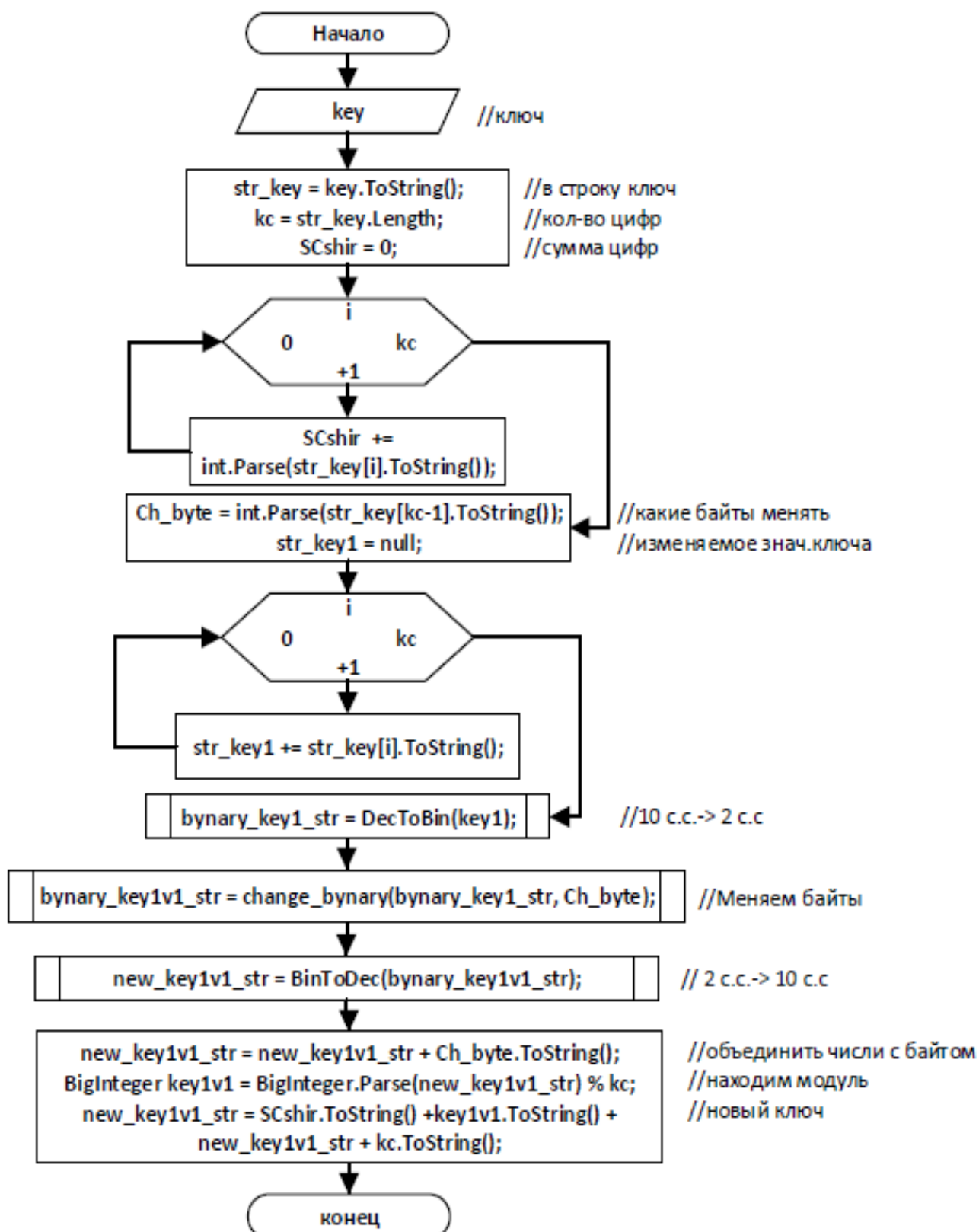


Рисунок 4.3 – Алгоритм формирования ключа по протоколу

Алгоритм декодирования ключа заключается в следующем:

1. Определить размерность первоначального ключа.
2. Определить алгоритм изменения ключа.
3. Определить правильность первоначального ключа.
4. Определить правильность зашифрованного ключа.
5. Проверить правильность зашифрованного ключа.
6. Декодировать, согласно алгоритму изменения ключа, тело изменённого ключа.
7. Проверить правильность первоначального ключа.
8. Проверить правильность размерности первоначального ключа.
9. Если *правильность первоначального ключа, правильность зашифрованного ключа и размерность первоначального ключа* верны, то и ключ верен. Иначе – произошла атака, и ключ был изменен.

Блок–схема данного алгоритма представлена на рисунках 4.4 – 4.6.

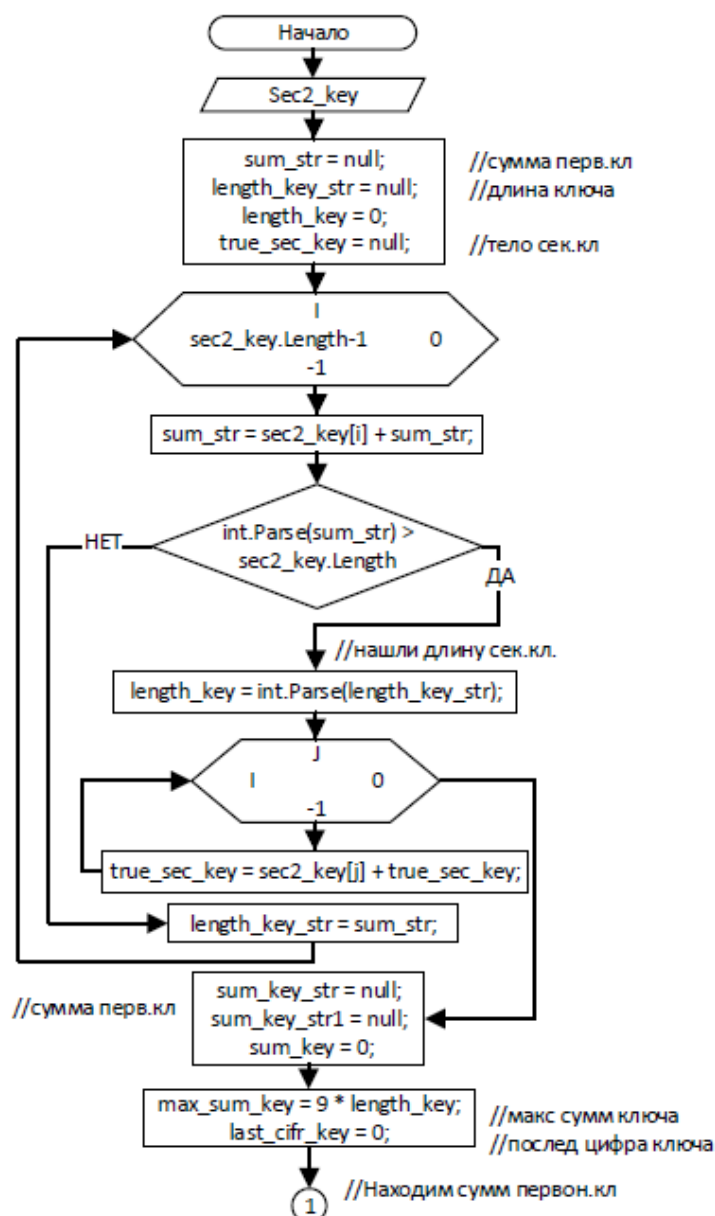


Рисунок 4.4 – Алгоритм декодирования, 1 часть

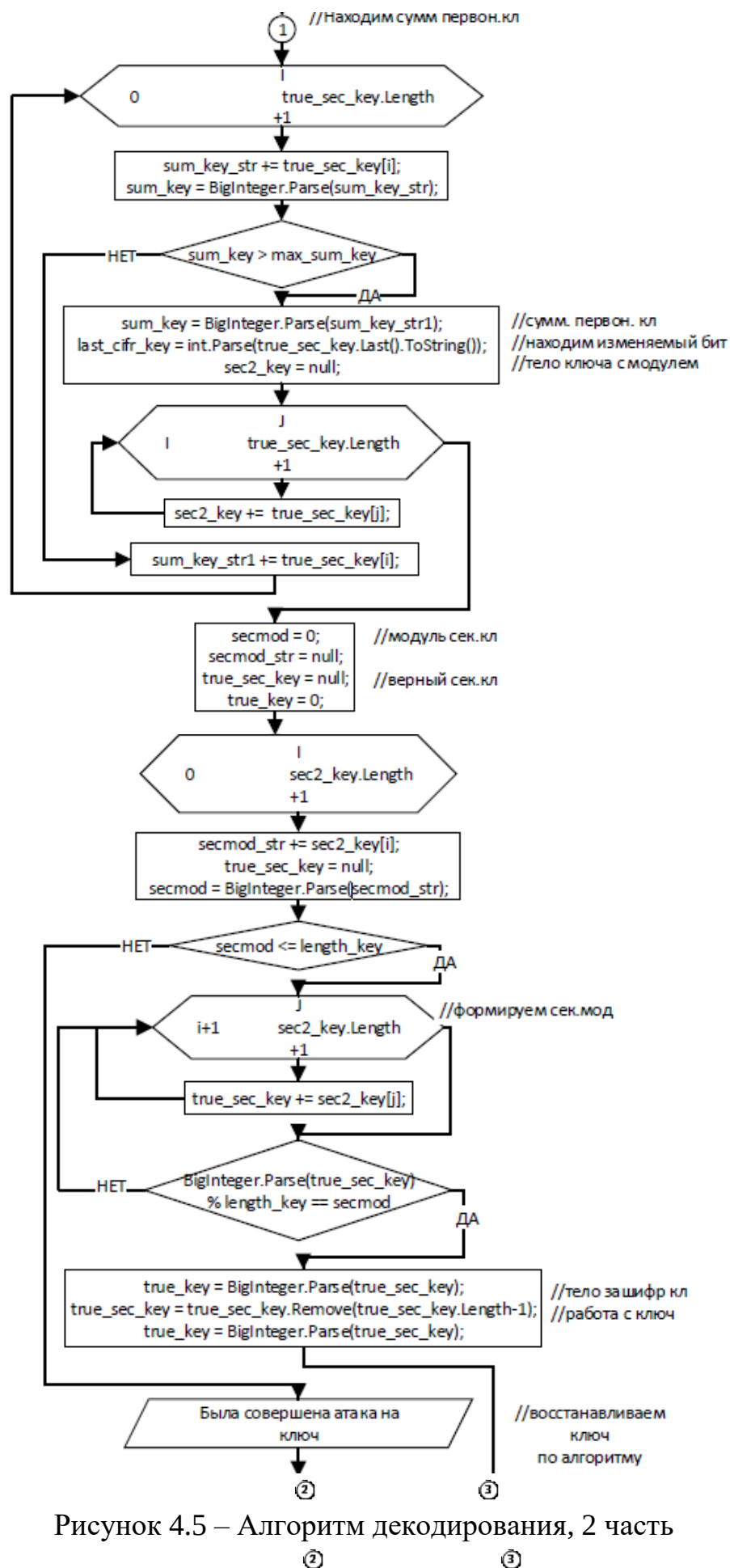


Рисунок 4.5 – Алгоритм декодирования, 2 часть

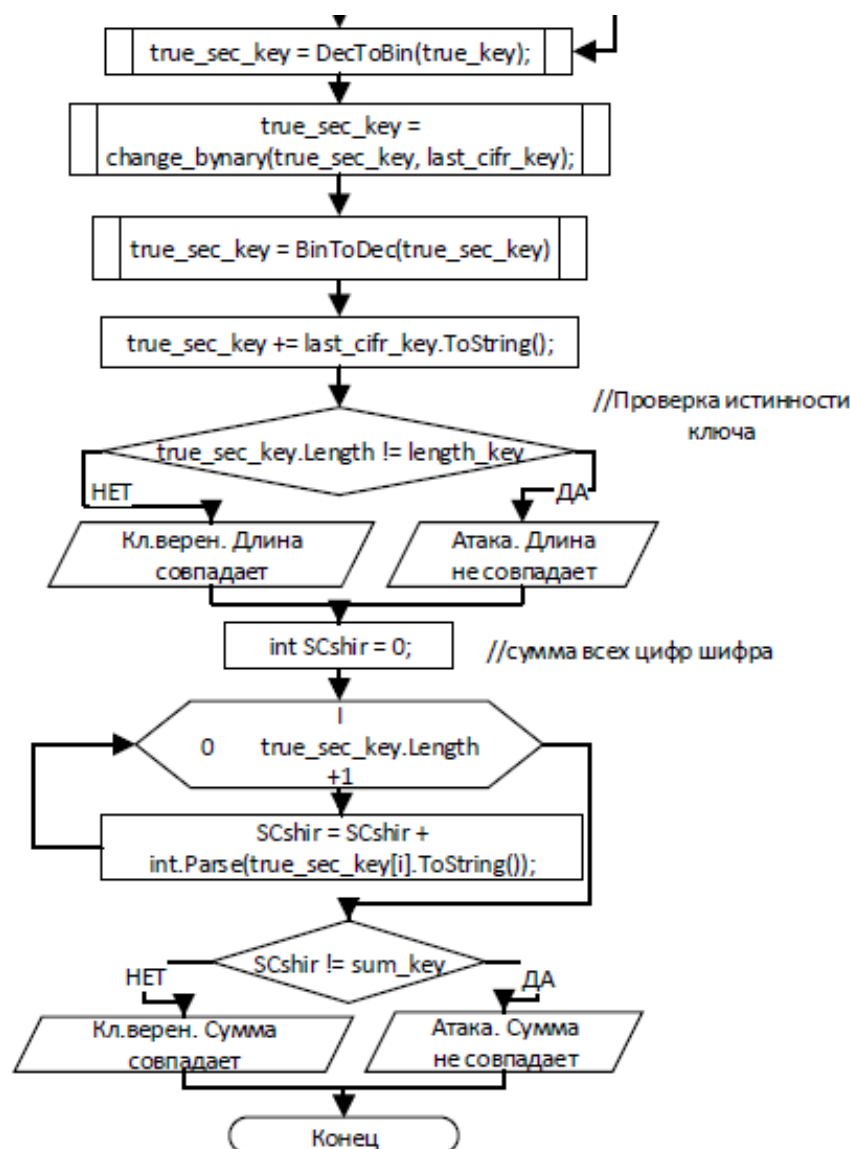


Рисунок 4.6 – Алгоритм декодирования, 3часть

4.2 Выбор языка программирования

В качестве языка программирования был выбран объектно-ориентированный язык программирования C# (C Sharp), относящийся к семье языков с C-подобным синтаксисом. В качестве среды разработки была выбрана Microsoft Visual Studio 2017 Enterprise. Главное преимущество данного программного продукта – разработка приложения с графическим интерфейсом, который позволяет пользователю быстро и легко разобраться в устройстве и схеме работы данного программного продукта, а также в нем реализованы различные библиотеки, содержащие криптографические функции и решена проблема работы с большими числами, что позволит полноценно реализовать работу протокола с целочисленным ключом выше типов `int` и `ulong`.

4.3 Программная реализация протокола

Общий вид диалогового окна программы, показывающую работу протокола, показан на рисунке 4.7.

The screenshot shows a window titled 'Form1' with two main sections: 'Отправитель' (Sender) and 'Получатель' (Receiver).

Отправитель (Sender):

- Ключ для аутентификации Получателя:** Two input fields with values '960568755' and '953005612', and a 'Сген. и отправить' button.
- Результат проверки - Отправитель:** A text area showing 'Results: Завершен'.
- Ключ:** An input field with '6157843191876543' and a 'Зашифровать ключ' button.
- Готовый ключ:** An input field with '787741364254389218316' and an 'Отправить' button.
- Уведомление о конце передачи: TRUE**
- Подтверждение об окончании передачи: TRUE**
- Results: Завершен**
- Действия:** A text area showing the process of generating and sending the key, including binary code and calculations.

Получатель (Receiver):

- Ключ для аутентификации от Отправителя:** An input field with '960568755' and a 'Сген. и отправить' button.
- Полученный ключ:** An input field with '787741364254389218316' and a 'Расшифровать ключ' button.
- Ключ:** An input field with '6157843191876543'.
- Result: - TRUE (module) - TRUE (lengthKey) - TRUE (su)**
- Подтверждение об окончании передачи**
- Results: Завершен**
- Действия:** A text area showing the process of receiving and verifying the key, including binary code and calculations.

Рисунок 4.7 – Внешний вид программы протокола

Сначала отправителю необходимо сформировать ключи аутентификации получателя, как показано на рисунке 4.8.

The screenshot shows the 'Отправитель' (Sender) section of the program. It has two input fields for 'Ключ для аутентификации Получателя' with values '960568755' and '953005612', and a 'Сген. и отправить' button.

Рисунок 4.8 – Ключи Отправителя для аутентификации Получателя

После того, как Получатель получил ключ от Отправителя, он его генерирует и отправляет обратно, как показано на рисунке 4.9.

The screenshot shows the 'Получатель' (Receiver) section of the program. It has one input field for 'Ключ для аутентификации от Отправителя' with the value '960568755', and a 'Сген. и отправить' button.

Рисунок 4.9 – Полученный ключ от Отправителя

Отправитель в свою очередь проверяет ключ Получателя, как показано на рисунке 4.10, и при успешной аутентификации продолжает работу, кодируя данные и отправляя их Получателю, как показано на рисунке 4.11.

Рисунок 4.10 – Проверка Получателя

Рисунок 4.11 – Кодирование данных по протоколу

При необходимости завершить сеанс, Отправителю нужно нажать кнопку «Завершить передачу», и тогда в поле «Results» можно будет увидеть, отреагировал или нет Получатель на сообщение о завершении. После получения данных Получатель их декодирует, и может увидеть результат своих действий. В случае обнаружения ошибки при декодировании, Получателю необходимо отправить уведомление о завершении передачи, которое позволит проинформировать Отправителя о том, что была совершена атака и ключ. Эти действия продемонстрированы на рисунке 4.12. В случае получения уведомления от Отправителя о завершении сеанса, ему также необходимо ответить на сообщение «Завершить передачу». Тем самым происходит подтверждение того факта, что данные были отправлены и получены.

Рисунок 4.12 – Процесс декодирования ключа Получателя

Оператор или администратор этого протокола может наблюдать все изменения в работе протокола в поле «Действия», выполняемые участниками сеанса, как продемонстрировано на рисунке 4.13.

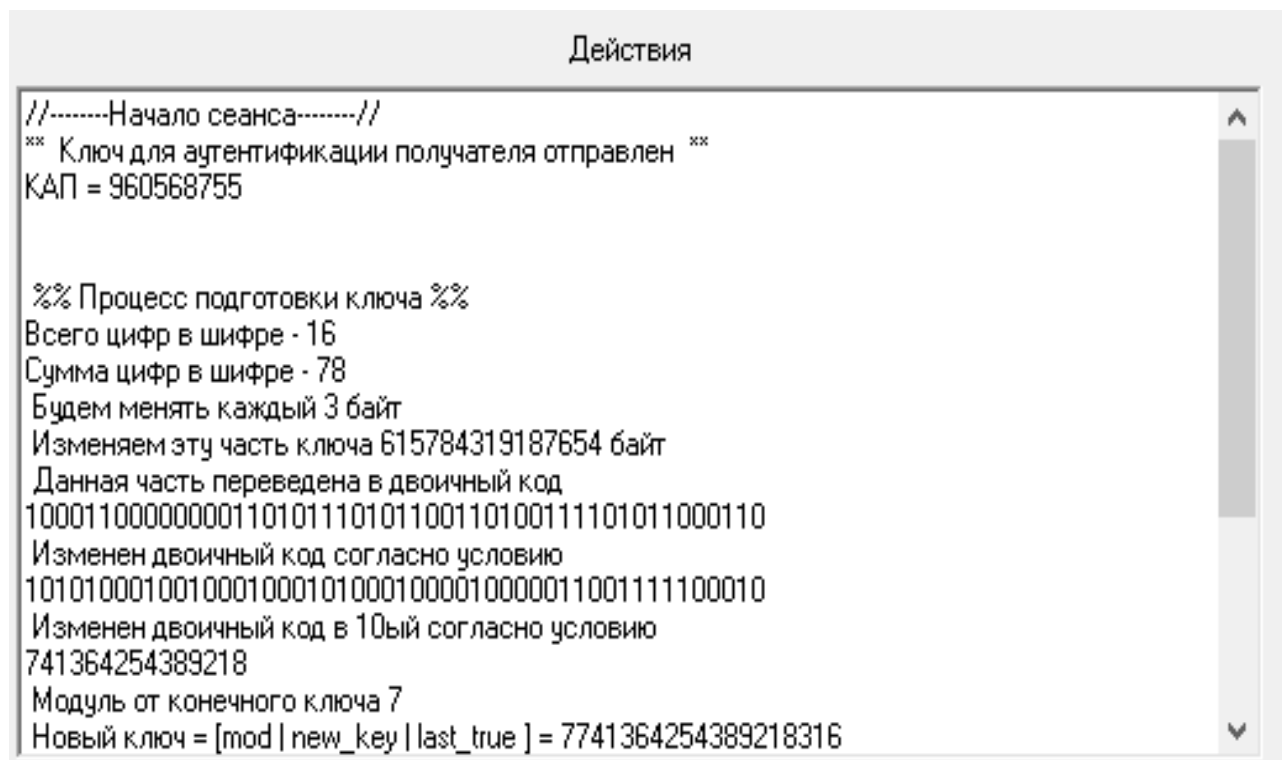


Рисунок 4.13 – Поле «Действия» для наблюдения работы протокола

4.4 Криптоанализ разработанного протокола

В разработанном протоколе соблюдены свойства безопасности протокола:

- аутентификация субъекта;
- аутентификация сообщения;
- защита от повтора;
- аутентификация источника;
- аутентификация ключа;
- подтверждение правильности ключа;
- защита от чтения назад;
- защищенная возможность договориться о параметрах безопасности;
- конфиденциальность;
- подотчетность;
- доказательство отправления;
- доказательство получения.

Данные свойства безопасности представлены в таблице ниже.

Таблица 4.1 – Свойства безопасности разработанного протокола

№	Код	Название	Соответствует
1	G1	Аутентификация субъекта	+
	G2	Аутентификация сообщения	+
	G3	Защита от повтора	+
2	G4	Неявная (скрытая) аутентификация получателя	
	G5	Аутентификация источника	+
3	G6	Авторизация (доверенной третьей стороны)	
4	G7	Аутентификация ключа	+
	G8	Подтверждение правильности ключа	+
	G9	Защита от чтения назад	+
	G10	Формирование новых ключей	+
	G11	Защитная возможность договориться о параметрах безопасности	+
5	G12	Конфиденциальность	+
6	G13	Обеспечение анонимности при прослушивании	+
	G14	Обеспечение анонимности при работе с другими участниками	
7	G15	Ограниченная защищенность от DoS–атак	
8	G16	Неизменность отправителя	
9	G17	Подотчетность	
	G18	Доказательство отправки	+
	G19	Доказательство получения	+
10	G20	Безопасное временное свойство	

Недостатки безопасности протокола:

- невозможность неявной (скрытой) аутентификации;
- невозможность обеспечения анонимности при прослушивании;
- невозможность обеспечения анонимности при работе с другими участниками;
- невозможность защититься от DoS–атак;
- невозможность определения инвариантности отправителя;
- не выполняется безопасное временное свойство.

Теперь рассмотрим используемый метод шифрования. Преимущества этого метода шифрования состоит в том, что атакующий должен знать алгоритм шифрования по значению *алгоритма изменения ключа* – их может быть от 1 до 9, при том, что эти алгоритмы четко не определены под номером и могут быть усовершенствованы и согласованы на усмотрение пользователей. Поэтому перебор вариантов на установление алгоритма может затянуться на довольно большой срок.

Контрольная сумма ключа или его длина (*размерность первоначального ключа*) статичная, но в зависимости от алгоритма шифрования, а у нас их 9, зашифрованный ключ может увеличиться или уменьшиться в размере, что создаст сложности в вычисления верного значения истинности зашифрованного ключа, т.е. значения *правильности зашифрованного ключа*.

Контрольная сумма *ППК* показывает правильность первоначального ключа. Нарушителю довольно непросто будет подменить сообщение так, чтобы все три критерия – *правильность первоначального ключа, правильность зашифрованного ключа и размерность первоначального ключа* подтвердили, что сообщение истинно. Для качественной атаки подмены ключа на данный протокол злоумышленнику необходимо будет создать довольно сложную математическую модель.

Авторами рекомендуется использовать данный протокол совместно с другими протоколами, что позволит качественнее защитить ключ и нейтрализовать недостатки и уязвимости одного из протоколов.

Выводы

Данный раздел посвящен разработке протокола. Приведены и описаны требования к протоколу, алгоритм работы протокола, алгоритм защиты передаваемой информации, блок-схемы кодирования и декодирования, описание и обоснование выбранного языка программирования, описана общая структура программы, а также сделан криптоанализ данной разработки, в котором найдены сильные и слабые стороны разработанного протокола.

Разработанная программа представляет объектно-ориентированное приложение с простым и понятным интерфейсом. В программе представлена работа двух субъектов разработанного протокола. Приведены рисунки, демонстрирующие работу каждого из них с дополнительными пояснениями автора.

РЕЗУЛЬТАТЫ 1 ГЛАВЫ

В данном исследовании были изучены современные криптографические протоколы, обоснованы возможности их импортозамещения, а также спроектирован и реализован новый криптографический протокол, который способен повысить криптостойкость указанных протоколов и защитить открытый ключ в открытом канале связи, что и является целью исследований.

Выполнен ряд следующих задач:

- анализ различных криптографических протоколов;
- анализ алгоритмов работы современных протоколов, согласно классификации;
- поиск уязвимостей и совместный криптоанализ протоколов;
- разработка нового протокола, совместимого с известными доквантовыми, квантовыми и постквантовыми протоколами, повышающего их криптостойкость при проведении кибератак на открытый ключ;
- разработка ПО, демонстрирующего работу защищенного протокола;
- криптоанализ и проверка надёжности работы разработанного протокола.

Это программное приложение является кроссплатформенным, и имеет интуитивно понятный интерфейс. В основе разработанного ПП лежит собственный разработанный алгоритм кодирования и декодирования открытого ключа.

Для реализации и анализа разработанного криптографического протокола задействованы свободные источники, однако они не дают подробного описания ввиду секретности разработанных алгоритмов. В инструкции для пользователей подробно описан сценарий их работы. Алгоритмы кодирования и декодирования передаваемых сообщений имеют подробное пошаговое описание.

ГЛАВА 2

РАЗРАБОТКА И АНАЛИЗ ЭФФЕКТИВНОГО ИМПОРТОЗАМЕЩАЮЩЕГО СРЕДСТВА ПРОТИВОДЕЙСТВИЯ DDOS-АТАКАМ

ОСНОВНЫЕ ПОЛОЖЕНИЯ

В сегодняшнем мире глобализации и развития информационных технологий не представляется возможным построение бизнеса и производства без использования новейших достижений науки и техники, связанных с информационными технологиями. Развитие нормативной базы в Российской Федерации, касающейся вопросов импортозамещения, упрочивает процесс внедрения этого вида технологий в современный отечественный рынок. Наиболее перспективными ответвлениями направления российских импортозамещающих информационных технологий являются развитие и применение криптографических средств защиты информации для различных целей, включая их использование государственными органами и коммерческими организациями, имплементация средств электронного документооборота, расширение применения схем шифрования на основе открытых ключей и средств защиты Web-ресурсов от DoS-атак.

Среди преимуществ применения подобного рода технологий выделяются: более эффективное ведение делопроизводства, которое достигается посредством снижения вероятности искажения передаваемой информации, а также её повреждения и утери, меньшее время передачи информационных пакетов по сети, а также оптимизация поисковых способностей соответствующих агрегаторов, что позволяет иметь возможность пользователю в наикратчайшие сроки получить искомые данные. Введение в эксплуатацию электронной цифровой подписи как криптографической системы способно в полной мере обеспечить выполнение основных свойств информации: *целостности* и *конфиденциальности*. Но в то же время, использование ЭЦП не предоставляет гарантии по обеспечению другого важнейшего свойства информационных ресурсов – *доступности*. Для того, чтобы имелась возможность непрерывного доступа к условным центрам сертификации со стороны пользователей системы, необходимо обеспечение безотказного функционирования сервиса инфраструктуры, осуществляющей обработку и генерацию открытых ключей.

В наши дни одними из наиболее актуальных разновидностей кибератак являются *атаки типа DoS (DDoS)*, целью которых является осуществить *отказ в обслуживании сервером*. В случае проведения данной атаки, её успешная реализация может ограничить, или обеспечить полную недоступность ресурсов информационной системы, расположенной на сервере, для всех без исключения её пользователей. В качестве последствий данной кибератаки могут оказаться как финансовые убытки для владельца информационной системы, так и выход из строя всей аппаратной части в целом [16].

Среди множества способов осуществления DDoS-атак наиболее эффективной является атака, направленная на обеспечение большой загруженности ресурсов, располагающихся на сервере, за счёт обработки большого числа невалидных запросов, отправляемых из сети ботнет, которая наиболее часто организуется из зараженных устройств реальных пользователей, имеющих выход в сеть, которые даже не подозревают тот факт, что с их устройства ведется какая-либо атака. Таким образом, вредоносное ПО устанавливается на незащищенные устройства, подключенные к сети, посредством внедрения в систему вирусов, червей, троянских коней. Так происходит заражение большого количества хостов сети, после которого контроль над этими машинами переходит к злоумышленнику. Затем атакующим субъектом производится реализация атаки, основанной, в первую очередь, на осуществлении одновременной отправки всем узлам зараженной сети команды, вследствие которой происходит активация программного обеспечения, внедренного в устройство пользователя, и переход данных хостов под контроль злоумышленника, таким образом являясь источником распределенной атаки, направленной на отказ в обслуживании.

Также стоит упомянуть, что реализация данного типа атак для злоумышленника имеет смысл с точки зрения получения финансовой прибыли. В качестве примера рассмотрим два способа: в первом случае организацию и осуществление атаки оплачивает конкурирующая организация с целью нанесения репутационного ущерба жертве атаки, а также привлечения себе клиентов, использующих услуги (продукцию) атакуемого ресурса; во втором случае атакующий субъект использует атаку с целью вымогательства финансовых средств, требуя оплату от жертвы для прекращения атаки.

На сегодняшний день имеется ряд факторов, препятствующих эффективному противодействию и отражению распределенных атак. Также достаточно сильно усложняет процесс поиска злоумышленника в связи с тем, что подавляющее большинство атак не предполагает установления сеанса связи с атакующим источником. Более того, во многих государствах в силу несовершенства законодательства представляется невозможным получение наказания злоумышленником даже в случае его поимки.

На мировом рынке предлагаются услуги защиты от DDoS-атак, которыми обеспечивается уменьшение отрицательного воздействия от данного рода воздействий на информационные ресурсы в некоторых случаях до 99%, но в то же время они обладают рядом недостатков, ограничивающих их применение.

Одним из главных ограничений является тот факт, что большинство высокоэффективных алгоритмов защиты от данных атак являются интеллектуальной собственностью организаций, выраженной в *коммерческой тайне разработчиков алгоритмов*, что не дает возможности многим компаниям иметь возможность сертифицировать данные продукты. Одним из недостатков также является высокая стоимость услуг защиты от DDoS-атак, которая является довольно дорогим удовольствием для многих организаций. К примеру, услуги компании Cloudflare, специализирующейся на подобной защите, для защиты сетей коммерческих организаций составляет порядка 10 тысяч долларов в месяц.

Подытожив вышеописанное, необходимо сделать вывод, что проблема противодействия и отражения атак типа «Отказ в обслуживании» является актуальной проблемой для российской инфраструктуры, в связи с повсеместным внедрением информационных технологий и глобализации Интернета, что, в свою очередь, требует эффективных импортозамещающих методов защиты данных, соответствующих адекватной стоимости оказания услуг.

В предлагаемой работе предлагается практическая реализация методики обнаружения одной из наиболее распространенных DDoS-атак на уровне L7 модели OSI – HTTP Flood. В основе этой методики находится математическая модель, которая воспроизводит взаимодействие клиентов с сервером. Рассмотренная в работе модель принимает во внимание индивидуальные характеристики некоторых параметров, описывающих работу сети и сервера, что влияет на возможность более эффективного обнаружения атаки. Более того, в разработке предлагается программная реализация указанной методики, выраженная в виде модуля расширения функциональности для системы отражения большого числа невалидных запросов с атакующей стороны HTTP_Flood_reject. Реализация подобного решения направлена на защиту критически важных ресурсов государственных, военных, промышленных, банковских и коммерческих организаций, а также физических лиц в целях предоставления услуг защиты информации от возможной кибератаки.

1 ОПИСАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

1.1 Предмет информационной системы

В конце XX века наше общество вступило в принципиально новую эпоху, которую принято называть эрой *высоких технологий*. В наши дни достаточно широкое применение приобретают новейшие достижения в области науки и техники, проникая во все сферы жизни общества. Ярким проявлением этого развития выступает сфера информационных технологий, проникая вглубь общества, предоставляет ему все больше возможностей для коммуникаций и всяческих коллабораций. Разновидностью информационных технологий является информационная система, элементы которой широко внедряются в жизнь современного общества.

Информационная система представляет собой систему аппаратных, программных и программно–аппаратных средств, осуществляющих создание, обработку, передачу, хранение и уничтожение информационных ресурсов, а также обмен данными посредством их взаимодействия.

Одним из самых распространенных примеров информационной системы является глобальная сеть Интернет, которая, благодаря своей глобализации, на текущий момент уже охватывает большую половину земного шара, таким образом предоставляя возможность людям из разных слоев общества иметь доступ к информационным ресурсам. Физически глобальная сеть Интернет представляет собой совокупность огромного числа глобальных и локальных сетей, которые находятся в собственности коммерческих и государственных учреждений и соединенных линиями связи на физическом уровне. Условно мировую паутину можно вообразить в виде огромной сети, которая состоит из небольших сетей различных размеров и величин, которые имеют возможность осуществлять взаимодействие с другими сетями в рамках одной большой глобальной сети посредством передачи данных. Имея довольно низкую стоимость услуг, которая в большинстве случаев выражается в виде определенной ежемесячной оплаты за использование телефонной (оптоволоконной) линии или мобильной связи, пользователи получают доступ к информационным ресурсам различного характера, предоставляемым государственными и коммерческими организациями всех стран, имеющих выход в глобальную сеть. На просторах сети Интернет пользователи имеют возможность получать информацию, касающуюся повседневной жизни, такую как наличие пробок на каком–либо отрезке маршрута, а также общественно важной жизни населения, как мировые новости или политическая ситуация в том или ином государстве.

Таким образом, благодаря всемирной сети Интернет люди по всему миру приобрели небывалую возможность иметь связь, которая, к тому же, недорогая и весьма надёжная, по всему земному шару. Касательно развития бизнеса и деловых отношений это делает ведение делопроизводства более эффективным как для небольших фирм, так и для многомиллионных корпораций, которые обладают инфраструктурой по всему земному шару. Данный факт сопровождается тем, что, как правило, использование инфраструктуры сети Интернет сопровождается меньшими финансовыми вложениями для осуществления коммуникации, нежели телефонная или спутниковая связь.

Среди услуг глобальной сети особняком выделяется электронная почта, использование которой также имеет достаточно много преимуществ, прежде всего финансовых, перед обычным письмом или СМС. Сравнивая с обычным письмом, явно выделяется преимущество по времени пересылки, которое в случае электронного письма занимает порядка нескольких секунд, а при обычном – до нескольких недель в зависимости от расстояния.

Другой полезной и широко применяемой услугой глобальной сети является WWW, что представляет собой систему работы с гипертекстом, которая также является эффективным средством поиска. Гипертекст объединяет разные документы на основе заданного набора слов. Данная технология осуществляется следующим образом: если в каком-либо наборе текста встречается новое слово, то система, которая осуществляет работу с гипертекстом, предоставляет возможность перехода к другому документу, в котором данное слово рассмотрено более подробно. Также WWW в некоторых случаях используется для более понятного интерфейса для работы с базами данных.

В современном мире, для успешного осуществления взаимодействия во всех уровнях жизни общества, а в особенности коммерческой, наблюдается рост зависимости субъектов информационных отношений от использования информационных технологий. Рассмотрим на рисунке 1.1 типовую схему корпоративной сети организации, с разветвленной структурой филиалов, расположенных удаленно, и работающей дистанционно.

На примере рассматриваемой информационной системы можно заметить, что в организации, коммуникационная сеть которой построена по подобной схеме, имеется возможность проводить конференции с участием сотрудников, которые работают в другом городе или даже государстве, без необходимости нахождения в одном помещении, что снижает затраты компании на необходимость во взаимодействии различных ее отделов и филиалов.

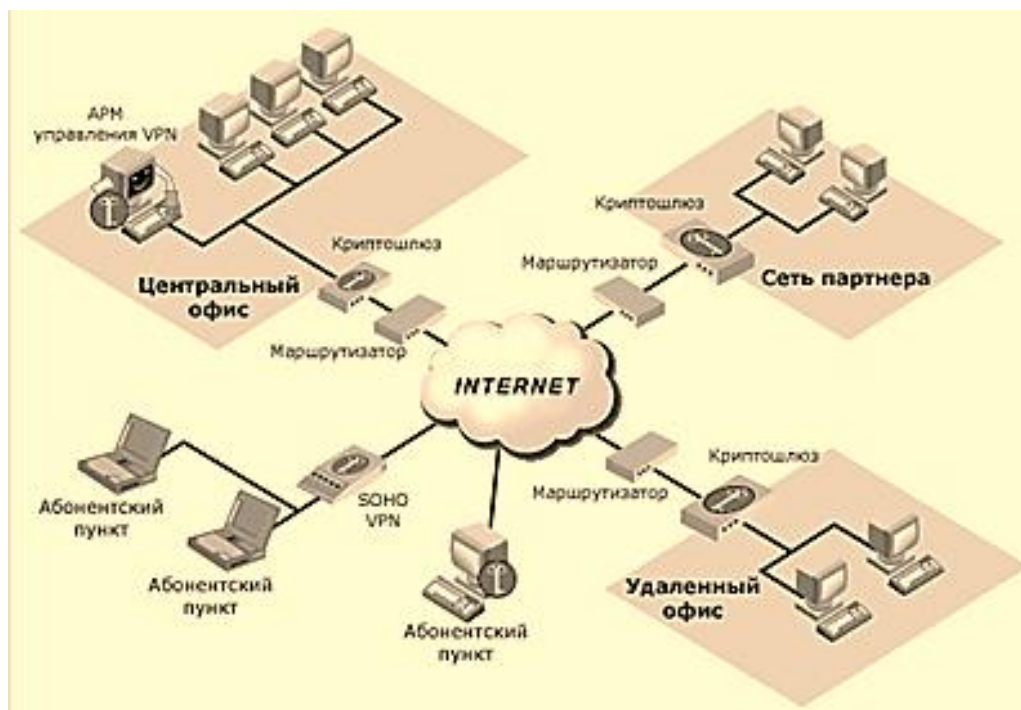


Рисунок 1.1 – Пример информационной системы

В подобных информационных системах происходит обработка информации различного содержания и назначения, среди чего может находиться как критически важная для компании информация, относящаяся к коммерческой тайне, так и персональные данные клиентов, использующих услуги этой организации. Любая организация несет ответственность за хранение, обработку, накопление и передачу такого рода информации, что также влечет за собой крупные риски в случае ее утечки, в первую очередь, финансовые и репутационные. Следовательно, ко всем информационным системам предъявляется определенный ряд требований по обеспечению основных услуг, что будет рассмотрено ниже. Причиной нарушения предоставления основных услуг могут быть как действия случайного характера, так и заранее продуманные и спланированные действия злоумышленника.

Классификация моделей возможных злоумышленников в информационной системе приведена в следующем пункте.

1.2 Модель нарушителя

Нарушителем является физическое лицо, осуществляющее преднамеренно или случайным образом действия, результатом которых является нарушение безопасности персональных данных в процессе их обработки техническими средствами в информационной системе, а также это пользователь, который осуществляет несанкционированный доступ к информации.

Стоит отметить, что под несанкционированным доступом в данном случае подразумевается, как получение, обработка и хранение информации, так и внесение в нее изменений и удаление. В рамках данной работы к понятию нарушителя расширяется в связи с тем, что злоумышленник также может и не являться пользователем системы, подверженной атаке.

В соответствии с нормативными документами, имеющими юридическую силу на территории Российской Федерации, определены два типа нарушителей по принципу наличия права легального доступа в помещения, в которых размещены аппаратные средства, обеспечивающие доступ к ресурсам персональных данных информационной системы (ИСПДн):

- нарушители, которые не обладают правом легального доступа на контролируруемую зону территории (помещения) – категория I;
- нарушители, которые обладают законным правом легального доступа на контролируемую зону территории (помещения) – категория II.

Также возможные потенциальные нарушители классифицируются на:

- внешние нарушители, которые производят атаки с территорий вне пределов контролируемой зоны ИСПДн;
- внутренние нарушители, которые производят атаки, находясь в пределах контролируемой зоны ИСПДн.

Также необходимо упомянуть, что лица категории II выступают в качестве внешнего нарушителя, находясь за пределами контролируемой зоны.

Внешними нарушителями касательно ИСПДн могут являться следующие субъекты: уволившиеся сотрудники организации, посторонние лица, желающие с какой-либо целью получить доступ к ПДн; лица, состоящие в преступных организациях. Внешний нарушитель способен осуществлять следующие деструктивные воздействия на информационную систему:

- перехват обрабатываемых техническими средствами ИСПДн ПДн за счёт их утечки по техническим каналам утечки информации, в том числе с использованием портативных, возимых, носимых, а также автономных автоматических средств разведки серийной разработки;
- деструктивные воздействия через элементы информационной инфраструктуры ИСПДн, которые в процессе своего жизненного цикла (модернизация, сопровождение, ремонт, утилизация) оказываются за пределами КЗ;
- несанкционированный доступ к информации с использованием специальных программных воздействий посредством программы вирусов, вредоносных программ, алгоритмических или программных закладок;

– перехват информации, передаваемой по сетям связи общего пользования или каналам связи, не защищенным от несанкционированного доступа (НСД) к информации организационно–техническими мерами;

– атаки на ИСПДн путем реализации угроз удаленного доступа.

Внутренний нарушитель (лица категории II) подразделяется на восемь групп в зависимости от способа и полномочий доступа к информационным ресурсам (ИР) информационной системы с персональными данными.

1. К первой группе относятся сотрудники предприятий, не являющиеся зарегистрированными пользователями и не допущенные к ИР ИСПДн, но имеющие санкционированный доступ в контролируемую зону (КЗ). К этой категории нарушителей относятся сотрудники различных структурных подразделений предприятий: энергетики, сантехники, уборщицы, сотрудники охраны и другие лица, обеспечивающие нормальное функционирование объекта информатизации.

2. Ко второй группе относятся зарегистрированные пользователи ИСПДн, осуществляющие ограниченный доступ к ИР ИСПДн с рабочего места. К этой категории относятся сотрудники предприятий, имеющие право доступа к локальным ИР ИСПДн для выполнения своих должностных обязанностей.

3. К третьей группе относятся зарегистрированные пользователи подсистем информационной системы с персональными данным, осуществляющие удаленный доступ к ПДн по локальной или распределенной сети предприятий.

4. К четвертой группе относятся зарегистрированные пользователи ИСПДн с полномочиями администратора безопасности сегмента (фрагмента) ИСПДн.

5. К пятой группе относятся зарегистрированные пользователи с полномочиями системного администратора ИСПДн, выполняющего конфигурирование и управление программным обеспечением и оборудованием, включая оборудование, отвечающее за безопасность защищаемого объекта: средства мониторинга, регистрации, архивации, защиты от несанкционированного доступа.

6. К шестой группе относятся зарегистрированные пользователи ИСПДн с полномочиями администратора безопасности ИСПДн, отвечающего за соблюдение правил разграничения доступа, за генерацию ключевых элементов, смену паролей и криптографическую защиту информации (СКЗИ). Администратор безопасности информационной системы с персональными данными осуществляет аудит тех же самых средств защиты объекта, что и системный администратор.

7. К седьмой группе относятся лица из числа программистов–разработчиков сторонней организации, являющихся поставщиками ПО и лица, обеспечивающие его сопровождение на объекте размещения ИСПДн.

8. К восьмой группе относятся персонал, обслуживающий технические средства информационной системы с персональными данными, а также лица, обеспечивающие поставку, сопровождение и ремонт ТС ИСПДн.

Далее в работе будет рассмотрена модель вероятных угроз в описанной информационной системе с персональными данными.

1.3 Модель угроз

В соответствии с данными, приведёнными в работе [17], угроза может быть выражена преднамеренными или случайными действиями физических лиц, иностранных разведывательных служб или организаций, в том числе криминальных, которые создают условия, при которых происходит нарушение безопасности персональных данных, вследствие чего возникает ущерб критически важных интересов государства, общества и личности. Таковой ущерб может быть выражен в виде нарушения некоторых характеристик информационных ресурсов, вызванного посредством деструктивного воздействия такого, как искажение, модификация или незаконное ознакомление, либо в виде подобного воздействия в отношении информационной системы. В качестве источников данных угроз имеет смысл рассматривать также объекты и явления, что, в свою очередь, представляет сложности при создании всеобъемлющей системы защиты информационных ресурсов. По этой причине во всех видах организаций создается модель угроз, содержащая классификацию вероятных угроз, их описание и способы реализации.

1.3.1 Классификация угроз в соответствии с IT–Baseline Protection Manual

Среди большого числа международных нормативных документов, регламентирующих классификацию угроз, находится Standard BSI. В данном документе рассматривается перечень вероятных угроз. А также приведены технические и организационные меры по осуществлению защиты. В соответствии с классификацией, угрозы делятся на 5 категорий, приведенных ниже [18].

1. Угрозы, которые возникают в связи с непредвиденными и непредсказуемыми событиями такими, как стихийное бедствие, революция, военные действия и прочее.
2. Угрозы, возникающие в связи с несовершенством организации и управления.
3. Угрозы, обусловленные человеческим фактором.
4. Угрозы, возникающие по причине технических неисправностей.
5. Угрозы, возникающие посредством заранее спланированных действий злоумышленников с атакующей стороны.

Также, рассматривая каждую из представленных выше вероятных угроз, в нормативном документе представлен всеобъемлющий перечень возможных угроз.

В пунктах, описанных ниже, будут рассмотрены некоторые из наиболее вероятных угроз, отнесенных к каждой из категорий.

1.3.1.1 Угрозы, которые возникающие в связи с непредсказуемыми событиями

Угрозы, относящиеся к данной категории, достаточно трудно предсказать в связи с тем, что их возникновение происходит абсолютно случайным образом во многих ситуациях. Одним из примеров подобной угрозы является проблема кадров, а именно — проблема, напрямую связанная с человеком как работником. В данном случае угроза может представлять собой потенциальный ущерб в виде болезни или смерти работника, а также участие персонала в различного рода забастовках, что ведет к возникновению пробелов и прерываний в выполнении рабочих задач.

Другими источниками угроз, входящих в данную категорию, являются всевозможные чрезвычайные происшествия, аварии и катастрофы как природного, так и техногенного характера, вызванные человеческим фактором.

Непосредственно к угрозам этой категории относятся слишком высокие и слишком низкие температуры, недопустимые угрозы, обусловленные человеческим фактором, уровень влажности, пыль и другое, что может вызвать неполадки в корректном функционировании информационной системы.

Большинство сложных систем построено таким образом, что все элементы взаимосвязаны друг с другом, или имеют зависимость от работы других подсистем, что в случае отказа в работе одного из элементов подсистемы вызовет недоступность информационной системы в целом. В качестве примера рассмотрим резкий скачок напряжения электрической сети, в результате которого произошел выход из строя мощного сервера, на котором обрабатывалась база данных пользователей какой-либо организации. Таким образом, данный сбой имеет большую вероятность привести к сбою работу всей информационной системы.

1.3.1.2 Угрозы, возникающие в связи с несовершенством организации и управления

Довольно часто происходит возникновение угроз в связи с недоработками организационного характера, среди которых можно выделить отсутствие или несовершенство нормативных документов и правил, пренебрежение доведением этой информации работникам организации, неквалифицированный мониторинг и аудит мер по информационной безопасности.

К примеру, если в политике безопасности информационной системы не указан в явном виде запрет на использование неучтённых носителей данных, или работник не ознакомлен с этим правилом, то увеличивается вероятность проникновения в корпоративную сеть вредоносного программного обеспечения, внедренного с помощью внешнего накопителя информации.

Другим примером угрозы, относящейся к рассматриваемой категории, является недостаточное финансирование, а также неэффективное использование финансовых и материальных ресурсов. Ярким примером касательно информационной системы является невозможность использования новейших операционных систем, в связи с чем в организации отсутствует возможность аудита выполненных пользователями действий.

Ошибки, осуществляемые в процессе проектирования информационной системы, также относятся к данной категории угроз. В реальном проекте ошибка при проектировании и расчета пропускной способности канала связи приведет с одной стороны к более высоким затратам на услуги связи, а с другой к более низкой скорости работы некоторых компонентов системы.

Более того, в некоторых случаях происходит тестирование с использованием реальных данных или неквалифицированное тестирование. Таким образом, в данную категорию попадает и понятие тестирования.

1.3.1.3 Угрозы, обусловленные человеческим фактором

В рассматриваемую категорию угроз прежде всего попадают те, которые представляют собой результат некорректных действий пользователя системы, вследствие которых повышается вероятность утраты конфиденциальности или целостности данных. Элементарным примером может служить распечатка конфиденциальных документов на принтере, после которой работник забывает забрать некоторые документы. В этом случае ознакомиться с конфиденциальной информацией сможет любой человек, находящийся в месте распечатки и не имеющий привилегий быть обладателем данного вида информации. В информационной системе потеря целостности или конфиденциальности вполне вероятна в случае некорректной настройки прав доступа к файлам и папкам.

Подытожив текущий пункт, отметим, что большинство угроз этой категории возникает в связи с некорректным администрированием и управлением системой.

1.3.1.4 Угрозы, возникающие по причине технических неисправностей

Как видно из названия данной категории, источниками угроз в данном случае являются неисправности, связаны с технической составляющей информационной системы. В качестве неисправности могут быть неполадки в сетях электро-, водо-, теплоснабжения, в сетях связи и др. При появлении подобной угрозы возникает достаточно высокая вероятность выхода из строя аппаратной части оборудования, работа которого является необходимой составляющей частью функционирования информационной системы или её элементов.

К угрозам рассматриваемой категории также можно отнести уязвимости и ошибки программного обеспечения. В качестве примера, в некоторых приложениях присутствует возможность применять криптографические методы защиты документов, однако она включает в себе применение криптографически нестойких алгоритмов защиты, которые могут быть легко взломаны при использовании специализированных программных средств.

Также автоматическая настройка CD–привода является возможной угрозой данной категории в связи с тем, что в моменты, когда включена функция автоматического определения диска, файл с названием «AUTORUN.INF» выполняется автоматически, не требуя разрешения и подтверждения у пользователя, вне зависимости от содержимого оптического диска.

1.3.1.5 Угрозы, которые возникают посредством заранее спланированных действий атакующей стороны

Данная категория включает в себе большее количество угроз, чем любая другая. Среди угроз встречаются те, которые могут быть осуществлены как физически, так и удаленно. Как пример, можно рассмотреть программное обеспечение «Adware», которое представлено в виде компьютерных червей, вирусов, троянских коней и пр. Более того, существуют угрозы, деятельность которых связана с анализом сетевого трафика с использованием специального программного обеспечения – «Sniffers». Осуществляя реализацию данной угрозы, атакующий имеет возможность получить несанкционированный доступ к конфиденциальной информации (электронная почта, пароли, персональные данные). Немаловажно, что в процессе выполнения анализа сетевого трафика злоумышленник имеет возможность получить информацию о внутренней структуре сети, узнав IP–адреса, используемые протоколы, порты, а также тип ПО, которое используется на компьютерах, подключенных к сети.

Так, злоумышленник получает возможность взять под свой контроль сетевое устройство, через которое маршрутизируется трафик, а, значит, помимо нарушения конфиденциальности информации, произойдет также нарушение её целостности.

Также, к рассматриваемой категории угроз относятся угрозы, связанные со спуфингом (Spoofing) такие, как ARP и DNS spoofing. Осуществление данных угроз предоставляет возможность злоумышленнику изменить маршрутизацию пакетов в локальных (ARP, DNS) и глобальных (DNS) сетях, таким образом злоумышленник получает возможность анализировать трафик, перенаправив его, например, свой на прокси-сервер. Одной из разновидностей подобных угроз является Web spoofing, которая состоит в том, что злоумышленник регистрирует доменное имя со схожим названием какого-либо web-сайта, причем большинство пользователей даже и не догадаются о подмене.

1.3.2 Классификация угроз по нарушаемым основным требованиям ИС

Ко всем видам информационных систем (ИС) определяется ряд требований по обеспечению основных параметров их функционирования. Среди всего прочего выделяются требования, описанные ниже.

1. *Обеспечение конфиденциальности* информационных ресурсов в информационной системе. Конфиденциальность информации заключается в том, что лицо, которое обладает доступом к некоторой специализированной информации, обязуется не предоставлять и не передавать эти данные третьему лицу, не получив согласия обладателя информационных ресурсов.

2. *Требование по обеспечению целостности* информационных ресурсов. Целостностью информации является свойство информационных ресурсов, особенностью которого является то, что данные не могут быть изменены нелегитимным пользователем информационной системы.

3. *Обеспечение доступности информации*. Под доступностью понимается свойство информационных ресурсов, состоящее в том, что легитимный пользователь системы имеет возможность использовать данные в определенных целях в рамках обозначенных правил пользования по требованию.

4. *Обеспечение аутентичности данных*. Под аутентификацией информационных ресурсов понимается процесс установления соответствия предъявляемого идентификатора субъекта ИС на его принадлежность данной системе.

5. *Обеспечение наблюдаемости ресурсов*. Наблюдаемость ресурсов заключается в том, что осуществляется процесс фиксирования деятельности пользователей ИС и выполняемых процессов с целью предотвращения нарушения политики безопасности и установления принадлежности выполняемых действий в системе.

1.3.2.1 Угрозы нарушения конфиденциальности данных

Реализация данной угрозы состоит в том, что информация, которая относится к разряду конфиденциальной, становится известной третьей стороне, которая не обладает правом на ознакомление с таковыми данными. В качестве примера можно привести следующие варианты реализации данной угрозы.

- *Анализ сетевого трафика и перехват.* Данный процесс осуществляется с использованием программного обеспечения, называемого сниффером.
- *Криптоанализ зашифрованной информации.*
- *Осуществление несанкционированного доступа* к информации, хранящейся в памяти устройств долговременной памяти.

1.3.2.2 Угрозы нарушения целостности данных

Данный тип угроз состоит в осуществлении несанкционированного удаления или изменения информационных ресурсов, обработка которых происходит в информационной системе. Среди примеров реализации подобных угроз:

- Изменение потока данных, маршрут следования которых проходит узел злоумышленника, а также с применением программного обеспечения.
- Внесение изменений в файловые ресурсы пользователей, которые располагаются в дистрибутивном дисковом пространстве.

1.3.2.3 Угрозы нарушения аутентичности информации

Угрозы, связанные с нарушением аутентичности, выражены в том, что злоумышленник выдает себя за легального пользователя информационной системы, что предоставляет ему возможность пользоваться правами и привилегиями легально. Ниже приведены примеры реализации подобной угрозы.

- На уровнях L3 и L4 (транспортном и сетевом) модели OSI злоумышленник может осуществить навязывание ложных сетевых адресов посредством технологии ARP-spoofing, а также навязывание доменных имен (DNS-spoofing).
- Довольно распространенная атака «Человек посередине (Man-in-the-middle)», которая состоит в том, что атакующая сторона каким-либо образом внедряет свои элементы инфраструктуры в канал связи между двумя легальными пользователями системы, таким образом просматривая данные, которые отправляются в этом канале связи и имея полный контроль над ними. Особенностью данного типа атаки является то, что на протяжении всего периода осуществления атаки злоумышленник является невидимым для легальной стороны.

1.3.2.4 Угрозы нарушения логирования и аудита

В данном случае угроза состоит в том, что злоумышленник изменяет принцип работы информационной системы таким образом, что не происходит логирования событий таких, как деятельность процессов и пользователей, установление индивидуальных идентификаторов процессов и пользователей. Среди примеров осуществления подобных атак приведены:

- Очистка или уничтожение журналов аудита.
- Выведение из строя, либо произвольное отключение работы системы логирования событий в информационной системе.
- Переполнение журнала аудита избыточными данными.

1.3.2.5 Угрозы нарушения доступности информации

Угроза нарушения доступности информационных ресурсов состоит в осуществлении неких действий, в результате чего происходит блокировка доступа для легитимных пользователей к определенному набору данных в информационной системе. Ниже приведены примеры подобных кибератак:

- Разрыв канала связи между некоторым количеством легитимных пользователей атакуемой информационной системы.
- Осуществление перегрузки ресурсов системы невалидными запросами со стороны злоумышленника. Результатом данной атаки является невозможность со стороны сервера обработать валидные запросы реальных пользователей. Атаки, которые реализуют подобную угрозу, носят название «Отказ в обслуживании» (Denial of service). В связи с тем, что цель данной работы состоит в разработке и реализации методики противодействия и обнаружения атаки типа HTTP flood, которая осуществляется на прикладном уровне модели OSI и является одной из наиболее распространенных атак в своем классе. В следующем разделе будут рассмотрены особенности реализации атак типа DDoS, и в особенности HTTP flood.

Выводы

Представленный раздел посвящен рассмотрению теоретических сведений, касающихся понятия информационной системы. Рассмотрены и описаны источники угроз информационной безопасности, введено понятие предмета информационной системы, стоит также отметить выделение модели нарушителя и модели угроз. Касательно второго понятия, была введена классификация по двум категориям: классификация угроз в соответствии с международным нормативным документов IT-Baseline Manual Protection, и классификация угроз по основным наиболее часто нарушаемым требованиям информационных систем, а также дана их подробная характеристика и рассмотрено описание.

2 ОСОБЕННОСТИ РЕАЛИЗАЦИИ АТАК ТИПА DDOS.

АТАКА HTTP FLOOD

Основной целью атак типа «Отказ в обслуживании» является нарушение доступности информационных ресурсов за счёт выведение объекта, подверженного атаке, из рабочего состояния, что, в свою очередь, делает ресурсы недоступными для легитимных пользователей информационной системы. Атака данного типа может иметь два источника появления, первый – за счёт использования уязвимостей в программном обеспечении информационной системы жертвы, второй – за счёт отправления настолько большого числа сетевых пакетов, что вызывает перегрузку сервера (например, HTTP flood).

Сравнивая оба метода, первый требует от злоумышленника обладать обширными знаниями и высокой квалификацией, а второй основан на использовании так называемой грубой силы. Рассмотрим второй вариант более подробно. Суть его состоит в том, что осуществляется большая загрузка вычислительных ресурсов сервера за счёт того, что с атакующей стороны отправляется большое число пакетов, которые должны быть обработаны сервером. Подобная высокая нагрузка на сервер приводит к тому, что сервер не имеет возможности обрабатывать запросы от валидных пользователей информационной системы, что в итоге может привести к значительному повышению обработки запросов на сервере, либо его выходу из строя.

Необходимо также принять во внимание, что среди подобного рода атак выделяют два типа аномалий: загрузка вычислительных ресурсов информационной системы, и загрузка пропускной способности канала связи. Предлагаемая в текущей работе методика ориентирована на защиту от атак второго типа, и далее будем рассматривать тот вариант, что вычислительных ресурсов сервера вполне достаточно, чтобы обработать все отправленные ему пакеты.

При реализации атак типа «Отказ в обслуживании» для атакующей стороны не имеют значения результаты обработки сервером отправляемых пакетов. Так, запросы злоумышленником могут быть отправлены с ложных и неактуальных юзер–агентов, а также с ложных IP–адресов, что, в свою очередь, выступает в качестве препятствия эффективному обнаружению и противодействию атакам.

Также для успешного осуществления DoS–атаки необходимо наличие высокой пропускной способности канала связи. В связи с этим фактором, данная атака производится одновременно с нескольких хостов. Подобная разновидность атаки, направленной на отказ в обслуживании, получила название DDoS.

Как уже упоминалось ранее, зачастую, для реализации распределенной атаки, злоумышленником используются рабочие машины, не являющиеся собственностью атакующей стороны, получившие название «Зомби». Среди способов инфицирования таких хостов наиболее распространенным является внедрение троянской программы в ПО компьютеров реальных пользователей сети. Далее, по команде злоумышленника, служащей началом атаки, внедренная программа трансформирует рабочую станцию, имеющую выход в глобальную сеть, в источник нелегитимных запросов, целью которых является перегрузка вычислительных ресурсов сервера информационной системы.

2.1 Классификация DDoS-атак

Среди наиболее распространенных атак типа «Отказ в обслуживании» выделяют следующие [19]:

- TCP SYN Flood;
- TCP Flood;
- Ping of Death;
- ICMP Flood;
- UDP Flood;
- HTTP Flood.

2.1.1 Атака на исчерпание системных ресурсов

Атака типа «Отказ в обслуживании», целью которой является исчерпание системных ресурсов хоста, на котором происходит обработка обращений, направлена на загрузенность системных ресурсов таких, как вычислительные мощности процессора, памяти и распределения дискового пространства.

В большинстве случаев, атакующая сторона в момент действия атаки уже имеет доступ к общим ресурсам системы, предпринимая попытку затруднить доступ к этим ресурсам сервиса легальных пользователей за счёт своих действий.

Таким образом, за счёт выполнения данных действий имеется возможность осуществления недоступности сервера, зависания выполняемых процессов, а также переполнения пространства диска для подключения реальных пользователей ИС.

Среди средств осуществления подобного рода атак выделяется программное средство PortFuck, главное диалоговое окно которого изображено на рисунке 2.1, при помощи которого совершается атака переполнением таблицы процессов. Данный программный модуль своими действиями открывает новые TCP-соединения на стороне атакуемого хоста до тех пор, пока ресурсы жертвы не окажутся переполненными.

Вне зависимости от мощности процессора, объема памяти и полосы пропускания линий связи переполнение однозначно наступит посредством осуществления описанной атаки в связи с тем, что для каждого отдельного TCP – соединения требуется некоторый определенный объем ресурсов для его открытия.

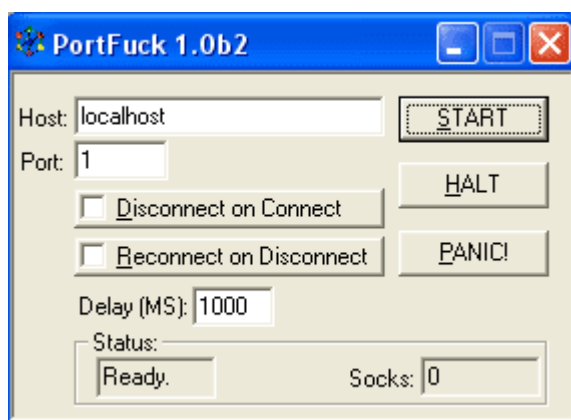


Рисунок 2.1 – Диалоговое окно утилиты PortFuck

В результате выполнения данной программы на примере нагрузка процессора возросла до 100%, после чего проводник Windows перестал отвечать на запросы легитимного пользователя (рисунок 2.2).

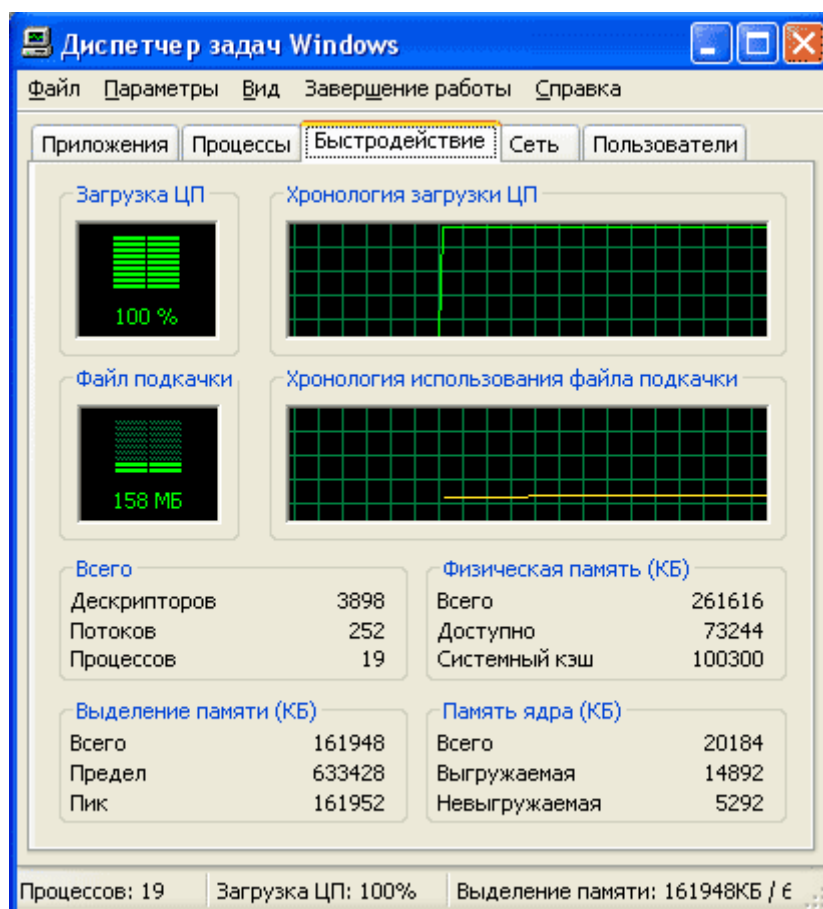


Рисунок 2.2 – Нагрузка процессора

Также на рисунке 2.3 показан результат тестового подключения рассматриваемой утилиты к 80 (HTTP) порту компьютера, полученный с использованием утилиты Attacker 3.0. В рассмотренном примере обработка запросов, поступающих на 80 порт хоста, осуществляться не будет, в связи с переполнениями подключений по указанному порту.

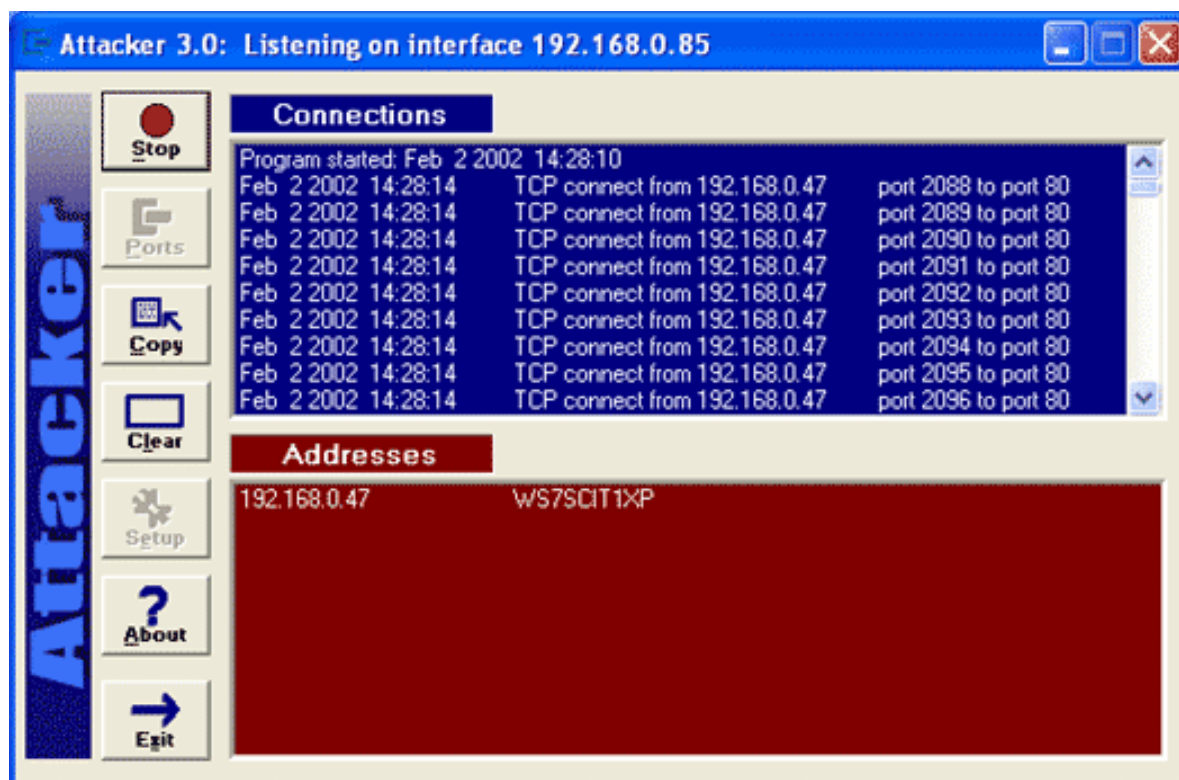


Рисунок 2.3 – Переполнение 80 порта подключениями

2.1.2 Атака на достижение недостатка ресурсов

Данная атака осуществляется с целью обеспечения загруженности системных ресурсов обрабатывающего хоста, путем повышенной нагрузки на оперативную память, процессор, а также физическую память дисков.

2.1.2.1 Отправка «тяжёлых» пакетов

В случае осуществления подобной атаки злоумышленник отправляет на сервер пакеты, которые при широкой полосе пропускания не насыщают ее, но в то же время достаточно в большом объеме происходит нагрузка на процессорное время. Таким образом, процессор не в состоянии обработать полученные пакеты, в связи с чем будет наблюдаться отказ в системе, что в результате приведет к недоступности ресурсов для легальных пользователей информационной системы.

2.1.2.2 Переполнение сервера файлами логирования

Файлы, в которые производится запись действий программного обеспечения, а также пользователей системы, называются *лог-файлами*. Обычно при создании подобных файлов на сервере устанавливается определенный лимит на количество записей, либо объем, занимаемый этим файлом на диске. В случае, когда указанное ограничение не установлено, атакующей стороной отправляются большие по объему пакеты, занимающие большое пространство на диске, что в последствие вызовет недоступность сервера в связи с отсутствием свободной памяти.

2.1.2.3 Недостаточная проверка данных пользователя

Подобная ошибка в проектировании системы защиты в большинстве случаев приводит к бесконечному (или сверхдлительному) времени выполнения цикла, что также может проявляться в увеличенном времени потребления процессорных ресурсов, в некоторых случаях приводящее к исчерпанию предоставляемой памяти.

2.2 Атака типа HTTP Flood

Атака HTTP Flood, схема которой изображена на рисунке 2.4, является волюметрической распределенной атакой, применяемой в целях обеспечения чрезмерной нагрузки на целевой сервер HTTP-запросами. После того, как целевой сервер является перегруженным указанными запросами, он не в состоянии отправлять ответы на трафик, полученный от реальных пользователей, что, в свою очередь, вызывает отказ в обслуживании сервером [20].

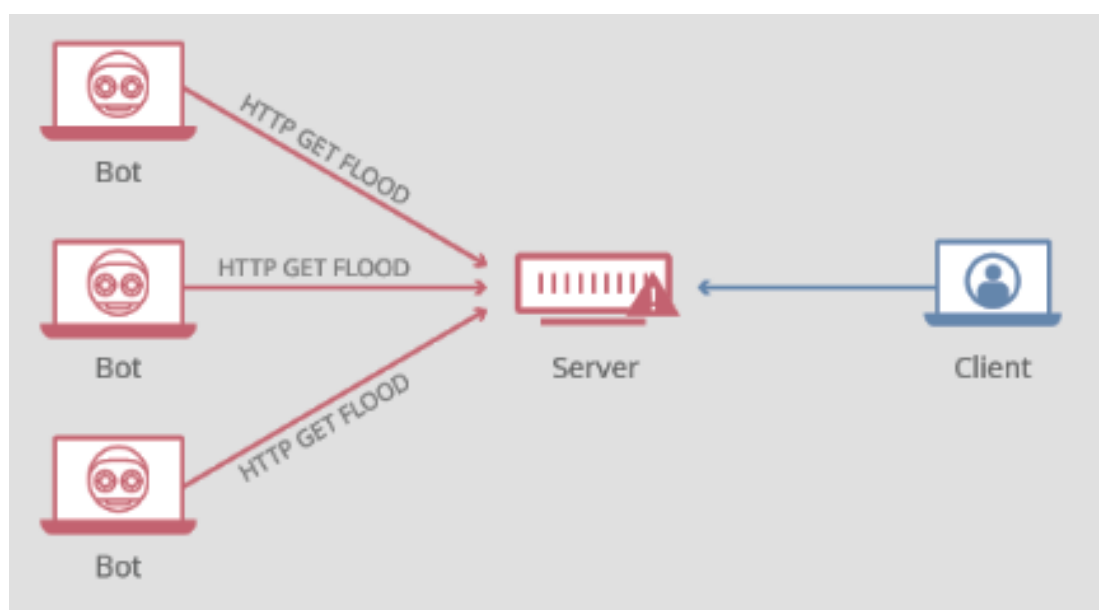


Рисунок 2.4 – Схема атаки типа HTTP Flood

Атака HTTP Flood осуществляется на уровне 7 модели OSI, который является уровнем приложений и на котором осуществляют работу протоколы HTTP, HTTPS, FTP и другие. Запросы HTTP являются основными запросами, применяемыми в браузерах повсеместно, а также наиболее используемыми для загрузки Web-страниц и отправки контента в глобальной сети Интернет. Отражение подобного рода атак представляет собой непростую задачу в связи с тем, что вредоносный трафик достаточно трудно отличить от нормального.

С целью добиться наиболее эффективного результата своей атаки, злоумышленники зачастую создают сети ботнет для максимизации влияния их атак. Заразив множество устройств вредоносным программным обеспечением, атакующая сторона получает возможность применить созданную сеть, организовав атаку с большим объемом трафика.

Среди HTTP Flood разновидностей атак выделяются два типа:

1. *Атака HTTP GET* – в случае осуществления данной атаки множество компьютеров и других устройств, подключенных к сети Интернет, направляются на некий объект (Web-сайт) с целью отправки огромного числа запросов на загрузку изображений, файлов, а также других ресурсов, расположенных на целевом сервере. В итоге целевой сервер оказывается переполненным количеством поступающих запросов и ответов, что вызывает отказ в обслуживании для легитимных пользователей системы.

2. *Атака HTTP POST* – при отправке формы на Web-сайте сервер должен обработать входящий запрос и направить необходимый запрос в базу данных. Процесс обработки данных формы и выполнения необходимых команд базы данных является относительно интенсивным по сравнению с объемом вычислительной мощности и пропускной способности, необходимой для отправки запроса POST. Эта атака использует неравенство в относительном потреблении ресурсов, отправляя много запросов POST непосредственно на целевой сервер, пока его емкость не будет насыщена и не произойдет отказ в обслуживании.

2.3 Атака ICMP Flood (Ping Flood)

В случае осуществления атаки подобного типа на целевой сервер отправляется большое число невалидных ICMP-пакетов (от Internet Control Message Protocol, протокол межсетевых управляющих сообщений, входящий в стек протоколов TCP/IP) с достаточно большого диапазона IP-адресов. Целью атаки является насыщение канала передачи данных и обеспечение высокой нагрузки атакуемого сервера поддельным трафиком, состоящим из запросов.

В связи с тем, что ICMP–пакеты отправляются без подтверждения о получении, то невалидный трафик достаточно тяжело выявить, что имеет аналогию с UDP трафиком. Также атака ICMP Flood может быть осуществлена ради получения необходимой информации о сервере (сканирование портов и адреса получателя) с целью совершения целенаправленной атаки на конкретные порты сервера.

2.4 Атака типа UDP Flood

При осуществлении атаки типа UDP (User Datagram Protocol) Flood на атакуемый сервер отправляется большое число UDP–пакетов из большого диапазона IP–адресов, в связи с чем сервер оказывается перенасыщенным ложными UDP–пакетами. К тому же, вследствие подобной атаки происходит перегрузка сетевых интерфейсов за счёт насыщения полосы пропускания. Благодаря тому, что в протоколе UDP отсутствует необходимость установления соединения, в сравнении с TCP, фильтрация UDP Flood представляет собой весьма непростую задачу с точки зрения валидации легитимного трафика. При активизации подобной атаки сеть поражается пакетами со случайными или статическими IP–адресами, что может привести сервер в состояние недоступности за счёт обладания информацией о доступности конкретного порта сервера, а также его IP–адреса [21].

2.5 Атака типа TCP SYN Flood

В первую очередь, клиентской стороной создается SYN–пакет (от слова synchronization, синхронизация) для установления новой сессии с сервером. TCP сессия уже является открытой, так как выполнен процесс «трехэтапного рукопожатия TCP», сервер производит мониторинг и обработку всех открытых пользовательских сессий. При осуществлении атаки SYN Flood сервер, на который производится атака, получает огромное количество нелегитимных SYN–запросов, в которых содержится ложный IP–адрес источника запроса. В данном случае происходит поражение сервера, за счёт которого вся память таблицы соединений оказывается заполненной, в результате чего происходит отказ в работе хоста за счёт значительного уменьшения производительности сервера.

2.6 Принципы выявления атак типа DDoS

Среди некоторых специалистов имеется мнение, что для выявления атак типа «Отказ в обслуживании» не требуется специальных средств, поскольку факт осуществления подобной атаки на сервис является весьма очевидным.

При некоторых факторах данное утверждение имеет право на существование. Но в то же время за всю историю существования подобного рода атак было отмечено большое количество случаев, когда существования атаки было замечено спустя несколько суток после её начала, что в качестве последствий может иметь превышение расходов на оплату интернет трафика, которое также может быть обнаружено лишь в момент оплаты счёта за использование сети провайдера. Более того, большинство из методов защиты являются неэффективными при установлении их на целевом объекте атаки, но в то же время обладают большей эффективностью при обнаружении на сетевых магистральных каналах.

Так, наиболее целесообразным способом установления защиты является установление системы защиты на промежуточном уровне между клиентом и сервером с целью эффективного отражения атак. Также для эффективного отражения необходимо обладать квалифицированными знаниями о типе, характере и других отличительных характеристиках различных атак типа «Отказ в обслуживании», что позволяет наиболее быстро с меньшими трудозатратами обеспечивать безопасность информационных систем. Службы и организации, занимающиеся обеспечением безопасности от таких атак, имеют возможность определять указанные характеристики трафика, а также производить необходимые настройки системы целевого сервера для минимизации ущерба.

Таким образом, у них имеется возможность установить приведенные условия, но что конкретно является причиной отказа в обслуживании – не представляется возможным в связи с тем, что данный факт может произойти вследствие нештатного события на целевом сервере.

Согласно правилам политики обеспечения безопасности, при установлении факта осуществления атаки типа «Отказ в обслуживании» требуется ее незамедлительная регистрация для произведения дальнейшего анализа. Затем при некоторых определенных условиях (продолжающаяся атака, либо необходимость внесения корректировок в правила фильтрации трафика) может потребоваться использование службы обеспечения безопасности для восстановления нормального функционирования сервиса, а также легитимизации поступающего трафика.

При обнаружении атак также в некоторых случаях является целесообразным использование функционала редиректа трафика по другим каналам связи, а также использование запасных серверов для сохранения копий. Стоит отметить, что обеспечение защиты информационных систем от атак рассматриваемого типа является нетривиальной задачей, для осуществления которой могут применяться различные методы защиты в зависимости от вида информации, подлежащей защите.

Так, методы обнаружения DDoS–атак разделяют условно на три группы:

- *сигнатурные*, основанные на анализе качества входящего трафика;
- *статистические*, на основе количественного анализа входящего трафика;
- *гибридные*, состоящие из комбинации указанных выше методов.

2.7 Наиболее известные DDoS–атаки

В 2012 году были осуществлены неоднократные попытки совершения крупномасштабных DDoS–атак на корневые серверы DNS. Планирование первой атаки было отмечено на 31 марта 2012 года, но в последствии данной атаки так и не было совершено. Основной задачей хакеров из неизвестной группировки Anonymous являлось произвести отказ всей глобальной сети Интернет, что заключалось в осуществлении кибератак на 13 корневых DNS серверов. Для совершения этой кибератаки была разработана утилита Ramp, работа которой заключалась в объединении менее значимых DNS–серверов и Интернет–провайдеров, за счёт чего и планировалось осуществление такой атаки [22].

В ноябре 2002 года была осуществлена атака, подобная рассмотренной выше, и которая до сих пор считается наиболее масштабной DDoS–атакой на корневые серверы DNS, благодаря чему атакующей стороне удалось вывести из строя 7 корневых серверов DNS. После этого, в августе того же года была совершена атака на компанию AT&T, которая является одной из крупнейших мировых телекоммуникационных компаний. Вследствие атаки, продолжавшейся длительностью 8 часов, были выведены из строя серверы DNS, находившиеся в инфраструктуре компании, после чего пользователи не имели возможности открыть официальный сайт компании, но также наблюдалась недоступность сайтов коммерческих организаций, входящих в инфраструктуру.

10 ноября 2012 г. пострадал крупнейший мировой провайдер хостинг услуг Go Daddy. В результате атаки, которая была успешно осуществлена, ущерб понесли как официальный сайт компании, так и более 33 миллионов доменных имен, которые были зарегистрированы с участием данной организации.

Несколько ранее злоумышленники, разработав вирус под названием «Mydoom» произвели атаку, вследствие которой была обеспечена недоступность сайта компании SCO, основным направлением деятельности которой является разработка программного обеспечения, что произошло 22 августа 2003 года. Стоит отметить, что сайт компании оставался недоступным в течение 3 суток.

Среди кибератак, осуществление которых происходит ежедневно, как на крупные, так и на мелкие сервисы, выделяется кибератака, осуществленная 15 сентября 2012 г., которая производилась на «мастодонта» услуг по DDoS–защите компанию CloudFlare, также обладающей инфраструктурой доставки контента CDN, чьи серверы расположены по всему земному шару. Указанная атака осуществлялась с мощностью 65 Гбит/с. Несмотря на то, что ранее компании удавалось отражать кибератаки мощностью несколько десятков Гбит/с, данную атаку, пик которой наблюдался в 15 сентября 2012 г. в 13:00 (как показано на рисунке 2.5), предотвратить на тот момент не представлялось возможным. Впоследствии, благодаря проведенному анализу осуществленной атаки, было выявлено, что для её проведения было необходимо как минимум 65 тысяч ботов, генерирующих трафик 1 Мбит/с. Но, в то же время, Интернет–провайдерами будет достаточно легко обнаружена подобная аномалия траффика, что в последствии приведет к блокировке Web–сайта. К тому же, создание столь большой ботнет–сети является для злоумышленников весьма недешевой задачей. После этого было окончательно установлено, что для совершенной кибератаки был применен метод приумножения DNS–запросов через открытые серверы DNS.

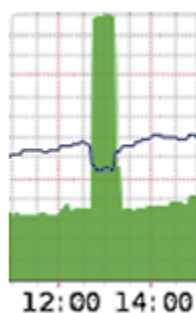


Рисунок 2.5 – Атака на CloudFlare 15 сентября 2012 года

18 марта того же года, согласно источнику, опубликованному в газете «The New York Times», была осуществлена одна из наиболее мощных атак типа «Отказ в обслуживании», которая была направлена на сервисы компании Spamhaus, основным направлением деятельности которой является блокирование, с дальнейшим занесением в черный список, IP–адресов и доменных имен проектов, рассылающих спам в большом количестве. Основным поводом послужило занесение данной организацией в черный список провайдера хостинг–услуг из Нидерландов «Cyberbunker», чье отмщение было выражено в кибератаке с пиковой мощностью до 300 Гбит/с, с использованием открытых DNS серверов. На 19 марта того же года наблюдалась атака, которая варьировалась мощностью от 30 Гбит/с до 90 Гбит/с, затем затихла на пару дней, и 22 марта 2012 года DDoS–атака была возобновлена с новой силой, мощность которой возросла до 120 Гбит/с, что показано на рисунке 2.6.

Защита от атаки осуществлялась компанией CloudFlare, которой было применено распределение трафика между дата-центрами, входящими в инфраструктуру организации, что дало понять Cyberbunker, что подобная атака не закончится успехом, после чего атака сменила вектор на вышестоящие пиры. Часть трафика блокировалось на уровне Tier2, оставшийся трафик поступал на Tier1, на котором мощность достигала до 300 Гбит/с. В тот момент, когда происходил пик атаки, интернет пользователи по всему миру не могли загрузить Web-сайты, находящиеся под защитой CloudFlare. В результате чего провайдерам по защите от DDoS-атак удалось выдержать столь мощную атаку, но в то же время на европейский направлениях наблюдалось увеличение времени отклика при доступе к Web-проектам. К примеру, в центре передачи и обмена трафика в Лондоне LINX 23 марта скорость упала более, чем в 2 раза, как показано на рисунке 2.6:



Рисунок 2.6 – Падение трафика в центре LINX

2.8 Известные методы защиты от DDoS-атак

Организация защиты от распределенных атак типа «Отказ в обслуживании» представляет собой достаточно сложное мероприятие. В первую очередь, необходимо упомянуть, что довольно непросто корректно определить источник угрозы, к тому же, владельцы хостов зачастую не подозревают, что их ресурс подвергается атаке в какой-то определенный момент времени и, более того, пользователи, с устройств которых осуществляется атака, совершенно не знают об этом факте. Также процесс, в ходе которого происходит идентификация паразитного трафика, сам по себе весьма трудоемкий. С точки зрения обеспечения безопасности атаки «Отказ в обслуживании» представляют собой наиболее опасную угрозу в сети, в связи с чем принятие необходимых эффективных мер по их противодействию является трудной задачей для организаций, предоставляющих услуги защиты от подобного рода атак. Таким образом, наиболее эффективный способ обнаружения подобных атак – осуществлять анализ аномальных поведений сетевого трафика.

В связи с тем, что вредоносный трафик имеет большие сходства с легитимным трафиком, его сложно отличить от реального трафика от легальных пользователей информационной системы. Благодаря чему следующие компоненты являются важными для организации стратегии обеспечения сетевой безопасности: межсетевые экраны, системы обнаружения вторжений, маршрутизация в «черные дыры», и другие. Также необходимо заметить, что использование указанных технологий не гарантирует обеспечение полной защиты от DDoS-атак.

2.8.1 Маршрутизация трафика в «черные дыры»

Данная технология применяется провайдером в целях блокировки всего трафика, который направляется на целевой сервер в наиболее раннем узле прохождения трафика, что обеспечивается тем, что трафик перенаправляется в «черную дыру» для того, чтобы обеспечить защиту сети провайдера и клиентов, использующих его услуги. Очевидно, данный метод не является удобным, так как наряду с вредоносным трафиком все запросы реальных пользователей системы также маршрутизируются, следовательно, пользователи не могут получить доступ к ресурсу, а клиент не получает легитимный трафик, соответственно, атакующая сторона в некоторой мере достигает своей цели.

2.8.2 Использование списков контроля доступа

Маршрутизирующие устройства, в которых реализован механизм списков контроля доступа для фильтрации вредоносного трафика, могут применяться в целях защиты от атак типа «Отказ в обслуживании».

Данная технология позволяет обеспечивать защиту от ICMP-атак, а также осуществлять фильтрацию неиспользуемых протоколов.

На сегодняшний день при осуществлении большинства атак используются актуальные протоколы, которые применяются для функционирования глобальной сети Интернет, следовательно, второй пункт не является актуальным для корректного обеспечения защиты. Также маршрутизаторы имеют возможность осуществлять блокировку зон с невалидными IP-адресами, в связи с чем злоумышленники подделывают корректные IP-адреса источников атак.

Таким образом, списки ACL можно назвать первой линией обороны от DDoS-атак, но данная технология не является актуальной и эффективной для обеспечения защиты от следующих видов атак:

- атаки лавинного типа: SYN, SYN–ACK и FIN. Использование данной технологии не позволяет осуществлять блокировку атаки SYN с произвольным набором объектов спуфинга, а также атаки ACK и RST на 80 (HTTP) порт целевого Web–сервер, так как в связи с тем, что некорректные IP–адреса источников постоянно изменяются, для блокировки следовало бы проследить и идентифицировать каждый отдельный некорректный источник атаки, что является неразрешимой задачей на сегодняшний день. Таким образом, единственным вариантом блокировки подобной атаки является блокировка всех поступающих подключений на сервер, что, в свою очередь, и является целью злоумышленника;
- использование технологии прокси; предпринимая попытки отражения направленной атаки со спуфингом, списки ACL блокируют весь трафик пользователей атакуемой системы, который поступает от одного IP или ограниченного диапазона адресов прокси, что происходит в связи с тем, что списки ACL не в состоянии произвести отличие вредоносных SYN от реальных;
- BGP протокол; в случае осуществления атаки с неопределенным произвольным набором объектов спуфинга на маршрутизатор BGP ACL списки не в состоянии произвести анализ трафика, который, благодаря лавинообразному эффекту, меняется довольно быстро, что также обуславливается тем, что рассматриваемая технология не производит различия некорректных адресов от реальных;
- Атаки уровня 7 модели OSI–уровня приложений; для того, чтобы блокировать клиентские соединения при помощи списков ACL необходимо производить настройку конфигурации огромного числа списков для каждой потенциальной атакуемой системы. Защита такого рода неактуальна.

2.8.3 Применение межсетевых экранов

Несмотря на то, что межсетевые экраны играют важнейшую роль в проектировании защиты каждой организации, их основным функционалом не является отражение атак типа «Отказ в обслуживании». Использование данной технологии не способно обеспечить всеобъемлющую защиту от атак типа «Отказ в обслуживании», так как межсетевые экраны обладают целым набором исходных параметров, среди которых в данном случае выделяется невозможность выявления аномалии трафика. Основной задачей межсетевых экранов является контроль доступа в частные сети и к хостам. Среди способов решения данной задачи выделяется отслеживание сеансов, инициированных изнутри, которые адресованы на внешний сервис с последующим приемом только особых откликов от ожидаемых источников на внешней стороне.

Стоит отметить, что подобная схема не носит применения к сервисам DNS, Web и подобным, так как их технология подразумевает открытость для общего доступа с целью обеспечения возможности внешних подключений. И в этом случае межсетевые экраны открывают канал связи, пропуская HTTP трафик к конкретному IP-адресу, программным образом настроенному на сервере. Данный способ не является эффективным в целях обеспечения защиты от DDoS-атак, так как злоумышленник может с легкостью воспользоваться открытым портом для атаки с разрешенного на подключение протокола. Таким образом, межсетевые экраны не обладают возможностью выявления аномалий трафика, анализируя его с открытых протоколов. Более того, в технологии межсетевых экранов отсутствует возможность осуществлять борьбу со IP-спуфингом, так как при активной атаке они имеют возможность осуществлять блокировку конкретного потока трафика, но не реализованы методы отделения нелегитимного трафика от легитимного.

2.9 Реакция на атаки «Отказ в обслуживании», вызванные вручную, и другие

В момент обнаружения атаки жертвой, в первую очередь, следует обращение в поддержку провайдера услуг (провайдер Интернет-услуг, хостинга или магистральный) с целью идентификации источника атаки. В случае применения злоумышленником технологии IP-спуффинга процесс отражения атаки может занять достаточно большой промежуток времени, так как необходимо будет обращаться к вышестоящему провайдеру услуг. В итоге, с некоторой вероятностью источник атаки все же будет идентифицирован, и его блокировка повлечет за собой блокировку всего входящего трафика, в том числе и легитимного.

Также специалистами защите информации в целях обеспечения безопасности от атак типа «Отказ в обслуживании» могут быть применены различные стратегии для достижения цели, а именно – использование резервных ресурсов, что выражается в организации резервной полосы пропускания или резервных сетевых устройств, при помощи которых в большинстве случаев будет возможным обеспечивать бесперебойную работу информационных систем при наличии атак. Нужно отметить, что данный метод не является рентабельным, в связи с тем, подразумевает собой введение в эксплуатацию дополнительных сетевых устройств с соответствующими интерфейсами, что не гарантирует обеспечения защищенности от DDoS-атак в случае увеличения её масштабов.

2.10 Анализ рынка

На сегодняшний день на мировом рынке представлены два основных производителя систем, обеспечивающих отражение атак типа «Отказ в обслуживании», Cisco Systems и Arbor Networks, а также некоторый набор программных продуктов, применение которых ориентировано для защиты от менее крупных DDoS-атак и более узконаправленных случаев. Стоимость обеспечения защищенности информационной системы от DDoS-атак может варьироваться в достаточно большом диапазоне – от сотен до десятков тысяч долларов ежемесячно, в зависимости от популярности Web-проекта и мощности кибератак, совершаемых на него.

2.10.1 Коммерческие решения, представленные на рынке

Производители аппаратного и программного обеспечения, обеспечивающего защиту от атак типа DDoS, в большинстве случаев не оказывают услуги по защите от атак, а являются вендорами, продающими свою продукцию. Рассмотрим более подробно наиболее эффективные коммерческие предложения сегодняшнего рынка.

2.10.2 Предложение по защите от компании Cisco

Компанией Cisco предлагается технология Clean PIPes, которая заключается в применении модулей Cisco Anomaly Detector и Cisco Guard и различных систем статистического анализа поступающего сетевого трафика, которые основаны на данных, поступающих с маршрутизаторов по протоколу Cisco Netflow. В данном случае модуль Anomaly Detector, а также системы статистического анализа трафика являются своего рода системой обнаружения атак типа DDoS, а модуль Cisco Guard выступает в качестве средства противодействия обнаруженной атаке. Рассматриваемое решение, предлагаемое компанией Cisco, предоставляет возможность не только осуществить блокировку атакующих IP-адресов, а разделить нелегитимный трафик от реального. К тому же, реализованы три основные функции, выполнение которых обеспечивает защиту от атак:

Обнаружение – осуществляется обучение эталонным параметрам трафика, на основании которых происходит определение аномалии в его структуре за счёт сравнения с контрольными характеристиками, что свидетельствует о начале атаки. Таким образом, какое-либо отклонение характеристик трафика от эталонных значений вызывает срабатывание сигнала тревоги.

Подавление – процесс очистки трафика, заключающийся в идентификации аномалий, обнаружении IP–спуфинга, а также блокировке нелегитимного трафика.

Изменение маршрута прохождения трафика и его возврат – заключается в том, что «грязный» трафик перенаправляется на фильтрующее устройство Cisco Guard, после чего он возвращается обратно в сеть, как изображено на рисунке 2.7.

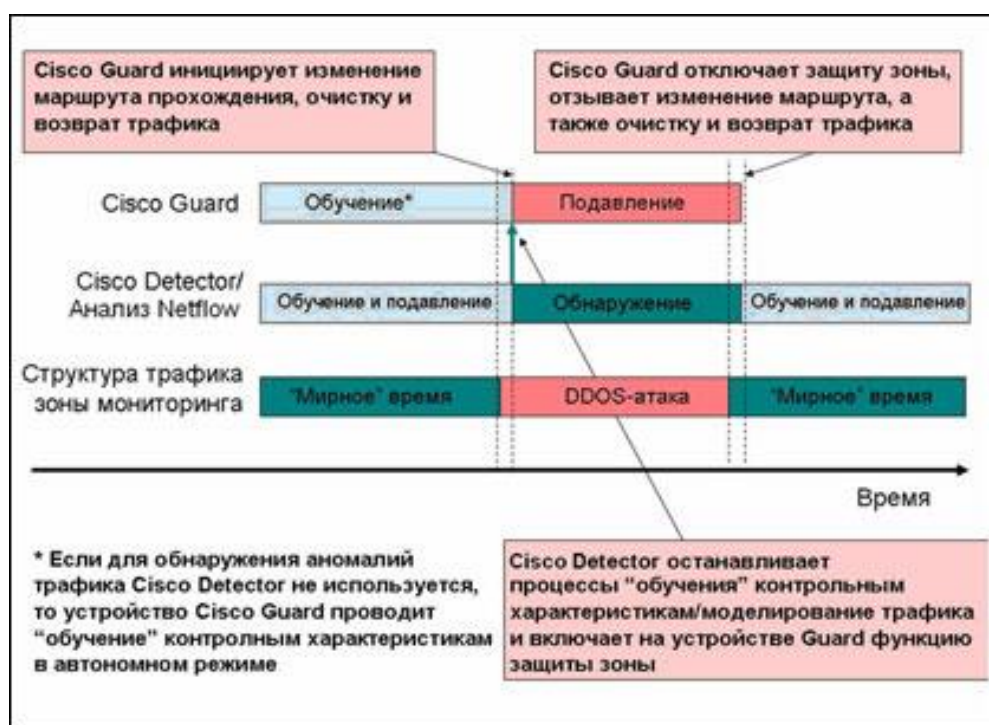


Рисунок 2.7 – Принцип работы технологии Clean IPes

2.10.2.1 Система защиты Web-сервера от компании ADOS

Основным предлагаемым решением является демон `ados_daemon`, которому предоставляется доступ к TCP пакетам, которые посылаются HTTP–клиентами до обработки TCP/IP стеком в ядре, и который принимает решение о дальнейшем пропуске пакета, либо его блокировке.

Данное решение принимается, исходя из результатом поиска IP–адреса источника пакета в формируемых в ходе работы демона списках IP–адресов. Так, поддерживается три вида списка: `Ban_list` – черный список – при наличии IP–адреса в этом списке осуществляется его блокировка; `White_list` – белый список – вне зависимости от содержимого пакета с IP–адресом источника, находящегося в этом списке, блокировка осуществлена не будет; `Track_list` – треклист – с указанных в данном списке IP–адресов были осуществлены запросы, но не до текущего момента времени не было дано решение о блокировке или пропуске.

Также согласно настройкам конфигурации ADOS любому типу запросов присваивается определенный вес, который назначается, исходя из нагрузки, создаваемой на Web-сервер.

При попадании запросов на сервер информация об IP-адресе источника и количестве запросов каждого класса помещается в трек-лист и хранится там в течение определенного промежутка времени *track_period*. С каждым запросом с конкретного IP-адреса увеличивается счётчик запросов, по окончании выводится информация о средней частоте запросов, которая рассчитывается по формуле 1.

$$rate = \frac{C_1 * W_1 + C_2 * W_2 + \dots + C_n * W_n}{track_period} \quad (2.1)$$

где C_i – количество запросов одного класса, W_i – вес этого класса.

Затем происходит сравнение вычисленного показателя с порогом срабатывания *flt_rate_threshold*, при превышении которого IP-адрес источника запроса помещается в список блокировок при условии, что он не находится в белом списке IP-адресов.

Более того, после помещения IP-адреса в список блокировок активируется процесс обратного резолвинга, который заключается в том, что, если IP-адрес принадлежит одному из «надёжных» доменов, и не удастся определить имя его хоста, то данный IP-адрес попадает в белый список адресов.

2.10.3 Некоммерческие Unix-продукты защиты от HTTP Flood

На сегодняшний день существует два способа относительно обеспечивающие защиту от DDoS-атак которые реализованы встроенными средствами Web-сервера, а также посредством внедрения внешних модулей.

2.10.3.1 Стандартный способ идентификации атаки

При наличии атаки на первоначальном этапе сетью ботнет происходит постепенное увеличение количества посылаемого трафика, таким образом, необходимо в максимально короткое время обеспечить защиту.

В первую очередь, следует произвести подсчёт количества процессов Web-сервера Apache, а также количество подключений на 80 (HTTP) порт следующими Shell-командами:

```
# ps aux | grep HTTPd | wc-l  
# netstat -na | grep ":80\ " | wc-l
```

Если значения превышают среднестатистические, то это с высокой вероятностью свидетельствует о наличии атаки.

Затем необходимо проанализировать список IP-адресов, с которых осуществляются запросы на подключение:

```
# netstat-na | grep ":80\ " | sort | uniq-c | sort-nr | less
```

Таким образом, невозможно однозначно утверждать о наличии DDoS-атаки, но в случае, если с одного и того же IP-адреса источника осуществляется большое количество запросов, ситуация становится все более очевидной, за исключением, что большое количество реальных пользователей используют NAT.

Затем необходимо произвести анализ пакетов при помощи tcpdump:

```
# tcpdump -n -i eth0 -w attack.log dst port 80 and host IP_сервера
```

С помощью указанной выше команды запросы, отправляемые на 80 порт Web-сервера, будут записаны в файл attack.log. В качестве показателя наличия атаки будет являться большое количество однообразных запросов («тяжелый» скрипт или корень сайта) с разных IP-адресов.

Далее, при выполнении следующей команды будет выведено 2 столбца, в первом из них количество запросов, а во втором IP-адрес источника запросов. Соответственно, при большом количестве запросов с одного IP-адреса вероятность принадлежности этого IP боту резко возрастает.

2.10.3.2 Фильтрация запросов по URL при помощи IPTABLES

В рассматриваемой технологии реализовано расширение string, использование которого предоставляет возможность осуществлять фильтрацию поступающих запросов, исходя из анализа их содержимого.

В случае осуществления атакующих запросов на конкретную страницу сайта (например, главная страница «GET / HTTP/1.1»), то имеется возможность отклонить все запросы, поступающие на нее следующей командой:

```
#iptables -I INPUT -p tcp -dport 80 -m string --string "GET / HTTP/1.1" --algo kmp -j DROP
```

Таким образом, Web-страница временно оказывается недоступной для подключений, и, тем не менее, Web-сайт будет защищен от кибератаки.

2.10.3.3 Фильтрация трафика по географическому принципу

В случае, если сайт ориентирован на определенную группу людей, проживающих на одной территории, либо разговаривающих на одном языке, то средствами iptables с применением расширения geoip имеется возможность осуществить блокировку всех запросов, поступающих, например, из других стран:

```
#iptables -A INPUT --dport 25 -m geoIP --src-cc RU -j ACCEPT
#iptables -A INPUT --dport 25 -j LOG --log-prefix "Blocked smtp from not RU:"
#iptables -A INPUT --dport 25 -j REJECT
```

2.10.3.4 Встроенный модуль Web-сервера Apache mod_evasive

В некоторых случаях для защиты от DDoS-атак целесообразно использовать модуль Web-сервера Apache mod_evasive, который осуществляет блокировку по тайм-ауту при превышении определенного числа запросов к домену с последующим ответом 403 Forbidden, что предоставляет возможность уменьшить нагрузку на целевой сервер в некоторых случаях при небольших атаках. Но, в то же время, при наличии более существенных атак подобный способ не является эффективным, потому что даже в случае блокировки необходимо сгенерировать ответ «403 Forbidden» на сервере на каждый полученный запрос, что требует затраты ресурсов. Модуль mod_evasive также может быть вызван внешним скриптом, в связи с чем имеется возможность осуществить блокировку атакующего IP-адреса на уровне межсетевого экрана.

2.10.3.5 Обеспечение защиты встроенными средствами Web-сервера Nginx

«Ахиллесовой пятой» атакующих ботов является их неумение читать Cookie-файлы, которое заключается в том, что наличие факта передачи Cookie-файла Web-серверу от пользователя позволяет идентифицировать нелегитимный трафик.

В первую очередь необходимо скрыть целевой Web-сервер путём, к примеру, указания запросов на 8080 порт, а на место Web-сервера настроить более быстрый Web-сервер Nginx, затем произвести настройку страницы-заглушки. В качестве примера актуален стандартный конфигурационный файл Web-сервера Nginx, и нужно добавить «cookie», дописав в конфигурационном файле следующую строку:

```
add_header Set-Cookie "type=this-is-not-bot".
```

Также в момент открытия страницы написать предупреждение: «Приносим извинения за предоставленные неудобства. Для перехода на сайт необходимо включить Cookie и перейти по ссылке».

После того, как пользователь Web-сайта получает указанное выше предупреждение, то следующим запросом он передает Cookie на Web-сервер:

```
#iptables -t nat -A PREROUTING -p tcp --dport 80 -m string --string "this-is-not-bot" --algo kmp -j REDIRECT --to-ports 8080
```

2.10.3.6 Ограничение количества запросов с одного IP-адреса

На Web-сервере Nginx была осуществлена реализация модуля ngx_http_limit_zone_module, при помощи которого владельцу ресурса предоставляется возможность ограничить количество отправляемых запросов с одного IP-адреса.

Таким образом, владелец сайта, к примеру, предполагает, что легитимный пользователь осуществляет не более двух запросов в единицу времени к одной странице его ресурса, что позволяет блокировать всех остальных пользователей информационной системы посредством межсетевого экрана, количество запросов от которых превышает данный эталонный показатель.

Для настройки рассматриваемой функции необходимо в конфигурационный файл Web-сервера Nginx добавить следующий скрипт:

```
error_log /var/log/nginx/error.log;  
<...>  
location =/ {  
    limit_conn one 3;  
    root /var/www/domain.ru;  
}
```

Таким образом, IP-адреса, с которых было отправлено более граничного количества допустимых запросов в единицу времени, попадают в error.log с комментарием «limiting connections by zone», как показано на рисунке 2.8. Далее на основе анализа файла error.log с хранящимися в нем IP-адресами имеется возможность осуществлять работу с черными списками целевого Web-сервера.

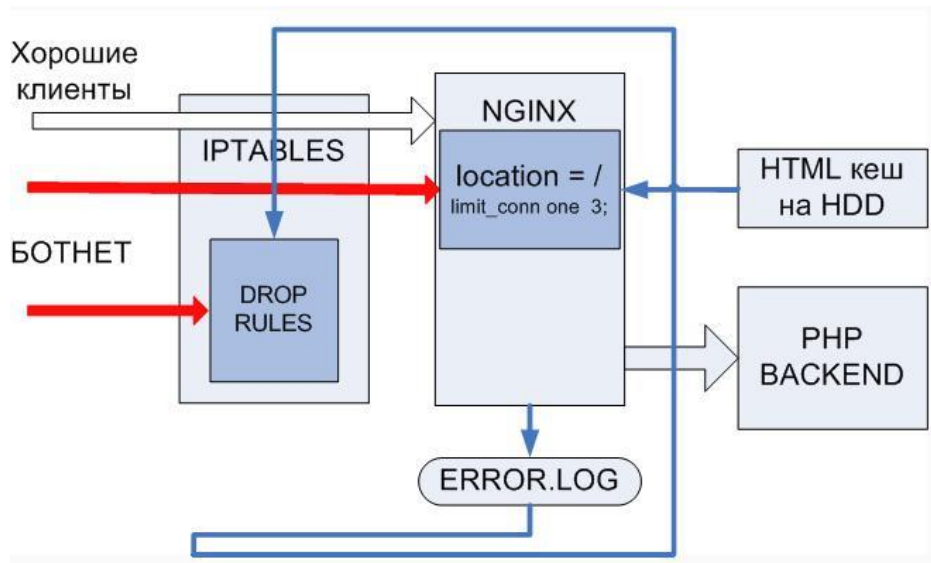


Рисунок 2.8 – Принципиальная схема фильтрации IP-адресов согласно лимиту запросов

2.10.3.7 Фильтрация запросов по URL

При помощи специальных средств Web-сервера Nginx имеется возможность осуществлять фильтрацию запросов по URL согласно некоторому критерию. Ниже приведен пример фильтрации согласно критерию «POST / HTTP/1.0» без URL (с пустым реферером):

```
set $add 1;
location / {
limit_except GET POST {
deny all; } set $ban "";
if ($HTTP_referer = "" ) {set $ban $ban$add;}
if ($request_method = POST ) {set $ban $ban$add;}
if ($query_string = "HTTP/1.0" ){set $ban $ban$add;}
if ($ban = 111 ) {
access_log /var/log/nginx/ban IP; return 404;
}
proxy_pass HTTP://127.0.0.1:8000;
}
```

Выводы

Данный раздел посвящен рассмотрению DDoS-атак и выделению их особенностей. Была произведена классификация, а также выделены наиболее распространенные виды DDoS-атак:

- TCP SYN Flood;
- TCP Flood;
- Ping of Death;
- ICMP Flood;
- UDP Flood;
- HTTP Flood.

Более того, в текущем разделе также рассмотрены основные существующие методы защиты от DDoS-атак, приведены самые мощные атаки, отмеченные историей. Был проанализирован рынок программного обеспечения и выделены 2 вендора поставки продукции защиты от атак – Cisco, ADOS.

3 АЛГОРИТМИЧЕСКАЯ РЕАЛИЗАЦИЯ

Разрабатываемая система отражения атак будет состоять из следующих составляющих (рисунок 3.1): прокси-сервера, на котором реализован алгоритм фильтрации поступающих запросов и целевой сервер, на котором физически расположена атакуемая информационная система.

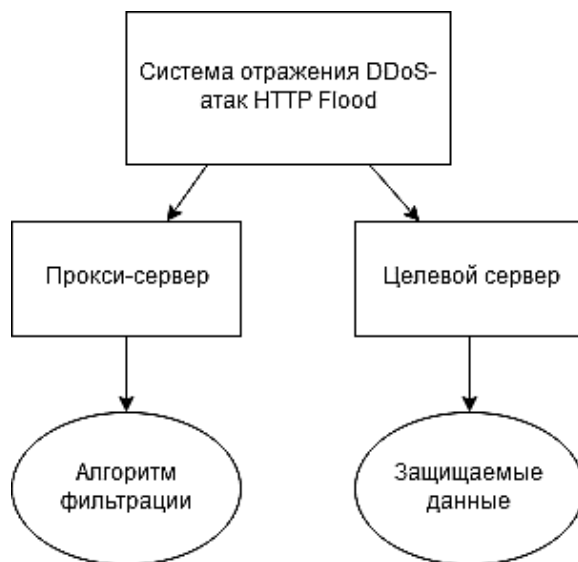


Рисунок 3.1 – Разрабатываемая система отражения атак

3.1 Отправка запросов в глобальной сети Интернет и проксирование

HTTP прокси-сервер, изображенный на рисунке 3.2, представляет собой сервер обработки HTTP-запросов, поступающих от клиентов. Таким образом, этот узел является промежуточным звеном между пользователем и целевым сервером вне зависимости от осведомленности обеих сторон о существовании указанного хоста. Использование подобного метода позволяет клиентам осуществлять отправку запросов и получать ответ как от целевого сервера, так и от прокси.

Первоначально при отправке запроса происходит подключение к прокси-серверу, затем с прокси совершается очередное подключение к целевому серверу, запрашивается требуемый клиентом ресурс и результат отдается клиентской стороне. Также в некоторых случаях запрос, совершаемый клиентской стороной, может быть модифицирован в соответствии с согласованной настройкой прокси-сервера с целевым сервером. К тому же, использование технологии проксирования при определенной настройке предоставляет возможность обеспечивать защиту компьютера или информационной системы от различного рода сетевых атак, но в иногда данная технология применяется злоумышленниками с целью скрывания целевого IP-адреса или перехвата незащищенных или слабо защищенных пользовательских данных [21].

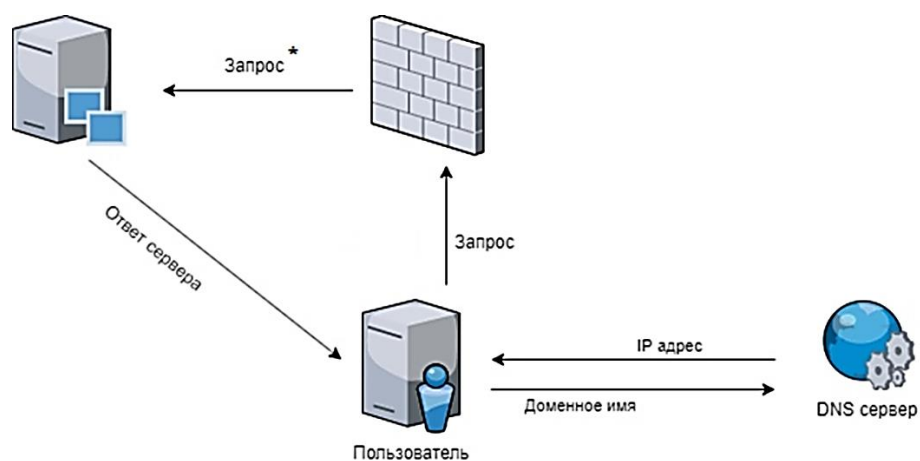


Рисунок 3.2 – Отправка запросов с участием прокси

Наиболее часто прокси-серверы применяются в ситуациях, описанных ниже.

- Доступ устройств, находящихся внутри локальной сети, в глобальную сеть.
- В некоторых случаях при частых отправках запросов в одном и тем же данным целесообразно использовать рассматриваемую технологию в целях кеширования ресурсов. Таким образом, при кешировании данных на прокси-сервере снижается нагрузка на целевой сервер, при этом пользователи быстрее получают запрашиваемые данные, так как отсутствует необходимость их обработки на сервере, более того, зачастую промежуточный узел физически находится ближе к пользователю. К тому же в случае внесения изменений в контент ресурса, его возможно модифицировать, применяя соответствующие корректировки в заголовки HTTP-запросов.
- Использование прокси-серверов с целью минимизации объема передаваемого трафика посредством сжатия контента.
- В коммерческих организациях прокси-сервер может служить хостом, при помощи которого осуществляется выход в глобальную сеть.
- Скрытие данных источника запросов путем подмены IP-адреса источника, так как в данном случае запросы отправляются с прокси-сервера.

Из всего разнообразия прокси-серверов выделяют два основных типа: прозрачный прокси и обратный прокси.

При технологии «Прозрачный прокси» трафик в неявном виде перенаправляется на прокси-сервер с использованием маршрутизатора.

Использование технологии «Обратный прокси» позволяет осуществлять ретрансляцию запросов клиентов из внешней сети на серверы (их может быть один или несколько), которые логически расположены во внутренней сети. Наиболее часто данная технология применяется в целях балансировки сетевой нагрузки между некоторым количеством целевых серверов, а также обеспечения их безопасности, выступая в качестве фаервола на уровне приложений L7 модели OSI.

При обращении к запрашиваемому ресурсу запросы с клиентской части отправляются не напрямую на целевой сервер, на котором расположены ресурсы, а на прокси в соответствии с настройкой записей DNS, где указан IP-адрес прокси-сервера. В случае отправки запроса клиентской частью открывается соединение с прокси-сервером, после чего клиент осуществляет отправку запроса, который затем поступает на прокси. Затем происходит процесс обработки запроса на прокси-сервере с последующим открытием соединения с целевым сервером, передавая запрос клиента. После этого целевой сервер отвечает на запрос прокси-сервера, который доставляет данные клиенту.

В общем, прокси-сервер представляет собой многофункциональный сервер с настроенной клиентской частью для конкретно используемого протокола.

Web-прокси представляет собой прокси-сервер и сервер, предоставляющий анонимность в виде Web-приложения, внедренного на Web-сервер, а также используется в роли промежуточного звена для отправки запросов и получения контента из информационных систем. Web-прокси могут быть использованы для:

- ускорения доставки контента;
- проведения тестирования информационных сервисов;
- доступа к внешней сети из локальной сети;
- анонимизации и сокрытия реального IP-адреса для доступа в ИС;
- получения доступа к заблокированным в государстве Web-сайтам.

Web-прокси устанавливается на Web-сервере, позволяя легитимным пользователям осуществлять запросы с IP-адресом этого сервера в качестве источника, что, в данном случае, обеспечивает анонимность пользователей с учётом определенных настроек Web-прокси сервера [22].

3.2 Разрабатываемый метод проксирования запросов

Клиентские запросы отправляются на прокси-сервер, на котором осуществляется их обработка, после чего на прокси также может быть сгенерирован ответ клиенту. В ином случае совершается отправка запроса, идентичного клиентскому запросу, на целевой сервер с последующей передачей ответа от него клиенту. Таким образом, в данной системе прокси-сервер может выступать как в качестве сервера, так и в качестве клиента. В момент приема запросов от клиентской части информационной системы он является сервером, а по отношению к целевому серверу он является клиентом.

На рисунке 3.3 схематически показана система с наличием прокси-сервера.

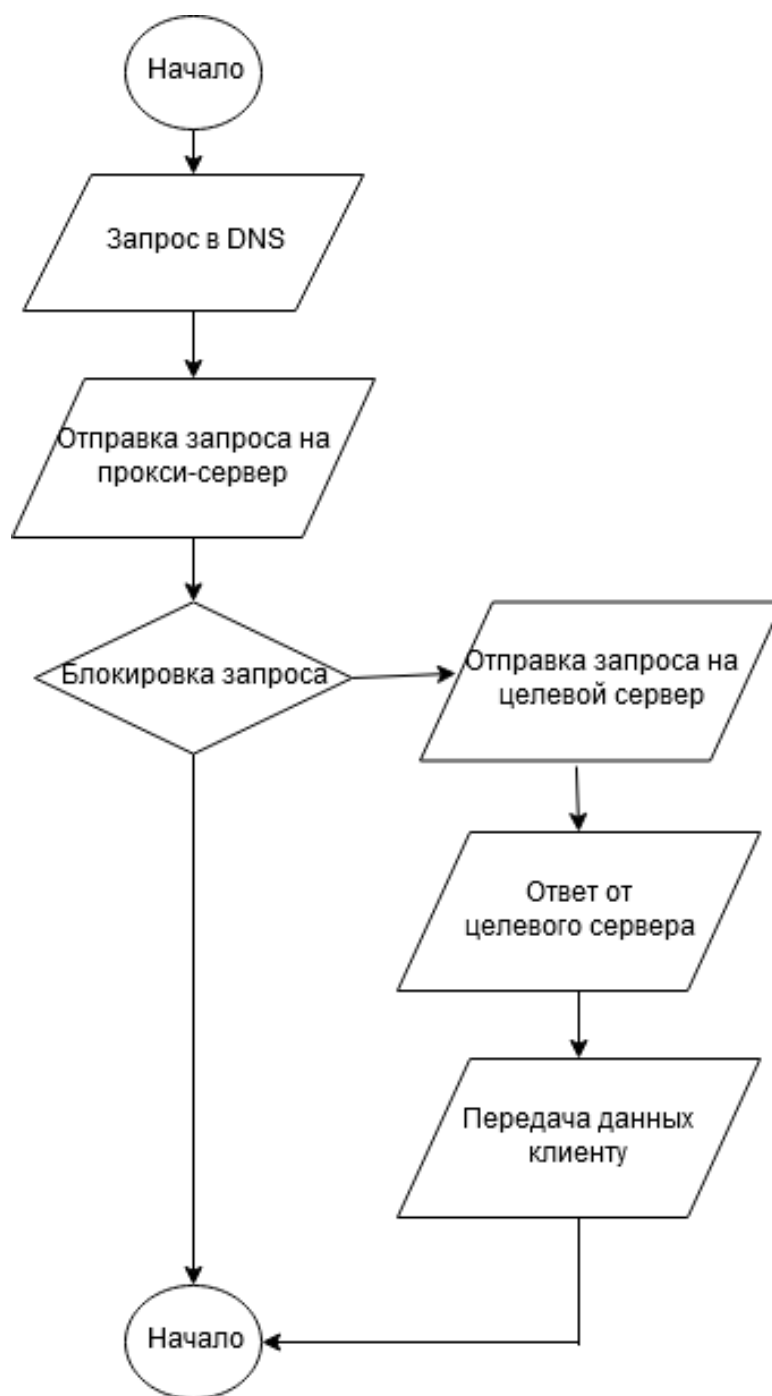


Рисунок 3.3 – Схема система с прокси-сервером

Рассмотрим процесс работы прокси-сервера более подробным образом.

Изначально сервер находится в состоянии ожидания получения клиентских запросов на 80 (HTTP) порт. Затем для каждого запроса отдельным образом создается новый поток для обработки данных, в котором осуществляется обработка запроса. После получения запроса на прокси-сервере осуществляется блокировка IP-адреса с последующим ответом 403 стороне, отправляющей запрос, либо отправление его на целевой сервер [21].

Во всех остальных случаях запрос будет послан удалённому серверу, с которого запрашивается документ, или другому прокси-серверу, если таковой был определён. При этом число неудачного использования кэша (Cache misses), будет увеличено на единицу. После обработки полученного запроса целевым сервером отправляется ответ 200, который сначала доставляется на прокси-сервер с дальнейшей передачей клиенту. На прокси-сервере также осуществляется хранение статистических данных в течение взаимодействия с целевым сервером.

3.3 Алгоритм обнаружения и фильтрации

В данном исследовании предлагается новый принцип обнаружения и блокировки запросов, изображенный на рисунке 3.4.

Таким образом клиент, либо злоумышленник при отправке запроса отправляет его напрямую на прокси-сервер, так как предполагается, что А-записи DNS атакуемого сайта направлены на IP-адрес прокси. В случае, если запрос невалидный, то он блокируется по тайм-ауту на 30 секунд с последующим увеличением в 2 раза при повторении ситуации.

Если же нагрузка на сервер возрастает до 95%, то на прокси-сервере измеряется время между запросами и производится расчет нормы запросов в единицу времени по формуле:

$$q = (C_1 * W_1 + \dots + C_n * W_n) / t,$$

и в дальнейшем, если количество запросов с одного IP-адреса превышает q на 75%, то этот IP-адрес блокируется на прокси-сервере по тайм-ауту.

Выводы

В этом разделе подробно описан принцип отправки запросов в глобальной сети Интернет, так же представлен и подробно описан принцип проксирования запросов и подробным образом рассмотрен предлагаемый алгоритм обнаружения и фильтрации аномалий трафика.

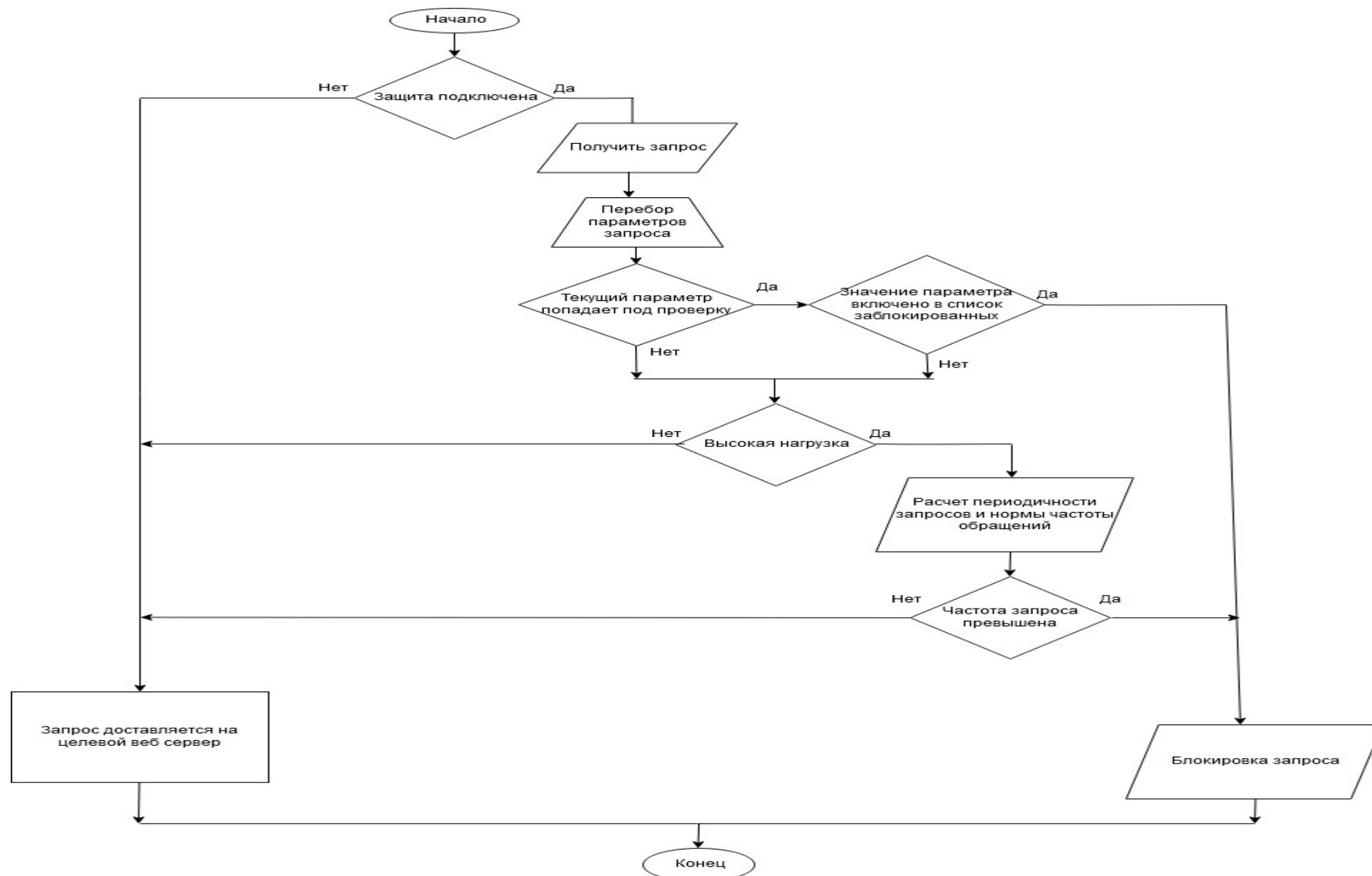


Рисунок 3.4 – Принцип отражения и блокировки запросов

4 ПРОГРАММНОЕ КОНСТРУИРОВАНИЕ

В текущем разделе приведено обоснование целесообразности выбранного языка программирования, дано описание разработанных методов и классов.

4.1 Обоснование выбора языка программирования

В качестве языка программирования для создания требуемого программного обеспечения были выбраны язык гипертекстовой разметки HTML и мультипарадигменный язык программирования JavaScript. Это связано с тем, что HTML является наиболее распространенным языком создания Web-сайтов.

В ходе работы также использовался язык программирования – JavaScript. Это язык управления сценариями просмотра гипертекстовых страниц Web на стороне клиента. Основная идея использования языка JavaScript состоит в возможности изменения значений атрибутов HTML-контейнеров и свойств среды отображения в процессе просмотра HTML-страницы пользователем. При этом перезагрузки страницы не происходит. На практике это означает, что можно изменить цвет фона страницы или картинку, интегрированную в документ, открыть новое окно или выдать предупреждение.

4.2 Обоснование выбора среды разработки

Существуют причины, по которым выбран проект Adobe Brackets. Его делает Adobe, и поддержка такой крупной корпорации — значит очень много. В случае со средой разработки Adobe Brackets – это настоящее открытое программное обеспечение Open Source — уровень открытости весьма высок. Проект был опубликован на сайте Github в начале мая этого года, несмотря на то, что он находится ещё в начальном состоянии. *Бэклог* проекта (список новых функций, изменений и исправлений) тоже открыт для всех — разработка ведётся по методологии управления проектами *scrum* с помощью спринтов (промежутков времени, за которые команда разработчиков создаёт часть программы, готовую к демонстрации и ценную для заказчика) по 2,5 недели. Дизайн и архитектура этого IDE заслуживают более подробного исследования.

Проект Brackets использован как интегрированная среда Web-разработки.

Принцип, лежащий в основе дизайна Brackets — минимализм и чистота интерфейса. Вместо того, чтобы создавать окна и вкладки, среда Brackets позволяет редактировать связанные участки кода из разных файлов.

Например, если подвести курсор к тегу, *id* или имени класса элемента в HTML-файле и нажать Ctrl+E (Quick edit), то фрагмент кода CSS, относящийся к этому элементу, откроется здесь же, в Inline-редакторе. Внеся любые изменения, можно нажать Ctrl+E повторно, и фрагмент закроется. Можно определить функцию в коде JavaScript из любого места кода, где эта функция вызывается.

4.3 Описание основных модулей программы

Рассмотрим классы DNS:

Класс *DNS128* – класс, который отвечает за DNS-резолв. Полей класс не имеет, и в нём реализованы следующие методы:

Поиск *name servers*. Метод возвращает строку-результат, с реальным именем сервера name server DNS.

Метод *Decrypt DNS* – на вход принимает два параметра: строку типа String и объект типа SecretKeySpec. Первый входной параметр – это строка, которую надо искать. Второй – это путь к DNS. Метод возвращает строку-результат.

Метод *Encrypt* – на вход принимает два параметра: строку типа String и объект типа SecretKeySpec. Первый входной параметр – это строка, которую надо зашифровать. Второй – это ключ для резолва.

Класс *Base64* содержит два метода перевода объекта типа String в байт-массив.

Метод *Encode* типа String – переводит массив байтов в строку и возвращает её. На вход принимает одномерный массив типа byte.

Метод *Decode* типа String – переводит строку в массив байтов и возвращает его. На вход принимает объект типа String.

Класс ***CipherMessage*** – его методы служат для конвертации типов.

Класс ***SideA*** – служит для генерации ключа со стороны А.

Класс ***SideB*** – служит для генерации ключа со стороны Б.

Класс ***TransportPublicKey*** предназначен для транспортировки ключа от стороны А к стороне Б. Этот класс методов не имеет.

Класс ***Connect_info*** – этот класс не имеет методов. Его объекты хранятся на сервере, в объекты этого класса записывается вся информация о пользователе, которой обладает сервер. У объекта имеется 4 поля:

Поле *name* типа String – хранит уникальное имя клиента.

Поле *address* типа String – хранит ip адрес и порт.

Поле *Socket* типа Socket – хранит сокет (программный интерфейс для обеспечения обмена данными), через который осуществляется связь с клиентом.

Поле *clientsFriendsList* содержащее ссылочную переменную на объект *ArrayList* – коллекция, которая хранит объекты класса *Friend*. В этой коллекции будет храниться список друзей клиента, чтоб сервер знал об изменении статуса каких уникальных имен надо оповещать клиента.

Класс *Dialogs* – объекты этого класса служат для хранения истории переписки с отдельным пользователем во время сеанса работы программы.

Объект класса содержит три поля *Private*. То есть доступ напрямую к ним запрещен. Для работы с полями были реализованы *Getters* и *Setters*.

Поле *Name* типа *String* – хранит уникальное имя собеседника.

Поле *Status* типа *Boolean* хранит статус последнего сообщения – было оно прочитано, или нет, во время переписки с легитимным пользователем.

Поле *DialogList* содержащее ссылочную переменную на объект *ArrayList* – коллекция в которой хранятся сообщения на время сеанса.

Класс *Enter* – класс, отвечающий за взаимодействие с индивидуальным входным ключом. *Enter* наследуется от класса *JFrame* и реализует интерфейс *Serializable*. Содержит четыре поля, для которых реализованы геттеры и сеттеры. Также класс содержит семь методов. Рассмотрим содержимое класса.

Поле *Name* типа *String* – хранит уникальное имя легитимного пользователя, полученное им из индивидуального входного ключа.

Поле *Address* типа *String* – в эту переменную записывается адрес индивидуального входного ключа с помощью метода *Start*.

Поле *Friends* содержащее ссылочную переменную на объект *ArrayList* – коллекция, в которой хранятся имена друзей в формате *String*.

Поле *FriendsClassFriend* содержащее ссылочную переменную на объект *ArrayList* – коллекция, в которой хранятся объекты класса *Friend*.

Метод *Start* типа *Void*. На вход параметров не принимает. Отвечает за генерацию диалогового окна для выбора расположения индивидуального входного ключа, дешифрования параметров из этого файла и записи всех необходимых входных данных для старта работы мессенджера, а именно имени пользователя и списка его друзей. В своем теле метод обращается к *Read*.

Метод *AddFriend* типа *Void*. Метод принимает на вход строку, в которой записано уникальное имя нового пользователя, которого клиент хочет добавить в список друзей в ключевом файле. Считывает уже существующий файл, добавляет в список друзей нового друга, перезаписывает файл.

Метод *DeleteFriend* типа *Void*. Метод принимает на вход строку, в которой записано уникальное имя пользователя, которого клиент хочет удалить из списка друзей в ключевом файле. Алгоритм работы метода аналогичен вышеописанному *AddFriend*.

Метод *Refresh* типа *Void*. На вход не принимает никаких параметров. Отвечает за обновления поля *FriendsClassFriend*.

Метод *Read* типа *String* – возвращает расшифрованное уникальное имя пользователя. На вход принимает строку с зашифрованным именем пользователя. Обращается к методу *ReadPassword*, который выводит диалоговое окно для ввода пароля пользователем, получает его хэш с помощью *GenerateHash* и использует полученную последовательность битов в качестве ключа для декодирования.

Метод *ReadPassword* типа *String* – возвращает введенный пользователем пароль.

Метод *GenerateHash* типа *String* – возвращает сгенерированный хэш.

Класс *Friend* – класс, объекты которого хранят информацию о клиенте в списке друзей. Реализует интерфейс *Serializable*. Класс содержит два поля, для которых в языке JavaScript реализованы специальные методы *геттеры* (от слова *get* – получать) и *сеттеры* (от слова *set* – устанавливать).

Поле *FriendName* типа *String* – хранит уникальное имя.

Поле *FriendStatus* типа *Boolean* – используется клиентом и сервером для определения статуса пользователя (онлайн или офлайн).

Класс *Keys* – класс, объекты которого используются для хранения сеансовых ключей шифрования. Содержит 2 поля, для которых реализованы специальные методы – *геттеры* («получатели») и *сеттеры* («установители»).

Поле *FriendName* типа *String* – хранит уникальное имя собеседника.

Поле *SeckretKeySpec* – содержит ссылочную переменную на объект класса *SeckretKeySpec*. Служит для хранения сеансового ключа.

Класс *Message* – объекты данного класса предназначены для обмена информацией между клиентом и сервером. Реализует интерфейс *Serializable*. Содержит 10 полей, для всех полей реализованы *геттеры* и *сеттеры*:

Поля *bKeyAsHexStringBase64*, *pAsValueAsHexBase64*, *gAsValueAsHexBase64*, *yAsValueAsHexBase64* типа *String* используются при генерации ключа по алгоритму Диффи – Хеллмана (позволяющему двум сторонам получить общий *секретный ключ*, используя незащищённый от прослушивания, но защищённый от подмены канал связи для обмена сообщениями с помощью симметричного шифрования);

Поле *Name* типа *String* служит для хранения уникального имени отправителя.

Поле *Address* типа *String* служит для хранения уникального имени получателя.

Поле *Data* типа *String* служит для хранения данных в передаваемом сообщении.

Поле *List* содержит одномерный массив типа *String* – служит для отправки сервером имен клиентов, находящихся онлайн.

Поле *Instructions* содержащее ссылочную переменную на объект *ArrayList* – коллекция, в которой хранятся объекты класса *Friend* – список друзей. *Setter* для этого экземпляра класса перегружен 4 раза. Число входных параметров – от 1 до 4.

Класс *Client* – класс, отвечающий за работу клиентской части мессенджера в целом. Содержит в себе 19 полей, 14 методов, 5 экземпляров внутренних классов:

Поле *Incoming* типа *TextArea* – отвечает за многомерное поле, в которое будут выводятся сообщения диалога (часть интерфейса);

Поле *Outgoing* типа *TextField* – отвечает за текстовое поле, в которое будут вводиться текстовые сообщения (часть интерфейса).

Поле *Fame* типа *JFrame* – объект класса *JFrame* представляет собой окно с рамкой и строкой заголовка с кнопками «Свернуть», «На весь экран» и «Закрыть».

Поле *Socket* типа *Socket* (программный интерфейс для обмена данными) – объекты этого типа создают соединения между вычислительными устройствами;

Поле *FlagToFirstGenerateKey* типа *Boolean* – переменная отвечает за то, чтобы генерация ключа со стороны А была выполнена один раз при запуске программы.

Поле *NotClone* типа *Boolean* – эта переменная служит для проверки состояния пользователя – находится ли пользователь онлайн или офлайн;

Поле *MyFriendsList* типа *ArrayList* – служит для хранения друзей клиента.

Поле *FriendsListForKeys* типа *ArrayList* – для хранения сеансовых ключей.

Поле *MyDialogsList* типа *ArrayList* – для хранения текущих диалогов клиента.

Поле *Data* – это одномерный массив типа *String*. Этот массив служит в качестве промежуточного места хранения списка друзей.

Поле *Model* типа *DefaultListModel* – элемент интерфейса, служащий для вывода списка друзей легитимного пользователя.

Выводы

В данном разделе был обоснован выбор как языка программирования, так и среды разработки программного средства. Были описаны основные классы и методы, которые были использованы в ходе выполнения проекта. Информация, полученная из описания, достаточна для понимания принципа работы как программы в целом, так и отдельных её частей.

5 ТЕСТИРОВАНИЕ МОДУЛЕЙ ПРОГРАММНОГО СРЕДСТВА

В текущем разделе будут описаны модули разработанного программного обеспечения, а также примеры работы программного средства. Программное средство реализовано в виде эмулятора *монитора атаки* и *монитора защиты*. Предполагается, что монитор атаки представляет собой организованную сеть *ботнет*, состоящую из зараженных устройств, подключенных к глобальной сети Интернет, а монитор защиты – программное обеспечение, установленное на прокси-сервере, при помощи которого осуществляется фильтрация трафика.

Рассмотрим более подробно составные части атакующей стороны. Но сначала проанализируем раздел «Параметры атаки», изображённый на рисунке 5.1.

Параметры атаки

Домен

donstu.ru

Ip

80.237.26.241

Нагрузка

15

Атака

Рисунок 5.1 – Параметры атаки

В данном окне находится три поля ввода. В поле «Домен» вводится доменное имя сайта, на который производится атака. В поле IP указывается IP–адрес сервера или хостинга, на котором физически расположен сайт, также эти данные могут быть автоматическим способом подтянуты из DNS при нажатии кнопки «Tab». В поле «Нагрузка» указывается среднее число невалидных запросов в секунду, которые также могут быть интерпретированы, как атакующие. Также на данном окне расположена кнопка «Атака», при нажатии на которую активизируются атакующие запросы на указанный IP–адрес.

На рисунке 5.2 изображены параметры указанного доменного имени, которые берутся из актуальной для домены зоны DNS, в частности, параметр А – для записи доменного имени.

Параметры цели

```
array(1) {  
  [0]=>  
  array(5) {  
    ["host"]=>  
    string(9) "donstu.ru"  
    ["class"]=>  
    string(2) "IN"  
    ["ttl"]=>  
    int(86397)  
    ["type"]=>  
    string(1) "A"  
    ["ip"]=>  
    string(13) "80.237.26.241"  
  }  
}
```

Рисунок 5.2 – Параметры домена

Основная часть монитора атаки находится в разделе «Список ботов» (рисунок 5.3), куда записываются IP-адрес источника невалидного запроса, юзер-агент и код ответа, который может быть получен как от целевого сервера, так и от прокси-сервера.

Список ботов		
168.132.23.49	URI::Fetch	200
180.21.245.224	InternetSeer.com	200
163.77.202.81	SuperBot	200
164.169.219.245	svetabot	no response
153.239.181.200	NearSite	no response
163.255.112.243	lwp-trivial	200
156.127.117.25	Offline Explorer	200
178.199.254.48	Snoopy	200
153.68.232.86	CazoodleBot	no response
177.211.107.59	LinksManager.com_bot	200
180.51.59.59	sucker	no response

Рисунок 5.3 – Список ботов

Рассмотрим составные части монитора защиты. На рисунке 5.4 изображен график поступающих запросов и заблокированных из них, где синей полосой обозначено количество запросов, поступающих от легальных пользователей информационной системы, красным – количество заблокированных запросов, а оранжевым – общая нагрузка на сервер, выраженная в количестве запросов в секунду.

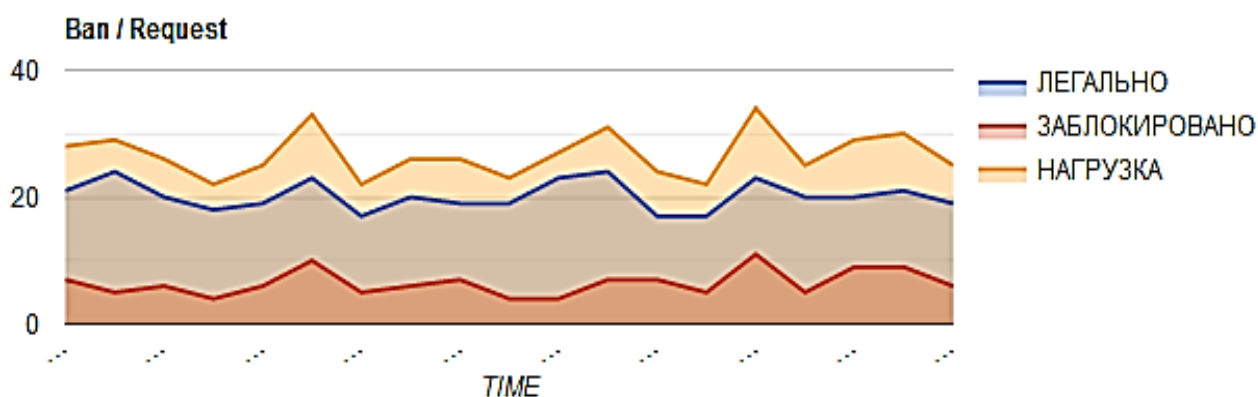


Рисунок 5.4 – График запросов и банов

Также имеется еще график (рисунок 5.5), показывающий нагрузку на процессор обрабатываемыми запросами, выраженную в процентном содержании.

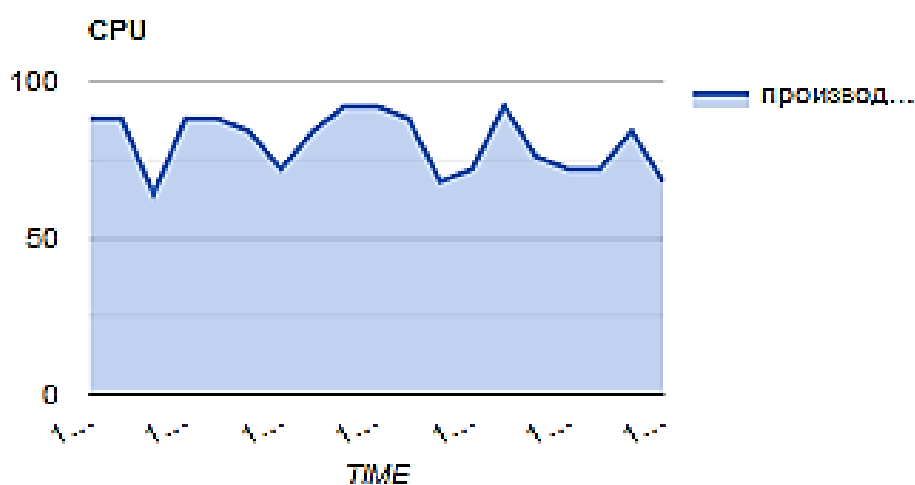


Рисунок 5.5 – Загруженность процессора

На мониторе защиты расположено окно конфигурации фильтра (рисунок 5.6), согласно которому осуществляется блокировка запросов на входе прокси-сервера. На данном окне присутствует возможность ручного добавления критериев, в соответствии с которыми осуществляются блокировки на прокси-сервере.

Параметры фильтра

		Добавить
----------------------	----------------------	---

agent	NearSite
ip	12
agent	WPScan
agent	IDBot
load	25
agent	CazoodleBot
agent	NearSite
agent	PECL::HTTP
agent	Web Sucker
agent	Express WebPictures
agent	sucker
agent	FlashGet
agent	SuperHTTP
agent	webalta
agent	NearSite
agent	Grafula
agent	Navroad
agent	Edge/15
	svetabot
agent	svetabot
agent	FHscan

Рисунок 5.6 – Параметры фильтра

Затем, при нажатии на какую-либо точки в любой момент времени на графике запросов и банов откроется дополнительное окно, отображающее все принятые запросы в конкретный момент времени, где красным цветом выделены те запросы, IP-адреса источников которых были заблокированы (рисунок 5.7).

Теперь, когда мы рассмотрели основной функционал программного обеспечения, детальным образом рассмотрим его в действии (рисунок 5.8).

Нагрузка: **19** в секунду

161.52.206.215	80.237.26.241	POST	XoviBot	donstu.ru	200
170.202.47.121	80.237.26.241	GET	FHscan	donstu.ru	blocked
167.180.34.150	80.237.26.241	POST	Widow	donstu.ru	200
169.141.229.160	80.237.26.241	POST	Surfbot	donstu.ru	200
164.97.46.42	80.237.26.241	GET	ecxi	donstu.ru	200
163.77.202.81	80.237.26.241	POST	SuperBot	donstu.ru	200
167.180.34.150	80.237.26.241	POST	Widow	donstu.ru	200
156.127.117.25	80.237.26.241	GET	Offline Explorer	donstu.ru	200
179.22.147.171	80.237.26.241	GET	Web Sucker	donstu.ru	blocked
167.157.132.96	80.237.26.241	GET	NearSite	donstu.ru	blocked
170.202.47.121	80.237.26.241	POST	FHscan	donstu.ru	blocked
177.166.181.52	80.237.26.241	POST	libwww	donstu.ru	200
177.166.181.52	80.237.26.241	POST	libwww	donstu.ru	200
163.255.112.243	80.237.26.241	POST	lwp-trivial	donstu.ru	200
178.199.254.48	80.237.26.241	GET	Snoopy	donstu.ru	200
158.155.88.71	80.237.26.241	GET	Edge/15	donstu.ru	blocked
153.234.207.164	80.237.26.241	GET	Edge/15	donstu.ru	blocked
180.170.252.123	80.237.26.241	POST	YandexBot	donstu.ru	200
169.207.254.137	80.237.26.241	GET	CheckHost	donstu.ru	200

Рисунок 5.7 – Нагрузка в конкретный момент времени

Монитор атаки

Список ботов		
170.11.191.63	YandexBot	—
164.13.141.189	Firefox/64	—
171.205.59.174	Firefox/66	—
162.161.97.74	YaBrowser/18.10	—
170.66.77.243	Firefox/62	—
176.220.14.246	Edge/17	—
180.170.252.123	YandexBot	—
175.239.133.43	YandexBot	—

Параметры атаки

Домен

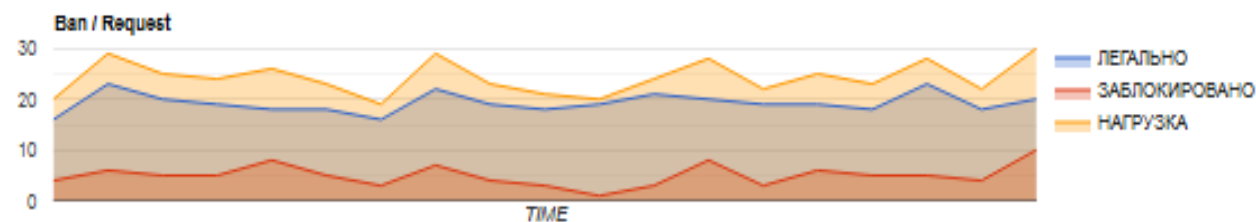
Ip

Нагрузка

16

Атака

Рисунок 5.8 – Начало атаки



Нагрузка: 18 в секунду

178.199.254.48	80.237.26.241	GET	Snoopy	donstu.ru	200
176.141.189.48	80.237.26.241	GET	Grafula	donstu.ru	blocked
164.97.46.42	80.237.26.241	POST	ecxi	donstu.ru	200
174.179.170.46	80.237.26.241	POST	WWW-Mechanize	donstu.ru	200
167.167.132.96	80.237.26.241	POST	NearSite	donstu.ru	blocked
160.18.11.60	80.237.26.241	POST	PECL::HTTP	donstu.ru	blocked
163.239.181.200	80.237.26.241	GET	NearSite	donstu.ru	blocked
180.105.69.52	80.237.26.241	GET	ExtractorPro	donstu.ru	200
178.102.37.111	80.237.26.241	POST	CazoodleBot	donstu.ru	blocked
180.61.59.59	80.237.26.241	POST	sucker	donstu.ru	blocked
179.254.143.39	80.237.26.241	GET	WWW-Mechanize	donstu.ru	200
163.89.61.17	80.237.26.241	POST	ExtractorPro	donstu.ru	200
161.26.69.103	80.237.26.241	GET	YaBrowser/18.9	donstu.ru	200
158.166.88.71	80.237.26.241	GET	Edge/15	donstu.ru	blocked
160.240.21.200	80.237.26.241	POST	Chrome/72	donstu.ru	200
162.86.199.113	80.237.26.241	GET	Edge/16	donstu.ru	200
160.57.25.196	80.237.26.241	POST	YaBrowser/19.4	donstu.ru	200
158.166.88.71	80.237.26.241	GET	Edge/15	donstu.ru	blocked

Рисунок 5.9 – Монитор защиты (а)

Итак, рассмотрим ситуацию, при которой некий сайт «donstu.ru» был атакован сетью ботнет со средней частотой отправления нелегитимных запросов 15 запросов в секунду, как показано на рисунке 5.8.

Как видим на рисунке 5.9, *а* и *б*, на мониторе защиты графики динамически изменяются ежесекундно, при поступающих запросах.

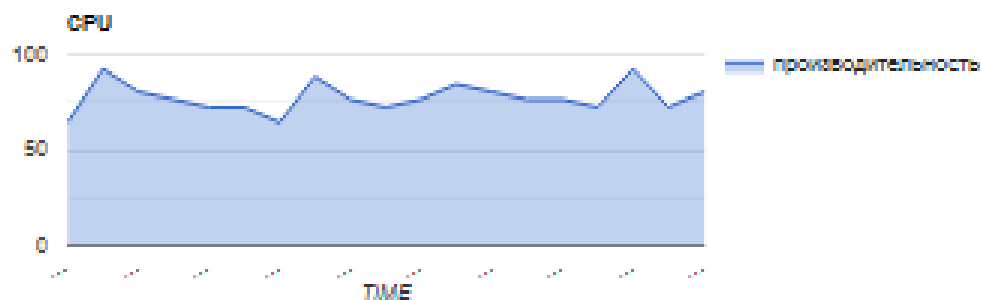
Также при выводе информации о поступивших запросах в некоторый определенный момент времени видим, что белым цветом выделены запросы, которые были отправлены на обработку целевому серверу, а красным цветом обозначены заблокированные запросы с соответствующим образом добавленным флагом «blocked» к запросу, что на атакующей стороне будет отображаться как «No response», как на рисунке 5.10, так как при блокировке IP-адреса прокси-сервером не предоставляется ответ атакующей стороне в целях минимизации ресурсозатратности на обработку запросов.

Далее, при необходимости внесения дополнительных корректировок в систему фильтрации имеется возможность добавления IP-адреса и юзер-агентов в черный список в разделе «Параметры фильтра» следующим образом: «IP» «176.141.189.48» или «agent» «IDBot».

Также необходимо отметить следующую особенность системы фильтрации: при достижении уровня загрузки процессора 90% производится расчет нормы числа запросов в секунду, после чего рассчитывается средняя частота запросов с конкретного IP-адреса, и, в случае, если она превышает норму на 75%, то IP-адрес источника блокируется по тайм-ауту.

Выводы

В данном разделе были рассмотрены все элементы функционала программы. В дальнейшем планируется расширение функционала и усовершенствование интерфейса. Полное описание программного средства находится в приложениях, а именно: руководство системного администратора, руководство программиста, руководство оператора и Исходный код программного средства.



Параметры фильтра

agent	NearSite
ip	12
agent	WPScan
agent	IDBot
load	25
agent	CazoodleBot
agent	NearSite
agent	PECL-HTTP
agent	Web Sucker
agent	Express WebPictures
agent	sucker
agent	FlashGet
agent	SuperHTTP
agent	webalta
agent	NearSite
agent	Grafula
agent	Navroad
agent	Edge/16
	svetabot
agent	svetabot
agent	FHscan

Рисунок 5.9 – Монитор защиты (б)

163.77.202.81 SuperBot	200
164.169.219.245 svetabot	no response
153.239.181.200 NearSite	no response
163.255.112.243 lwp-trivial	200
156.127.117.25 Offline Explorer	200
178.199.254.48 Snoopy	200
153.68.232.86 CazoodleBot	no response
177.211.107.59 LinksManager.com_bot	200
180.51.59.59 sucker	no response

Рисунок 5.10 – Флаги с ответами на клиентской стороне

РЕЗУЛЬТАТЫ 2 ГЛАВЫ

В сегодняшнем мире особенно остро стоит проблема защиты информации и обеспечения её безопасности для всех пользователей глобальной сети Интернет.

Вне зависимости от того, какого рода информация передается по каналам связи, будь то конфиденциальные данные, или же данные общего назначения, необходимо обеспечивать её безопасность, а в случае атак типа «Отказ в обслуживании» – гарантировать доступность передаваемых и получаемых информационных ресурсов. Очень многие производители программного обеспечения заинтересованы в разработке такого ПО, с помощью которого имеется возможность обнаруживать и блокировать DDoS-атаки, в особенности, атаки типа HTTP Flood, которые являются наиболее распространенными видами DDoS-атак.

В данной работе предпринята попытка создания импортозамещающего программного средства для обнаружения и блокировки DDoS-атак, в котором реализован новый алгоритм, предназначенный для защиты данных любой степени секретности от атаки типа HTTP Flood. В эмуляционном программном обеспечении создан интуитивно понятный дружественный программный интерфейс, доступный при запуске с любого устройства. В ходе работы также была произведена классификация наиболее часто осуществляемых атак типа DDoS и предоставлена новая методика фильтрации, основанная на методе проксирования L7 запросов.

Результатов работы стало импортозамещающее программное средство, реализованное на практике, визуально отображающее эффективность разработанного алгоритма. Приведено тестирование разработанного программного продукта, выполнен детальный обзор проведенных испытаний в отчёте, где проиллюстрированы основные этапы работы программного обеспечения.

ГЛАВА 3

ИМПОРТОЗАМЕЩЕНИЕ ПРОГРАММНОГО МОНИТОРА БЕЗОПАСНОСТИ ДЛЯ ОТРАЖЕНИЯ КИБЕРАТАК ТИПА «УДАЛЁННЫЙ ДОСТУП»

ОБЩИЕ ПОЛОЖЕНИЯ

В настоящее время эффективная работа, связанная с анализом и обработкой информации, невозможна без использования компьютерных сетей. Частные сети фирм, в том числе и российских, предоставляют доступ к базам данных и новостям. Кроме того, пользователь получает возможность коллективной работы и оперативных консультаций с коллегами и экспертами. Подобные услуги оказываются необходимыми для бизнесменов, врачей, учителей, госчиновников, журналистов и т.п. Для обеспечения такого рода работы в рамках некоторой структуры, например, фирмы, используют локальные сети.

Однако физическая реализация локальной сети не всегда возможна или удобна. Часто складывается ситуация, когда подразделения фирмы географически удалены друг от друга, или работники находятся не в помещении офиса, а работают с клиентами на местах, выезжают в командировки. В этом случае важно организовать работу таким образом, чтобы, находясь в отдалении от локальной сети фирмы, они, тем не менее, могли подключиться к ней.

Для того, чтобы обеспечить доступ к услугам локальной сети при отсутствии непосредственного подключения к ней, служат средства удалённого доступа к сети. Эти средства используют для связи удаленного (то есть не имеющего непосредственного подключения к локальной сети) компьютера с сетью коммутированные линии телефонных сетей общего пользования, аналоговые выделенные линии, цифровые линии сети ISDN (Integrated Services Digital Network – цифровая сеть с интеграцией служб), сети коммутации пакетов общего пользования. Они обеспечивают передачу данных на любые расстояния.

Безусловным преимуществом таких систем является снижение затрат на обслуживание данной категории пользователей. Во-первых, пропадает необходимость выделения офисного пространства выездным или домашним работникам (например, менеджерам по продажам, программистам, дизайнерам, журналистам, переводчикам). Во-вторых, не нужно приобретать дополнительно мебель, компьютер и т.д. В-третьих, увеличивается эффективность работы компании ее филиалов, что достигается за счёт автоматизации многих бизнес-процессов. Таким образом, удаленный доступ — это не роскошь, а еще один компонент успешного ведения современного бизнеса.

Организация системы удаленного доступа – задача не из лёгких. Конечно, можно обойтись и телефонными переговорами, отвлекая остальных сотрудников и руководство от выполнения их собственной работы. Получить необходимые данные вовремя и быстро принять решение — значит успешно вести свой бизнес.

Единого рецепта построения, который подходил бы абсолютно всем клиентам от небольшой фирмы, в которой требуется организовать доступ для нескольких человек, до крупной корпорации, где удаленно работают несколько тысяч сотрудников, не существует. Сколько пользователей – столько и решений. Одним из основных факторов, влияющим на выбор, следует назвать размер компании. Исходя из её потребностей и возможностей, определяется «прожиточный минимум» удаленного пользователя, то есть пакет необходимых ему программно–аппаратных средств. Теперь дело за малым: «подобрать» такое оборудование и методику организации удаленного доступа, которые решали бы данные задачи, а главное, без переплаты за массу других ненужных функций.

В то же время не следует забывать о том, что данные в подобных системах передаются на большое расстояние и, часто, по сетям общего пользования, доступ к которым не ограничен, что определяет их возможную уязвимость.

Актуальность безопасности передаваемых данных и их защиты от несанкционированного доступа в системах удаленного доступа требует особенно пристального внимания и применения специальных методик, которые гарантировали бы достаточную конфиденциальность и достоверность данных. В противном случае, хакер, подключившийся в качестве удаленного клиента или перехватывая трафик реально существующего клиента, сможет похитить данные, являющиеся государственной, военной, промышленной, банковской или коммерческой тайной, или сфальсифицировать их.

Задача, решаемая в данной работе, связана с решением обозначенной выше проблемы и может быть сформулирована следующим образом: проанализировать и исследовать существующие способы обеспечения безопасности работы в системах удаленного доступа, применяемых в России и конкретно в Ростовской области, и выработать рекомендации по выбору конкретного способа, используя новое, *импортзамещающее* программное обеспечение.

Цель работы — проанализировать и исследовать существующие способы обеспечения безопасности работы в системах удаленного доступа и реализовать программу по защите от несанкционированного доступа к системе.

Объектом исследования являются виды атак типа «Удаленный доступ».

Для достижения поставленной цели были поставлены *следующие задачи*:

- анализ доступных программ в сфере удаленного контроля ПК;
- исследование различных вирусов удаленного доступа;
- изучение применяемых методов и средств удаленного доступа;
- реализация *импортзамещающего* программного обеспечения для удаленного контроля ПК, включающую в себя *монитор защиты* против различных типов сетевых кибератак удалённого доступа.

1 АНАЛИЗ ПРОБЛЕМЫ

Удалённый доступ — очень широкое понятие, которое включает в себя различные типы и варианты взаимодействия компьютеров, сетей и приложений. Существует огромное количество схем взаимодействия, которые можно назвать удалённым доступом, но их объединяет использование глобальных каналов или глобальных сетей при взаимодействии. Кроме того, для удалённого доступа, как правило, характерна несимметричность взаимодействия, то есть, с одной стороны, имеется центральная крупная сеть или центральный компьютер, а с другой — отдельный удалённый терминал, компьютер или небольшая сеть, которые должны получить доступ к информационным ресурсам центральной сети.

За последние год – два количество предприятий, имеющих территориально распределённые корпоративные сети, значительно возросло. Поэтому для современных средств удалённого доступа очень важны свойства хорошей масштабируемости и поддержки большого количества удалённых клиентов.

Программы удалённого администрирования — программы или функции операционных систем, позволяющие получить удалённый доступ к компьютеру через Интернет или ЛВС и производить управление и администрирование удалённого компьютера в реальном времени. Программы удалённого администрирования предоставляют почти полный контроль над удалённым компьютером: они дают возможность удалённо управлять рабочим столом компьютера, возможность копирования или удаления файлов, запуска приложений, полный контроль над диспетчером задач и т.д. Данные программы могут быть как легальными, так и нелегальными.

Троян удалённого доступа (Remote Access Trojan) — это разновидность вредоносного программного обеспечения, которое позволяет хакеру удалённо получить контроль (отсюда и название) над компьютером [17].

Данные виды вирусов актуальны на сей день, так как проблема состоит в том, что они не идентифицируются как вирусы, и антивирусная программа блокировать их не будет. Эксперты компании Fidelis Cybersecurity изучили уязвимости в ПО, наиболее популярные у злоумышленников (на территории России) в первом квартале 2019 г. Хотя «ценность» уязвимостей быстро снижается сразу же после публикации *патчей* (программных «заплаток», фрагментов кода для внесения изменений в компьютерные файлы) для них, злоумышленники не отказываются от применения уязвимостей и ошибок в программах (*багов*) так быстро и сразу.

Фактически хакеры продолжают использовать уязвимости до тех пор, пока с их помощью можно взломать хоть сколь-нибудь интересующие их цели.

Согласно отчёту Fidelis Cybersecurity, многие из наиболее актуальных угроз первого квартала 2019 г. – это старые, хорошо известные *баги*, *патчи* для которых вышли еще несколько лет назад. Около трети всех обнаруженных в первом квартале проблем (эксплойты, уязвимости) оказались датированы ещё 2017 г. и ранее [18].

К примеру, самыми активными вредоносами на территории России были названы H-Worm (Houdini) и njRAT – два трояна удаленного доступа (RAT), известные с 2012 г. Более того, эксперты считают, что разработчики этих «вредоносных инструментов», вероятно, связаны друг с другом и работают вместе.

1.1 Методы и средства удаленного доступа

Под средствами удаленного доступа понимаются средства, необходимые для связи небольших локальных сетей и даже удалённых отдельных компьютеров с центральной локальной сетью предприятия. В качестве отдельных удалённых узлов могут также выступать банкоматы или кассовые аппараты, требующие доступ к центральной базе данных о легальных клиентах банка, пластиковые карточки которых необходимо авторизовать на месте. Банкоматы или кассовые аппараты обычно рассчитаны на взаимодействие с центральным компьютером по сети X.25, которая в свое время специально разрабатывалась как сеть для удалённого доступа неинтеллектуального терминального оборудования к центральному компьютеру.

Метод удаленного доступа – это метод, при котором пользователь удалённого терминала с помощью специального программного обеспечения подключается по глобальной сети к другому компьютеру, в качестве локального узла. Этот способ часто используется на мини-компьютерах, но мало распространен в ЛВС. Удалённое управление (Remote control) – это метод, который позволяет удалённому пользователю получить контроль над локальными компьютерами в локальной вычислительной сети корпорации (т.е. управлять одним из ПК в ЛВС). Скорость проведения сеанса и его возможности зависят от характеристик управляемого ПК, т.к. именно на нём выполняется обработка всех сетевых команд. Коды клавиш, нажимаемых на удалённом ПК, посылаются в управляемый ПК, а все изменения на экране управляемого ПК выводятся на экран удалённого ПК. Используемые файлы и прикладные программы не загружаются в удалённый ПК. Передача данных идёт через модем со скоростью 2400–57600 бит/с. Недостаток метода: для выполнения одной работы задействованы два ПК.

Метод удаленного узла (Remote node) основан на использовании сервера удалённого доступа, который служит своего рода «регулирующим» и позволяет отдельному удалённому ПК или ЛВС связываться с центральной ЛВС. Программное обеспечение удалённого ПК, реализующее функции удалённого узла, позволяет ему функционировать как полноценному пользователю ЛВС.

Таким программным обеспечением может быть NT Work Station. Как только связь установлена, телефонные линии становятся «прозрачными», и пользователь может работать со всеми ресурсами сети, как будто он сидит за компьютером, непосредственно подключенным к ЛВС.

Сервер удалённого доступа может быть реализован в виде модема со встроенным специальным ПО; либо быть сервером ЛВС, на котором выполняются программы удалённого узла. Желательно, чтобы на удалённом ПК, помимо сетевого системного ПО, находилось всё прикладное ПО, необходимое для сеанса связи: все выполняемые (*.exe) файлы и необходимые Windows–приложения. В противном случае необходимо будет передавать их с сетевого сервера на канал связи. Так как они имеют, как правило, большие объёмы данных, то такая передача ПО потребует значительных затрат времени. Удалённый узел, как и каждый полноценный пользователь ЛВС, имеет свой сетевой адрес. Сетевая операционная система преобразует сетевые пакеты, которые нужно передавать через модем, из формата протокола IP или IPX в формат, совместимый со стандартом последовательной передачи [21].

С появлением всё большего количества программ, поддерживающих архитектуру «клиент–сервер», усиливается тенденция на использование программного обеспечения для удалённых узлов, из-за того, что такие программы позволяют обрабатывать большие файлы данных на серверах локальной вычислительной сети, а на удалённый ПК передать только результаты обработки.

1.2 Виртуальные частные сети

Виртуальная частная сеть (VPN) представляет собой подключение типа «точка–точка» в частной или общедоступной сети, например, в Интернете. VPN–клиенты используют специальные TCP / IP–протоколы, называемые туннельными протоколами, обеспечивающие установление защищенного канала обмена данными между двумя компьютерами. С точки зрения взаимодействующих компьютеров между ними организуется выделенный канал типа «точка–точка», хотя в действительности, данные передаются через Интернет, как и любые другие пакеты.

При обычной реализации VPN клиент инициирует виртуальное подключение типа «точка–точка» к серверу удаленного доступа через Интернет. Сервер удаленного доступа отвечает на вызов, выполняет проверку подлинности вызывающей стороны и передает данные между VPN–клиентом и частной сетью организации.

Для эмуляции канала типа «точка–точка» к данным добавляется заголовок (выполняется инкапсуляция). Этот заголовок содержит сведения о маршрутизации, обеспечивающие прохождение данных по общей или публичной сети до конечной точки. Для эмуляции частного канала и сохранения конфиденциальности данные шифруются. Пример VPN–подключения представлен на рисунке 1.1.

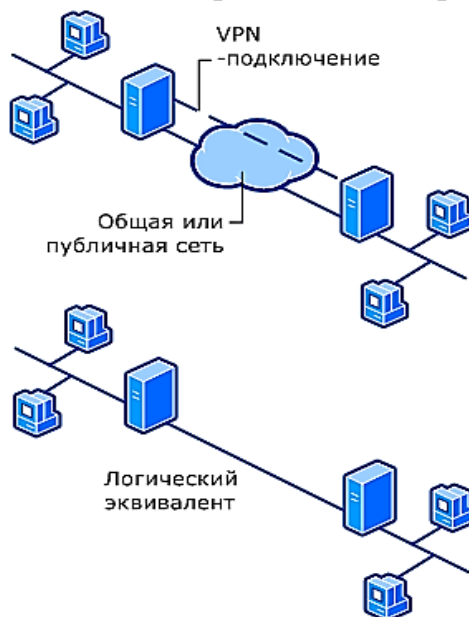


Рисунок 1.1 – VPN–подключение

1.3 Типы VPN–подключений

Существует два типа VPN–подключений.

VPN–подключение удалённого доступа дает пользователям возможность работать дома или в дороге, получая доступ к серверу частной сети посредством инфраструктуры общедоступной сети, например, Интернета. С точки зрения пользователя, VPN–подключение представляет собой подключение типа «точка–точка» между клиентским компьютером и сервером организации. Реальная инфраструктура общей или публичной сети не имеет значения, поскольку данные передаются так, как если бы они передавались по выделенному частному каналу.

VPN–подключение типа «сеть–сеть» (иногда называется VPN –подключением типа «маршрутизатор–маршрутизатор») предназначено для маршрутизации подключений между различными филиалами организации, а также между организациями через общедоступную сеть, обеспечивая защиту подключений.

Если сети соединены через Интернет (рисунок 1.2), то маршрутизатор, с поддержкой VPN, пересылает пакеты данных другому такому же маршрутизатору через VPN–подключение. С точки зрения маршрутизаторов, VPN–подключение на логическом уровне функционирует как выделенный канал уровня передачи данных.

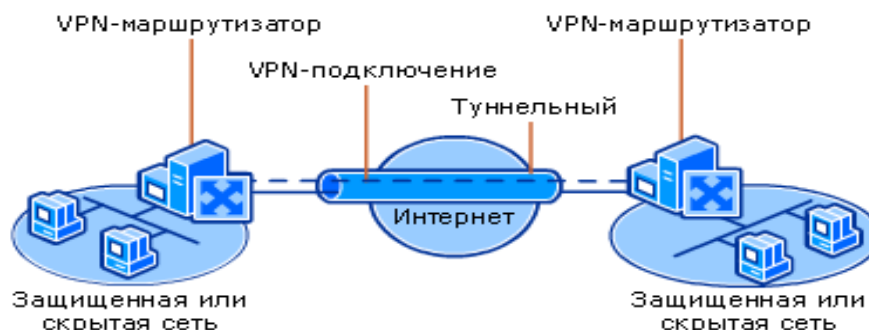


Рисунок 1.2 – VPN–подключение между двумя удаленными сайтами через Интернет

С помощью VPN–подключений можно соединить две частные сети. VPN–сервер обеспечивает маршрутизируемое подключение к сети, к которой присоединен VPN–сервер. Вызывающий маршрутизатор проходит проверку подлинности на отвечающем маршрутизаторе, и в целях взаимной проверки подлинности отвечающий маршрутизатор проходит проверку подлинности на вызывающем маршрутизаторе. При VPN–подключении типа «сеть–сеть» пакеты, отправляемые с любого из маршрутизаторов через VPN–подключение, обычно формируются не на маршрутизаторах. Схема VPN–подключения между двумя удаленными сайтами через Интернет представлена на рисунке 1.2.

Выводы

Данный раздел посвящен обзору ПО удалённого доступа, применяемых в настоящее время. Описаны программы удалённого администрирования, виды вредоносных программ, угрожающих безопасности данных, а также программно–аппаратные методы и средства удалённого доступа. Рассмотрена работа виртуальных частных сетей (VPN), а также типы VPN–подключений с точки зрения их уязвимости.

Исходя из проведенных исследований, необходимость иметь *суверенитет* Российской Федерации в сфере программного обеспечения удалённого доступа является актуальной и приоритетной задачей на сегодняшний день.

Решить эту задачу может помочь разработка новых алгоритмов и реализация *импортозамещающего* программного обеспечения, предназначенного для удалённого контроля компьютерной сети, включающую в себя *монитор защиты* против различных типов сетевых кибератак удалённого доступа.

2 ИССЛЕДОВАНИЕ ЛЕГАЛЬНЫХ И НЕЛЕГАЛЬНЫХ ПРОГРАММ ПО ПРЕДОСТАВЛЕНИЮ УДАЛЕННОГО ДОСТУПА

2.1 Программы удаленного администрирования

Программы удалённого администрирования (Remote Administration Tools, RAT) используются для удалённого управления рабочими станциями или другими компьютерными устройствами, и являются разновидностью потенциально опасного программного обеспечения. Через них можно выполнять практически любые действия с удаленной системой: передавать файлы, вести наблюдение за действиями пользователя, производить настройки системы, управлять функциями ввода / вывода и т.д. Чаще всего такие программы состоят из двух компонентов – серверной и клиентской частей на зараженном компьютере, предоставляющий доступ к системе. Существуют варианты, позволяющие управлять из браузера – для тех, кому необходимо иметь доступ с любого компьютера без предварительно установленных программ.

В легальных целях они используются для доступа к ресурсам другого компьютера (например, с домашнего компьютера можно управлять рабочим ПК), и для удаленной технической поддержки пользователей с низкой квалификацией в решении различных технических проблем.

Однако существует и третий, *нелегальный вариант* – удаленное подключение злоумышленника, с целью получения полного контроля над компьютером и всеми возможностями, которые этот контроль предоставляет. Однако только кражей информации и выводом компьютера из строя арсенал инструментов RAT–троянов не ограничивается: большинство таких программ предоставляет хакеру возможность «подсмотреть» в Web–камеру, прослушать данные из микрофона, узнать местоположение зараженного устройства (если на нём есть GPS), дают они и доступ к рабочему столу (RDP) и к командной строке. И это только минимальный набор возможностей RAT–троянов.

Хакеры используют RAT–трояны для слежки за человеком или сборе подробной информации о нем. При этом целью может стать кто угодно: известный политический деятель или обычный человек, о жизни которого кто–то хочет узнать подробней. Имея физический доступ к серверу, злоумышленник может украсть идентификатор и пароль для клиента, либо создать собственные. На машине клиента можно поработать в отсутствие владельца, либо, опять же, скопировать пароль доступа.

Другой тип атаки – заражение *трояном–кейлоггером* (это регистрирующее устройство, регистрирующее действия пользователя — нажатия клавиш на клавиатуре ПК, движения и нажатия клавиш мыши и т.д.), либо иной программой, крадущей пароли. Еще одним объектом воздействия может стать канал, по которому владелец управляемого компьютера передает пароль на подключение. Для передачи данных используют мобильные телефоны (SMS или устный разговор), электронную почту, Skype, любой из мессенджеров, и, если злоумышленник сумеет получить доступ к каналу передачи данных – он получит пароль и найдет возможность управлять чужим ПК.

2.2 Исследование легального ПО для удаленного контроля компьютера

Речь идет о *средствах удаленного администрирования* для Windows 10 и Windows 7, хотя многие из этих программ позволяют подключаться к удаленному рабочему столу на других ОС, в том числе с планшетов и смартфонов Android и iOS.

Для чего могут понадобиться такие программы? В большинстве случаев они используются для удалённого доступа к рабочему столу и действий для обслуживания компьютера системными администраторами и в сервисных целях. С точки зрения обычного пользователя, удалённое управление компьютером через Интернет или по локальной сети может быть полезным: например, вместо установки виртуальной машины с Windows на ноутбуке Linux или Mac можно подключаться к уже имеющемуся удалённому компьютеру с нужной операционной системой, и это лишь один из возможных сценариев.

В обновлении Windows 10 версии 1607 (август 2016) появилось новое, встроенное, очень простое приложение для удаленного рабочего стола — *Быстрая помощь*, которое подойдет для самых начинающих пользователей. Кратко об использовании программы: это удаленный доступ к рабочему столу в приложении «Быстрая помощь» (Quick Assist) Windows 10 (открывается в новой вкладке).

Удалённый рабочий стол Microsoft хорош тем, что для удалённого доступа к компьютеру с его помощью не требуется установки какого-либо дополнительного ПО. При этом протокол RDP, который используется при доступе, в достаточной мере защищён и работает надёжно. Скриншот программы показан на рисунке 2.1.

Но есть и недостатки. В то время, как подключиться к удалённому рабочему столу можно, и не устанавливая дополнительных программ всех версий Windows 7, 8 и Windows 10 (а также других ОС, в том числе Android и iOS, загрузив бесплатный клиент Microsoft Remote Desktop), в качестве сервера, к которому можно подключиться, может быть использован только компьютер или ноутбук с ОС Windows Pro и выше.

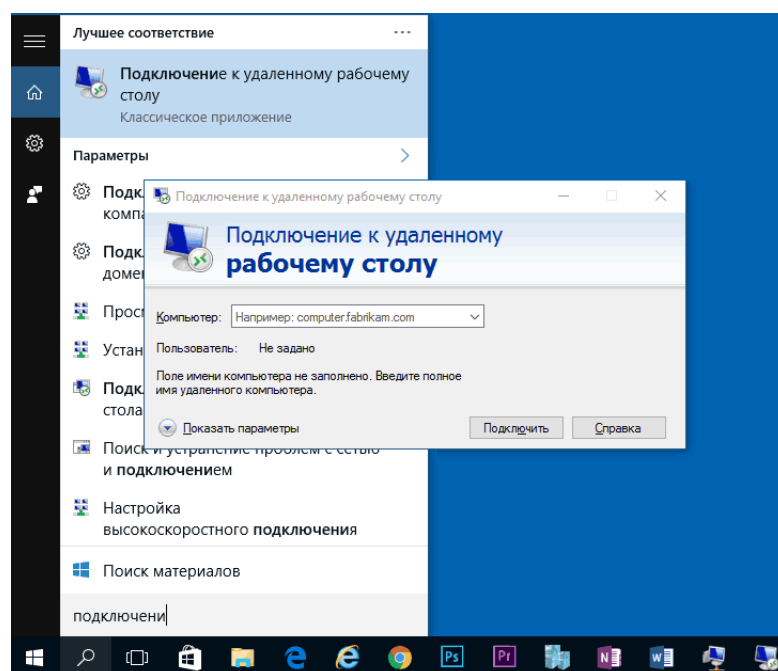


Рисунок 2.1 – Удаленный рабочий стол Microsoft (Microsoft Remote Desktop)

Ещё одно ограничение — без дополнительных настроек и изысканий, подключение к удаленному рабочему столу Microsoft работает только в том случае, если компьютеры и мобильные устройства находятся в одной локальной сети (например, подключены к одному роутеру в домашних условиях) или же имеют статические IP в Интернете (и находятся не за маршрутизаторами).

TeamViewer — самая известная программа для удалённого рабочего стола Windows и других ОС. Она проста в использовании, функциональна, работает через Интернет и бесплатна для частного использования. Кроме этого, может работать без установки на компьютер, что полезно, если вам требуется лишь однократное подключение. Скриншот программы представлен на рисунке 2.2.

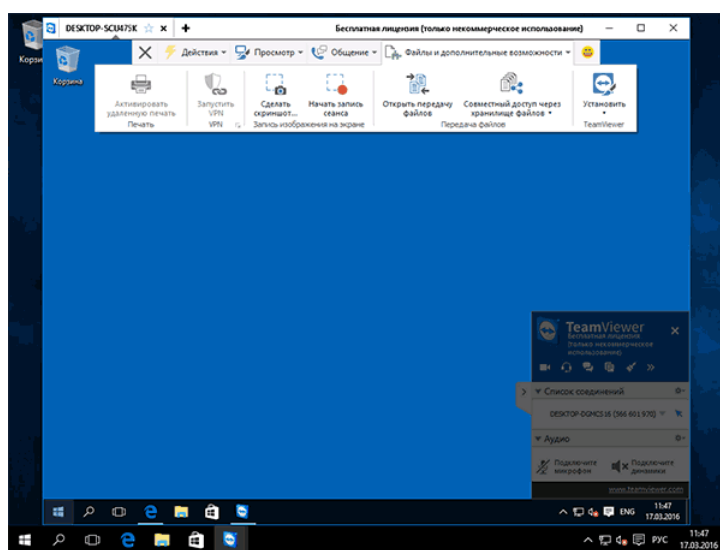


Рисунок 2.2 – TeamViewer

TeamViewer доступен и в виде «большой» программы для Windows 7, 8 и Windows 10, Mac и Linux, совмещающей в себе функции сервера и клиента и позволяющей настроить постоянный удаленный доступ к компьютеру, в виде модуля TeamViewer QuickSupport, не требующего установки, который сразу после запуска выдает ID и пароль, которые требуется ввести на компьютере, с которого будет выполняться подключение. Дополнительно есть вариант TeamViewer Host, для обеспечения возможности подключения к конкретному компьютеру в любое время. С недавних пор появился TeamViewer в виде приложения для Google Chrome; также есть приложения для iOS и Android.

Функции, доступные при сеансе удаленного управления ПК в TeamViewer:

- запуск VPN соединения с удаленным компьютером;
- возможность удаленной печати;
- создание скриншотов и запись удаленного рабочего стола;
- общий доступ к файлам или просто передача файлов;
- голосовой и текстовый чат, переписка, переключение сторон.

Также приложение TeamViewer поддерживает Wake-on-LAN, перезагрузку и автоматическое переподключение в безопасном режиме.

Недостатки – TeamViewer бесплатна для личного использования, однако, фактически бесплатных пользователей система распознает как «использующих ПО в коммерческих целях» и ограничивает длительность сессии в несколько минут. Разработчик на сайте предоставляет возможность снятия данной блокировки: пользователь должен заполнить форму на сайте с указанием описания своего некоммерческого использования для последующего рассмотрения службой поддержки, но даже после получения уведомления о снятии блокировки, фактически, компьютер пользователя все равно заблокирован, что может рассматриваться как выдавливание бесплатных пользователей.

Google имеет собственную реализацию удалённого рабочего стола, работающую как приложение для Google Chrome (доступ будет обеспечен не только к Google Chrome на удаленном компьютере, а ко всему рабочему столу). Поддерживаются все настольные ОС, на которые можно установить браузер Google Chrome. Для Android и iOS также имеются официальные клиенты в магазинах приложений. Скриншот программы представлен на рисунке 2.3.

Для использования Google Chrome Remote Desktop потребуется загрузить расширение браузера из официального магазина, задать данные для доступа (пин –код), а на другом компьютере — подключиться с использованием этого же расширения и указанного пин-кода. При этом для использования удалённого рабочего стола Google Chrome обязательно требуется войти в свой аккаунт Google (не обязательно один и тот же аккаунт на разных компьютерах).

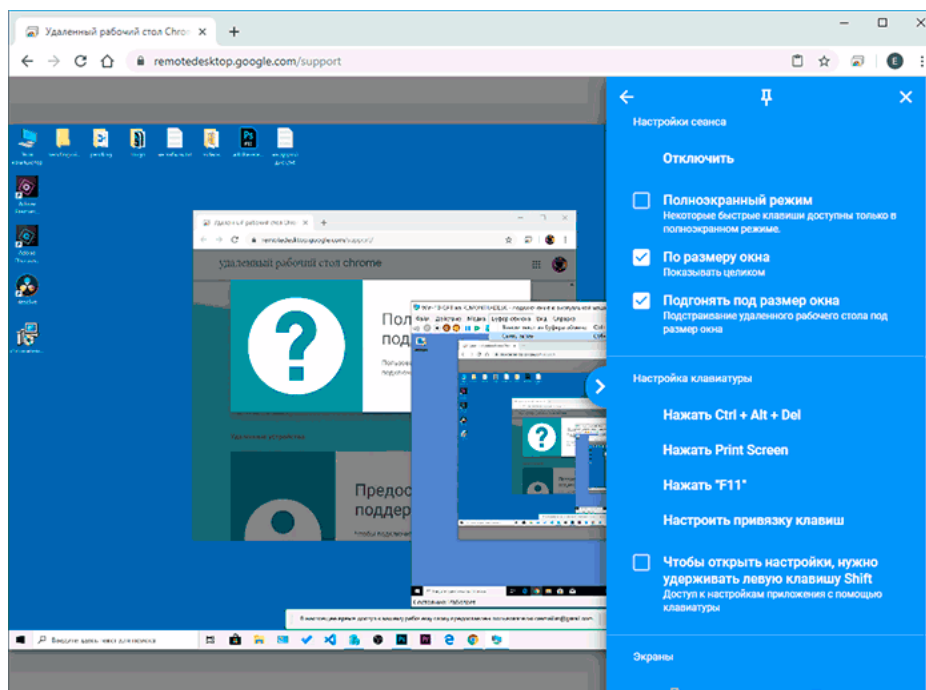


Рисунок 2.3 – Удаленный рабочий стол Google Chrome

Среди преимуществ способа — безопасность и отсутствие необходимости установки дополнительного ПО, если вы и так пользуетесь браузером Chrome. Из недостатков — ограниченная функциональность, и большая загрузка ЦП.

2.3 Теоретические сведения о трояках

Вирус типа «троян» — он же троянский конь — особый тип программного обеспечения, созданный для несанкционированного удалённого проникновения на компьютер пользователя. В отличие от обычного вируса, они не имеют функции размножения и дальнейшего распространения по сети. Однако троян открывает дверь для проникновения других вирусов, которые приготовлены хакером, троян создавшего или запустившего. По своему действию он напоминает принцип троянского коня, подаренного греками несдавшимся защитникам древнего города Трои. Та же судьба уготовлена и пользователю, который запускает на компьютере скачанное приложение. Нередко троян используется хакером для «зомбирования» компьютера и превращения его в одного из бесчисленных участников *ботнета* с целью проведения DDOS-атак на целевой сервер более крупной жертвы: Web-ресурса, работа которого кому-то очень сильно мешает [19].

Иногда заражённый трояном компьютер незаметно от пользователя передаёт из хранилищ системы сохранённые пароли, кейлоггеры с помощью открытых трояном портов передают информацию о набранных на клавиатуре сочетаниях клавиш, что отображает на экране злоумышленника всю информацию, которую печатала жертва атаки.

Хакеры нередко прибегают и к обычной уловке, когда спустя определённое время после «закладки» трояна они сканируют сеть на наличие компьютеров, где установка трояна для них удалась. Об этом будет свидетельствовать определённый открытый порт операционной системы. Действие трояна для жертвы атаки будет сопровождаться следующими последствиями [18, 19]:

- приведение операционной системы в нерабочее состояние, с помощью замены критических для системы файлов на другие, нужные хакеру файлы;
- повреждение данных;
- форматирование носителей данных (все типы дисков);
- внезапное переполнение жёсткого диска (диск чем-то забит);
- включение компьютера в состав ботнет для проведения DDOS атак;
- отключение антивируса и фаерволла с целью нанесения удара;
- кража паролей к электронным счётам;
- загрузка дополнительного деструктивного ПО;
- возможность подключения к монитору и Web-камере пользователя;
- запись происходящего на экране и аудиозаписи;
- модификация реестра;
- использование заражённого компьютера в качестве прокси с целью запутывания следов истинного компьютера, с которого и производится атака.

Что троян обычно ищет на вашем компьютере?

- информацию по кредитным картам;
- данные аккаунтов (социальные сети, электронная почта);
- специальные документы;
- расписания и другую календарную информацию.

Самые известные трояны для Windows:

1. *NetBus* – *утилита для удалённого контроллинга Windows*. Создана в 1998 г. шведским программистом Карлом-Фредриком Найктером на языке программирования Delphi. Создатель трояна заявлял, что этот троян изначально задумывался как программа-шутка. Однако его применение ознаменовалось резонансным скандалом, связанным с созданием целой базы «весёлых» картинок с участием несовершеннолетних, в результате чего некоторые высокопоставленные учёные мужи университета Лунда закончили свою карьеру.

2. *Back Orifice* – *утилита для удалённого контроллинга Windows*. Названием послужила игра слов от более серьёзного ПО Microsoft BackOffice Server от одноимённой корпорации. Работает по принципу клиент-сервер. Небольшая программка-сервер устанавливается на компьютер, затем сервер начинает общение с клиентской частью трояна через сетевые протоколы, работая с графическим интерфейсом или другой системой компьютера. Любимый порт, по которому сервер общается с клиентом в компьютере – 31337.

3. *Zeus – троян-червь, предназначенный для ОС семейства Windows.* Основное назначение – кража банковской информации у держателей карт с помощью перехвата набранных пользователем-жертвой данных в открытом сеансе браузера. На компьютеры жертв попадал через фишинговые сайты или несанкционированной загрузки и последующей установки ПО из сети. Антивирусом не определялся.

4. *Vundo – троян, вызывающий выскакивающие здесь и там окна с рекламным содержанием, нередко компрометирующего характера.* Использовал бреши в популярных *плагилах* к не менее популярным браузерам. Существует немало модификаций трояна, что вызывает определённые трудности в обнаружении и удалении Vundo. Нередко троян успешно борется с антивирусным и антишпионским программным обеспечением.

Некоторые типы троянов умело маскируются под обычные файлы, приклеиваясь к программам и используя уязвимости, которые до сих пор не прикрыты создателями ПО. Присутствие такого трояна ничем его не выдаёт, а действие для системы может быть фатальным – хакер сможет удалять и создавать на удалённом компьютере всё, что пожелает [18, 19].

2.4 Классификация троянов и их основные цели

Трояны на Command Shell. Их основное направление – предоставить контроль над командным процессором компьютера жертвы. Самый известный из этих типов троянов – *Netcat*. Это троян-вложение, маскирующийся в документе под файл-приложение. Троян вкладывается в документ, например, в документ PDF. Документ открывается жертвой – и троян запускается.

Почтовый троян. Хакеры получают контроль над компьютером жертвы после отправки сообщения. Фактически, никакие средства персональной защиты (например, настроенный фаервол) помешать им не смогут. Теперь хакеры, через почтовые команды, могут рыскать по папкам жертвы, прикрываясь официально запущенным процессом. Самый известный среди таких троянов – *RemoteByMail*.

Троян-искажатель. Суть проста: вы думаете, что запускаете утилиту Калькулятор от Windows (*calc.exe*) – но запускается троян, точь-в-точь похожий на калькулятор. В создании искажателей используется особый вид программ – редакторов, позволяющих просматривать, редактировать, извлекать и заменять целые цепочки в коде «родной» программы на нужные взломщику. Причём программа – троян может не иметь по функционалу ничего общего с настоящей, но благодаря возможности выудить с помощью редакторов иконки, символы Windows, от реальной утилиты – трояна не отличить: те же фреймы и диалоговые окна – всё натурально. Типичный представитель таких троянов – *Restorator*.

Трояны–ботнеты. В этом случае понятие *трояна* связано с понятием *вирус*. Причина этого в том, что троян заражает огромное количество компьютеров (чем больше, тем лучше, и здесь важно не столько количество ПК, сколько диапазон IP–адресов и количество под–сетей). Создаётся собственная сеть из заражённых компьютеров, в каждом из которых находится троян, ждущий команды из своего командного центра. В час *X*, по команде сверху, с вашего компьютера (или через него) пойдут DDoS–атаки на сайты, спам–рассылки по почтовым адресам и др. Самые известные среди ботнет–программ – Illusion Bot и NetBot Attacker.

Прокси – трояны. Этот вид троянов устанавливается на ПК жертвы с одной целью – использовать компьютер ничего не подозревающего владельца в качестве *прокси–сервера* (посредника между пользователем и сайтами), используя его IP и MAC адреса предающих устройств. Используя это оборудование, можно заразить тысячи машин, а их владельцы и подозревать об этом не будут. Самый известный представитель троянов в своём роде имеет название W3bProxу Troj4nCr34t0r.

FTP–трояны ставят на компьютер жертвы FTP–серверы. После установки в системе распахиваются FTP–порты. Теперь взломщик может скачать с вашего компьютера любой файл. Типичный представитель троянов: TinyFTPD.

VNC–троян. Системой идентифицируется в качестве отдельно запускаемого приложения, так что вполне вероятно, антивирус может на него внимания не обратить. Подключается в машине жертвы через просмотрщик VNC. И скрытым процессом запускается сервер. В числе их WinVNC и VNC Stealer.

HTTP(S)–трояны в идеальном их виде способны проникать сквозь фаервол системы и работать в противоположном направлении в прямом коридоре протокола передачи данных HTTP. Троян активирует некое приложение, которое определяется фаерволом системы как программа, запущенная от имени пользователя, и поэтому этот процесс происходит незаметно для пользователя. Типичный представитель троянов такого типа – HTTP Rat.

SHTTPD–трояны. SHTTPD – это маленький HTTP–сервер, упакованный хакером в программу, предлагаемую для скачивания. Как только вы её запустите, начнёт работать и троян, превращая ваш компьютер в незаметный для вас сервер.

ССТТ–трояны (Covert Channel Tunneling Tool) – это инструменты прокладки скрытого канала для передачи данных. Использует целый спектр техник *эксплойта*, создавая случайные каналы передачи данных в их общем потоке инициированного системой контроля доступа к сети. Это позволяет хакеру создать дополнительную серверную оболочку с доступом из внутренней сети (и наоборот), создавая каналы передачи данных по протоколам HTTP, TCP и UDP между внешним сервером и программным блоком внутренней сети.

Банковский троян. Слово «банковский» не является точным определением, а лишь выделяет трояны этого типа по общей схеме работы с жертвами. Троян не может быть всеобъемлющим, и на разных этапах обмана «подключает» различные техники и инструменты. Задача же банковских троянов очевидна – выудить финансовую информацию о жертве. Сложность их реализации состоит не столько в техническом исполнении, сколько в правильном применении техники социальной инженерии. Жертва не должна почуять подвох и забить тревогу раньше времени ни на одном из этапов взлома: от перехвата личных вводимых пользователем идентификационных данных и подделки онлайн страниц интернет-приложений банков до компрометирования форм обратной связи в текущем окне браузера жертвы. По такому же принципу работает всем известный Zeus.

Деструктивные трояны. Эти программы не воруют денег, но ущербом от заражения ими может послужить крах операционной системы в целом: от форматирования локальных и сетевых томов до затирания загрузочной записи Windows. Пакость таких троянов усугубляется элементарностью команд: они могут быть написаны на простых языках программирования типа VBScript, и не определяться антивирусом. Пример трояна, который способен проникнуть на компьютер, и, не обнаружив себя, предоставить хакеру полный доступ к системе, приведён в статье «Как написать троян на VBScript?»

Трояны – уведомители. Их цель – отослать хакеру расположение IP-адреса жертвы всякий раз, когда он выходит в сеть. Для этого используются различные каналы: ISQ, электронная почта, net команды и т.д.

Трояны–кредитки. Они могут входить в состав банковских карт, специализируясь на воровстве данных, содержащихся на титуле банковской карты: её номер, три цифры CVV2, деталей банковского счёта. Для этого применяются поддельные сайты, внешний облик которых напоминает Интернет-банк жертвы, созданный с одной лишь целью: заставить человека ввести нужную информацию о своей банковской карте в соответствующих полях таблицы.

Трояны – шпионы. Основная их задача – шпионство. Один из самых ярких представителей, модифицируемых от одного взломщика к другому, PhoneSnoop. Изначально был нацелен на коммуникаторы премиум класса семейства BlackBerry – одну из самых рекламируемых марок в США (в России мало известную). Принцип их действия таков: на коммуникатор, заражённый PhoneSnoop-ом, с определённого номера проходит звонок, троян автоматически поднимает трубку и сразу переходит на домашнюю страницу устройства, не вызывая подозрений хозяина. Хакер теперь может слышать всё происходящее.

2.5 Исследование RAT–троянов

Троян удаленного доступа, или RAT (Remote Access Trojan), является одним из самых вредоносных типов хакерских программ, которые только можно представить обычному пользователю. Они могут нанести всевозможные повреждения и быть ответственны за потери дорогостоящих данных. С ними нужно активно бороться, так как они широко распространены в России и очень опасны.

Рассмотрим, что они представляют собой, и как они функционируют, и что можно сделать обычному пользователю, чтобы защититься от них.

Ведь RAT можно рассматривать почти как кибероружие, и некоторые из них были разработаны и использованы для этих целей. В контексте компьютерного вредоносного программного обеспечения, троян представляет собой вредоносное ПО, распространяемое по сети. Что касается удалённого доступа RAT, то это связано с функционалом вредоносной программы. Он позволяет его разработчику иметь удалённый доступ к зараженному компьютеру. И когда злоумышленник получает такой удалённый доступ, у него практически нет ограничений на то, что он может сделать. Это может варьироваться от изучения вашей файловой системы, просмотра ваших действий на экране, сбора ваших учётных данных для входа в систему или шифрования ваших файлов, чтобы затем потребовать выкуп. Он также может украсть ваши данные или, что еще хуже, данные вашего клиента. После установки RAT ваш компьютер может стать концентратором, откуда атаки будут запущены на другие компьютеры в локальной сети, что позволит обойти любую защиту периметра.

К сожалению, RAT–трояны существуют уже более десяти лет. Считается, что эта технология сыграла свою роль в широкомасштабном разграблении американских технологий китайскими хакерами еще в 2003 г. Расследование в Пентагоне выявило кражу данных у американских оборонных подрядчиков, причём секретные данные о разработке и испытаниях были переданы в места расположения в Китае. Возможно, вы помните, как отключались энергосистемы на восточном побережье США в 2003 и 2008 годах. Они также были прослежены в Китае и, по-видимому, им способствовали RAT–трояны.

Хакер, который может запустить RAT в компьютерную систему, может воспользоваться любым программным обеспечением, которое есть в распоряжении пользователей зараженной системы, которые даже не заметят этого.

Злонамеренный разработчик RAT может взять под контроль все объекты противника: электростанции, телефонные сети, ядерные объекты или газопроводы.

Таким образом, RAT не только представляют угрозу для корпоративной безопасности. Эти трояны могут позволить специальным подразделениям и связанным с ними организациям атаковать противника в случае кибервойны. Их можно рассматривать как кибероружие. Хакеры по всему миру используют RAT для слежки за интересующими их компаниями, а также кражи их данных и денег.

Между тем проблема борьбы с атаками RAT теперь стала вопросом национальной безопасности для многих стран, в том числе России. Вот почему отражение этих атак с помощью отечественных импортозамещающих технологий стало столь важной задачей – это вопрос суверенитета российских IT-технологий.

Первоначально использовавшиеся для промышленного шпионажа и саботажа китайскими хакерами, сегодня RAT достаточно оценён в США, которые интегрировали их в свой военный арсенал. Теперь они являются частью стратегии наступления, известной как «гибридная война».

Рассмотрим некоторые из самых известных RAT-троянов [18, 19].

Back Orifice — это американский RAT, который существует с 1998 г. Это своего рода предок современных RAT-троянов. Первоначальная схема эксплуатировала уязвимость в Windows 98. Более поздние версии, которые работали в более новых операционных системах Windows, назывались *Back Orifice 2000* и *Deep Back Orifice*. Этот RAT способен скрывать себя в операционной системе, что делает его особенно трудным для обнаружения. Однако сегодня большинство систем защиты от вирусов используют в качестве сигнатур исполняемые файлы *Back Orifice* и поведение *окклюзии*. Отличительной особенностью этого программного обеспечения является то, что оно имеет простую в использовании консоль, которую злоумышленник может использовать для навигации и просмотра зараженной системы. После установки эта серверная программа связывается с клиентской консолью по стандартным сетевым протоколам. Например, известно, что используется номер порта 21337.

DarkComet был создан еще в 2008 г. французским хакером Жаном– Пьером Лесуэром, но привлек внимание сообщества кибербезопасности только в 2012 г., когда было обнаружено, что африканское хакерское подразделение использует эту систему для нацеливания на правительство и вооруженные силы США. *DarkComet* характеризуется простым в использовании интерфейсом, который позволяет пользователям, практически не имеющим технических навыков, выполнять хакерские атаки. Это позволяет шпионить через *кейлоггинг*, захват экрана и сбор пароля. Управляющий хакер также может управлять функциями питания удаленного компьютера, позволяя включать или выключать компьютер удаленно.

Сетевые функции зараженного компьютера также можно использовать для использования компьютера в качестве прокси-сервера и маскировки его личности во время рейдов на других компьютерах. Проект DarkComet был заброшен его разработчиком в 2014 г., когда было обнаружено, что он используется сирийским правительством, чтобы шпионить за его гражданами.

Mirage — известный RAT-троян, используемый спонсируемой государством китайской хакерской группой. После очень активной шпионской кампании с 2009 по 2015 год группа замолчала. *Mirage* был основным инструментом группы с 2012 года. Обнаружение варианта *Mirage*, названного *MirageFox* в 2018 г., является намеком на то, что группа может вернуться в действие.

MirageFox был обнаружен в марте 2018 г., когда он использовался для слежки за правительственными подрядчиками Великобритании. Что касается оригинальной версии *Mirage* RAT, она использовалась для атак на нефтяную компанию на Филиппинах, тайваньских военных, канадскую энергетическую компанию и другие цели в Бразилии, Израиле, Нигерии и Египте. Этот RAT поставляется встроенным в PDF. Открытие его приводит к выполнению скриптов, которые устанавливают RAT-трояны. После установки его первое действие — отчитаться перед системой управления и контроля с проверкой возможностей зараженной системы. Эта информация включает в себя скорость процессора, объем памяти и использование, имя системы и имя пользователя.

2.6 Троян NJRAT

NJRAT (*Remote Access Trojan, Backdoor.Ratenjay*), также известный как *Bladabindi*, является троянской программой удаленного доступа, которая позволяет владельцу программы контролировать зараженный компьютер конечного пользователя. Впервые он был обнаружен в июне 2013 г. с некоторыми вариантами, прослеженными до ноября 2012 г. Он был создан хакерской организацией из разных стран под названием *Sparclyheason* и часто использовался против различных целей (противника) на Ближнем Востоке. Он может быть распространен через фишинг и зараженные диски. Центр Microsoft по защите от вредоносных программ оценивает его как «серьезную угрозу» [21].

В июле 2014 г. в Индии был зарегистрирован всплеск атак *njRAT*. Пытаясь отключить возможности *njRAT*, в 2014 г. Microsoft закрыла четыре миллиона Web-сайтов, пытаясь отфильтровать трафик через домены *no-ip.com*.

В марте 2016 г. Softpedia сообщила, что спам–кампании, распространяющие трояны удаленного доступа, такие как njRAT, были направлены на Discord (программное обеспечение среди молодежи для общения). В октябре 2016 г. Softpedia сообщила о появлении взломанной загрузки VMware, которая будет загружать njRAT через Pastebin. Завершение процесса приведет к сбою компьютера.

Web–сайт Исламского государства был взломан в марте 2017 г. для показа поддельной загрузки обновления Adobe Flash Player, которая вместо этого загрузила трояна njRAT. Скриншот главного меню представлен на рисунке 2.4.

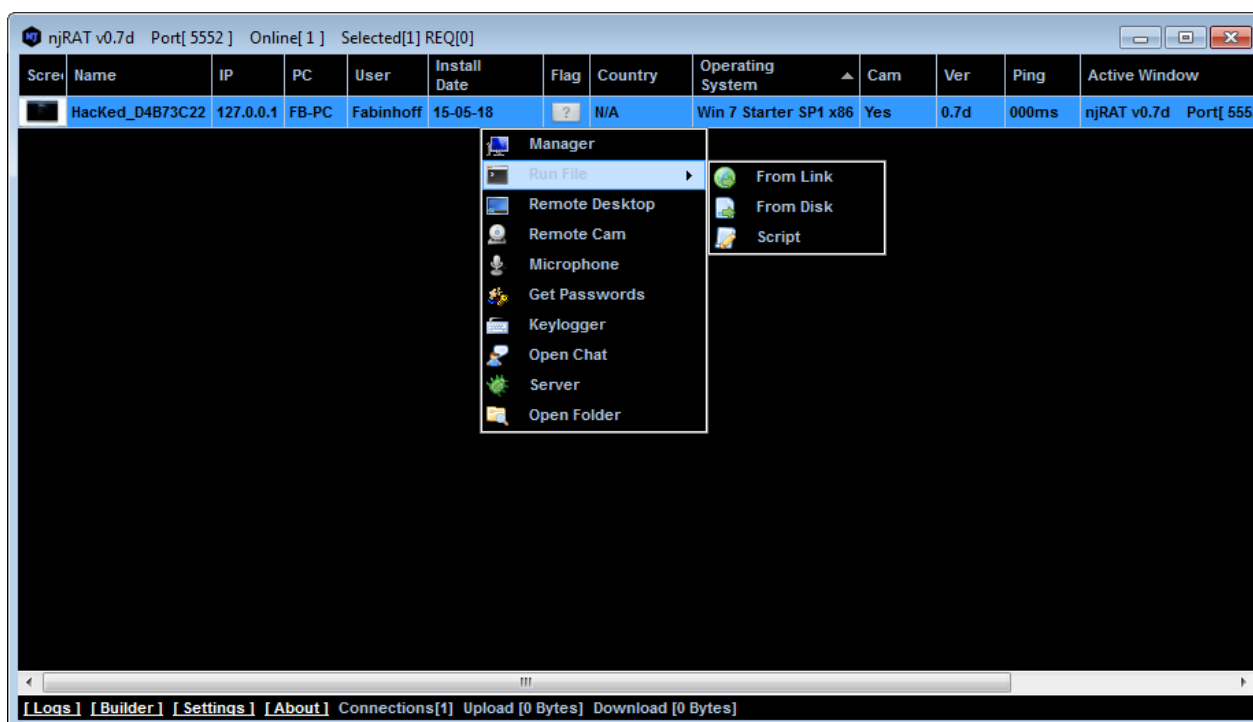


Рисунок 2.4 – Главное меню вируса NJRAT

Вот только некоторые функции njRAT:

- собирает информацию обо всех компонентах компьютера, обо всей системе и об открытых окнах во время первоначального заражения;
- может запустить интерфейс командной оболочки для выполнения команд;
- есть модуль, похищающий пароли, сохраненные в Web–браузерах жертвы;
- изменяет брандмауэр Windows, чтобы позволить ему общаться через него;
- может просматривать файловые системы, используя модуль файлового менеджера и качать либо загружать их без видимости для жертвы;
- способен удалять файлы на компьютере жертвы;
- способен регистрировать нажатия клавиш;
- может создавать, удалять или изменять ключи или значение реестра;
- определяет, есть ли у системы жертвы камера во время первоначального заражения, а также может её включать и выключать;

- добавляет постоянство через раздел реестра:
HKCU \ Software \ Microsoft \ CurrentVersion \ Run;
- может загружать и скачивать файлы с компьютера жертвы;
- может быть настроен для распространения через съёмные диски.

Проблема этого трояна состоит в том, что у него открытый исходный код, и он постоянно модифицируется. По данным Zscaler, новая итерация, именуемая njRAT Lime Edition, способна выполнять ранее не свойственные этим зловредам функции: шифровать файлы и требовать выкуп, отслеживать операции с криптовалютой, проводить DDoS-атаки типа Slowloris и ARME (Apache remote memory exhaustion).

Как показал анализ, обновленный njRAT пытается зашифровать файлы в Windows-папках LocalApplicationData, ApplicationData, ProgramFiles, Desktop, Favorites, MyMusic, MyPictures и Recent («Недавние документы»). Для этого на машине жертвы генерируется ключ AES-256; по окончании процесса, запущенного по команде, к имени зашифрованных файлов добавляется расширение .lime. Поскольку вредоносная программа использует симметричный алгоритм, расшифровка осуществляется тем же ключом; как оказалось, njRAT способен сам расшифровать все взломанные файлы.

Куплю–продажу биткойнов и соответствующие платежи троян отслеживает, используя список запущенных процессов. Обнаружив цель, вирус пытается с её помощью выявить криптокошелек, к которому жертва обращается для проведения транзакции. Поскольку такие кошельки хранят криптовалюту и могут быть привязаны к банковским картам или аккаунтам, то они представляют большой интерес для злоумышленников [22].

DDoS-атаки njRAT проводит, получив соответствующую команду с C&C-сервера. Маломощные атаки Slowloris нацелены на истощение ресурсов путем удержания открытых соединений на сервере. Такие неспешные DDoS-атаки уровня приложений можно проводить с одной – единственной машины. Целью удаленных атак ARME является истощение памяти на сервере Apache; в свое время авторы подобных атак применяли эксплойт к уязвимости CVE-2011-3192 в HTTP-серверах Apache, позволявший вызвать отказ в обслуживании.

Зловред njRAT Lime умеет также регистрировать нажатия клавиш, воровать пароли, удалять из Chrome куки-файлы и сохраненные учетные данные, отключать монитор, зачитывать вслух полученные с C&C тексты (с помощью функции TextToSpeech), активировать Диспетчер задач, менять обои на рабочем столе, скачивать и выкладывать файлы с помощью торрент-клиента. Кроме того, он способен распространяться через USB-накопители, отслеживая их подключение к зараженной машине. Копируя себя на флэшку, RAT-троян создает короткий путь к этой копии и соответствующую иконку.

Для обмена с центром управления njRAT Lime использует динамический DNS-сервис и кастомный протокол TCP. Этот зловред собирает и отправляет на C&C большой объем информации, куда входит имя системы, имя пользователя, версия Windows и используемая архитектура, наличие Web-камеры, текущее окно, ЦП, видеокарта, объем памяти, установленный антивирус, время заражения.

Новый вариант *njRAT* также снабжен богатым набором средств защиты от обнаружения и анализа. Он умеет отслеживать запуск в виртуальной машине и песочнице, используя стандартные запросы WMI. Помимо этого, зловред отслеживает и проводит мониторинг запущенных процессов и, обнаружив имя, грозящее ему раскрытием (Process Explorer, Wireshark, Taskmgr, Virustotal, SmartSniff и т. п.), пытается принудительно завершить этот процесс.

Выводы

Данный раздел посвящен исследованию легальных и нелегальных программ, предоставляющих удалённый доступ. Среди них программное обеспечение удалённого администрирования, удалённого контроля компьютера, а также различные троянские программы. Дана классификация троянов, исследован их вредоносный потенциал. Особое место уделено RAT-троянам удалённого доступа, а также NJRAT (*Backdoor.Ratenjay* или *Bladabindi*), которые могут рассматриваться как кибероружие гибридных кибервойн. Изучен их функционал, дающий злоумышленникам огромные возможности атаковать свои жертвы.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ МОНИТОРА ЗАЩИТЫ

Разрабатываемый *монитор защиты* и противодействия атакам будет состоять из следующих компонентов: клиентской части, серверной части, сервер сбора и обработки входящей информации. Графический интерфейс на стороне пользователя делать не целесообразно, так как программа будет работать в фоновом режиме. На стороне Сервера будет Администраторская панель, разработанная в виде Web-страницы. Такой подход обеспечит максимальный охват поддерживаемых устройств и не будет привязан только лишь к одному компьютеру. Очевидно, для разработки такого интерфейса необходимо использовать язык программирования JavaScript, язык разметки гипертекста HTML, а также таблицы стилей CSS. Так же сама серверная часть будет написана на языке программирования C#. Этот язык программирования был выбран из-за того, что в последнее время его много улучшают, средний порог вхождения, а также большое количество синтаксического сахара, что упрощает разработку. Принципиальная схема работы приложения указана на рисунке 3.1.



Рисунок 3.1 – Схема работы программы

3.1 Основные функции разработанного ПО

Данная программная реализации будет иметь в себе функции, которые помогут защитить информационную систему от злоумышленников. Один из методов, которым будет пользоваться данная программа, является обнаружение, основанное на сигнатурах. То есть программа, просматривая файл или пакет, будет обращаться к словарю с известными вирусами.

В случае соответствия какого-либо участка кода просматриваемой программы известному коду (сигнатуре) вируса в словаре, программа будет выполнять определенные действия. А именно программа может заморозить файл, удалить его либо отправить на рассмотрение. Для достижения достаточно продолжительного успеха при использовании этого метода необходимо периодически пополнять словарь известных вирусов новыми определениями. Для этого была использована база данных вирусов VirusTotal. Это одна из самых больших онлайн баз данных вирусов. При загрузке файла на сайт, он проверяет его на многих известных антивирусах. Пример представлен на рисунке 3.2.

52 / 69

52 engines detected this file

b78fb092e151db613cba51d7f2532547e48c6f4712809a485f272e2ab55776a5
NJRat 0.7D.exe

8.54 MB Size | 2019-12-17 02:06:35 UTC 2 days ago

assembly peexe

DETECTION	DETAILS	RELATIONS	BEHAVIOR	COMMUNITY
Ad-Aware	① Gen:Variant.MSILPerseus.114755	AhnLab-V3	① Trojan.Win32.NJRat.C2464784	
Alibaba	① Backdoor.MSIL/Bladabindi.3dd9884d	ALYac	① Gen:Variant.MSILPerseus.114755	
Antiy-AVL	① Trojan.Win32.TSGeneric	Arcabit	① Trojan.MSILPerseus.D1C043	
Avast	① Win32.Malware-gen	AVG	① Win32.Malware-gen	
BitDefender	① Gen:Variant.MSILPerseus.114755	BitDefenderTheta	① Gen:NN.ZemsiIF.33550.@p0@aOnRmpe	
CAT-QuickHeal	① Trojan.MsilperseusFC.S7086142	ClamAV	① Win.Malware.Daqc-6598298-0	
Comodo	① Malware@#1ftda4iuevg4x	CrowdStrike Falcon	① Win/malicious_confidence_100% (W)	

Рисунок 3.2 – Скриншот работы сайта VirusTotal

Как мы видим, хеш-сумма многих известных вирусов уже занесена в базу данных, поэтому поиск составляет десятые доли секунды. Сайт VirusTotal представляет публичный *API* ключ (от *Application Programming Interface* – специально создаваемой совокупности номера и проверочного кода) для всех зарегистрированных пользователей и даёт доступ к ресурсам своей базы данных, без запуска их сайта. Так, API VirusTotal позволяет загружать и сканировать файлы или URL-адреса, получать доступ к завершённым отчётам о сканировании, и автоматически комментировать процессы без необходимости использования интерфейса Web-сайта. Другими словами, он позволяет вам создавать простые сценарии для доступа к информации, генерируемой VirusTotal [22].

Для использования API нужно зарегистрироваться в VirusTotal Community. Если у вас есть действительная учётная запись VirusTotal Community, то вы найдете свой личный ключ API в разделе личных настроек. Этот ключ – все, что нужно для использования VirusTotal API.

Хотя антивирусные программы, созданные на основе поиска соответствия определению вируса в словаре, при обычных обстоятельствах могут достаточно эффективно препятствовать вспышкам заражения компьютеров, авторы вирусов стараются держаться на полшага впереди таких программ–антивирусов, создавая «полиморфические» вирусы в которых некоторые части участки кода перезаписываются, модифицируются, шифруются или искажаются так, чтобы невозможно было обнаружить совпадение с определением в словаре вирусов. Именно поэтому, только сигнатурами данное ПО не ограничиться.

Системный реестр или реестр Windows – это база данных для хранения настроек операционной системы, установленных программ, параметров пользователя и оборудования. Все изменения, которые происходят на компьютере тут же записываются и отображаются в реестре. Если мы загрузили и установили какую–то программу или просто поменяли обои рабочего стола, то все эти изменения вносятся в реестр. Поэтому защита и мониторинг реестра является важной частью информационной безопасности системы.

В редакторе реестра Windows имеется 5 корневых разделов, а именно:

- HKEY_CLASSES_ROOT или HKCR;
- HKEY_CURRENT_USER или HKCU;
- HKEY_LOCAL_MACHINE или HKLM;
- HKEY_USERS или HKU;
- HKEY_CURRENT_CONFIG или HKCC.

Каждый из этих разделов хранит в себе различную информацию, например, в HKCR здесь сведения обеспечивают выполнение необходимой программы при открытии файла с использованием проводника. Раздел HKCU является корневым для данных конфигурации пользователя, вошедшего в систему в настоящий момент. Здесь хранятся папки пользователя, цвета экрана, параметры панели управления и т.д. В разделе HKLM содержатся параметры конфигурации, относящиеся к данному компьютеру (для всех пользователей). Наиболее интересным является подраздел Software, который включает в себя настройки всех установленных в системе приложений. В разделе HKU содержатся все активные загруженные профили пользователей компьютера. Раздел HKEY_CURRENT_USER является подразделом HKEY_USERS.

HKCC является кустом реестра, который является частью реестра Windows. Он не хранит никакой информации сам по себе, но вместо этого действует как указатель или ярлык для раздела реестра, в котором хранится информация об используемом в настоящее время профиле оборудования.

Чтобы показать насколько просто можно внести изменение в реестр, создадим на рабочем столе обычный текстовый файл и сохраним его с расширением.reg. Пример файла представлен на рисунке 3.3.

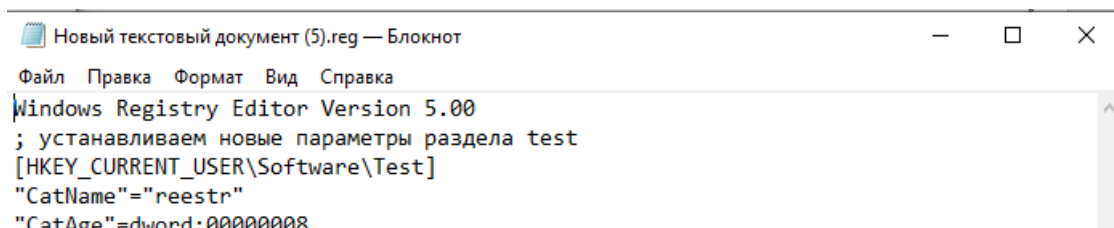


Рисунок 3.3 – Пример кода файла изменения реестра

Данный код создает в разделе HKEY_CURRENT_USER\Software\ папку test, в которой создает строковый параметр с названием CatName и значением reest, и параметр dword с названием CatAge и значением 8.

Многие вирусы, при попадании на компьютер, сразу проникают в реестр и меняют там нужные параметры. Так, RAT-троян NJRAT добавляет себя в автозагрузку с помощью пути HKCU\Software\Microsoft\CurrentVersion\Run. С помощью редактирования реестра можно отключить диспетчер задач. Если перейти по пути HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Policies и создать там подраздел System, в котором образовать новый параметр DWORD с названием DisableTaskMgr и значением 1, то при открытии диспетчера задач увидим надпись, что он отключен администратором. Пример показан на рисунке 3.4.

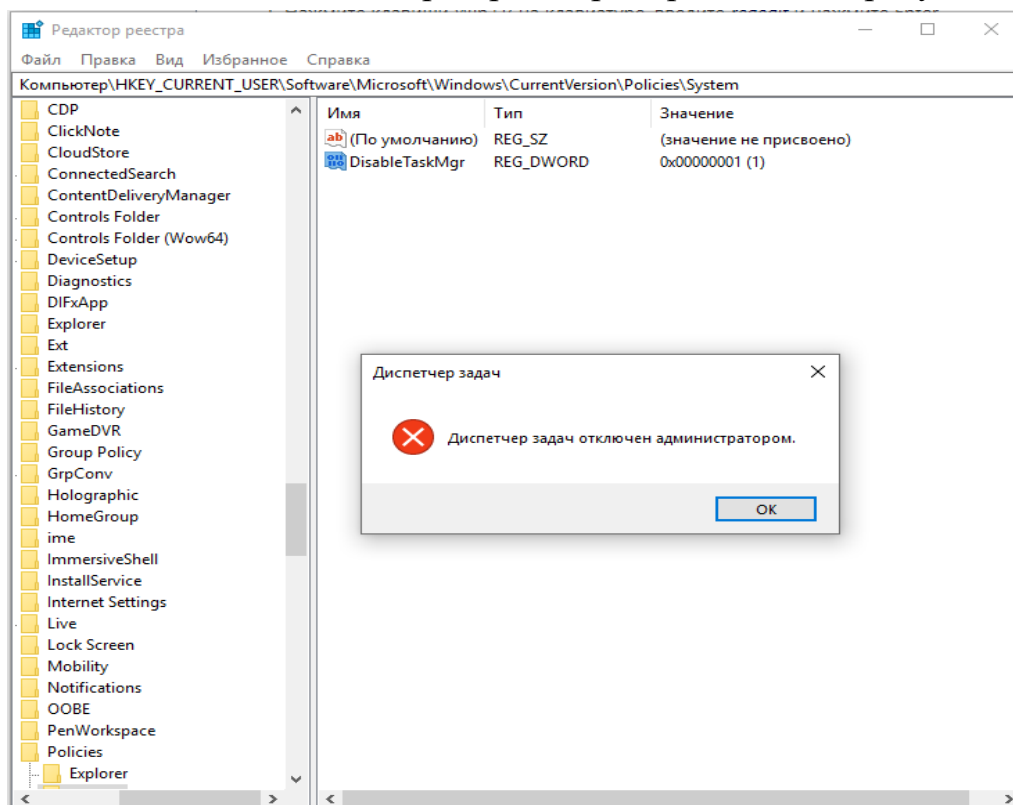


Рисунок 3.4 – Отключение диспетчера задач через реестр

Исходя из вышесказанного, был разработан алгоритм проверки разделов реестра. Алгоритм, при запуске ПК, делает отпечаток главных разделов реестра, которые часто используются вирусом, хеширует его и с некоторой периодичностью проверяет, были ли изменены данные раздела реестра (рисунок 3.5).

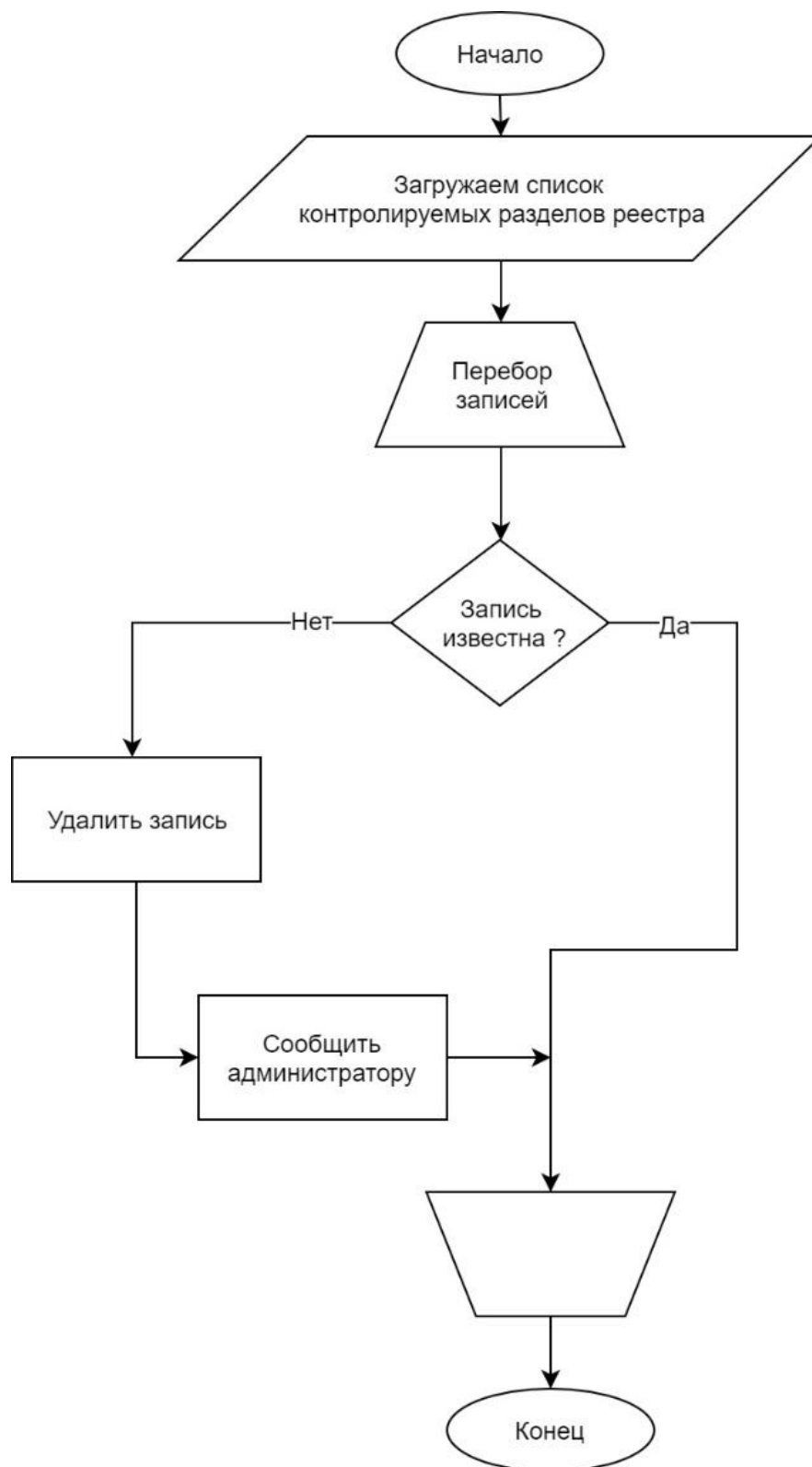


Рисунок 3.5 – Блок–схема проверки реестра

Например, такие как список доступных расширений, отключение диспетчера задач или других приложений. Если программа замечает подозрительные действия, то посылает оповещение администратору. Алгоритмом хеширования был выбран MD5. Шифрование MD5 — это способ хеширования, предполагающий создание оттисков или сумм для последующего процесса проверки их на подлинность. Таким образом, можно проверить, сохранилась ли информация в целостности. Данный алгоритм был выбран из-за того, что у него высокая чувствительность на изменения и относительно небольшой размер. Размер важен из-за того, что у нас клиент-серверное приложение, и для того, чтобы не тратить много трафика, был выбран данный алгоритм хеширования. Пример работы алгоритма и пример оповещения представлены на рисунке 3.6.

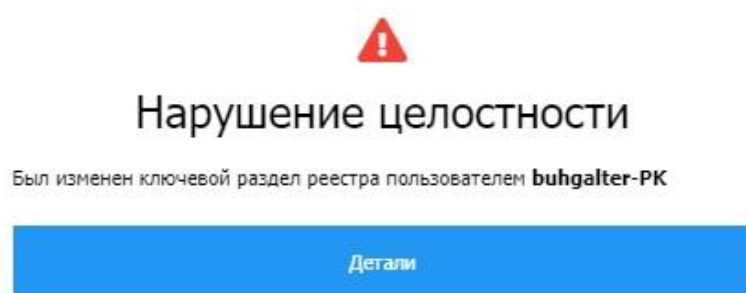


Рисунок 3.6 – Пример оповещения администратора об изменении раздела реестра

Еще одной функцией данной программы является проверка и мониторинг файлов автозагрузки. ПО будет проверять список автозагрузок на легальность использованных приложений. Будет происходить проверка и на электронную цифровую подпись. Пример работы данного алгоритма представлен на рисунке 3.7.

При запуске ПК программа будет загружаться и проверять реестр, проверять автозагрузку и сканировать систему. Если замечается некоторая подозрительная активность, то уведомление тут же отправляется администратору сети.

Еще одной функцией будет запрет на доступ к консольной утилите diskpart. Это утилита для управления разделами жестких дисков. Утилита является мощным, однако опасным инструментом для работы с жёсткими дисками, что требует определенных познаний, способностей и понимания того, что вы делаете. С помощью этой утилиты можно назначить букву скрытому системному разделу, изменить разделы диска, а также с помощью команды clean all можно очистить любой диск, при чем как поверхностно, то есть произойдет удаление разделов и информация будет помечена как удаленная, так и полностью, что значит удаление разделов и полное затирание информации, без возможности ее восстановить. Так что, если вирус захочет полностью затереть жесткий диск этой программой, это не составит большого труда.

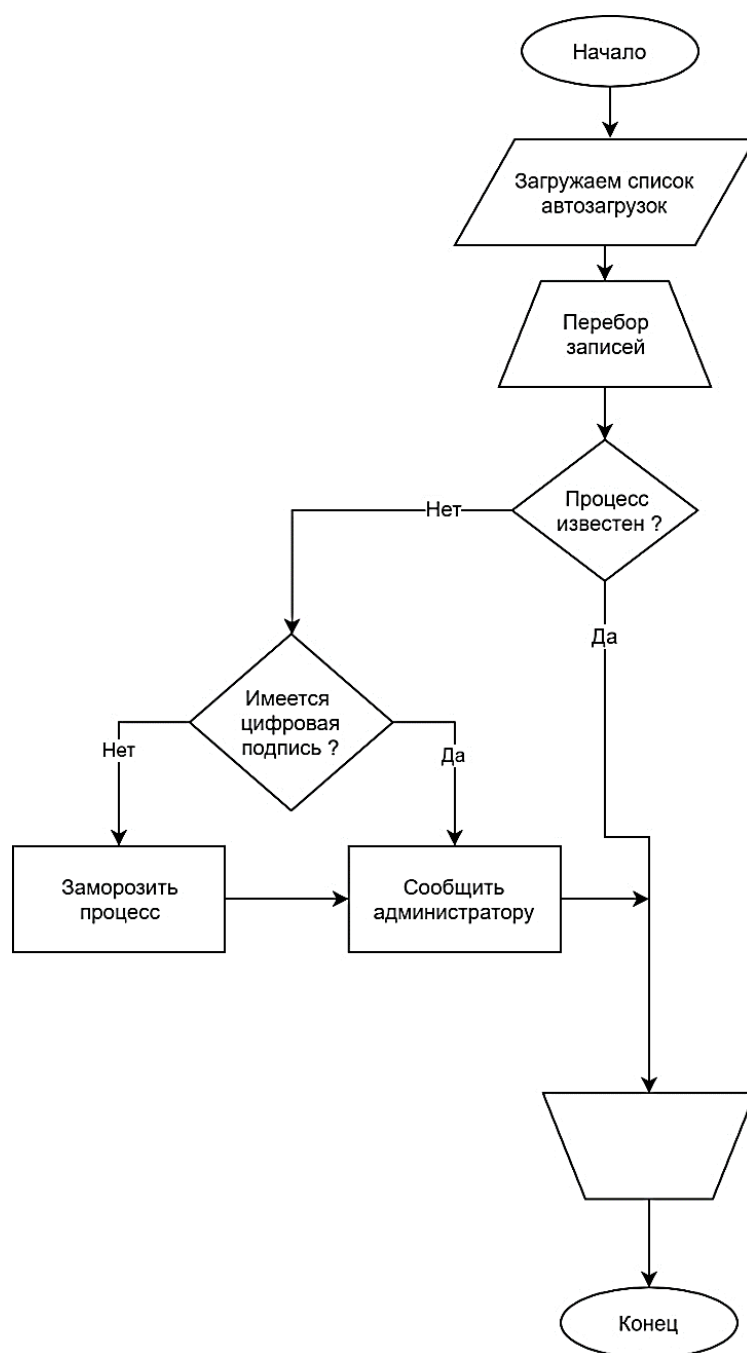


Рисунок 3.7 – Принцип работы проверки автозагрузки приложений

Данное ПО не представляет собой антивирус, оно предоставляет собой некий монитор, который предотвратит загрузку и запуск многих вирусов, например, таких как Retya, и ему подобных. Так как данная программа будет тут же замораживать неопознанный процесс, и держать его в таком состоянии до выяснения администратором функций этого процесса. Далее, многие вирусы используют в своем теле именно защиту от Антивируса, и обходят её, так как многие антивирусы известны и их можно разобрать. Разработанное программное обеспечение ещё не успело набрать популярность. Конечно, можно поставить на свой компьютер программу SecretNet, но это уже будет отдельная система, с которой надо учиться работать, а на многих предприятиях он просто не нужен.

Помимо этих средств защиты, в данном программном обеспечении будет осуществлен удаленный контроль, с наименованием компьютера, с просмотром операционной системы, доступными опциями, а также доступом к командной строке и процессам. Пример представлен на рисунке 3.8.

Список ПК			
Название	ОС	Опции	Операции
Олег-ПК	Windows 10	VPN 	Командная строка Процессы
Seven-TN	Windows 7		Командная строка Процессы
Бухгалтерия	Windows 10	VPN	Командная строка Процессы

Рисунок 3.8 – Список удаленных ПК

Администратор сможет в фоновом режиме подключаться к ПК пользователя и в полном объеме пользоваться командной строкой, как показано на рисунке 3.9.

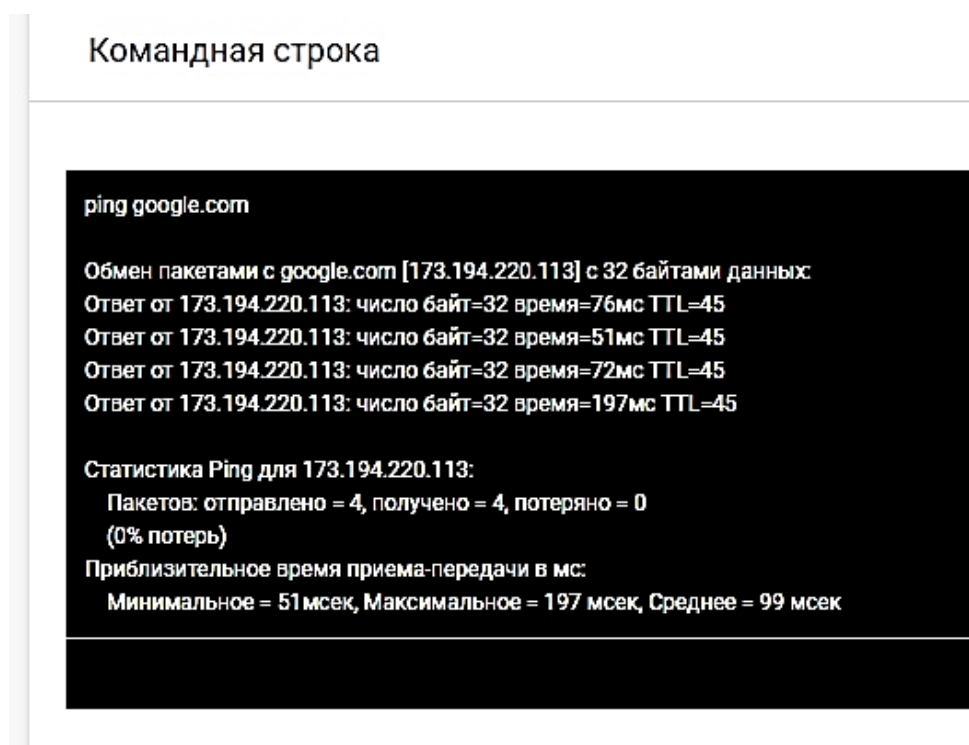


Рисунок 3.9 – Пример пользование командной строкой

Пример отображения процессов и пример отображение уведомлений можно увидеть на рисунках 3.10 и 3.11 соответственно.

Процессы			
Наименование	CPU	Память	Операции
Google Chrome	7.7%	1056 МБ	⏸ ⏴ ⏵
paint.net	0.1%	3.1 МБ	⏸ ⏴ ⏵
Visual Studio	2%	215 МБ	⏸ ⏴ ⏵
uTorrent	0%	10 МБ	⏸ ⏴ ⏵
Apache HTTP Server	0.2%	0.1 МБ	⏸ ⏴ ⏵

Рисунок 3.10 – Список запущенных процессов на ПК пользователя

Монитор		
Событие	Примечание	Дата
Нарушение HASH 	Изменился ключевой раздел реестра Подробнее	18.12.2019 11:58:44
Вирусная активность 	Обнаружен вредоносный файл / перемещен в карантин Подробнее	17.12.2019 12:42:15

Рисунок 3.11 – Пример уведомлений на панели администратора

Выводы

В данном разделе представлены алгоритмы реализации программного средства. По итогам изучения разработанных ранее программных реализаций, были намечены некоторые полезные функциональные дополнения, реализация которых сделает приложение более конкурентоспособным, а процесс мониторинга – удобным и незаметным. Основные функции данной программной реализации:

- работа приложения в фоновом режиме;
- запрет на использование некоторых системных программ;
- проверка автозагрузки и регистра реестров;
- возможность замораживания процессов либо удаление файлов;
- возможность использование удаленно командной строки;
- возможность проверки запущенных процессов;
- автозапуск программы и первоначальная проверка ПК;
- заморозка подозрительных процессов и отправка отчёта администратору.

РЕЗУЛЬТАТЫ 3 ГЛАВЫ

В данном исследовании было реализовано программное обеспечение в виде монитора безопасности, представляющего собой реализацию алгоритма защиты информационной системы от несанкционированного доступа и атак удаленного доступа типа «троянский конь», что и является целью работы.

Был выполнен ряд следующих задач:

- проведение анализа существующих программ в сфере удаленного контроля ПК;
- изучение методов и средств удаленного доступа;
- программная реализация алгоритма защиты информационной системы от несанкционированного доступа;
- предоставление безопасного удаленного доступа к информационной системе, включающей в себя монитор защиты от атак.

Данное приложение является кроссплатформенным и имеет интуитивно понятный интерфейс. В основе разработанного приложения лежит собственный разработанный алгоритм выбора доверительных и не доверительных программ.

Для реализации и анализа разработанного программного обеспечения были задействованы свободные источники, однако они не дают подробного описания в виду секретности разработанных алгоритмов.

В работе для всех пользователей подробно описан сценарий их работы. Алгоритмы работы клиентской и серверной части имеют пошаговые инструкции для администратора и пользователей.

ГЛАВА 4

ИМПОРТОЗАМЕЩАЮЩЕЕ ПРОГРАММНОЕ СРЕДСТВО БЕЗВОЗВРАТНОГО ГАРАНТИРОВАННОГО УНИЧТОЖЕНИЯ КРИТИЧЕСКИ ВАЖНОЙ ИНФОРМАЦИИ ДЛЯ ПОЛЬЗОВАТЕЛЯ КОМПЬЮТЕРНОЙ СИСТЕМЫ

ОБЩИЕ ПОЛОЖЕНИЯ

«Кто владеет информацией, тот владеет миром» – Натан Ротшильд, основатель огромной банковской династии, сказал эти слова еще в первой половине 19-го века, и был прав. Более того, с тех пор актуальность этих слов неумолимо растет день ото дня, так как информация уже давно стала первым по значимости ресурсом.

В наше время страшно даже представить ситуацию, в которой хакерам–злоумышленникам по той или иной причине удаётся получить доступ к нашим персональным данным, таким, как электронная почта, банковские карты и операции с ними, фотографии и многое другое. Значимость этих вещей для современного человека столь велика, что последствия от их потери могут быть бесконечно печальными. К тому же, даже форматирование диска перед его продажей вовсе не является гарантом того, что ваша ценная информация не попадет в руки третьих лиц.

Процедура удаления файлов базовыми средствами ОС Windows приводит к тому, что удаляемые файлы перемещаются в корзину, а не безвозвратно исчезают с компьютера. В этом случае восстановление информации, якобы удаленной с компьютера, не становится какой-либо проблемой для опытного специалиста, и выполняется буквально за пару нажатий клавиш мыши. Очистка корзины также не даёт желаемого результата, так как новая информация будет записываться в свободные области дискового пространства, а не на место удаленных данных. Восстановление информации станет возможным путём использования специального программного обеспечения. Форматирование носителя информации также не решает проблему, так как в этом случае очищается лишь таблица размещения файлов.

Далеко не все пользователи понимают, что удаление файла всё равно оставляет его содержание на жёстком диске. Байты, составляющие файлы, продолжают находиться на носителе, но при этом теряют связь с самой файловой системой.

Чтобы безвозвратно удалить данные, нужно в места хранения удаляемой информации записать новые данные. Пока этого не произойдёт, существует вероятность восстановления информации путём считывания носителя данных не через ОС, а с помощью специализированного программного обеспечения, коммерческих программ восстановления удалённых данных или другими методами.

Цель работы – изучение разновидностей способов гарантированного уничтожения информации, наиболее известных алгоритмов и программных реализаций в этой области, а также разработка импортозамещающего аналога, более функционального и безопасного программного средства гарантированного и безвозвратного уничтожения критически важной информации пользователя.

Объектом исследования являются алгоритмы и программные реализации средств гарантированного уничтожения информации (ГУИ).

Предмет исследования – области применения средств ГУИ.

Для достижения поставленной цели были поставлены *следующие задачи*:

- изучение понятия безвозвратного затирания данных;
- проведение анализа сложности и обзор подходов к безвозвратному гарантированному уничтожению критически важной информации пользователя;
- анализ существующих алгоритмов безвозвратного затирания данных;
- обзор наиболее известных программных реализаций средств ГУИ;
- описание разработанного импортозамещающего программного средства.

1 АНАЛИЗ ПРОБЛЕМЫ

С течением времени информационные технологии все более основательно проникают во все сферы деятельности не только всего человеческого общества в целом, но и во все уголки жизни отдельно взятой личности, спускаясь при этом на всё более низкий уровень. Если раньше человек был уверен в том, что принадлежащая ему информация находится под его защитой и полным контролем, находясь в руках правообладателя, то сейчас, сохраняя свою информацию на того или иного рода накопителе, пользователь ощущает все большее количество угроз её целостности. Ситуация осложняется тем, что личные персональные данные могут попасть в руки злоумышленника даже после её удаления с диска, так как информация о ней все еще остается на физическом носителе, а значит, эти данные могут быть кем-то восстановлены. Во избежание этого следует принять меры для уничтожения данных.

Такой процесс представляет собой определённый набор операций, направленных на безвозвратное затирание информации, которое выполняется программными или аппаратными средствами необратимого удаления данных.

Уничтожение данных — это упорядоченный набор операций, направленных на безвозвратное затирание информации, осуществляемое программными или аппаратными средствами необратимого удаления данных, в том числе и остаточных.

Обычно к подобного уровня удалению данных прибегают государственные учреждения, военные, промышленные, банковские структуры, различные коммерческие компании и др., в целях сохранения государственной, коммерческой, банковской, а также медицинской конфиденциальной информации [23].

В настоящее время разработано достаточно немалое количество программного обеспечения для безопасного и безвозвратного удаления данных.

Многие из них находятся в свободном доступе, а некоторые и с открытым исходным кодом. *Уничтожение данных* используется также в средствах программного шифрования информации для безопасного удаления временных файлов и уничтожения исходных данных, поскольку в противном случае, используя классическое удаление, существует возможность восстановления исходного файла лицом, желающим получить доступ к личной или секретной информации. Поэтому в подобных случаях пользователь, желающий уничтожить на носителе ту или иную информацию, должен иметь гарантии её полного уничтожения.

Затирание информации – важнейший компонент системы информационной безопасности. Твердотельные накопители (Solid State Drive, SSD), выполненные на основе электрически стираемых перепрограммируемых постоянных запоминающих устройств, заставляют усомниться в эффективности методов гарантированного уничтожения информации с накопителей на жёстких магнитных дисках – НЖМД.

В то же время, удаление отдельных файлов и полное затирание с них личной персональной информации изучены достаточно хорошо и в полном объёме.

В последние годы большой интерес у потребителей стали вызывать твердотельные накопители. Как правило, их внешний вид и разъёмы аналогичны обычному НЖМД, но при этом они обладают преимуществами по сравнению с традиционными магнитными дисками. SSD обладают большим быстродействием, они более лёгкие и компактные, потребляют меньше электроэнергии, обеспечивают произвольный доступ к любой области данных без дополнительных издержек, имеют меньшее время задержки обращения к данным, более устойчивы к неблагоприятным воздействиям окружающей среды (теплу, холоду, вибрациям, влажности, ударам), при эксплуатации не производят лишних колебаний, и при правильном использовании имеют весьма долгий срок службы.

Среди носителей информации в последние годы лидирующее место стали занимать накопители типа SSD по причине их высокого быстродействия, экономичности, надёжности и простоты использования.

В процессе развития информационных технологий и их проникновения в персональную жизнь человека, проблема возможности гарантированного удаления информации с носителей становится все более актуальной в рамках не только коммерческого, но и личного использования в том числе.

2 АНАЛИЗ СЛОЖНОСТИ И ОБЗОР ПОДХОДОВ К БЕЗВОЗВРАТНОМУ ГАРАНТИРОВАННОМУ УНИЧТОЖЕНИЮ ИНФОРМАЦИИ

2.1 Анализ сложности средств ГУИ

Обычно при удалении файлов на уровне файловой системы реального удаления не происходит, а этот адрес помечается как свободный, для дальнейшей записи в него информации. Исходя из этого, многие государственные стандарты и программное обеспечение гарантированного уничтожения информации прибегают к многократной перезаписи содержимого ячеек памяти для гарантированного затирания информации на накопителях. К настоящему времени так и не были опубликованы примеры восстановления данных с жёсткого магнитного диска, который подвергался очистке такого типа. Именно поэтому многие пользователи полагаются на программные методы уничтожения информации.

В случае твердотельных накопителей (Solid State Drive, SSD), при перезаписи методом FTL (Flash Translation Layer) будут вноситься новые данные в другие ячейки памяти на этом носителе, что обновит карту соответствия так, что последняя записанная информация появится по тому же логическому адресу. Для программного обеспечения, обращающегося в процессе работы к диску, разницы не существует, но на самом деле, по итогу этих операций, старая информация всё равно останется на микросхемах флэш-памяти. Помимо этого, алгоритмом самовосстановления будут генерироваться дополнительные копии данных.

В результате, алгоритмы уничтожения информации на основе их перезаписи, которые хорошо работают для жёстких дисков, являются неэффективными для накопителей SSD. Такие алгоритмы подразумевают, что перезаписывание данных в множестве логических адресов приводит к перезаписи тех же кластеров физического носителя, где находятся исходные данные. Перезапись данных на SSD влечет не физическое, а только логическое удаление, то есть данные становятся невозможно извлечь с помощью SATA или SCSI-интерфейса. Количество остаточной информации на накопителях SSD может быть довольно большим. Было протестировано в рабочих условиях несколько накопителей SSD, которые имели реальный физический объем на от 6 до 25% больше логического объёма. В течение исследования, на одном SSD было порядка 1000 мелких файлов. После этого диск был разобран, и запущен процесс поиска содержимого файлов на микросхемах памяти накопителя, в результате которого было найдено около 16 дубликатов некоторых файлов. Они были созданы контроллером SSD во время работы алгоритмов распределения нагрузки и самовосстановления.

Одни диски производят шифрование информации для увеличения производительности, другие, с той же целью, еще дополнительно сжимают информацию. Существуют оценки, согласно которым хорошо сжатая информация записывается на 25% быстрее, чем плохо поддающаяся компрессии. Это добавляет дополнительный уровень сложности, отсутствующий в НЖМД, так как усложняет оценку корректности проведенной процедуры удаления данных.

2.2 Обзор подходов к гарантированному уничтожению информации

Большинство современных накопителей имеет встроенные команды для запуска контроллером процедуры очистки накопителя. Поскольку производитель обладает полным знанием о внутреннем устройстве накопителя, эти встроенные команды должны быть очень надежны. Однако внедрение этих команд не является обязательным требованием стандарта изготовления таких накопителей.

Среди специфических команд ATA для гарантированного удаления информации существует команда «ERASE UNIT», которая затирает все доступные пользователю данные на накопителе путем записи во все ячейки единиц или нулей. Существует также расширенная команда «ERASE UNIT ENH», которая перезаписывает оставшиеся на накопителе данные с помощью специальной последовательности, определяемой производителем накопителя (предполагая, что изготовитель знает эффективную последовательность, затирающую остаточную информацию на аналоговом уровне лучше кого-либо другого).

Второй метод затирания данных предполагает использование обычных команд ввода-вывода для перезаписи каждого блока по его логическому адресу. Многократная программная перезапись лежит в основе очень многих стандартов и инструментов гарантированного удаления данных. Все стандарты и инструменты, о которых будет идти речь, используют схожий подход: они последовательно перезаписывают всё адресное пространство накопителя целиком, используя от 1 до 35 различных битовых последовательностей. Типичным примером является Руководящий документ Гостехкомиссии России по АС [24], предусматривающий ряд требований к механизму уничтожения информации для систем определенных классов защищенности. Так, для классов 3А и 2А очистка осуществляется двукратной произвольной записью в освобождаемую область памяти, ранее использованную для хранения защищаемых данных (файлов).

Размагничивание является быстрым и эффективным средством уничтожения информации на жёстких дисках (и самих дисков), так как разрушает низкоуровневое форматирование, вместе со всеми данными, и повреждает двигатель диска.

Технология хранения данных, используемая в микросхемах флэш-памяти, не основана на намагничивании, поэтому не следует ожидать прямого стирания данных с запоминающих элементов в результате процедуры размагничивания. Однако сильное переменное магнитное поле, излучаемое системой размагничивания, индуцирует мощные токи в металлических слоях микросхемы. Эти токи могут повредить микросхемы, сделав чтение данных с них невозможным [25].

Многие из недавно появившихся SSD-накопителей шифруют данные по умолчанию, так как это обеспечивает повышенную безопасность. Шифрование также обеспечивает быстрый способ очистки устройства, так как удаление ключа шифрования из хранилища ключей в теории делает невозможным чтение данных. Преимуществом этого подхода является скорость. Опасность же заключается в том, что метод полагается на правильное затирание контроллером накопителя внутреннего хранилища информации, содержащего ключ шифрования и все производные значения, которые могут быть полезны при криптоанализе. С учётом обнаруженных в некоторых реализациях команд безопасного удаления ошибок, было бы слишком оптимистичным полагаться на корректную очистку хранилища производителем. Кроме того, невозможно проверить, что удаление данных действительно произошло [26].

Существует гибридный подход, который может обеспечить как скорость, так и проверяемость гарантированного удаления информации. При применении этого подхода затирается хранилище ключей, а затем выполняется уничтожение каждого блока флэш-памяти. По окончании удаления накопитель переходит в *проверочное состояние*, когда его можно разобрать и убедиться, что удаление блоков прошло успешно, и информация уничтожена безвозвратно.

Затирание отдельных файлов необходимо для обеспечения безопасности данных в рабочих системах. Для систем типа персональных компьютеров, ноутбуков, планшетов и мобильных телефонов, в которых операционная система, программы и все пользовательские данные находятся на одном и том же накопителе SSD, затирание отдельных файлов является единственным вариантом удаления данных, не нарушающим рабочее состояние системы.

Удаление файла является более тонкой операцией, чем удаление всего адресного пространства. Эта операция требует удаления данных из одного или нескольких диапазонов логических адресов, оставляя прочее содержание накопителя без изменений. Ни для жёстких дисков, ни для SSD не определены специальные команды удаления конкретных областей.

Существует много программных утилит, осуществляющих удаление отдельных файлов. Все они используют тот же подход, что и программные инструменты полного стирания дисков, — многократно перезаписывают файл несколькими битовыми последовательностями, а затем удаляют его. Некоторые программы гарантированного удаления также многократно перезаписывают свободное пространство (не выделенное никакому файлу). В процессе обычного удаления информации с диска происходит лишь удаление адресов, ведущих к этой информации. Для решения этой проблемы принято производить многократную перезапись областей памяти с удаляемой и уничтожаемой информацией псевдослучайными последовательностями единиц и нулей.

Но, как показала практика, этот способ эффективен только для жёстких дисков НЖМД и не подходит для SSD, потому что это приводит лишь к логическому, а не физическому удалению информации. Среди наиболее эффективных методов гарантированного удаления выделяют *размагничивание*, так как оно является быстрым и эффективным средством уничтожения жёстких дисков, разрушающим низкоуровневое форматирование (вместе со всеми данными) и повреждает двигатель диска, а также использование встроенных команд для запуска контроллером процедуры очистки накопителя [27].

Выводы

В этом разделе проанализирована сложность средств гарантированного уничтожения информации, и сделан обзор подходов к решению этой задачи. Хотя существует достаточно много программных утилит, осуществляющих удаление отдельных файлов, методы их работы эффективны только для жёстких дисков НЖМД, но не подходит для SSD, так как они приводят только к логическому, а не физическому удалению информации. Среди методов гарантированного удаления информации выделяют размагничивание и использование встроенных команд для запуска контроллером процедуры очистки накопителя, что может повредить двигатель диска.

3 ГАРАНТИРОВАННОЕ УНИЧТОЖЕНИЕ ИНФОРМАЦИИ

3.1 Способы уничтожения данных

Есть несколько способов удалить файл с жёсткого диска наверняка, или сделать доступ к нему невозможным. Некоторые из способов предотвращения подобной утечки данных с жёсткого диска включают [26]:

- размагничивание диска;
- переписывание данных;
- физическое уничтожение носителя;
- кодировку информации.

Уничтожение важных данных на жёстком диске компьютера или с другого носителя – это лучший способ гарантировать, что ценные секретные или персональные конфиденциальные данные не смогут быть восстановлены даже в условиях лучших компьютерных лабораторий. Уничтожение информации имеет принципиально важное значение при перемещении компьютеров, на которых хранилась ценная информация, из безопасного места в менее надёжное помещение.

Ещё более эффективный подход к удалению данных – это размагничивание жёсткого диска. Данный метод выводит жёсткий диск из строя, и при его использовании больше нельзя будет применять такой НЖМД по назначению. Для этих целей, например, используется специальный прибор – Degausser.

Если же пользователь хочет удалить данные с жёстких дисков или других носителей, и при этом иметь гарантию того, что жёсткий диск будет пригоден к использованию, то следует воспользоваться *стиранием данных*. Это хорошее решение при повторном использовании компьютера, например, в другом отделе государственной структуры. Например, если в какой-либо организации компьютер передаётся новому сотруднику в другой отдел, то данные будут удалены, но компьютерные диски всё равно остаются в рабочем состоянии. Можно назвать наиболее распространенные программы по стиранию информации:

- Acronis Drive Cleanser;
- Secure Eraser;
- HDShredder;
- DBAN;
- Wise Disk Cleaner и т.д.

Другой метод избавления от данных представляет собой *переписывание всей памяти* на всех секциях жёсткого диска. Одной процедуры переписывания памяти будет вполне достаточно для полной уверенности в том, что предыдущая информация уничтожена безвозвратно, и никогда не сможет быть восстановлена.

Важно помнить, что простое удаление файла операционной системой всего лишь приводит к удалению пути к файлу, цифровая информация при простом удалении будет сохранена на жёстком диске. Даже повторное форматирование или разделение жёсткого диска на секции – это ещё не гарантия, что файлов не остается. Файлы сохраняются, даже если пользователь не может их увидеть [27].

В крайнем случае, если есть потребность в полном удалении файлов без возможности их восстановления, то физическое уничтожение носителя информации является наилучшим выбором. Носители информации можно уничтожить несколькими способами, включая непосредственное повреждение поверхности магнитного диска или барабана абразивными материалами. Коррозийные химикаты дадут те же результаты, при условии, что поверхность, на которую производится запись, будет полностью открыта для воздействия [26, 27].

3.2 Стандарты уничтожения информации

3.2.1 Стандарт ГОСТ Р 50739-95

Российский алгоритм ГОСТ Р 50739–95 является рекомендованным алгоритмом, проверенным временем. К достоинствам алгоритма можно отнести высокое быстродействие, а к недостаткам – тот факт, что этот алгоритм не предусматривает многократной записи.

Согласно ГОСТ Р 50739–95 определено, что стирание производится путем записи в неё маскирующей информации (Раздел 5.1.5 ГОСТ Р 50739–95), однако количество циклов и содержание маскирующей информации не указывается. Получается, что даже после одного цикла записи маскирующей информации на носитель, например, чередованием кодов нуля и единицы (0×55, 0×AA), остаточная намагниченность от прошлых данных остается (эффект остаточной информации) и её можно легко прочитать с помощью специального оборудования.

3.2.2 Стандарт DoD 5220.22–М

DoD 5220.22–М – это программный метод очистки данных, используемый в различных программах уничтожения файлов и данных для перезаписи существующей информации на жестком диске или другом устройстве хранения.

Стирание жёсткого диска с помощью метода очистки данных DoD 5220.22–М предотвратит возможность использования программных методов восстановления файлов и данных от подъёма информации с диска, а также устранил возможность применения большинства, если не всех, аппаратных методов восстановления.

Такой метод затирания работает следующим образом:

- 3 цикла перезаписи единицами;
- 3 цикла перезаписи нулями;
- цикл перезаписи псевдослучайной последовательностью.

Метод уничтожения DoD 5220.22–М был первоначально определен программой Национальной промышленной безопасности США, и является одним из наиболее распространенных методов стирания информации, используемых в программном обеспечении для уничтожения данных.

Как правило, не имеет большого значения, какой метод очистки данных использовать. Поскольку большинство пользователей стирают жёсткий диск только перед продажей диска или установкой новой операционной системы, то не имеет большого значения, сколько случайных символов записывается на диск.

Большинство пользователей, которые пытаются восстановить данные на приобретенном жёстком диске, вероятно, используют стандартные инструменты. Хотя они работают с помощью обнаружения удалённых данных, им обычно не удаётся справиться с правильным использованием метода очистки данных.

Однако при выборе метода очистки данных нужно решить, сколько времени потребуется для очистки диска. Если нужно очистить большой жёсткий диск, то запись нулей займет гораздо меньше времени, чем DoD 5220.22–М, который будет намного быстрее метода Гутмана, который совершит более 30 проходов. Кроме того, нужно учитывать, выполняются ли проверки после прохождения пропусков, поскольку некоторые программные средства могут реализовывать метод DoD 5220.22–М. Так, осуществляя проверку каждой записи в конце любого прохода, процесс займет гораздо больше времени, чем при использовании другого метода очистки, который вообще не проверяет результат уничтожения информации (например, безопасное стирание) или ждёт до самого конца последнего прохода, чтобы в конце концов проверить, что данные были перезаписаны.

Другой фактор, который может определить, какой метод стирания данных используется, – это фактические данные, используемые для перезаписи жёсткого диска. Некоторые методы очистки, такие как запись нуля, просто используют ноль вместо записи псевдослучайных символов. Возможно, использование генератора ПСЧ уменьшает вероятность того, что данные могут быть восстановлены.

Большинство программ уничтожения данных поддерживают несколько методов очистки данных в дополнение к DoD 5220.22–М, таких как безопасное стирание, запись нуля, случайные данные и метод Брюса Шнайера.

3.2.3 Алгоритм Брюса Шнайера

Метод Брюса Шнайера — алгоритм безопасного удаления данных с жёсткого диска компьютера. Эксперт с мировым именем в узких профессиональных кругах, он предложил следующий алгоритм из семи шагов [28]:

- заполнение единицами поверх имеющихся данных;
- заполнение нулями поверх имеющихся данных;
- запись псевдослучайных данных;
- запись псевдослучайных данных;
- запись псевдослучайных данных;
- запись псевдослучайных данных;
- запись псевдослучайных данных.

Нет, это не ошибка, эксперт действительно рекомендует *пять раз* заполнить НЖМД псевдослучайными данными. Хотя сам Брюс Шнайер впоследствии заявил: «Последние исследования с помощью туннельных микроскопов показали, что даже пяти раз может быть недостаточно». Так, может быть, десять раз?

Все программные методы безопасного удаления информации можно разделить на 3 уровня по степени надёжности полученного результата.

Нулевой уровень: вместо полной перезаписи в загрузочный сектор, в основную и резервную таблицу разделов записываются последовательности нулей.

Первый уровень: записываются последовательности нулей и единиц в тот сектор, где находится информация, подлежащая уничтожению.

На втором уровне перезапись информации происходит несколько раз, причём записываются не только нули и единицы, но и другие выбранные заранее или псевдослучайные последовательности значений.

Метод Шнайера относится ко второму уровню. После первых двух проходов (записи нулей и единиц) программный доступ к перезаписанным данным фактически невозможен, но тем не менее существует возможность и вероятность восстановления информации из-за наличия остаточной намагниченности краевых областей дорожек диска. Последние проходы записывают случайные (шумовые) коды, после чего магнитная структура диска над дорожками, где хранились данные, разрушается. Причём здесь перезапись повторяется несколько раз, что увеличивает вероятность перезаписи краевых дорожек. Чем большее количество раз происходит перезапись, тем лучше. При этом задача восстановления данных резко усложняется.

Недостатками данного метода является то, что:

- перезапись информации применяется только на исправном диске;
- надёжность уничтожения информации достаточно низкая;
- процесс перезаписи может занимать длительное время.

Положительные стороны также имеются, среди которых:

- простота реализации метода Брюса Шнайдера;
- возможность последующего использования носителей [29].

3.2.4 Алгоритм Питера Гутмана

Метод Гутмана — это алгоритм безопасного удаления данных (файлов) с жёсткого диска компьютера. Метод разработан Питером Гутманом и Коллином Пламбом. Метод состоит из 35 проходов, ориентированных на уничтожение записей, закодированных методами MFM и различными модификациями RLL.

Выбор проходов предполагает, что пользователь не знает механизм кодирования, используемый в жёстком диске, и потому включает в себя проходы, разработанные специально для трёх различных типов приводов. Если пользователь знает, какой тип кодировки использует привод, то он может выбрать только те проходы, которые предназначены для его диска. Для диска с различными механизмами кодирования требуются различные проходы.

Большинство проходов разработаны для жёстких дисков, закодированных по схемам MFM и RLL. В относительно современных дисках старые методы кодирования не используются, что делает многие проходы метода Гутмана лишними. Кроме того, примерно с 2001 года в конструкции жестких дисков ATA, IDE и SATA включена поддержка стандарта «Secure Erase», что устраняет необходимость применения метода Гутмана при очистке всего диска.

Состав перезаписываемой сессии:

- в процессе первых четырёх проходов записываются псевдослучайные последовательности (выбранные символы) в каждый байт каждого сектора;
- с 5 по 31 проход записывается псевдослучайная последовательность символов;
- в последние четыре прохода записываются псевдослучайные символы.

Каждый проход с 5 по 31 был разработан с учетом конкретной схемы магнитного кодирования, то есть как целевой проход. На диске записываются все дорожки, хотя таблица показывает только битовые проходы для дорожек, которые специально ориентированы на каждой схеме кодирования. Конечный результат должен скрывать любые данные на диске, так что только самые передовые технологии физического сканирования (например, с помощью магнитного силового микроскопа) привода, вероятно, смогут восстановить данные.

3.3 Программная очистка данных

3.3.1 Общая информация

Программная очистка данных – процесс необратимого удаления данных, сохранившихся в области, логически считающейся свободным пространством, с использованием особых методов удаления данных. Процесс происходит во время перезагрузки компьютера, после процесса загрузки операционной системы. Этот метод рекомендуется, если предполагается продать, подарить или повторно использовать диск. Вот первая тройка ПО для очистки жёстких дисков [30]:

- DBAN: программа очистки дисков при перезагрузке операционной системы, мультиплатформенная, бесплатная;
- DiskWipe: программа не требует установки, поддерживает все последние версии Windows (XP, Vista, Windows 7), бесплатная;
- CCleaner: поддерживает Windows (XP, Vista, Windows 7), бесплатная.

ПО для очистки данных успешно работает на дисках, не имеющих плохих запарченных секторов. Но оно не может стереть информацию на поврежденных дисках. Для эффективного уничтожения данных необходимо повторить процесс стирания данных по меньшей мере трижды. Полный цикл стирания может занять несколько часов или дней в зависимости от ёмкости диска.

3.3.2 Программа очистки «Darik's Boot and Nuke (DBAN)»

Это программное средство представляет собой загрузаемый образ ISO, который впоследствии можно записать на компакт-диск и, загрузившись с него, выбрать жёсткий диск для удаления данных. Работа с ним осуществляется следующим образом: нужно включить компьютер, жёсткие диски которого нужно очистить, затем следует убедиться, что он будет загружаться с привода для компакт-дисков, а затем вставить компакт-диск с программой DBAN.

После корректной загрузки появится экран приветствия и приглашение к выбору входных параметров boot, как показано на рисунке 3.1.

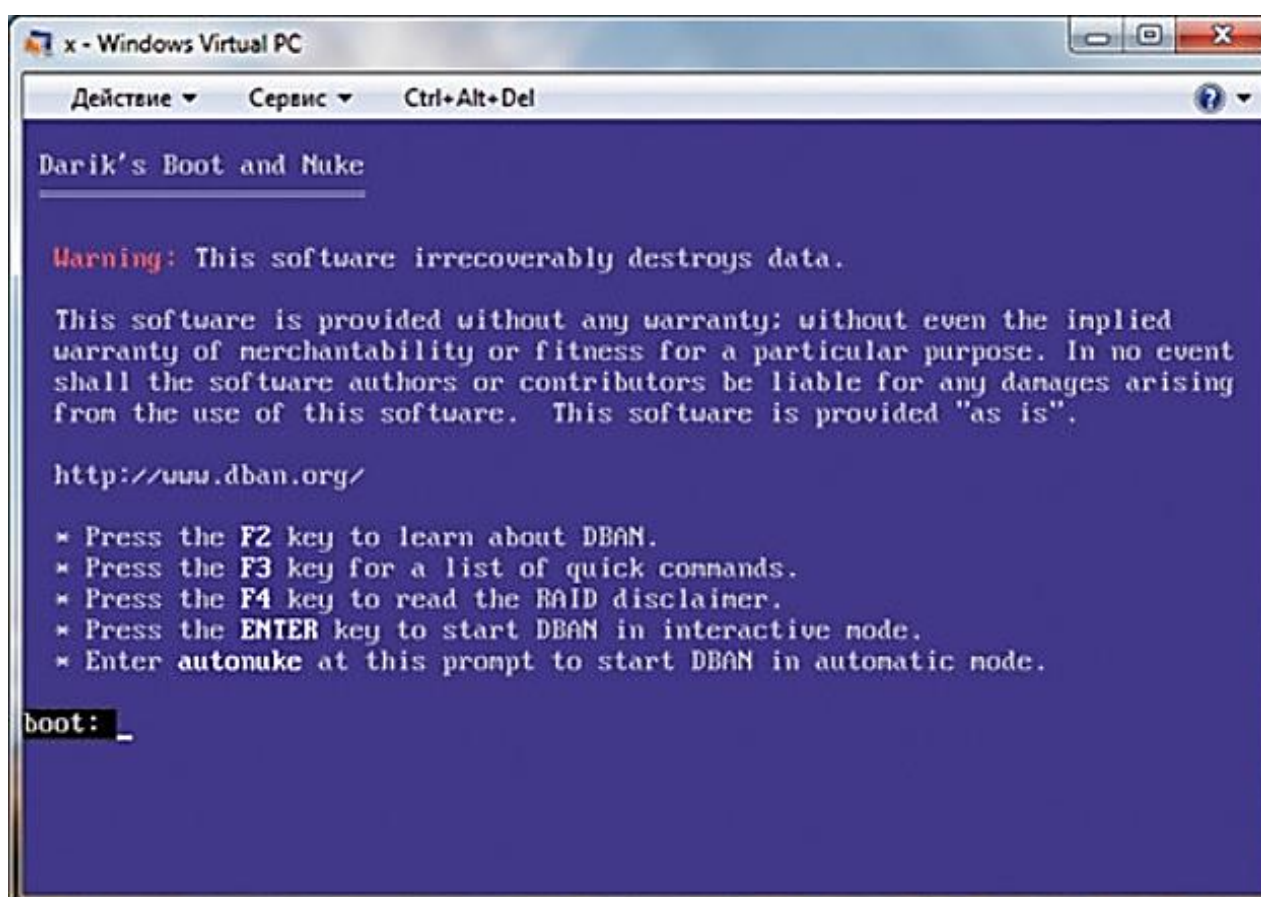


Рисунок 3.1 – Приглашение boot: _

Нажатием клавиши «М» осуществляется переход к ручному выбору метода очистки. Рекомендуется выбирать метод Gutmann Wipe для удаления конфиденциальных данных, как показано на рисунке 3.2. После выбора метода очистки происходит выбор диска, как показано на рисунке 3.3. При этом методе осуществляется 35 проходов по жесткому диску.



Рисунок 3.2 – Выбор метода Gutmann Wipe

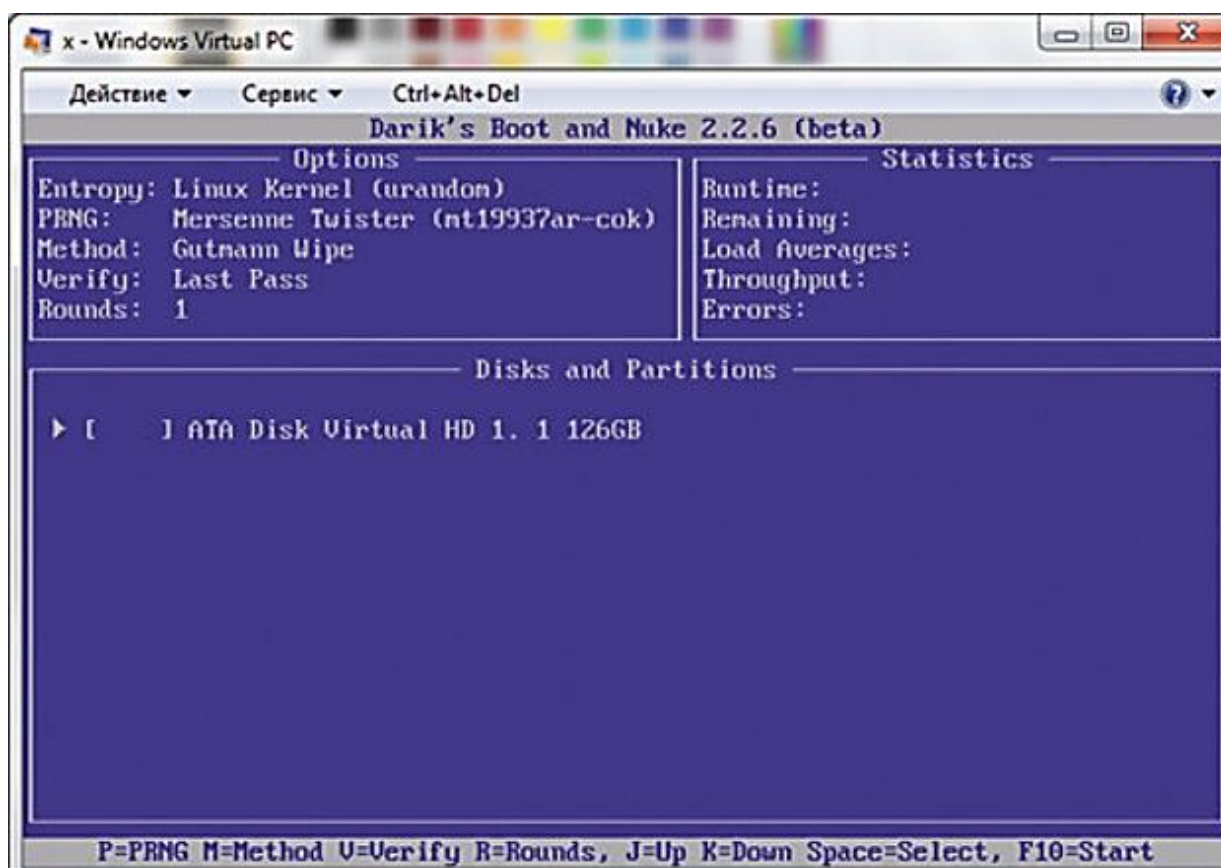


Рисунок 3.3 – Выбор диска для удаления данных по Gutmann Wipe

По нажатию клавиши «Пробел» осуществляется переход к экрану запуска процесса удаления данных, как показано на рисунке 3.4.

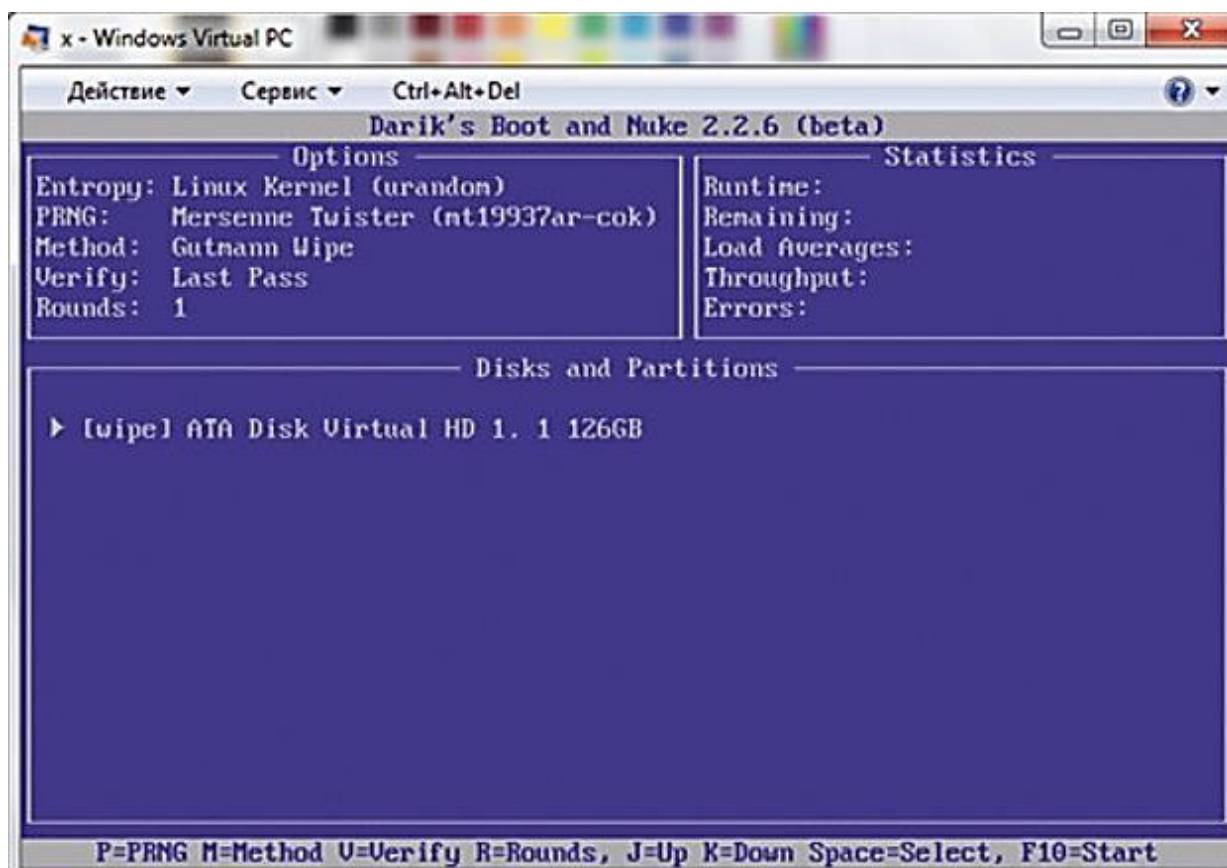


Рисунок 3.4 – Экран запуска процесса удаления данных

Необходимо убедиться, что был выбран нужный диск (раздел) для удаления данных. Если процесс проходит нормально, нужно нажать клавишу F10 для начала процесса удаления, который может занять до 12 часов.

3.3.3 Программа очистки *DiskWipe*

Disk Wipe — это бесплатное приложение для Windows, предназначенное для гарантированного уничтожения данных, не требующее установки. С Disk Wipe можно стереть все содержимое диска и сделать невозможным восстановление данных. Disk Wipe позволяет удалить данные как с раздела, так и со всего жесткого диска, используя алгоритмы Dod 5220-22.M, US Army, Peter Guttmann. Поддерживаются интерфейсы S-ATA, IDE, SCSI, USB, FIREWIRE. Интерфейс главной формы программы показан на рисунке 3.5.

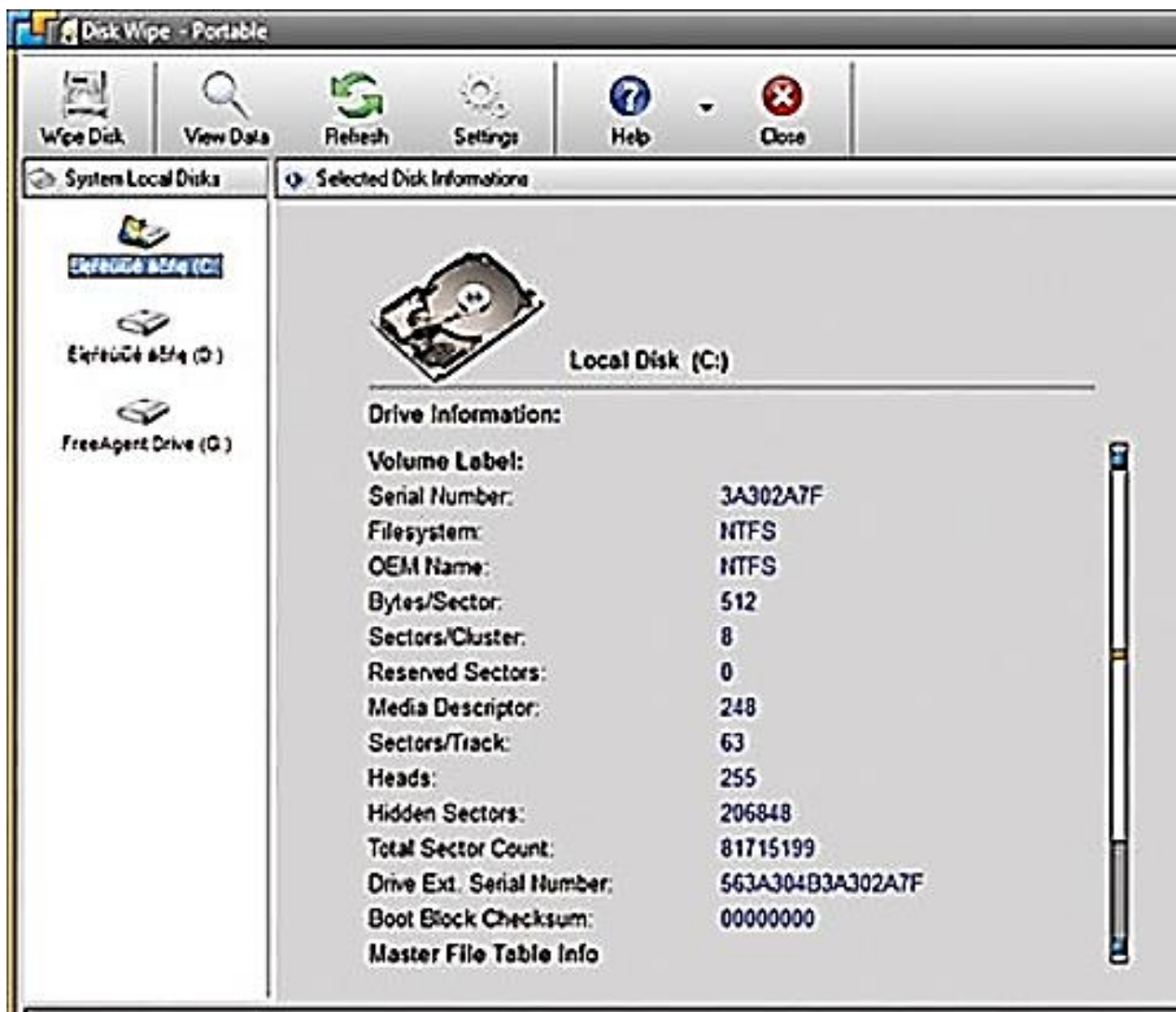


Рисунок 3.5 – Экран Disk Wipe

3.3.4 Программа очистки CCleaner

CCleaner — это бесплатная программа для оптимизации компьютера, совмещающая в себе модуль очистки системы, удаляющий все ненужные и временные файлы, и полнофункциональный модуль очистки реестра. CCleaner чистит следующие компоненты системы:

1. Internet Explorer — временные файлы кэша, историю посещений, cookies-файлы, скрытые файлы Index.dat, историю загрузок.
2. Корзину Recycle Bin.
3. Буфер обмена.
4. Временные файлы Windows
5. Журналы Windows.
6. Список последних документов (в меню «Пуск»).
7. Историю исполненных команд (в меню «Пуск»).

8. Историю помощника поиска в Windows XP.
9. Устаревшие данные функции Prefetch в Windows XP.
10. Дампы памяти после сбоев Windows.
11. Фрагменты файлов, остающиеся после работы команды Chkdsk.

Дополнительные функции позволяют очистить:

- Кэш очередности меню.
- Кэш сообщений системного лотка.
- Кэш размеров и адресов Windows.
- Историю помощи пользователю.
- Файлы журналов IIS.
- Дополнительные папки.

CCleaner не только удаляет старые файлы и другие данные стандартных компонентов Windows, но и позволяет производить очистку временных файлов и списков последних документов во многих других программах: Firefox, Opera, Safari, Media Player, eMule, Kazaa, Google Toolbar, Netscape, Microsoft Office, Nero, Adobe Acrobat Reader, WinRAR, WinAce, WinZip и других.

CCleaner использует модуль очистки реестра для обнаружения различных проблем и несоответствий в операционной системе. Он проверяет расширения файлов, элементы управления технологий ActiveX, ClassIDs, ProgIDs, программу удаления, общую библиотеку динамической компоновки DLL, шрифты, ссылки файлов помощи, пути приложений, значки и неправильные ярлыки.

Кроме того, предоставляется функция безвозвратного удаления файлов с жёстких дисков несколькими циклами перезаписи, что не позволяет восстановить их впоследствии никаким способом. Диск при этом не уничтожается.

Как показано на рисунке 3.6, для безопасного затирания свободного места на диске можно использовать следующие алгоритмы:

- простое стирание (1 проход);
- DOD 5220.22-M (3 цикла);
- NSA (7 циклов);
- Gutmann (35 циклов).

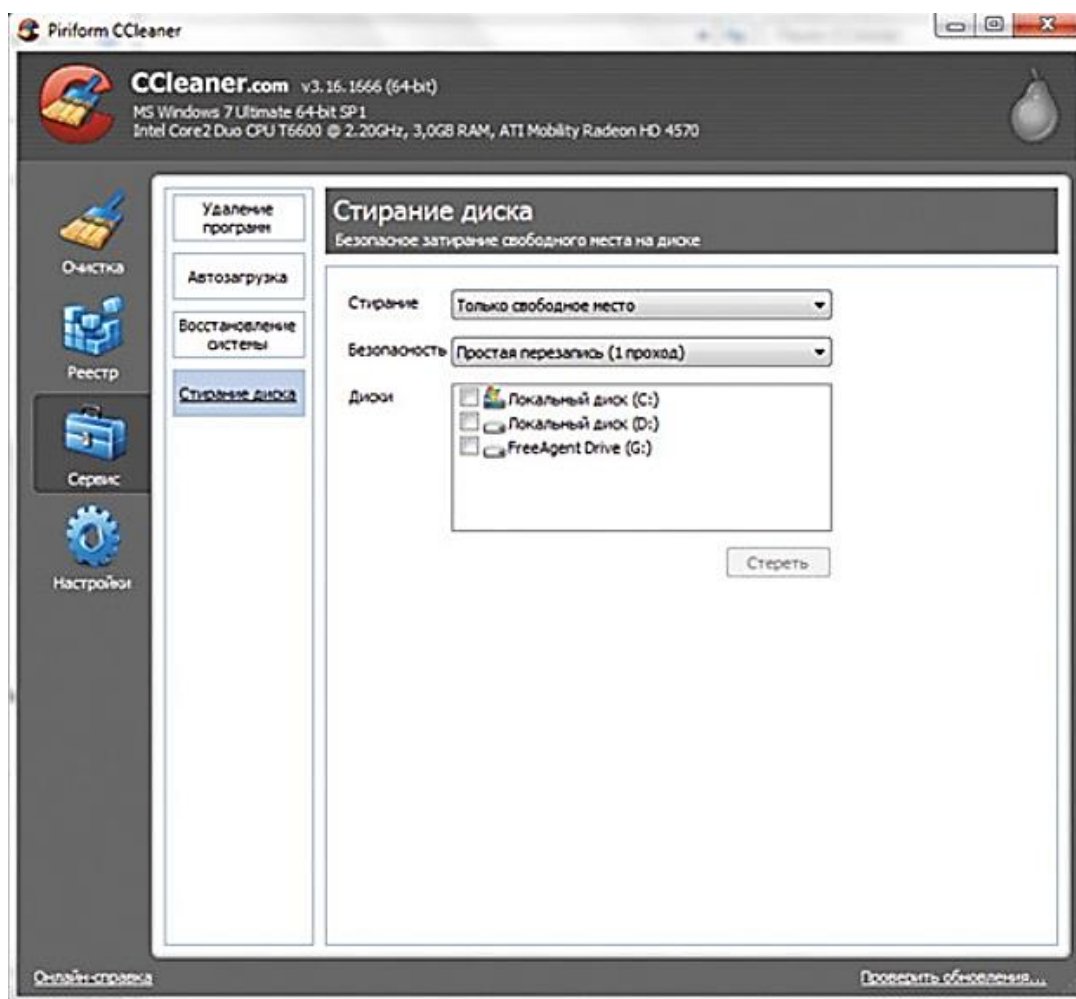


Рисунок 3.6 – Невосстановимое стирание с помощью CCleaner

Выводы

Подводя итог, можно сказать, что гарантированное удаление информации возможно осуществить как программными, так и физическими методами. В отличие от физических методов удаления, которые эффективны почти всегда, программные алгоритмы не столь надежны, так как не всегда можно быть точно уверенным, что никакой критической информации на диске не осталось.

Выделяют множество стандартов программного затирания информации, которые разработаны как Министерствами обороны разных стран, так и иными специалистами в области информационной безопасности.

Все рассмотренные алгоритмы основаны на перезаписи секторов, в которых хранилась критическая информация, но отличаются количеством циклов и видом данных, которыми происходит перезапись. Также существует разнообразное программное обеспечение, работающее на основе этих алгоритмов, которые отличаются функционалом и надежностью [31].

4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Всю разработанную систему «экстренной эвакуации файлов» условно можно разделить на три части: интерфейс пользователя для создания и настройки наборов горячих клавиш, класса, работающего с Google диском и модуля шифрования, позволяющего генерировать открытый и закрытый ключи, их импорта и экспорта в файл и шифрования. Приложение позволяет незаметно отслеживать нажатия заданных горячих клавиш с последующим отправлением, указанных файлов на Google Диск или их шифрованием (в зависимости от наличия подключения к сети «Интернет») с последующим затираньем файла и удалением из системы. Приложение показывает интерфейс при запуске, не отображается на панели задач и в треке. Это позволяет скрыть факт наличия программы на компьютере. Также приложение запоминает выбранные настройки, что позволяет задавать их один раз и использовать комбинации горячих клавиш после перезапуска приложения или всей системы.

Для корректной работы приложения необходимо подключить Google drive API и получить файл настроек в Google developers console. Для этого необходимо иметь профиль Google, перейти на страницу сервиса и зарегистрировать программу. Теперь подключаем Google drive API, как показано на рисунке 4.1, и устанавливаем NuGet пакет в сборку приложения.

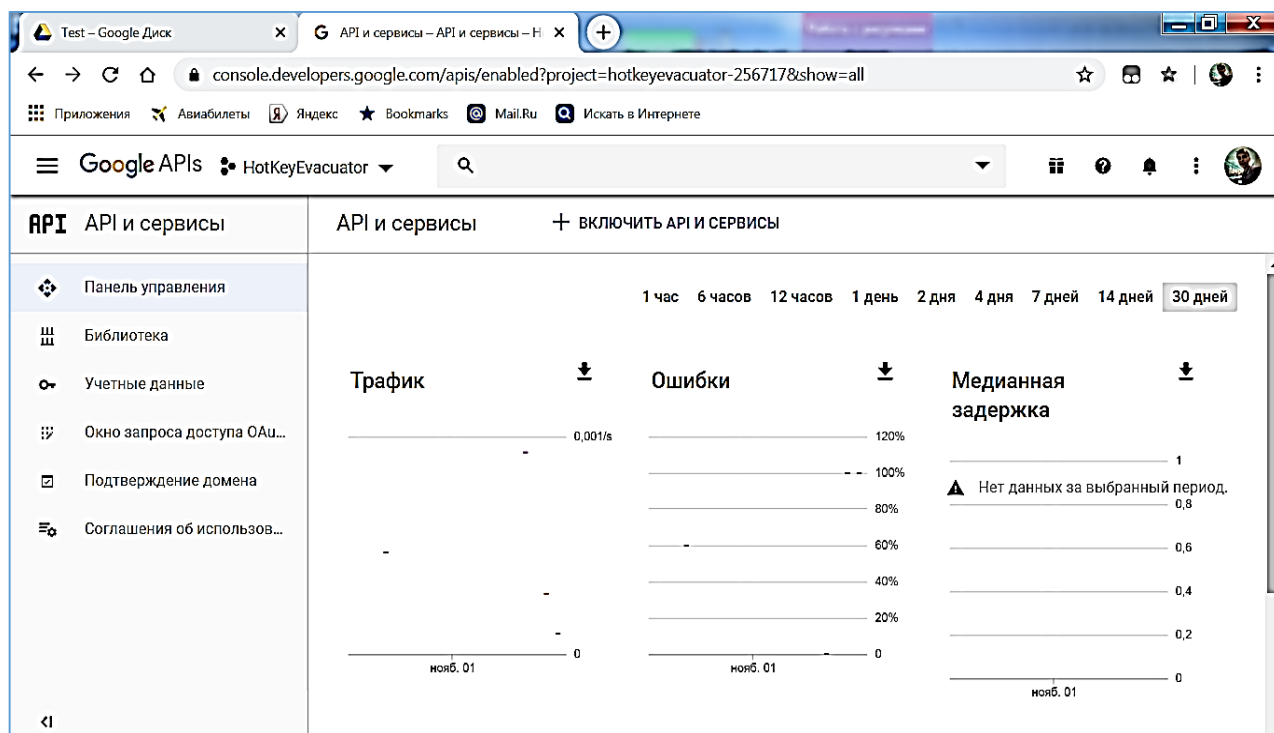


Рисунок 4.1 – Подключенный API

Это позволит использовать функции API. Но для доступа к диску необходимо получить файл – клиент с настройками. Без него Google не сможет выдать токен для доступа. Указываем необходимый уровень доступа к данным, и скачиваем json файл с настройками, после чего его также добавляем в сборку приложения. При первом запуске откроется вкладка в браузере с запросом доступа к Google drive API из указанного при настройке приложения. Эта процедура проводится один раз и в дальнейшем не будет выдавать приложение.

Программное приложение хранит настройки и при каждом запуске получает информацию из памяти о комбинациях горячих клавиш и файлах, связанных с ними. После того, как приложение получит словари с наборами клавиш и файлами, оно регистрирует их в системе, для корректного реагирования на их нажатие. Описанный процесс отображен на рисунке 4.2.

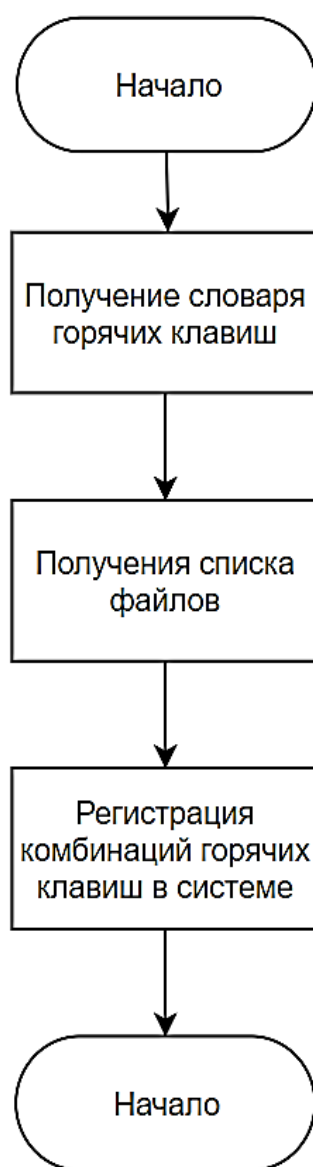


Рисунок 4.2 – Получение настроек и регистрация горячих клавиш

Запись в память приложения происходит при закрытии формы настроек, что позволяет избежать потери данных при аварийном завершении всего приложения.

Приложение имеет интерфейс настроек. Но он не отображается при запуске программы. Чтобы увидеть окно с интерфейсом необходимо нажать комбинацию клавиш, заданную константой и не входящую в словарь с комбинациями для работы с файлами. Форма, представленная на рисунке 4.3, содержит кнопку вызова настроек шифрования, кнопку выключения приложения и набор панелей для задания комбинаций горячих клавиш. Каждая панель имеет три комбобокса для выбора клавиш, поле для ввода пути к файлу, кнопку для выбора файла с помощью диалогового окна и кнопки сохранения, удаления настройки. При сохранении панель фиксируется и ниже добавляется пустая панель.

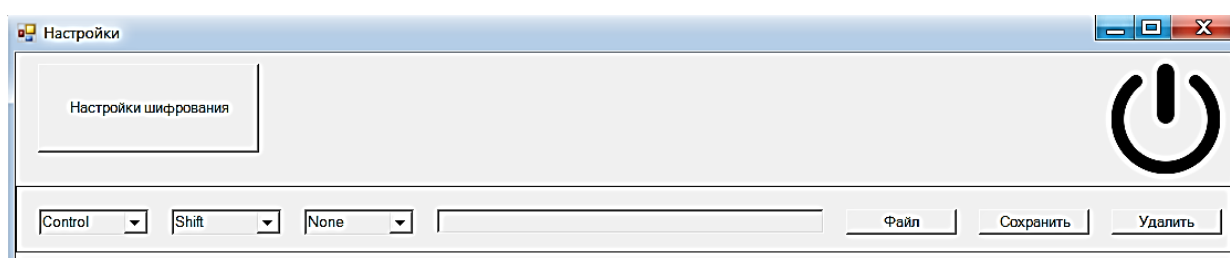


Рисунок 4.3 – Интерфейс настроек

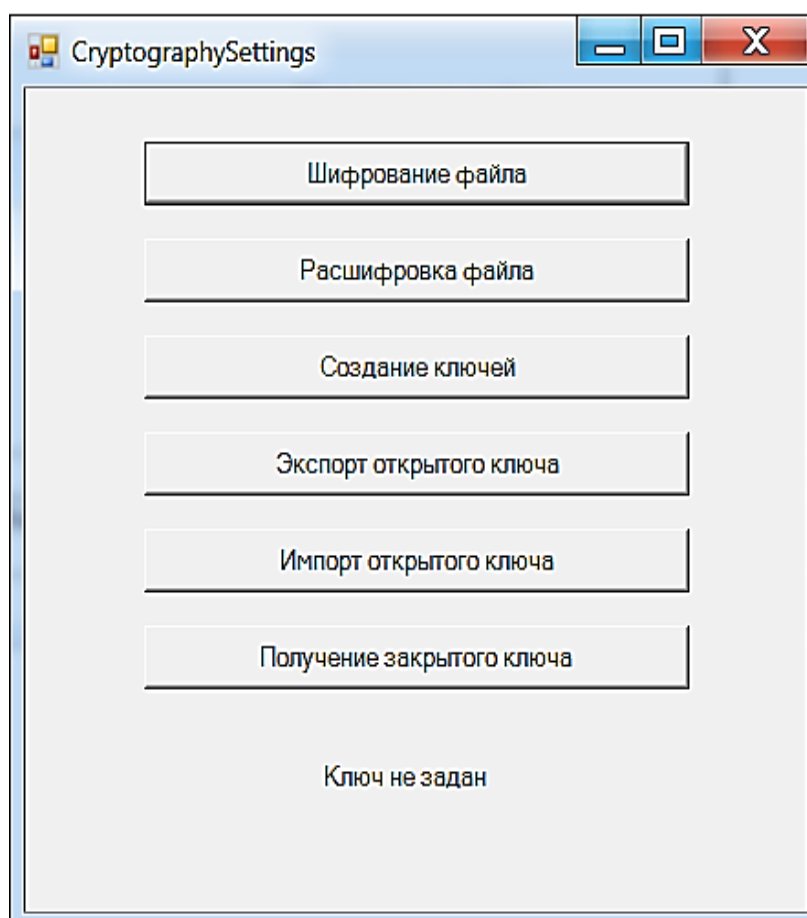


Рисунок 4.4 – Настройки шифрования

Настройка шифрования также имеет интерфейс, изображенный на рисунке 4.4, который позволяет сгенерировать открытый и закрытый ключи, импортировать открытый ключ в файл или экспортировать его из файла, а также расшифровать ранее зашифрованный документ. Логика шифрования вынесена в отдельный класс Cryptography.cs, так как методы будут использоваться без вызова интерфейса.

При нажатии зарегистрированных горячих клавиш программа проверяет наличие связанных файлов. Если таковые отсутствуют в системе, ничего не произойдет. Если файлы в наличии, приложение проверяет подключение к сети “Интернет” и доступность Google диска. Дальнейшее выполнение программы будет происходить по двум направлениям, в зависимости от наличия подключения к сети, это показано на рисунке 4.5.



Рисунок 4.5 – Процесс выполнения действия эвакуации

При наличии подключения к сети будет выполнена отправка файлов в облако, показанная на рисунке 4.6.

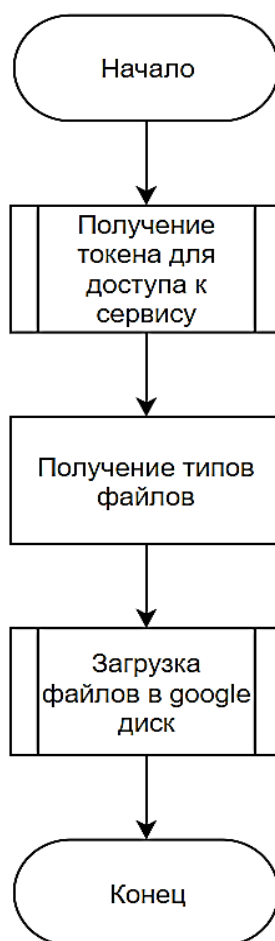


Рисунок 4.6 – Отправка файлов в облако

Сначала приложение запрашивает у сервиса Google drive токен для доступа к хранилищу. Так как приложение зарегистрировано и у нас есть файл-клиент, полученный ранее от сервиса, без необходимости подтверждения получаем токен. После данной процедуры получаем формат файлов, которые будут загружены в облако. Это действие необходимо, так как метод API для отправки файлов требует указания расширения файлов. После получения всей необходимой информации файлы загружаются на Google Диск.

При отсутствии подключения к сети, приложение производит шифрование файлов, данный процесс показан на рисунке 4.7. Важно помнить, что заранее должен быть подготовлен файл с открытым ключом. Программа экспортирует открытый ключ, после чего генерирует закрытый ключ. После получения ключевой пары, приложение зашифрует файлы и поместит их в указанную директорию. Рекомендуется в наборе файлов для эвакуации указать файл с открытым ключом, чтобы избежать возможности расшифрования файлов.

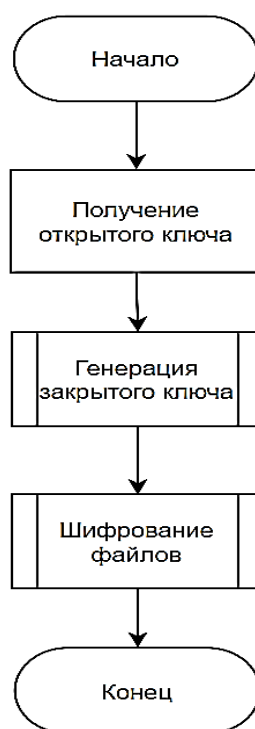


Рисунок 4.7 – Процесс шифрования файлов

После эвакуации файлов с компьютера, экземпляры, оставшиеся на локальном диске, затираются нулевыми битами и удаляются. Затирание позволит избежать корректного восстановления файла. Даже если файл будет восстановлен, он будет содержать несвязанный набор информации. Таким образом, исключается возможность получения указанных нами файлов сторонними лицами [30, 31].

Выводы

По итогам изучения существующих программных реализаций были намечены некоторые полезные функциональные дополнения, реализация которых сделает программное приложение более конкурентноспособным, а процесс затирания информации на дисках – удобным и незаметным:

- работа приложения в фоновом режиме;
- маскирование программного приложения для осуществления процесса затирания информации в Диспетчере задач Windows под системный процесс;
- запуск процедуры затирания информации путем нажатия горячих клавиш, комбинацию которых выбирает сам пользователь в окне настроек программы, которое вызывается также нажатием горячих клавиш;
- эвакуация файлов на Google Диск, предваряющая затирание.

По итогу написания программного приложения все функциональные дополнения были реализованы и протестированы.

РЕЗУЛЬТАТЫ 4 ГЛАВЫ

В данном исследовании реализовано импортозамещающее программное средство, позволяющее выполнить экстренное безвозвратное и гарантированное уничтожение критической информации, что позволит не только уберечь секретную информацию от несанкционированного доступа третьих лиц, но и оставить её в руках легитимного пользователя для дальнейшего использования.

Проведена исследовательская работа, в ходе которой были изучены общие теоретические аспекты использования гарантированного уничтожения информации, произведен детальный анализ сложности и обзор подходов к ГУИ, рассмотрены существующие стандарты уничтожения информации, а также проведен обзор существующих программных комплексов со схожим функционалом.

Как было выяснено, в основе всех программных алгоритмов лежит одна и та же операция перезаписи, но все они отличаются количеством циклов перезаписи и составом информации, которая записывается в ячейки памяти.

По итогу изучения существующих реализаций были выделены их достоинства и недостатки, с учётом которых было спроектировано своё, новое импортозамещающее программное средство, позволяющее высокоэффективно и незаметно безвозвратно уничтожать информацию в работающей системе без вреда для неё, оставляя диски работоспособными.

Подводя итог, стоит отметить, что были выполнены все ранее поставленные задачи, что позволило достигнуть поставленную цель – изучить разновидности способов гарантированного уничтожения информации, наиболее известные программные алгоритмы и реализации в этой области и реализовать программное средство гарантированного уничтожения критически важной информации пользователя.

ГЛАВА 5

РАЗРАБОТКА ИМПОРТОЗАМЕЩАЮЩЕГО ПРОГРАММНОГО СРЕДСТВА ФОРМИРОВАНИЯ ЭЛЕКТРОННОЙ ПОДПИСИ ПРИ ОРГАНИЗАЦИИ СИСТЕМЫ ЭЛЕКТРОННОГО ДОКУМЕНТООБОРОТА ОТЕЧЕСТВЕННОГО УЧРЕЖДЕНИЯ

ОБЩИЕ ПОЛОЖЕНИЯ

Документы – это критически важный элемент в любой организации, независимо от её масштаба и структуры. Будь то небольшая коммерческая фирма или крупное государственное ведомство, от качества документов, скорости их формирования и оборота, может зависеть само существование организации. Недостаточная документированность несёт в себе немалую опасность для существования организации. С другой стороны, излишняя бюрократизация может сильно замедлить её основные, жизненно важные внутренние процессы. Но в наше время, когда главной ценностью становится информация, и её объёмы очень быстро растут, документирование всё большего количества сведений, актов, соглашений и многих других форм информации становится необходимым и неизбежным.

Бурное развитие информационных технологий принесло не только большие объёмы информации, но и новые, более эффективные средства и методы работы с ней. Речь, само собой, идёт о недорогом высокоскоростном интернете, доступных смартфонах и обязательно имеющихся в любой организации достаточно мощных и производительных персональных компьютерах. Все эти условия привели к желанию, и, главное, возможности переосмысления работы с документами, в том числе юридически значимыми. Стало возможным организовать полный цикл жизни документов в виде цифрового документооборота, совсем без участия физических, бумажных носителей.

Использование электронных документов даёт большой ряд бесспорных преимуществ перед документами бумажными:

- значительное уменьшение времени передачи документов – электронный документ передаётся почти мгновенно на любое расстояние;
- снижение количества ошибок в документах, а также упрощение исправления обнаруженных ошибок;
- экономия на расходных материалах, почтовых услугах;
- избавление от необходимости вести бумажные архивы, занимающие много места, а также создающие потенциальную возможность утери документов.

Было бы странно и неэффективно не использовать такие возможности, поэтому текущее законодательство позволяет полноценно использовать электронные документы любой значимости, вместо документов на бумажных носителях. Исключение составляют только документы, содержащие секретные сведения, составляющие государственную тайну.

Однако организация электронного документооборота предполагает не только отказ от бумажных носителей в пользу электронных цифровых файлов, но и обеспечение безопасности документов от несанкционированного доступа. Ключевым условием для этого является использование электронной подписи.

Электронная подпись (ЭП) – это специальная информация в электронном виде, полученная с помощью преобразования подписываемого документа и ключа электронной подписи. Способы формирования и предоставляемые возможности электронной подписи отличаются в зависимости от её вида.

Так как без применения электронной подписи невозможно говорить об электронном обороте документов даже не самой высокой значимости, а внедрение и грамотное обеспечение такого документооборота являются актуальными задачами в самых разных организациях, то задачи, связанные непосредственно с электронной подписью, точно так же важны и актуальны.

Электронные подписи используются всё чаще. В настоящее время в РФ отмечается существенный рост рынка электронной подписи, что прямо обусловлено увеличением количества выданных сертификатов ЭП. За период с 2006 г. по 2015 г. произошёл рост данного рынка с 1,5 млрд. руб. до 14,8 млрд. руб., то есть в 9,9 раз. Вместе с ростом заинтересованности в законном использовании электронных подписей, растёт и преступный интерес. В 2019 году стало известно об участившихся случаях производства мошеннических сделок с использованием ЭП. Все перечисленные факты указывают на то, что электронная подпись перспективна и актуальна, а так как на данный момент она проходит стадию активного освоения и внедрения, число задач самого разного рода, с ней связанных, в ближайшее время будет только расти [32].

Объект исследования: информационные технологии организации электронного документооборота, применяемые в государственных учреждениях России.

Предмет исследования: теоретические и практические аспекты использования электронной цифровой подписи при организации электронного документооборота в государственных структурах Российской Федерации.

Цель работы: создание импортозамещающего программного средства с применением технологий удалённого (облачного) формирования и проверки ЭП с помощью технологии блокчейн. Достижение цели предполагает:

- изучение теоретических и правовых основ использования электронной подписи;
- рассмотрение и анализ существующих аналогичных программных продуктов;
- разработка архитектуры и алгоритмов работы итогового программного средства;
- практическая реализация и проверка работоспособности разработанного программного средства на практике.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ЭЛЕКТРОННОЙ ПОДПИСИ

1.1 Собственноручная подпись

Обычной подписью принято считать традиционную, собственноручную, личную подпись, которая представляет собой написанную от руки последовательность текстовых символов и декоративных росчерков. Текстовая часть подписи обычно содержит фамилию, имя и отчество владельца подписи, которые также могут быть записаны и сокращённо.

Подпись используется для аутентификации (подтверждения подлинности) разнообразных документов, или, другими словами, для закрепления и подтверждения факта совершения какого-либо действия подписывающим лицом. Например, подпись в договоре подтверждает, что именно подписывающее лицо заключило данный договор и согласно со всеми его положениями.

При определённых условиях личная подпись может использоваться и для идентификации подписывающего. По различным характеристикам подписи, эксперт–графолог может определить настоящего владельца подписи. Как минимум, эксперту для этого необходимо иметь экземпляр рассматриваемой подписи неустановленного лица и экземпляр подписи, обладатель которой точно известен.

Собственноручная подпись считается полноценным реквизитом документов, что указано в национальном стандарте ГОСТ Р 7.0.97–2016 «Система стандартов по информации, библиотечному и издательскому делу. Организационно–распорядительная документация. Требования к оформлению документов». Лаконичное определение подписи содержит другой стандарт, ГОСТ Р 7.0.8–2013 СИБИД «Делопроизводство и архивное дело. Термины и определения». Оно определяет личную подпись конкретного человека как «реквизит, содержащий собственноручную роспись должностного или физического лица».

Хорошая, сложная личная подпись естественным образом обладает устойчивостью к подделыванию. Чем больше в такой подписи элементов, уникальность которых зависит от индивидуальных свойств почерка подписывающего человека, тем сложнее все эти элементы в точности повторить, а значит, тем надёжнее данная подпись.

Однако, слишком высокая сложность подписи может стать не только преимуществом, но и недостатком подписи. Сложную подпись может быть тяжело воспроизвести не только злоумышленнику, но и владельцу подписи. Подпись должна быть достаточно легко повторяема для её владельца.

В идеальном случае воспроизвести подпись должно быть возможно не только легко, но и быстро. Если же подпись настолько сложна, что даже владелец не в состоянии в точности её повторить, он будет вынужден, строго говоря, каждый раз использовать новую подпись. А это, в свою очередь, почти полностью избавляет подпись от её ключевых качеств: аутентифицируемости и идентифицируемости.

Таким образом, основными свойствами личной подписи являются:

- устойчивость к подделке мошенников;
- повторяемость при многократном написании;
- быстрота написания, обусловленная привычкой владельца подписи.

Этими свойствами обусловлена возможность использования подписи для аутентификации и идентификации. А совокупность рассмотренных свойств и возможностей подписи, в свою очередь, обеспечивает приемлемость и распространённость использования подписи при оформлении документов и договорённостей самой разной степени юридической важности, вплоть до федеральных законов и документов, содержащих сведения, составляющие государственную тайну.

1.2 Электронная подпись

Электронная подпись – это сформированная по определенному алгоритму информация в электронном виде, связанная с другой, подписываемой информацией и подписывающим лицом (лицами) таким образом, что с помощью специальных средств электронной подписи можно подтвердить или опровергнуть подлинность (целостность) подписанной информации, установить факт подписания, авторов подписей, а также другие вспомогательные сведения.

Данного определения достаточно, чтобы понять, что у электронной подписи есть много общего с подписью собственноручной. Прежде всего, очевидно, цели существования обоих видов подписей идентичны: аутентификация документов, идентификация подписавших лиц. Оба вида подписей, теоретически, при определённых обстоятельствах, могут быть подделаны.

Есть и немало отличий электронной подписи от собственноручной, обусловленных их разной природой. Во–первых, электронная подпись, даже при её формировании по одному и тому же алгоритму, при подписании разных документов категорически не может иметь одинаковый вид. Во–вторых, так как формирование подписи производит вычислительное устройство, а не рука человека, теряет актуальность свойство простоты воспроизведения подписи её обладателем. В–третьих, с небольшой оговоркой, при использовании электронной подписи перестаёт быть актуальным свойство быстроты её формирования.

Конечно, время выполнения криптографических операций, производящихся при формировании надёжных цифровых подписей, может возрастать при увеличении уровня надёжности этих подписей. Но при современном уровне развития технологий речь идёт даже не о секундах, требующихся на написание подписи рукой, а о долях секунды, достаточных для формирования криптографически стойких электронных подписей. Наконец, электронная подпись в совокупности с сертификатом электронной подписи позволяют точно идентифицировать владельца подписи, без необходимости выполнения специальных экспертиз. К тому же, электронная подпись может содержать в себе и дополнительную информацию, например, дату и время формирования подписи.

Первое упоминание электронной подписи появилось относительно недавно. В 1976 году Уитфилд Диффи и Мартин Хеллман в своей работе «New Directions in Cryptography» предположили, что на основе классических криптографических систем возможно впервые создать полностью электронный, цифровой механизм, аналогичный собственноручной подписи владельца [33].

Годом позднее, Рональд Ривест, Ади Шамир и Леонард Адлеман разработали широко известный, и до сих пор часто применяемый криптографический алгоритм RSA. В статье «A Method for Obtaining Digital Signatures and Public-Key Cryptosystems» учёные описали свой созданный алгоритм, а также способы его использования для формирования цифровых подписей [34].

Эти исследования послужили уверенным начальным толчком для дальнейшего теоретического и практического развития криптографии в области цифровой подписи. После RSA были разработаны и другие схемы цифровой подписи, например, алгоритмы Майкла Рабина и Ральфа Меркла [35].

Важным этапом в развитии цифровых подписей стало первое строгое определение требований безопасности к алгоритмам цифровой подписи. Они были разработаны Шафи Гольдвассером, Сильвио Микали и Рональдом Ривестом в 1984 г. Они также описали модели возможных кибератак на электронные цифровые подписи, и разработали схему GMR, устойчивую к описанным атакам.

История обособленного развития цифровой подписи в России началась в 1994 году, когда главным управлением безопасности связи ФАПСИ был разработан первый российский стандарт ЭЦП – ГОСТ Р 34.10–94 «Информационная технология. Криптографическая защита информации». Процедуры выработки и проверки электронной цифровой подписи на базе асимметричного криптографического алгоритма. В 2002 г. на смену этому стандарту пришёл одноимённый ГОСТ Р 34.10–2001, криптографический алгоритм которого основан уже на вычислениях в группе точек эллиптической кривой.

В соответствии с этим стандартом, термины «электронная цифровая подпись» и «цифровая подпись» являются синонимами. Следующим этапом развития этой технологии стал стандарт ГОСТ Р 34.10–2012 «Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи». Актуальным на данный момент является принятый 1 июня 2019 г. межгосударственный стандарт ГОСТ 34.10–2018 Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи.

Можно выделить две основные схемы построения цифровой подписи. Первая схема основана на использовании алгоритмов симметричного шифрования. Она предусматривает существование доверенного третьего лица – арбитра. Вторая схема основана на использовании алгоритмов асимметричного шифрования, и является на сегодняшний день наиболее распространённой.

Симметричные алгоритмы шифрования предполагают использование одного ключа шифрования, используемого как для зашифрования, так и для расшифрования данных. Такой ключ обязательно должен держаться в тайне, и именно поэтому схемы электронной подписи на основе таких алгоритмов предполагают наличие арбитра. Авторизацией документа в такой схеме является сам факт зашифрования документа ключом шифрования, и передача его арбитру.

Асимметричные алгоритмы шифрования предполагают использование двух разных ключей: секретного, для зашифрования, и публичного, для расшифрования. Секретным ключом обладает только владелец подписи. Упрощённое графическое представление асимметричной схемы цифровой подписи показано на рисунке 1.1.

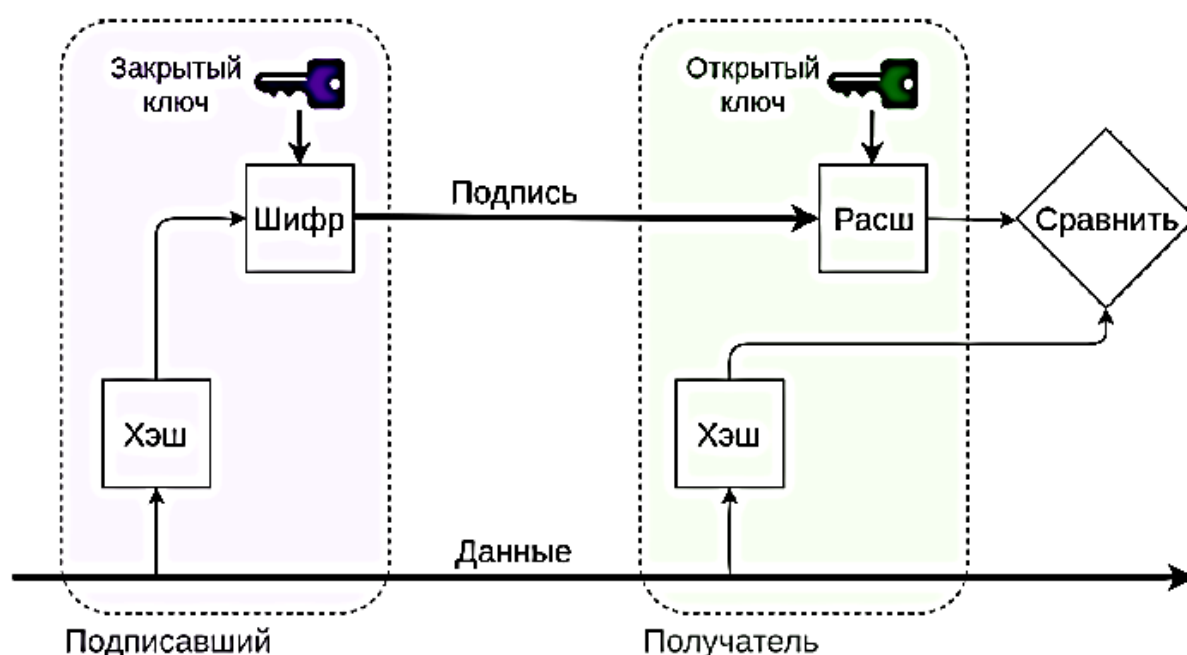


Рисунок 1.1 – Асимметричная схема цифровой подписи

Для подписания документа владелец подписи, как правило, с помощью хэш-функции формирует хэш от содержимого документа, шифрует полученный хэш асимметричным алгоритмом шифрования с использованием своего секретного ключа. Результатом этих преобразований и является подпись документа. Проверить подпись может любой желающий, при наличии подписанного документа, самой подписи и открытого ключа, связанного с закрытым ключом, которым подпись была создана. Для проверки подписи проверяющему необходимо рассчитать хэш от содержимого документа с помощью той же хэш-функции, которая была использована подписывающим. К подписи документа необходимо применить алгоритм шифрования, используя открытый ключ. Такое применение алгоритма шифрования, фактически, будет являться расшифрованием зашифрованного ранее хэша. Если рассчитанный на основе документа хэш полностью совпадает с расшифрованным, подпись считается верной. Если же хэши не совпали, подпись неверна, и подписанной информации доверять нельзя.

Подпись может оказаться неверной в следующих случаях:

1. Проверяющий ошибся, и неверно выбрал алгоритм шифрования или хэширования, либо же выбрал сразу два неверных алгоритма. Успешно и корректно проверить цифровую подпись возможно только используя в точности такие же криптографические алгоритмы, какие были выбраны владельцем подписи.

2. Проверяющий обладает неверным открытым ключом, который не связан с закрытым ключом подписывающего.

3. Подписанный документ или сама подпись были изменены. Другими словами, была нарушена целостность информации.

В первых двух случаях проблему можно исправить, просто уточнив недостающие исходные данные и исправить саму схему формирования и проверки цифровой подписи. Для обнаружения третьего случая цифровая подпись и существует. Если по какой-либо причине, будь то помехи в канале связи или действия злоумышленника, подписанная информация или подпись были искажены, задача системы электронной подписи – сообщить об этом искажении.

1.3 Правовое регулирование в области использования ЭП

1.3.1 *Федеральный закон «Об электронной подписи»*

В отличие от собственноручной подписи, описания которой в российском законодательстве нет, за исключением ранее упомянутых национальных стандартов, подпись электронная описана достаточно подробно.

Главным документом в области использования электронных подписей является Федеральный закон от 06.04.2011 N 63–ФЗ «Об электронной подписи». Согласно статье 2 данного закона, электронная подпись – это информация в электронной форме, присоединённая к другой информации в электронной форме (подписываемой информации) или иным образом связанная с такой информацией, и которая используется для определения лица, подписывающего информацию. Данным законом определено три вида электронной подписи: простая, усиленная неквалифицированная, усиленная квалифицированная.

1.3.2 Простая электронная подпись

Простая электронная подпись не предъявляет никаких конкретных требований к используемым алгоритмам. Пункт 2 статьи 5 рассматриваемого закона определяет простую подпись как «электронная подпись, которая посредством использования кодов, паролей или иных средств подтверждает факт формирования электронной подписи определенным лицом».

Существует множество примеров использования простой электронной подписи. Можно сказать, что каждый Интернет–пользователь социальных сетей и мессенджеров, то есть почти каждый человек, в повседневной жизни сталкивается с простой подписью. Поскольку упомянутые сервисы требуют от каждого пользователя пройти процедуру идентификации и аутентификации с помощью кодов и паролей, каждую произведённую единицу информации, будь то личное сообщение или публичная заметка, можно считать подписанной простой электронной подписью. Подтверждение факта формирования любой единицы информации конкретным лицом обеспечивается процедурами идентификации и аутентификации, или, в терминах закона, кодами и паролями.

Условия использования простой электронной подписи определены статьями 6 и 9 федерального закона «Об электронной подписи». Пункт 2 статьи 6 указывает, что документ, подписанный простой подписью, признаётся равнозначным документу на бумажном носителе в случаях, предусмотренных другими законами и нормативно–правовыми актами, а также соглашениями между участниками электронного взаимодействия. При этом ко всем таким актам предъявляются требования, описанные в статье 9. А именно, нормативно–правовые акты и соглашения должны предусматривать:

- правила определения лица, подписывающего электронный цифровой документ, по его простой электронной цифровой подписи;
- обязанность лица, создающего и (или) использующего ключ простой электронной цифровой подписи, соблюдать его конфиденциальность.

Помимо этого, статья 9 устанавливает важное ограничение, состоящее в недопустимости применения простой электронной цифровой подписи для подписания электронных документов, содержащих сведения, составляющие государственную тайну, или в информационной системе, содержащей сведения, составляющие государственную тайну.

1.3.3 Неквалифицированная электронная подпись

К неквалифицированной электронной подписи закон уже предъявляет ряд конкретных требований. Такая подпись должна:

1. Быть получена в результате криптографического преобразования информации с использованием ключа электронной цифровой подписи.
2. Позволять определить лицо, подписавшее электронный документ.
3. Позволять обнаружить факт внесения изменений в электронный документ после момента его подписания с помощью электронной цифровой подписи.
4. Создаваться с использованием средств электронной подписи.

Средствами электронной цифровой подписи, упомянутыми в п. 4, называются криптографические средства, используемые для создания ключей электронной подписи, самих подписей, а также для произведения проверки подписей.

Такая подпись может использоваться во всех ситуациях, позволяющих использование простой электронной подписи. Кроме этого, документ, подписанный неквалифицированной электронной подписью, может признаваться равносильным документу на бумажном носителе, где помимо собственноручной подписи используется печать. Возможность использования подобной электронной подписи вместо печати должна быть предусмотрена правовыми актами или соглашениями.

В отличие от простой подписи, обязанности участников электронного взаимодействия, использующих любой из видов усиленной подписи, не должны быть закреплены в дополнительных правовых актах или соглашениях, так как они уже закреплены в самом законе, о чём сказано в статье 10.

Эти обязанности включают в себя:

1. Требование обеспечения конфиденциальности ключей электронных подписей.
2. Требование уведомлять участников электронного взаимодействия о нарушении конфиденциальности ключа в случае, если это произойдёт.
3. Запрет на использование ключа электронной подписи при наличии оснований полагать, что его конфиденциальность была нарушена.
4. При использовании квалифицированной электронной цифровой подписи – требование использования только средств электронной цифровой подписи, получивших подтверждение соответствия требованиям закона.

1.3.4 Квалифицированная электронная подпись

Самые серьёзные требования предъявляются к последнему виду – квалифицированной электронной цифровой подписи. Помимо того, что такая подпись должна соблюдать все требования упомянутых ранее видов, она также должна обрабатываться только средствами электронной цифровой подписи, имеющими подтверждение соответствия требованиям рассматриваемого закона.

У квалифицированной электронной цифровой подписи обязательно должен быть квалифицированный сертификат, в котором указан ключ её проверки. Она признаётся действительной только в том случае, если обратное не признано решением суда, и при этом соблюдаются все следующие условия:

1. Квалифицированный сертификат подписи создан и выдан аккредитованным удостоверяющим центром (УЦ) с действительной на момент выдачи аккредитацией.
2. Квалифицированный сертификат действителен на момент подписания документа, либо на момент проверки, если момент подписания неизвестен.
3. Успешно подтверждены как обладание владельцем сертификата использованной подписи, так и целостность подписанной информации.

Такая подпись может использоваться для подписания любых документов, в том числе и содержащих сведения ограниченного доступа, и даже сведения, составляющие государственную тайну. Исключением могут быть только отдельные документы, признаваемые действительными только на бумажном носителе. Такие документы должны определяться отдельными законами РФ, или иными нормативно–правовыми актами.

1.3.5 Другие законы и нормативно правовые акты

Упоминание электронной подписи есть во многих других нормативно-правовых актах. Большинство актов, основной сферой действия которых являются отношения в области использования электронных подписей, регулируют использование квалифицированной электронной подписи. В числе таких актов: Приказ ФСБ России от 27.12.2011 № 795 «Об утверждении Требований к форме квалифицированного сертификата ключа проверки электронной подписи», Приказ ФСБ РФ от 27.12.2011 № 796 «Об утверждении Требований к средствам электронной подписи и Требованиям к средствам удостоверяющего центра», постановления Правительства России, связанные с использованием ЭП органами исполнительной власти и местного самоуправления.

Федеральный закон от 27.07.2010 № 210–ФЗ «Об организации предоставления государственных и муниципальных услуг» в статье 21.1 закрепляет возможность граждан обращаться за получением государственных и муниципальных услуг с помощью электронных документов, подписанных электронной подписью.

Федеральный закон от 05.04.2013 № 44–ФЗ «О контрактной системе в сфере закупок товаров, работ, услуг для обеспечения государственных и муниципальных нужд» (статья 5) устанавливает допустимость проведения электронных торгов с применением электронного документооборота и электронных подписей.

Существует и множество других, отраслевых нормативно–правовых актов, закрепляющих возможность и способы применения электронной подписи в различных отношениях, таких как уплата налогов и подача отчётности.

1.4 Алгоритмы электронной подписи

1.4.1 Алгоритм RSA

RSA – асимметричный алгоритм шифрования, описанный в статье учёных Рональда Риверста, Ади Шамира и Леонарда Адлемана «A Method for Obtaining Digital Signatures and Public–Key Cryptosystems», опубликованной в 1978 г. Криптосистема RSA стала первой системой, пригодной и для шифрования, и для цифровой подписи, и до сих пор используется в большом числе криптографических приложений. Для шифрования используется операция возведения в степень по модулю большого числа. Данный алгоритм основан на вычислительной сложности задачи факторизации больших целых чисел. Для дешифрования за разумное время (обратное преобразование) необходимо уметь вычислять функцию Эйлера от данного большого числа, для чего необходимо знать разложение числа на простые множители, т.е. выполнить факторизацию большого числа [34].

При защищённой передаче данных, как и во многих асимметричных алгоритмах шифрования, для зашифрования используется открытый ключ, а для расшифрования – закрытый. Для формирования и проверки электронной подписи может использоваться тот же самый алгоритм, что и для шифрования данных, но при условии использования закрытого ключа при подписании, то есть в начале процесса, а открытого – при проверке подписи.

Пара ключей шифрования RSA создаётся по следующему алгоритму:

1. Выбираются два различных случайных простых числа p и q заданного размера (например, 1024 бита каждое).

2. Вычисляется их произведение $n=p \cdot q$, которое называется модулем.

3. Вычисляется значение функции Эйлера от числа n :

$$\varphi(n)=(p-1) \cdot (q-1).$$

4. Выбирается целое число, удовлетворяющее соотношению $(1 < e < \varphi(n))$, взаимно простое со значением функции $\varphi(n)$:

- число e называется открытой экспонентой;
- обычно в качестве e берут простые числа, содержащие небольшое количество единичных бит в двоичной записи, например, простые из чисел Ферма: 17; 257 или 65537; так как в этом случае время, необходимое для шифрования с использованием быстрого возведения в степень, будет существенно меньше;
- слишком малые значения e , например, 3, потенциально могут ослабить безопасность схемы RSA, так как хакер сможет их легко подобрать.

5. Вычисляется число d , мультипликативно обратное к числу e по модулю $\varphi(n)$, то есть число, удовлетворяющее сравнению

$$d \cdot e \equiv 1 \pmod{\varphi(n)};$$

– число d называется секретной экспонентой. Обычно оно вычисляется при помощи расширенного алгоритма Евклида.

6. Пара (e, n) публикуется в качестве открытого ключа RSA.

7. Пара (d, n) играет роль закрытого ключа RSA и держится в секрете.

Схемы создания и проверки подписей по алгоритму RSA показаны на рисунке 1.2 и состоят из шагов, указанных далее.



Рисунок 1.2 – Схема формирования и проверки ЭЦП алгоритма RSA

Создание подписи:

1. Взять открытый текст m .
2. Создать цифровую подпись s с помощью своего секретного ключа $\{d, n\}$:

$$s = SA(m) = md \pmod{n}.$$

3. Передать пару $\{m, s\}$, состоящую из сообщения и подписи.

Проверка подписи:

1. Принять пару $\{m, s\}$.
2. Взять открытый ключ $\{e, n\}$ Алисы.
3. Вычислить прообраз сообщения из подписи: $m' = PA(s) = se \pmod{n}$.
4. Проверить подлинность подписи (и неизменность сообщения), сравнив m и m' .

1.4.2 Алгоритмы DSA и ECDSA

Digital Signature Algorithm (DSA) – это алгоритм электронной цифровой подписи, предложенный Национальным институтом стандартов и технологий США (НИСТ) в августе 1991 г. DSA входит в состав Digital Signature Standard (DSS) – стандарта США в области электронных цифровых подписей. Последняя версия данного стандарта вступила в силу в июле 2013 г. и обозначается как FIPS–186–4 (Federal Information Processing Standards). Алгоритм основан на вычислительной сложности нахождения дискретных логарифмов в конечных полях.

Существует также и родственный алгоритм Elliptic Curve DSA (ECDSA), определённый, в отличие от DSA, не над кольцом целых чисел, а в группе точек эллиптической кривой. Однако он уже несколько раз подвергался критике, суть которой сводится к тому, что уровень обеспечиваемой безопасности у данного алгоритма не превосходит уровень оригинального алгоритма DSA.

Алгоритм DSA так же, как и ГОСТ 34.10, подписывает не сам текст документа, а его хэш, из-за чего требует использования надёжной хэш-функции, как можно меньше подверженной *коллизиям* (совпадениям хэш-последовательностей, сформированных на основе различных данных). В первой версии стандарта использовалась хеш-функция SHA–1 (Secure Hash Algorithm), в последней версии можно использовать любой алгоритм семейства SHA–2.

В августе 2015 г. был опубликован FIPS–202, описывающий новую хеш-функцию SHA–3, но она пока ещё не включена в стандарт DSS.

Параметры схемы электронной цифровой подписи:

1. Криптографическая хеш-функция $H(x)$.
2. Простое число q , размерность которого N в битах совпадает с размерностью в битах значений хеш-функции $H(x)$.
3. Простое число p , такого, что $(p-1)$ делится на q . Битовая длина p обозначается $L(2^{L-1}) < p < 2^L$.
4. Число g такое, что его мультипликативный порядок по модулю p равен q . Для его вычисления можно воспользоваться формулой $g = h^{(p-1)/q} \bmod p$, где h – некоторое произвольное число, $h \in (1; p-1)$ такое, что $g \neq 1$. В большинстве случаев значение $h = 2$ удовлетворяет этому требованию.
5. Секретный ключ – число $x \in (0, q)$.
6. Открытый ключ, вычисляемый по формуле $y = g^x \bmod p$.

Открытыми параметрами являются числа (p, q, g, y) . Закрытый параметр только один – число x . При этом числа (p, q, g) могут быть общими для группы пользователей, а числа x и y являются соответственно закрытым и открытым ключами конкретного пользователя.

При подписывании сообщения используются секретные числа x и k , причём число k должно выбираться случайным образом (на практике псевдослучайным) при вычислении подписи каждого следующего сообщения.

Так как параметры (p, q, g) могут быть использованы для нескольких пользователей, то на практике пользователей зачастую делят по некоторым критериям на группы с одинаковыми значениями (p, q, g) . По этой причине указанные параметры называются *доменными параметрами*.

Формирование подписи сообщения производится по такому алгоритму:

1. Выбор случайного числа $k \in (0, q)$.
2. Вычисление $r = (g^k \bmod p) \bmod q$.
3. Выбор другого k , если $r=0$.
4. Вычисление $s = k^{-1}(H(m)+x \cdot r) \bmod q$.
5. Выбор другого k , если $s=0$.
6. Подписью является пара (r, s) общей длины $2N$.

Проверка подписи включает в себя следующие шаги:

1. Вычисление $w = s^{-1} \bmod q$.
2. Вычисление $u_1 = H(m) \cdot w \bmod q$.
3. Вычисление $u_2 = r \cdot w \bmod q$.
4. Вычисление $v = (g^{u_1} \cdot y^{u_2} \bmod p) \bmod q$.
5. Подпись верна, если $v = r$.

1.4.3 Межгосударственный стандарт ГОСТ 34.10–2018

Наиболее актуальные для Российской Федерации алгоритмы формирования и проверки электронной подписи описаны в стандарте ГОСТ 34.10–2018 «Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи». Этот стандарт чаще всего используется в отечественных средствах по работе с ЭЦП.

Алгоритмы стандарта основаны на операциях в группе точек эллиптической кривой, определённой над конечным простым полем. Их криптографическая стойкость основывается на сложности вычисления дискретного логарифма в группе точек эллиптической кривой, а также на стойкости соответствующей хэш–функции. В качестве хэш–функции используется алгоритм, часто упоминаемый под названием «Стрибог», описанный в стандарте ГОСТ 34.11–2018 «Информационная технология. Криптографическая защита информации. Функция хэширования». Длина хэш–кода, получаемого с помощью этого алгоритма, может составлять 256 или 512 бит.

В результате подписания сообщения алгоритмом ГОСТ 34.10–2018 формируется цифровая подпись размером 512 или 1024 бит, которая затем добавляется к подписываемому сообщению. Помимо самой подписи стандартом предусмотрена возможность добавить текстовое поле произвольного содержания.

Например, там могут быть указаны автор подписи и дата её формирования.

Параметры схемы подписи:

- простое число p – модуль эллиптической кривой ($p > 2^{255}$);
- эллиптическая кривая (ЭК) E задаётся своим инвариантом $J(E)$ или коэффициентами $a, b \in F_p$, где F_p – конечное поле из p элементов. Инвариант $J(E)$ связан с коэффициентами a и b следующим образом:

$$J(E) = J(E) = 1728 \frac{4a^3}{4a^3 - 27b^2} \pmod{p}; \text{ при } 4a^3 - 27b^2 \not\equiv 0 \pmod{p};$$

- целое число m – порядок группы точек ЭК, m должно быть отлично от p ;
- простое число q , порядок некоторой циклической подгруппы группы точек эллиптической кривой, то есть выполняется соотношение $m = nq$, для некоторого $n \in \mathbb{N}$. Также q лежит в пределах $2^{254} < q < 2^{256}$;
- точка $P = (x^P, y^P)$ эллиптической кривой E , являющаяся генератором подгруппы порядка q , то есть $q \cdot P = \mathbf{0}$ и $k \cdot P \neq \mathbf{0}$ для всех $k = 1, 2, \dots, q-1$, где $\mathbf{0}$ – нейтральный элемент группы точек эллиптической кривой E ;
- $h(M)$ – хеш-функция (ГОСТ Р 34.11–2012), которая отображает сообщения M в двоичные векторы длины 256 (512) бит.

Каждый пользователь электронной цифровой подписи имеет личные ключи:

- ключ шифрования d – целое число, лежащее в пределах $0 < d < q$;
- ключ расшифрования $Q = (x_Q, y_Q)$, вычисляемый как $Q = d \cdot P$.

Дополнительные требования:

$$p^t \not\equiv 1 \pmod{q}, \forall t = 1..B, \text{ где } B \geq 31; J(E) \neq 0 \text{ и } J(E) \neq 1728.$$

Алгоритм формирования подписи представлен в виде блок-схемы на рисунке 1.3 и состоит из следующих шагов:

1. Вычисление хеш-функции от сообщения M : $\bar{h} = h(M)$.
2. Вычисление $e = z \bmod q$, и если $e = 0$, то положить $e = 1$, где z – целое число, соответствующее $\bar{h}$.
3. Генерация случайного числа k такого, что $0 < k < q$.
4. Вычисление точки ЭК $C = kP$, и по ней нахождение $r = x_C \bmod q$, где x_C – это координата x точки C . Если $r = 0$, то вернуться к предыдущему шагу.
5. Нахождение $s = (rd + ke) \bmod q$. Если $s = 0$, то вернуться к шагу 3.
6. Формирование ЭЦП $\xi = (\bar{r} | \bar{s})$, где $\bar{r}$ и $\bar{s}$ – векторы, соответствующие r и s .

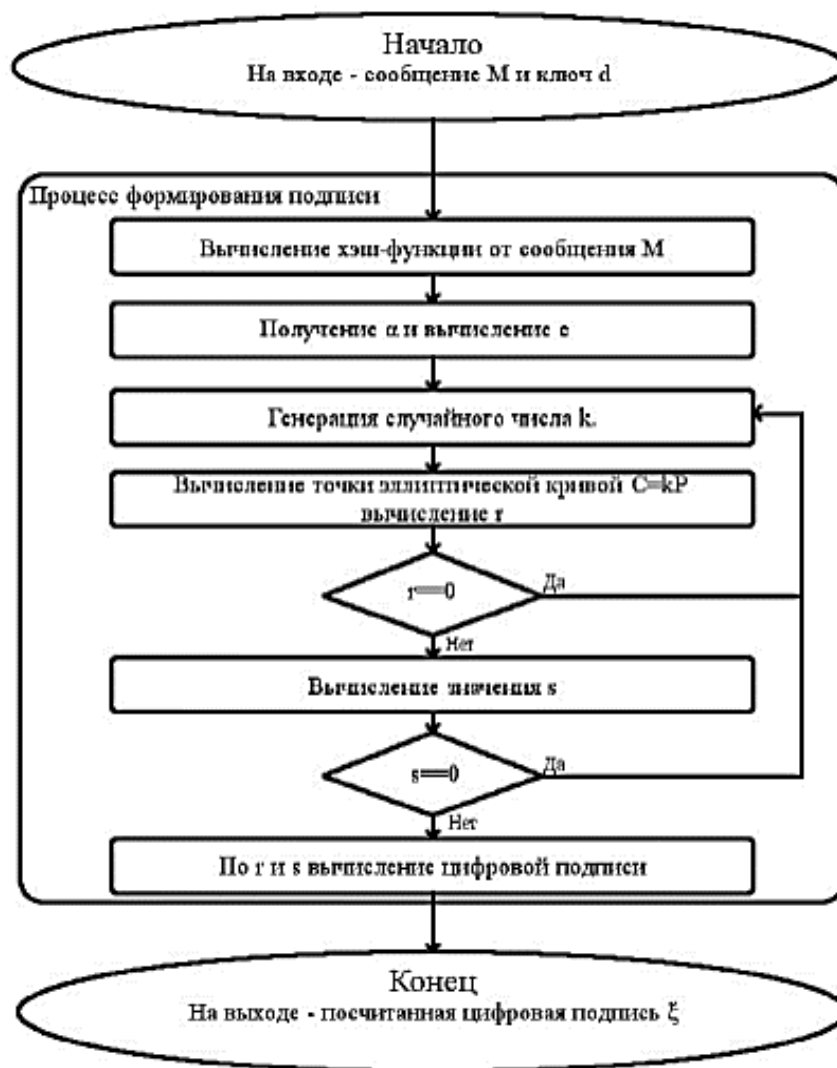


Рисунок 1.3 – Формирование электронной подписи по ГОСТ 34.10-2018

Алгоритм проверки электронной цифровой подписи представлен в виде блок-схемы на рисунке 1.4, и включает в себя следующие действия:

1. Вычисление по цифровой подписи ξ чисел r и s , учитывая, что $\xi = (\bar{r} | \bar{s})$, где r и s – числа, соответствующие векторам $\bar{r}$ и $\bar{s}$. Если хотя бы одно из неравенств $r < q$ и $s < q$ неверно, то и сама подпись не верна.
2. Вычисление хэш-функции от сообщения M : $\bar{h} = h(M)$.
3. Вычисление $e = z \bmod q$, и если $e=0$, то положить $e=1$, где z — целое число, соответствующее значению $\bar{h}$.
4. Вычисление $v = e^{-1} \bmod q$.
5. Вычисление $z_1 = sv \bmod q$ и $z_2 = -rv \bmod q$.
6. Вычисление точки эллиптической кривой $C = z_1 \cdot P + z_2 \cdot Q$ и определение $R = x_c \bmod q$, где x_c – координата x точки C .
7. В случае равенства $R = r$ подпись верна, иначе – не верна.

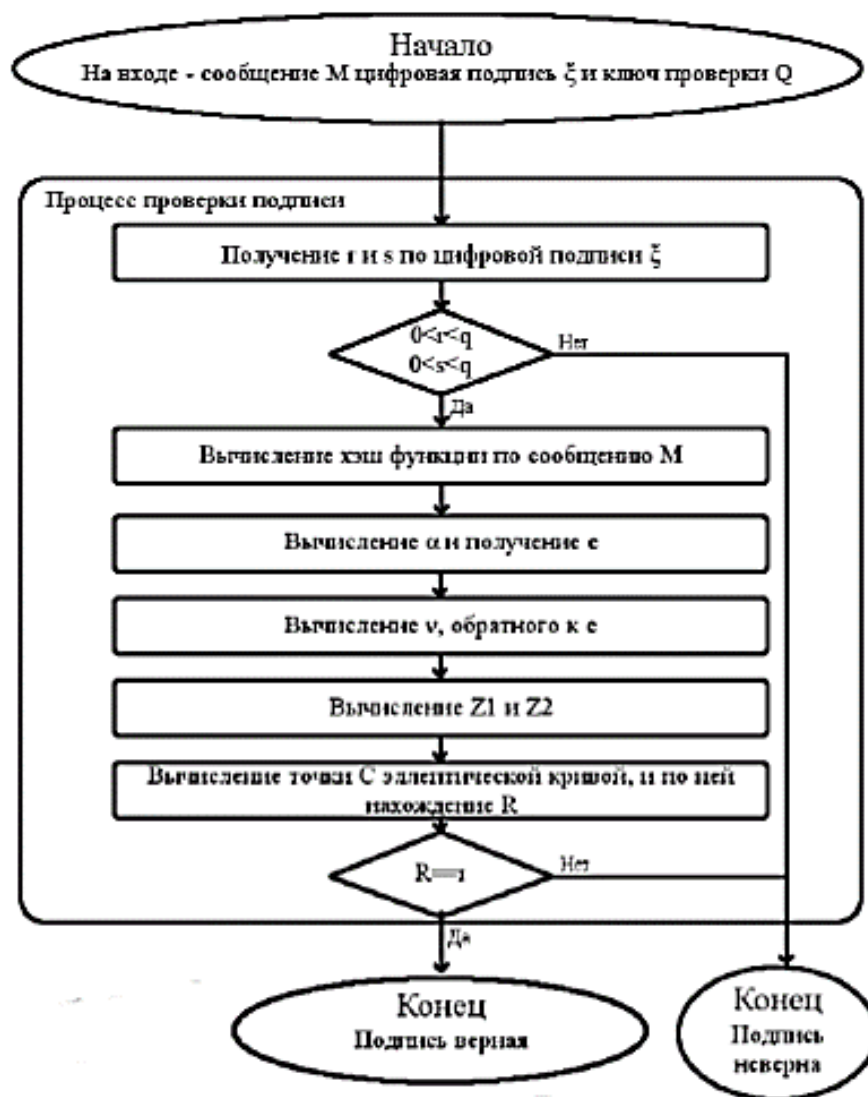


Рисунок 1.4 – Проверка электронной подписи по ГОСТ 34.10–2018

Выводы

В ходе работы по изучению теоретических основ электронной подписи, были рассмотрены определения собственноручной и электронной подписей, кратко приведена история развития электронной подписи, а также изучены классический алгоритм RSA, применяемый в США стандарт DSA и наиболее актуальный в Российской Федерации стандарт ГОСТ 34.10–2018.

При изучении правовых основ использования электронных подписей, были рассмотрены основные положения закона от 06.04.2011 № 63-ФЗ «Об электронной подписи», а также перечислены другие связанные с электронной подписью нормативные правовые акты Российской Федерации.

Таким образом, в результате работы над данным разделом была полностью выполнена первая из поставленных задач.

2 СУЩЕСТВУЮЩИЕ ПРОГРАММНЫЕ КОМПЛЕКСЫ ОБРАБОТКИ ЭЛЕКТРОННЫХ ПОДПИСЕЙ

2.1 КристоПро

ООО «КРИПТО–ПРО» – крупнейший поставщик криптографических услуг на российском рынке. Компания разрабатывает программные продукты, призванные решать самые разные задачи, в том числе и работу с ЭП.

КристоПро CSP 5.0 – новое поколение криптопровайдера, развивающее три основные продуктовые линейки компании КристоПро: КристоПро CSP (классические токены и другие пассивные хранилища секретных ключей), КристоПро ФКН CSP, Рутокен CSP (неизвлекаемые ключи на токенах с защищенным обменом сообщениями) и КристоПро DSS (ключи в облаке) [36].

Все преимущества продуктов этих линеек не только сохраняются, но и преумножаются в КристоПро CSP 5.0: шире список поддерживаемых платформ и алгоритмов, выше быстродействие, удобнее пользовательский интерфейс. Главное – единообразная работа со всеми ключевыми носителями, включая ключи в облаке. КристоПро CSP позволяет решать следующие задачи:

- формирование и проверка электронной цифровой подписи;
- обеспечение конфиденциальности и контроля целостности информации посредством её шифрования и имитозащиты;
- обеспечение аутентичности, конфиденциальности и имитозащиты данных при соединении по протоколам TLS, и IPsec;
- контроль целостности системного и прикладного ПО для его защиты от несанкционированных изменений и нарушений доверенного функционирования.

Поддерживается множество криптографических алгоритмов, в том числе известные криптосистемы RSA, ECDSA, ГОСТ 34.10–2012.

Минимальная цена на программу «КристоПро CSP» версии 5.0 составляет 2700 рублей за лицензию на право использования программы на одном рабочем месте.

Помимо основного продукта, фирма поставляет и множество других, вспомогательных решений, не только программных, но и программно–аппаратных.

Например, КристоПро HSM 2.0 – это высокопроизводительный программно–аппаратный криптографический модуль, снабженный датчиками вскрытия, защитой ключевой информации с использованием разделенных по схеме «3 из 5» ключей, доверенной операционной системой, безопасными механизмами аудита и контроля подключений, обеспечивающий защиту криптографических ключей класса КВ2 (Приказ ФСБ РФ от 27 декабря 2011 г. № 796 «Об утверждении Требований к средствам электронной подписи и Требований к средствам удостоверяющего центра»).

Существует и другое программно–аппаратное решение – КристоПро DSS, предназначенный для централизованного, защищенного хранения закрытых ключей пользователей, а также для удаленного выполнения операций по созданию электронной подписи (ЭП) с использованием ПАКМ КристоПро HSM.

КристоПро DSS предоставляет пользователям интерактивный Web–интерфейс для управления криптографическими ключами и создания электронной подписи под документами, которые пользователь загружает на КристоПро DSS. Для работы с таким интерфейсом в случае неквалифицированной подписи необходим лишь Web–браузер.

2.2 КристоАРМ

КристоАРМ – программа, разрабатываемая ООО «Цифровые технологии» (Йошкар-Ола), и предназначенная для шифрования и расшифрования данных, создания и проверки электронной подписи с использованием сертификатов открытых ключей, для работы с сертификатами и криптопровайдерами. «КристоАРМ», наряду со стандартными криптопровайдерами (входящими в поставку Windows), использует реализацию криптоалгоритмов в сертифицированных ФСБ РФ криптопровайдерах компании «КРИПТО–ПРО» [37].

Используя криптопровайдер «КристоПро CSP», программа «КристоАРМ» позволяет работать с сертифицированными средствами, создавать электронную цифровую подпись, равнозначную собственноручной.

Функциональные возможности КристоАРМ:

- шифрование и расшифрование файлов произвольного формата преобразования файлов функциями системы криптографической защиты информации (СКЗИ);
- создание и проверка корректности одной или нескольких ЭЦП;
- выполнение операций подписи и шифрования за одно действие;
- управление цифровыми сертификатами и ключами пользователя, списками отозванных и доверенных сертификатов;
- управление криптопровайдерами;
- совместимость с отчуждаемыми ключевыми носителями Рутокен, eToken;
- отправка подписанных и зашифрованных файлов по e–mail.

Существует несколько вариантов и комплектаций программы КристоАРМ, различающихся как по предоставляемым возможностям, так и по цене.

«КристоАРМ Старт» – бесплатная версия, не требующая ввода лицензионного ключа. Предоставляет возможность выполнения операций подписи и шифрования с использованием стандартных криптопровайдеров, входящих в поставку Windows.

При наличии установленного СКЗИ «КриптоПро CSP» можно проверить ЭЦП на электронном документе (но не поддерживается функция создания ЭЦП).

«КриптоАРМ Стандарт» – базовая версия: шифрование и расшифрование файлов произвольного формата, создание и проверка корректности ЭЦП, полностью поддерживает СКЗИ «КриптоПро CSP».

«КриптоАРМ Стандарт PRO» включает в состав дополнительные модули TSP и OSCP, модуль Усовершенствованной подписи.

2.3 Общие черты существующих программных средств

Программы обеих рассмотренных фирм соответствуют требованиям российского законодательства, поддерживают различные алгоритмы и форматы подписей. В общем виде предлагаемые решения состоят из:

- программно–аппаратного комплекса для защищённого хранения ключей ЭП;
- серверной части для настройки собственного удостоверяющего центра, либо для централизованного защищённого создания и проверки подписей;
- клиентского приложения, с которыми при операциях с электронной подписью непосредственно взаимодействуют пользователи информационной системы.

При этом, в случае использования локальных решений, предполагающих наличие ключей электронной подписи у их владельцев, например, на смарт–карте, возникают некоторые ограничения и неудобства.

Очевидное неудобство – необходимость каждого владельца подписи держать ключ при себе, и самостоятельно обеспечивать контроль за его безопасностью. Также данные решения предполагают установку на каждое рабочее место специального программного обеспечения, которое на этом же месте и будет формировать электронные цифровые подписи. Это означает, что установка, настройка, а также эксплуатация (создание подписей) могут быть нетривиальны.

Облачные решения, например, «КриптоПро DSS», более удобны для конечного пользователя, так как избавляют его от необходимости следить за сохранностью физического ключа и использовать не всегда интуитивно понятное ПО, установленное на личном рабочем месте.

Однако, облачные решение вызывают небезосновательные вопросы к безопасности, сохранности и конфиденциальности хранимых удалённо, на чужом оборудовании, ключей электронных подписей, а также вызывают сомнения в доверии к подписям, сформированным удалённо [38].

Выводы

Проанализированы самые крупные и популярные в России игроки на рынке программного обеспечения в области работы с электронными подписями, а именно, КристоПро и КристоАРМ. Выявлены их достоинства и недостатки.

И классические, и облачные схемы электронной подписи давно применяются на практике, но облачная схема пользуется меньшим доверием с точки зрения безопасности, в противовес своему главному достоинству – простоте использования.

Криптопровайдер КристоПро CSP выполняет следующие функции:

- создание и верификацию электронной цифровой подписи;
- обеспечение конфиденциальности и контроля целостности информации с помощью её шифрования и имитозащиты (защиты системы шифровальной связи или другой криптосистемы от навязывания ложных данных);
- обеспечение аутентичности, конфиденциальности и имитозащиты данных при соединении по протоколам TLS, и IPsec;
- контроль целостности системного и прикладного ПО для его защиты от несанкционированных изменений и нарушений доверенного функционирования.

Минимальная цена на программу «КристоПро CSP» версии 5.0 составляет 2700 рублей за лицензию на право использования программы на одном рабочем месте.

Функциональные возможности программы КристоАРМ:

- шифрование и расшифрование файлов произвольного формата преобразования файлов функциями системы криптографической защиты информации (СКЗИ);
- создание и проверка корректности одной или нескольких ЭЦП;
- выполнение операций подписи и шифрования за одно действие;
- управление цифровыми сертификатами и ключами пользователя, списками отозванных и доверенных сертификатов;
- управление криптопровайдерами (Cryptography Service Provider, CSP), т.е. независимыми модулями, позволяющими осуществлять криптографические операции в операционных системах линейки Microsoft Windows, управление которыми происходит с помощью функций CryptoAPI;
- совместимость с отчуждаемыми ключевыми носителями Рутокен, eToken;
- отправка подписанных и зашифрованных файлов по e-mail.

«КристоАРМ Старт» – бесплатная версия, не требующая ввода лицензионного ключа. Предоставляет возможность выполнения операций подписи и шифрования с использованием стандартных криптопровайдеров, входящих в поставку Windows.

Таким образом, решена вторая задача данного исследования, состоящая в рассмотрении и анализе существующих программных продуктов обработки ЭП, применяемых в Российской Федерации.

3 РАЗРАБОТКА ИМПОРТОЗАМЕЩАЮЩЕГО ПРОГРАММНОГО КОМПЛЕКСА ФОРМИРОВАНИЯ ЭЛЕКТРОННОЙ ЦИФРОВОЙ ПОДПИСИ

3.1 Высокоуровневая архитектура программного комплекса

Компоненты комплекса ЭЦП предлагается разработать на языке программирования Python. Этот язык широко распространён и используется в системах различной сложности, в том числе и на предприятии прохождения преддипломной практики. Он также является и отличным языком для учебных целей, так как обладает простым синтаксисом. Отдельное преимущество языка программирования Python – динамическая, но достаточно строгая типизация. Разработчик может не указывать используемые типы данных явно, но при несовпадении типов в какой-либо операции, интерпретатор языка об этом сообщит.

Графический интерфейс целесообразно разработать в виде Web-страницы. Такой подход обеспечит максимальный охват поддерживаемых устройств, так как любой современный компьютер и смартфон позволяют просматривать Web-страницы. Для разработки подобного интерфейса следует использовать языки разметки гипертекста JavaScript и HTML, а также таблицы стилей CSS.

Поскольку облачная электронная подпись является наиболее удобным и предпочтительным вариантом для конечного пользователя, то представляется разумным выбор именно такой базовой архитектуры разрабатываемого комплекса.

Несмотря на привлекательность, облачная схема подвергается критике ИТ-специалистов из-за удалённости всех секретных данных от их владельцев. Чтобы увеличить уровень доверия к облачной схеме формирования подписей, возможно добавить в неё дополнительный элемент, повышающий её надёжность.

Такой элемент – открытое хранилище сформированных подписей. Поскольку подписи и сертификаты ключей их проверки не являются секретной информацией, их не нужно защищать от прочтения любым лицом. Наоборот, они должны быть доступны всем желающим, чтобы они могли убедиться в том, что проверяемые ими подписи действительно верны и были легитимно сформированы в рассматриваемой системе. Однако, они должны быть защищены от изменения злоумышленником. Дополнительное свойство хранилища, которое могло бы увеличить пользу от его использования, а также повысить доверие к системе в целом – это возможность сохранять достоверное время его наполнения, или, другими словами, время формирования подписей.

Для достижения всех перечисленных свойств отлично подходит технология блокчейн. Предлагаемая схема облачной электронной подписи графически показана на рисунке 3.1. Она состоит из следующих частей:



Рисунок 3.1 – Общая структура предлагаемого программного комплекса

- защищённый криптографический модуль, производящий криптографические операции с документами, подписями и ключами подписей;
- защищённое хранилище ключей электронных подписей, входящее в состав криптографического модуля;
- блокчейн–хранилище сформированных подписей;
- центральный Web–сервер;
- пользовательский Web–интерфейс.

3.1.1 Криптографический модуль и защищённое хранилище ключей

Закрытые ключи подписей хранятся защищённо на стороне сервиса, «в облаке». Безопасность ключей обеспечивается шифрованием и всего хранилища, и каждого ключа отдельно. Доступ к определённому ключу есть только у его владельца, прошедшего процедуры идентификации и аутентификации. Никто, кроме самой системы, не обладает доступом к закрытым ключам. Даже владельцы ключей инициируют создание своих ключей по факту регистрации в системе, но самих ключей при этом не получают. Они могут только управлять их использованием, посредством авторизации в системе.

Процедура подписания документа происходит «в облаке», в криптографическом модуле. Она показана на рисунке 3.2. Чтобы подписать документ, пользователь должен пройти процедуру авторизации. Только при успешном её прохождении система сможет понять, каким именно ключом должен быть подписан документ. Сформированная подпись не только отправляется пользователю и добавляется в блокчейн–хранилище подписей. Помимо самой битовой последовательности подписи, в блокчейн добавляется и сертификат её открытого ключа, содержащий данные о владельце подписи, а также о сроках её действия.

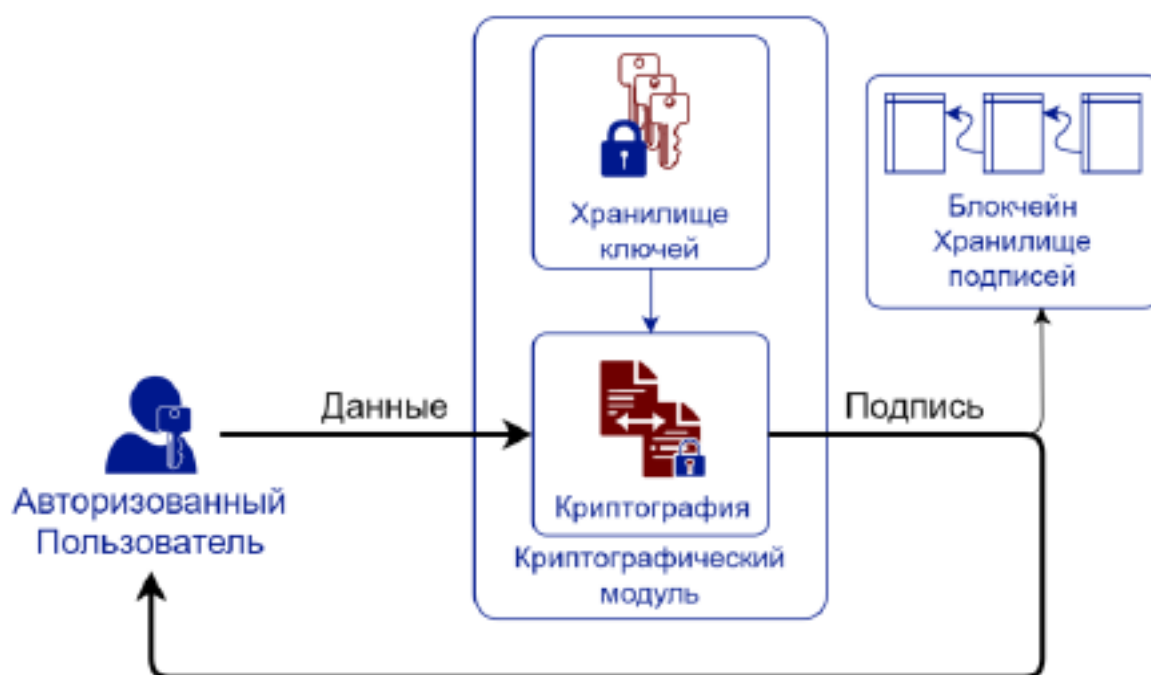


Рисунок 3.2 – Схема формирования электронной подписи

Как в подписях, так и в блоках подписей сохраняется информация о времени создания, что должно дополнительно увеличивать уровень доверия. Для проверки, помимо обычной процедуры сравнения хэшей, загруженная пользователем подпись отыскивается в блокчейне, и только в случае её нахождения, подпись считается верной и легитимно сформированной. Если же подпись оказывается верной по обычной процедуре проверки, но отсутствует в хранилище подписей, то этот факт служит автоматическим сигналом о возможной компрометации системы.

3.1.2 Блокчейн–хранилище подписей

Блокчейн (англ. Blockchain), что в переводе с английского языка означает «цепочка блоков», представляет собой структуру, обладающую всеми свойствами связного списка, а также рядом дополнительных свойств.

Во–первых, каждый блок связан со всеми предыдущими определённой закономерностью. Обычно эта закономерность состоит в том, что хэш–суммы всех блоков соответствуют определённому шаблону или правилу, и при этом хэш–сумма каждого последующего блока вычисляется от последовательности данных, обязательно включающих в себя хэш–сумму предыдущего блока. Именно в этой связи и проявляется «цепочка» блоков.

Во–вторых, каждый блок формируется с помощью решения определённой задачи, направленной на соблюдение закономерности, и требующей для решения затрат определённого количества вычислительных ресурсов.

Примером закономерности может служить условие: «первые 32 бита хэш–суммы блока являются нулевыми». Пример хэш–суммы, соблюдающей это правило:

000000002d1ef7ab4d9273d1906381b44f2f6f2588a3efb96a49188331984753

Задачей, направленной на соблюдение закономерности, является нахождение *примеси*, при добавлении которой к данным хэш–сумма примет приведённый вид. Решение таких задач называют «*майнингом*», то есть «добыванием» блоков.

Проверить соблюдение закономерности в любом блоке – простая задача. Поэтому любой пользователь, просматривающий блокчейн, может легко убедиться в целостности данных, содержащихся в нём.

В случае нарушения этой закономерности в определённом блоке, он и все последующие блоки считаются недействительными, поскольку каждый бит информации блока участвует в формировании хэшей всех последующих блоков. Изменение любого бита в блоке $N-2$ цепи из N блоков повлечёт изменение хэш сумм блоков $N-2$, $N-1$ и N . Из этого вытекают два важнейших последствия:

1. Для злоумышленного изменения содержащихся в блокчейне данных нужно заново решить задачу на соблюдение закономерности в изменяемом блоке, а также во всех последующих блоках. Выполнение такого массива вычислений может, должно быть и обычно является слишком сложной задачей для злоумышленника.
2. Чем больше блоков в блокчейне, тем сложнее подменить любой из ранних блоков.

Теоретически, злоумышленник может обладать вычислительной мощностью, достаточной, чтобы подменить весь необходимый «хвост» блокчейна. Чтобы исключить такую возможность, блокчейн–хранилища обычно организуют не централизованно, на одном сервере, а распределённо. Копия блокчейна находится в каждом узле сети, и, если в каком–то из узлов блокчейн перестаёт совпадать с копиями в других узлах, то правильными считаются копии большинства узлов. Чем больше число копий блокчейна, т.е. чем шире сеть «майнеров», тем меньше шансы злоумышленника на успех. Например, в блокчейн–сети криптовалюты Bitcoin «майнером» может стать любой желающий. Присоединяясь к сети, участник добавляет свои вычислительные ресурсы к общей распределённой системе, и тем самым снижает долю, значимость ресурсов злоумышленника, а значит, уменьшается и вероятность успеха его злонамеренных действий.

Структура блокчейн–хранилища подписей, применяемого в разрабатываемой облачной системе, представлена на рисунке 3.3. Как видно из рисунка, каждый блок может содержать неограниченное, но не нулевое количество подписей. В нём также обязательно содержатся собственные сведения, такие как собственный хэш блока, хэш предыдущего блока, примесь (соль) для формирования собственного хэша, дату формирования и идентификатор, и другие.

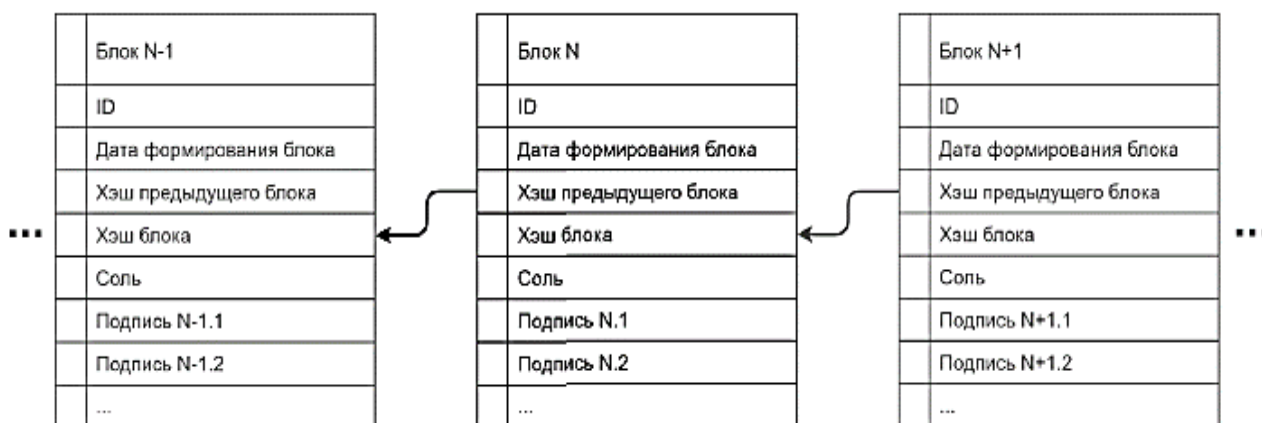


Рисунок 3.3 – Структура блокчейн–хранилища подписей

3.1.3 Центральный Web–сервер и Web–интерфейс

Все компоненты системы разработаны независимыми, общаются между собой посредством http–запросов, через лаконичный программный интерфейс REST. Это обеспечивает возможность масштабирования системы, необходимого прежде всего для построения распределённого блокчейн–хранилища подписей.

Взаимодействие всех элементов системы обеспечивается центральным Web–сервером. Он обрабатывает http–запросы пользователя на регистрацию, подписание и проверку подписей, а после обработки перенаправляет полученные данные и запросы криптографическому модулю или хранилищу подписей. Только этот сервер доступен конечным пользователям. Интерфейс остальных элементов системы доступен только центральному Web–серверу, но закрыт для доступа извне.

С точки зрения пользователя, всё взаимодействие с системой сводится к заполнению формы на Web–странице. Причём, для проверки подписи, необходимо просто загрузить подписанный файл и подпись. А для подписания, кроме загрузки документа нужно также пройти процедуру авторизации, то есть ввести логин и пароль, чтобы «указать» сервису используемый ключ электронной подписи и подтвердить владение им. Подробно Web–интерфейс будет рассмотрен далее, при демонстрации работы разработанного программного комплекса.

3.2 Основные программные функции

Запуск программы осуществляется с помощью консольной команды, которую в общем виде можно представить следующим образом:

```
python __main__.py -a {rsa, dsa, gost} -k {key}
```

Первый аргумент (`-a` или `--algo`) принимает один из трёх возможных значений, соответствующих поддерживаемым алгоритмам электронной подписи (RSA, DSA, ГОСТ 34.10-2018), а второй аргумент (`-k` или `--key`) – ключ шифрования, которым шифруется хранилище ключей подписей.

Например, чтобы запустить программный комплекс так, чтобы ключи подписей всех вновь зарегистрированных пользователей, а в последствии и сами подписи, формировались по алгоритму DSA, а хранилище ключей шифровалось с ключом «01234689abcdef», необходимо запустить следующую команду:

```
python __main__.py -a dsa -k 01234689abcdef
```

Данная команда запускает программу, которая обрабатывает входные аргументы и затем порождает три подпроцесса: процесс центрального Web-сервера, процесс криптографического модуля и процесс блокчейн-хранилища подписей. Такой способ запуска удобен для первичного знакомства и демонстрации работы программного комплекса. Однако, для реального использования этого способа стоит избегать. Каждый из указанных подпроцессов должен быть запущен отдельно, желательно, на физически разных компьютерах.

Рассмотрим основные программные функции, играющие ключевую роль в работе комплекса. Полный текст данных функций далее.

get_args() – функция, обрабатывающая аргументы командной строки, и возвращающая обработанные аргументы в виде словаря (структура *dict* языка Python).

register() – функция обработки запроса на регистрацию нового пользователя. Как и во всех других функциях обработки http-запросов, прежде всего, происходит валидация запроса, то есть проверка корректности входных данных. Если данные некорректны, функция не выполняет свои основные операции, а возвращает http-ошибку с кодом «400 Bad request». Если переданные данные корректны, в данном случае это логин и пароль, функция обращается к методу *add_key* модуля *keys_storage*, передавая логин и пароль, и получая в ответ двоичное представление файла сертификата только что созданного ключа электронной подписи. Сам ключ подписи формируется, сохраняется и используется только в рамках хранилища ключей, которым и является модулем *keys_storage*. Полученный сертификат отправляется пользователю, что для него сигнализирует об успешной регистрации.

add_key(self, login: str, raw_pw: str) – метод модуля *keys_storage* для создания и сохранения нового ключа электронной подписи, а также формирования сертификата ключа проверки подписи. Ключ генерируется по алгоритму, выбранному при запуске криптографического модуля.

sign() – функция обработки криптографическим модулем запроса на подписание документа. Сначала производится авторизация пользователя. В случае успешной авторизации, ключ подписи пользователя отыскивается в защищённом хранилище и расшифровывается. Далее ключ и данные для подписания передаются в модуль криптографических операций, используя специальную функцию *_sign(private_key, data_bytes)*. Модуль операций сам определяет, какому алгоритму принадлежит переданный ключ, и применяет именно этот алгоритм. Сформированная подпись вместе с сертификатом открытого ключа проверки подписи отправляется в блокчейн-хранилище подписей с помощью *http*-запроса. Затем готовая подпись пересылается в виде скачиваемого файла пользователю, запросившему подписание документа.

verify() – функция обработки запроса на проверку подписи. В отличие от предыдущих операций, данная не требует авторизации. Любой желающий может успешно произвести данный запрос, отправив подписанные данные и подпись. Загруженная подпись, прежде всего, отправляется в блокчейн-хранилище подписей в рамках запроса на поиск подписи в блокчейне. Если подпись была сформирована данной системой, она присутствует и в блокчейне, и поэтому будет отправлена вместе с сертификатом ключа проверки подписи в ответе на запрос поиска подписи. Если же подпись в блокчейне не найдена, она считается недействительно, о чем система оповещает пользователя. Если подпись найдена, то теперь в распоряжении криптографического модуля имеются и подпись, и сертификат ключа проверки данной подписи, содержащий и сам ключ проверки. Все эти данные отправляются в модуль криптографических операций для подтверждения или опровержения действительности проверяемой подписи. Результат проверки, состоящий в заключении о верности или неверности подписи, отправляется пользователю, запросившему проверку.

3.3 Демонстрация работы программного комплекса

Полный цикл работы с программным комплексом начинается с его запуска консольной командой, описанной ранее. Для демонстрации, как показано на рисунке 3.4, запустим его в режиме работы с алгоритмом RSA и следующим ключом шифрования хранилища подписей:

d8f0e535fccad9fd4ee11cfd3cf1138cd9d53e13bebddcd2daf993c47c014b8e

```
nikita@nikita-mint ~/dev/learn/digi-signature
Файл Правка Вид Поиск Терминал Справка
(ds) nikita@nikita-mint ~/dev/learn/digi-signature $ python __main__.py --algo rsa --key d8f8e535fccad9fd4ee11cfd3cf1138cd9d53e1
3bebdddcd2daf993c47c814b8e
* Serving Flask app "control_server.control_server" (lazy loading)
* Serving Flask app "signatures_storage.storage_server" (lazy loading)
* Environment: production
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
WARNING: This is a development server. Do not use it in a production deployment.
* Debug mode: off
Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:8989/ (Press CTRL+C to quit)
* Running on http://127.0.0.1:8990/ (Press CTRL+C to quit)
* Serving Flask app "secure_crypto.secure_crypto_server" (lazy loading)
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:8991/ (Press CTRL+C to quit)
Окно или вкладка откроются в текущем сеансе браузера.
Окно или вкладка откроются в текущем сеансе браузера.
```

Рисунок 3.4 – Запуск системы

Программа, работающая в демонстрационном режиме, автоматически откроет и графический интерфейс пользователя в новом окне браузера. Но на данный момент в системе не зарегистрировано ни одного пользователя. Чтобы создать тестового пользователя, необходимо перейти к интерфейсу администратора, по адресу /admin. Данный интерфейс показан на рисунке 3.5.

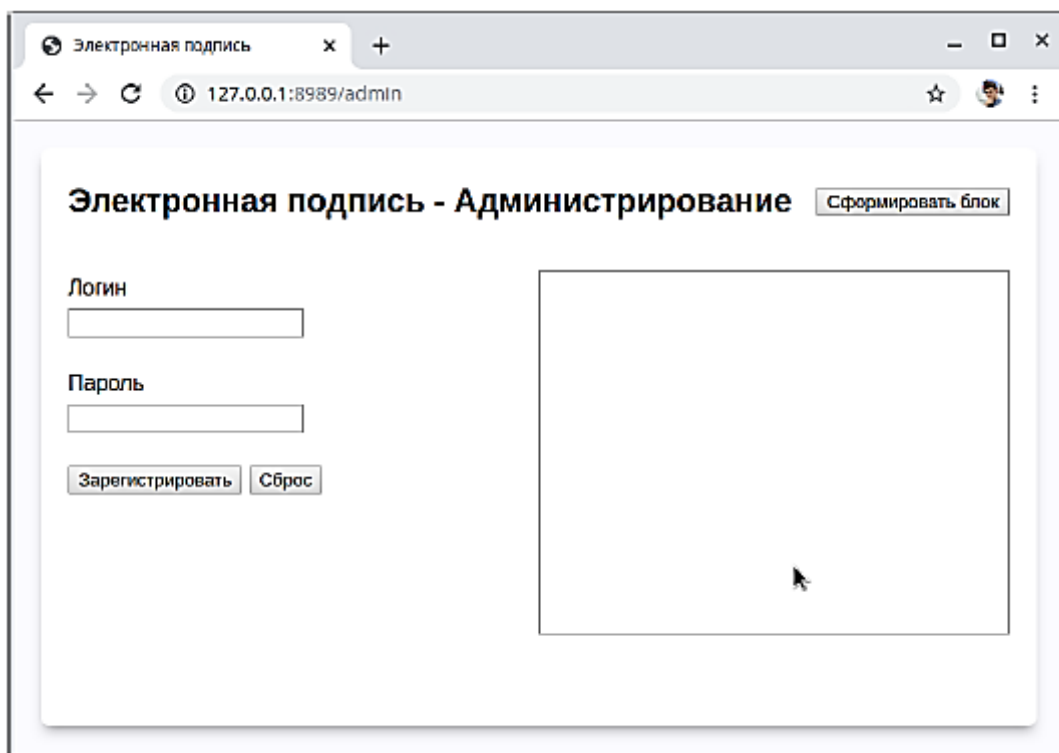


Рисунок 3.5 – Интерфейс администратора

Для регистрации пользователя необходимо ввести логин и пароль, а затем нажать кнопку «Зарегистрировать». При успешной регистрации, в текстовом поле в правой части интерфейса появится соответствующее сообщение, а под формой ввода логина и пароля появится ссылка для скачивания сертификата ключа проверки подписи созданного только что ключа электронной подписи, как показано на рисунке 3.6. Регистрация может и не получиться, например, если в системе уже был зарегистрирован пользователь с введённым именем.

В таком случае, в правой части программного интерфейса появится сообщение с рекомендацией пользователю выбрать другой логин.

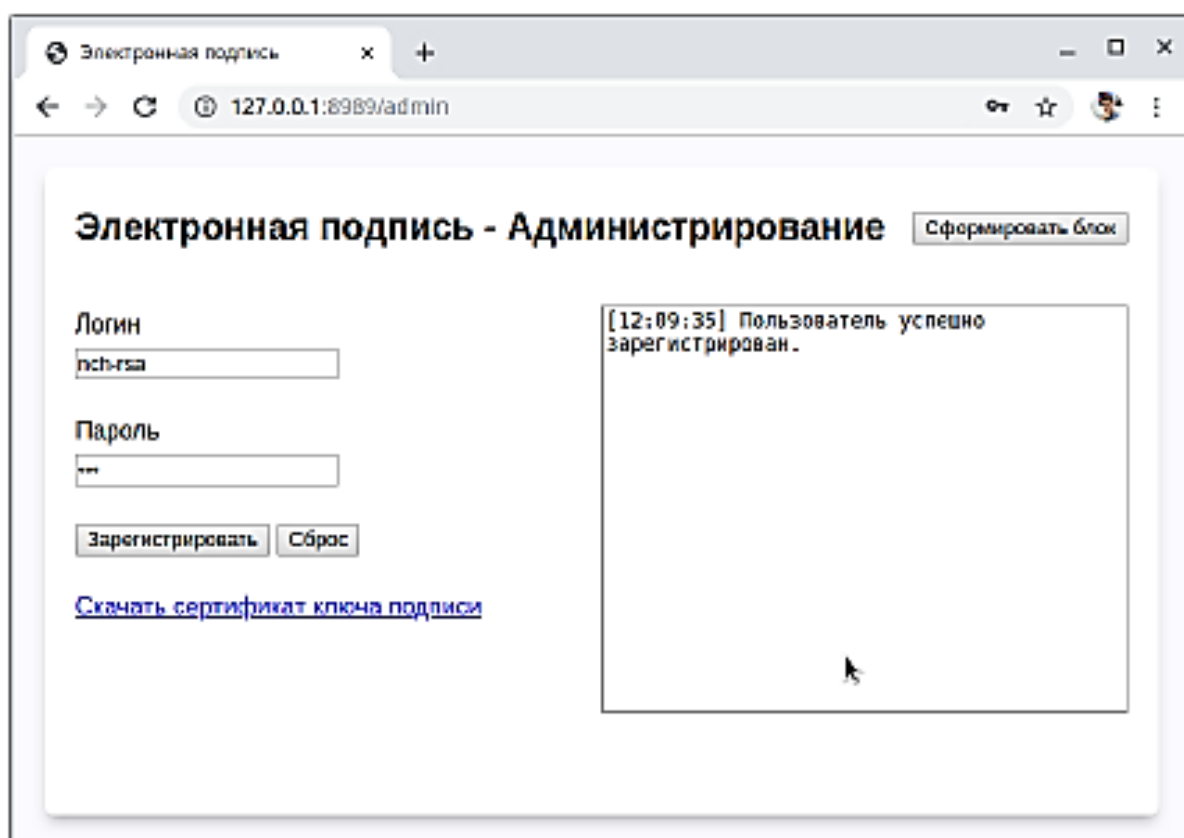


Рисунок 3.6 – Успешная регистрация нового пользователя

Интерфейс администратора содержит также и кнопку «Сформировать блок». Она позволяет принудительно запустить формирование нового блока в блокчейн-хранилище подписей. Данная функция требуется достаточно редко, так как блоки в блокчейн-хранилище формируются автоматически, через определённый, заранее настраиваемый промежуток времени. Содержимое сертификата ключа проверки подписи показано на рисунке 3.7.

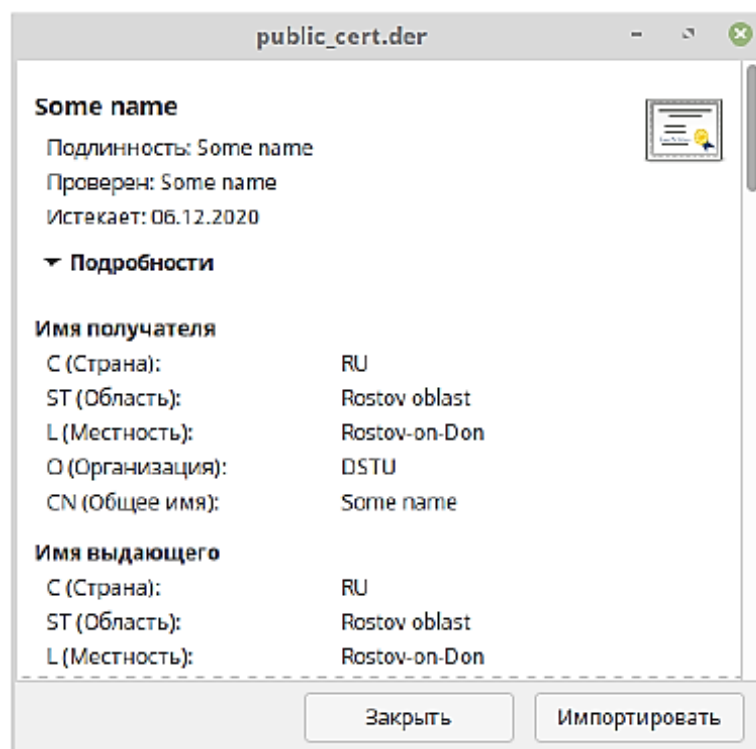


Рисунок 3.7 – Сертификат ключа подписи зарегистрированного пользователя

Чтобы убедиться в сохранности сформированного ключа электронной подписи, можно попробовать просмотреть хранилище ключей. Как видно на рисунке 3.8, данные в хранилище совершенно не читаемы, и при попадании к злоумышленнику окажутся для него бессмысленным набором данных.



Рисунок 3.8 – Зашифрованное хранилище ключей подписей

Теперь, когда пользователь зарегистрирован, можно перейти к интерфейсу пользователя, показанному на рисунке 3.9. Данный интерфейс похож на интерфейс администратора, но на месте кнопки формирования блока содержит переключатель режимов работы, помимо текстовых полей ввода логина и пароля содержит поле загрузки подписываемых данных, а вместо кнопки «Зарегистрировать» содержит кнопку «Подписать».

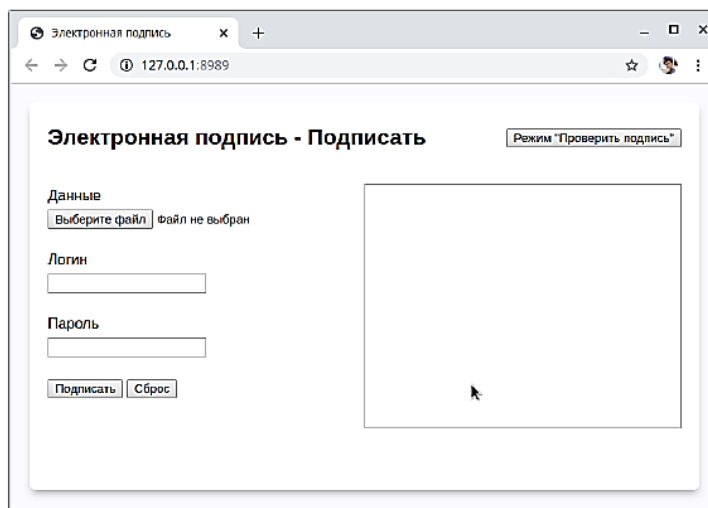


Рисунок 3.9 – Интерфейс пользователя в режиме формирования подписи

Для подписания может быть загружен файл любого формата. Для примера взято изображение с именем «img.png». При корректном заполнении формы, после нажатия кнопки «Подписать», интерфейс показывает сообщение об успешном подписании, а также ссылку для скачивания сформированной подписи, как показано на рисунке 3.10.

В процессе формирования подписи, согласно разработанной схеме, она не только отправляется пользователю, но и сохраняется в блокчейн-хранилище. В этом можно убедиться, просмотрев данное хранилище, как показано на рисунке 3.11.

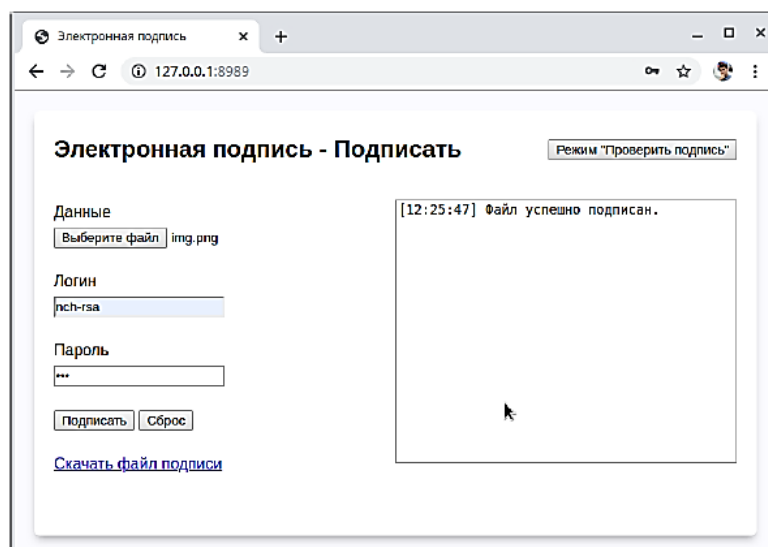


Рисунок 3.10 – Успешное формирование подписи

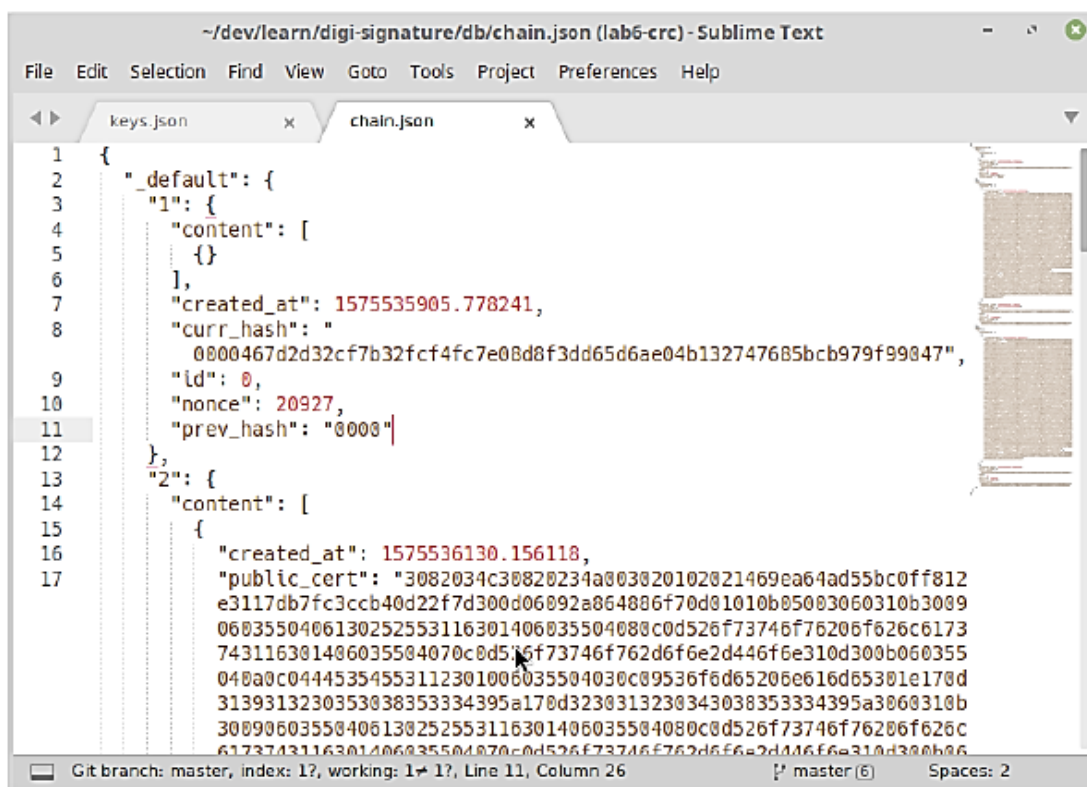


Рисунок 3.11 – Содержимое блокчейн-хранилища подписей

Скачаем подпись и с помощью кнопки переключения режимов перейдём к интерфейсу проверки подписи. В данном режиме интерфейс упрощается, и требует только загрузки подписанных данных и подписи. Вместо кнопки «Подписать», интерфейс в данном режиме содержит кнопку «Проверить подпись». Загрузив ранее подписанный документ и скачанную подпись, и запустив проверку подписи, при успешном её осуществлении, в правой части интерфейса появится заключение о верности или неверности подписи, как показано на рисунке 3.12.

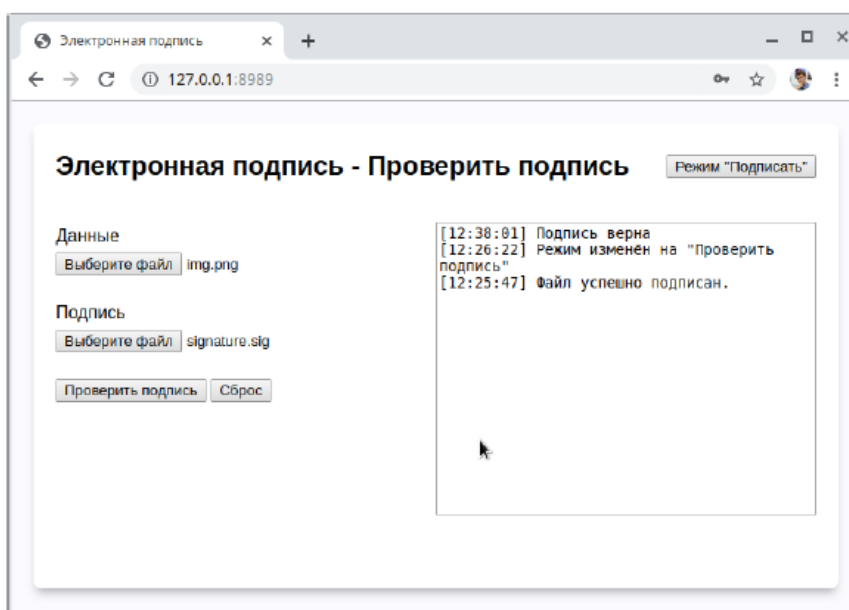


Рисунок 3.12 – Интерфейс пользователя в режиме проверки подписи

Выводы

В ходе работы над этим разделом была спроектирована архитектура итогового программного комплекса, состоящая из пользовательского интерфейса, центрального Web-сервера, криптографического модуля, в состав которого входят модуль криптографических операций и защищённое хранилище ключей электронных подписей, и блокчейн-хранилище сформированных подписей. Все эти элементы спроектированы с расчётом на масштабирование и расширяемость. Они могут работать независимо друг от друга, а общение между ними происходит с помощью http-запросов.

В рамках демонстрации полного цикла работы с программой показаны простота и производительность работы с системой. Для полноценной работы всего программного комплекса достаточно выполнить одну консольную команду с двумя аргументами: алгоритмом электронной подписи и ключом шифрования хранилища подписей. Продемонстрирован, в тестовом режиме, графический интерфейс пользователя и администратора системы.

Таким образом, в результате работы над этим разделом исследования были решены две последние поставленные задачи, состоящие в разработке архитектуры и алгоритмов работы итогового программного средства, а также в его практической реализации и проверке работоспособности.

РЕЗУЛЬТАТЫ 5 ГЛАВЫ

В предлагаемой работе реализован импортозамещающий программный комплекс, реализующий облачную схему работы с электронной подписью, и включающий в себя дополнительный элемент, призванный увеличить уровень доверия как к разработанному комплексу в целом, так и к каждой сформированной с его помощью подписи отдельно.

Проведена исследовательская работа, в ходе которой были изучены и проанализированы как общие теоретические и правовые аспекты использования электронных подписей, так и частные программные реализации различных схем работы с электронной цифровой подписью, пользующиеся в государственных учреждениях России наибольшим спросом и коммерческим успехом.

В ходе исследовательской работы было определено, что предпочтительной, но не пользующейся достаточным доверием является облачная схема формирования и проверки электронной подписи. Было решено произвести попытку устранения недостатка, состоящего в недоверии к облачной электронной подписи, путём добавления дополнительного элемента – блокчейн-хранилища для сформированных системой электронных подписей.

Сформулирована архитектура системы, направленная как на увеличения доверия к самой себе, так и на простоту настройки и эксплуатации. Запуск всей системы осуществляются одной консольной командой. Графический интерфейс является доступным для использования широким кругом лиц, так как реализован в виде Web-страницы, а значит, может быть использован на самых разных устройствах. Интерфейс прост и включает в себя минимально возможный набор элементов, что упрощает взаимодействие с ним. Каждый из компонентов рассчитан как на работу в системе, так и на самостоятельное существование, что обеспечивает простоту масштабирования и расширяемости всей системы, что особенно важно для блокчейн – хранилища электронных подписей

Программный комплекс работает корректно и может быть использован при организации электронного документооборота в любой отечественной организации, допускающей применение неквалифицированной электронной подписи.

ГЛАВА 6

РЕАЛИЗАЦИЯ ИМПОРТОЗАМЕЩАЮЩЕГО КРИПТОГРАФИЧЕСКОГО АЛГОРИТМА ДЛЯ ШИФРОВАНИЯ ДАННЫХ В СУБД SQL SERVER

ОБЩИЕ ПОЛОЖЕНИЯ

Немаловажным аспектом безопасности в отечественных предприятиях, использующих сервера с системами управления базами данных, является резервное копирование баз данных. Существует множество способов реализации безопасного резервного копирования. Один из возможных способов – это создание зашифрованной резервной копии с помощью алгоритмов шифрования.

Система управления базами данных MS SQL Server предполагает несколько алгоритмов шифрования, доступных российскому пользователю, и среди них есть такие стандарты шифрования, как «AES» и «TDES».

Однако перечисленные стандарты шифрования не являются современными. Стандарт «TDES» был выпущен в 1978 г., а стандарт «AES» – появился в 1998 г.

В России стандартом шифрования является ГОСТ Р 34.12, последняя редакция которого приходится на 2018 г. Но существует редакция немного старше, 2015 г., и в её случае уже проанализирована криптостойкость и сложность алгоритмов шифрования и, согласно анализу экспертов, эта редакция является очень надежной.

Объектом исследования является резервное копирования баз данных.

Предметом исследования является методы резервного копирования баз данных с использованием шифрования в СУБД MS SQL Server.

Целью работы является создание импортозамещающего программного приложения, осуществляющего резервное копирование баз данных в СУБД MS SQL Server с использованием алгоритма шифрования ГОСТ Р 34.12 «Кузнечик».

Для достижения поставленной цели необходимо решить ряд задач:

- изучить методы резервного копирования доступные в MS SQL Server, а также определить их преимущества и недостатки;
- рассмотреть шифрование в MS SQL Server, сравнить доступные алгоритмы в MS SQL Server с ГОСТ Р 34.12 «Кузнечик»;
- разработать собственный оригинальный алгоритм создания зашифрованной резервной копии и восстановления базы данных;
- разработать импортозамещающее программное приложение для создания резервных копий и восстановления баз данных в MS SQL Server с использованием алгоритма шифрования ГОСТ Р 34.12 «Кузнечик».

Для разработки программного приложения, обеспечивающего безопасное создание резервных копий, потребуется ознакомиться с технологией MS SQL VDI. Необходимо реализовать интерфейс для передачи потока резервной копии непосредственно в разработанное программное приложение.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Основные сведения

База данных (БД)— это структурированное хранилище с информацией.

Сами по себе, базы данных не могут существовать полноценно, в первозданном виде это лишь набор информации. Для того что бы манипулировать информацией необходима система управления базой данных (СУБД). Под управлением базой данных понимаем возможность индивидуального или коллективного редактирования информации, ее сортировку, полное или частичное копирование и перемещение, а также возможность объединять несколько баз данных в одну общую систему. Для осуществления всего вышеперечисленного были созданы программные продукты, являющиеся программным обеспечением баз данных – СУБД. Основные компоненты СУБД представлены на рисунке 1.1.

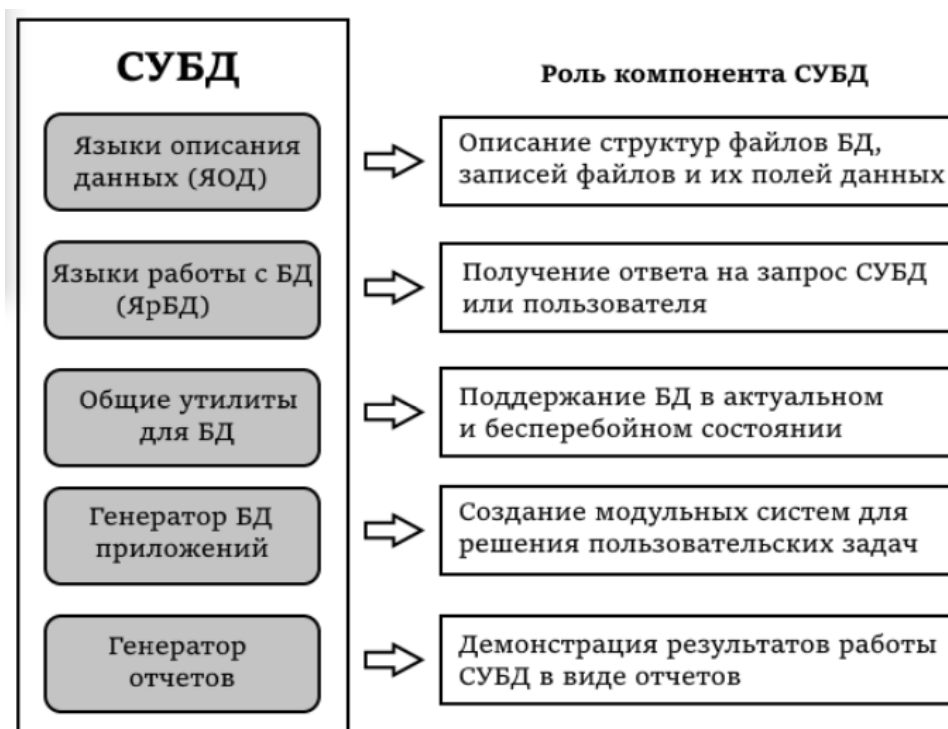


Рисунок 1.1 – Компоненты СУБД

Система управления базами данных (СУБД) — это специализированное программное обеспечение, основной задачей которого является обеспечение бесперебойного доступа к базе данных, а также к методам её администрирования.

1.2 Администрирование БД

Крайне важным аспектом бесперебойной работы базы данных является поддержание СУБД, и этой работой занимаются администраторы баз данных.

Администратор базы данных—это ИТ–специалист, отвечающий за выработку требований к базе данных, её проектное проектирование, реализацию, эффективное использование и сопровождение. Общие задачи администрирования баз данных:

- подготовка к эксплуатации баз данных и серверов баз данных. Это может включать установку и настройку экземпляров SQL Server на физических или виртуальных серверах, или создание новых виртуальных машин на основе шаблонов изображений, а также создание баз данных и распределение их данных и файлов журналов на устройствах хранения;

- сохранение файлов баз данных и объектов. После того, как база данных создана и заполняется данными, для оптимальной работы требуется ее постоянное обслуживание и оптимизация. Это предполагает уменьшение фрагментации, которая появляется по мере того, как записи добавляются и удаляются, сохранение файлов данных соответствующего размера и обеспечение последовательной структуры логических и физических данных;

- управление восстановления в случае сбоя базы данных. Базы данных часто имеют решающее значение для деловых операций, поэтому главной задачей DBA является планирование соответствующей стратегии резервного копирования и восстановления для каждой базы данных, что позволило бы осуществить восстановление базы данных в случае сбоя;

- импорт и экспорт данных. Данные часто передаются между системами, поэтому администраторам БД необходимо выполнять экспорт или импорт данных;

- применение безопасности к данным. Серверы баз данных организации содержат данные, которые позволяют бизнесу работать. Нарушение безопасности может быть дорогостоящим и трудоемким для восстановления, приводить к потере доверия клиентов. Администратор БД должен реализовывать такие политики безопасности, которые обеспечивают пользователям доступ к необходимым данным, но при этом соблюдают правовые нормы бизнеса по защите своих активов, а также снижают риски, связанные с нарушением безопасности;

- мониторинг и устранение неполадок СУБД. Многие операции по администрированию БД являются реактивными, т.е. они предполагают принятие мер для устранения неполадок и возникающих проблем. Опытные администраторы БД осуществляют упреждающий подход, чтобы попытаться обнаружить потенциальные проблемы до того, как они начнут влиять на операции с данными.

Большинство администраторов БД знакомы с системами, которыми они управляют, и знают задачи, которые должны выполняться ежедневно. Однако даже опытные администраторы не полагаются исключительно на свою память.

Администраторы БД обычно составляют и ведут документацию, которая включает в себя такие сведения, как:

- параметры конфигурации и расположения файлов;
- контактная информация персонала;
- стандартные правила и графики технического обслуживания;
- процедуры аварийного восстановления.

Ведение документации является важной частью администрирования баз данных. Подробная книга записей может иметь неоценимое значение, особенно в случае, когда новый администратор должен взять на себя ответственность за управление БД, или при возникновении неожиданной чрезвычайной ситуации в отсутствие администратора. При сбое сервера, чётко задокументированные шаги восстановления базы данных уменьшают чувство паники у пользователей системы и обеспечивают быстрое решение проблемы.

1.3 Резервное копирование. Виды резервного копирования

Большинство СУБД наделены возможностью создания резервной копии базы данных. Создание резервной копии необходимо для восстановления БД в стабильном состоянии, в случае сбоев и ошибок. Резервное копирование осуществляется самой системой управления базами данных, но оно также может выполняться вручную администратором, или же быть запланировано и выполняться автоматически в указанном временном интервале.

Резервная копия состоит из базы данных и всех log-файлов, требуемых для приведения копии базы данных и к согласованному состоянию во время резервирования. Следует отметить, что вне зависимости от типа выполняемого резервного копирования резервируемые данные должны быть неповрежденными. Поэтому перед выполнением резервного копирования следует проверять стабильность базы данных и проводить эту процедуру резервного копирования только тогда, когда база данных находится в устойчивом состоянии.

Виды резервного копирования различаются по многим параметрам. Каждый способ обладает как своими достоинствами, так и недостатками.

Следует рассмотреть каждый способ по отдельности.

а) Полное резервное копирование. Метод создает полную копию базы данных. *Преимущества:* быстрое и корректное восстановление данных; простое управление; все данные содержатся в одной резервной копии; не может быть конфликта версий резервных копий. *Недостатки:* требует много места для хранения резервных копий; высокая загрузка сети и самой СУБД; длительное выполнение самой процедуры резервного копирования.

б) *Инкрементное резервное копирование*. Метод использует полную копию, как начальную точку. Затем выполняется резервное копирование только блоков данных, которые были изменены с момента последнего резервного задания, с заданным периодом выполнения задания. В зависимости от политики хранения резервных копии, через определенный период создается новая полная копия для повторения цикла. *Преимущества*: высокая скорость резервного копирования (копируются только блоки изменённых данных); меньше места для хранения (по сравнению с полным); большее количество точек восстановления. *Недостатки*: низкая скорость восстановления данных (необходимо восстановить как начальную полную копию, так и все последующие блоки); малая надежность (зависит от целостности всех блоков в цепочке).

в) *Дифференциальное резервное копирование*. Метод является промежуточным решением между полным резервным копированием и инкрементными резервными копиями. Подобно инкрементному резервному копированию, отправной точкой для дифференциальной резервной копии является создание полной резервной копии и резервное копирование только измененных данных. Однако, в отличие от инкрементных резервных копий, дифференциальная резервная копия сохраняет не данные, которые были изменены с момента последней резервной копии, а данные, которые были изменены с момента первоначальной полной резервной копии. Таким образом, полная резервная копия является постоянной точкой отсчета для последовательных резервных копий. *Преимущества*: резервное копирование быстрее, чем полное, но медленнее, чем инкрементное; восстановление быстрее, чем инкрементное, но медленнее чем полное; более надежный способ (для восстановления требуется только полная и последняя резервная копия). *Недостатки*: каждый последующий *бекап* выполняется дольше по времени и занимает больше дискового пространства в хранилище.

1.4 СУБД Microsoft SQL Server

Microsoft SQL Server — система управления реляционными базами данных (РСУБД), разработанная корпорацией Microsoft. Основным используемым языком является Transact–SQL. Это реляционная распределенная клиент–серверная СУБД. Основным инструментом является SQL Server Management Studio (SSMS). Он представляет собой графический пользовательский интерфейс (GUI) и интерфейс сценариев Transact–SQL для управления компонентом ядра БД и базами данных.

Transact–SQL (T–SQL) – модифицированный язык SQL. Все приложения, взаимодействующие с экземпляром MS SQL Server, независимо от их реализации и пользовательского интерфейса, отправляют серверу инструкции Transact–SQL.

Microsoft SQL Server обладает всеми необходимыми инструментами, для поддержания базы данных в стабильном состоянии. Важно понимать, что SQL Server не является единым монолитным приложением, а структурирован как ряд компонентов. Компоненты SQL Server и их назначение приведены в таблице 1.1 Основными компонентами являются Database Engine, SSAS, SSIS, SSRS.

Таблица 1.1 – Компоненты Microsoft SQL

Компонент	Описание
Database Engine	Ядро платформы SQL Server. Обеспечивает высокую производительность и масштабируемость реляционных баз данных на основе языка SQL. Может использоваться для размещения данных, для их дальнейшей обработки с помощью транзакций в интерактивном режиме и для создания хранилищ данных. SQL Server также включает в себя оптимизированный движок базы данных, который использует технологию «inmemory» для повышения производительности коротких транзакций.
Analysis Services	SQL Server Analysis Services (SSAS) – аналитические службы – обеспечивают функциональность OLAP (Online Analytical Processing) и анализа данных для приложений бизнес-аналитики. Они позволяют организации собирать данные из нескольких источников, например, реляционных БД, и обрабатывать их различными способами.
Integration Services	SQL Server Integration Services (SSIS) – службы интеграции, инструмент масштаба предприятия для извлечения, преобразования и интеграции данных из различных источников и их перемещения в один или несколько целевых источников данных. Предназначены для слияния данных из разнородных источников и загрузки их в хранилища, витрины данных и пр.
Reporting Services	SQL Server Reporting Services (SSRS) – службы отчетов – включают Диспетчер отчетов и сервер отчетов. Представляют собой полномасштабную серверную платформу для создания, управления и распространения отчетов. Позволяют использовать для обработки и хранения отчетов сочетание возможностей SQL Server и IIS. SSRS может быть установлен самостоятельно или интегрирован с Microsoft SharePoint.

Компонент	Описание
Master Data Services	SQL Server Master Data Services (MDS) среда для создания бизнес-правил, гарантирующих качество и точность основных данных. Бизнес-правила могут применяться для запуска бизнес-процессов, выполняющих проверки и управляющих потоками данных.
Data Quality Services	SQL Server Data Quality Services (DQS) среда для создания хранилища метаданных базы знаний, что позволяет улучшить качество данных организации. Процессы очистки данных позволяют изменять или удалять неполные и некорректные данные, процессы сопоставления позволяют выявлять и объединять дублирующиеся данные.
StreamInsight	SQL Server StreamInsight предоставляет платформу для создания приложений, которые выполняют обработку сложных событий для потоков данных в реальном времени.
Full-Text Search	Полнотекстовый поиск – это функция компонента «Database Engine», который обеспечивает механизм сложного семантического поиска для текстовых данных.
Replication	«Database Engine» включает в себя репликации, набор технологий для синхронизации данных между серверами для удовлетворения потребностей распространения данных
Power View for SharePoint Server	Power View – компонент SQL Server Reporting Services. Обеспечивает интерактивное исследование данных, их визуализацию и презентацию. Power View также доступен в Excel.

1.4.1 Модели восстановления в Microsoft SQL Server

Одним из наиболее важных аспектов является обеспечение регулярного копирования данных, чтобы в случае сбоя можно было восстановить БД. Несмотря на то, что компьютерная индустрия уже на протяжении десятилетий знает о необходимости надежных стратегий резервного копирования, трагические истории потери данных по-прежнему остаются обычным явлением.

Модели восстановления SQL-сервера. SQL-сервер поддерживает три типа моделей восстановления базы данных. Все эти модели сохраняют данные в случае аварийной ситуации, но между ними существуют важные различия, которые необходимо учитывать при выборе модели восстановления для базы данных.

Описание моделей восстановления приведено в таблице 1.2. У каждой базы данных может быть своя модель восстановления. Выбор модели осуществляется в окне *Свойства БД* на странице *Параметры*.

Таблица 1.2 – Модели восстановления базы данных

Модель восстановления	Описание
Простая (Simple)	Предназначена для восстановления до точки последней архивации. Стратегия для данной модели должна включать полные и разностные операции резервного копирования. Включив простую модель восстановления, нельзя выполнять резервное копирование журналов транзакций. Автоматически в контрольной точке происходит усечение журнала (очистка всех неактивных записей транзакций), чтобы минимизировать пространство. Эта модель идеально подходит для большинства системных БД.
Полная (Full)	Предназначена для восстановления БД до точки сбоя или на определенный момент времени. При использовании данной модели протоколируются все операции, включая массовые операции и массовую загрузку данных. Стратегия должна включать следующие архивы: полные, разностные и архивы журнала транзакций (или только полные архивы и архивы журнала транзакций), т.е. требуется резервное копирование журнала. Исключает потерю данных из-за поврежденного или отсутствующего файла данных.
Неполное протоколирование (Bulk Logged)	Эта модель сокращает пространство, занимаемое журналом транзакций, сохраняя большую часть функциональности полной модели восстановления. Выполняется минимальное протоколирование массовых операций и массовых загрузок без контроля отдельных операций, что может повысить производительность операций массового копирования. Если сбой произойдет прежде, чем будет выполнена полная или разностная архивация, массовые операций и массовые загрузки придется повторить вручную. Стратегия резервного копирования для данной модели должна включать те же архивы, что и для полной модели.

1.4.2 Резервное копирование в MSSQL

Резервное копирование баз данных и журналов. Типы резервных копий SQL Server поддерживает несколько типов резервных копий, которые можно комбинировать для реализации правильной стратегии резервного копирования и восстановления для конкретной базы данных на основе бизнес-требований и целей восстановления. Описание типов резервных копий приведено в таблице 1.3.

Таблица 1.3 – Описание типов резервных копий в SQL Server

Тип резервной копии	Описание
Полная (Full)	В полную резервную копию БД включаются все объекты, системные таблицы и данные, а также фрагменты журналов транзакций, соответствующие времени архивации. Полная резервная копия позволяет полностью восстановить БД на момент завершения резервного копирования.
Разностная (Differential)	Разностные резервные копии предназначены для архивации данных, которые изменились со времени последнего резервного копирования. При резервном копировании сохраняются только изменения, поэтому архивация занимает меньше времени, и ее можно выполнять чаще. В разностные копии также включаются фрагменты журналов транзакций, необходимые для восстановления БД после завершения резервного копирования
Копия Журнала Транзакций (Transaction Log)	При резервном копировании журнала транзакций в архиве сохраняются изменения, произошедшие со времени последнего резервного копирования журнала транзакций, а затем выполняется его усечение, в ходе которого стираются выполненные или отмененные транзакции. В отличие от полных или разностных архивов, в архиве журнала транзакций записано его состояние на момент начала резервного копирования (а не на момент его завершения).
Копия файлов и файловых групп (File / File Group)	Эти резервные копии позволяют архивировать не всю БД, а только указанные файлы и файловые группы. Резервные копии этого типа используют при работе с большими БД, чтобы сэкономить время. Архивация файлов и файловых групп имеет ряд особенностей. Одновременно с ней надо

Тип резервной копии	Описание
	архивировать и журнал транзакций. Поэтому данным способом НЕ выполняют архивацию, если включен параметр Усечение журнала на контрольной точке (Truncate Log On Checkpoint). Если объекты БД охватывают несколько файлов или файловых групп, надо выполнить архивацию ВСЕХ этих файлов или файловых групп.
Частичная (Partial)	Содержит данные только из некоторых файловых групп БД, включая данные первичной файловой группы, все файловые группы, доступные для чтения/записи, а также любые дополнительно указанные файлы, доступные только для чтения. Частичные резервные копии могут оказаться полезны в тех случаях, когда необходимо исключить файловые группы только для чтения. Частичная резервная копия базы данных, доступной только для чтения, содержит только первичную файловую группу.
Копия заключительного фрагмента журнала (Tail-log Backup)	В резервную копию заключительного фрагмента журнала попадают все записи, резервная копия которых еще не была создана (заключительный фрагмент журнала, «хвост»), что позволяет предотвратить потерю работы и сохранить неповрежденную цепочку журналов. Для восстановления базы данных SQL Server на последний момент времени необходимо предварительно выполнить резервное копирование заключительного фрагмента журнала её транзакций. Заключительный фрагмент журнала становится последней рассматриваемой частью резервной копии в плане восстановления базы данных.
Копия только для копирования (Copy Only)	Специальная резервная копия, независимая от обычной последовательности резервных копий SQL Server. Обычно создание резервного копирования приводит к изменению БД и влияет на то, как будут восстанавливаться последующие резервные копии. Однако иногда приходится выполнять резервное копирование БД для особых нужд, когда это не сказывается на общем процессе резервного копирования и восстановления. Этой цели и служат резервные копии только для копирования (БД или журналов транзакций).

1.4.3 Шифрование в MSSQL

Для резервных копий основополагающим требованием является обеспечение защиты данных от аппаратных сбоев и пр. Однако данные в резервной копии могут быть конфиденциальными, поэтому необходимо убедиться, что носитель резервных копий защищен от несанкционированного доступа.

В большинстве организаций это может быть выполнено путем хранения носителей резервных копий в местах с защищенной файловой системой. В ситуации, когда данные в резервной копии требуют дополнительной безопасности, можно шифровать резервные копии таким образом, чтобы они могли быть восстановлены только на экземпляре SQL Server, содержащем ключ шифрования. Резервное копирование с шифрованием в SQL Server основывается на алгоритмах стандарта шифрования, включая AES 128, AES 192, AES 256 и Triple DES.

Главным достоинством шифрования резервных копий в MS SQL является то, что сам процесс происходит непосредственно при формировании резервной копии. Таким образом, файл резервной копии сохраняется на диск уже в зашифрованном виде. В этом случае обеспечивается высокая скорость создания зашифрованной резервной копии, так как шифрование происходит «на лету», а также не нарушается целостность системы.

Главным недостатком шифрования MS SQL Server является то, что расширить доступный арсенал шифров не представляется возможным. Политика программного продукта MS SQL Server не подразумевает расширение доступного набора шифров. Пользователю необходимо выбирать среди имеющихся вариантов.

1.5 Альтернативные решения резервного копирования в MS SQL

Как и в любой области, существуют альтернативы. В случае создания зашифрованных резервных копий все доступные варианты делятся на три группы. Каждая из этих групп имеет свои достоинства и недостатки. Для полного понимания необходимо рассмотреть каждую группу подробнее.

В первую группу следует включить стороннее программное обеспечение, которое нацелено на упрощение администрирования MS SQL Server. В этом случае приложение самостоятельно отправляет на сервер MS SQL запросы и настраивает планировщик задач. В свою очередь, пользователь оперирует интуитивно понятным интерфейсом программы, где обычно предоставляются параметры и настройки в упрощенном варианте.

Такие программы, как «Effector saver» и «Handy Backup» предоставляют пользователю весь необходимый инструментарий, для создания резервных копий, планирование расписания, а также восстановления базы данных.

Сами программы не обладают своим набором алгоритмов шифрования, а пользуются лишь тем, что предлагает сам MS SQL Server. Поэтому необходимо предварительно настроить шифрование на самом сервере MS SQL. Отсутствует возможность расширить список доступных алгоритмов другими алгоритмами.

Во вторую группу входит целый кластер приложений, позволяющих зашифровать произвольный набор данных, в том числе и готовый файл резервной копии MS SQL Server. Программы «Files Cipher» и «dsCrypt» обладают множеством криптоалгоритмов и позволяют зашифровать произвольный набор файлов.

В качестве главного недостатка следует отметить то, что для осуществления шифрования файл должен быть уже создан физически и находиться в доступной для программ директории. Таким образом, подобное шифрование является уязвимым. В случае, когда у злоумышленника есть доступ к разделу памяти, где файлы хранятся в открытом виде, он может прихватить файл до его шифрования. В случае использования подобных программ для шифрования резервных копий, основной уязвимостью будет являться то, что предварительно будет создан файл резервной копии в незашифрованном виде.

В третью группу входят приложения, которые позволяют зашифровать раздел жесткого диска, таким образом, шифруя на лету все данные, которые записываются на этот диск. Такие программы, как «Truecrypt» и «BestCrypt» обладают большим набором крипто алгоритмов, но, как и в случае всех остальных, перечисленных выше программ, в список криптоалгоритмов не входит российский ГОСТ.

Следует отметить тот факт, что запись резервной копии на зашифрованный том жесткого диска не подразумевает, что файл будет зашифрован. В случае шифрования тома жесткого диска производительность работы с ним будет снижена, что может вызвать дополнительные неудобства в работе.

Выводы

Изучены основные теоретические сведения, касающиеся администрирования базы данных MS SQL, а также изучены основные методы реализации резервного копирования и шифрования. Детально исследован инструментарий для администратии MS SQL. Рассмотрены основные компоненты: Database Engine, Analysis Services и Integration Services; а также основные инструменты управления базами данных MS SQL: SQL Server Management Studio, SQL Server Configuration Manager и Sqlcmd. Проанализированы основные методы реализации резервного копирования базы данных, а также ее шифрования с помощью встроенного инструментария MS SQL. Произведен анализ альтернативных решений и их возможностей для создания резервных копий с последующим шифрованием.

У каждой группы альтернативных решений найдены как свои достоинства, так и недостатки. Каждая группа применима при определенных обстоятельствах, когда выбранное решение будет самым эффективным.

2 АНАЛИЗ АЛГОРИТМОВ ШИФРОВАНИЯ

В современном мире, где каждый год возрастает мощность процессоров и скорость передачи данных, стандарты, выпущенные всего несколько лет тому назад, могут очень быстро устареть, и стать неактуальными.

В случае криптоалгоритмов, стандарты обычно проектируются с запасом мощности и надежности, таким образом, обеспечивая стабильную защиту на несколько десятилетий. Но, несмотря на предполагаемый запас, зачастую в ходе эксплуатации разработанных стандартов обнаруживаются уязвимости, которые могут очень сильно сократить предполагаемый срок службы стандарта. Примером может служить стандарт DES, предполагаемое время службы которого, по прогнозам экспертов, должно было продлиться вплоть до 2000-х годов, но на практике срок службы оказался намного короче. DES был национальным стандартом в США и существовал в качестве такого с 1977 по 1980 г. Стандарт DES оказался уязвимым к множеству атак, и в 1980 г. был заменен на TDES.

Следует отметить и популярность стандартов. Чем известнее стандарт, чем больше программных решений на нем основано, тем больше атак на него совершается, и, таким образом, значительно ускоряется поиск возможных уязвимостей и сокращается предположительный срок надёжности стандарта.

2.1 Алгоритм «Кузнечик»

Блочный шифр «Кузнечик» – это отечественная импортозамещающая разработка симметричного блочного шифрования, входящего в состав ГОСТ Р 34.12–2015. Он пришёл на смену уже устаревшего алгоритма шифрования «Магма», входящего в состав ГОСТ 28147–89. Основу алгоритма составляет *подстановочно–перестановочная сеть (SP–сеть)*, в отличие от *сети Фейстеля*, используемой в шифрах «Магма» и AES. Шифр на основе *SP–сети* получает на вход блок и ключ и совершает несколько чередующихся раундов, состоящих из стадий подстановки и стадий перестановки. Каждый раунд включает в себя линейное и нелинейное преобразование, плюс операцию наложения – *итерационного ключа*. Всего таких раундов девять и один последний неполный раунд, в котором выполняется только наложение последнего (десятого) итерационного ключа.

Итерационные (или раундовые) ключи получаются путем определённых преобразований на основе мастер–ключа, длина которого составляет 256 бит. Этот процесс начинается с разбиения мастер–ключа пополам; таким образом, получается первая пара раундовых ключей.

Для генерации каждой последующей пары раундовых ключей применяется восемь итераций *сети Фейстеля*, при этом в каждой итерации используется константа, которая вычисляется путем применения линейного преобразования алгоритма к значению номера итерации.

Алгоритм блочного шифрования «Кузнечик» основан на последовательном применении нескольких однотипных раундов, каждый из которых содержит три преобразования: сложение с раундовым ключом, преобразование блоком подстановок и линейное преобразование. У отечественной импортозамещающей разработки проводится девять раундов шифрования.

«Кузнечик» применяет подстановочно–перестановочную сеть в качестве основного алгоритма, а для создания раундового ключа использует сеть Фейстеля.

У алгоритма «Кузнечик» первые два раундовых ключа получаются разбиением входного ключа пополам. Затем, для выработки очередной пары раундовых ключей, используется восемь итераций *сети Фейстеля*. В качестве раундовых ключей здесь используется последовательность счётчика, прошедшая через линейное преобразование этого алгоритма.

2.2 Алгоритм AES

Advanced Encryption Standard (AES) – симметричный алгоритм блочного шифрования, который был принят в качестве стандарта правительством США по итогам проведенного конкурса, между технологическими институтами. Он заменил устаревший *Data Encryption Standard* (DES), который, по итогам прошедших лет, больше не соответствовал требованиям безопасности.

Алгоритм AES позволяет использовать длины ключей 128/192/256 бит и число раундов 10/12/14 соответственно. Следует отметить, что алгоритм AES использует подстановочно–перестановочную сеть в основе алгоритма, а раундовый ключ генерирует посредством собственного метода, на основе расширения ключа.

В AES шифруемый блок и его промежуточные состояния в ходе шифрования представляются в виде матрицы байтов, 4 x 4. Каждый раунд содержит следующие преобразования: сложение и раундовым ключом, байтовая подстановка, побайтовый циклический сдвиг в строках матрицы и матричное умножение. В случае AES число раундов зависит от длины ключа, 10 раундов для 128 бит, 12 раундов для 192 бит и 14 раундов для длины ключа в 256 бит.

Алгоритм выработки раундовых ключей в алгоритме AES более сложен, чем в российском стандарте. Так, алгоритм AES позволяет использовать длины ключей 128, 192 и 256 бит, в то время как алгоритм «Кузнечик» может использовать длину ключа только в 256 бит.

Ключ и ключевая последовательность представляются в виде матриц 4–х байтовых слов, и начальный участок последовательности заполняется словами из ключа. Последующие слова ключевой последовательности вырабатываются по рекуррентному соотношению, группами, кратными размеру ключа.

Полученные из потока 4–байтовые слова группируются в ключевые элементы необходимого размера, равного размеру шифруемого блока, и затем используются на последующих раундах шифрования.

2.3 Алгоритм «Triple DES»

TDES (Triple DES, 3DES) – тройной DES, усовершенствованный блочный алгоритм DES. Симметричный блочный криптографический алгоритм, созданный на основе алгоритма DES с целью устранения главного недостатка — малой длины ключа (56 бит), который может быть взломан методом перебора ключа. В основе функции шифрования лежит сеть Фейстеля. Основным принципом архитектуры сети Фейстеля является то, что процесс шифрования состоит из нескольких однотипных раундов. Каждый раунд обрабатывает одну из половин шифруемого блока, затем эти половины меняются местами, и на следующем раунде процесс повторяется с неизменной на предыдущем раунде частью.

Алгоритм TDES в каждом из трех повторений может использовать либо разные ключи, либо их комбинировать. В каждом повторе алгоритм TDES получает раундовые ключи из изначального ключа, но делит его не пополам, как в случае «Кузнечика», а на 8 частей. Перед делением его длина расширяется до 64 бит путём добавления контрольных битов целостности.

Полученный после начальной перестановки 64–битовый блок IP(T) участвует в 16 циклах преобразования сетей Фейстеля. В каждом раунде происходит его сложение с раундовым ключом, преобразование с помощью блока подстановок, линейное преобразование и перестановка [39].

2.4 Сравнение алгоритмов AES, «Triple DES» и «Кузнечик»

Несмотря на схожесть архитектур, в отличие от AES, у алгоритма «Кузнечик» есть несколько особенностей: линейное преобразование может быть реализовано с помощью регистра сдвига; ключевая развертка реализована с помощью *сети Фейстеля*, в которой в качестве функции используется раундовое преобразование исходного алгоритма.

В архитектуре *SP–сеть*, которую используют алгоритмы шифрования AES и «Кузнечик», шифруемый блок представляется в виде матрицы байтов. Шифрование происходит аналогично архитектуре *сети Фейстеля* за несколько раундов, но за каждый раунд шифруется полный блок, а не его половина. Это означает, что для обеспечения сопоставимой сложности, алгоритму DES необходимо выполнить вдвое больше раундов преобразования, чем «Кузнечику» или AES. Сравнение алгоритмов представлено в таблице 2.1.

Таблица 2.1 – Сравнение алгоритмов «Кузнечик, «AES» и «TDES»

Название шифра	Длина ключа (бит)	Число раундов шифрования	Раундовые преобразования	Год выпуска стандарта
«Кузнечик»	256	9	XOR с раундовым ключом, нелинейное преобразование с блоками подстановки S, Линейное преобразование L	2015 (2018) г.
AES	128, 192, 256	10, 12, 14	S – преобразование, перестановка, перемножение на матрицу слева в поле $GF(2^8)$, XOR с раундовым ключом	1998 г.
TDES	3 x 56	16	Расширение (перестановка), XOR с раундовым ключом, S–преобразование	1978 г.

В сравнении с иностранной разработкой AES, отечественный шифр «Кузнечик» появился значительно позднее. Эти алгоритмы очень схожи, но в них есть существенные отличия. В «Кузнечике» были исправлены многие недостатки, выявленные за 17 лет работы алгоритма AES.

Проведенное выше сопоставление параметров алгоритмов шифрования ГОСТ Р 34.12–2015 и AES показало, что во многом схожи, включая архитектуру. Однако российский импортозамещающий алгоритм является более простым, как для реализации, так и для понимания его работы. По основным параметрам криптостойкости ни один алгоритм не обладает особым преимуществом перед другими. В случае алгоритма TDES можно сделать однозначный вывод о том, что он уступает в надёжности, так как длина ключа в его случае составляет 168 бит, что меньше, чем у алгоритмов «Кузнечик» и AES–256.

Выводы

Сравнительный анализ алгоритмов, проведённый в этом разделе, показал, что алгоритмы шифрования AES и «Кузнечик» во многом схожи, оба основаны на архитектуре *SP-сети* и поддерживают ключ длиной 256 бит.

Но в то же время, AES появился намного раньше, является более известным и популярным алгоритмом шифрования, и массово используется в программных продуктах, в том числе и в продуктах Microsoft. Именно это и является его недостатком, так как число атак на алгоритм AES значительно превышает число атак на алгоритм «Кузнечик». Это дает злоумышленнику большую вероятность найти уязвимость в шифре AES. При этом возрастает шанс сохранить криптостойкость алгоритма шифрования «Кузнечик».

Также следует отметить тот факт, что отечественный алгоритм «Кузнечик» проще в понимании и реализации, но при этом не уступает в качестве.

Алгоритм TDES уступает другим алгоритмам практически по всем параметрам, что делает его использование ненадёжным и не рекомендуемым.

Можно сделать вывод о предпочтительности использования отечественного алгоритма шифрования «Кузнечик», тем более в импортозамещающих российских разработках в области кибербезопасности информационных систем.

3 АЛГОРИТМИЧЕСКОЕ КОНСТРУИРОВАНИЕ

3.1 Концепция программного продукта

Разрабатываемое импортозамещающее программное приложение должно быть атомарным для пользователя. Пользователь не должен знать о той бизнес-логике, которая обрабатывает его запросы. После запроса пользователя приложение должно ответить результатом, вне зависимости от успеха. Так же приложение должно быть отказоустойчивым, так как всегда есть возможность того, что пользователь укажет неправильные данные, выполнит действия в неправильном порядке или во время работы программы произойдет сбой.

Разрабатываемое приложение (рисунок 3.1) будет осуществлять резервное копирование базы данных, а также её восстановление из резервных копий. Такие резервные копии должны быть зашифрованы, что необходимо для их безопасного хранения на физическом диске пользователя. В качестве алгоритма шифрования выбран ГОСТ Р 34.1—2015 (алгоритм «Кузнечик»). Необходимо вести учёт действий пользователя, для этого приложение должно обладать своей собственной базой данных для ведения учёта и хранения пользовательских данных.

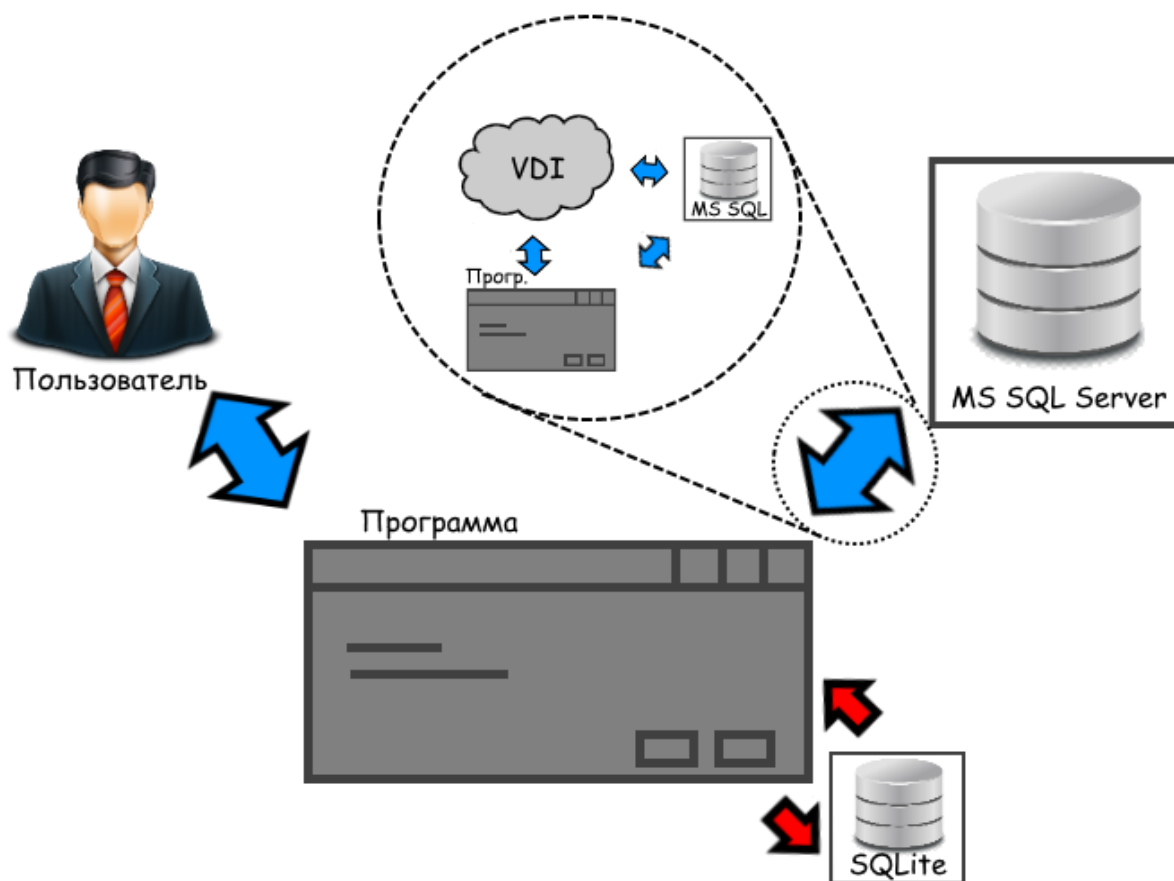


Рисунок 3.1 – Схема архитектуры разрабатываемого приложения

В качестве такой базы данных может послужить портативная СУБД MSQlite 3.0. Главная проблема, с которой придется столкнуться на этапе разработки, это то, что MS SQL поддерживает только своё шифрование «из коробки», и не предоставляет сторонним разработчикам возможности расширить доступный набор алгоритмов. В то же время компания–разработчик Microsoft предусмотрела альтернативный вариант, и в его случае сторонним разработчикам предоставляется интерфейс VDI, через который можно перенаправить поток резервной копии непосредственно в стороннее приложение. Таким образом, созданная база данных будет перенаправлена в разрабатываемое приложение, итоговая схема которого показана на рисунке 3.1. Исходя из представленной схемы, очевидно, что пользователь взаимодействует исключительно с разрабатываемым приложением.

Пользователю должен быть предоставлен доступ к логу событий, в котором, в хронологическом порядке, будут записаны все взаимодействия с базами данных на MS SQL Server [40, 41].

3.2 Алгоритм создания резервной копии

Одной из важных функций в разрабатываемой программном обеспечении является создание резервной копии. Функция реализующая механизм резервного копирования должна подключиться к серверу MS SQL для того, чтобы инициализировать интерфейс VDI для последующей передачи потока резервной копии базы. Параметры для получения нужно хранить в настройках самой программы, для автоматизации процесса и упрощения работы с сервером.

Полученный поток может быть зашифрован алгоритмом «Кузнечик». Путь для сохранения резервной копии пользователь так же указывает в настройках программы. Все действия программы с сервером следует документировать в лог событий. Так можно будет восстановить хронологию последних действий программы. Алгоритм создания резервной копии представлен на рисунке 3.2.

Можно заметить, что в случае ошибок в настройках пользователя, сбоев во время подключения к серверу или ошибок во время получения потока резервной копии, алгоритм прерывается и уведомляет пользователя о возникшей неисправности. Эта часть алгоритма должна учитывать все возможные исключения и обрабатывать их в корректной форме, чтобы максимально точно донести до пользователя природу возникшей ошибки.

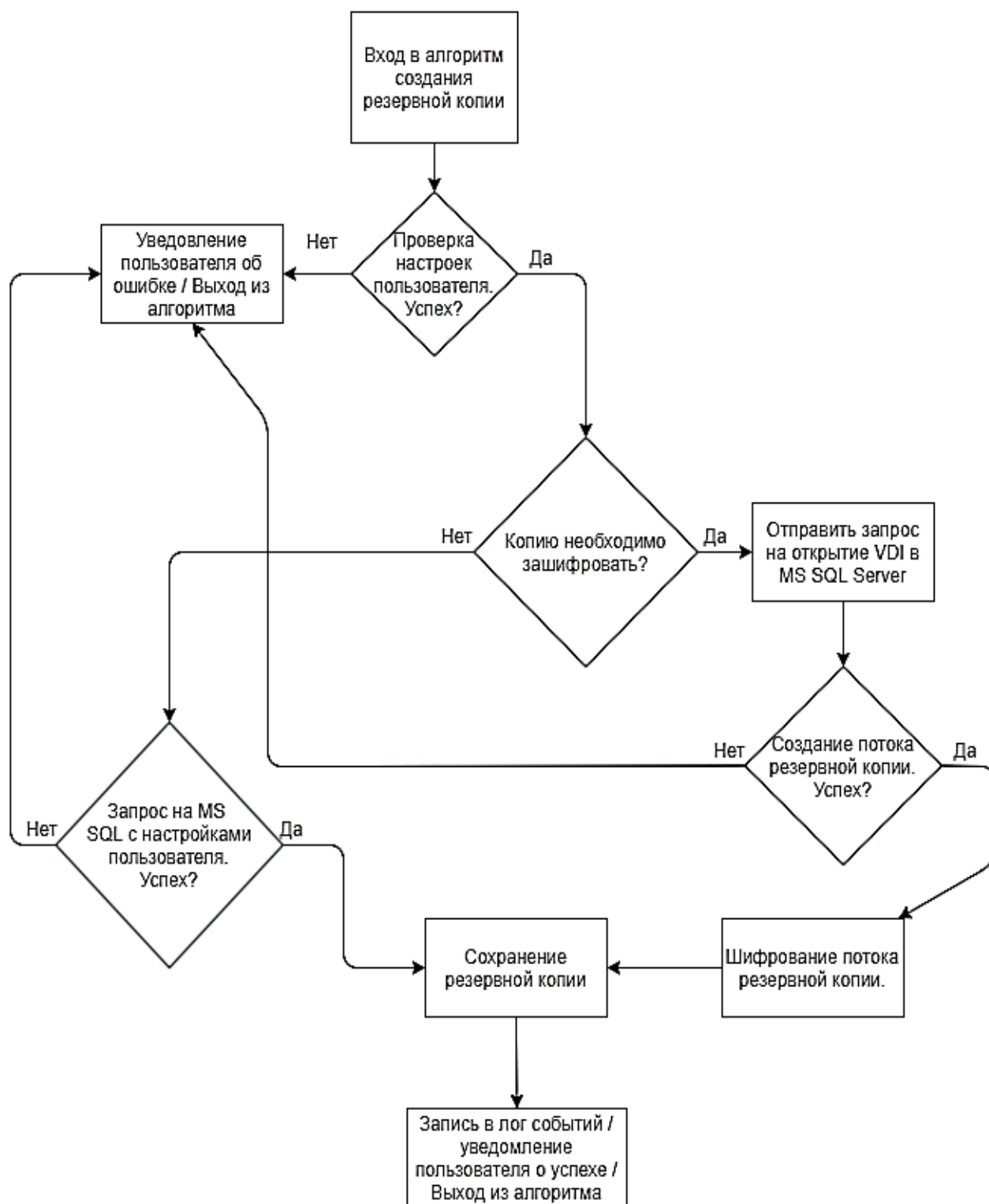


Рисунок 3.2 – Алгоритм создания резервной копии

Перед завершением алгоритма, результат работ должен быть записан в лог событий. Это необходимо для того, чтобы пользователь мог просмотреть результаты работы, а также в случае необходимости узнать, с какими параметрами осуществлялось резервное копирование.

3.3 Алгоритм восстановления базы данных из резервной копии

Представленный на рисунке 3.3 алгоритм является обратно направленным алгоритмом создания резервной копии. Следует заметить, что в независимости от того, шифровалась резервная копия или нет, конечные действия по восстановлению базы данных происходят непосредственно на сервере MS SQL.

При выходе из алгоритма успех работы, как и в случае с алгоритмом создания резервной копии, должен быть так же записан в лог событий.

На вход в алгоритм поступают параметры от пользователя. В эти параметры входят настройки подключения к MS SQL Server, а также, в случае шифрования резервной копии, поступают параметры ключа.

В ходе выполнения алгоритм может прерваться ошибкой, и тогда выполняется выход из алгоритма с последующим уведомлением пользователя об ошибке.

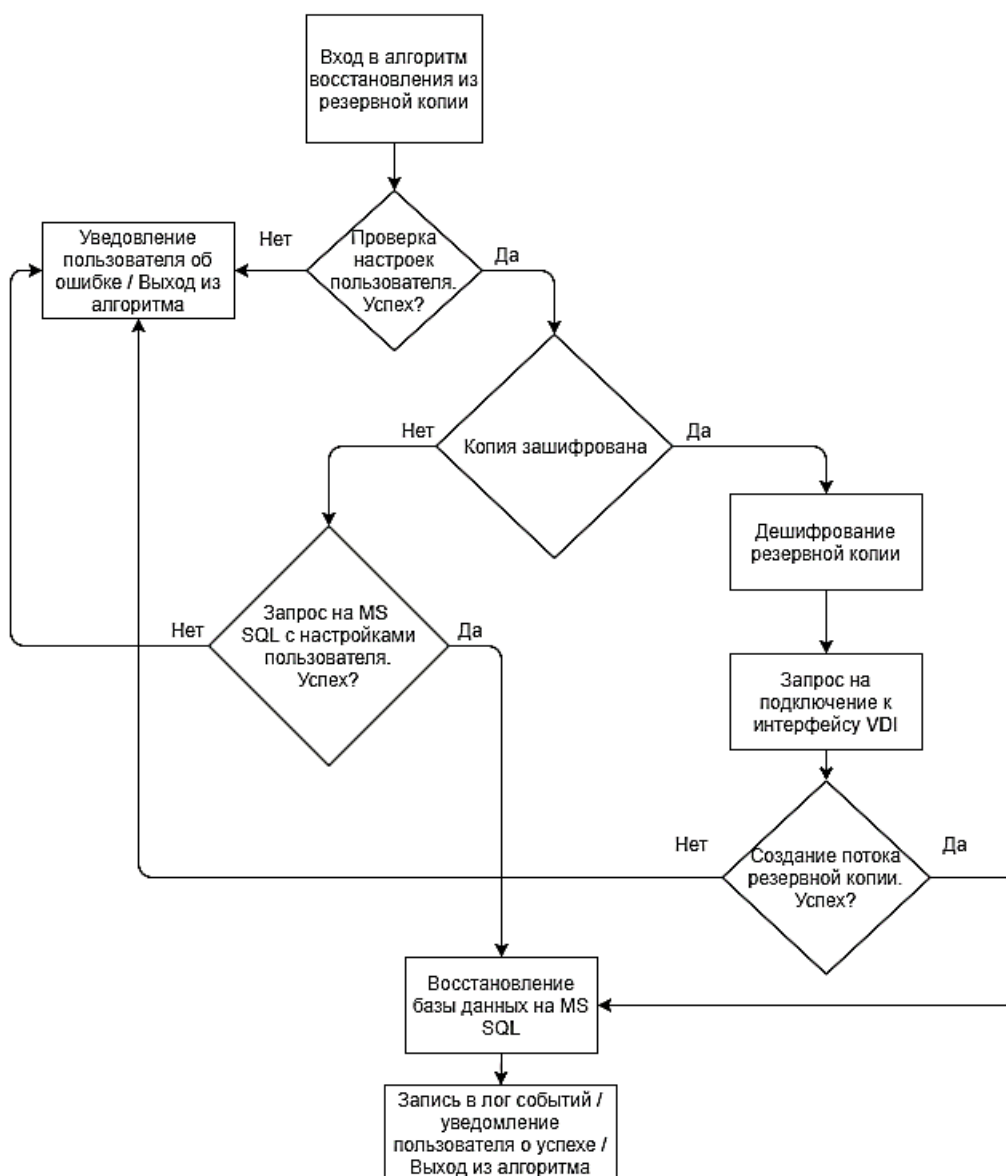


Рисунок 3.3 – Восстановление базы данных из резервной копии

3.4 Лог событий

В случае резервного копирования, ведение учёта действий является важной частью всего процесса, так как знать результат выполнения действий с базой данных необходимо, чтобы в случае сбоев можно было определить природу ошибки. Для этого в СУБД существует *лог событий*, в который записываются все действия пользователя, а также ведется учёт сбоев с их детальным описанием.

В разрабатываемом импортозамещающем программном приложении лог событий представляет собой совокупность таблиц во встроенной базе данных. Структура этих таблиц представлена на рисунке 3.4.

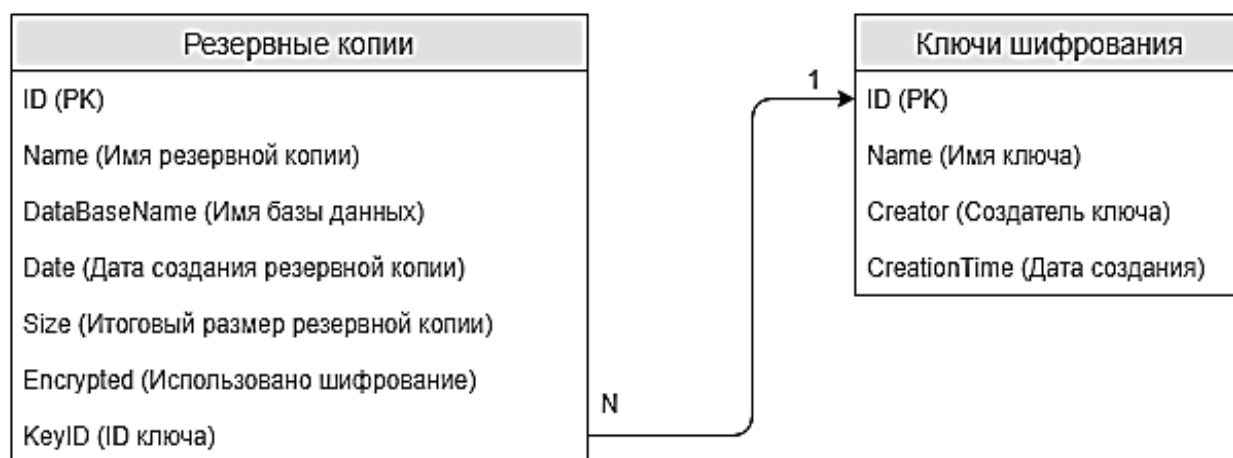


Рисунок 3.4 – Таблицы лога событий

Следует отметить, что запись в таблицы осуществляется только в случае успеха. Таким образом, просмотрев лог событий, можно проверить результат операции. Таблицы связаны между собой по ID ключа шифрования. В данном случае прослеживается связь один ко многим, так как одним и тем же ключом может быть зашифровано множество резервных копий, но она резервная копия зашифрована лишь одним конкретным ключом. В таблице ключей хранится информация о создателе ключа и дате создания. Имя создателя в таком случае соответствует логину из MS SQL Server, от имени которого пользователь создает ключ [40, 41].

3.5 Обеспечение безопасного резервного копирования. Интерфейс MS SQL VDI

Проблема, с которой можно столкнуться при создании зашифрованной резервной копии – это передача самого файла резервной копии в разрабатываемую программу. Так как MS SQL не позволяет расширить предлагаемый пользователю набор шифров, то шифровать резервную копию после её создания должна программа. Но тогда, перед тем, как передать файл резервной копии, он будет создан на носителе, и в этот момент времени есть риск, что он будет доступен для нарушителя.

Чтобы избавить разрабатываемый программный продукт от такой уязвимости, следует воспользоваться интерфейсом VDI, через который СУБД будет передавать поток создаваемой резервной копии непосредственно в программу.

VDI (Virtual Device Interface) – SQL Server предоставляет интерфейс прикладного программирования API (Application Programming Interface), являющийся интерфейсом виртуального устройства резервного копирования (VDI), который позволяет независимым поставщикам программного обеспечения интегрировать SQL Server в свои продукты для обеспечения поддержки операций резервного копирования и восстановления баз данных.

Эти API, представляющие собой набор готовых классов, процедур, функций, структур и констант, предоставляемых приложением (библиотекой, сервисом) или операционной системой для использования во внешних программных продуктах, специально разработаны для обеспечения максимальной надёжности и производительности. Они поддерживают полный спектр функций резервного копирования и восстановления SQL Server, в том числе полный спектр возможностей для быстрого создания снимков и резервного копирования [41].

Таким образом, создание файла резервной копии на физическом носителе в незашифрованном виде будет исключено. В случае завершения передачи потока резервной копии, соединение через VDI интерфейс завершается автоматически.

Выводы

В этом разделе была рассмотрена основная концепция разрабатываемого программного продукта. Разработаны алгоритмы создания резервной копии, а также восстановления базы данных из резервной копии.

Проанализирован процесс взаимодействия между таблицами базы данных в логике событий. Разработана структура взаимодействия разрабатываемого программного продукта с интерфейсом VDI.

4 ПРОГРАММНОЕ ПРОЕКТИРОВАНИЕ

4.1 Стек технологий и требования к пользователю

Для разработки приложения использован язык программирования C# на платформе .NET Framework 4.6. Этот выбор обусловлен тем, что C#, так же, как и MS SQL, разработан компанией Microsoft и предоставленный разработчику арсенал платформы отлично взаимодействует с технологией MS SQL.

Также использован интерфейс MS SQL VDI через программу–посредник PackDB. В этом случае PackDB будет являться вложенным ресурсом и помещаться в сборку во время компиляции. Для ведения лога событий требуется локальная база данных MSQlite, а для взаимодействия с платформой .Net Framework будет использована бесплатная библиотека Dapper [42].

Приложение будет использовать смешанную аутентификацию MS SQL. Для работы с приложением у пользователя должна быть учётная запись на MS SQL Server, которая должна состоять в группе sysadmin. Это является обязательным условием, так как команды, которые будет посылать приложение от имени этого пользователя, потребуют соответствующие права.

Также у пользовательской учетной записи Windows должны быть права на создание глобальных объектов (Create Global Objects). Эти права необходимы для создания резервной копии из стороннего приложения, которым и является разрабатываемая программа, настройки и требования которой рассмотрены далее.

Программа состоит из трех отдельных модулей:

- модуль пользовательского интерфейса;
- модуль шифрования резервных копий;
- модуль взаимодействия с MS SQL Server и интерфейсом VDI;
- модуль логирования событий.

Каждый модуль выполняет свою задачу и работает обособленно, что и является главным преимуществом такой архитектуры. Тот факт, что каждый модуль выполняет свою задачу, независимо от других, позволяет, в случае модификации модуля, легко заменить файл сборки с ним.

4.2 Модуль пользовательского интерфейса

Разрабатываемая программа является оконным приложением и включает в себя следующие компоненты интерфейса: главную форму; меню с настройками программы и справкой; окна настроек программы; работу с резервными копиями; окна создания резервной копии и восстановления базы данных; и дополнительный функционал – окна мастера создания ключей шифрования и окно лога событий.

Главная форма представлена на рисунке 4.1, она включает в себя меню с настройками и справкой, меню работы с резервными копиями, меню дополнительного функционала, а также окно вывода действий программы.

В меню настроек содержатся настройки соединения с сервером и настройки директорий. Кнопки создания резервной копии и восстановления базы данных вызывают отдельные формы, в которых пользователь сможет произвести необходимые действия. Кнопки дополнительного функционала вызывают форму мастера создания ключей, а также форму лога событий.

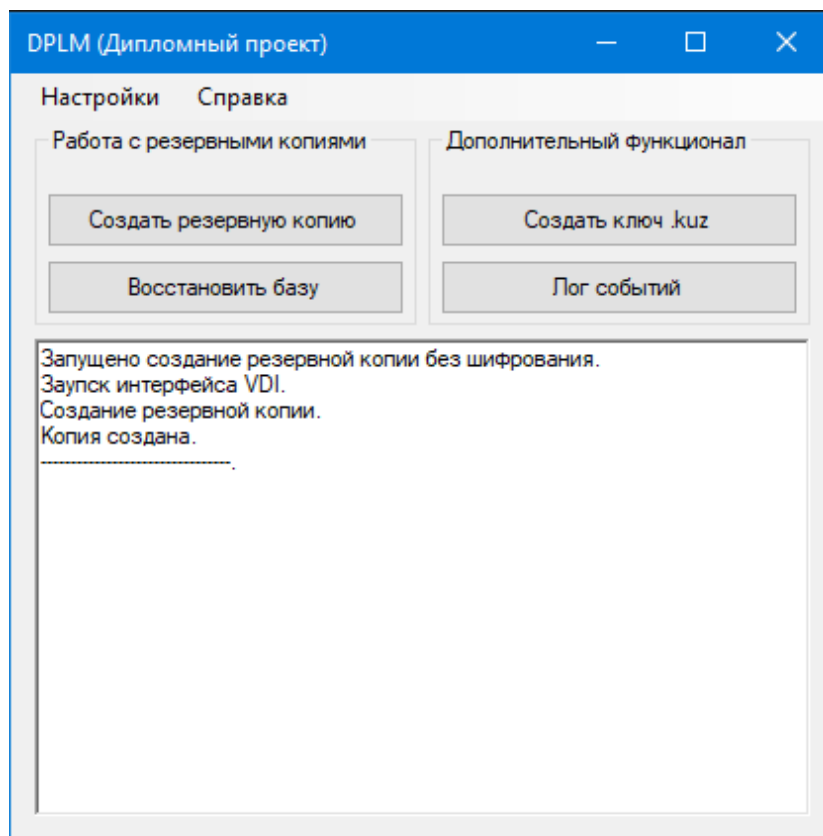


Рисунок 4.1 – Главная форма

Окна настроек программы представлены на рисунках 4.1 и 4.2. Эти окна вызываются из соответствующего подменю в меню «Настройки».

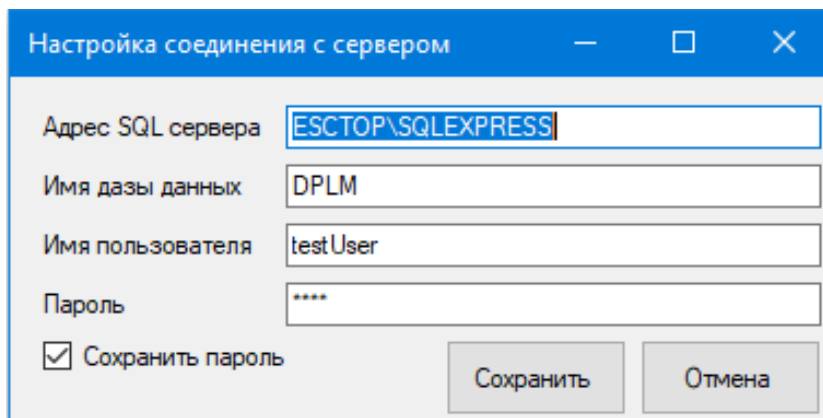


Рисунок 4.2 – Окно настройки соединения с сервером

В окне «Настройка соединения с сервером» пользователю необходимо указать адрес соединения с сервером, имя обрабатываемой базы данных, а также логин и пароль учетной записи на сервере MS SQL. По усмотрению пользователя пароль может быть сохранен, в противном случае, при каждом соединении с сервером программа будет запрашивать ввести пароль заново.

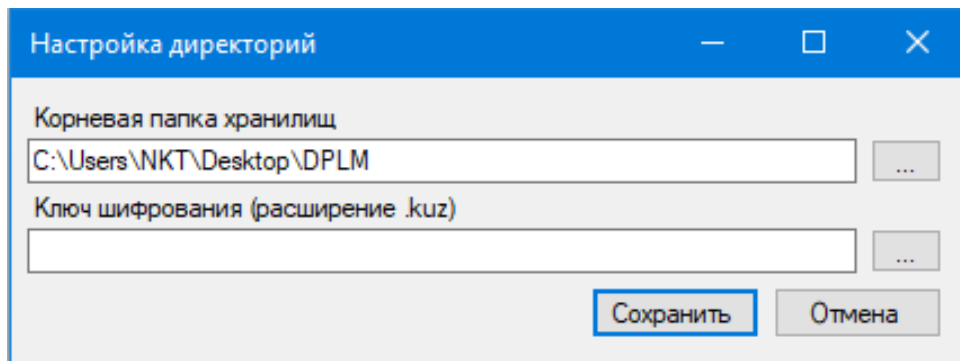


Рисунок 4.3 – Окно настройки директорий

В окне «Настройка директорий» пользователь может задать путь для сохранения резервных копий по умолчанию, а также путь к используемому ключу шифрования. В случае, если пользователь не установит эти параметры программа будет запрашивать их по необходимости.

Меню «Работа с резервными копиями» включает в себя две кнопки, кнопку «Создать резервную копию» и «Восстановить базу», каждая из них вызывает соответствующие окна, представленные на рисунках 4.4 и 4.5.

В окне «Создание резервной копии» пользователю необходимо указать путь сохранения файла резервной копии, так же он может изменить имя, которое по умолчанию заполняется автоматически. Имя базы данных заполняется самой программой и берется из настроек пользователя. Пользователь так же может зашифровать резервную копию, поставив галочку напротив этого параметра. Путь к ключу шифрования так же берется из настроек пользователя. В случае, если он пуст, программа предложит указать путь к ключу вручную.

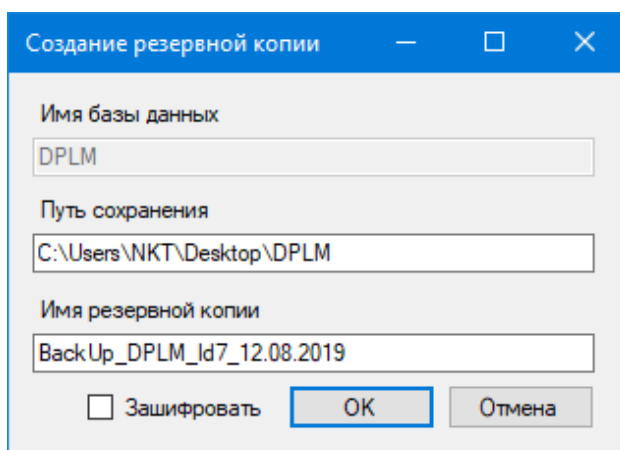


Рисунок 4.4 – Окно «Создание резервной копии»

Окно восстановления базы данных, показанное на рисунке 4.5, вызывается при нажатии на кнопку «Восстановить базу». В этом окне пользователю необходимо указать путь к резервной копии, а также путь к ключу шифрования, если резервная копия зашифрована. Название базы данных и пользователь, от имени которого будет проводиться восстановление, заполняются автоматически, причём параметры берутся из настроек пользователя.

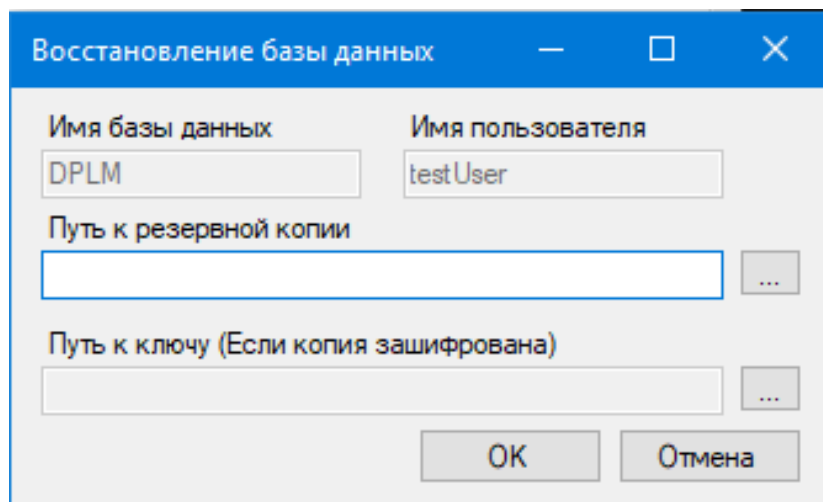


Рисунок 4.5 – Окно «Восстановление базы данных»

При нажатии на кнопку «ОК» будет произведена проверка на корректность указанных параметров, в случае ошибки программа уведомит пользователя о нарушении, как показано на рисунке 4.6.

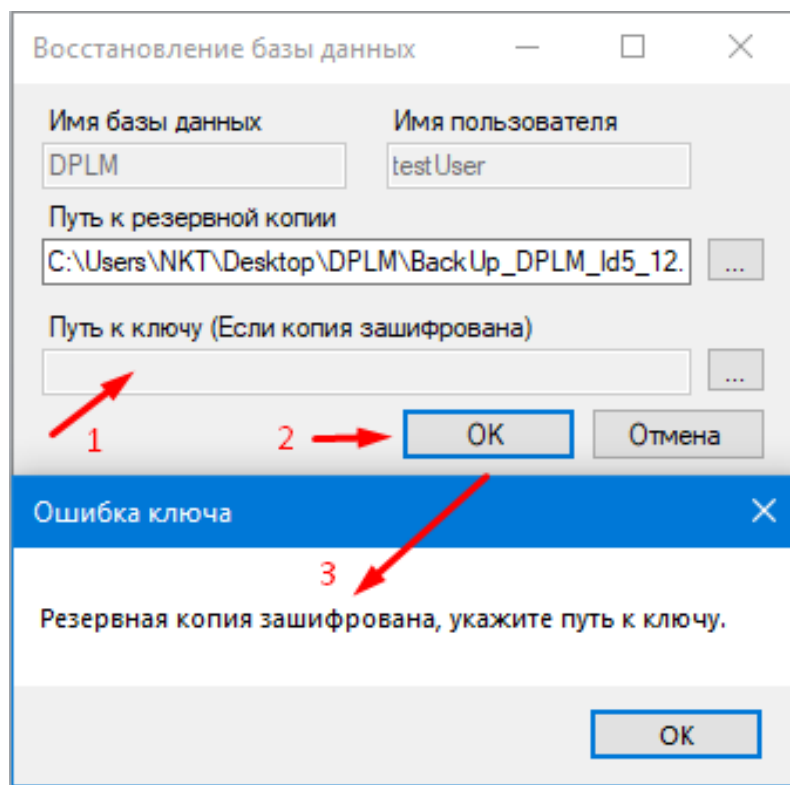


Рисунок 4.6 – Окно ошибки, вызванное валидатором

На рисунке 4.6 видно, как пользователь указал путь к резервной копии, но не указал путь к ключу. После нажатия на кнопку «ОК» программа запустила валидатор, который установил, что копия зашифрована, а путь к ключу отсутствует, и выдала исключение, уведомив пользователя об ошибке.

Меню с дополнительными функциями включает в себя две кнопки. Кнопка «Создать ключ.kuz» вызывает окно «Мастер создания ключей» представленный на рисунке 4.7. В этом окне пользователю будет представлен автоматически сгенерированный ключ шифрования, который рекомендуется изменить вручную, так как созданный ключ сгенерирован автоматически, что может оказаться возможной уязвимостью. В поле «Имя пользователя» будет указано имя учётной записи MS SQL из настроек. Дата создания устанавливается автоматически, указав текущее время. Ключ шифрования представлен в HEX-формате, для удобства редактирования. Пользователю нужно указать путь сохранения, по которому такой ключ будет создан. Также пользователю представлено предупреждение, в котором сказано о том, что автоматическая генерация ключей не является самым надёжным решением, и пользователь может самостоятельно внести изменения в ключ.

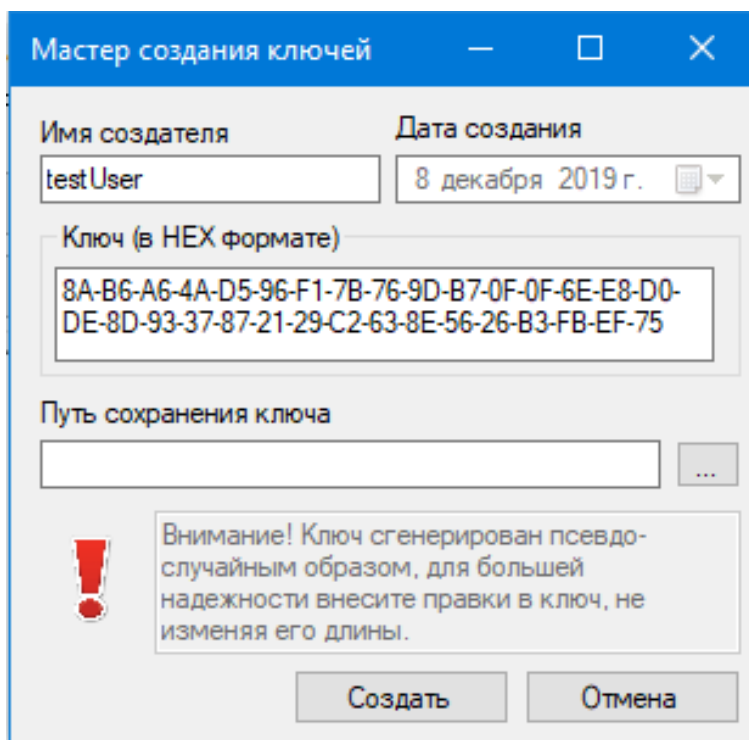
The image shows a Windows-style dialog box titled "Мастер создания ключей". It has a blue title bar with standard window controls. Inside, there are two input fields at the top: "Имя создателя" containing "testUser" and "Дата создания" showing "8 декабря 2019 г." with a calendar icon. Below these is a large text box labeled "Ключ (в HEX формате)" containing a long hexadecimal string. Underneath is a text box for "Путь сохранения ключа" followed by a browse button "...". At the bottom, there is a warning box with a red exclamation mark icon and the text: "Внимание! Ключ сгенерирован псевдо-случайным образом, для большей надежности внесите правки в ключ, не изменяя его длины." Below the warning are two buttons: "Создать" and "Отмена".

Рисунок 4.7 – Окно «Мастер создания ключей»

При нажатии на кнопку «ОК» валидатор проверит введенные пользователем параметры, и, в случае нарушения, выдаст предупреждение об ошибке, как показано на рисунке 4.8. В этом случае пользователь нарушил формат записи ключа. После нажатия на кнопку «Создать» валидатор обнаружил несоответствие и уведомил пользователя о допущенной ошибке.

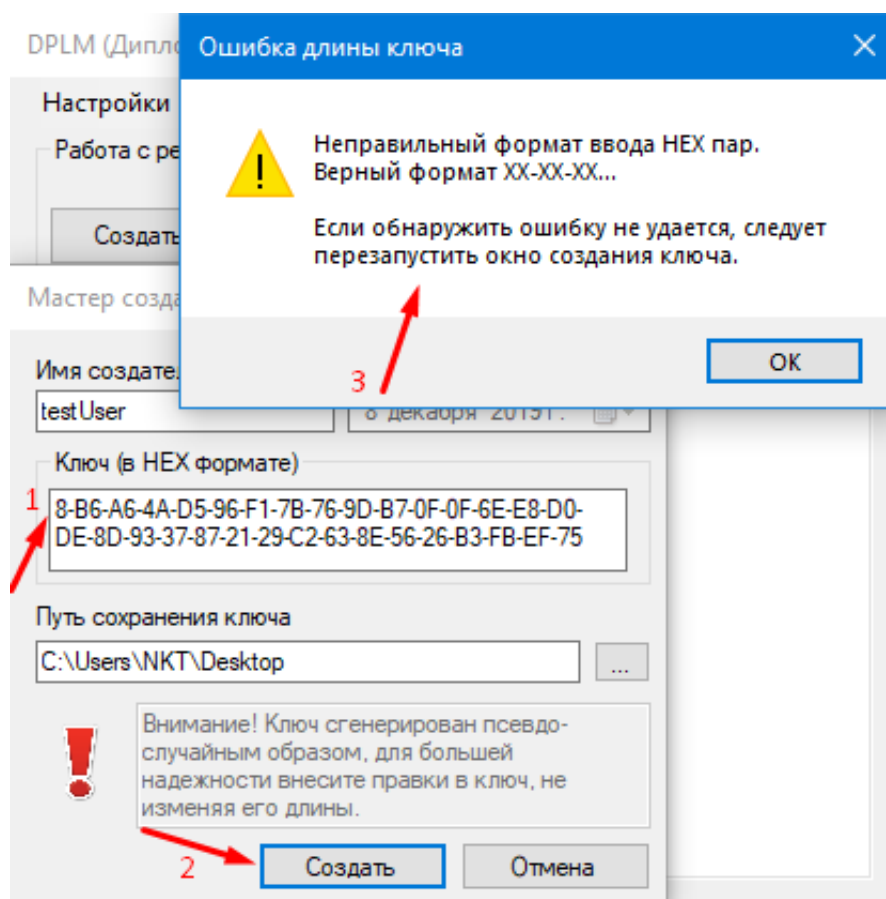


Рисунок 4.8 – Окно ошибки валидатора

Кнопка «Лог событий» вызовет форму, в которой будет показана сводная информация о работе программы. Окно лога событий представлено на рисунке 4.9. В нём можно увидеть дополнительную информацию о созданных ключах, резервных копиях и активных подключениях. Нажав соответствующие кнопки, можно увидеть сводные таблицы с результатом.

Лог событий								
Id	Имя резервной копии	Имя базы данных	Дата создания	Размер	Шифрование	ID ключа	Ключи	
1	1.bak	1	22.11.2019	1.17 MB	Нет			
2	BackUp_DPLM_Id2_12.06.2...	1	06.12.2019 17:27:44	4,08 MB	Да		Резервные копии	
3	BackUp_DPLM_Id3_12.06.2...	1	06.12.2019 17:48:38	4,08 MB	Нет			
4	BackUp_DPLM_Id4_12.06.2...	1	06.12.2019 17:50:27	4,08 MB	Да	2		
5	BackUp_DPLM_Id5_12.07.2...	DPLM	07.12.2019 16:29:40	4,08 MB	Да	3		
6	BackUp_DPLM_Id6_12.08.2...	DPLM	08.12.2019 9:55:33	4,08 MB	Нет		Активные подключения	

Рисунок 4.9 – Окно «Лог событий»

4.3 Модуль шифрования резервных копий

В модуле шифрования реализован блочный симметричный шифр «Кузнечик» из ГОСТ 34.12–2015. Реализация подразумевает работу в двух направлениях, шифрования и расшифрования. Оба алгоритма реализованы согласно ГОСТ. Блок схема с раундом шифрования представлена на рисунке 4.10.

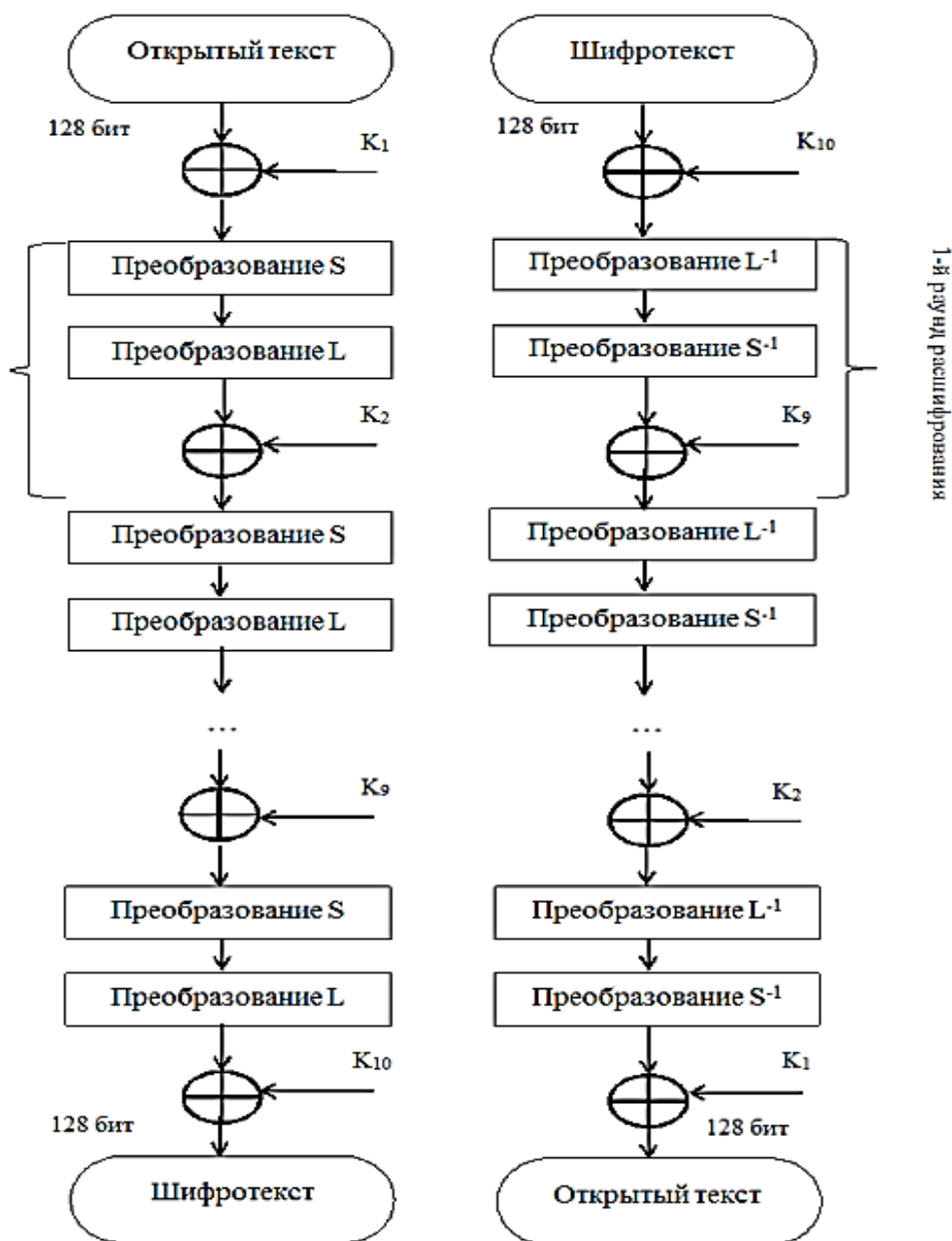


Рисунок 4.10 – Схема работы шифра «Кузнечик»

Модуль шифрования состоит из двух классов: «Ghost2015» и «KuzKey».

В теле класса «Ghost2015» содержится два публичных метода:

- `Encrypt(byte [] file, byte[] masterKey);`
- `Decrypt(byte [] file, byte[] masterKey).`

Результатом этих методов в обоих случаях является массив байтов. Внутри класса «Ghost2015» реализованы приватные методы преобразований и перестановок.

В классе «KuzKey» содержится структура ключа шифрования, которая включает в себя следующие параметры:

- «OwnerName» – имя владельца (создателя) ключа;
- «CreationDate» – время создания ключа;
- «HexKey» – ключ в формате HEX;
- «Error» – сообщение об ошибке при считывании ключа.

Данный класс содержит следующие методы:

- GetCipherKey();
- ReadKeyFromFile(string path);
- WriteItSelfAsync(string pathToSave).

«GetCipherKey» – метод, преобразующий HEX представление ключа в массив байт, используется при передаче массива байт в класс «Ghost2015».

«ReadKeyFromFile» – статический метод, позволяющий создать экземпляр класса «KuzKey» из сохраненного файла с расширением «.kuz».

Структуру файла «.kuz» можно увидеть на рисунке 4.11.

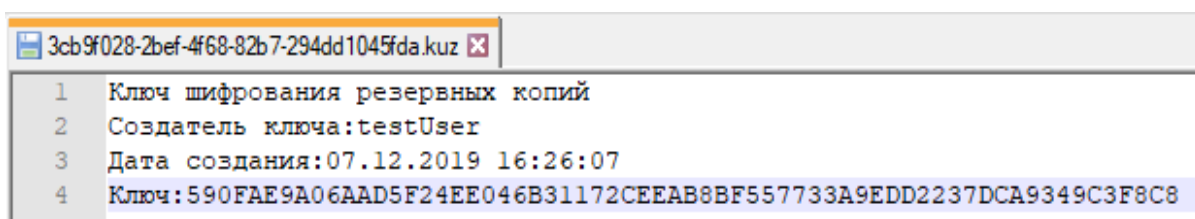


Рисунок 4.11 – содержимое файла с расширением «.kuz»

«WriteItSelfAsync» – асинхронный метод, позволяющий записать хранимые параметры структуры «KuzKey» в файл с расширением «.kuz».

Следует обратить внимание на то, что в файл форматом «.kuz» записывается текстовое представление ключа, используя кодировку UTF8.

Таким образом, содержимое файла можно просмотреть в любом текстовом редакторе, и в случае необходимости внести поправки.

4.4 Модуль взаимодействия с MS SQL и интерфейсом VDI

Написание драйвера для интерфейса VDI является сложной и объемной задачей, так как требует четкого понимания всей архитектуры VDI и программного взаимодействия посредством интерфейсов COM портов. В данном проекте разработка драйвера для VDI будет избыточной и можно воспользоваться бесплатно распространяемой программой PackDB от компании forsoft.

PackDB – консольное приложение для выполнения резервного копирования, восстановления и копирования баз данных MS SQL Server с использованием интерфейса VDI. PackDB так же может предоставить выходной поток через параметр stdout или получить входной поток через параметр stdin.

Таким образом, можно использовать данную программу для предоставления потока резервной копии базы данных в разрабатываемую программу и обратно через поток ввода/вывода консольного приложения PackDB.

Модуль программы содержит в себе класс контроллер «BackUpController» в теле которого существует 4 метода:

- CreateBackUpWithoutEncryption(string fullFileName);
- CreateBackUpWithEncryption(string fullFileName, string KeyPath);
- UploadBackUpWithoutEncryption(string backUpPath);
- UploadBackUpWithEncryption(string backUpPath, string KeyPath).

Перечисленные методы являются асинхронными, что позволит вызвать их выполнение в отдельном потоке.

CreateBackUpWithoutEncryption – метод для создания простой резервной копии на вход получает путь для сохранения созданной копии.

CreateBackUpWithEncryption – метод создания зашифрованной резервной копии, который на вход получает и путь для сохранения, и путь к ключу на диске.

UploadBackUpWithoutEncryption – метод восстановления базы данных из резервной копии без шифрования. На вход получает путь к резервной копии.

UploadBackUpWithEncryption – метод восстановления базы данных из зашифрованной резервной копии, на вход получает путь к зашифрованной резервной копии, а также к ключу которым эта копия была зашифрована.

Методы шифрования, использованные в теле методов, принадлежат модулю шифрования. Так, этот программный модуль отвечает исключительно за создание резервных копий и восстановление базы данных в случае её повреждения, а также за передачу полученного потока в модуль шифрования.

4.5 Модуль логирования событий

Для ведения лога событий и документации потребуется локальная база данных. Так как архитектура приложения не подразумевает, что данное программное обеспечение будет использоваться асинхронно по сети, то для ведения лога будет достаточно интегрированной базы данных в самом приложении, с такой задачей замечательно справится компактная база данных SQLite.

SQLite—это портативная СУБД, являющаяся по совместительству встраиваемой в проект библиотекой. Основным достоинством SQLite является высокая скорость работы, которая достигается отсутствием соединений по типу «сервер–клиент» и «клиент–сервер», а также возможность хранить все данные в одном файле.

Для работы с SQLite существует множество бесплатных библиотек, но в случае данной разработки будет использована библиотека Dapper.

Dapper – это библиотека объектно–реляционного сопоставления для платформы Microsoft .NET: он предоставляет платформу для сопоставления объектно–ориентированной модели предметной области с традиционной реляционной базой данных. Благодаря своему малому весу, Dapper предоставляет высокую производительность и позволяет выполнять запросы быстрее, чем EF Core.

Модуль лога событий состоит из следующих классов:

- «BackUpInfo»;
- «KeyInfo»;
- «SQLiteDataAccess».

В модуль лога событий включена база данных со следующими таблицами:

- «DataBaseBackUpList»;
- «Keys».

«BackUpInfo» – это абстрактный класс, являющийся моделью таблицы «DataBaseBackUpList» из базы данных. Позволяет упростить взаимодействие модели данных с кодом программы, считывая и сохраняя данные из базы данных, используя библиотеку Dapper.

«KeyInfo» – также абстрактный класс над таблицей «Keys» и, аналогично классу «BackUpInfo», служит моделью представления данных для программы.

«SQLiteDataAccess» – класс–контроллер, содержит в своем теле методы для взаимодействия с базой данных. Эти методы нацелены на запись в таблицы базы данных SQLite, а также на считывание данных из этих таблиц.

Модуль логирования тесно интегрирован в другие модули, так как ведет запись в реальном времени, после завершения ключевых операций в программе.

Методы вызываются из модуля пользовательского интерфейса, для отображения содержимого таблиц базы данных в окне лога событий. Строка подключения к базе данных SQLite интегрирована в проект и должна оставаться неизменной. Но несмотря на это, пользователь программы может открыть файл базы данных «BackUpInfo.db», используя стандартный редактор баз данных от SQLite.

Выводы

В этом разделе была создана программная реализация разрабатываемого приложения. Выбран стек технологий, который потребовался для реализации.

В его состав вошли следующие технологии:

- .Net Framework 4.6;
- SQL и T-SQL для работы с MS SQL Server;
- SQLite и Dapper.

В разрабатываемом приложении были успешно реализованы алгоритмы, описанные в предыдущем разделе. Рассмотрены следующие модули приложения:

- Модуль пользовательского интерфейса;
- Модуль шифрования резервных копий;
- Модуль взаимодействия с MS SQL Server и интерфейсом VDI;
- Модуль логирования событий.

По итогу программной разработки создано импортозамещающее ПО, которое обеспечивает выполнение всех поставленных задач.

РЕЗУЛЬТАТЫ 6 ГЛАВЫ

В современном мире много внимания уделяется безопасному хранению информации. Существует множество готовых решений, которые могут позволить их пользователям надежно защитить информацию от злоумышленника. Системы управления базами данных также наделены этой возможностью и предоставляют администратору баз данных необходимый арсенал для обеспечения должного уровня безопасности информации в системе. Но существующий инструментарий в СУБД MS SQL Server не подразумевает использование российского ГОСТа в качестве алгоритма шифрования резервных копий.

В ходе работы были проанализированы методы резервного копирования, а также доступные администратору, алгоритмы шифрования резервных копий. Согласно проведенному анализу российский алгоритм «Кузнечик» является отличной альтернативой постепенно устаревающему AES, поскольку не уступает в качестве и скорости работы. При том, что российский стандарт вышел намного позже, он уже успел отлично себя зарекомендовать.

В работе представлены алгоритмы создания зашифрованной резервной копии, а также восстановления базы данных из зашифрованной резервной копии. Данные алгоритмы подразумевают использование российского стандарта в совокупности с технологией MS SQL VDI, позволяющей передавать поток создаваемой резервной напрямую в разработанное программное средство, что исключает возможность утечки копии базы данных до её шифрования.

На основании разработанных алгоритмов было сконструировано программное обеспечение, включающее в себя пользовательский интерфейс, модуль шифрования, модуль логирования и модуль взаимодействия с MS SQL Server.

В результате проделанной работы были достигнуты все поставленные цели:

- изучены методы резервного копирования доступные в MS SQL Server, а также определены их преимущества и недостатки;
- рассмотрено шифрование в MS SQL Server, и проведено сравнение между доступными алгоритмами в MS SQL Server и ГОСТ Р 34.12 «Кузнечик»;
- разработаны алгоритмы создания зашифрованной резервной копии и восстановления базы данных, в случае её повреждения;
- разработано программное обеспечение для создания резервных копий, и восстановления баз данных в MS SQL Server с использованием алгоритма шифрования ГОСТ Р 34.12 «Кузнечик».

Реализованное импортозамещающее программное обеспечение позволяет надежно защитить резервные копии баз данных в MS SQL Server.

К недостаткам данного программного обеспечения можно отнести сложность предварительной настройки, ограничение в выборе используемой операционной системы, повышение нагрузки на сервер MS SQL и увеличение времени выполнения создания резервной копии.

Однако данная программная реализация имеет ряд преимуществ, в их числе упрощенное взаимодействие с сервером для конечного пользователя, а также шифрование посредством криптоалгоритма ГОСТ 34.12 Р 2015 «Кузнечик».

Разработанное импортозамещающее программное приложение предназначено для использования в отечественных структурах – государственных, военных, промышленных, банковских, научных, медицинских, образовательных и др. – везде, где возникает необходимость в безопасном хранении важной информации, требуется надёжная защита данных от действий злоумышленников различного рода и необходимо обеспечение должного уровня безопасности и защиты данных в информационных системах.

ГЛАВА 7

РАЗРАБОТКА ИМПОРТОЗАМЕЩАЮЩЕЙ СИСТЕМЫ РАЗГРАНИЧЕНИЯ ДОСТУПА ДЛЯ РАБОТЫ С ПРОГРАММНЫМ СРЕДСТВОМ, ПОСТРОЕННЫМ ПО МИКРОСЕРВИСНОЙ АРХИТЕКТУРЕ ДЛЯ ЗАЩИТЫ ИНФОРМАЦИИ ОТ НЕСАНКЦИОНИРОВАННОГО ДОСТУПА

ОБЩИЕ ПОЛОЖЕНИЯ

Перспективным направлением разработки архитектуры современных программных приложений является построение систем с применением микросервисной архитектуры, где приложение представляется в виде нескольких небольших сервисов, каждый из которых работает в отдельном потоке, а модули общаются между собой посредством протокола HTTP. Этот принцип модульности позволяет создавать проекты, модули которых слабо связаны между собой, что обеспечивает хорошую горизонтальную масштабируемость.

В связи с тем, что общение между сервисами осуществляется посредством сетевого взаимодействия, возникает необходимость обезопасить каждый из сервисов, чтобы потенциальный злоумышленник не смог получить доступ к секретным или конфиденциальным данным, которые ему не принадлежат.

Поскольку микросервисная архитектура является новым принципом в построении программных приложений, в том числе импортозамещающих, разработка системы разграничения доступа и применение моделей доступа для таких программных средств – слабо изученный вопрос, требующий изучения, поэтому исследования данного вопроса является актуальной задачей.

Объект исследования – используемые современные алгоритмы авторизации.

Предмет исследования: алгоритмы, позволяющие ограничить доступ к приложению, построенные по микросервисной архитектуре.

Целью исследования является разработка архитектуры защиты программного средства, построенного по микросервисной архитектуре, обеспечивающей недопущение несанкционированного доступа к секретной или конфиденциальной информации, обрабатываемой в приложении, и её реализация.

Для достижения поставленной цели необходимо решить ряд задач:

1. Изучить классификацию алгоритмов авторизации.
2. Выявить их достоинства и недостатки.
3. Провести анализ существующих алгоритмов авторизации и обосновать выбор алгоритма обеспечения защиты информации от НСД.
4. Разработать архитектуру слоя безопасности, обеспечивающего защиту данных.
5. Выбрать язык программирования с используемым стеком технологий и реализовать импортозамещающее программное приложение, демонстрирующее работающую систему разграничения доступа.

Для решения поставленных задач были использованы методы авторизации, основы построения защищенных программных приложений, а также основы теории систем управления базами данных.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Основные сведения

Идентификация и аутентификация – понятия, применяемые, когда заходит речь об ограничении доступа субъектов информационных систем к объектам информационных систем (пользователей к аппаратным и программным ресурсам).

Общим шаблоном действий систем, где внедрены эти понятия, является следующий порядок действий: от субъекта система получает идентификационный параметр (идентификацию), проверяет его подлинность (аутентификацию), а впоследствии предоставляется доступ (авторизацию), либо получает отказ в доступе.

Наличие вышеперечисленных процедур является обязательным условием любых информационных систем, поскольку любые модели доступа основаны на работе субъектов с объектами. Дадим более строгие определения этих понятий.

Идентификация — присвоение субъектам и объектам доступа личного идентификатора и сравнение его правильности с заданным идентификатором.

Аутентификация (установление подлинности) — проверка принадлежности субъекту доступа предъявленного им идентификатора и подтверждение его подлинности. Другими словами, аутентификация заключается в проверке того, является ли подключающийся субъект тем, за кого он себя выдает.

Общая процедура идентификации и аутентификации пользователя при его доступе в АС представлена на рисунке 1.1. Если в процессе аутентификации подлинность субъекта установлена, то система защиты информации должна предоставить те или иные права доступа данному пользователю в зависимости от того, какой моделью разграничения доступа пользуются в текущем случае.

По контролируемому компоненту системы, способы аутентификации можно разделить на аутентификацию партнеров по общению и аутентификацию источника данных. *Аутентификация партнеров по общению* используется при установлении (и периодической проверке) соединения во время сеанса. Она служит для предотвращения таких угроз, как маскарад и повтор предыдущего сеанса связи. *Аутентификация источника данных* — это подтверждение подлинности источника отдельной порции данных.

По направленности аутентификация подразделяется на *одностороннюю* (когда пользователь доказывает свою подлинность системе, например, при входе в систему), и *двустороннюю* (взаимную, когда подлинность должны подтвердить обе стороны, и пользователь, и система).

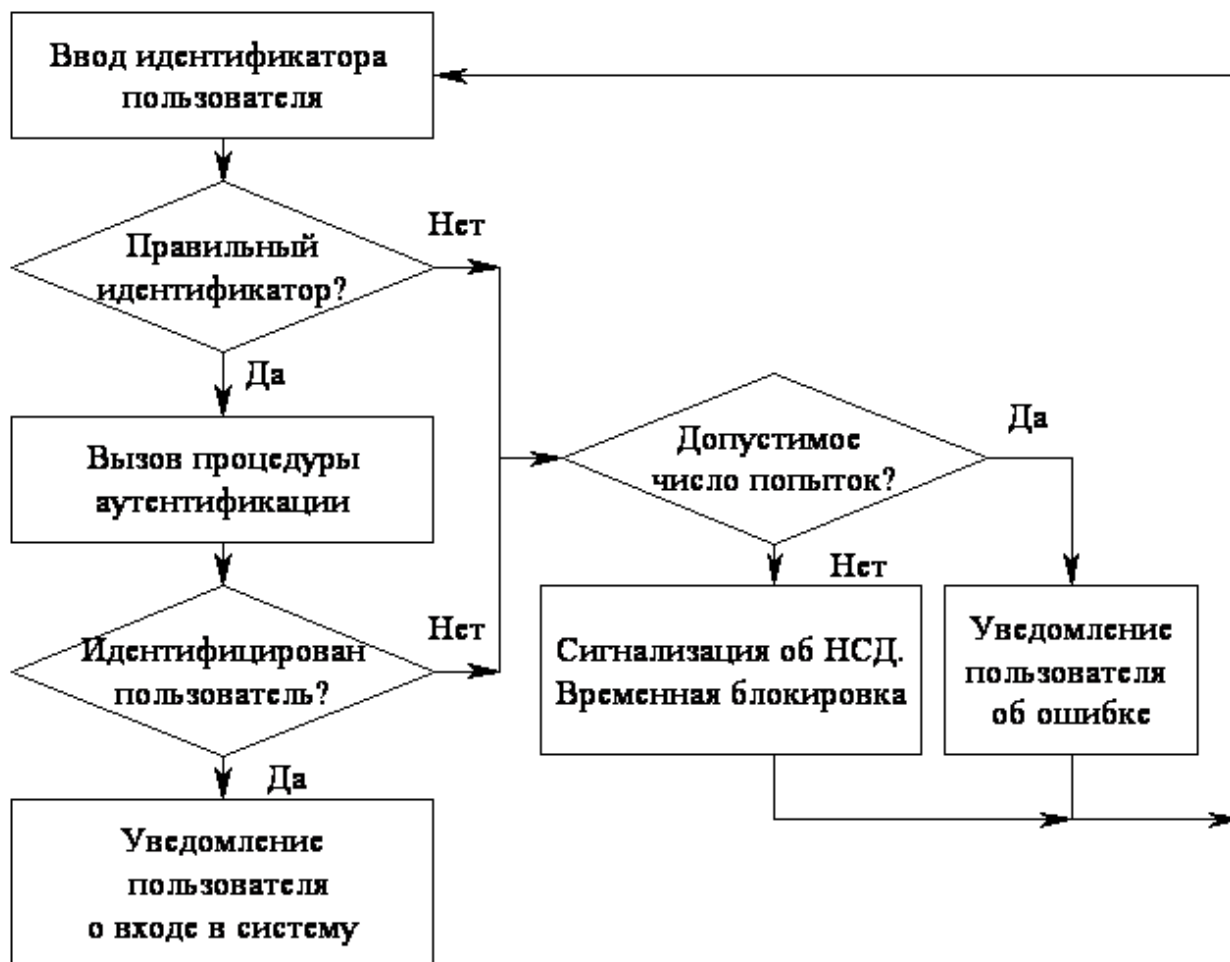


Рисунок 1.1 – Процедура идентификации и аутентификации

1.2 Виды аутентификации

Основным классификационным признаком методов аутентификации является тот признак, по которому устанавливается подлинность субъекта:

1. Методы аутентификации, использующие в своей основе секретную информацию, которую знает только субъект – пароль.
2. Методы аутентификации, применяющие в своей основе некий уникальный материальный предмет – жетон, карточку, ключ и т.д.
3. Методы, использующие в своей основе биометрические параметры человеческого организма: отпечатки пальцев, радужная оболочка, модель лица.
4. Методы аутентификации, использующие в своей основе информацию, ассоциированную с пользователем (с его координатами).

Рассмотрим эти группы подробнее.

1.2.1 Авторизация паролем

Наиболее распространенными простыми и привычными являются методы аутентификации, основанные на *паролях* – секретных идентификаторах субъектов. Субъект, вводя свой пароль, посылает его на модуль безопасности, который, в свою очередь сравнивает с тем паролем, который хранится внутри своей базы. Для обеспечения повышенной безопасности в базе хранится не сам пароль в явном виде, а его хэш, вычисленный одним из распространенных методов. И на модуль приходит не сам пароль, а уже хэш от пароля, который вычисляется на клиентской части для того, чтобы он в явном виде не отсылался по сетевым каналам передачи информации. В случае совпадения хэшей, модуль безопасности либо предоставляет пользователю доступ к информационной системе, в соответствии с используемой моделью разграничения доступа, либо фиксирует факт попытки несанкционированного доступа к системе, т.е. её взлома.

Парольные методы следует более глубоко разделить по сроку жизни используемых паролей: методы с *константными, постоянно используемыми паролями* (со временем сами пароли не меняются, они могут быть изменены только по воле субъекта); и методы с *динамичными паролями*, где для каждого факта аутентификации требуется получить и ввести в систему новый пароль.

В подавляющем большинстве автоматизированных систем используются *постоянные*, или *многоцветные* пароли. В этом случае пароль остается постоянным, и меняется только в тех особых случаях, когда заканчивается установленный администратором срок жизни пароля, либо, когда субъект сам захочет изменить свой пароль. У этого метода есть как преимущества, так и недостатки. Из плюсов – упрощается администрирование системы, поскольку не нужно реализовывать архитектуру для применения динамических паролей. Из минусов – повышается вероятность рассекречивания секрета либо путем *подсматривания* при наборе пользователем этого пароля, либо путем *перехвата* сеанса связи и раскрытия этого пароля (либо его хэша). Также существует вероятность подбора пароля, если тот достаточно прост и создан без применения специальных символов, написан в одном регистре или имеет какую-либо смысловую нагрузку, например, имя кошки, номер почтового индекса, дата рождения. В случае простого короткого пароля можно с помощью *брута* выяснить, каким паролем защищен тот или иной пользователь. (*Брут*, или *брутфорс*, термин, произошедший от английского выражения «brute force» — метод «грубой силы» или полного перебора – один из популярных у хакеров методов взлома паролей на серверах и в различных программах).

Более надежным способом считается использование *одноразовых* или *динамически меняющихся* паролей для каждого входа в систему.

Известны следующие методы парольной защиты, основанные на одноразовых паролях: методы *модификации схемы простых паролей*; методы типа «*запрос–ответ*»; а также *функциональные методы*.

В первом случае пользователю присваивается сразу же несколько паролей, каждый из которых будет верен. Модуль безопасности запросит порядковый номер пароля из списка, и пользователь должен будет ввести этот пароль. Когда список паролей будет исчерпан, субъекту выдается новый список паролей. Все эти пароли генерируются случайным образом.

В методе «запрос–ответ» система задает те или иные вопросы, ответы на которые могут быть известны только настоящему владельцу.

В основе функциональных методов лежат специальные функции, которые способны изменять пароль пользователя. Это позволяет обеспечить возможность изменения (по некоторой формуле) паролей пользователя во времени. Указанная функция должна удовлетворять следующим требованиям:

- для заданного пароля x легко вычислить новый пароль;
- зная x и y , сложно или невозможно определить функцию $y = f(x)$.

Наиболее известными примерами функциональных методов являются: метод функционального преобразования и метод «рукопожатия».

Идея метода *функционального преобразования* состоит в регулярном изменении функции самого пароля. Параметры, которые будут меняться, могут быть разными: либо текущая дата, либо время, прошедшее с момента регистрации, либо какая–нибудь другая мера. Несложно заметить, что с течением времени пароль пользователя будет представлен в виде следующей последовательности: $x, f(x), f(f(x))$ и так далее [43].

Метод «*рукопожатия*» состоит в следующем. Функция парольного преобразования известна только субъекту и модулю безопасности. При входе в информационную систему подсистема аутентификации выдает случайную последовательность x , которая передается пользователю. Пользователь вычисляет результат функции $y = f(x)$ и возвращает его в систему. Система проверяет результат вычисления клиента с полученным ей результатом, и в случае, если эти результаты совпадают, пользователю предоставляется доступ к системе и аутентификация считается успешной.

Достоинством этого метода является тот факт, что передача секретных данных в канале связи сведена к минимуму, что снижает вероятность утечки или перехвата информации злоумышленником.

Для случаев, когда пользователю необходимо проверить подлинность другого пользователя или узла информационной системы, с которой он работает, метод рукопожатия является наиболее оптимальным, поскольку между конечными точками никакой конфиденциальной информации не передается. Необходимо только обеспечить передачу секретной функции, в чём может помочь известный протокол Диффи–Хеллмана.

Стоит отметить, что такой способ не является полностью защищенным, поскольку этот метод не исключает возможности того, что злоумышленник может подключиться к каналу связи и перехватывать пакеты, тем самым подменяя содержание смысловой части этих пакетов. В таком случае необходимо реализовать подпись содержимого этих пакетов, чтобы избежать перехвата.

1.2.2 Авторизация уникальным идентификатором

В последнее время получили распространение комбинированные методы идентификации, требующие, помимо знания пароля, *наличие карточки* (token) – специального устройства, подтверждающего подлинность субъекта.

Карточки разделяют на два типа [44]:

- пассивные (карточки с памятью);
- активные (интеллектуальные карточки).

Самыми распространенными являются пассивные карточки с магнитной полосой, которые считываются специальным устройством, имеющим клавиатуру и процессор. При использовании указанной карточки пользователь вводит свой идентификационный номер. В случае его совпадения с электронным вариантом, закодированным в карточке, пользователь получает доступ в систему. Это позволяет достоверно установить лицо, получившее доступ к системе и исключить несанкционированное использование карточки злоумышленником (например, при её утере легитимным пользователем). Такой способ часто называют двухкомпонентной аутентификацией.

Иногда (обычно для физического контроля доступа) карточки применяют сами по себе, без запроса личного идентификационного номера.

К достоинству использования пассивных карточек относят тот факт, что обработка аутентификационной информации выполняется устройством чтения – *ридером* без передачи данных в память компьютера. Это исключает возможность электронного перехвата данных по каналам связи.

Недостатки пассивных карточек следующие: они существенно дороже паролей, требуют специальных устройств чтения – *ридеров*, их использование подразумевает специальные процедуры безопасного учета и распределения. Их также необходимо оберегать от злоумышленников, и, естественно, не оставлять в устройствах чтения. Известны случаи подделки пассивных карточек.

Интеллектуальные карточки, кроме памяти, имеют собственный встроенный чип – микропроцессор. Это позволяет реализовать различные варианты парольных методов защиты: многоразовые пароли, динамически меняющиеся пароли, обычные запрос–ответные методы. Все карточки обеспечивают двухкомпонентную аутентификацию. К достоинствам интеллектуальных карточек следует добавить их многофункциональность. Их можно применять не только для целей безопасности при аутентификации, но и, например, для различных финансовых операций. Сопутствующим недостатком карточек является их высокая стоимость.

1.2.3 Авторизация биометрией

Методы аутентификации, основанные на использовании биометрических параметров человека, обеспечивают достаточно высокую надежность защиты объектов, решая проблемы утраты паролей и личных идентификаторов, поскольку эти параметры привязаны к человеку и скопировать эти данные является очень затруднительной задачей. Однако такие методы нельзя использовать при идентификации процессов или данных (объектов данных), поскольку технология является новой, и специалистами не был разработан единый стандарт для распределенных систем. Также сказывается и относительная дороговизна оборудования, а также его внедрения в информационные системы. Это обуславливает их использование пока только на особо важных объектах и системах.

Для идентификации субъекта могут использоваться такие параметры человеческого организма, как радужная оболочка глаза, отпечатки пальцев или ладони, узор ушной раковины, инфракрасной картинке капиллярных сосудов. Используется опознание по почерку и его параметрам, по потожировым выделениям, по тембру голоса, что широко распространено в мобильных устройствах и начинает распространяться в банковских системах, и даже по ДНК.

Новым направлением идентификации личности является использование биометрических характеристик в интеллектуальных расчётных карточках, в жетонах–пропусках и в элементах сотовой связи. Например, при расчёте в магазине, предъявитель карточки кладет палец на сканер в подтверждение, что карточка действительно принадлежит ему.

Перечислим наиболее часто используемые биометрические характеристики и атрибуты, а также соответствующие им технические системы.

Отпечатки пальцев: такие устройства имеют достаточно компактные размеры и самую низкую стоимость по сравнению с остальными техническими устройствами. В настоящее время такие приборы пропагандируются правоохранительными органами разных стран в связи с тем, что им выгодно собирать базу отпечатков пальцев с целью повышения раскрываемости преступлений.

Геометрия руки: соответствующие устройства распознавания руки используются на таких промышленных технологических объектах, на которых из-за повышенного загрязнения рук работников, или из-за применения на производстве химических реактивов затруднительно использовать сканеры отпечатков пальцев.

Радужная оболочка (или сетчатка) глаза: подобные сканирующие устройства распознавания обладают наивысшей точностью. Вероятность того, что система безопасности пропустит человека, которого пропускать не стоило – около 0,1%.

Голос: проверка голоса может служить средством авторизации в любых мобильных устройствах. Помимо мобильных устройств, данный метод применим и для доступа в информационные системы и со стационарных ПК. Устройства, необходимые для этого (звуковая карта и микрофон) стоят порядка 1000 рублей, что достаточно недорого. Вероятность ошибки составляет 2–5%. Минусом данного метода может быть временное или постоянное изменение голоса вследствие различных факторов – человек болен или подвержен эмоциональному потрясению.

Ввод с клавиатуры: при вводе ключевой фразы с клавиатуры отслеживается как корректность введенной фразы, так и частотные характеристики, описывающие интервалы нажатия между парами клавиш, свойственных тому или иному человеку.

Подпись: для контроля рукописной подписи используются дигитайзеры.

1.2.4 Аутентификация по местоположению

Новым методом является аутентификация по геоинформации, характерной субъектам. Этот метод реализован на основе системы космической навигации – GPS или ГЛОНАСС. Пользователь, имеющий аппаратуру, способную связаться со спутниками, многократно посылает координаты определенных спутников, которые находятся для субъекта в зоне приёма. Подсистема аутентификации, траектории движения спутников, может с достаточно высокой точностью определить местоположение субъекта. Высокая надежность аутентификации определяется тем, что орбиты спутников подвержены колебаниям, предсказать которые достаточно трудно. Кроме того, координаты спутников и пользователей меняются во времени, что позволяет снизить вероятность перехвата [45].

Плюсом этого метода является тот факт, что аппаратура GPS достаточно проста и надёжна в использовании, а также стоит сравнительно недорого. Это позволяет использовать её в тех случаях, когда авторизованный пользователь должен получать доступ к информационной системе только в тех случаях, когда он находится в строго определенном месте.

Суммируя возможности средств аутентификации, можно классифицировать их по уровню информационной безопасности на три категории:

1. Статическая аутентификация.
2. Устойчивая аутентификация.
3. Постоянная аутентификация.

Первая категория обеспечивает защиту только от несанкционированного доступа в системах, где нарушитель не может во время сеанса работы прочесть аутентификационную информацию. Примером статической аутентификации являются традиционные постоянные пароли. Их эффективность зависит от сложности угадывания паролей и от того, насколько хорошо они защищены. Для компрометации статической аутентификации нарушитель может подсмотреть, подобрать, угадать или перехватить аутентификационные данные и т. д.

Устойчивая аутентификация использует динамические характеристики аутентификации, изменяющиеся с каждым сеансом работы. Реализациями устойчивой аутентификации являются системы, использующие одноразовые пароли и электронные подписи. *Усиленная аутентификация* обеспечивает защиту от атак, где злоумышленник может перехватить аутентификационную информацию и попытаться использовать ее в следующих сеансах работы. Однако устойчивая аутентификация не обеспечивает защиту от активных хакерских атак, в ходе которых маскирующийся злоумышленник может оперативно (в течение одного сеанса аутентификации) перехватить, модифицировать и вставить изменённую информацию в поток передаваемых данных.

Постоянная аутентификация обеспечивает идентификацию каждого блока передаваемых данных, что предохраняет их от несанкционированного доступа, модификации или вставки. Примером реализации подобной категории аутентификации является использование алгоритмов генерации электронных подписей для каждого бита пересылаемой информации.

1.2.5 Многофакторная аутентификация

Перспективным направлением в развитии современных систем ограничения доступа к программным приложениям является внедрение в информационные системы многофакторной аутентификации.

Многофакторная аутентификация – это расширенная аутентификация, где пользователю необходимо предоставить более одного доказательства о том, что этот пользователь действительно тот человек, за кого себя выдает.

В использовании данного метода лежит 3 фактора, которые выделяют специалисты в области защиты информации. Какую совокупность этих факторов использовать, зависит от того, насколько высоки требования к надёжности и защищенности программного обеспечения, а также от того, какие средства были выделены на то, чтобы построить эффективную систему защиты.

Перечислим эти факторы.

1. *Фактор знания.* Это те секретные знания, которыми должен обладать только подлинный владелец. Зачастую это может быть либо пароль (числовой или символьный), или вопрос, ответ на который не может быть известен больше никому.

2. *Фактор владения.* Под этим термином подразумевается то, что может быть в распоряжении только у валидного субъекта – легитимного пользователя. В текущее время широкое распространение получила аутентификация по SMS, которое приходит на указанный при регистрации пользователем номер телефона. В этом случае уникальным предметом является SIM–карта.

3. *Фактор неотъемлемого свойства.* Под этим понятием подразумевают личную биометрическую уникальность человеческого организма, и с точки зрения субъекта является самым простым требованием, поскольку не надо ни запоминать пароль, который он может забыть, ни иметь всегда под рукой устройство аутентификации.

Как заявляют многие эксперты, многофакторная аутентификация – намного более эффективный метод защиты информационных систем, поскольку выкрасть пароль и устройство намного сложнее, чем просто выкрасть или подсмотреть секрет.

Однако у этого способа аутентификации есть и свои минусы. Рассмотрим самый распространенный способ с применением мобильного телефона и почты. Во–первых, пользователю необходимо ввести свой номер телефона, что, в случае непредвиденной утечки, может привести к получению на этот номер спама и фишинговых сообщений. Во–вторых, требуется обязательно наличие связи, иначе SMS может попросту не дойти. Также среди минусов можно выделить то, что мобильное устройство зачастую залогинено в почтовом сервисе, что в случае утери поставит под угрозу все аккаунты, привязанные к почтовому ящику.

1.3 Разграничение доступа

После успешного выполнения процедур идентификации и аутентификации необходимо предоставить субъекту – легитимному пользователю доступ к объектам информационной системы в соответствии с той политикой разграничения доступа, которая была принята и установлена для конкретного пользователя.

Обычно полномочия пользователя представлены в виде таблицы, где списком перечислены объекты, к которым предоставляется доступ, а также права, которыми наделен пользователь в отношении того или иного объекта. В качестве этих объектов могут быть как массивы информации, так и аппаратные мощности информационной системы: количество отведенной оперативной памяти, время использования процессора и другие ресурсы.

Выделяют следующие методы разграничения доступа:

- разграничение доступа по спискам;
- использование матрицы установления полномочий;
- по уровням секретности и категориям;
- парольное разграничение доступа.

При разграничении доступа по спискам задаются соответствия:

- каждому пользователю – список ресурсов и прав доступа к ним;
- каждому ресурсу – список пользователей и их прав доступа к данному ресурсу.

Списки позволяют установить права с точностью до пользователя. Здесь нетрудно добавить права или явным образом запретить доступ. Списки используются в большинстве операционных систем и СУБД.

Использование матрицы установления полномочий подразумевает применение матрицы доступа (таблицы полномочий). В указанной матрице строками являются идентификаторы субъектов, имеющих доступ в автоматизированную систему (АС), а столбцами – объекты (информационные ресурсы) АС. Каждый элемент матрицы может содержать имя и размер предоставляемого ресурса, право доступа (чтение, запись и др.), ссылку на другую информационную структуру, уточняющую права доступа, ссылку на программу, управляющую правами доступа и др.

Этот метод предоставляет более унифицированный и удобный подход, т. к. вся информация о полномочиях хранится в виде единой таблицы, а не в виде разнотипных списков. *Недостатками матрицы* являются её громоздкость, и не вполне оптимальное использование ресурсов (большинство клеток – пустые).

Разграничения доступа *по уровням секретности и категориям* состоят в том, что ресурсы автоматизированной системы разделяются в соответствии с уровнями секретности или уровнями категорий.

При разграничении по уровню секретности выделяют несколько уровней, например, *общий доступ, конфиденциально, секретно, совершенно секретно*. Полномочия каждого пользователя задаются в соответствии с максимальным уровнем секретности, к которому он допущен. Пользователь имеет доступ ко всем данным, имеющим уровень (гриф) секретности не выше, чем он имеет.

При разграничении по категориям задается и контролируется ранг категории, соответствующей пользователю. Соответственно, все ресурсы АС декомпозируют по уровню важности, причем определенному уровню соответствует некоторый ранг персонала (типа: *руководитель, администратор, пользователь*).

Парольное разграничение, очевидно, представляет использование методов доступа субъектов к объектам по паролю. При этом используются все методы парольной защиты. Очевидно, что постоянное использование паролей создает неудобства пользователям и временные задержки. Поэтому указанные методы используют в исключительных ситуациях.

На практике обычно сочетают различные методы разграничения доступа. Например, первые три метода усиливают парольной защитой.

Заметим, что руководящие документы могут регламентировать два основных вида (или принципа) разграничения доступа:

- дискретное управление доступом;
- мандатное управление доступом [46].

Дискретное управление доступом представляет собой разграничение доступа между поименованными субъектами и поименованными объектами. Субъект с определенным правом доступа может передать это право любому другому субъекту. Подобный вид управления доступом организуется на базе методов разграничения по спискам или с помощью матрицы.

Мандатное управление доступом регламентирует разграничение доступа субъектов к объектам, основанное на конфиденциальности информации, характеризуемой меткой, содержащейся в объектах, и официальном разрешении (допуске) субъектов обращаться к информации этого уровня конфиденциальности. Иначе, для реализации мандатного управления доступом каждому субъекту и каждому объекту присваивают классификационные метки, отражающие их место в соответствующей иерархии. С помощью этих меток субъектам и объектам должны быть назначены классификационные уровни, являющиеся комбинациями уровня иерархической классификации и иерархических категорий. Данные метки должны служить основой мандатного принципа разграничения доступа. Ясно, что методы разграничения доступа по уровням секретности и категориям являются примерами мандатного управления доступом.

Выводы

Рассмотрены основные виды алгоритмов авторизации, идентификации и аутентификации, выделены их сильные и слабые стороны. Обозначены основные модели разграничения доступа, благодаря которым возможно управлять правами доступа для различных пользователей.

Основные преимущества приложений, построенных по микросервисной архитектуре – хорошая горизонтальная масштабируемость и малая связность отдельных модулей, что влечет за собой удобство при поддержании программного средства в перспективе и предсказуемость поведения программного обеспечения в целом, при возможном отказе одного из модулей.

Таким образом, ввиду перспективности такой архитектуры в будущем, необходимо разработать и реализовать импортозамещающую систему авторизации, для защиты от несанкционированного доступа к данным, обрабатываемым отдельными сервисами.

2 ПРИНЦИПЫ И ТЕХНОЛОГИИ МИКРОСЕРВИСОВ

Микросервисы – это архитектура программного обеспечения, в котором серверные приложения строятся путём разбиения одного монолитного приложения на множество однозадачных сервисов.

2.1 Основные принципы работы микросервисной архитектуры

Несмотря на то, что канонического определения микросервисов не существует, есть определенный набор характеристик (принципов), позволяющих установить стиль разработки программного обеспечения. Рассмотрим эти принципы:

- в качестве программных интерфейсов для взаимодействия с микросервисами, как правило, используют интерфейс RESTful API и асинхронный обмен сообщениями через очереди (RabbitMQ, ZeroMQ). Микросервисы позволяют управлять информационными ресурсами с помощью узкого множества операций: чтения и обновления отдельных его частей. А все взаимодействия между сервисами строятся на стороне клиента без запоминания на сервере. В этой связи необходимо придерживаться паттерна Design for Failure. Этот паттерн крайне необходим, поскольку сеть по своей природе очень ненадежна: пакеты могут не доходить либо из-за плохо настроенной маршрутизации, либо из-за обрыва канала связи между конечными точками. Помимо этого, сервисы могут быть работоспособность сервисов может быть подвержена различным настройкам файрвола, из-за чего тот может перестать пропускать определенные типы пакетов, которые могут показаться ему подозрительным. В этой связи каждый из сервисов должен быть построен так, чтобы он учитывал эти колебания пропускной способности сети, и он должен уметь восстанавливать свою работоспособность в случае, если он был либо изолирован от всех, либо один из сервисов, с которым он ввиду своей архитектуры взаимодействовал, был недоступен;

- в отличие от монолитных приложений, которые развертываются в одном процессе, в микросервисной архитектуре каждая обособленная единица развертывается в изолированном окружении, как указано на рисунке 2.1. Это позволяет дать независимость командам разработки, занимающихся разными сервисами, предоставляя возможность в выборе среды разработки и стека используемых технологий для каждого из микросервисов. Отсюда же вытекает то, что у каждого сервиса должно быть свое изолированное хранилище данных: общей базы данных нет и быть не может. Особенно это удобно для тех случаев, когда разные СУБД могут лучше отрабатывать запросы в частных случаях. Для высоконагруженных систем это может дать ощутимый прирост в производительности.

Так, для хранения большого массива текстовых данных может подойти Oracle, для хранения геотипов хорошо подходит PostgreSQL с его легковесным расширением Postgis, а для хранения пар «ключ–значение», что хорошо подходит для хранения обычных файлов, существуют NoSQL базы Berkeley DB или Apache Cassandra;

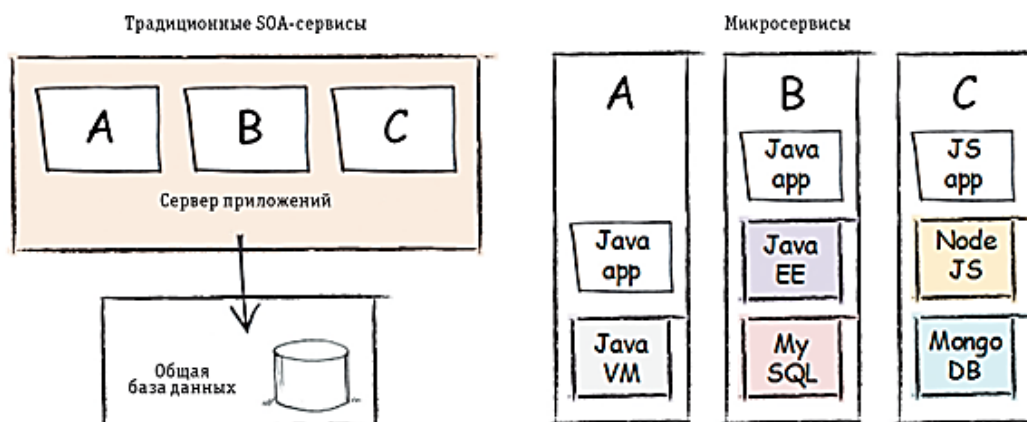


Рисунок 2.1 – Структура микросервисного приложения

– микросервисы реализуют конкретную потребность, они предоставляют конкретный функционал для каждого частного случая. В случае, если для разных задач требуется несколько иное поведение микросервиса, создается несколько микросервисов, реализующих один и тот же функционал и имеющих один и тот же интерфейс взаимодействия, но у каждого из них будет разное поведение. Это отличный инструмент для реализации обработчиков ошибок и исключений, расширения типичного поведения алгоритмов путем добавления дополнительных сценариев поведения, поддержки новых функций.

2.2 Технические аспекты работы с микросервисами

Работая с микросервисами, нужно учитывать, что для их полноценного функционирования необходима хорошо развитая сетевая инфраструктура, в которой эти сервисы содержатся, поскольку именно посредством сетевого взаимодействия происходит общение одного сервиса с другими. Для этого необходимо предусмотреть *отказоустойчивую топологию* для локальной сети между всеми сервисами: иметь дублирующий канал или устройства, которые будут реплицировать основные критические узлы такой системы.

Вторым важным аспектом в работе с подобной архитектурой является максимально простое горизонтальное масштабирование приложения. Должна быть предусмотрена возможность запускать несколько одинаковых сервисов на разных аппаратных мощностях, чтобы они реплицировали друг друга, выполняя тем самым две цели: *увеличение отказоустойчивости* аппаратно–программного комплекса в целом, и *повышение надёжности* из-за возможного распределения нагрузки между разными экземплярами одинаковых микросервисов.

2.3 Постановка задачи разработки

Для создания приложения, удовлетворяющего перечисленным выше особенностям этой архитектуры и поддерживающего ролевое разграничение доступа, необходимо сконструировать архитектуру приложения таким образом, чтобы она позволяла без перекомпиляции исходного кода подменять один сервис на другой, либо же подменить текущий запущенный сервис на другой без остановки в работе отдельного функционала; была иерархически построена так, чтобы была возможность подключения модуля безопасности, ограничивающего доступ неавторизованным пользователям, а также реализовывала распределение нагрузки между разными экземплярами сервисов, представляющих и реализующих одну и ту же бизнес-логику. Подобная архитектура позволит:

- избежать несанкционированного доступа со стороны нелегальных субъектов к конфиденциальным данным;
- избежать убытков, связанных как с техническим обслуживанием оборудования, на котором развернуты сервисы, так и с обновлением самих программных компонентов на более новые версии.
- повысить надежность программного средства, обеспечив достаточную стойкость к высоким нагрузкам благодаря возможности репликации функционала без сложных манипуляций.

С этой целью необходимо выбрать язык программирования, стек технологий и платформу, на которой будет разворачиваться данная архитектура, поскольку для разных целей разные языки программирования могут быть более или менее эффективными. Также у каждой платформы есть свои аспекты в программировании, забывать о которых не стоит, адресация путей и папок в Windows и UNIX системах, например.

Выводы

В этом разделе было определено понятие «микросервисная архитектура». Также рассмотрены основные нюансы принципы работы микросервисной архитектуры, такие как взаимозаменяемость сервисов, изолированность окружения каждого из микросервисов, а также сетевое взаимодействие микросервисов друг с другом. После этого была дана характеристика технических и эксплуатационных особенностей при использовании этой архитектуры и поставлена задача разработки, где были описаны основные характеристики и функции, которые должна будет реализовывать задуманная архитектура.

3 АЛГОРИТМИЧЕСКОЕ КОНСТРУИРОВАНИЕ

3.1 Общая архитектура приложения

Для того чтобы скрыть внутреннюю структуру приложения от пользователя, нужно сделать единый интерфейс взаимодействия пользователя с конечным программным продуктом. Отдельные сервисы могут находиться как локально на разных портах, так и на удаленных машинах. В этой связи следует создать отдельный сервис, который будет «знать» о других сервисах, чтобы разработчики могли воспользоваться этими сведениями для решения поставленных целей.

С помощью информации, собранной на сервисе–регистраторе, можно создать сервис–шлюз, который будет «прокладывать» маршрут к конечной, интересующей пользователя точке, и отдавать запрошенную информацию согласно правам доступа авторизованного пользователя. Для поддержания масштабируемости программного приложения предлагается реализовать возможность создания нескольких экземпляров одного и того же сервиса. Этим будут решены следующие проблемы: программное обеспечение, имеющее в своей структуре такой механизм, будет более отказоустойчивым, поскольку если откажет в работе один из нескольких сервисов, остальные смогут взять его работу на себя.

Непосредственно на сервис–шлюз наиболее целесообразно ставить средства идентификации, аутентификации и авторизации, поскольку именно он будет обладать сведениями о маршрутах до всех конечных локальных узлов.

Для достижения поставленных целей будет целесообразно использовать аутентификацию паролем и подтверждение во время всего, поскольку разрабатываемая архитектура и доступ к сервисам будет иметь лишь ограниченный доверенный круг лиц, а авторизация нужна будет по большей части для того, чтобы не повредить непреднамеренно данные. В конечном счёте, архитектура конечного программного приложения будет выглядеть так, как показано на рисунке 3.1.

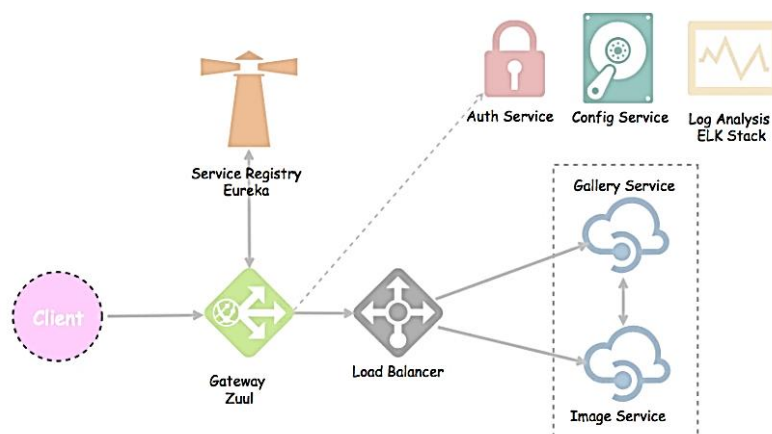


Рисунок 3.1 – Архитектура конечного программного продукта

3.2 Сервис регистрации сервисов

Архитектуре разрабатываемого программного продукта необходим сервис регистрации сервисов. На него будет возложен следующий функционал:

1. Регистрировать сервисы, появившиеся в окружении.
2. Присваивать символьные имена для зарегистрированных сервисов и хранить их в памяти до тех пор, пока они функционируют.
3. Рассылать реестр активных сервисов всем зарегистрированным сервисам.

Предложенный функционал необходим для возможности горизонтального масштабирования и отсутствия сильной связки между различными сервисами для сохранения основных преимуществ микросервисной архитектуры. Данный сервис позволит быть прослойкой между взаимодействующими сервисами, который будет говорить о том, на каком адресе какой сервис развернут. В таком случае придется лишь однажды задать в зависимостях символьное имя сервиса, который будет использоваться в ходе работы другого сервиса, а сервис–регистратор будет отдавать по этому имени адрес нужного объекта этой системы.

Также, это позволит безболезненно регистрировать более чем один экземпляр одного и того же сервиса для повышения отказоустойчивости системы в целом и для распределения нагрузки: предполагается, что с целью перестраховки от перегрузки тех или иных узлов приложения возможно будет подключить несколько их экземпляров для того, чтобы каждый из них работал поочередно, тем самым нагрузка между ними распределялась.

3.3 Сервис–шлюз

Сервис–шлюз берет на себя роль роутера: именно он, согласно реестру, собранному сервисом регистрации, будет переадресовывать запросы по фактическим адресам, которые имеются в окружении.

Этот сервис будет единственной точкой входа в систему, будет принимать все запросы для того чтобы переслать их на конечный необходимый сервис. Именно на этот сервис можно повесить дополнительный функционал по обеспечению безопасности: аутентификация, логирование действий, распределение запросов между различными инстансами одного и того же сервиса с целью распределения нагрузки и фильтрация запросов. В целом структура взаимодействия двух сервисов отображена на рисунке 3.2.

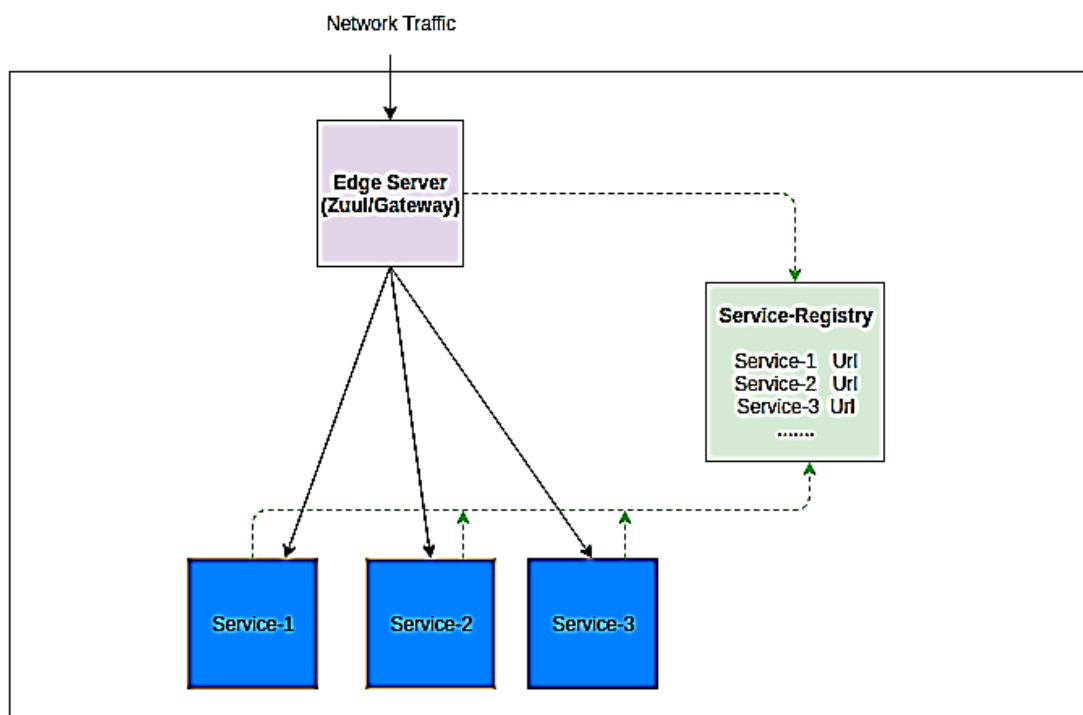


Рисунок 3.2 – Схема взаимодействия сервисов

3.4 Сервис аутентификации

Этот сервис будет прикрепляться к сервису–шлюзу, и проверять валидность (работоспособность) установленного пользователем подключения.

Для данных целей достаточно будет использовать подтверждение при помощи токена. Аутентификация на этом сервисе будет происходить так:

1. Пользователь отправляет запрос на получение токена, подтверждающего его личность, как легитимного пользователя, прошедшего аутентификацию.
2. Сервер проверяет пару логин / пароль и высылает в качестве ответа токен;
3. Пользователь прикрепляет каждый раз токен, благодаря которому сервер предоставляет ему, как легитимному пользователю, доступ к сервису.

Реализовать безопасность можно двумя способами.

Первый способ заключается в том, что токен может проверяться каждый раз в службе аутентификации. Для этого шлюз должен каждый раз вызывать стороннюю службу, прежде чем разрешить запросам перейти к любой другой службе. В этом случае логика безопасности лежит целиком и полностью на сервисе аутентификации. Однако это может вызвать проблемы в том случае, если система будет высоко нагружена: лишние запросы на проверку токена могут пагубно сказаться на *полезной пропускной способности канала* связи между сервисами: служебных запросов на проверку токена, не несущих полезной нагрузки с точки зрения конечного пользователя, станет больше.

Второй способ можно реализовать, сделав проверку токенов на стороне шлюза, в то время как служба аутентификации будет проверять учетные данные и выдавать токены. Но с точки зрения безопасности оба способа хороши: ни один, ни другой не даст доступ к внутренним сервисам без аутентификации.

В качестве генерируемого токена предлагается выбрать стандарт JSON Web Token (JWT). Эти токены, как правило, используются для передачи данных для аутентификации в клиент–сервисных приложениях. Они создаются сервером, подписываются секретным ключом и высылаются клиенту, который в последующем использует его для подтверждения своей подлинности.

Токен состоит из трех частей: заголовок, полезная нагрузка и подпись. Первые два объекта являются объектами JSON определенной структуры, а третий – вычисляется на основе первых двух и выбранного алгоритма. Часто для компактного представления первые две части кодируются в Base64–URL.

Существует два вида токенов: Access–токены и Refresh–токены.

Первый вид токенов предоставляет доступ его владельцу к защищенным ресурсам сервера. Зачастую имеет определенный короткий срок жизни и может в себе нести дополнительную информацию.

Второй вид токенов позволяет клиентам запрашивать новые Access–токены и имеют более длительный срок жизни.

Выводы

В этом разделе рассмотрены различные варианты реализации программного приложения, что даёт возможность выбрать язык программирования и стек технологий, на котором целесообразно реализовать выбранную архитектуру.

Такая архитектура позволит ограничить доступ к приложению со стороны неавторизованных пользователей, что позволит избежать следующих проблем:

1. Утечки информации ввиду возможности реализации угрозы НСД.
2. Повреждения информации, путём выдачи доступа только легитимным пользователям, к тем данным, которые нужны для выполнения задачи.

4 ПРОГРАММНОЕ ПРОЕКТИРОВАНИЕ

4.1 Структура программы

Итак, архитектура приложения должна состоять из следующих компонентов:

1. Сервис регистрации – сервис, функционал которого состоит в регистрации активных сервисов и в ведении реестра подключений;
2. Сервис–шлюз, который, опираясь на реестр сервиса регистрации, должен прокладывать маршрут как до промежуточных узлов, таких, как сервис аутентификации, так и до конечных узлов, представляющих бизнес–логику разрабатываемого программного приложения;
3. Сервис аутентификации, проверяющий идентификационные данные пользователей и генерирующий токен, подтверждающий личность пользователя;
4. Сервисы, представляющие бизнес–логику приложения.

4.2 Выбор языка программирования и технологий

Поставленную задачу по разработке программного обеспечения было принято реализовывать на языке программирования Java. Выделим основные достоинства:

1. *Кроссплатформенность* – за счёт того, что программа на языке Java предварительно компилируется в байт–код, который впоследствии исполняется в JVM на любой платформе, приложение, написанное на Windows, запустится на любой платформе, поддерживающей Java Virtual Machine;
2. *Автоматическое управление памятью* – разработка программы на языке Java упрощается тем, что в JVM работает Garbage Collector, который отслеживает неиспользуемые в памяти объекты и освобождает занимаемое ими место;
3. *Безопасность* – подразумевается, что из–за отсутствия указателей, намного сложнее получить несанкционированный доступ к памяти, а также в Java есть Security Manager, в котором можно указать правила доступа для приложения, а также запускать приложения в песочнице, устраняя их уязвимости.

Итак, язык Java отлично подходит для достижения поставленной цели.

В качестве основного фреймворка, используемого в задуманной архитектуре, будет использоваться Spring. Данный фреймворк позволяет без рутинной настройки разворачивать и запускать на локальной машине сервисы. Это позволит проводить достаточно легко отладку приложения с целью проверки его на корректность работы, а также впоследствии этот сервис позволяет развернуть написанный сервис в отдельном контейнере приложений, например, Tomcat.

Фреймворком, используемым для компиляции и сборки написанных сервисов, выберем Maven. С его помощью возможно достаточно просто указать требуемые для каждого из отдельных модулей зависимости, а впоследствии этот сборщик загрузит их из своего репозитория и будет хранить их локально, что позволит быстро перенести проект на любой компьютер, и продолжить там доработку.

Для простоты разработки и тестирования каждый из сервисов будем реализовывать посредством фреймворка Spring Boot, который берет на себя конфигурирование и запуск контейнера с сервисом посредством среды разработки.

По этой причине, для всех модулей, используемых в проекте, необходимо прописать следующие строки программного кода:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

А также укажем для Maven, каким плагином должна проходить сборка:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

4.3 Реализация сервиса–регистратора

Сервис регистрации отдельных сервисов можно реализовать, используя Spring Cloud. Внутри этого пакета классов есть реализованный класс `SpringEurekaServer`. Для подключения в сервисе этого класса и развертке сервера регистрации Eureka необходимо в файле Maven указать следующую зависимость:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

Далее следует в классе, реализующем загрузку точки входа в контекст приложения Spring, добавить аннотацию к этому классу `@EnableEurekaServer`. После этого в файле `application.properties`, представляющем из себя список строк ключ–значение, укажем основные параметры класс, такие как имя сервиса; порт, на котором он будет работать, а также флаги, указывающие, что сервис сам себя регистрировать и заявлять о себе в реестре не должен.

```
spring.application.name=eureka-server
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

После сборки и запуска сервиса будет доступно по адресу `localhost:8761/eureka` визуальный интерфейс данного сервиса, как на рисунке 4.1, позволяющий отследить в реальном времени зарегистрированные сервисы.

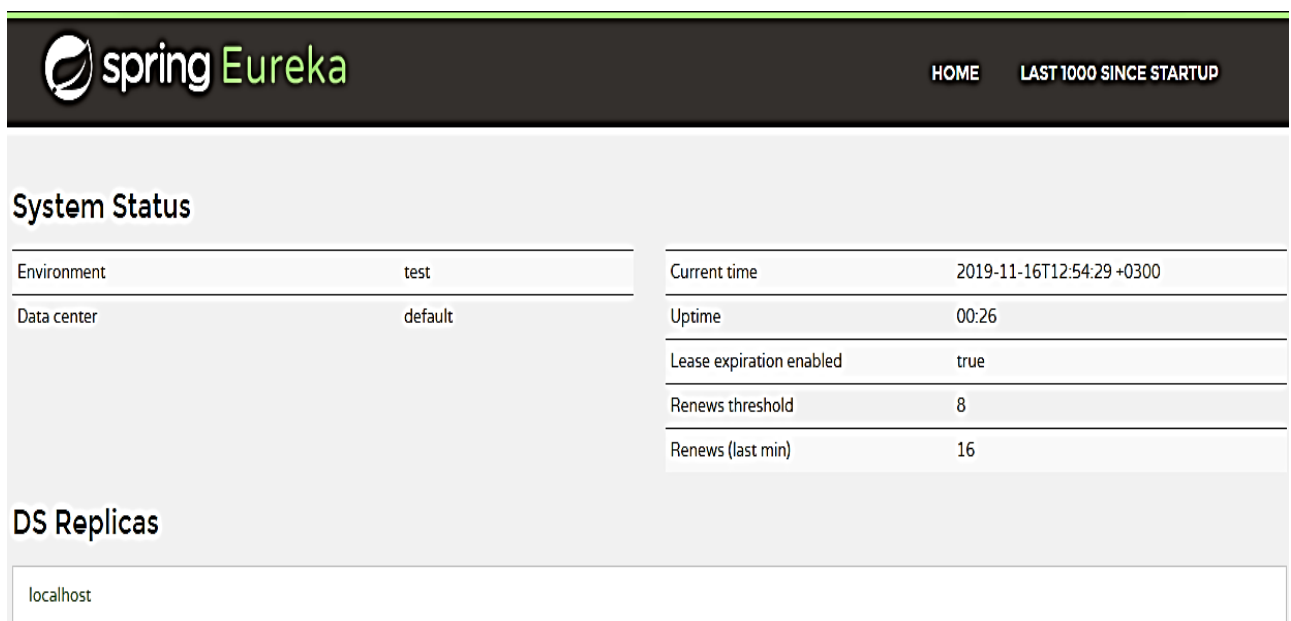


Рисунок 4.1 – Визуальное представление сервиса регистрации

Клиенты сервиса Eureka будут заявлять о себе сами на сервис регистрации. А впоследствии этот сервис будет рассылать конечным узлам хэш этого реестра. Если хэш, хранимый на конечных узлах сервисов, будет отличаться от рассылаемого варианта, то им будет разослана новая версия этого реестра.

4.4 Реализация сервиса–шлюза

В пакете Spring Cloud имеется заготовка для сервиса–шлюза. Zuul имеет возможность принимать запросы, а после перенаправлять их конечным узлам–сервисам. Для запуска подобного сервиса в основном классе приложения понадобится указать несколько аннотаций: `EnableEurekaClient` и `EnableZuulProxy`. Первая аннотация включает клиент Eureka: после запуска *инстанса* сервис будет посылать периодические эхо–запросы на сервис регистрации, тем самым заявляя о себе. Приемная сторона будет запоминать, откуда пришел эхо–запрос и будет добавлять запись об этом в своем реестре. Время жизни такой записи – полторы минуты. Запись в реестре сотрется после того, как в течение 90 секунд не будет поступать сигналов. Это будет значить, что узел стал недоступен. Укажем в конфигурационном файле основные настройки для этого сервиса:

```
zuul.routes.auth-service.sensitive-headers=Cookie, Set-Cookie
zuul.routes.auth-service.strip-prefix=false
zuul.routes.auth-service.path=/auth/**
zuul.routes.auth-service.service-id=AUTH-SERVICE
zuul.routes.gallery-service.path=/gallery/**
zuul.routes.gallery-service.service-id=gallery-service
zuul.ignored-services=*
server.port=8762
spring.application.name=zuul-server
eureka.client.service-url.default-zone=http://localhost:8761/eureka/
```

В этих строках задаётся имя сервиса, порт, на котором его следует развернуть, указывается адрес сервиса–регистратора, перечисляются доступные через этот шлюз сервисы (параметр `zuul.routes.auth-service.service-id`) и путь (параметр `zuul.routes.gallery-service.path`). Делается запрос, по которому можно будет получить ответ от этих сервисов.

Также необходимо создать логику, отвечающую за ограничение доступа к сервису неавторизованных пользователей и отвечающую за разграничение прав доступа. Сделаем это после создания сервиса авторизации.

4.5 Реализация сервиса авторизации

Дадим имя сервису авторизации `auth-name` и укажем ему адрес сервиса-регистратора, а также скажем, на каком порту он должен работать. Помимо основного класса, которое является точкой старта приложения, нам потребуется еще три класса, которые будут расширять библиотечные интерфейсы и реализовывать методы, отвечающие непосредственно за безопасность.

Класс `UserDetailsServiceImpl` – реализация интерфейса `UserDetailsService`, который служит для проверки, существует ли пользователь, который был введен при запросе авторизации, или нет. Если пользователя не существует, то тогда будет выброшено исключение `UsernameNotFoundException`.

Для демонстрационных целей несколько пользователей жёстко прописаны в коде. В реально работающих приложениях источником пользователей может быть запрос к базам данных или какой-то список, хранящийся в файловой системе.

Класс `SecurityCredentialsConfig` значительно расширяет библиотечный класс `WebSecurityConfigurerAdapter` и представляет собой конструирование класса с параметрами, необходимыми под текущую задачу. Внутри класса мы добавляем фильтры, задаем маски запросов и инициализируем объекты, отвечающие за проверку корректности данных и за высчитывание хэша от пароля.

Класс `JwtUsernameAndPasswordAuthenticationFilter` – один из тех фильтров, что были инициализированы в классе `SecurityCredentialsConfig`. В теле этого фильтра мы переопределяем стандартные параметры, такие как прослушиваемый путь (меняем `“/login”` на `“/auth”`). Также в этом фильтре описан метод генерации токена: в его генерации участвует такая информация, как имя авторизованного пользователя, присваиваемые ему права и хэш пароля пользователя.

После успешной аутентификации пользователю в заголовке ответа возвращается токен, который пользователь должен предъявлять при каждом запросе к любому сервису, прикрепленному этому шлюзу.

4.6 Тестирование архитектуры и примеры работы

Для работы с сервисами, интегрированными в эту архитектуру, необходимо посылать запросы на шлюз. Шлюз будет проверять авторизационные данные и в зависимости от этого предоставлять доступ или отказывать в доступе.

Любые сервисы и любое их количество могут быть расположены в локальной сети, тогда как доступ в сети общего пользования будет иметь только шлюз. Таким образом, извне невозможно будет получить доступ напрямую к отдельным сервисам, они будут доступны только через шлюз.

Запустим сервисы, представляющие архитектуру, а также пару сервисов, реализующих бизнес–логику приложения. На сервисе Eureka мы увидим, что все они зарегистрированы и готовы к работе, как показано на рисунке 4.2.

Для отладки будем использовать сервис Postman, который позволяет конструировать http запросы с удобным графическим интерфейсом: можно заполнить заголовок, указать тип запроса, а также передать необходимые параметры. После получения ответа есть возможность разобрать его и посмотреть его структуру, чтобы понять, верно ли сконфигурировано приложение. Сделаем запрос к сервису image-service или gallery-service. Как мы можем заметить на рисунке 4.3, отображается сообщение, что в доступе было отказано (ошибка 401).

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
AUTH-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-1UL7PV8:auth-service:9100
GALLERY-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-1UL7PV8:gallery-service:8300
IMAGE-SERVICE	n/a (1)	(1)	UP (1) - DESKTOP-1UL7PV8:image-service:8200
ZUUL-SERVER	n/a (1)	(1)	UP (1) - DESKTOP-1UL7PV8:zuul-server:8762

General Info	
Name	Value
total-avail-memory	419mb
environment	test
num-of-cpus	4
current-memory-usage	156mb (37%)
server-uptime	00:26
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	

Рисунок 4.2 – Список зарегистрированных сервисов

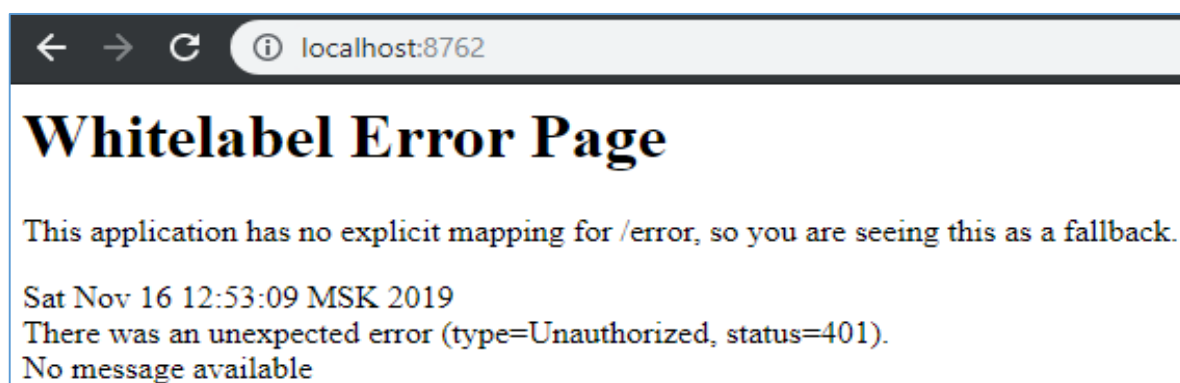


Рисунок 4.3 – Ошибка об отказе в доступе

Отправим http-запрос на адрес с данными для аутентификации под обычным пользователем, чтобы получить токен. Как видно на рисунке 4.4, в заголовке ответа пришел токен с другими полями со служебной информацией. Этот токен представляет собой зашифрованные данные о легитимном пользователе, приведенные к кодировке base64.

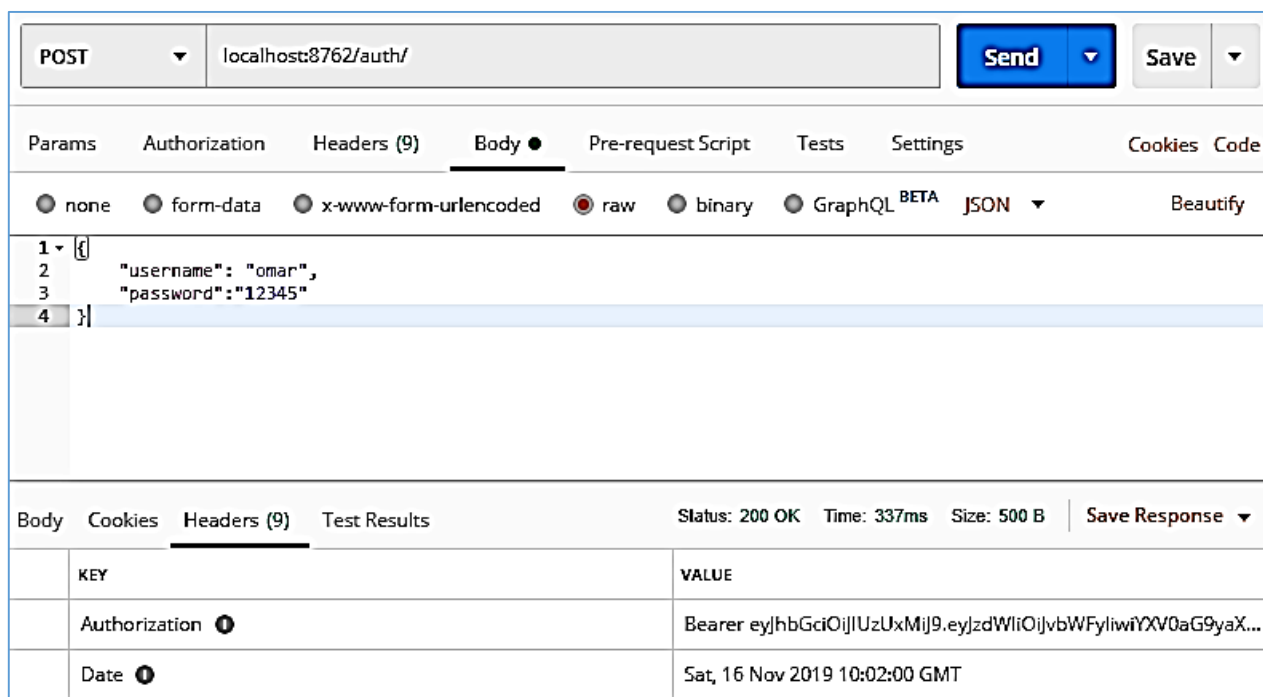


Рисунок 4.4 – Ответ от сервиса авторизации с токеном

Теперь сделаем те же самые запросы, что и ранее, только прикрепим к ним в заголовок токен. Контроллер, работающий на серверной части приложения, извлёк из заголовка токен, расшифровал его и извлек информацию о том, кому принадлежит эта сессия. После этого сравнил роль, которая выдана этому пользователю, с тем уровнем доступа, который был присвоен субъекту. Результат можно увидеть на рисунке 4.5.

Попробуем сделать запрос к административной части. На рисунке 4.6 можем видеть, что пользователю не хватает прав, поэтому мы видим ошибку.

Сменим логин под администратором. Повторно отправим на сервис авторизации данные для входа в аккаунт администратора. Полученный в заголовке токен прикрепим к тому же запросу и сделаем запрос снова, чтобы проверить, что администратор действительно получит доступ к этому разделу. Результат отображен на рисунке 4.7.

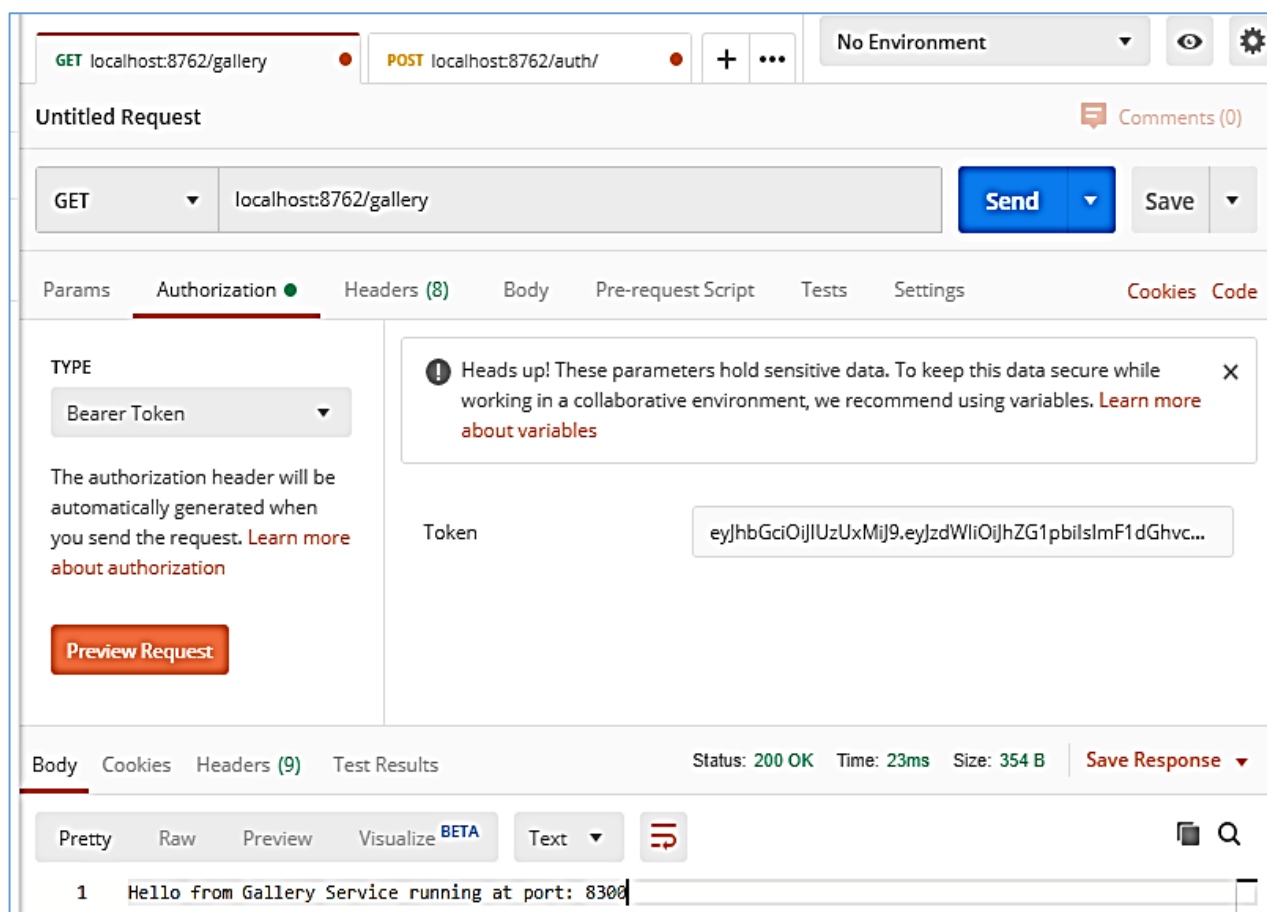


Рисунок 4.5 – Результат ответа на запрос с прикрепленным токеном

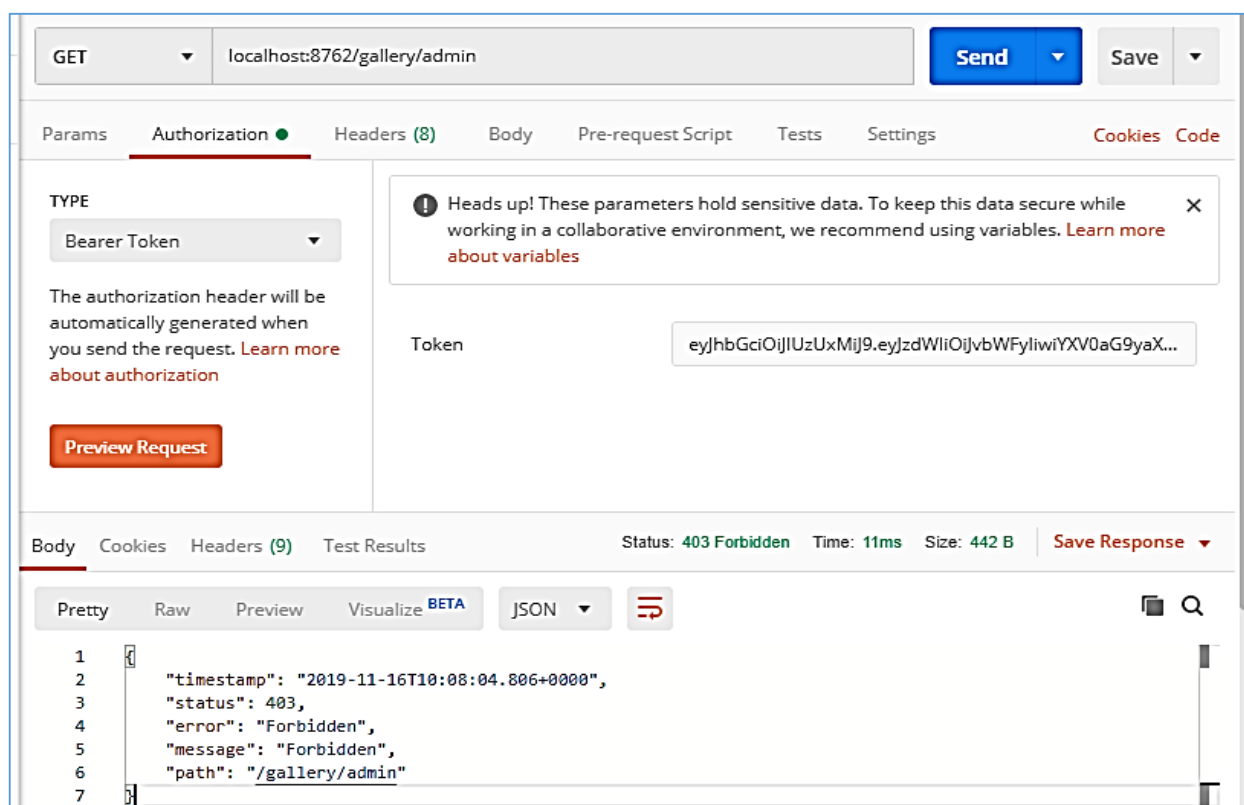


Рисунок 4.6 – Ответ от сервиса простому пользователю

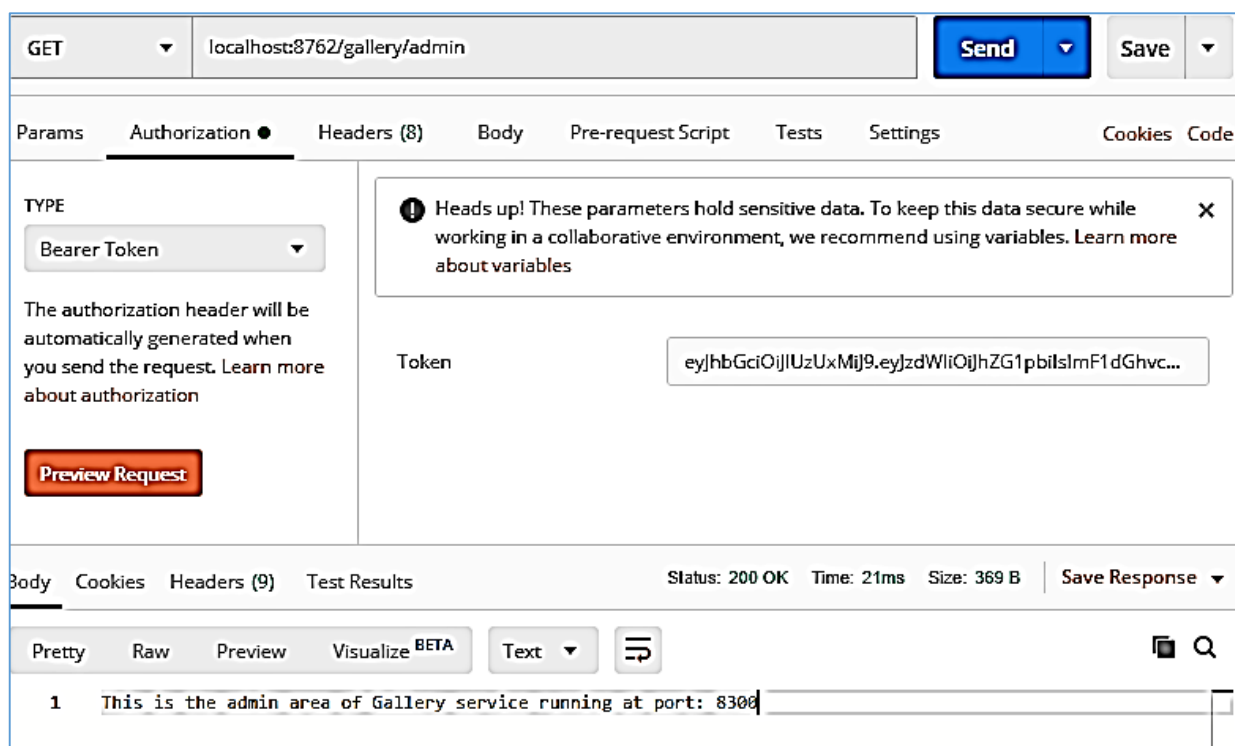


Рисунок 4.7 – Ответ от сервиса администратору

Выводы

В результате программной реализации задуманной архитектуры, получен набор изолированных сервисов, доступ к которым ограничен неавторизованным пользователями. Путём присвоения каждому из пользователей ролей и выдаче меток страницам, отвечающих за необходимые полномочия пользователей, была решена задача по ролевому разграничению доступа пользователей к сервисам.

Таким образом, разработано импортозамещающее программное обеспечение с оригинальной микросервисной архитектурой, обеспечивающее разграничение доступа к информационной системе для защиты её от несанкционированного доступа и несанкционированной модификации со стороны злоумышленников.

РЕЗУЛЬТАТЫ 7 ГЛАВЫ

Работа выполнена с целью разработки архитектуры импортозамещающего программного средства, построенного по микросервисной архитектуре, с целью разграничения доступа к информационной системе для защиты её от НСД.

Цель достигнута, и поставленные задачи выполнены:

1. Изучена классификация алгоритмов авторизации.
2. Выявлены достоинства и недостатки данных алгоритмов.
3. Проведен анализ существующих алгоритмов авторизации, выбран лучший метод реализации архитектуры, обеспечивающий защиту данных от НСД.
4. Разработана архитектура приложения, отвечающая поставленным задачам.
5. Выбран язык программирования и соответствующий стек технологий для реализации данной архитектуры, сама архитектура была реализована и испытана серией последовательных запросов к приложению.

Данная архитектура позволяет надёжно защитить от несанкционированного доступа программное средство, распределенное на нескольких вычислительных машинах, а также даёт возможность, с помощью дискреционной или мандатной модели безопасности компьютерных систем, выдавать или ограничивать права доступа пользователя на отдельные информационные объекты.

Реализованная архитектура устойчива к возрастающим, со временем, на эту систему нагрузкам. Горизонтальное масштабирование осуществляется простым добавлением к общей структуре приложения дубликата сервиса, на который приходится повышенная нагрузка. При автоматической регистрации сервиса, система разграничения нагрузки позволит снизить её в автоматическом режиме.

К минусам данной архитектуры относится тот факт, что в случае отказа сервиса авторизации, новые пользователи системы не смогут получить прав к информационным ресурсам, а также повышенные требования к сетевым ресурсам для взаимодействия сервисов между собой. Из-за сетевой прослойки, идентичное приложение, построенное на монолитной архитектуре, из-за отсутствия сетевого взаимодействия между компонентами, могло бы работать быстрее.

К плюсам можно отнести то, что проверка токена работает на стороне шлюза, а не на стороне сервиса авторизации, благодаря чему в случае отказа сервиса авторизации уже авторизованные пользователи смогут продолжить работу с программным средством. Также, как уже описывалось выше, данное программное средство обладает высокой устойчивостью к нагрузкам.

В дальнейшем, на разработанной архитектуре планируется реализовать бизнес-логику для внутренней структуры предприятия, которая позволяла бы вести внутренний справочник по различным поставляемым изделиям.

ГЛАВА 8

**РАЗРАБОТКА ИМПОРТОЗАМЕЩАЮЩЕЙ СИСТЕМЫ
БЕЗОПАСНОЙ АУТЕНТИФИКАЦИИ ПОЛЬЗОВАТЕЛЕЙ
ЭЛЕКТРОННЫХ ПЛАТЕЖНЫХ СИСТЕМ SECRET TOKEN**

ОБЩИЕ ПОЛОЖЕНИЯ

В последние десятилетия достижения науки и новейших технологий, как никогда прежде, начали определять динамику экономического роста России, уровень благосостояния населения, конкурентоспособность государства в мировом сообществе, степень обеспечения его национальной безопасности и равноправной интеграции в мировые экономические процессы.

Вместе с тем, развитие процесса информатизации мирового сообщества породило целый комплекс негативных последствий. Действительно, высокая сложность и одновременно уязвимость всех электронных систем, на которых базируются региональные, национальное и мировое информационные пространства, а также фундаментальная зависимость от их стабильного функционирования инфраструктур государств, приводят к возникновению принципиально новых угроз. Эти угрозы связаны, прежде всего, с потенциальной возможностью использования информационно–телекоммуникационных систем в целях, несовместимых с задачами поддержания международной стабильности и безопасности, уважения прав и свобод человека. Поэтому в настоящее время все более актуальным становится проблема аутентификации пользователей, имеющих доступ к общественным и личным информационным ресурсам.

Самым надежным методом многофакторной аутентификации в настоящее время является применение персонального аппаратного устройства – токена. Такие устройства могут стать надежной альтернативой как парольным, так и биометрическим методам, в виду простоты их внедрения и эксплуатации.

Объектом исследования данной работы являются платежные системы (ПС).

Предмет исследования – угрозы безопасности для пользователя, возникающие в электронных платежных системах в процессе аутентификации.

Целью данной работы является изучение возможностей применения токенов для обеспечения безопасности пользователя на этапе аутентификации при совершении действий в электронных платежных системах.

Задачи исследования:

- анализ существующих электронных платежных систем и выявление проблем безопасности аутентификации в платежных системах;
- изучение существующих способов аутентификации пользователей и их возможностей по защите пользователей;
- исследование возможностей применения Secret Token для усиления защиты пользователей электронных платежных систем;
- разработка алгоритма работы Secret Token на базе USB устройства.

1 ОПИСАНИЕ ЭЛЕКТРОННЫХ ПЛАТЕЖНЫХ СИСТЕМ

1.1 Требования к системе защиты электронных платежных систем

Для обеспечения безопасности электронных платежных систем (ЭПС) нужны как организационные меры, так и аппаратно–программные технические средства защиты информации. На рисунке 1.1 представлены архитектура современной ЭПС, и вероятные угрозы безопасности её функционирования.

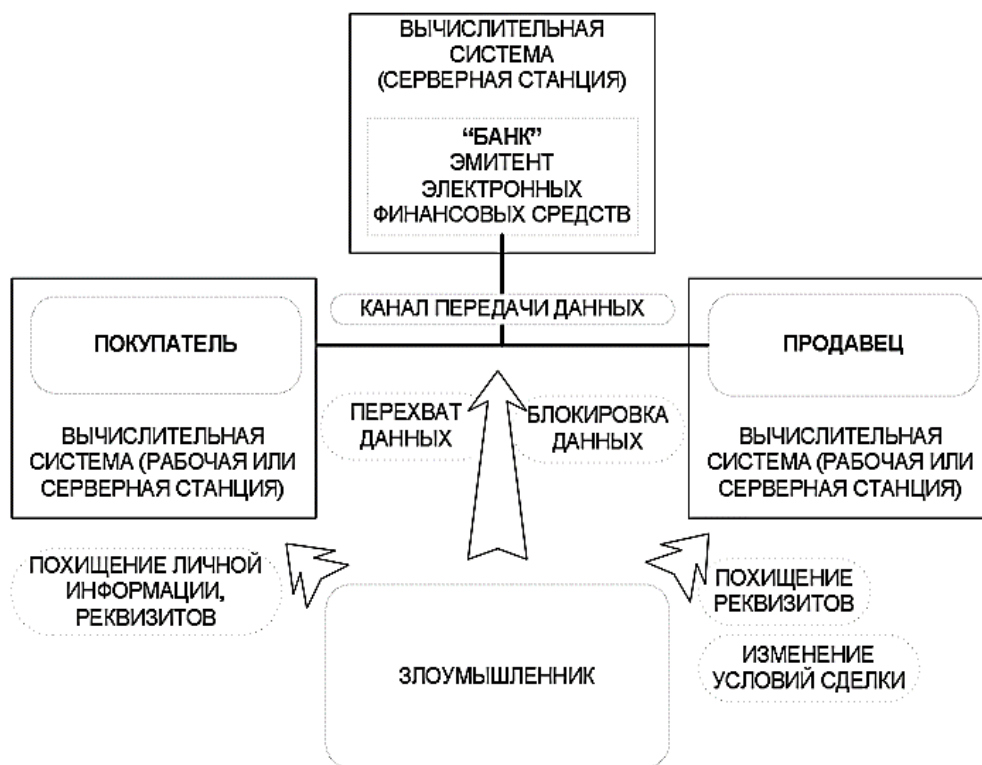


Рисунок 1.1 – Архитектура электронных платежных систем и возможные угрозы

На схеме показано, что существует вероятность не только единичного, но и комплексного воздействия злоумышленников на канал передачи данных. Атака возможна на всех участников электронного платежа. Пользователь электронной платежной системы может подвергнуться следующим угрозам [45]:

- несанкционированному просмотру и копированию его персональных данных;
- несанкционированному копированию персональной финансовой информации;
- несанкционированному копированию электронных чеков.

Для повышения безопасности чаще всего используют несколько факторов аутентификации одновременно. В то же время, не всякая комбинация нескольких методов действительно является многофакторной аутентификацией. Например, биометрическую идентификацию можно рассматривать лишь как дополнительный метод подтверждения личности пользователя.

Можно выделить следующие требования к системе защиты платежных систем:

- в качестве мер защиты от несанкционированного проникновения в банковские сети необходимы антивирусная, межсетевая и криптографическая защита;
- требуется многофакторная аутентификация участников сделки;
- должен применяться автоматизированный процесс оценки степени риска каждого из платежей, проводимых через электронную платёжную систему;
- необходима надежная защита персональной конфиденциальной информации, в том числе паролей и иных идентификационных данных;
- требуется такая система обмена реквизитами, которая бы обеспечивала идентификацию участников, но не раскрывала дополнительной информации о них;
- нужна прозрачность совершенных транзакций и сохранение их историй как для пользователя, так и для владельца платежной системы или продавца.

1.2 Уязвимости на этапе входа в электронный кошелек

Аутентификация – необходимая процедура для доступа к платежной системе. Сервис, которым пользуется клиент, берет на себя генерирование ключей и одноразовых паролей [46]. Именно на данном этапе злоумышленник может начать получать необходимые данные для взлома электронного кошелька.

Получить логин пользователя он сможет без проблем – осуществить перехват идентификатора сессии для мошенника несложно, что даст ему нужные сведения.

Когда пользователь заходит в свой электронный кошелек, ему присваивается номер (этот номер сохраняется в Cookie браузера и передается сайту при открытии каждой страницы) – для того, чтобы сохранить сведения о действиях в данной сессии. На сервере будет храниться файл, который отвечает за идентификацию пользователя. Если мошенник сумеет получить номер текущей сессии, то сможет выдать себя за клиента банка и получить множество данных.

1.3 Атаки на этапе входа в электронный кошелек

Угрозы могут поджидать пользователя уже на этапе входа в электронный кошелек – в момент аутентификации. Ввиду того, что для проверки подлинности пользователя используются криптографические алгоритмы, он может даже не догадываться о существующей угрозе. Проверить подлинность протокола можно только со стороны сервера. Пользователь даже не знает о преследующей его угрозе.

Как правило, атаки на протоколы аутентификации не связаны со взломом криптографических алгоритмов как таковых. Злоумышленник, в первую очередь, пользуется уязвимостями, которые присущи самому протоколу.

Всевозможные виды атак на протоколы аутентификации пополняются постоянно, и перечислить их полные списки даже не представляется возможным. Тем не менее, эти атаки можно разделить на несколько категорий.

В рамках данной работы рассмотрены типичные методы кибератак на алгоритмы аутентификации. Среди них можно выделить [47]:

- атаку с повторной передачей сообщения;
- атаку с помощью параллельного сеанса;
- атаку с помощью отражения сообщения;
- атаку с помощью чередования сообщений;
- атаку на основе неправильной интерпретации и т. д.

Перехват информации возможен, когда злоумышленник будет пытаться стать посредником между клиентом и электронным кошельком. Атака выглядит так:

- находясь на канале связи между пользователем и электронным кошельком, хакер перехватывает запросы пользователя и отправляет их на сайт от своего имени;
- мошенник транслирует все действия пользователя и сайт определяет его как своего клиента, так как он вводит необходимый пароль, который может знать только пользователь. В данном случае будет невозможно определить, что клиент не тот, за кого себя выдает, так как IP источника идентичен на всех этапах протокола.
- для завершения атаки хакеру достаточно выдать пользователю какую-нибудь ошибку HTTP и прервать связь. Клиент воспримет это как проблему с сервером.

Чтобы обеспечить защиту от подобной атаки, необходимо изначально защитить себя от угрозы перехвата соединения. О этом речь пойдет далее.

Выводы

Подводя итоги состоявшемуся анализу платежных систем, угрозам их безопасности и требований к системам защиты от технического мошенничества, можно утверждать, что наиболее перспективным направлением развития систем защиты является создание новых средств сбора дополнительной информации об участниках сделки. Современные сервисы электронных платежных систем надёжно защищены криптографическими алгоритмами, но обычный пользователь может и не знать о совершающемся акте мошенничества и перехвата информации. Для наибольшего уровня защиты, серверу требуется собирать дополнительные сведения о пользователе, которые бы автоматизировано собирались не только при входе в личный кабинет клиента, а постоянно во время работы пользователя на сервере.

2 АУТЕНТИФИКАЦИЯ И ВОЗМОЖНОСТИ ЗАЩИТЫ ПОЛЬЗОВАТЕЛЕЙ

В современном мире невозможно представить себе не только офис какой-либо компании, но и дом обычного человека без компьютеров. В тоже время, многие данные, которые хранятся на ПК, являются конфиденциальными, составляют коммерческую или персональную тайну и нуждаются в защите.

Для того, чтобы это было осуществимо, необходимо выполнение двух условий: знать, кто получил доступ к информации, и быть уверенным, что пользователь действительно является тем, за кого он себя выдает.

Для достижения указанных выше условий существуют процедуры проверки подлинности личности пользователя – его идентификации и аутентификации.

Идентификация – это процесс распознавания личности субъекта по его идентификатору (т.е. некой уникальной информации – числу, строке символов).

Аутентификация – это процедура проверки соответствия личности субъекта и того, за кого он пытается себя выдать, с помощью некой уникальной информации, известной только ему, т.е., другими словами – проверка подлинности его личности.

Идентификация в данном случае заключается в процессе поиска уникального идентификатора в некой базе данных. Чтобы пользователь мог доказать свою подлинность, проверятся соответствие между субъектом и его идентификатором.

Фактор аутентификации — это некая уникальная информация, которая предоставляется субъектом системе при его аутентификации.

Всего различают четыре фактора аутентификации:

- субъект имеет нечто (дискету, токен и т.д.);
- субъект знает нечто (пароль, логин и т.д.);
- субъект обладает уникальной биологической особенностью (например, отпечатком пальца, сетчаткой глаза, формой ушной раковины, структурой ДНК и др.);
- субъект находится в определенном месте (IP–адрес, данные от радиометки и т.д.).

Самый распространенный и часто используемый способ аутентификации пользователей — *парольный*. Однако это слабый уровень защиты, который не соответствует современным требованиям к информационной безопасности. Его надёжность заключается в сложности ключевых слов пользователя и условий хранения им этих данных. По уровню защиты данных аутентификацию можно разделить на три категории: статическую, устойчивую и постоянную.

Статическая аутентификация защищает от несанкционированного доступа только в тех системах, в которых злоумышленник не может во время сеанса работы прочесть аутентификационную информацию. Но в тоже время, узнав пароль, мошенник получает полный доступ ко всей конфиденциальной информации.

При устойчивой аутентификации применяются динамические данные, которые меняются на каждом из сеансов работы пользователя. Такими данными могут быть одноразовые пароли. Но такой тип аутентификации изначально обладает уязвимостью от активных атак, когда злоумышленник в течение сеанса перехватывает данные, модифицирует их и вставляет в поток передаваемой информации. Этот вид аутентификации сложнее, нежели в предыдущем случае, но все равно не отвечает всем современным требованиям безопасности в сети.

Постоянная аутентификация обеспечивает идентификацию передаваемых данных в течение всего сеанса, что и предохраняет их от несанкционированного изменения. Это самый надежный из существующих способов аутентификации.

В рамках данного исследования проведен обзор самых распространенных протоколов постоянной аутентификации. Из них наиболее известны [48]:

- EAP (RFC 3748 – Proposed Standard);
- IPSec (RFC 4302 – Proposed Standard, RFC 2401 – Proposed Standard);
- SSH (RFC 4251 – Proposed Standard, RFC 4252 – Proposed Standard);
- TLS v 1.2 (RFC 5246 – Proposed Standard).

2.1 Протокол аутентификации EAP

EAP (от англ. Extensible Authentication Protocol, расширяемый протокол аутентификации) — это фреймворк аутентификации, который часто используется в беспроводных сетях и соединениях точка–точка [49].

EAP используется для выбора метода аутентификации, передачи ключей и обработки этих ключей подключаемыми модулями.

EAP – это довольно популярный формат, он используется в IEEE 802.11 (WiFi). Около ста методов EAP из IEEE 802.1X были приняты в качестве официальных механизмов аутентификации в стандартах WPA и WPA2.

В процессе аутентификации можно выделить трёх основных участников процесса: аутентификатора, узел (клиент) и сервер аутентификации.

Аутентификатор — это участник процесса, который требует провести аутентификацию (WiFi точка доступа, сетевой коммутатор и т. д.).

Узел или клиент — это участник процесса, который будет аутентифицирован (компьютер, ноутбук, планшет, смартфон, телефон и т. д.).

Сервер аутентификации — это участник процесса, способный, по некоторым данным от узла или клиента, аутентифицировать его.

В некоторых случаях сервер аутентификации и аутентификатор могут совпадать и быть одним и тем же устройством.

Взлом данного протокола даст злоумышленнику возможность удаленного доступа к ПК, а, следовательно, и к конфиденциальной информации, которая содержится на нём. Такое соединение возможно взломать в непосредственной близости к аутентификатору. Здесь злоумышленнику необходимо перехватить ключи, что для него сделать несложно из-за открытости каналов передачи.

2.2 Протокол аутентификации IP Security

IP Security — это комплект протоколов, касающихся вопросов шифрования, аутентификации и обеспечения защиты при транспортировке IP-пакетов. Протокол основан на согласовании сторонами применяемых криптографических алгоритмов, алгоритмов управления ключевой информацией и их параметров. Такое согласование выполняется предварительно, перед началом использования протокола. Для установки защищенного соединения, оба участника процесса должны согласовать алгоритмы аутентификации и передаваемые ключи.

Протокол IPSec получил неоднозначную оценку. По мнению специалистов, присутствует чрезмерная сложность и избыточность протокола [50].

Так, Niels Ferguson и Bruce Schneier в своей работе "A Cryptographic Evaluation of IPsec" отмечают, что они обнаружили серьёзные проблемы безопасности практически во всех главных компонентах IPsec. Эти авторы также отмечают, что набор протоколов требует серьёзной доработки для того, чтобы он обеспечивал хороший уровень безопасности. В работе приведено описание ряда атак, использующих как слабости общей схемы обработки данных, так и слабости криптографических алгоритмов. То есть протокол несовершенен ввиду своей распространённости, а значит распространены и методы его взлома.

2.3 Протокол аутентификации SSH

Secure Shell (SSH) – это криптографический сетевой протокол для безопасной работы сетевых служб в незащищенной сети. Типичные приложения включают удаленную командную строку, вход в систему и удаленное выполнение команд, но любая сетевая служба может быть защищена с помощью SSH.

SSH предоставляет безопасный канал по незащищенной сети в архитектуре клиент–сервер, соединяющий клиентское приложение SSH с сервером SSH.

SSH использует криптографию с открытым ключом для аутентификации удаленного компьютера и позволяет ему аутентифицировать пользователя, если это необходимо. Существует несколько способов использования SSH; один из них – использовать автоматически генерируемые пары открытых и закрытых ключей для простого шифрования сетевого подключения, а затем использовать аутентификацию пароля для входа в систему [51].

Другой способ заключается в использовании вручную сгенерированной пары открытого и закрытого ключей для выполнения проверки подлинности, что позволяет пользователям или программам входить в систему без указания пароля. В этом случае любой пользователь может создать соответствующую пару различных ключей (public и private). Открытый ключ размещается на всех компьютерах, которые должны разрешить доступ к владельцу соответствующего закрытого ключа (владелец сохраняет секретный ключ). Хотя аутентификация основана на закрытом ключе, сам ключ никогда не передается через сеть во время аутентификации. SSH проверяет, является ли тот же человек, предлагающий открытый ключ, также владельцем соответствующего закрытого ключа.

SSH также поддерживает аутентификацию на основе паролей, которая шифруется автоматически созданными ключами. В этом случае злоумышленник может имитировать законную сторону сервера, запросить пароль и получить его (атака «человек в середине»). Однако это возможно только в том случае, если обе стороны никогда не аутентифицировались раньше, поскольку SSH запоминает ключ, который ранее использовал серверная сторона. Клиент SSH выдает предупреждение перед принятием ключа нового, ранее неизвестного сервера. Аутентификация паролем может быть отключена.

Принятие открытого ключа злоумышленника без проверки позволит авторизовать несанкционированного хакера как действительного пользователя.

2.4 Протокол аутентификации TLS

TLS (англ. Transport Layer Security— протокол защиты транспортного уровня) – это криптографический протокол, обеспечивающий защищённую передачу данных между узлами в сети Интернет. TLS используют асимметричное шифрование для аутентификации, симметричное шифрование для конфиденциальности и коды аутентичности сообщений для сохранения целостности сообщений [52].

Этот криптографический протокол аутентификации широко используется в приложениях, работающих с сетью Интернет: в Web-браузерах, электронной почте, в сервисах обмена мгновенными сообщениями, и т.д.

TLS даёт возможность клиент–серверным приложениям осуществлять связь в сети таким образом, что производить прослушивание пакетов и осуществлять несанкционированный доступ будет невозможно.

Так как большинство протоколов связи может быть использовано как с TLS, так и без TLS, то при установке соединения необходимо явно указать серверу, хочет ли клиент устанавливать протокол TLS.

Сейчас в большинстве ОС предусмотрено использование TLS по умолчанию. Но важным моментом является то, что сервер также должен поддерживать TLS.

Для обеспечения безопасности данных пользователя в протоколе существуют основные шаги процедуры создания защищённого сеанса связи:

- запрос клиентом у сервера защищенного соединения;
- отбор сервером наиболее надёжных и проверенных алгоритмов;
- отправка клиенту цифрового сертификата;
- проверка клиентом сертификатов;
- получение общего секретного сеансового ключа клиентом и сервером.

Однако, при возникновении проблем на некоторых из шагов, указанных выше, само подтверждение связи может завершиться с ошибкой, и, как итог, безопасное соединение установлено не будет [53].

Первые уязвимости протокола были обнаружены еще в 2009 г.

Злоумышленнику возможно взломать https-соединение, а значит, добавлять собственные запросы в сообщения, отправленные от клиента к серверу.

Если пользователь не обращает внимания на индикацию браузера о том, что сессия является безопасной (обычно значок замка), уязвимость может быть использована для атаки типа человек посередине.

2.5 Хеш-функция как алгоритм защиты в протоколе аутентификации

Хеширование (англ. hashing) — это преобразование входного массива данных произвольной длины в выходную битовую строку фиксированной длины.

Существует много алгоритмов хеширования с различными характеристиками (разрядность, вычислительная сложность, криптостойкость и др.). Простейшим примером хеш-функции можно назвать вычисление контрольной суммы.

Среди множества существующих хеш-функций принято выделять криптографически стойкие функции. Криптостойкая хеш-функция прежде всего должна обладать стойкостью к коллизиям двух видов [54].

Первый тип заключается в том, что для заданного сообщения $\sim M$ должно быть практически *невозможно подобрать* другое сообщение $\sim M'$, имеющее такой же хеш. Это свойство также называется необратимостью хеш-функции.

Второй тип заключается в том, что должно быть практически невозможно подобрать пару сообщений (M, M'), имеющих одинаковый хеш.

Для криптографических хеш-функций важно, чтобы при малейшем изменении аргумента значение функции сильно изменялось. В частности, значение хеша не должно давать утечки информации даже об отдельных битах аргумента.

Для сверки используется хеш-значение проверяемой информации. Различают два основных направления этого применения.

Например, контрольная сумма может быть передана по каналу связи вместе с основным текстом. На приёмном конце контрольная сумма может быть рассчитана заново и её можно сравнить с переданным значением. Если будет обнаружено расхождение, то это означает, что при передаче возникли искажения. В таком случае требуется запросить повтор на передачу контрольной суммы.

В большинстве случаев парольные фразы не хранятся на целевых объектах, а сохраняются лишь их хеш–значения. Хранить парольные фразы нецелесообразно, так как в случае несанкционированного доступа к файлу с фразами злоумышленник узнает все парольные фразы, и сразу сможет ими воспользоваться, а при хранении хеш–значений он узнает лишь хеш–значения, которые *необратимы* в исходные данные, в данном случае в парольную фразу. В ходе процедуры аутентификации вычисляется хеш–значение введённой парольной фразы, и сравнивается с сохранённым значением. Примером, в данном случае, могут служить операционные системы GNU / Linux и Microsoft Windows XP. В них хранятся лишь хеш–значения парольных фраз из учётных записей пользователей.

Например, при записи текстовых полей в базе данных может рассчитываться их хеш–код, и данные могут помещаться в раздел, соответствующий этому хеш–коду. Тогда при поиске данных надо будет сначала вычислить хеш–код текста и сразу станет известно, в каком разделе их надо искать, то есть, искать надо будет не по всей базе, а только по одному её разделу (это сильно ускоряет поиск).

Бытовым аналогом хеширования в данном случае может служить помещение слов в словарь по алфавиту. Первая буква слова является его хеш–кодом, и при поиске мы просматриваем не весь словарь, а только нужную букву.

2.6 Алгоритм хеширования SHA–256

SHA–256 (от англ. Secure Hashing Algorithm) — это популярный криптографический алгоритм хеширования, разработанный National Security Agency — Агентством национальной безопасности США. Задача SHA—256 состоит в том, чтобы сделать из случайного набора данных определённые значения с фиксированной длиной, которое послужит идентификатором этих данных.

Полученное значение сравнивается с дубликатами исходных данных, извлечь которые невозможно. Основная сфера применения алгоритма — использование в различных приложениях или сервисах, связанных с защитой информации, где функция и получила широкое распространение. Также она используется как технология для майнинга криптовалют [55].

Хэш–сумма не является технологией шифрования данных в классическом понимании, этим обусловлена невозможность расшифровки данных в обратную сторону. Это односторонняя шифровка для любого количества данных. Все алгоритмы SHA базируются на методе Меркла – Дамгарда: данные разделяют на равномерные группы, каждая из которых проходит через одностороннюю функцию сжатия. В результате этого длина данных уменьшается.

У такого метода есть два значительных достоинства:

- быстрая скорость шифрования и невозможная расшифровка без ключей;
- минимальный риск появления коллизий (одинаковых образов).

Для хэш–функции, в которой L это количество бит в хэш–значении, нахождение сообщения, соответствующего этому хэш–значению может быть выполнено с помощью метода грубой силы путем перебора 2^L различных сообщений. Такие атаки имеют место быть, и их результативность зависит от значения L и от вычислительной мощности компьютера. Второй критерий, представляющий собой нахождение двух различных сообщений, которые в результате дают одно и то же хэш значение, называется нахождением коллизии. *Нахождение коллизии* в среднем требует перебора только $2^{L/2}$ сообщений.

Некоторые из применений криптографических функций, такие как хранение паролей, лишь частично затронуты атакой на коллизии. Нахождение пароля, который подойдет к атакуемому аккаунту требует метода грубой силы, а также доступ к хэш–значению оригинального пароля, что может быть тривиальным или нет. Обратное же получение пароля из хэш значения не является возможным.

В случае же, например, с электронной цифровой подписью документов, мошенник не может просто скопировать подпись с существующего документа. Злоумышленнику необходимо создать два документа и заставить обманом подписать владельца подписи неверный документ.

Структура SHA–256 для пустого сообщения представляет собой строку:

e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

Даже малейшее изменение в сообщении приведет (с бесконечно большой вероятностью) к совершенно другому хэш значению, благодаря эффекту снежного кома. Например, бит хэш значения. добавление точки к концу предложения меняет 111 из 224 бит оригинального сообщения.

Оригинальное сообщение:

SHA–224("The quick brown fox jumps over the lazy dog")

0x 730e109bd7a8a32b1cb9d9a09aa2325d2430587ddbc0c38bad911525

Значение после добавления точки:

```
SHA-224("The quick brown fox jumps over the lazy dog.")  
0x 619cba8e8e05826e9b8c519c0a5c68f4fb653e8a3d8aa04bb2c8cd4c
```

В данной работе применяется JWT (от английского JSON Web Token) использующий алгоритм HS-256, который в свою очередь является модификацией SHA-256. JWT состоит из заголовка, информационной нагрузки и подписи. Заголовок определяет какой алгоритм используется для генерации подписи и его структура выглядит примерно следующим образом:

```
header = '{"alg":"HS256","typ":"JWT"}',
```

HS-256 означает, что данный токен подписан с помощью HMAC-SHA256. Цифровая нагрузка несет в себе информацию, которую мы хотим передать, структура данного сегмента может выглядеть следующим образом:

```
payload = '{"loggedInAs":"admin","iat":1422779638}'.
```

В спецификации JWT есть рекомендация использовать в качестве одного из полей передаваемого информационного объекта временной штамп, который представлен здесь полем «iat».

Подпись создается с помощью кодирования алгоритмом base64url заголовка и информационной нагрузки, разделенных запятой. Так же в итоговом кодировании присутствует секретный ключ, хранящийся на сервере и как уже было отмечено ранее, меняющийся со временем. Небольшой кусок кода, представленный ниже является примером кодирования токена.

```
Key = 'secretToken'  
unsignedToken = encodeBase64(header) + '.' + encodeBase64(payload)  
signature = HMAC-SHA256(key, unsignedToken)
```

В случае аутентификации, когда пользователь успешно проходит операцию входа в систему с помощью своих данных, JWT будет сгенерирован и возвращен пользователю, который автоматически сохраняет его локально. Данный метод отличен от стандартного подхода, в котором сессия создается на сервере и пользователю возвращается cookie.

Когда у пользователя появляется желание получить доступ к защищенному пути или ресурсу на сервере он должен отослать токен, как в правило в заголовке авторизации. Данный механизм является механизмом без состояния, так как состояние пользователя никогда не сохраняется в памяти сервера.

Защищенные части сервера проверяют подлинность предоставленного токена, и если он присутствует и является верным, пользователю будет дана возможность получить доступ к запрашиваемой информации. Так как JWT является самостоятельным источником информации, вся необходимая информация уже хранится в нем, что уменьшает необходимость обращения к базе данных и таким образом увеличивает производительность системы.

Выводы

Несмотря на разнообразие протоколов аутентификации, применяемых в отечественных организациях, все они имеют свои уязвимости. Протокол TLS до сих пор считается одним из самых безопасных стандартов, при условии, что пользователь понимает риск использования ложных учётных данных, и поддерживается практически всеми производителями беспроводного оборудования, а также разработчиками сетевого программного обеспечения.

Исследуемый метод может аутентифицировать как клиента, так и сервер (т.е. он является методом взаимной аутентификации).

Но даже несмотря на требование предустановленного сертификата на стороне клиента, существует уязвимость, которой может воспользоваться злоумышленник, чтобы взломать нужный ему код, дополнительно узнав данные сертификата клиента.

Наилучшей безопасности можно добиться, храня сертификаты клиентов на стороннем носителе, к которому злоумышленник добраться не сможет. Таким носителем может стать переносимое USB-устройство, *токен*, хэш от которого будет являться уникальным кодом легитимного пользователя.

3 ПРОГРАММНОЕ КОНСТРУИРОВАНИЕ USB-ТОКЕНА

USB-токен – это небольшое устройство, похожее на обычную флэшку. *Аппаратный токен*, или *USB-ключ*, предназначен для обеспечения информационной безопасности пользователя. Используется для идентификации его владельца, безопасного удалённого доступа к информационным ресурсам, но в основном – для упрощения аутентификации пользователя. Внутри него записан специальный уникальный код, заменяющий другие способы двойной аутентификации.

Токены предназначены для электронного удостоверения личности (например, клиента, получающего доступ к банковскому счёту), при этом они могут использоваться как вместо пароля, так и вместе с ним. В некотором смысле токен — это электронный ключ для доступа к чему-либо.

Обычно аппаратные токены обладают небольшими размерами, что позволяет носить их в кармане или кошельке, часто они оформлены в виде брелоков. Некоторые предназначены для хранения криптографических ключей, таких как электронная подпись или биометрические данные (например, детали дактилоскопического узора). В одни токены встроена защита от взлома, в другие — мини-клавиатура для ввода PIN-кода или кнопка вызова процедуры генерации, и дисплей для вывода сгенерированного ключа. Токены обладают разъёмом USB, функциями RFID или беспроводным интерфейсом Bluetooth для передачи сгенерированной последовательности ключей на клиентскую систему.

Существуют также *программные токены*, которые выдаются пользователю после успешной авторизации, и являются ключом для доступа к специальным службам. Однако программные токены часто используются злоумышленниками, для несанкционированного доступа к аккаунту. Здесь они не рассматриваются.

Современные флэш-накопители информации с USB-интерфейсом образуют класс массовых мобильных устройств хранения данных, увеличивающих производительность труда пользователей персональных компьютеров (ПК).

Опрос компании SanDisk показал, что 77% служащих используют свои флэш-накопители как для личных, так и связанных с работой задач. При этом значительная часть хранимых данных относится к категории конфиденциальной информации: записи клиентов (25%), финансовая информация (17%), бизнес-планы (15%), контактная информация служащих (13%), маркетинговые планы (13%), интеллектуальная собственность (6%), коды программ (6%).

Приведенные данные доказывают, что флэш-накопители будут доступными и востребованными мобильными устройствами хранения деловой и личной информации в ближайший период (десятилетие) развития информационных систем.

Логично, что флэш-накопитель может стать альтернативой специальному USB-токену. Уникальным ключом может служить идентификационный код устройства. Владелец флэш-карты с помощью специальной программы сможет использовать её в качестве ключа. Процесс будет состоять из нескольких этапов:

- Установка нужного программного обеспечения на компьютер;
- Запуск требуемого для подключения Web-ресурса;
- Регистрация в сети с предварительно подключенным USB-устройством.
- Предварительное подключение, при следующих сеансах, использованного при начальной регистрации USB-устройства.

Для пользователя эта процедура выглядит максимально просто (рисунок 3.1).

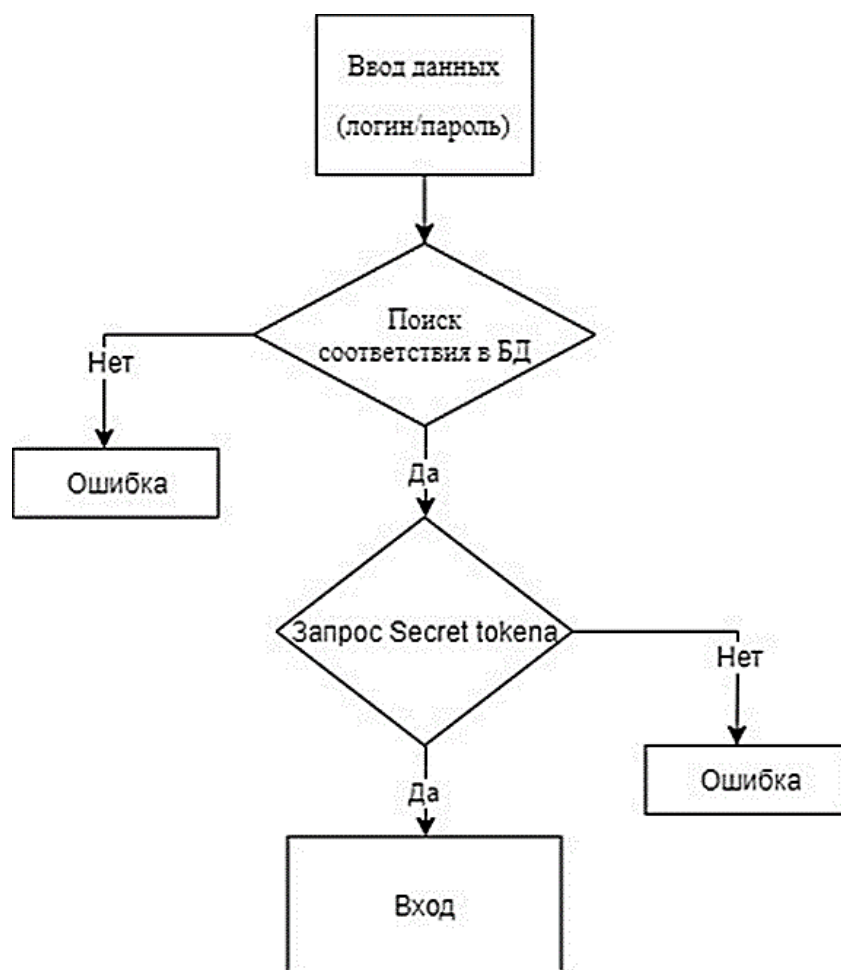


Рисунок 3.1 – Блок схема принципа работы Secret Token

На данный момент, для обеспечения этого процесса разработано программное приложение SECRET TOKEN, которое включает в себя в том числе:

- Приложение для браузера Google Chrome «USB INFO»;
- Расширение для браузера Google Chrome «Com.chrome.dstu.money_app».

Помимо указанных выше разработок, также создан Web-ресурс и база данных.

3.1 Обоснование выбора языков программирования

При написании программного приложения для браузера Google Chrome «Com.chrome.dstu.money_app» использовался язык программирования Python.

Для многих разработчиков основное преимущество языка Python заключается в удобочитаемости, ясности и более высоком качестве написания программного кода. Программный код на языке Python читается легче, а значит, многократное его использование и обслуживание выполняется гораздо проще, чем использование программного кода на других языках. Единообразие оформления программного кода на языке Python облегчает его понимание даже для тех, кто не участвовал в его создании. Кроме того, язык Python поддерживает самые современные механизмы многократного использования программного кода. Сценарии языка Python могут легко взаимодействовать с другими частями приложения, благодаря различным механизмам интеграции. Эта интеграция позволяет использовать Python для настройки и расширения функциональных возможностей программных продуктов.

Для создания расширения USB INFO был выбран язык программирования JavaScript, специально разработанный для создания интерактивных HTML–документов. Это объектно–ориентированный язык разработки встраиваемых приложений, выполняющих как на стороне клиента, так и на стороне сервера.

Клиентские приложения выполняются браузером просмотра Web–документов на машине пользователя, серверные приложения выполняются на сервере.

Одним из уникальных преимуществ языка JavaScript является его широкая распространённость и кроссплатформенность. Язык поддерживается во всех операционных системах, во всех видах браузеров, на настольных компьютерах и на ноутбуках, а также на мобильных устройствах – планшетах и на смартфонах. Очень важно ещё и то, что JavaScript–приложения работают, не требуя установки их на компьютеры пользователей, а поддерживаются браузерами.

3.2 Программная реализация приложения SECRET TOKEN

Приложение SECRETTOKEN работает с пользовательской моделью программы.

Одним из основных составляющих данного программного продукта является установление безопасного соединения между клиентом и сервером.

SECRET TOKEN не имеет особых требований, поэтому он может работать с пользовательскими моделями, которые основаны на любой системе базы данных.

Ниже перечислены три обязательных элемента:

- `is_authenticated`: свойство, которое является `True`, если пользователь имеет действительные учетные данные, или `False`, иным образом;
- `is_active`: свойство, которое является `True`, если учетная запись пользователя активна, или `False`, иным образом;
- `get_id`: метод, который возвращает уникальный идентификатор для пользователя в виде строки (Secret Token).

Данные свойства являются довольно универсальными и легко реализуются на практике. Для пользователя создаются следующие обязательные элементы: логин, пароль, адрес электронной почты и секретный токен. Если первые три вводит сам пользователь, то секретный токен считывается с помощью программного приложения. В качестве секретного кода, программа берет хэш от уникального кода ID (серийного номера), предварительно подключенного и выбранного в качестве такового USB-устройства. За каждым пользователем закрепляется история совершенных транзакций [56].

Листинг 1 – Фрагмент кода класса пользователя

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), index=True, unique=True)
    email = db.Column(db.String(120), index=True, unique=True)
    password_hash = db.Column(db.String(128))
    secret_token = db.Column(db.String(140)) # секретный токен
    transactions = db.relationship("Transaction", backref='author', lazy='dynamic') # связь
                                                    с транзакциями пользователя

    def __repr__(self):
        return '<User {}>'.format(self.username)

    def set_password(self, password): #метод установки нового пароля
        self.password_hash = generate_password_hash (password)

    def check_password(self, password): #метод проверки пароля
        return check_password_hash(self.password_hash, password)
```

SECRET TOKEN отслеживает вошедшего пользователя, сохраняя его уникальный идентификатор в пользовательском сеансе, и место хранения, назначенное каждому пользователю, который подключается к приложению. Каждый раз, когда пользователь, вошедший в систему, переходит на новую страницу, SECRET TOKEN извлекает идентификатор пользователя из сеанса, а затем загружает этого пользователя в память.

Поскольку SECRET TOKEN «ничего не знает» о базах данных, ему нужна помощь расширения в загрузке пользователя. О нём речь пойдет в следующей части работы. А пока вернемся к функционированию приложения.

Основной маршрут операций, которые видит пользователь будет заключаться в следующем: в случае успешной авторизации будет показана история транзакций, в противном случае – пользователю будет предложено сперва зарегистрироваться / авторизоваться.

Листинг 2 – Фрагмент кода добавления новой транзакции

```
@app.route('/', methods=['GET', 'POST'])
@app.route('/index', methods=['GET', 'POST'])
@login_required
def index():
    form = TransactionForm()
    if form.validate_on_submit():
        transaction = Transaction(body=form.post.data, money=form.money.data,
author=current_user)
        db.session.add(transaction)
        db.session.commit()
        flash('Transaction successfully completed!')
        return redirect(url_for('index'))
```

Одним из свойств, необходимым для эффективного функционирования программного приложения SECRET TOKEN, было свойство `is_authenticated`, которое проверяет, есть ли пользователь уже в системе.

Первым шагом является загрузка пользователя из базы данных. Имя пользователя пришло с формой представления, так что теперь модно запросить базу данных, чтобы найти пользователя. Если имеется совпадение для имени пользователя, которое было предоставлено, далее возможно проверить, является ли пароль, который также пришел с формой, действительным. Это делается путем вызова `check_password`. Таким образом, теперь есть два возможных условия ошибки: имя пользователя может быть недействительным, или пароль может быть неверным для пользователя. В любом из этих случаев пользователь будет перенаправлен обратно на этап входа, чтобы пользователь мог повторить попытку.

Если имя пользователя и пароль оба правильные, то проверяется `login_user` функцию, которая исходит из SECRET TOKEN. Эта функция будет регистрировать пользователя как вошедшего в систему, так что это означает, что любые будущие страницы, на которые пользователь переходит, будут иметь `current_user` переменную, установленную для этого пользователя.

Также нужно будет предложить пользователям возможность выхода из приложения. Это можно сделать с помощью функции `logout_user`:

Листинг 3 – Фрагмент кода разлогина

```
@app.route('/logout')
def logout():
    logout_user()
    return redirect(url_for('index'))
```

SECRET TOKEN предоставляет очень полезную функцию, которая заставляет пользователей войти в систему, прежде чем они смогут просматривать определенные страницы приложения. Если пользователь, который не вошел в систему, попытается просмотреть защищенную страницу, SECRET TOKEN автоматически перенаправит пользователя на форму входа в систему.

Листинг 4 – Фрагмент кода логина

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    # Если текущий юзер уже авторизован - покажем историю транзакций
    if current_user.is_authenticated:
        return redirect(url_for('index'))
    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.data).first()
        # Найдём в базе запись о пользователе по значению секретного токена
        # HASH (SERIAL ID USB)
        st_user = User.query.filter_by(secret_token=form.secret_token.data).first()
        # Не найден пользователь с таким именем или введен неверный пароль
        if user is None or not user.check_password(form.password.data):
            flash('Invalid username or password')
            return redirect(url_for('login'))
        # Не найдена в БД запись о пользователе с переданным значением
        # секретного токена или значения секретных токенов не совпадают
        if st_user is None or user.id != st_user.id:
            flash('Invalid secret_token, report the problem to your system administrator')
            return redirect(url_for('login'))
        login_user(user, remember=form.remember_me.data)
        next_page = request.args.get('next')
        if not next_page or url_parse(next_page).netloc != "":
            next_page = url_for('index')
        return redirect(next_page)
    return render_template('login.html', title='Sign In', form=form)
```

Способ, которым SECRET TOKEN защищает функцию просмотра от анонимных пользователей, заключается в вызове `@login_required`. Когда пользователь, не вошедший в систему, обращается к функции представления, защищенной с помощью `@login_required`, сервер собирается перенаправить на страницу входа, но он будет включать некоторую дополнительную информацию в этом перенаправлении, чтобы приложение могло затем вернуться на первую страницу. Сразу после того, как пользователь вошел в систему, вызвав функцию SECRET TOKEN `login_user()`, будет получено значение аргумента `next` строки запроса. Приложение USB INFO предоставляет переменную, содержащую всю информацию, которую клиент отправил вместе с запросом.

Последний элемент функциональности, который необходимо построить – это регистрационная форма, чтобы пользователи могли зарегистрироваться через Web-форму. Выглядит следующим образом:

Листинг 5 – Фрагмент кода регистрации пользователя

```
class RegistrationForm(FlaskForm):
    username = StringField('Username', validators=[DataRequired()])
    email = StringField('Email', validators=[DataRequired(), Email()])
    password = PasswordField('Password', validators=[DataRequired()])
    password2 = PasswordField('Repeat Password', validators=[DataRequired(),
EqualTo('password')])
    submit = SubmitField('Register')
    secret_token = PasswordField('SecretToken', validators=[DataRequired()])
    def validate_username(self, username):
        user = User.query.filter_by(username=username.data).first()
        if user is not None:
            raise ValidationError('Please use a different username.')
    def validate_email(self, email):
        user = User.query.filter_by(email=email.data).first()
        if user is not None:
            raise ValidationError('Please use a different email address.')
    def validate_secret_token(self, secret_token):
        return True
```

3.3 Программная реализация дополнительного приложения для SECRET TOKEN «USB INFO»

Для функционирования приложения необходимо дополнение, которое будет помогать находить и выбирать USB–устройство при регистрации пользователя, и в дальнейшем при работе пользователя в сети посылать сигнал о том, подключено оно или нет. Это своего рода скрипт–помощник приложения. Такое расширение постоянно проверяет, подключено ли USB–устройство к сети.

Работа приложения заключается в двух основных действиях:

- Если данные получены, то происходит переход на следующую страницу;
- Если был сигнал об удалении устройства, то поле с данными очищается, в том числе логин и пароль пользователя; и дальнейшая работа в сети уже невозможна.

Листинг 6 – Фрагмент кода расширения для браузера

Login helper

```
'use strict';
function updateCypherContent() {
  chrome.storage.sync.get('selected_device', function(data) {
    let security_content = document.getElementById('secret_token');
    if (data.selected_device) { //
      if (security_content) {
        security_content.value =
data.selected_device.hashedException.toString();
      }
    });
    updateCypherContent();
    chrome.extension.onMessage.addListener(function(request){
      if (request.command) {
        if (request.command.on_device_removed) {
          updateCypherContent()

chrome.extension.onMessage.addListener(function(request){
  if (request.command) {
    if (request.command.on_device_removed) { //
      window.location.replace("http://127.0.0.1:5000/logout");
    }
  }
}
```

Также необходим алгоритм, который будет преобразовывать уникальный серийный номер устройства в хэш и шифровать его. Для шифрования была использована асинхронная функция вычисления SHA–256.

Листинг 7 – Фрагмент кода вычисления функции SHA-256

```
async function digestMessage(message) {
  const msgUint8 = new TextEncoder().encode(message);
    // encode as (utf-8) Uint8Array
  const hashBuffer = await crypto.subtle.digest('SHA-256', msgUint8); // hash the
  message
  const hashArray = Array.from(new Uint8Array(hashBuffer)); // convert buffer
  to byte array
  const hashHex = hashArray.map(b => b.toString(16).padStart(2, '0')).join(""); // convert
  bytes to hex string
  return hashHex;
}
```

При регистрации пользователя за ним закрепляется выбранное устройство. Это извлекаемое устройство, и нужно, чтобы данные могли постоянно обновляться.

Листинг 8 – Фрагмент кода обновление устройства

```
var devices = {};
function appendToDeviceSelector(device) {
  var el = document.createElement('option');
  el.setAttribute('value', device.device);
  el.textContent =
    'Product 0x' + ('0000' + device.productId.toString(16)).slice(-4) +
    ' Vendor 0x' + ('0000' + device.vendorId.toString(16)).slice(-4);
  device_selector.add(el);
};
function appendDeviceInfo(name, value) {
  // метод, который выведет на device_info информацию о выбранном устройстве
  var el = document.createElement('b');
  el.textContent = name + ':'
  device_info.appendChild(el);
  device_info.appendChild(document.createTextNode(value));
  device_info.appendChild(document.createElement('br'));
}
```

Приложение берет информацию об устройстве по индексу в списке. При запуске подается запрос операционной системе и создается список всех подключенных к компьютеру внешних устройств.

Листинг 9 – Фрагмент кода работы списка подключенных устройств

```
var device = devices[device_selector.options.item(index).value];
appendDeviceInfo(
  'Product ID',
  '0x' + ('0000' + device.productId.toString(16)).slice(-4));
```



```

appendDeviceInfo(
  'Vendor ID',
  '0x' + ('0000' + device.vendorId.toString(16)).slice(-4));
chrome.usb.openDevice(device, function(handle) {
  if (chrome.runtime.lastError !== undefined) {
    var el = document.createElement('em');
    el.textContent = 'Failed to open device: ' +
      chrome.runtime.lastError.message;
    device_info.appendChild(el);
  } else {
    populateDeviceInfo(handle, function () {
      chrome.usb.closeDevice(handle);
    });
  }
});
}
}
chrome.usb.getDevices({}, function(found_devices) {
  if (chrome.runtime.lastError !== undefined) {
    console.warn('chrome.usb.getDevices error: ' +
      chrome.runtime.lastError.message);
  }
});

```

Так как устройство извлекаемо, то необходимо постоянное динамическое обновление списка. Если устройство извлекается, то программное приложение отправляет данную информацию в основную программу.

Листинг 10 – Фрагмент кода обрыва связи с устройством

```

if (chrome.usb.onDeviceRemoved) {
  chrome.usb.onDeviceRemoved.addListener(function (device) {
    delete devices[device.device];
    for (var i = 0; i < device_selector.length; ++i) {
      if (device_selector.options.item(i).value == device.device) {
        device_selector.remove(i);
        deviceSelectionChanged();
        break;
      }
    }
  });
  sendMessageToExtension({command: {on_device_removed: true}}); //

```

Эта двойная связь типа: операционная система – программное приложение обеспечивает непрерывный мониторинг за всеми подключенными устройствами.

3.4 Реализация программного приложения SECRET TOKEN

В тестовом виде процесс работы выглядит так (рисунок 3.2):

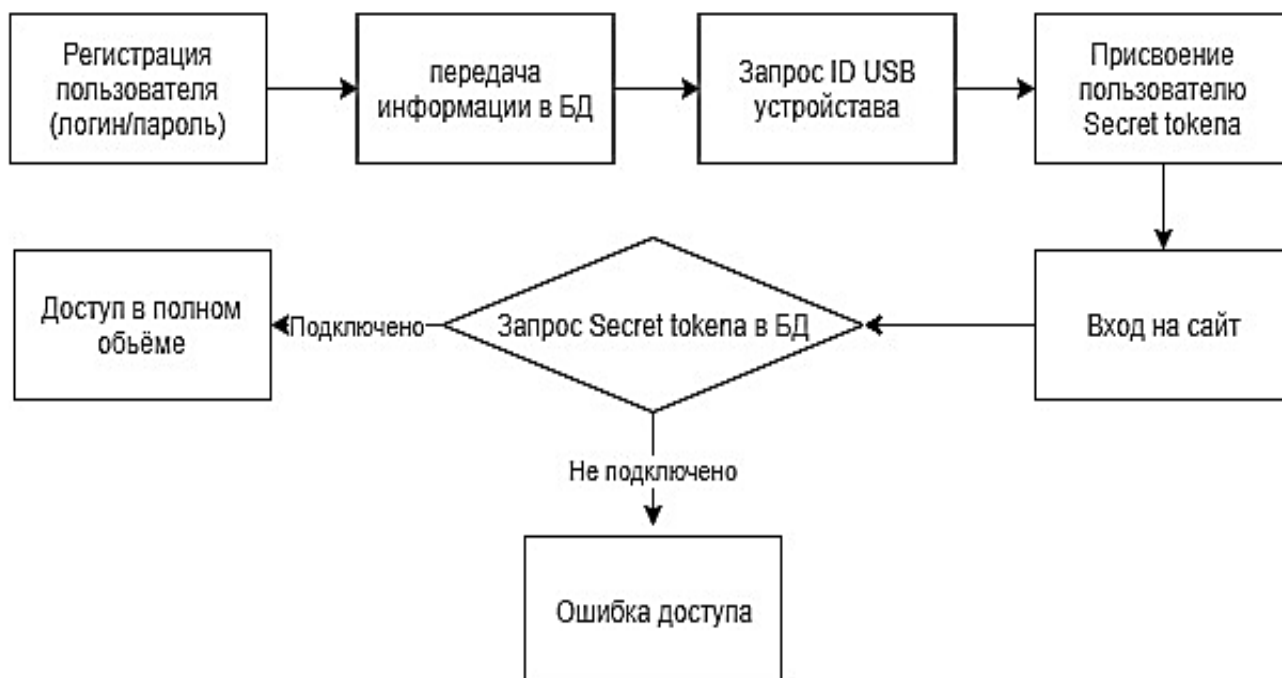


Рисунок 3.2 – Блок схема принципа работы SECRET TOKEN

Весь процесс аутентификации пользователя состоит из 7 шагов. Остановимся на них подробнее, перечислив задачи, решаемые на каждом шаге:

Шаг 1. Запуск Web-ресурса через терминал ОС показан на рисунке 3.3.

```
polyak@polyak-VirtualBox: ~/Рабочий стол/web_app
Файл Правка Вид Поиск Терминал Справка
polyak@polyak-VirtualBox:~$ cd Рабочий\ стол
polyak@polyak-VirtualBox:~/Рабочий стол$ cd web_app
polyak@polyak-VirtualBox:~/Рабочий стол/web_app$ source venv/bin/activate
(venv) polyak@polyak-VirtualBox:~/Рабочий стол/web_app$ flask run
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

The screenshot shows a terminal window titled 'polyak@polyak-VirtualBox: ~/Рабочий стол/web_app'. The user navigates to the 'web_app' directory and runs 'flask run'. The output shows the application is running on http://127.0.0.1:5000/.

Рисунок 3.3 — Запуск серверной части приложения

Шаг 2. Переходим на страницу <http://127.0.0.1:5000/> на рисунке 3.4.

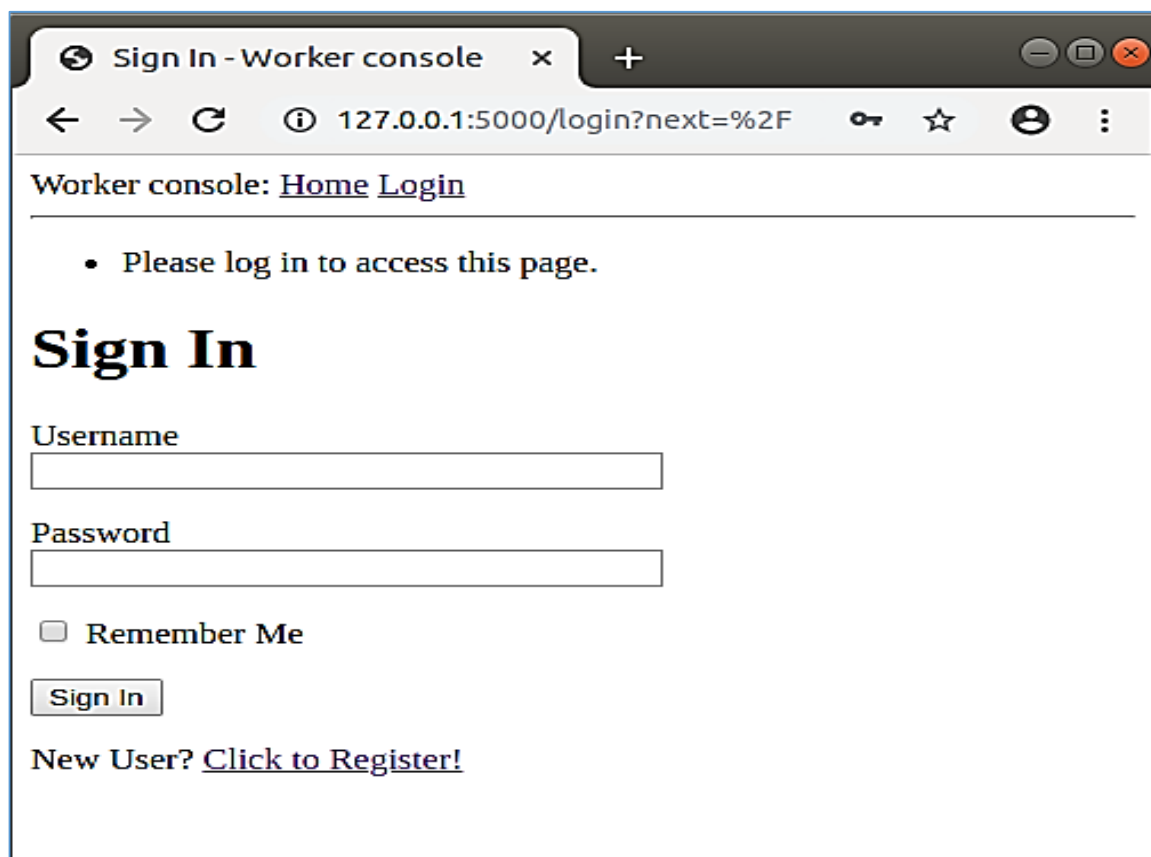


Рисунок 3.4 — Страница авторизации

Шаг 3. Для регистрации пользователя необходимо настроить расширение. Для этого запускаем приложение «com.chrome.dstu.money_app» (рисунок 3.5).

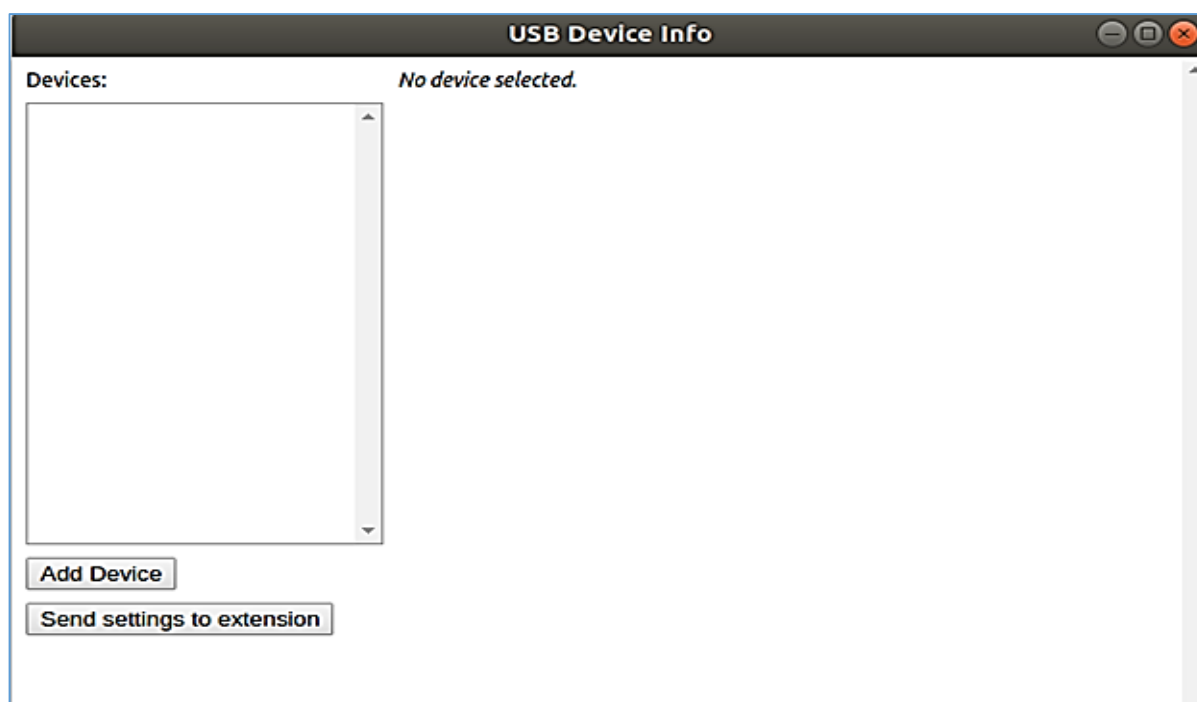


Рисунок 3.5 — Приложение «com.chrome.dstu.money_app»

Шаг 4. Выбираем подключенное USB-устройство в списке на рисунке 3.6.

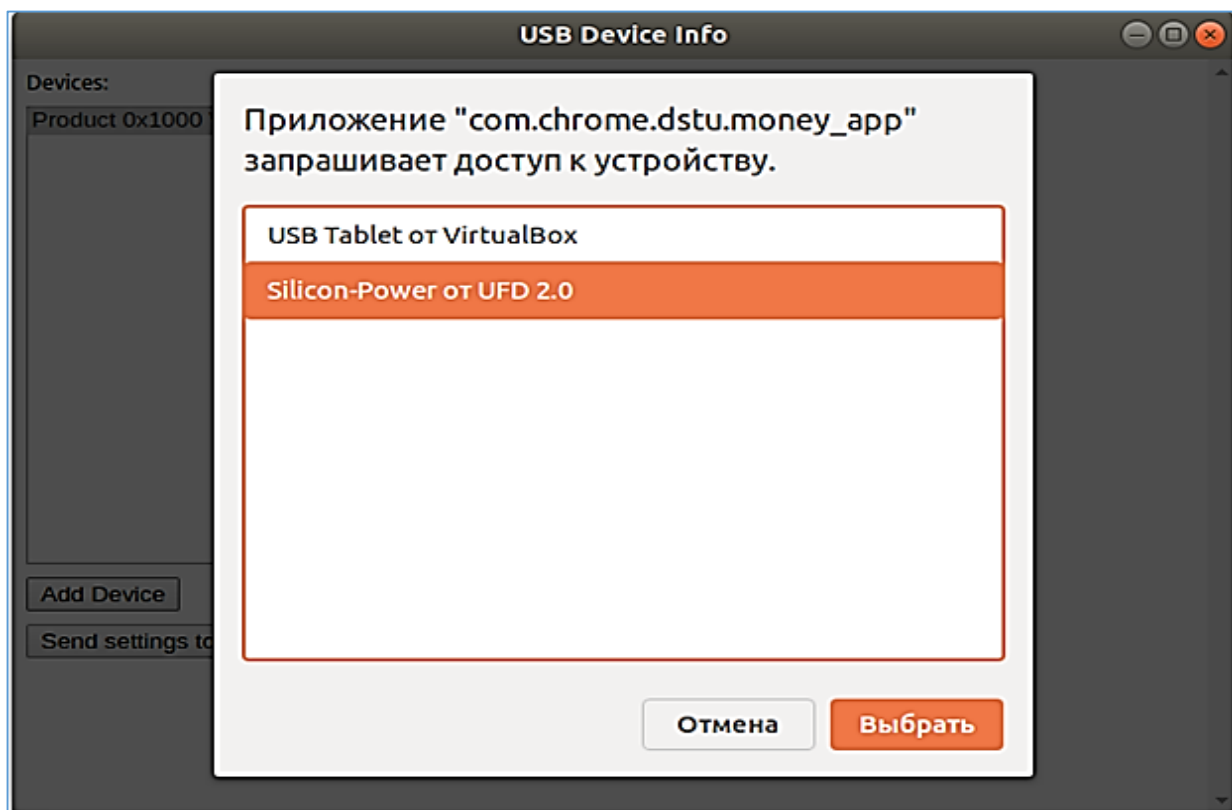


Рисунок 3.6 — Список устройств

На рисунке 3.7 показана подробная информация о выбранном устройстве

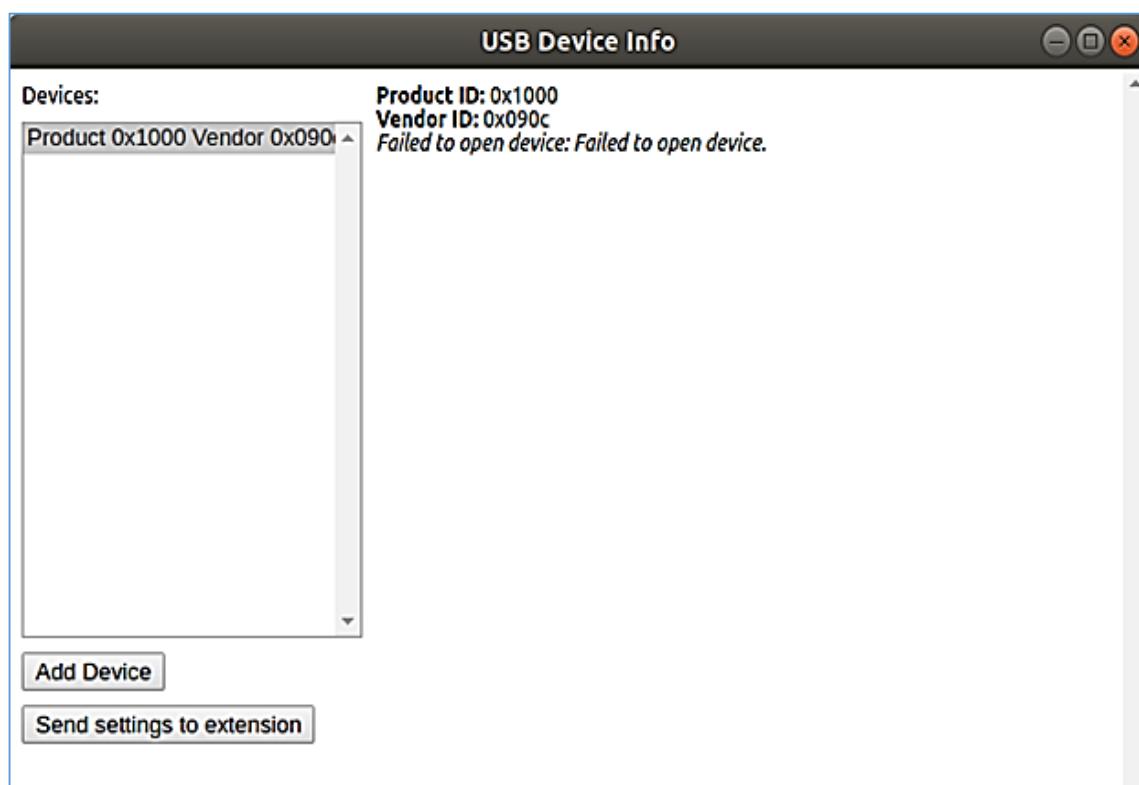
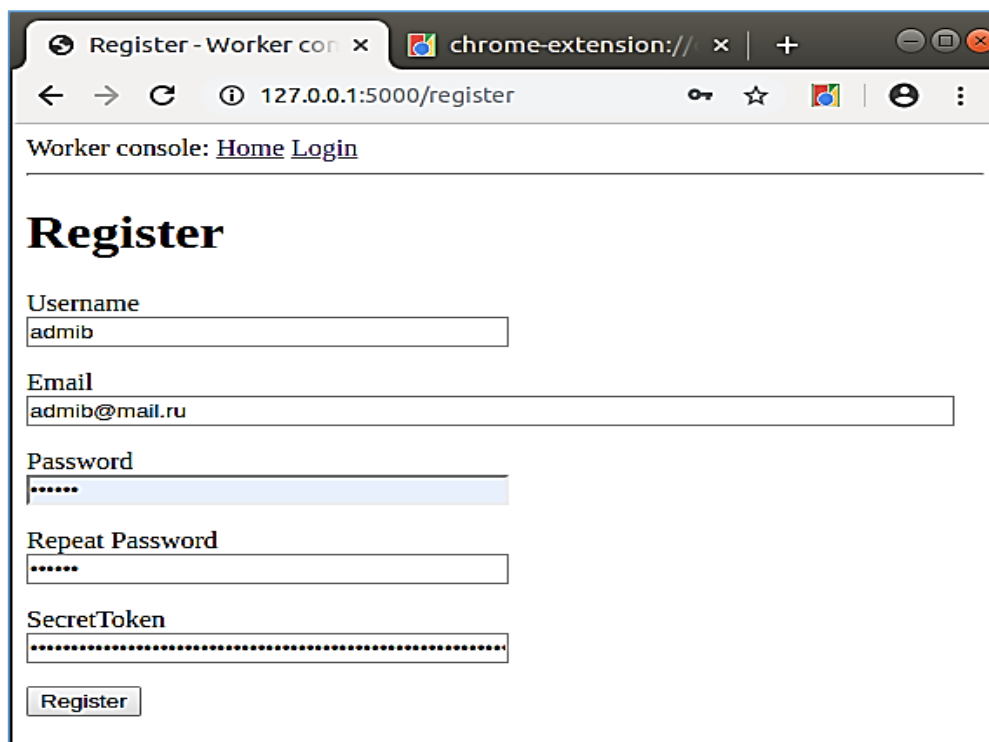


Рисунок 3.7 — Информация о выбранном устройстве

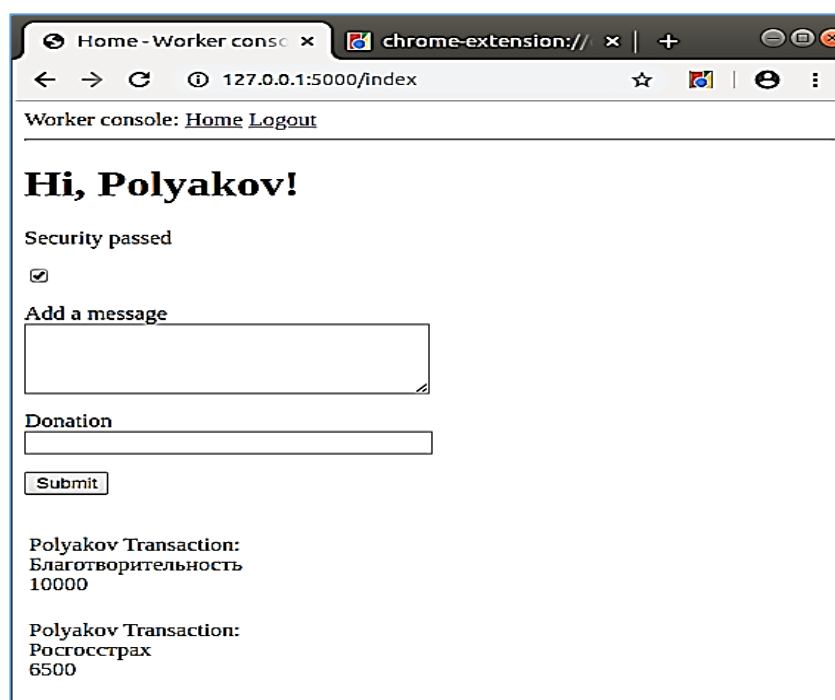
Шаг 5. Выбранное устройство передается от приложения к странице настройки расширения. Теперь оно является токеном. На странице регистрации появляется новое поле – SecretToken на рисунке 3.8.



The screenshot shows a web browser window with the address bar displaying 'chrome-extension:// 127.0.0.1:5000/register'. The page title is 'Register - Worker console'. Below the title, there is a link 'Home Login'. The main heading is 'Register'. The form contains the following fields: 'Username' with the value 'admib', 'Email' with the value 'admib@mail.ru', 'Password' with masked characters '.....', 'Repeat Password' with masked characters '.....', and 'SecretToken' with masked characters '.....'. A 'Register' button is located at the bottom of the form.

Рисунок 3.8 — Регистрация пользователя

Шаг 6. Обновляем страницу. При успешной активации токена будет произведен вход в личный кабинет на рисунке 3.9.



The screenshot shows a web browser window with the address bar displaying 'chrome-extension:// 127.0.0.1:5000/index'. The page title is 'Home - Worker console'. Below the title, there is a link 'Home Logout'. The main heading is 'Hi, Polyakov!'. The page displays a 'Security passed' message with a checkmark icon. Below this, there is a text input field labeled 'Add a message'. Further down, there is a text input field labeled 'Donation' and a 'Submit' button. At the bottom, there are two transaction records: 'Polyakov Transaction: Благотворительность 10000' and 'Polyakov Transaction: Росгосстрах 6500'.

Рисунок 3.9 — Активация токена

Теперь информация о пользователях и произведенных ими транзакциях будет храниться в базе данных на рисунках 3.10 и 3.11.

The screenshot shows a database application interface with a menu bar (Файл, Редактирование, Вид, Инструменты, Справка) and a toolbar. The 'Данные' (Data) tab is active, showing the 'user' table. The table has columns: id, username, email, password_hash, and secret_token. A single row of data is visible for user 'Polyakov'.

	id	username	email	password_hash	secret_token
1	1	Polyakov	Polyakov@mail.ru	pbkdf2:sha256...	b1c7dee29dc404d312958c3388e9c054d44b0134b76847b6ac5e99d9...

Рисунок 3.10 — База данных пользователей

The screenshot shows the same database application interface, but with the 'transaction' table selected. The table has columns: id, body, timestamp, user_id, and money. Two rows of data are visible, representing transactions for 'Благотворительность' and 'Росгосстрах'.

	id	body	timestamp	user_id	money
1	1	Благотворительность	2019-11-11 19:22:45...	1	10000
2	2	Росгосстрах	2019-11-11 19:23:47...	1	6500

Рисунок 3.11 — База данных транзакций

Шаг 7. При извлечении USB устройства происходит очищение кэша и пользователя перебрасывает на страницу ввода логина и пароля на рисунке 3.12.

The screenshot shows a web application with a sign-in page and a USB device management overlay. The sign-in page has fields for Username (Polyakov) and Password (*****), a 'Remember Me' checkbox, and a 'Sign In' button. The USB device management overlay shows a list of devices (currently empty) and buttons for 'Add Device' and 'Send settings to extension'.

Sign In

Username: Polyakov

Password: *****

☐ Remember Me

New User? [Click to Register!](#)

USB Device Info

Now selected:

List of devices

Devices: No device selected.

Рисунок 3.12 — Очистка кэша

При попытке стороннего пользователя зайти через автоматически сохраненный логин и пароль или при введении их на любом другом ПК, на странице будет возникать ошибка на рисунке 3.13.

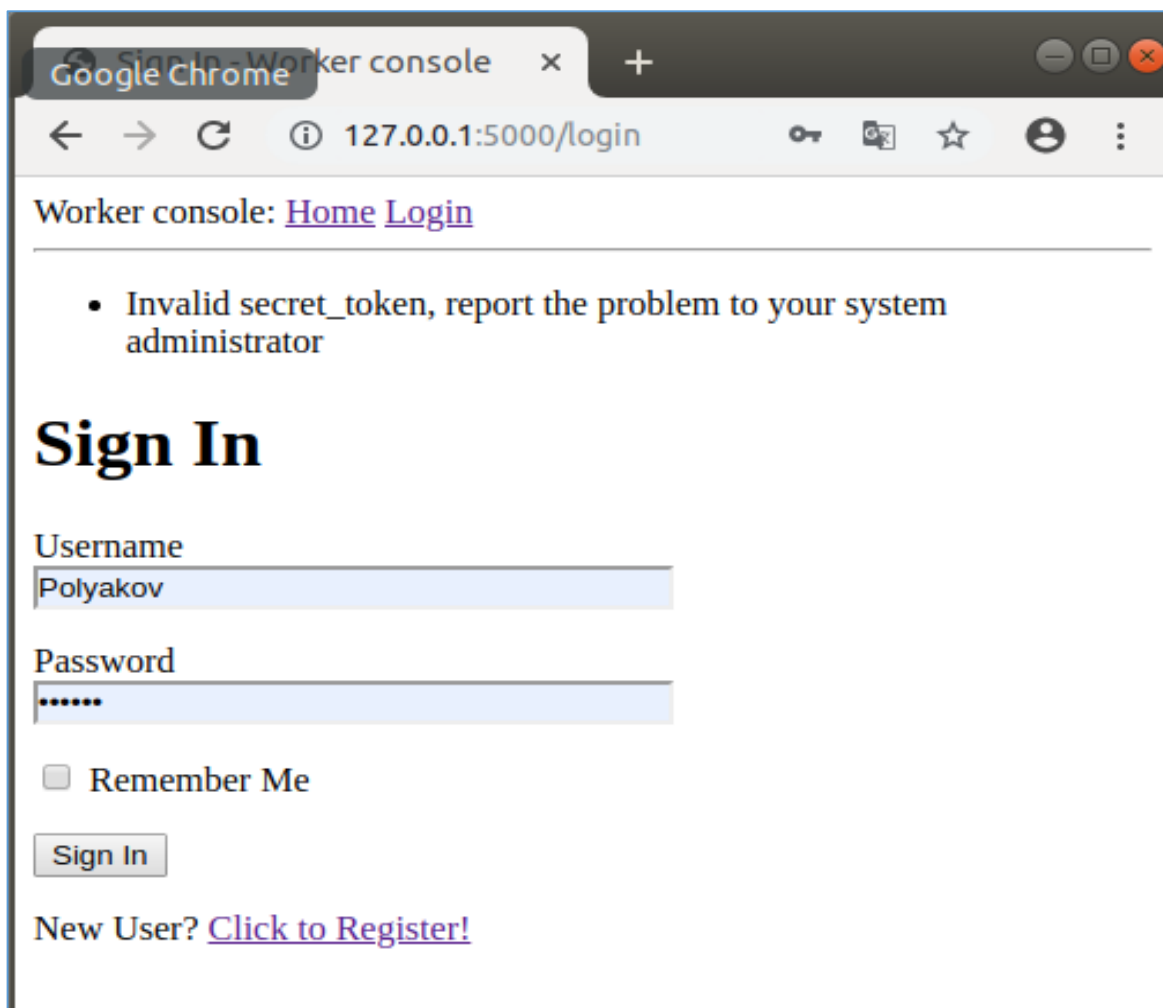


Рисунок 3.13 — Ошибка входа

Выводы

В результате проведенных исследований и проделанной работы получен и протестирован импортозамещающий алгоритм, позволяющий обеспечить высокий уровень безопасности пользователя в сети с помощью USB-устройства, без предустановленных на него программ. Данные возможности протестированы на базе операционной системы семейства Linux и браузера Google Chrome.

Таким образом, реализовано импортозамещающее программное средство SECRET TOKEN, позволяющее пользователю проводить безопасные транзакции в отечественных электронных платёжных (банковских, коммерческих) системах, обеспечивая при этом защиту передаваемых данных в канале связи.

РЕЗУЛЬТАТЫ 8 ГЛАВЫ

В рамках проведенных исследований был выполнен обзор существующих угроз для пользователей электронных платежных систем, изучены основные протоколы аутентификации пользователей на Web-ресурсах [57].

Выполнен обзор криптографического алгоритма шифрования SHA-256, проведен анализ его работы, а также рассмотрены хэш-функции.

В качестве операционной системы, на базе которой предусмотрено функционирование данного приложения, выбрана ОС семейства Linux.

Разработанное программное средство SECRET TOKEN предоставляет возможность организациям различного уровня, в том числе обычному пользователю реализовать безопасные транзакции в рамках электронных платежных систем, обеспечивая при этом защиту передаваемой информации в канале связи, контролировать частоту содержимое передаваемого трафика, а также постоянную регистрацию событий в базе данных [58].

В ходе тестирования, реализованное программное средство SECRET TOKEN подтвердило возможность сетевой передачи, обработки и учёта информации в защищенном виде. Это доказывает эффективность его использования в различных отечественных платёжных (банковских, коммерческих) системах.

В ходе данной работы была разработана и протестирована программа безопасной аутентификации пользователя с помощью Web USB – токена.

Разработанные клиентская и серверная часть приложения удовлетворяют всем представленным в первом разделе требованиям безопасности [59].

Приложение прошло проверки и было протестировано на распространенные уязвимости аутентификации платёжных систем, и успешно прошло все тесты.

Разработанное программное средство является расширяемым и легко конфигурируемым. Его можно применять в проектах любой сложности на большинстве современных ОС, с возможностью доступа с любого браузера.

Таким образом, в результате работы были достигнуты поставленные задачи, а значит, и основная цель – возможность применения USB-токена для обеспечения безопасности пользователя на этапе аутентификации при совершении действий в электронных платежных системах.

ГЛАВА 9

СОЗДАНИЕ ИМПОРТОЗАМЕЩАЮЩЕЙ СИСТЕМЫ ОБНАРУЖЕНИЯ И РАСПОЗНАВАНИЯ ЛИЦА ЧЕЛОВЕКА В КАДРЕ С ЦЕЛЬЮ АВТОРИЗАЦИИ ПОЛЬЗОВАТЕЛЕЙ С ПОМОЩЬЮ ВИДЕОКАМЕРЫ

ОБЩИЕ ПОЛОЖЕНИЯ

За последние десять лет распознавание лиц людей стало популярной темой исследований в области компьютерного зрения и одним из наиболее успешных способов анализа и понимания изображений. Из-за характера проблемы, в ней заинтересованы исследователи не только в области информатики, но и в области нейробиологии и психологии. Успехи в исследованиях компьютерного зрения смогут предоставить полезную информацию о том, как работает мозг человека.

В настоящее время наблюдается непреходящий интерес к проблеме распознавания лиц. Под *распознаванием* понимается идентификация изображения неизвестного лица с одной из известных персон, или, в случае авторизации и аутентификации, сравнение лица в кадре с эталонным изображением в базе.

Задача распознавания лиц актуальна как в области интеллектуальных сред, так и в системах безопасности. Во многих странах, в числе которых Китай, Япония и США, созданы системы распознавания лиц, внедрённые в систему общественной безопасности и давно используются Федеральными службами в биометрических технологиях трёхмерного распознавания лиц. Использование точной и надёжной системы распознавания лиц требует немалых ресурсов, но за последнее время технологии значительно развились, и скоро эта проблема будет решена.

Объект исследования – компьютерное зрение, анализ изображений, понимание биометрических технологий при решении задачи распознавания лиц людей.

Предмет исследования – идентификация изображения лица пользователя с целью его авторизации и аутентификации с помощью видеокамеры.

Целью работы является исследование алгоритмов обнаружения и распознавания лица человека в кадре, и создание, на основе этих методов, импортозамещающей компьютерной программы, позволяющей осуществлять авторизацию пользователей при помощи видеокамеры, используя их уникальные биометрические показатели.

Для достижения описанной выше цели был выявлен ряд задач, таких, как исследование *сред глубокого обучения* для обнаружения лица в кадре, исследование и применение в совокупности *алгоритмов распознавания личности*, создание *расширяемой базы данных сотрудников*, формирование *тренировочной базы* для алгоритмов распознавания личности, создание *программного средства авторизации* пользователя с *мандатной системой доступа* к информационной системе.

1 ИДЕНТИФИКАЦИЯ ЧЕЛОВЕКА ПО БИОМЕТРИЧЕСКИМ ДАННЫМ

Биометрическая идентификация – это предъявление пользователем своего уникального биометрического параметра и процесс его сравнения со всей базой имеющихся биометрических данных. Для извлечения такого рода персональных данных используются специальные устройства – биометрические считыватели.

Биометрические системы контроля доступа удобны для пользователей тем, что носители информации находятся всегда при них, не могут быть утеряны либо украдены. Биометрический контроль доступа считается более надёжным, т.к. идентификаторы не могут быть переданы третьим лицам или скопированы.

Системы распознавания лиц людей используют компьютерные алгоритмы, чтобы выделить конкретные отличительные детали лица человека. Эти детали, такие как расстояние между глазами или форма подбородка, затем преобразуются в математическое представление и сравниваются с данными о лицах других людей, собранными в базе данных распознавания лиц. Данные о конкретном лице часто называют шаблоном лица и отличаются от фотографии, поскольку они предназначены только для включения определённых деталей, которые можно использовать для различения одного лица от другого [60].

Современные методы идентификации человека делятся на класс физических и класс поведенческих методов. *Поведенческие* методы оценивают действия индивидуума по подписи или тембру голоса, предоставляя пользователю некоторую степень контроля над его поступками. *Физические* методы идентификации, основанные на биометрических признаках человека, наиболее полно и часто реализуемы в современных интеллектуальных системах.

К уникальным характеристикам, по которым производится идентификация человека, относятся отпечатки пальцев, форма ушной раковины, рисунок вен на лице и ладони, узор радужной оболочки и изображение сетчатки глаза и др. Лицо, голос и запах каждого человека также индивидуальны.

1.1 Методы биометрической идентификации

Существует два основных класса методов биометрической идентификации: статические (похожие на неподвижную фотографию) и динамические (в движении).

Первый класс статических методов основан на физиологических признаках человека, присутствующих с ним на протяжении всей его жизни. К ним относится идентификация: по отпечатку пальца; по лицу; по радужной оболочке и сетчатке глаза; по геометрии руки; по термограмме лица; по ДНК; по рисунку вен на лице и на руке; по форме ушной раковины на основе акустических характеристик уха, и др.

Второй класс динамических методов берёт за основу характеристики поведения людей, а именно, подсознательные движения в процессе повторения какого-либо обыденного действия: рукописного и клавиатурного почерка, модуляций голоса, манеры говорить и двигаться, особенностей походки, мимики лица, и др. К таким методам относится идентификация по голосу; по рукописному и клавиатурному почерку, по мимике лица, по манере говорить, по жестикуляции, и по другим перечисленным динамическим характеристикам человека.

Одним из приоритетных видов поведенческой биометрии является манера печатать на клавиатуре – клавиатурный почерк. При её определении фиксируется скорость печати, давление на клавиши, длительность нажатия на клавишу, промежутки времени между нажатиями, использование определённых клавиш, и др.

Отдельным биометрическим фактором может служить манера использования мыши. Помимо этого, поведенческая биометрия охватывает еще очень много факторов, которые не связаны с компьютером, — походка, например, особенности того, как человек поднимается по лестнице, переходит дорогу, и так далее.

Существуют также комбинированные системы идентификации, которые используют несколько биометрических характеристик, что позволяет удовлетворить самые строгие требования к надёжности и безопасности систем контроля доступа.

1.2 Критерии биометрической идентификации

Существуют определенные показатели, которые используют для определения эффективности *биометрической идентификации*. Это такие показатели, как:

FAR (False Acceptance Rate) – коэффициент ложного пропуска;

FMR (False Match Rate) – вероятность ложного совпадения параметров, того, что система неверно сравнивает входной образец с несоответствующими шаблонами, заложенными в базу данных для идентификации;

FRR (False Rejection Rate) – коэффициент ложного отказа;

FNMR (False Non-Match Rate) – вероятность ложного несовпадения параметров, того, что система может ошибиться в определении совпадений между входным образцом и соответствующим шаблоном из базы данных;

График ROC (Receiver Operating Characteristic Curve) – это графическая визуализация компромисса между коэффициентами ложного пропуска и ложного отказа, т.е. биометрическими характеристиками FAR и FRR.

Коэффициент отказа в регистрации (FER, Failure Equality Registration) – показатель безуспешных попыток создать шаблон из входных данных (при их низком качестве), что и вызывает отказ в регистрации.

Коэффициент ошибочного удержания (FTC, Failure Technical Control) – вероятность того, что автоматизированная система не способна определить биометрические входные данные, когда они представлены корректно.

Ёмкость шаблона – максимальное количество наборов данных, которые могут храниться в системе биометрической идентификации.

В России использование биометрических данных регулируются Статьей 11 Федерального закона «О персональных данных» № 152–ФЗ от 27.07.2006 г.

1.3 Идентификация по геометрии лица

Существует огромное множество методов распознавания личности по геометрии лица, и все они основаны на том, что черты лица и форма черепа для каждого человека индивидуальны. Эта область биометрики привлекательна для многих учёных, потому что люди узнают друг друга, в первую очередь, по лицу.

Эта область делится на два направления: 2D– и 3D–распознавание. У каждого из методов есть свои достоинства и недостатки, однако многое зависит ещё и от области применения и требований, предъявленных к конкретному алгоритму.

1.3.1 Идентификация по 2D–изображению

2D распознавание лица – по статистике, это один из самых неэффективных методов биометрии. Он появился очень давно и применялся, в основном, в криминалистике, что и способствовало его развитию. Затем появились компьютерные интерпретации метода, в результате чего он стал более надёжным, но, безусловно, уступал и с каждым годом все больше уступает другим биометрическим методам идентификации личности. В настоящее время из-за плохих статистических показателей он применяется в мультимодальной, или, как ее еще называют, перекрестной биометрии, или в социальных сетях.

Преимущества метода. В отличие от большинства биометрических методов, здесь не требуется дорогостоящее оборудование. При наличии необходимой аппаратуры возможно распознавание на значительных расстояниях от камеры.

Недостатки. Низкая статистическая достоверность. Предъявляются строгие требования к освещению. Для многих алгоритмов характерна неприемлемость каких-либо внешних помех, как, например, очки, борода, некоторые элементы причёски, закрывающие лицо. Обязательно изображение лица анфас, с весьма небольшими отклонениями. Также большинство алгоритмов не учитывают возможные изменения мимики лица, то есть выражение лица должно быть нейтральным. Изменения причёски, усы, борода, парики, грим – неприемлемы.

1.3.2 Идентификация по 3D–модели

Реализация данного метода представляет собой довольно сложную задачу. Но несмотря на это в наши дни существует множество методов по 3D распознаванию лица. Их невозможно сравнить друг с другом, так как используются различные сканеры и базы. Далеко не все из них выдают FAR и FRR и используются абсолютно различные подходы.

Переходным от 2D к 3D является метод, реализующий накопления информации по лицу. Он имеет лучшие характеристики, чем 2D–метод, но для его реализации также требуется всего лишь одна камера. При занесении субъекта в базу данных делается множество снимков лица под разными углами, и алгоритм соединяет изображения воедино, создавая 3D–шаблон. А при распознавании используется несколько кадров видеопотока. Однако пока этот метод является лишь экспериментальным.

Наиболее классическим является метод проецирования шаблона. Он состоит в том, что на объект (лицо) проецируется сетка. Далее камера с большой скоростью делает снимки, и затем специальная программа обрабатывает полученные изображения. Луч, падающий на искривленную поверхность, изгибается – чем больше кривизна поверхности, тем сильнее изгиб луча. Изначально при этом применялся источник видимого света, подаваемого через «жалюзи». Затем видимый свет заменили на инфракрасный, ввиду того, что он обладает рядом преимуществ. На первом этапе обработки отбрасываются изображения, на котором лица не видно вообще или присутствуют посторонние предметы, мешающие идентификации. По полученным снимкам восстанавливается 3D–модель лица, на которой выделяются и удаляются ненужные помехи (прическа, борода, усы и очки). Затем производится анализ модели – выделяются антропометрические особенности, которые в итоге записываются в уникальный код и заносятся в базу данных. Время захвата и обработки изображения составляет около двух секунд.

Преимущества метода. Отсутствие необходимости контактировать со сканирующим устройством. Низкая чувствительность к внешним факторам, как на самом человеке (очки, бороды, изменение прически), так и в его окружении (освещенность, поворот головы). Высокий уровень надежности.

Недостатки метода. Дороговизна оборудования. Изменения мимики лица и помехи на лице ухудшают статистическую надежность метода. Метод еще недостаточно хорошо разработан, особенно в сравнении с давно применяющейся дактилоскопией, что затрудняет его широкое применение.

2 МЕТОДЫ ОБНАРУЖЕНИЯ ОБЪЕКТОВ И ГЛУБОКОЕ ОБУЧЕНИЕ

2.1 Анализ методов обнаружения объектов

Задача обнаружения лица на изображении часто является первым шагом в процессе решения задачи более высокого уровня — распознавания лица, деталей лица или его мимики. Кроме того, информация о присутствии и количестве лиц на изображении может быть полезна в системах автоматического учета числа посетителей; системах пропускного контроля в учреждениях, аэропортах и метро; автоматических системах предотвращения несчастных случаев; интеллектуальных интерфейсах «человек–компьютер»; в фототехнике для автоматической фокусировки на лице человека, а также для стабилизации изображения лица с целью облегчения распознавания эмоций; для расширения зоны стереовидения при создании систем 3D отображения. Существующие алгоритмы обнаружения лиц можно разбить на четыре категории [61]:

- эмпирический метод;
- метод характерных инвариантных признаков;
- распознавание с помощью шаблонов, заданных разработчиком;
- метод обнаружения по внешним признакам, обучающиеся системы.

2.1.1 Эмпирический метод

Эмпирический подход предполагает создание алгоритма, реализующего набор правил, которым должен отвечать фрагмент изображения, для того чтобы быть признанным человеческим лицом. Этот набор правил является попыткой формализовать эмпирические знания о том, как именно выглядит человеческое лицо на изображениях, и чем руководствуется человек при принятии решения: лицо он видит или нет. Самые простые правила эмпирического подхода:

- центральная часть лица имеет однородную яркость и цвет;
- разница в яркости между центром и верхней частью лица значительна;
- лицо содержит в себе два симметрично расположенных глаза, нос и рот, резко отличающиеся по яркости относительно остальной части лица.

Метод сильного уменьшения изображения для сглаживания помех, а также для уменьшения вычислительных операций предварительно подвергает изображение сильному уменьшению. На таком изображении проще выявить зону равномерного распределения яркости (предполагаемая зона нахождения лица), а затем проверить наличие резко отличающихся по яркости областей внутри: именно такие области можно с разной долей вероятности отнести к «лицу».

Для определения областей изображения, где находится «лицо», метод построения гистограмм, строит вертикальную и горизонтальную гистограммы. В обнаруженных областях–кандидатах происходит поиск черт лица человека.

Данный подход использовался на заре развития компьютерного зрения ввиду малых требований к вычислительной мощности процессора для обработки изображения. Рассмотренные выше методы имеют неплохие показатели по выявлению лица на изображении при однородном фоне, они легко реализуемы с помощью машинного кода. Впоследствии было разработано множество подобных алгоритмов. Но эти методы оказались абсолютно непригодными для обработки изображений, содержащих большое количество лиц или сложный задний фон. Также они очень чувствительны к наклону и повороту головы.

2.1.2 Метод характерных инвариантных признаков

Методы характерных инвариантных признаков образуют второе семейство способов детектирования лиц. Здесь виден подход к проблеме с другой стороны: нет попытки в явном виде формализовать процессы, происходящие в человеческом мозге. Сторонники подхода стараются выявить закономерности и свойства изображения лица неявно, найти инвариантные особенности лица, независимо от угла наклона и положения. Основные этапы алгоритмов этой группы методов:

- детектирование на изображении явных признаков лица: глаз, носа, рта;
- обнаружение: границы лица, форма, яркость, текстура, цвет;
- объединение всех найденных инвариантных признаков и их верификация.

Метод обнаружения лиц в сложных сценах предполагает поиск правильных геометрических расположений черт лица. Для этого применяется гауссовский производный фильтр с множеством различных масштабов и ориентаций. После этого производится поиск соответствия выявленных черт и их взаимного расположения с помощью случайного перебора.

Суть *метода группировки признаков* в применении второй производной гауссовского фильтра для поиска интересующих областей изображения. Далее группируются края вокруг каждой такой области при помощи порогового фильтра. А затем используется оценка при помощи байесовской сети для комбинирования найденных признаков, таким образом происходит выборка черт лица. Методы этой группы в качестве достоинств имеют возможность распознавать лицо в различных положениях. Но даже при небольшом загромождении лица другими объектами, возникновении шумов или засветке процент достоверного распознавания сильно падает. Большое влияние также оказывает сложный задний фон изображения.

Основа рассмотренных подходов – эмпирика, которая одновременно является их сильной и слабой стороной. Большая изменчивость объекта распознавания, зависимость вида лица на изображении от условий съёмки и освещения, позволяют без колебаний отнести обнаружение лица на изображении к задачам высокой сложности. Применение эмпирических правил позволяет построить модель изображения лица, и свести задачу к выполнению некоторого количества относительно простых проверок изображения по шаблону в базе данных.

Однако, несмотря на безусловно разумную посылку – попытаться использовать и повторить уже успешно функционирующий инструмент распознавания — человеческое зрение, методы первой категории пока далеки по эффективности от своего прообраза, поскольку исследователи, решившие избрать этот путь, сталкиваются с рядом серьезных трудностей. *Во-первых*, процессы, происходящие в мозгу во время решения задачи распознавания изображений изучены далеко не полностью, и тот набор эмпирических знаний о человеческом лице, которые доступны исследователям на сознательном уровне, далеко не исчерпывает инструментарий, используемый мозгом подсознательно. *Во-вторых*, трудно эффективно перевести неформальный человеческий опыт и знания в набор формальных правил, поскольку чересчур жесткие рамки правил приведут к тому, что в ряде случаев лица не будут обнаружены, и, напротив, слишком общие правила приведут к большому количеству случаев ложного обнаружения.

2.1.3 Распознавание с помощью шаблонов

Шаблоны задают некий стандартный образ изображения лица, например, путем описания свойств отдельных областей лица и их возможного взаимного расположения. Обнаружение лица с помощью шаблона заключается в проверке каждой из областей изображения на соответствие заданному шаблону.

Особенности подхода: используются два вида шаблонов – недеформируемые и деформируемые; шаблоны заранее запрограммированы и необучаемы; используется корреляция для нахождения лица на изображении.

Метод детектирования лица при помощи *трёхмерных форм* предполагает использование шаблона в виде пар отношений яркостей в двух областях. Для определения местоположения лица необходимо пройти всё изображение на сравнение с заданным шаблоном. Причём делать это нужно с различным масштабом.

Модели распределения опорных точек являются статистическими моделями, которые представляют объекты, форма которых может измениться.

Полезная особенность метода — способность выделить форму переменных объектов в пределах учебного набора с небольшим количеством параметров формы. Эта компактная и точная параметризация может использоваться для разработки эффективных систем классификации.

К достоинствам распознавания с помощью шаблонов можно отнести относительную простоту реализации и неплохие результаты на изображениях с не очень сложным задним фоном. А главным недостатком является необходимость калибровки шаблона вблизи с изображением лица. Большая трудоёмкость вычисления шаблонов для различных ракурсов и поворотов лица ставят под вопрос целесообразность их использования [62].

2.1.4 Метод обнаружения объектов по внешним признакам

Изображению (или его фрагменту) ставится в соответствие вычисленный вектор признаков, который используется для классификации изображений на два класса — *лицо* / *не лицо*. Обычно поиск лиц на изображениях с помощью методов, основанных на построении математической модели изображения лица, заключается в полном переборе всех прямоугольных фрагментов изображения всевозможных размеров, и проведения проверки каждого из фрагментов — на наличие лица. Поскольку схема полного перебора обладает такими безусловными недостатками, как *избыточность* и *большая вычислительная сложность*, то применяются различные методы сокращения количества изучаемых фрагментов.

Основные принципы методов [63]:

- схоластика: каждый сканируется окном и представляется векторами ценности;
- блочная структура: полученное изображение разбивается на пересекающиеся или непересекающиеся участки различных масштабов, а затем производится оценка, с помощью алгоритмов оценки весов векторов.

Для обучения алгоритмов требуется библиотека вручную подготовленных изображений человеческих лиц и «не лиц», а также любых других изображений.

Важнейшей задачей является выделить сильные классификаторы. Именно они будут иметь наивысший приоритет для проверки найденных признаков в изображении. Количество же более слабых классификаторов стоит уменьшить за счёт похожести друг на друга, а также удалении классификаторов, возникших за счёт шумовых выбросов. Перечислим основные методики выполнения этих задач:

- искусственные нейронные сети;
- метод главных компонент (Principal Component Analysis (PCA));
- факторного анализа (Factor Analysis);

- линейный дискриминантный анализ (Linear Discriminant Analysis);
- метод опорных векторов (Support Vector Machines (SVM));
- наивный байесовский классификатор (Naive Bayes classifier);
- скрытые Марковские модели (Hidden Markov model);
- метод распределения (Distribution-based method);
- совмещение факторного анализа (ФА) и метода главных компонент;
- разреженная сеть окон (Sparse network of windows SNoW);
- активные модели (Active Appearance Models);
- адаптированное улучшение и основанный на нём Метод Виолы–Джонса и др.

Рассмотрим особенности некоторых из них. На сегодняшний день метод искусственных нейронных сетей является наиболее распространенным способом решения интеллектуальных задач распознавания лиц на изображении.

Искусственная нейронная сеть (ИНС) — это математическая модель, представляющая собой систему соединённых и взаимодействующих между собой нейронов. Нейронные сети не программируются в привычном смысле этого слова, они обучаются. Технически обучение заключается в нахождении коэффициентов связей (синапсов) между нейронами.

Метод опорных векторов (Support Vector Machines) применяется для снижения размерности пространства признаков, не приводя к существенной потере информативности тренировочного набора объектов. Применение метода главных компонент к набору векторов линейного пространства позволяет перейти к такому базису пространства, что основная дисперсия набора будет направлена вдоль нескольких первых осей базиса, называемых главными осями. Натянутое на полученные таким образом главные оси подпространство является оптимальным среди всех пространств, в том смысле, что наилучшим образом описывает тренировочный набор. Это набор алгоритмов, схожих с алгоритмами вида «обучение с учителем», использующихся для задач классификации и регрессионного анализа. Метод опорных векторов основан на поиске линейного разделения классов, он принадлежит к семейству линейных классификаторов.

Цель тренировки большинства классификаторов – минимизировать ошибку классификации на тренировочном наборе (называемую эмпирическим риском). В отличие от них, с помощью метода опорных векторов можно построить классификатор, минимизирующий верхнюю оценку ожидаемой ошибки классификации (в том числе и для неизвестных объектов, не входивших в тренировочный набор). Применение метода опорных векторов к задаче обнаружения лица заключается в поиске гиперплоскости в признаковом пространстве, отделяющей класс *изображений лиц* от *изображений «не-лиц»*.

Возможность линейного разделения столь сложных классов, как изображения лиц и «не–лиц» представляется маловероятной. Однако, классификация с помощью опорных векторов позволяет использовать аппарат ядерных функций для неявного проецирования векторов–признаков в пространство потенциально намного более высокой размерности (еще выше, чем пространство изображений), в котором классы могут оказаться линейно разделимы. Неявное проецирование с помощью ядерных функций не приводит к усложнению вычислений, что позволяет успешно использовать линейный классификатор для линейно неразделимых классов.

Сегодня самым перспективным, в плане высокой производительности и низкой частоты ложных срабатываний, а также обладающим большим процентом верных обнаружений лиц, выглядит *метод Виолы–Джонса*.

Основные принципы этого метода таковы:

- используются изображения в интегральном представлении, что позволяет вычислять быстро необходимые объекты;
- используются *признаки Хаара*, с помощью которых происходит поиск нужного объекта (в данном контексте, лица и его черт);
- используется *бустинг* (от англ. boost – улучшение, усиление) для выбора самых подходящих признаков для искомого объекта на данной части изображения;
- все признаки поступают на вход классификатора, который анализирует изображение и выдаёт результат – «истина» либо «ложь»;
- используются каскады признаков для быстрого отбрасывания окон, где не найдено изображение требуемого лица.

Обучение классификаторов идет очень медленно, но результаты поиска лица очень быстры, именно поэтому нужно заострить внимание на методе распознавания лиц на изображении. Алгоритм хорошо работает и распознает черты лица под небольшим углом, примерно до 30 градусов. При угле наклона больше 30 градусов процент обнаружений резко падает, что не позволяет в стандартной реализации детектировать повернутое под любым углом лицо человека, что в значительно затрудняет или делает невозможным использование алгоритма в современных производственных системах, с учётом их растущих потребностей.

В общем виде, задача обнаружения лица и черт лица человека на цифровом изображении выглядит так: имеется изображение, на котором есть искомые объекты. Оно представлено двумерной матрицей пикселей размером $w \times h$, в которой каждый пиксел имеет своё цифровое значение:

- от 0 до 255, если это черно–белое изображение;
- от 0 до 255^3 , если это цветное изображение (компоненты R, G, B).

В результате своей работы алгоритм должен определить лица и их черты, а также пометить их. Поиск осуществляется в активной области изображения прямоугольными признаками. С их помощью и описываются черты лица:

$$Rect_i = \{x, y, w, h, a\},$$

где x, y – координаты центра i -го прямоугольника, w – ширина, h – высота, a – угол наклона прямоугольника к вертикальной оси изображения.

Иными словами, применительно к рисункам и фотографиям используется подход на основе сканирующего окна (Scanning Window): сканируется изображение окном поиска (так называемое, окно сканирования), а затем применяется классификатор к каждому положению. Система обучения и выбора наиболее значимых признаков полностью автоматизирована и не требует вмешательства человека, поэтому данный подход работает быстро.

Чтобы производить какие-либо действия с данными, используется интегральное представление изображений по методу Виолы–Джонса. Оно позволяет быстро рассчитывать суммарную яркость произвольного прямоугольника на данном изображении, причем какой бы прямоугольник не был, время расчёта неизменно.

Интегральное представление изображения – это матрица, совпадающая по размерам с исходным изображением. В каждом элементе её хранится сумма интенсивностей всех пикселей, находящихся левее и выше данного элемента. Элементы матрицы рассчитываются по следующей формуле:

$$L(i, y) = \sum_{i=0, j=0}^{i \leq x \leq y} I(i, j),$$

где $I(i, j)$ – яркость пиксела исходного изображения. Каждый элемент матрицы $L(x, y)$ представляет собой сумму пикселей в прямоугольнике от (0,0) до (x,y).

В стандартном методе Виолы – Джонса используются прямоугольные признаки (рисунок 2.1), которые называются *хаароподобными вейвлетами*:

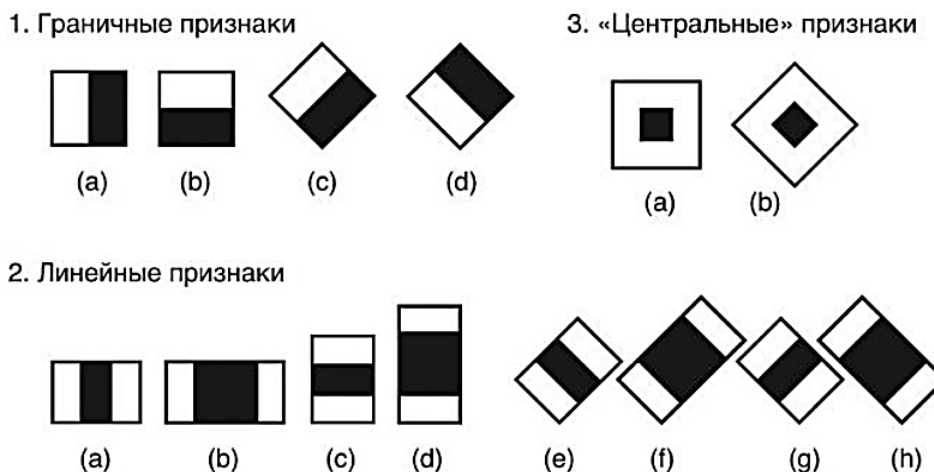


Рисунок 2.1 – Прямоугольные хаароподобные признаки

Вычисляемым значением такого признака будет $F = X - Y$, где X – сумма значений яркостей точек закрываемых светлой частью признака, а Y – сумма значений яркостей точек закрываемых тёмной частью признака. Для их вычисления используется понятие интегрального изображения, рассмотренное выше. Признаки Хаара дают точечное значение перепада яркости по оси X и Y соответственно.

Алгоритм сканирования окна с признаками выглядит так: есть исследуемое изображение, выбрано окно сканирования, выбраны используемые признаки. Далее окно сканирования начинает последовательно двигаться по изображению с шагом в 1 ячейку окна (допустим, размер самого окна есть 24×24 ячейки).

Сканирование производится последовательно для различных масштабов, причём масштабируется не само изображение, а сканирующее окно (т.е. изменяется размер ячейки). Все найденные признаки, соответствующие выбранным заранее, попадают к классификатору, который и «выносит окончательный вердикт».

В процессе поиска вычислять все признаки на маломощных настольных компьютерах просто невозможно и нереально. Следовательно, классификатор должен реагировать только на определенное, необходимое подмножество всех признаков. С точки зрения логики понятно, что классификатор нужно обучить нахождению лиц по определенному, заранее установленному подмножеству. Это можно сделать, только обучая вычислительную машину автоматически.

В контексте алгоритма, имеется множество объектов (изображений), разделённых некоторым образом на классы. Задано конечное множество изображений, для которых известно, к какому классу они относятся (к примеру, это может быть класс «фронтальное положение носа»). Это множество называется *обучающей выборкой*. Классовая принадлежность остальных объектов не известна.

Требуется построить алгоритм, способный классифицировать произвольный объект из исходного множества. Для решения проблемы такого, столь сложного обучения, существует технология *бустинга* – комплекса методов, способствующих повышению точности аналитических моделей.

Эффективная модель, допускающая мало ошибок классификации, называется «сильной». «Слабая» же, напротив, не позволяет надёжно разделять классы или давать точные предсказания, делает в работе большое количество ошибок.

Алгоритм бустинга для поиска лиц таков:

1. Определение слабых классификаторов по прямоугольным признакам;
2. Для каждого перемещения сканирующего окна вычисляется прямоугольный признак на каждом примере;
3. Выбирается наиболее подходящий порог для каждого признака;
4. Отбираются лучшие признаки и лучший подходящий для них порог;
5. «Перевзвешивается» выборка.

Каскадная модель сильных классификаторов – это, по сути, то же дерево принятия решений, где каждый узел дерева построен таким образом, чтобы детектировать почти все интересующие образы и отклонять регионы, не являющиеся образами. Помимо этого, узлы дерева размещены таким образом, что чем ближе узел находится к корню дерева, тем из меньшего количества примитивов он состоит, и тем самым требует меньшего времени на принятие решения. Данный вид каскадной модели хорошо подходит для обработки изображений, на которых общее количество детектируемых образов сравнительно мало. В этом случае алгоритм может быстрее принять решение о том, что данный регион не содержит образ, и перейти к следующему.

Стоит отметить, что в том случае, если на вход системе обнаружения подаётся цветное изображение, то можно существенно увеличить скорость работы алгоритма, обрабатывая изображение предварительно, при помощи цветового кодирования. Кроме этого, использование цветового кодирования помогает уменьшить число ложных срабатываний [64].

На сегодня, алгоритм Виолы–Джонса является самым востребованным, ввиду своей высокой скорости работы и высокой точности срабатывания.

2.2 Глубокое и машинное обучение

Глубокое обучение – одна из самых влиятельных и быстрорастущих областей в искусственном интеллекте. Однако получить интуитивное понимание глубокого обучения довольно сложно, поскольку термин «глубокое обучение» охватывает множество различных алгоритмов и методов. Глубокое обучение также является вспомогательной дисциплиной машинного обучения в целом, поэтому прежде всего нужно понять, что такое машинное обучение.

Проще говоря, машинное обучение – это метод, позволяющий компьютерам выполнять конкретные задачи без явного кодирования каждой строки алгоритма, используемого для выполнения конкретной задачи. Существует множество различных алгоритмов машинного обучения, но одним из наиболее часто используемых алгоритмов является *многослойный персептрон*.

Многослойный персептрон также называют *нейронной сетью*, и он состоит из серии узлов / нейронов, соединенных вместе. В многослойном персептроне есть три разных слоя: входной слой, скрытый слой и выходной слой.

Входной слой переносит данные в сеть, где ими манипулируют узлы в среднем / скрытом слое. Узлы в скрытом слое – это математические функции, которые могут манипулировать данными, поступающими с входного слоя, извлекая соответствующие шаблоны из входных данных.

Так нейронная сеть «учится». Нейронные сети получили свое название от того факта, что они вдохновлены структурой и функциями человеческого мозга.

Соединения между узлами в сети имеют значения, называемые *весами*. Эти значения, по сути, являются предположениями о том, как данные в одном слое связаны с данными в следующем слое. Когда сеть обучается, веса корректируются, и цель состоит в том, чтобы веса / предположения о данных, в конечном итоге, сошлись на значениях, которые точно представляют значимые шаблоны в данных.

Функции активации присутствуют в узлах сети, и они преобразуют данные нелинейным образом, позволяя сети изучать сложные представления данных. Они умножают входные значения на значения веса, и добавляют член смещения.

Глубокое обучение – это термин, присваиваемый архитектурам машинного обучения, которые объединяют многие многослойные персептроны, так что существует не один скрытый слой, а много скрытых слоев. Чем «глубже» глубокая нейронная сеть, тем более сложные паттерны она может выучить.

Сети глубоких слоев, состоящие из нейронов, называются полностью связанными сетями, или полностью связанными слоями, ссылаясь на тот факт, что данный нейрон поддерживает соединение со всеми окружающими его нейронами. Полностью связанные сети могут быть объединены с другими функциями машинного обучения, для создания различных архитектур глубокого обучения.

2.2.1 *Среда для глубокого обучения Torch*

Torch представляет собой научную вычислительную среду с широкой поддержкой алгоритмов машинного обучения, в которой упор сделан на графические процессоры (GPU). Она отличается удобством в использовании и высоким быстродействием, благодаря простому и быстрому языку скриптов, LuaJIT и базовой программе на основе C / CUDA.

Важные особенности среды. Она основывается на языке LuaJIT с высокой степенью оптимизации, и обеспечивает доступ к ресурсам низкого уровня, например, для работы с обычными указателями языка C, и стремится обеспечить максимальную универсальность и скорость при создании научных алгоритмов, в то же время упрощая процесс. Имеет большую экосистему пакетов, в том числе в области машинного обучения, машинного зрения, обработки сигналов, параллельной обработки, изображений, видео, аудио и сетей, которые развиваются по инициативе и на основе поддержки сообщества Lua. Это простые в использовании библиотеки популярных нейронных сетей и средств оптимизации, отличающиеся максимальной универсальностью при реализации сложных топологий нейронных сетей. Вся среда (включая Lua) является автономной, и её можно перенести на любую платформу без изменений.

2.2.2 *Среда для глубокого обучения Caffe*

Caffe представляет собой среду глубокого обучения, которая создавалась с учетом выражения, скорости и модульного принципа. Она была создана компанией Berkeley AI Research (BAIR) при участии членов сообщества. Янгквин Джиа (Yangqing Jia) создал проект во время своей работы над докторской диссертацией в Калифорнийском университете в Беркли. Данная платформа включает в себя предустановленные наборы обучаемых нейронных сетей и также стал известен своими возможностями обработки изображений поддержкой пакета прикладного ПО MATLAB.

Важные особенности среды:

- сравнительно быстрая работа при визуальной обработке изображений;
- выразительная архитектура, способствующая инновационным разработкам;
- расширяемый код, стимулирующий активные разработки;
- высокая скорость – *Caffe* предлагает один из самых быстрых вариантов внедрения свёрточных нейронных сетей (CNN).

Используется для научно–исследовательских проектов, прототипов стартапов и масштабных приложений в области компьютерного зрения, речи и мультимедиа.

2.2.3 *Среда для глубокого обучения TensorFlow*

TensorFlow – это популярный инструмент для обучения нейросетей, разработанный компанией Google, которые активно использует собственный фреймворк глубокого обучения для таких крупномасштабных сервисов, как Gmail и Google Translate. TensorFlow уже применили к своим сервисам и такие значительные бренды как Uber, Airbnb, Dropbox и многие другие. Он оказал огромное влияние на разработку интеллектуальных нейросетевых систем.

Наиболее удобным клиентским языком работы с TensorFlow является Python, но доступны и экспериментальные интерфейсы на JavaScript, C++, Java и Go. Сообщество Open source разработало также решения для C# и Julia.

Следствием популярности фреймворка стала подробная документация, включающая не только версию на официальном сайте, но и сторонние источники. Документация описывает как структуру фреймворка TensorFlow, так и инструменты, позволяющие отследить сам процесс обучения нейронной сети, например, визуализация с TensorBoard. Важно, что проект учитывает не только мощные вычислительные кластеры, но и возможность запуска моделей на мобильных платформах, как iOS и Android.

Тем не менее, TensorFlow – это низкоуровневый инструмент. Необходимо тщательно продумывать архитектуру нейронной сети, правильно оценивать размерность и объемы входных и выходных данных. Таким образом, работа с TensorFlow требует написания значительного количества программного кода.

TensorFlow оперирует статическим вычислительным графом. То есть вначале мы определяем граф, далее запускаем вычисления и, если необходимо внести изменения в архитектуру, заново обучаем модель.

Такой подход выбран ради эффективности, но многие современные нейросетевые инструменты умеют учитывать уточнения в процессе обучения без существенной потери скорости обучения. В этом плане основным конкурентом TensorFlow является фреймворк глубокого обучения PyTorch.

Поскольку у нас нет большого набора данных (например, 6000 изображений лиц) для обучения модели нашего проекта была использована предварительно обученная модель. Все выбранные среды поддерживают парки моделей. Кроме того, существуют преобразователи, позволяющие использовать модели, обученные с помощью другой библиотеки. Библиотека Caffe первой вошла в парк моделей и имеет самый широкий выбор моделей.

Выводы

В этом разделе проанализированы методы обнаружения объектов, связанных с задачами распознавания человеческих лиц. Дана классификация современных алгоритмов обнаружения лиц, в том числе: эмпирический метод, метод характерных инвариантных признаков, метод распознавания с помощью шаблонов, метод обнаружения объектов по внешним признакам, среди которых самым перспективным в плане высокой производительности и низкой частоты ложных срабатываний является метод Виолы–Джонса. Охарактеризован метод глубокого обучения – один из влиятельных и быстрорастущих областей искусственного интеллекта, изучены научные вычислительные среды с широкой поддержкой алгоритмов машинного обучения – Torch, Caffe, TensorFlow, и др.

Результаты, полученные в этом разделе, получают дальнейшее развитие в процессе программного конструирования и разработки программного приложения распознавания человеческих лиц в кадре с целью авторизации пользователей информационной системы с помощью видеокамеры.

3 РАСПОЗНАВАНИЕ ЛИЦ

Распознавание лиц является несложной задачей для людей. Эксперименты показали, что даже дети от одного до трёх дней от роду способны различать знакомые им лица. А насколько сложно это может быть для компьютера? Оказывается, что на сегодняшний день вообще мало известно о человеческом распознавании объектов (лиц). Используются ли внутренние элементы (глаза, нос, рот) или внешние элементы (форма головы, линия роста волос) для успешного распознавания лица? Как сами люди анализируют изображение, и как его кодирует человеческий мозг? Учёные Дэвид Хьюбел и Торстен Визел показали, что в нашем мозге есть специализированные нервные клетки, реагирующие на специфические локальные особенности сцены, такие, как линии, края, углы или движение.

Поскольку люди не рассматривают окружающий мир как разрозненные части, зрительная кора мозга должна, каким-то образом, объединять различные источники информации в нужные шаблоны. Автоматическое распознавание человеческих лиц сводится к извлечению этих значимых функций из изображения, их полезному представлению и выполнению их некоторой классификации.

Распознавание лиц на основе геометрических особенностей лица, вероятно, является наиболее интуитивным подходом к распознаванию лиц. Одна из первых автоматизированных систем распознавания лиц использовала точки маркера (положение глаз, ушей, носа и т.д.) для построения вектора объектов (расстояние между точками, угол между ними и т.д.). Распознавания проводили путём расчёта евклидова расстояния между векторами признаков зонда и эталонным изображением. Такой метод устойчив по отношению к изменениям освещения по своей природе, но имеет огромный недостаток: точная регистрация точек маркера затруднена, даже при использовании современных алгоритмов.

Некоторые из последних работ по геометрическому распознаванию лиц использовали 22-мерный вектор признаков, и эксперименты с большими наборами данных показали, что одни только геометрические объекты не несут достаточно информации для надёжного распознавания лиц.

Метод Eigenfaces использует целостный подход к распознаванию лиц: изображение лица – это точка из многомерного пространства изображений, и можно найти представление более низкой размерности, где классификация становится легкой. Подпространство нижнего измерения находится с анализом главных компонентов, который определяет оси с максимальной дисперсией. Хотя этот вид преобразования является оптимальным с точки зрения реконструкции изображения, он не учитывает метки классов, а также имеет и другие недостатки.

Возможна ситуация, когда дисперсия генерируется из внешних источников. Например, это освещение. Оси с максимальной дисперсией не обязательно содержат какую-либо дискриминационную информацию, поэтому классификация становится невозможной. Поэтому для распознавания человеческих лиц применяется специфическая для класса проекция с линейным дискриминантным анализом. Основная идея этого метода состоит в том, чтобы минимизировать дисперсию в классе, одновременно максимизируя дисперсию между классами [65].

3.1 Библиотеки открытого доступа

Существует несколько библиотек открытого доступа, которые можно использовать в обнаружении и распознавании объектов и лиц. Наиболее распространенные библиотеки – это OpenCV и Dlib.

Библиотека OpenCV (Open Source Computer Vision) – это библиотека компьютерного зрения с открытым исходным кодом) – библиотека алгоритмов компьютерного зрения, обработки изображений и численных алгоритмов общего назначения с открытым кодом. Реализована на C/C++ и Python, но также имеет кроссплатформенную реализацию для C#, которая называется EmguCV.

Dlib – это кроссплатформенная библиотека программного обеспечения общего назначения, написанная на языке программирования C++, также имеет реализацию для C# под названием DlibDotNet. Это программное обеспечение с открытым исходным кодом, выпущенное по лицензии Boost Software, включает в себя огромный функционал, в который входит машинное обучение и обработка изображений.

3.1.1 Анализ библиотеки DlibDotNet

В библиотеке имеются две модели обнаружения человеческих лиц:

- HoG Face Detector (Гистограмма направленных градиентов);
- Deep Learning Based Face Detector (Обнаружение с помощью глубокого обучения).

Первая модель обнаружения лиц основана на функциях HoG и SVM и состоит из 5 фильтров HOG – вид спереди, вид слева, вид справа, вид спереди, но повернутый влево, и вид спереди, но повернутый вправо. Выходные данные представляются в виде списка граней с координатами (x, y) диагональных углов.

Достоинства данной модели:

- самый быстрый метод, функционирующий на процессоре;
- работает хорошо для фронтальных и слегка нефронтальных граней;
- облегченная модель по сравнению с остальными тремя;
- работает при небольшой окклюзии (временном сокрытии части изображения).

Недостатки:

- модель не способна обнаружить маленькие лица, поскольку обучена для минимального размера лица 80×80 пикселей. Необходимо убедиться, что размер человеческого лица на изображении больше этих значений. Однако, можно обучить свой собственный детектор лица для изображений лиц меньшего размера.
- ограничительная рамка часто исключает часть лба и даже часть подбородка;
- не очень хорошо работает при значительной окклюзии (временном сокрытии);
- не работает для боковой грани и экстремальных нефронтальных граней, например, в случае, если взгляд направлен вниз или вверх.

Вторая модель обнаружения человеческих лиц основана на использовании метода Maximum–Margin Object Detector (MMOD) с функциями на основе CNN. Процесс обучения для этого метода очень прост, не требуется большое количество данных для обучения детектора пользовательских объектов.

Плюсы данной модели:

- работает для разных ориентаций лица;
- устойчив к окклюзии (временном сокрытии части изображения);
- очень быстро работает на GPU;
- очень легкий тренировочный процесс;

Недостатки:

- очень медленно работает на процессоре;
- не обнаруживает маленькие лица, так как тренируется для минимального размера лица 80×80 пикселей. Однако, возможно обучить свой собственный детектор лица для изображений лиц меньшего размера;
- ограничительная рамка даже меньше, чем у детектора HoG.

3.1.2 Анализ библиотеки OpenCV 4.1

В библиотеке есть также две основных модели обнаружения лиц:

- Haar Cascade Face Detector (обнаружение при помощи признаков Хаара);
- Dnn Face Detector (обнаружение при помощи глубокого обучения);

Детектор лица на основе каскадов Хаара был самым современным алгоритмом в области обнаружения изображений лиц в течение многих лет, с 2001 г., когда он был впервые представлен Паулем Виолой и Майклом Джонсом. В последние годы было много усовершенствований данного метода, и поэтому библиотека OpenCV имеет достаточное количество моделей в своей коллекции.

Плюсы данной модели:

- работает на процессоре практически в режиме реального времени;
- имеет сравнительно простую архитектуру;
- обнаруживает лица в разных масштабах.

Недостатки:

- метод дает много ложных предсказаний, а значит, не слишком надёжен;
- не работает с нефронтальными изображениями и их фрагментами;
- не работает при окклюзии (временном сокрытии части изображения).

Вторая модель OpenCVDnn была включена в OpenCV с версии 3.3. Она основана на детекторе Single-Shot-Multibox и использует архитектуру ResNet-10 в качестве магистрали. Модель обучена с использованием изображений человеческих лиц, доступных в Интернете, но источник не разглашается [66].

OpenCV предоставляет две модели для такого детектора человеческого лица:

- версия с плавающей запятой 16 оригинальной реализации Caffe;
- 8-битная квантованная версия с использованием Tensorflow.

В данной модели изображение преобразуется в большой двоичный объект и передается обученной модели с помощью функции `forward()`. Выход обнаружений представляет собой 4D-матрицу, где третье измерение перебирает обнаруженные лица. Четвертое измерение содержит информацию о ограничительной рамке и счёт для каждого лица. Например, обнаружения $[0, 0, 0, 2]$ дают оценку достоверности для первого лица, а обнаружения $[0, 0, 0, 3:6]$ дают ограничивающую рамку.

Выходные координаты ограничительной рамки нормированы между $[0, 1]$. Координаты должны быть умножены на высоту и ширину исходного изображения, чтобы получить правильную ограничивающую рамку на изображении.

Метод имеет следующие достоинства:

- наиболее точный из четырех методов;
- работает в режиме реального времени на процессоре;
- работает для различных ориентаций лица – вверх, вниз, влево, вправо, под любым углом, в профиль, на лицевой стороне и т. д.;
- работает даже при существенной окклюзии;
- обнаруживает лица в различных масштабах (и большие, и крошечные лица);

Детектор на основе DNN преодолевает все недостатки детектора на основе каскада Хаара, без ущерба для любых преимуществ, предоставляемых Хааром.

3.1.3 Сравнительный анализ моделей из библиотек Dlib и OpenCv

Выявлены плюсы и минусы каждого метода, сделаны следующие заключения:

В большинстве случаев, размер лица на изображении заранее неизвестен. Значит, лучше использовать метод OpenCV-DNN, так как он обладает достаточным быстродействием и очень точен, даже для граней небольшого размера. Он способен обнаруживать лица под разными углами.

Для средних и больших размеров изображения модель Dlib HoG является самым быстроедействующим методом на процессоре. Но он не обнаруживает лица малого размера (меньше, чем 70 x 70 пикселей). Если заранее известно, что приложение не будет иметь дело с лицами очень маленьких размеров, тогда детектор лиц на основе HoG – лучший вариант. Если будет использоваться графический процессор, то детектор человеческого лица MMOD является лучшим вариантом, так как на графическом процессоре он обладает достаточно высоким быстроедействием и обеспечивает обнаружение под различными углами.

Для изображений с высоким разрешением, детекторы HoG и MMOD могут не работать при уменьшении изображения. Но зато для этого можно использовать метод OpenCV–DNN, так как он обнаруживает лица малого размера.

Проанализировав каждую из моделей, было принято решение использовать модель OpenCv Dnn для обнаружения лиц в кадре в реальном времени, а также алгоритмы распознавания лица EigenFaceRecognizer, FisherFaceRecognizer и LBPHFaceRecognizer, реализация которых имеется в данной библиотеке.

3.2 Алгоритм EigenFaceRecognizer

Проблема с представлением изображения, которое нам дано, заключается в его высокой размерности. Двумерные изображения в градациях серого $p \times q$ охватывают векторное пространство $m = p \times q$, поэтому изображение с разрешением 100×100 пикселей уже находится в пространстве изображений размером 10 000. *Вопрос:* все ли измерения одинаково полезны для нас? Мы можем принять решение, только если есть какие-либо расхождения в данных, поэтому мы ищем компоненты, составляющие большую часть информации.

Метод анализа основных компонентов (РСА) был независимо предложен Карлом Пирсоном в 1901 г. и Гарольдом Хотеллингом в 1933 г. Идея состоит в том, что многомерный набор данных часто описывается коррелированными переменными, и поэтому большую часть информации составляют лишь несколько значимых измерений. Поэтому было предложено превратить набор возможных коррелированных переменных в меньший набор некоррелированных переменных.

Метод РСА находит направления, называемые основными компонентами, с наибольшей дисперсией в данных, а затем использует их в алгоритме.

Описание алгоритма:

Пусть $X = \{x_1, x_2, \dots, x_n\}$ – случайный вектор с наблюдениями $x_i \in R^d$.

1. Вычислить среднее μ : $\mu = \frac{1}{n} \sum_{i=1}^n x_i$.

2. Вычислить ковариационную матрицу S : $S = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)(x_i - \mu)^T$.

3. Вычислить собственные значения λ_i и собственные векторы v_i из S :

$$Sv_i = \lambda_i v_i, \quad i=1, 2, \dots, n.$$

4. Вычислить порядок собственных векторов по убыванию их собственных значений. K главных компонент являются собственными векторами, соответствующими k наибольшим собственным значениям.

Тогда k главных компонент наблюдаемого вектора x определяются как:

$$y = W^T (x - \mu), \quad \text{где } W = (v_1, v_2, \dots, v_k).$$

Реконструкция от основания СПС (собственные переменные системы) дается как:

$$x = W y + \mu, \quad \text{где } W = (v_1, v_2, \dots, v_k).$$

Затем метод Eigenfaces выполняет распознавание человеческого лица:

- Проецирование всех обучающих образцов в подпространство PCA.
- Проецирование изображения запроса в подпространство PCA.
- Нахождение ближайшего соседа – между проецируемыми обучающими изображениями и проецируемым изображением запроса.

Вместе с тем, остается одна нерешённая проблема. Пусть дано 400 изображений размером 100×100 пикселей. Анализ главных компонент решает ковариационную матрицу $S = X X^T$, где $\text{size}(X) = 10\,000 \times 400$, в нашем примере. В результате получается матрица $10\,000 \times 10\,000$, то есть, примерно 0,8 ГБ. Решение этой проблемы неосуществимо, и поэтому нужно применить хитрость.

Из линейной алгебры известно, что матрица $M \times N$ с $M > N$ может иметь только $N - 1$ ненулевых собственных значений. Значит, вместо этого можно взять разложение по собственным значениям $S = X^T X$ размера $N \times N$:

$$X^T X v_i = \lambda_i v_i,$$

и в результате вычислений получить исходные собственные векторы $S = X X^T$, взятых с левым умножением матрицы данных:

$$X X^T (X v_i) = \lambda_i (X v_i).$$

Полученные собственные векторы ортогональны. Их нужно нормализовать на единицу длины, чтобы получить ортонормированные собственные векторы.

3.3 Алгоритм FisherFaceRecognizer

Анализ основных компонентов (РСА), который является ядром метода Eigenfaces, находит линейную комбинацию функций, которая максимизирует общую дисперсию данных. Хотя это, несомненно, мощный способ представления данных, он не учитывает какие-либо классы, и поэтому при отбрасывании компонентов может быть потеряно много дискриминационной информации.

Возможна ситуация, когда дисперсия в данных генерируется внешним источником, например, светом. Компоненты, идентифицированные РСА, не всегда содержат какую-либо дискриминационную информацию. Спроецированные образцы, поэтому, смазываются вместе, что и делает классификацию невозможной.

Линейный дискриминантный анализ выполняет уменьшение размерности в зависимости от класса. Он изобретен британским статистиком Р.А. Фишером, который успешно использовал его для классификации цветов в своей статье «Использование нескольких измерений в таксономических задачах» 1936 г. Чтобы найти комбинацию признаков, которая лучше всего разделяет классы, линейный дискриминантный анализ максимизирует соотношение между классами к разбросу внутри классов, а не максимизирует общий разброс. Идея проста: одни и те же классы должны плотно объединяться, в то время как разные классы находятся как можно дальше друг от друга в представлении более низкой размерности. Это также было признано и другими учёными, которые применяли дискриминантный анализ для распознавания лиц в своих исследованиях.

Описание алгоритма:

Пусть X – случайный вектор с выборками из классов c :

$$X = \{X_1, X_2, \dots, X_c\}$$

$$X_i = \{x_1, x_2, \dots, x_n\}.$$

Матрицы рассеяния S_B и S_W рассчитываются как:

$$S_B = \sum_{i=1}^c N_i (\mu_i - \mu)(\mu_i - \mu)^T;$$

$$S_W = \sum_{i=1}^c \sum_{x_i \in X_i} (x_i - \mu_i)(x_i - \mu_i)^T;$$

где μ – общее среднее значение: $\mu = \frac{1}{N} \sum_{i=1}^N x_i$;

а μ_i – это среднее значение класса $i \in \{1, \dots, c\}$: $\mu_i = \frac{1}{|X_i|} \sum_{x_i \in X_i} x_i$.

Классический алгоритм Р.А. Фишера теперь ищет проекцию W , которая максимизирует критерий отделимости классов:

$$W_{opt} = \arg \max_W \frac{|W^T S_B W|}{|W^T S_W W|}.$$

Решение этой оптимизационной задачи дается путём решения общей задачи

$$S_B v_i = \lambda_i S_W v_i;$$

на собственные значения: $S_W^{-1} S_B v_i = \lambda_i v_i$.

Осталась решить одну проблему: ранг S_W не более $(N-c)$ для N выборок и c классов. В задачах распознавания образов число выборок N почти всегда меньше размера входных данных (количество пикселей), поэтому матрица рассеяния S_W становится сингулярной. Такое решение было найдено путём выполнения анализа основных компонентов данных, и проецирования выборок в $(N-c)$ – мерное пространство. Затем был проведен линейный дискриминантный анализ сокращённых данных, поскольку матрица S_W больше не являлась единичной.

Задача оптимизации может быть переписана следующим образом:

$$W_{pca} = \arg \max_W |W^T S_T W|;$$

$$W_{pca} = \arg \max_W \frac{|W^T W_{pca}^T S_B W_{pca} W|}{|W^T W_{pca}^T S_W W_{pca} W|}.$$

Матрица преобразования W , которая проецирует выборку в $(c-1)$ – мерном пространстве, определяется следующим образом: $W = W_{fld}^T W_{pca}^T$.

Метод Fisherfaces изучает специфическую для класса матрицу преобразования, поэтому они не воспринимают освещение так же очевидно, как метод *Eigenfaces*. Вместо этого дискриминантный анализ обнаруживает черты лица, чтобы различать людей. Важно отметить, что производительность метода *Fisherfaces* сильно зависит от входных данных. Если *Fisherfaces* используется только для хорошо освещённых изображений, но этот алгоритм пытается распознать лица в плохо освещённых сценах, то метод *Fisherfaces*, скорее всего, найдет неправильные компоненты (потому что его функции не могут быть преобладающими на плохо освещённых изображениях). Это логично и неизбежно, так как метод не мог изучить освещение.

3.4 Алгоритм LBPHFaceRecognizer

Алгоритмы, применяемые в методах Eigenfaces и Fisherfaces, используют целостный подход к распознаванию лиц. Данные рассматриваются как вектор, где-то в многомерном пространстве изображения. Большая размерность всегда плохо сказывается на быстродействии, и поэтому идентифицируется низко-размерное подпространство, где (вероятно) сохраняется полезная информация.

Подход Eigenfaces максимизирует общий разброс, который может привести к проблемам, если дисперсия генерируется внешним источником, потому что компоненты с максимальной дисперсией по всем классам не всегда полезны для классификации. Поэтому, чтобы сохранить некоторую дискриминационную информацию, применяется линейный дискриминантный анализ, а затем он оптимизируется, как и описано в методе Fisherfaces.

Method Fisherfaces отлично работал, по крайней мере, для ограниченного сценария, который предполагался в нашей модели. Однако настоящая реальность далека от математической идеализации. Невозможно гарантировать идеальные настройки освещения на 10 различных фотографиях человека. А что же говорить, если для каждого человека есть только одно изображение? Поэтому наши ковариационные оценки для подпространства могут быть в корне неверными, как и распознавание человеческого лица. Чтобы получить хорошие показатели распознавания, понадобится как минимум 8 (± 1) изображений для каждого человека, и метод Fisherfaces здесь может не слишком помочь.

Поэтому некоторые исследования были сосредоточены на извлечении из изображений местных особенностей. Идея состоит в том, чтобы не рассматривать всё изображение как многомерный вектор, а описывать только локальные особенности объекта. Функции, которые извлекаются таким образом, будут неявно иметь низкую размерность. Идея правильна, но станет очевидным, что изображение, получаемое на выходе, страдает не только от изменения освещенности. Нужно подумать о таких вещах, как масштабирование, перенос или поворот изображений, ведь локальное описание должно быть хотя бы немного устойчивым к этим вещам.

Методология Local Binary Patterns берет свое начало в 2D-текстурном анализе. Основная идея состоит в том, чтобы суммировать локальную структуру в изображении, сравнивая каждый пиксел с его окрестностями. Суть состоит в том, чтобы взять пиксел в качестве центра, и ограничить его соседей. Если интенсивность центрального пиксела больше либо равна интенсивности его соседа, то нужно обозначить его 1, и 0, в противном случае. В конечном итоге получается двоичное число для каждого пиксела, например, 11001111. Так, с 8 окружающими пикселями получится 2^8 возможных комбинаций, называемых локальными двоичными образцами, или кодами LBP (Local Binary Patterns).

Описание алгоритма:

Более формальное описание оператора LBP может быть дано как:

$$LBP(x_c, y_c) = \sum_{p=0}^{p-1} 2^p s(i_p - i_c),$$

где (x_c, y_c) – центральный пиксел с интенсивностью i_p ; i_c – определяющая интенсивность соседнего пиксела; s – это знаковая функция, определяемая как:

$$s(x) = \begin{cases} 1, & \text{if } x \geq 0; \\ 0, & \text{else} \end{cases}.$$

Это описание позволяет захватывать очень мелкие зернистые детали на изображениях. Фактически, авторы смогли конкурировать с современными результатами для классификации текстур. Вскоре после того, как оператор был опубликован, было отмечено, что фиксированная окрестность не в состоянии кодировать детали, различающиеся по масштабу. Таким образом, оператор был расширен для использования переменной окрестности. Идея состоит в том, чтобы выровнять неограниченное количество соседей по кругу с переменным радиусом, что позволяет захватывать следующие окрестности (рисунок 3.1):

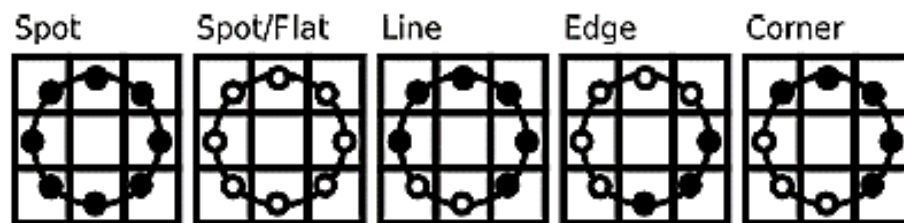


Рисунок 3.1 – Наглядный пример работы алгоритма

Для точки (x_c, y_c) положение соседа (x_p, y_p) , $p \in P$, может быть рассчитано как:

$$x_p = x_c + R \cos\left(\frac{2\pi p}{P}\right);$$

$$y_p = y_c - R \sin\left(\frac{2\pi p}{P}\right),$$

где R – радиус круга, а P – количество точек выборки.

Оператор является расширением исходных кодов LBP, поэтому его иногда называют расширенным LBP (также называемым циклическим LBP). Если координата точки на окружности не соответствует координатам изображения, точка получится интерполированной. В современных информационных технологиях насчитывается много различных схем интерполяции, но реализация OpenCV выполняет билинейную интерполяцию [67]:

$$f(x, y) \approx [1-x, x] \begin{bmatrix} f(0,0) & f(0,1) \\ f(1,0) & f(1,1) \end{bmatrix} \begin{bmatrix} 1-y \\ y \end{bmatrix}.$$

По определению, оператор LBP устойчив к монотонным преобразованиям серой шкалы. Можно легко убедиться в этом, посмотрев на изображение искусственно модифицированного LBP (рисунок 3.2):

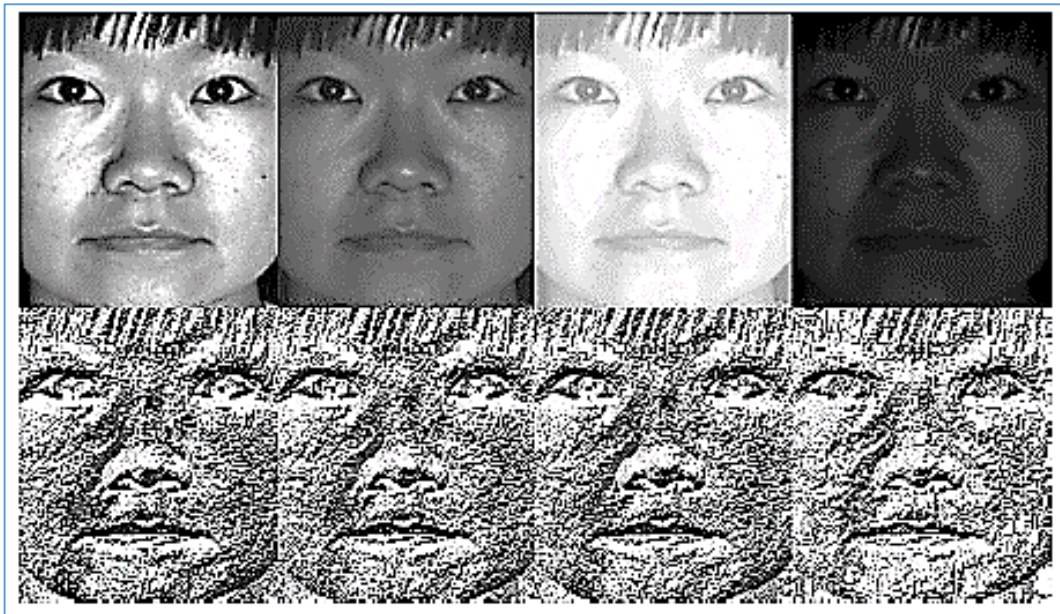


Рисунок 3.2 – Наглядная устойчивость метода к освещению

В модель распознавания лиц осталось включить пространственную информацию. Такое представление должно разделить изображение LBP на m локальных областей и извлечь гистограмму из каждого. Затем пространственно улучшенный вектор признаков получается путём конкатенации локальных гистограмм (без их объединения). Эти гистограммы называются Local Binary Patterns Histograms или гистограммы локальных бинарных образцов (паттернов).

Выводы

В этом разделе проанализированы распознавания человеческих лиц. Изучены метод *Eigenfaces*, использующий целостный подход к распознаванию лиц. Рассмотрены библиотеки открытого доступа *OpenCV* и *Dlib*, проведён сравнительный анализ используемых в них моделей. Исследованы алгоритмы:

- *EigenFaceRecognizer* и метод анализа основных компонентов (PCA),
- *FisherFaceRecognizer* и метод линейного дискриминантного анализа (LDA),
- *LBPHFaceRecognizer* и методология локальных двоичных образцов (LBP).

Результаты, полученные в этом разделе, получают дальнейшее развитие в процессе разработки структуры программного средства, программного конструирования и кодирования программного приложения для распознавания человеческих лиц в кадре для авторизации пользователей с помощью видеокамеры.

4 СТРУКТУРА ПРОГРАММНОГО СРЕДСТВА

Разрабатываемое программное средство будет состоять из двух отдельных приложений, написанных для администратора и пользователей соответственно. Также для них отдельно будут созданы две разные базы данных, причём доступ к ним будет иметь только администратор. Сделано это для того, чтобы в случае взлома базы данных пользователей, злоумышленник не смог установить себе уровень доступа администратора, и не получил доступ к защищаемым данным.

Программа реализована на языке программирования C#, и написана в среде разработки Visual Studio. В ней имеется встроенная подключаемая библиотека System.Data.SqlClient, которая используется для работы с базами данных. Для их удобного отображения встроен элемент DataGridView, который позволяет редактировать данные в удобном формате, и имеет гибкий инструментарий.

Для написания функционала по распознаванию лиц использована последняя на данный момент времени версия библиотеки OpenCv4.1.1, а точнее, её *кроссплатформенная реализация EmguCv*, которая позволит взаимодействовать с OpenCv на языке C#. В последней версии библиотеки появилась возможность подключать обученные на нейронных сетях модели для обнаружения лиц. Это позволит во много раз увеличить точность и качество обнаружения, и облегчит работу в дальнейшем. Используемая модель обучена на снимках размером 300 x 300 пикселей с применением 140000 тысяч итераций, и обеспечивает невероятную точность в обнаружении, даже при вращении и повороте головы в сторону, но из этого следует, что и на вход данной модели нужно подавать изображение соответствующего размера. В библиотеке реализованы 3 необходимых алгоритма распознавания лиц, рассмотренные в теоретической части.

Описать работу программы можно при помощи блок–схем алгоритмов. На рисунке 4.1 показана работа программы и вызываемых функций, выполняемых сразу после запуска приложения. При этом сперва происходит инициализация формы и компонентов, необходимых для работы программы. Первыми объявляются переменные, с помощью которых можно взаимодействовать с обученным на нейронных сетях моделями для обнаружения лиц, и идет загрузка данных моделей. Далее происходит подключение к базе данных пользователей.

После этого, в случае успешного подключения, объявляются 3 алгоритма распознавания человеческих лиц: EigenFaceRecognizer, FisherFaceRecognizer и LBPHFaceRecognizer. Последний обладает преимуществом по сравнению со своими предшественниками, так как он устойчив к изменению освещения, однако есть смысл использовать все три алгоритма в комплексе, и принимать решение, основываясь на совокупности их предсказаний.

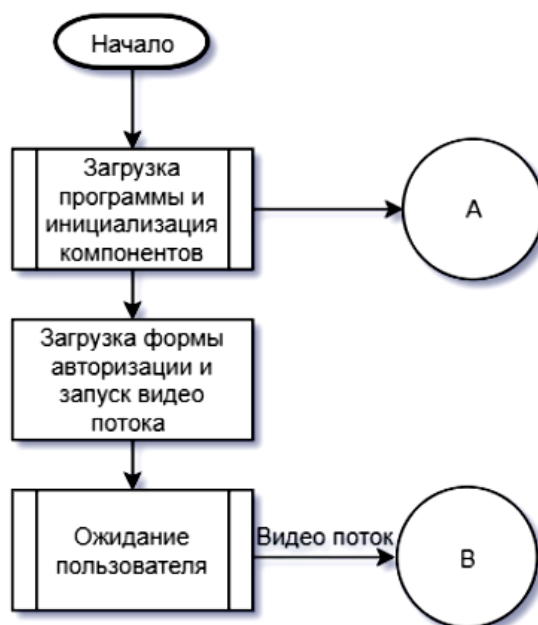


Рисунок 4.1 – Алгоритм работы программы авторизации

Затем, на вход данным алгоритмам подаются два массива: фотографии пользователей и соответствующие им (их названию) метки. Причем не важно в какой последовательности идут изображения и метки, главное, чтобы присутствовали идентичные названия и их количество было равным. Работу данной части программы можно увидеть на рисунке 4.2.



Рисунок 4.2 – Инициализация необходимых компонентов и алгоритмов

После того, как программа загрузилась, и все необходимые компоненты были подключены, происходит запуск Web-камеры и считывание видеопотока, который будет отображаться в соответствующем окне. Сразу при запуске камеры, каждый фрейм (кадр) будет передаваться в функцию, внутри которой он будет обрабатываться и приводиться к нужному виду. После обработки, происходит поиск лица по на изображении. В случае, когда лицо найдено, вокруг него будет нарисована рамка, чтобы наглядно показать работу детектора. Затем обнаруженное лицо обрабатывается и подается на вход алгоритму распознавания и производится предсказание метки. Если вероятность принадлежности лица текущего пользователя лежит в допустимом диапазоне, происходит авторизация.

Ввиду эффективности алгоритма LBPН, было принято решение определять корректность по его предсказанию. На рисунке 4.3 изображена блок-схема алгоритма работы функции обработки кадров с видеокамеры.

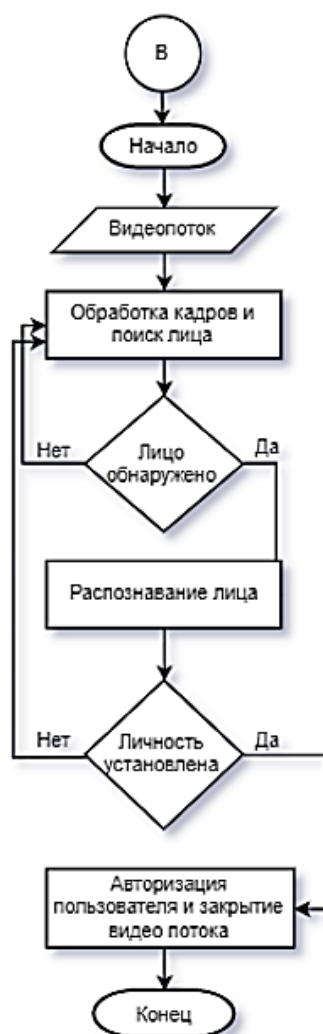


Рисунок 4.3 – Функция обнаружения и распознавания лица

Распознавания личности по алгоритму LBP позволит сократить количество необходимых фотографий для базы пользователей, ввиду того, что не нужно будет делать различные вариации освещения.

5 ПРОГРАММНОЕ КОНСТРУИРОВАНИЕ

В разработанном программном средстве используется интерпретатор библиотеки OpenCv4.1.1–EmguCv4.1.1, распространяемой на основе открытого кода. Для создания программного средства использовался язык C# и интегрированная среда разработки программных приложений Microsoft Visual Studio 2017 Community. Сохраняемые и используемые фотографии имеют формат BMP, данные о пользователях хранятся в файл базы данных MS SQL Server расширения MDF, а использование данных файлов внутри созданного программного обеспечения обеспечивается при помощи встроенной библиотеки, общение с которой происходит путем подключения пространства имен System.Data.SqlClient.

5.1 Описание основных функций

5.1.1 Описание данных о сотрудниках и работа с ними

Для работы с таким программным средством необходимо, чтобы администратор первым делом зарегистрировал пользователя, а также сформировал базу фотографий и соответствующих им меток. Чем больше будет фотографий в базе, тем лучше будет работать алгоритм распознавания личности. Желательно использовать различные вариации мимики, поворотов и наклонов головы, а об освещении можно не переживать, так как особенностью используемого алгоритма является устойчивость к его смене, что позволяет в разы сократить объём БД.

Для хранения и обработки информации о пользователях, используется реляционная база данных, общение с которой происходит с помощью sql-запросов. В базе хранится такая информация, как: id пользователя, его ФИО, телефон для связи, дата рождения и регистрации. Для *двухфакторной аутентификации*, которая обязательна для администратора, а пользователям устанавливается по их желанию, также хранится пароль, который можно использовать в комплексе.

Для удобства добавления и исправления данных в базе, они отображаются с использованием компонента DataGridView, который позволяет редактировать данные в удобном формате, и имеет гибкий инструментарий.

На рисунке 5.1 представлен наглядный пример базы данных.

Пользователи Администраторы Доступные файлы					
	Id	ФИО	PIN	Date of birth	Date of registration
	777	Тишкин Федор ...	c0b85819bbcc0c1827e...	30.03.1996	06.11.2019
	111	TestUser	a63a7d76e7e331b6b49...	20.02.2002	03.11.2019

Рисунок 5.1 – Пример отображения данных

Доступ к базам данных имеет только администратор. Со стороны пользователя БД никак не отображаются, а только проходит подключение к базе данных пользователей в момент распознавания личности. Затем соединение закрывается.

5.1.2 Работа алгоритма обнаружения

Для осуществления работы алгоритма распознавания личности сперва необходимо научить программу обнаруживать человеческие лица в кадре. Для этого в этой программе используется заранее обученная, при помощи глубокого обучения, модель. Для обучения был использован фреймворк *Caffe*.

Разработчики потратили много времени для сбора фотографий и обучения этой модели. Предоставление открытого доступа к ней обуславливается тем, что на обучение такой модели с нуля потребовалось бы очень много времени и ресурсов, тем более для обычных пользователей, не имеющих современного оборудования.

Для применения этой модели в последних версиях библиотеки *OpenCv* была добавлена возможность подключать обученные модели обнаружения лиц, среди которых наиболее популярны модели от *Caffe* и *TensorFlow*. Несмотря на низкую, по сравнению с *TensorFlow*, популярность, в работе используется модель от *Caffe*, так как фреймворк специализирован именно на обработке изображений.

В итоге, как оказалось, используемый метод обнаружения лиц работает очень точно, и способен обнаружить и одновременно вывести порядка 20 лиц. Также учитывается вращение и поворот головы на 90 градусов.

Обнаружение и распознавание лица в кадре осуществляется с помощью одной функции, логически разделенной на две последовательной части. В первой части происходит захват текущего кадра видеопотока, и дальнейшее его изменение до необходимого для работы модели векторного представления.

После этого кадр подается на вход используемой модели, которая возвращает 4-х мерный массив с результатом обнаружения. Полезными значениями являются координаты предполагаемого лица, рекомендуемая ширина рамки и коэффициент уверенности модели в том, что в кадре в нужном месте обнаружено именно лицо. Значение порога для данного коэффициента можно устанавливать в коде. Чем больше значение, тем точнее будет результат. Если в кадре обнаружено только одно лицо, то текущий кадр передается на вход алгоритму распознавания. Если лиц больше двух, то алгоритм распознавания пропускается, и в качестве результата выводится сообщение «Not found».

5.1.3 Обучение алгоритма распознавания

Для осуществления главной функции программного средства используется алгоритм локальных бинарных шаблонов – Local Binary Patterns (LBP).

Особенностью данного алгоритма является тот факт, что отбрасывается необходимость делать множество снимков с различным освещением. Это позволяет сократить размер базы фотографий и распознавать личность с большой точностью в помещении с меняющимся освещением.

Для формирования базы данных, в которых хранятся фотографии сотрудников, администратору предоставлен определённый функционал, который станет ему доступен после того, как он выполнит авторизацию. При переходе на соответствующую форму открывается окно с двумя областями, в которых выводится видеопоток в реальном времени, а также возможный конечный результат снимка. Пример такого окна показан на рисунке 5.2.

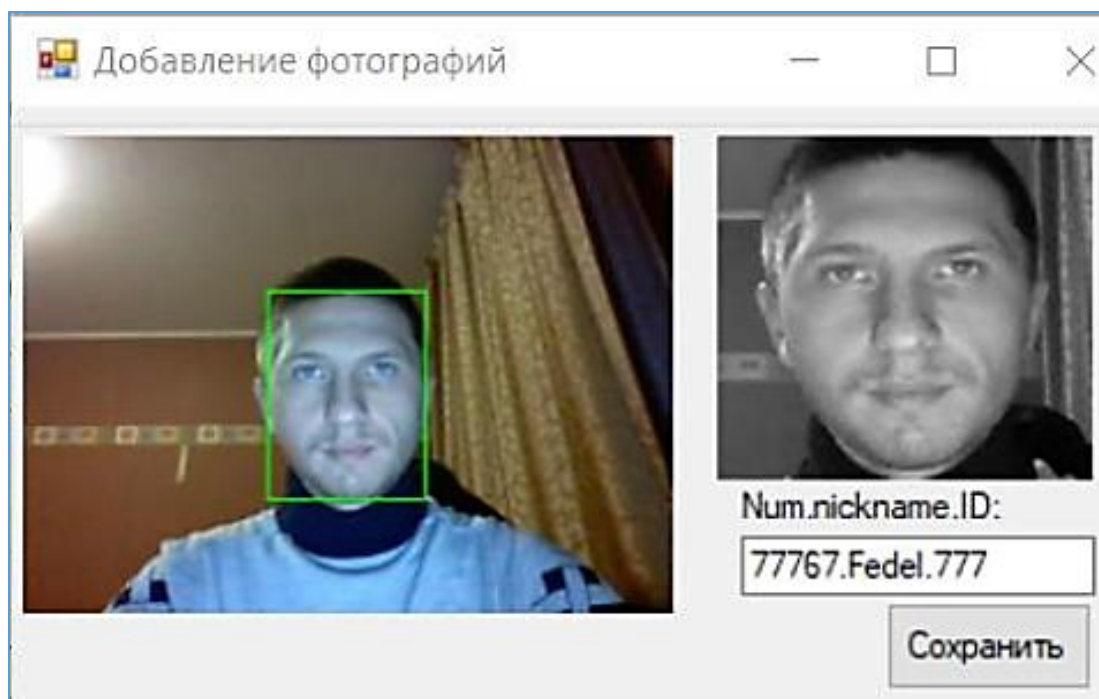


Рисунок 5.2 – Графический интерфейс добавления фотографии

При добавлении необходимо ввести метку, которая состоит из трёх частей, разделенных точками, а именно: номер метки – любое целое число, никнейм – любые символы, id пользователя в базе данных – целое число.

Номер метки – это одновременно и название сохраняемого изображения. Хранится она в виде числа, потому что для тренировки алгоритма нужно подавать фотографии и соответствующие им метки именно в виде числа, которые он потом выдает в качестве результата предсказания.

Порядок их написания не важен, главное, чтобы количество фотографий и меток было одинаково, и числовые метки начинались с новой строки, ввиду архитектуры разрабатываемого программного приложения.

Никнейм использовался для тестирования работы программы, когда база данных пользователей еще не была построена, и было принято решение оставить этот параметр в качестве удобства, как дополнительный.

Идентификатор пользователя тоже записывается и хранится рядом с меткой, чтобы можно было установить принадлежность обнаруженной личности определенному пользователю, имеющемуся в базе данных.

При некорректном вводе информации о сохраняемом изображении или возникновении иной другой ошибки будет выдано соответствующее сообщение.

После добавления в базу данных пользователя, формирования базы данных для фотографий и списка соответствующих меток, все необходимые для работы компоненты определены. При запуске программы и инициализации компонентов, наряду с другими функциями происходит вызов тренера алгоритма распознавания. При этом, на входе подаются фотографии, загруженные в список изображений на шаг раньше, и текстовый файл с номерами меток.

При обнаружении лица в кадре, изображение вокруг лица определенным образом обрезается, обесцвечивается и подается на вход предсказателю алгоритма, который на выходе предоставляет две переменные: возможная метка пользователя и дистанция. Вторая переменная показывает, насколько близко (или аналогично), по расчётам алгоритма, находится переданное лицо с самым похожим изображением в базе. Если результат равен 0, то алгоритм точно уверен в правильности распознавания. Но такое возможно, только если передать в точности изображение, которое находится в базе фотографий, поэтому при таком результате программа может принять решение, что осуществляется попытка фальсификации и запретит доступ. Если значение равно -1, то это означает, что алгоритму не удалось распознавать личность.

5.1.4 Описание графической части программы

Для того, чтобы исключить возможность пользователям (или хакерам, получившим, каким-то образом, несанкционированный доступ) получить доступ к базе данных и установить себе права администратора, было принято решение разделить пользователей и администраторов на две базы данных. Для этого было принято решение разделить программное средство на два проекта. Один устанавливается на компьютере или учётной записи администратора, другая часть – на компьютеры обычных пользователей.

Отличается также и функционал программ. Пользователь может лишь авторизоваться и получить тем самым доступ к зашифрованным файлам. На рисунке 5.3 представлен основной интерфейс пользователя после авторизации.

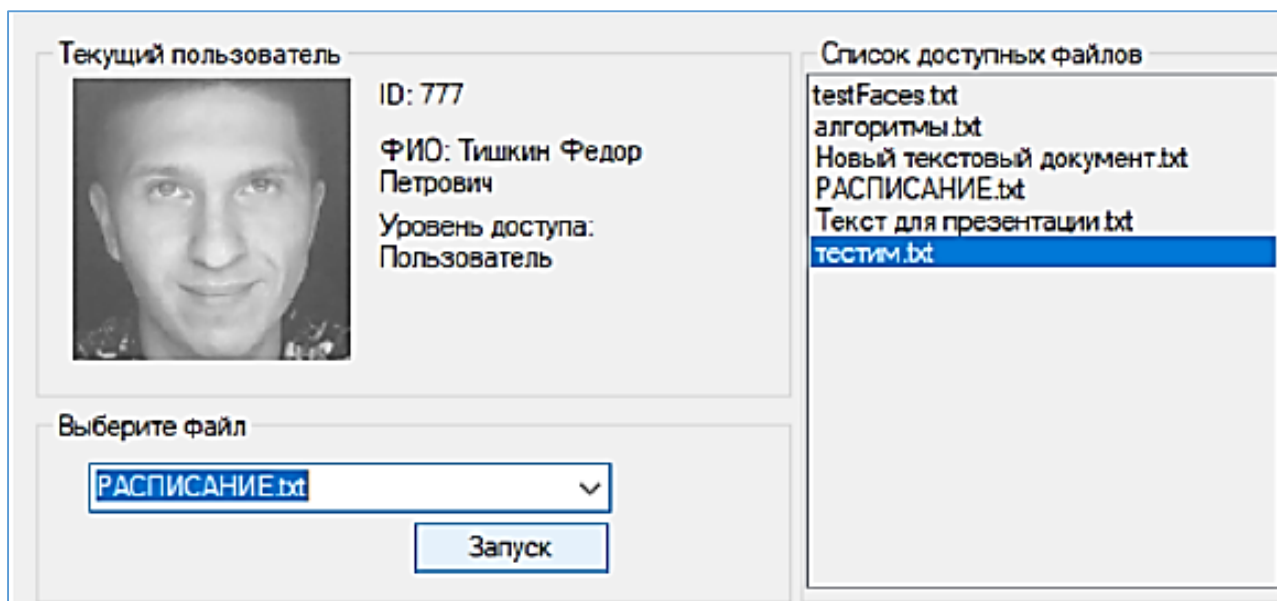


Рисунок 5.3 – Главное программное окно пользователя

Для запуска доступных файлов пользователю нужно выбрать необходимый зашифрованный файл во всплывающем окне и нажать кнопку «Запуск». Приведен список всех доступных для запуска файлов.

Пользователи с правами администратора имеют намного больший функционал. Помимо того, чтобы осуществлять простой запуск зашифрованных файлов, администратор имеет возможность их удалять и добавлять. При добавлении файлы шифруются с использованием алгоритма шифрования AES, который дополнительно добавляется в виде библиотеки, и хранятся в папке ресурсов, которая расположена в директории программы администратора.

Интерфейс главного окна имеет 3 вкладки, на которых отображается база данных пользователей, база данных администраторов и вкладка для работы с файлами. Кнопки сохранения изменений исходных данных о пользователях и добавления фотографий локализованы на панели инструментов, в верхней части экрана.

При добавлении в базу данных нового пользователя необходимо записать пароль, но его сохранение в чистом виде – это грубейшее нарушение правил безопасности. Для удобства в нижней части интерфейса предоставлен необходимый функционал, который предназначен для генерации случайного пароля и идентификатора, а также инструмент для хеширования, основанный на алгоритме MD5 с засаливанием. На рисунке 5.4 представлен пример отображения главного окна программы, которое открывается после авторизации администратора.

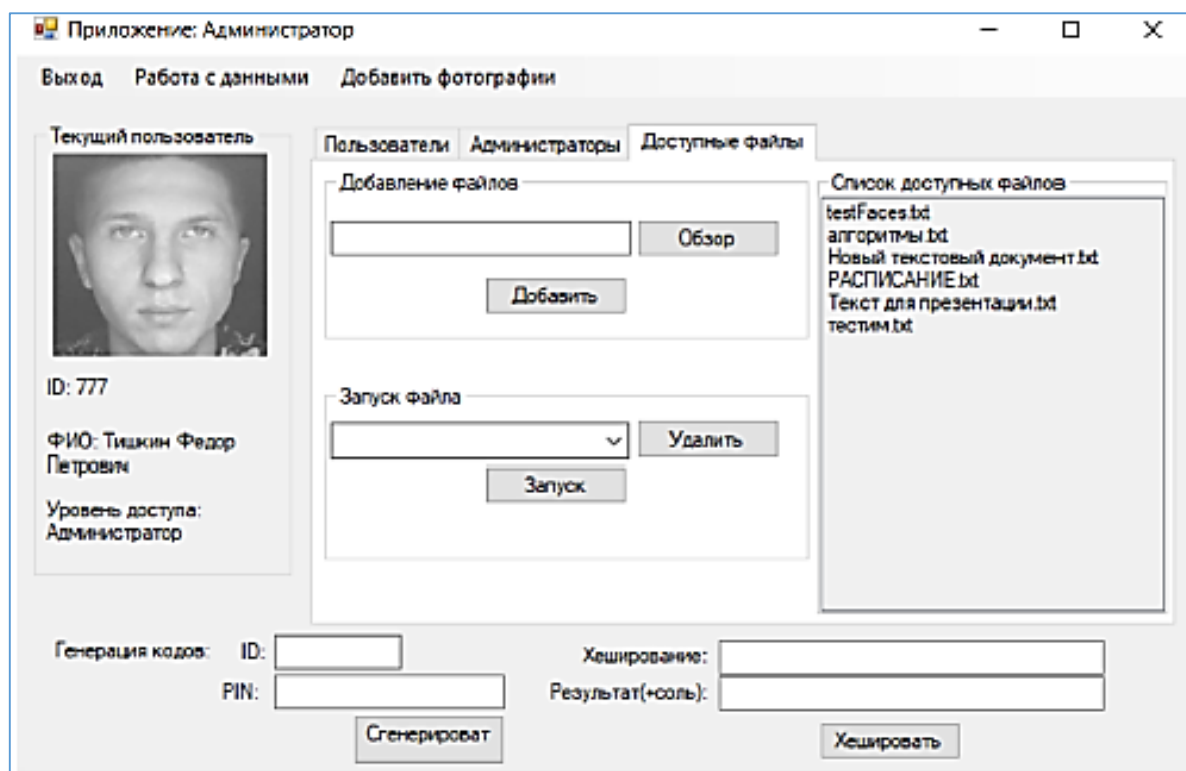


Рисунок 5.4 – Главное программное окно администратора

Запуск программы занимает около 5 – 6 секунд, это время отводится на инициализацию компонентов и подключение необходимых для полноценной работы модулей. Главный функционал начинает вступает в действие при нажатии кнопки «Start», после чего включается камера и захватывается каждый кадр видеопотока. Если лицо человека обнаруживается, то оно обводится в рамку. На рисунке 5.5 представлен пример успешной авторизации администратора.

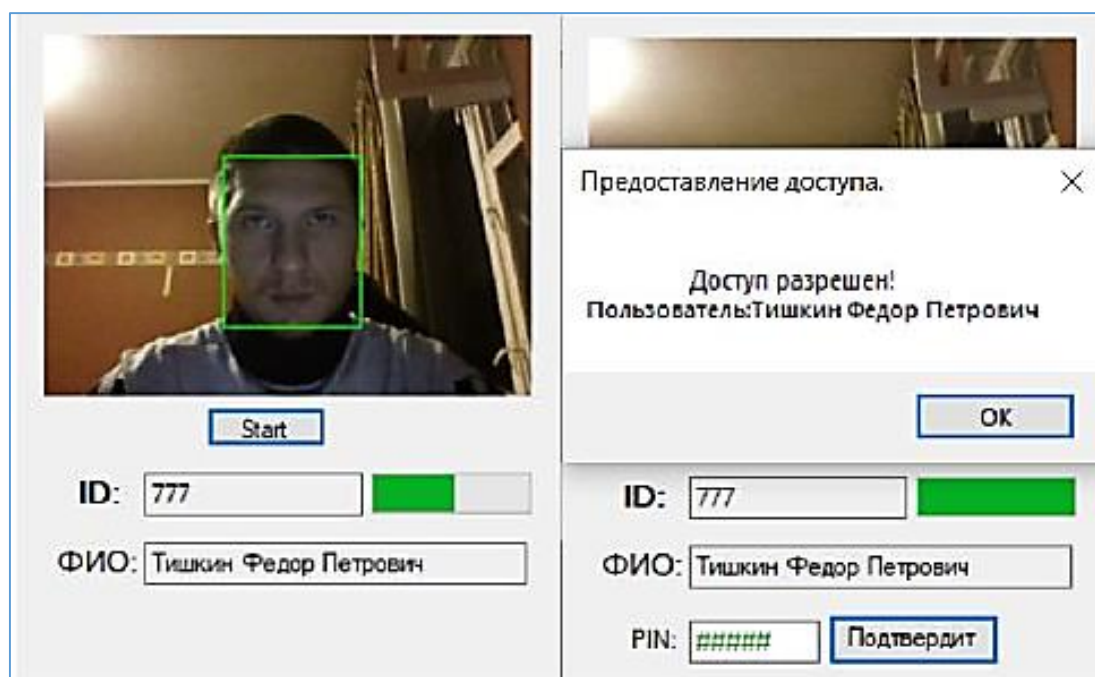


Рисунок 5.5 – Главное окно администратора при успешной авторизации

Также при работе алгоритма распознавания запускается счётчик кадров, если более чем в одном кадре был обнаружен один и тот же человек. Этот счётчик заполняет полосу прогресса авторизации, и по достижении определенного числа осуществляется авторизация. Если лицо пропадает из кадра, или обнаруженная личность изменяется, то счётчик через две секунды сбрасывается. Если до истечения времени последний пользователь возвращается в кадр, то счётчик продолжает увеличивать значение. После заполнения шкалы прогресса, администратору необходимо ввести пароль, для дополнительного подтверждения личности. Обычному пользователю пароль вводить не нужно.

Выводы

В результате проведенных исследований и проделанной работы разработано и протестировано импортозамещающее программное средство, позволяющее обнаружить и распознать лицо человека в кадре, с целью авторизации пользователя (сотрудника организации) с помощью видеокамеры. Для хранения и обработки информации используется реляционная база данных, общение с которой происходит с помощью sql-запросов. В системе используется двухфакторная аутентификация. Доступ к базам данных имеет только администратор. Алгоритм обнаружения использует библиотеку OpenCv с возможностью подключения обученных модулей обнаружения лиц – моделей от Caffe и TensorFlow. Для обучения алгоритма распознавания используется алгоритм локальных бинарных шаблонов – Local Binary Patterns (LBP). Используются 3 алгоритма распознавания человеческих лиц: EigenFaceRecognizer, FisherFaceRecognizer и LBPHFaceRecognizer. Графическая часть программы обрабатывает каждый кадр видеопотока. После успешной авторизации администратору необходимо ввести пароль, для дополнительного подтверждения личности. Обычному пользователю пароль вводить не нужно.

Таким образом, реализовано импортозамещающее программное средство, позволяющее осуществлять авторизацию пользователей при помощи видеокамеры, используя их личные уникальные биометрические показатели – черты лица.

РЕЗУЛЬТАТЫ 9 ГЛАВЫ

В работе исследованы биометрические данные пользователей, используемые для аутентификации, и выбран самый удобный и надежный из них – черты лица. Проанализированы методы обнаружения и алгоритмы распознавания лица в кадре и выбраны наилучшие варианты, опираясь на их преимущества и недостатки.

Импортозамещающее программное приложение, реализованное в рамках проведенных исследований, использует модуль компьютерного зрения EmguCv, который является интерпретатором библиотеки OpenCv для языка C#.

Функционал этой библиотеки используется для обнаружения лица в кадре и распознавания личности человека. Для хранения информации о пользователях используется реляционная база данных. Программное средство реализовано на Visual Studio 2017 Community с использованием языка программирования C#.

Программное приложение предназначено для отечественных организаций, офисов и домашнего пользования, когда появляется необходимость ограничить доступ к данным определенного круга лиц пользователей.

Приложение служит примером, показывающим реальные возможности применения видеотехнологий, которые только начинают набирать популярность в мире. Очевидно, что их применение в будущем будет только расширяться.

ГЛАВА 10

РАЗРАБОТКА ИМПОРТОЗАМЕЩАЮЩЕГО ПРОГРАММНОГО ПРИЛОЖЕНИЯ ПОЧТОВОГО КЛИЕНТА СО СКРЫТОЙ ПЕРЕДАЧЕЙ ИНФОРМАЦИИ НА ОСНОВЕ СТЕГАНОГРАФИЧЕСКИХ МЕТОДОВ

ОБЩИЕ ПОЛОЖЕНИЯ

В современном мире информация, ввиду своей распространенности и востребованности, является одним из самых ценнейших ресурсов на планете. Развитие технологий и средств коммуникаций всё больше повышают значимость информации как в повседневной жизни человека, так и в сферах бизнеса, науки, государственного управления и многих других. Повсеместное применение информации приводит к необходимости обеспечить её безопасность как от несанкционированного доступа, так и от повреждения или уничтожения злоумышленниками. Комплексные системы защиты информации могут предполагать использование методов как стеганографии, так и криптографии с целью обеспечения целостности и конфиденциальности информации.

Применение *стеганографии* как метода защиты информации от несанкционированного доступа можно уследить еще в период античности. Данный метод предполагает не только шифрование, но и *сокрытие* самого факта наличия информации в некотором объекте или канале передачи. Ввиду наибольшей распространенности цифровой информации в деятельности современного человека, наибольшую актуальность представляет именно *цифровая стеганография*, где в качестве контейнеров для хранения информации выступают электронные документы, изображения, видеозаписи и прочие файлы, так как обнаружение факта наличия скрытой информации практически невозможно без использования специальных технических средств [68].

Тема стеганографии как инструмента защиты информации рассматривается в работах многих отечественных авторов. Большой интерес представляют стеганографические контейнеры, являющиеся изображениями. Это обусловлено:

- практической значимостью защиты аудио– и видео– (мультимедийной) информации от незаконного копирования и распространения;
- значительным объёмом цифрового изображения, позволяющего варьировать между объёмом скрываемого сообщения и стойкостью к обнаружению и изменению;
- тем, что при встраивании информации известен конечный объём контейнера;
- тем фактом, что нет ограничений на встраивание скрываемой информации в режиме реального времени, как, например, в потоковом видео;
- тем, что многие изображения имеют области, имеющие шумовую структуру и хорошо подходящих для встраивания информации.

Множество имеющихся стеганографических приложений ограничены и неудобны в своем применении – они прячут сообщения непосредственно в файлах, причём только определенных форматов.

Передачу заполненного стеганографического контейнера должен осуществлять уже непосредственно сам пользователь, что может привести к утрате контейнера во время его хранения, либо передачи. Ввиду это, может оказаться востребованным приложение, осуществляющее и внедрение, и передачу сообщения.

Почтовые клиенты являются распространенными приложениями в обиходе как в сфере бизнеса, так и у рядового пользователя ПК. Учитывая их использование в целях передачи и чтения сообщений, они являются отличными кандидатами для интеграции со стеганографическими методами скрытия информации.

Таким образом, представляется актуальной и современной задача реализации методов скрытия данных в электронных письмах.

Объект исследования: стеганографические методы обработки информации.

Предмет исследования: возможность внедрения стеганографических сообщений в электронные письма.

Цель исследования: реализация приложения, скрывающего сообщения в электронных письмах в ходе передачи информации.

Для достижения поставленной цели необходимо решить ряд следующих задач:

- определить и изучить возможные методы вложения скрытых сообщений в электронные письма и сообщения;
- исследовать процесс реализации почтового клиента с возможностью редактирования различных компонентов электронных писем;
- реализовать программу, осуществляющую вложение, обнаружение и чтение скрытых сообщений в электронных письмах.

Методы исследования: для решения поставленных задач использовались алгоритмы сокрытия данных в графических изображениях и в тексте, а также программа, написанная на языке C#.

1 ОБЗОР СОВРЕМЕННОЙ ЦИФРОВОЙ СТЕГАНОГРАФИИ

1.1 Общие сведения и терминология

Стеганография — метод хранения и передачи данных в некотором контейнере, скрывая сам факт передачи и существования информации в нём.

В отличие от криптографии, которая скрывает содержимое сообщения и делает невозможным его понимание, стеганография скрывает факт наличия скрываемой информации в контейнере. Поэтому основная цель стеганографии — найти такие области контейнера, которые можно заменить на зашифрованное секретное сообщение, которое нужно передать по каналам связи.

Секретным сообщением t называется информация, которую необходимо скрыть. Сообщение t представляется в виде бинарной последовательности bt или матрицы действительных чисел $b_{i,j}$ [68, 69].

Контейнером C называется цифровой объект, в котором необходимо скрытно передать сообщение T . *Пустой контейнер* — это контейнер, в котором отсутствует вложение. *Заполненный контейнер* — контейнер, в котором оно присутствует.

Различают два основных типа контейнеров: *поточковый* и *фиксированный*. Поточковый представляет собой непрерывную цифровую последовательность (видеопоток), фиксированный имеет predetermined размер и характеристики.

Ключ — это секретная информация, необходимая для встраивания и извлечения стеганографического сообщения из контейнера.

В любом контейнере существует информационная и служебная область. В информационной области хранятся данные, которые используются для формирования образа контейнера перед его выводом. Служебная часть контейнера содержит данные, которые не используются для формирования (заголовки, теги).

Так как в современном мире информация представляется в цифровой форме, то в качестве контейнеров часто используют: неподвижные изображения, видеофайлы, аудиофайлы, текстовые документы. Скрываемая информация может являться текстовым, числовым, графическим, аудио и видеофайлом — главное, чтобы она была представлена в цифровом виде.

Обмен сообщениями может производиться через письма электронной почты; Web-сайты; информацию, хранящейся на съёмных носителях; телевизионные и радиоканалы для передачи сообщений. Это используется злоумышленниками в своих целях для передачи сообщений по открытым каналам связи. Поэтому разработка эффективных методов обнаружения вложений в различных контейнерах и их исследование является важной и актуальной задачей.

Для их поиска были разработаны различные методы обнаружения, которые учитывают статистические свойства контейнера и алгоритмы вложения и извлечения информации. Для оценки эффективности методов учитываются вероятность ложного обнаружения стеганографические вложения, вероятность его пропуска, а также качество контейнера после вложения.

1.2 Стеганографические системы

1.2.1 Принцип работы стеганографических систем

Задачу встраивания и выделения сообщений из другой информации выполняет стеганографическая система (стегосистема), которая состоит из следующих основных элементов, представленных на рисунке 1.1:

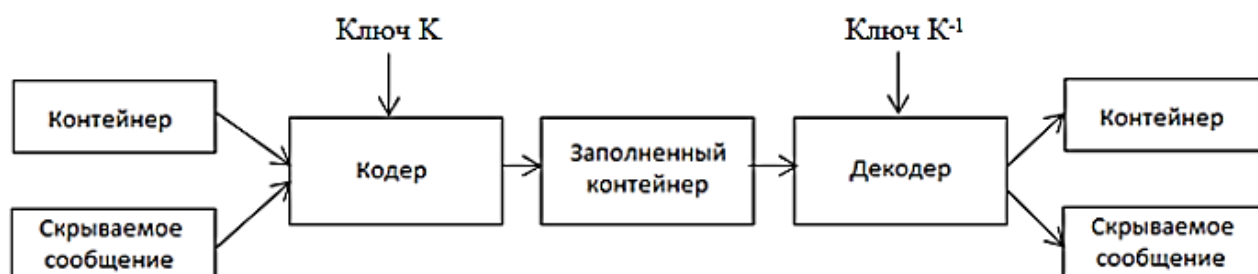


Рисунок 1.1 – Структурная схема типичной стеганографической системы

Кодер представляет собой устройство, предназначенное для осуществления вложения скрытого сообщения в другие данные с учетом их модели.

Декодер – устройство, восстанавливающее скрытое сообщение.

Некоторые реализации стегосистем также включают прекодер и стегодетектор. *Прекодер* – это устройство, предназначенное для преобразования скрываемого сообщения к виду, удобному для встраивания в стегоконтейнер. *Стегодетектор* – это устройство, предназначенное для определения наличия стеганографического (скрытого) сообщения. Данные, содержащие скрытое сообщение, могут подвергаться преднамеренным атакам или случайным помехам.

Как показано на рисунке 1.1, в стеганографической системе происходит объединение двух типов информации так, чтобы они могли быть различимы двумя принципиально разными детекторами. В качестве одного из детекторов выступает система выделения стегосообщения, в качестве другого – человек.

Прежде, чем осуществить вложение скрытого сообщения в контейнер, оно должно быть преобразовано к какому-то подходящему виду. Например, если в качестве контейнера выступает графическое растровое изображение, то и сообщение зачастую представляется как двумерный массив битов.

Чтобы повысить устойчивость заполненного контейнера к искажениям, часто выполняется его помехоустойчивое кодирование, или применяются широкополосные сигналы. Первоначальную обработку выполняет прекодер.

1.2.2 Стеганографические протоколы

Различают три типа стеганографических протоколов [69]:

- протоколы чистой стеганографии;
- протоколы стеганографии с секретным ключом;
- протоколы стеганографии с открытым ключом.

Чистая стеганография определяется как стеганографическая система, которая не требует использования шифра и ключей. Этот метод не безопасен, так как и отправитель, и получатель могут полагаться только на своё предположение, что никакие другие стороны не знают о секретном сообщении, что обычно не так.

Стеганография с секретным ключом определяется как стеганографическая система, которая требует обмена секретного ключа (стего–ключа) до общения. Стеганография с секретным ключом принимает сопроводительное сообщение, и встраивает секретное сообщение внутрь него с помощью секретного ключа (стего–ключа). Только стороны, которые знают секретный ключ, могут расшифровать и прочесть секретное сообщение. Процесс обмена секретного ключа является уязвимым для перехвата. Преимущество стеганографии с секретным ключом заключается в том, что даже если оно перехвачено, то только стороны, знающие секретный ключ, могут извлечь секретное сообщение.

Стеганография с открытым ключом определяется как стеганографическая система, которая использует открытый ключ и частный ключ для обеспечения тайной связи между сторонами, желающими общаться секретно. Отправитель будет использовать открытый ключ во время процесса кодирования, и только закрытый ключ, который имеет прямую математическую связь с открытым ключом, может расшифровать секретное сообщение. Такая система является наиболее перспективной в реализации в практических системах, поскольку она может использовать гораздо более надежные и исследованные технологии из области криптографии с открытым ключом. Она также имеет несколько уровней безопасности, поскольку нежелательные стороны должны сначала заподозрить использование стеганографии, и тогда они должны будут найти способ взломать алгоритм, используемый системой с открытым ключом, прежде чем они смогут перехватить секретное сообщение.

1.2.3 Характеристики стеганографической системы

Эффективная компьютерная стеганографическая система должна обладать следующими желаемыми характеристиками [70]:

- *секретность*: посторонний человек не должен иметь возможность извлекать скрытые данные с носителя, не обладая знаниями о правильном секретном ключе, используемом в процедуре извлечения стеганографических данных;
- *незаметность*: среда, после встраивания скрытых данных, должна быть неотличима от первоначальной оригинальной среды; у непосвящённых лиц не должно возникать сомнений и подозрений по поводу того, что над данными исходного контейнера совершались манипуляции с целью передачи скрытой информации;
- *высокая ёмкость*: максимальная длина скрытого сообщения, которое может быть встроено в контейнер, должна быть как можно большей;
- *сопротивление*: секретные данные должны быть в состоянии выжить, когда носителем манипулируют, например, по какой-то схеме сжатия с потерями;
- *точное извлечение*: извлечение скрытых данных из носителя должно быть точным и надежным, и не приводить к их потере, хотя бы частичной.

Основным фактором в определении подобных характеристик является метод внедрения информации с контейнер, который, в свою очередь, зависит уже от передаваемого сообщения и типа используемого контейнера (документ, графическое изображение, аудио— или видео —, или мультимедийный файл).

Следует понимать, что незаметность предполагает не только защиту контейнера от раскрытия хранящейся в нём информации в результате обычного человеческого наблюдения, но и от обнаружения с помощью статистических методов исследования информации, которые бы позволили определить факт изменения контейнера с целью хранения в нём информации.

1.3 Методы стеганографии

Основные направления *компьютерной стеганографии*—это методы, связанные с использованием специальных свойств различных форматов хранения данных, а также методы, связанные с использованием избыточности информации при представлении данных, и особенностями восприятия человека, чей глаз способен видеть низкочастотный шум гораздо лучше, чем высокочастотный, искажающий оттенок. Высококонтрастные и яркие участки всегда являются более заметными.

Глаз человека также чувствителен к большим участкам изображения, длинным и тонким объектам, некоторым цветам и местоположению объекта. Методы стеганографии можно разделить по следующим критериям:

- по области встраивания – в пространственной или в области преобразования:
- сокрытие данных в пространственной области — для внедрения выполняются изменения яркости или цветовых составляющих. К алгоритмам, способным выполнять данные преобразования, можно отнести следующие методы: Kutter, Bruyndonckx, Langelaar, Pitas, Rongen, PatchWork, Bender, Marvel;
- сокрытие данных в области преобразования — для внедрения чаще всего используют вейвлет-преобразования или дискретные косинусные преобразования (ДКП). Их эффективность основана на способности отделения информативных областей изображения от менее информативных. Алгоритмы, способные скрыть данные в области преобразования: Koch, Benham, A25, Hsu, Tao, A17, A18, Friderich;
- графический метод, который вкладывает данные в текст, представленный в графическом виде, при помощи изменения различных графических свойств контейнера. В качестве контейнера может быть использован BMP-файл, а секретной информацией может являться текст.

1.4 Соккрытие данных в изображениях

Большинство исследований посвящено использованию изображений в качестве стегоконтейнеров. Это обусловлено следующими причинами:

- существованием практически значимой задачей защиты фотографий, картин, видео от незаконного тиражирования и распространения;
- большим объёмом цифрового представления изображений, что позволяет внедрять информацию большого объёма, либо повышать *робастность* внедрения;
- заранее известным размером контейнера, отсутствием ограничений, накладываемых требованиями реального времени;
- наличием в большинстве реальных изображений текстурных областей с шумовой структурой, и поэтому хорошо подходящих для встраивания информации;
- слабой чувствительностью человеческого глаза к незначительным изменениям цветов изображения, его яркости, контрастности, содержанию в нём шума, искажениям вблизи контуров;
- хорошо разработанными методами цифровой обработки изображений.

Последняя причина вызывает и значительные трудности в обеспечении робастности: чем более совершенными становятся методы сжатия, тем меньше остается возможностей для встраивания посторонней информации.

1.5 Система человеческого зрения

1.5.1 Человеческое зрение и алгоритмы сжатия изображений

Свойства *системы человеческого зрения* (СЧЗ) можно разделить на две группы: низкоуровневые («физиологические») свойства и высокоуровневые («психофизиологические») свойства. Вплоть до середины 90-х годов XX века исследователи принимали во внимание, главным образом, низкоуровневые свойства зрения. В последние годы наметилась тенденция построения стеганографических алгоритмов с учётом и высокоуровневых характеристик СЧЗ.

Выделяют три наиболее важных низкоуровневых свойства, влияющих на заметность постороннего шума в изображении: чувствительность к изменению яркости изображения, частотная чувствительность и эффект маскирования.

Чувствительность к изменению яркости представляет собой способность человека к обнаружению разницы в яркости в похожем на монотонное изображении. Пример эксперимента: берется изображение, представляющее собой заполненный одним цветом квадрат; после того, как человек полностью сфокусируется на изображении, у центральной области квадрата постепенно повышают яркость до тех пор, пока человек не обнаружит различие в цвете. Проведенные опыты показали, что человек более чувствителен к изменению яркости, если она находится в диапазонах малых или больших значений.

Частотная чувствительность системы человеческого зрения проявляется в том, что человек гораздо более восприимчив к низкочастотному, чем к высокочастотному шуму. Это связано с неравномерностью амплитудно–частотной характеристики системы зрения человека. Экспериментально её можно определить при помощи того же опыта, что и при яркостной чувствительности. Но на этот раз в центральном изменяются пространственные частоты до тех пор, пока изменения не станут заметными [71].

Элементы системы человеческого зрения разделяют поступающий видеосигнал на отдельные компоненты. Каждая составляющая возбуждает нервные окончания глаза через ряд подканалов. Выделяемые глазом компоненты имеют различные пространственные и частотные характеристики, а также различную ориентацию (горизонтальную, вертикальную, диагональную).

В случае одновременного воздействия на глаз двух компонентов со сходными характеристиками возбуждаются одни и те же подканалы.

Это приводит к *эффекту маскирования*, заключающегося в увеличении порога обнаружения видеосигнала в присутствии другого сигнала, обладающего аналогичными характеристиками. Поэтому, аддитивный шум гораздо заметнее на гладких участках изображения, чем на высокочастотных, то есть в последнем случае наблюдается маскирование. Наиболее сильно эффект маскирования проявляется, когда оба сигнала имеют одинаковую ориентацию и местоположение.

Высокоуровневые свойства СЧЗ пока что не имеют такую же частоту применения в стеганографических алгоритмах, как низкоуровневые, но в последнее время практикуется их интеграция в новые алгоритмы. Отличием высокоуровневых свойств является то, что эти свойства проявляются после первичной обработки изображения глазами человека, и поэтому они являются менее весомыми показателями в процессе составления алгоритмов внедрения информации в изображения. Перечислим основные из этих свойств:

- чувствительность к контрасту – разница контраста вызывает внимание;
- чувствительность к размеру – большие объекты, располагающиеся на изображении, захватывают внимание человека в первую очередь;
- чувствительность к форме – длинные и тонкие объекты вызывают больший интерес, чем круглые однородные формы;
- чувствительность к цвету – определенные цвета вызывают у человека более сильную реакцию, чем другие – этот эффект усиливается, если фон заднего плана отличается от цвета фигур, расположенных на нём;
- чувствительность к местоположению – человек склонен рассматривать изображения, начиная с центра, а затем движется к краям картинки;
- чувствительность к ближайшим объектам – человек обычно более внимателен к объектам, находящимся на переднем плане изображения, чем на фоне;
- если на изображении есть люди, в первую очередь человек обратит свое внимание на них; рассматривая фотографии, человек обращает первоочередное внимание на лицо, глаза, рот, руки и фигуры людей;
- чувствительность к внешним раздражителям – внешние факторы, которые влияют на поведенческие особенности человека во время осмотра изображения (могут применяться как до, так и во время осмотра).

1.5.2 Принципы сжатия изображений

Под сжатием понимается уменьшение числа бит, требующихся для цифрового представления изображений. В основе сжатия лежат два фундаментальных явления: уменьшение статистической и *психовизуальной избыточности*.

Можно выделить три типа статистической избыточности:

- пространственная, или корреляция между соседними пикселями;
- спектральная, или корреляция между соседними частотными полосами;
- временная, или корреляция между соседними кадрами (для видео).

Высокие коэффициенты сжатия достижимы лишь с использованием психовизуальной избыточности изображения, то есть пренебрежения его визуально незначимыми частями. И тут уж не обойтись без знания системы человеческого зрения. «Выброшенные» части изображения заменяют нулями (константами), и если их много – применяют *кодер длин серий*. В реальных алгоритмах сжатия осуществляют обнуление не пикселей изображения, а спектральных коэффициентов. Преимущество такого подхода заключается в том, что близкие к нулю спектральные коэффициенты имеют тенденцию располагаться в заранее предсказуемых областях, что приводит к появлению длинных серий нулей и повышению эффективности кодирования. Большие по величине коэффициенты («значимые») подвергают более или менее точному квантованию и также сжимают кодером длин серий. Последним этапом алгоритма сжатия является применение *энтропийного кодера* (арифметического или *кодера Хаффмана*).

Восстановленное после сжатия изображение, естественно, отличается от исходного. При прочих равных условиях, чем больше сжатие, тем больше искажение. Для оценки качества восстановленного изображения можно использовать меру среднеквадратического искажения, определяемую как:

$$CKO = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2 ,$$

где N – число пикселей в изображении, $x_i, \hat{x}_i$ – значение пикселей исходного и восстановленного изображений соответственно. Гораздо чаще применяется модификация этой меры – *пиковое отношение сигнал / шум*, определяемое как

$$ПОСШ = 10 \log_2 \frac{N 255^2}{\sum_{i=1}^N (x_i - \hat{x}_i)^2} ;$$

где 255 – это максимальное значение яркости полутонового изображения (т.е. 8 битов на пиксел). Восстановленное изображение считается приемлемым, если $ПОСШ \geq 28 - 30$ дБ (в среднем). Перечисленные объективные меры искажения не всегда коррелируют с субъективным восприятием изображений, однако лучшего метода до сих пор не разработано.

ПОСШ не всегда хорошо согласуется с визуально наблюдаемой ошибкой. Пусть имеется два изображения, которые полностью одинаковы, кроме небольшой области. Хотя визуально разность между этими изображениями хорошо заметна, *ПОСШ* будет примерно одинаковым. Учёт системы человеческого зрения в схеме сжатия является самой трудной задачей. Было проведено множество исследований, но в силу возникших трудностей с математическим описанием системы зрения человека более подходящей меры найдено не было.

1.5.3 Соккрытие информации в пространственной области

Методы сокращения информации в пространственной области используют пиксельные уровни яркости и значения цвета непосредственно для кодирования битов сообщения. Эти методы являются одними из самых простых схем в встраивании и извлечении информации. Основным недостатком этих методов является количество шумовых добавок, которые непосредственно влияют на пиковое соотношение сигнал / шум и статистические свойства изображения.

Кроме того, эти алгоритмы встраивания, в основном, применимы к сжатию изображений без потерь, таких, как, например, изображения TIFF. Для сжатия с потерями, как, к примеру, изображения в формате JPEG, некоторые биты сообщения теряются на этапе сжатия изображения.

Выделяют следующие методы данного класса:

- метод наименьшего значащего бита (LSB);
- метод разницы между значениями пикселей (PVD);
- метод встраивания данных на основе крайних областей (EBE);
- метод встраивания случайных пикселей (RPE);
- метод отображения пикселей в скрытые данные (PMM);
- метод маркировки или связывания;
- метод интенсивности пикселей, или значения уровня яркости (GLV);
- текстурный метод;
- методы, основанные на гистограмме;
- методы, основанные на расширении спектра;
- методы, основанные на цветовой палитре.

Наиболее распространенный алгоритм, принадлежащий этому классу методов – это метод наименьшего значащего бита (LSB). Этот метод представляет собой технику замены, в которой в наименее значащем бите используется двоичное представление уровней яркости или цвета в пикселях, чтобы записывать туда биты сообщения.

Выводы

Цифровая стеганография является дисциплиной, изучающей внедрение скрытых сообщений в контейнеры, которые могут являться электронными документами, изображениями и другими файлами. Внедрение и последующее извлечение сообщений происходит посредством стегосистем, которые включают в себя методы первичной обработки и детектирования сообщений в контейнере.

Наибольшее распространение имеет использование изображений в качестве контейнеров. Обусловлено это избыточностью информации и повсеместном использовании изображений в профессиональной, коммуникационной и развлекательной сферах. Стеганографические системы для контейнеров–изображений строятся на основе особенностей системы человеческого зрения.

Различают два класса алгоритмов по области скрытия: области преобразования и пространственные области.

Алгоритмы в пространственной области воздействуют напрямую на цвет и яркость изображения, а алгоритмы в области преобразования прячут информацию в коэффициенты преобразований изображения, что позволяет защитить информацию от искажений при сжатии.

2 АЛГОРИТМИЧЕСКАЯ РЕАЛИЗАЦИЯ

Разработку почтового клиента можно разделить на три отдельных части: установки связи с почтовым сервисом, реализация стеганографических алгоритмов сокрытия информации, реализация криптографического шифрования сообщений (применяется опционально). Исследуем каждую часть отдельно.

2.1 Связь с почтовым сервисом

Связь с почтовым сервисом будет устанавливаться через протоколы SMTP и POP3. Чтобы воспользоваться этими протоколами, потребуется узнать, как установиться соединение со специальными серверами. Каждый протокол отвечает за свою отдельную функцию, и требует особого подхода к реализации.

2.1.1 Протокол SMTP

SMTP (Simple Mail Transfer Protocol — простой протокол передачи почты) — это сетевой протокол, предназначенный для передачи электронной почты в сетях TCP / IP. Формально он используется, как стандарт для отправки почты, с 1982 г., хотя его зарождение началось еще в семидесятых годах двадцатого века. С тех пор вышло много дополнений и расширений к нему, как, например, протокол *ESMTP*, прописанный в спецификации RFC с 2008 г.

Почтовые серверы могут использовать данный протокол как для пересылки, так и для приёма сообщения, но клиентские приложения обычно ограничиваются в использовании SMTP только с целью отправки почты. На рисунке 2.1 представлена схема общения по протоколу SMTP.

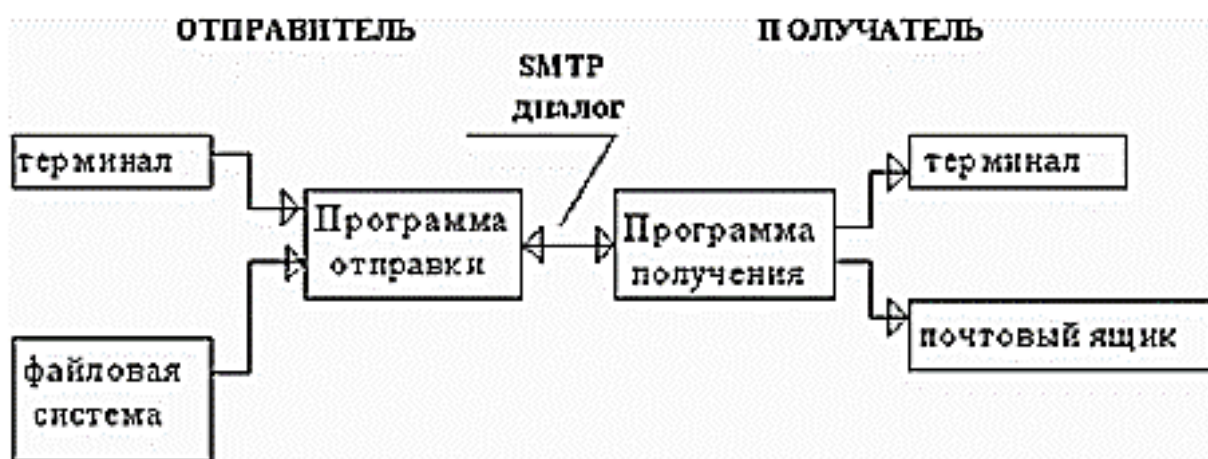


Рисунок 2.1 – Схема взаимодействия по протоколу SMTP

Передача данных по протоколу SMTP осуществляется с помощью текстовых запросов. В них входят следующие команды:

- HELO – команда приветствия, вводится вместе с полным доменным именем клиента, либо же с адресом его электронной почты;
- EHLO–HELO для ESMTP, позволяет использовать расширенный набор команд;
- MAIL – команда, в которой указывается адрес отправителя и адрес возврата письма в случае его неудачной доставки;
- RCPT – команда, в которой указывается адрес получателя письма;
- DATA – команда, в которой происходит передача текста сообщения;
- QUIT – команда, в которой разрывается соединение с сервером;
- HELP – команда, в которой выдаётся на экран список доступных команд;
- VRFY – команда проверки почтового ящика на существование;
- EXPN – команда выдачи списка получателей текущего письма;
- RSET – команда отмены всех переданных данных о письме на сервер.

Соединение с сервером SMTP происходит по конкретному адресу и порту. Обычно настройки SMTP – соединения имеют следующий вид:

- порт – 25, 587, 465 (SSL);
- адрес – smtp. [Имя домена];
- для установления соединения обычно используется протокол SSL.

В разработанном в данном исследовании программном приложении отправка писем осуществляется с помощью встроенных библиотек Microsoft Visual Studio.

2.1.2 Протокол POP3

POP3 (Post Office Protocol Version 3 — протокол почтового отделения, версия 3) это Интернет–протокол прикладного уровня, который используется почтовыми клиентами для извлечения электронных писем с удалённого сервера по TCP–соединению. Альтернативным протоколом с тем же функционалом является протокол IMAP (Internet Message Access Protocol).

Суть работы протокола POP заключается в выгрузке с почтового сервера писем и последующее их удаление, хотя большинство почтовых клиентов может их не удалять и оставить их нетронутыми. Вдобавок, многие почтовые сервисы сами ограничивают возможность удаления писем, оставляя её возможной лишь при прямом вхождении в почтовый ящик.

Сеанс общения в протоколе POP3 имеет три состояния:

- авторизация;
- транзакция;
- обновление.

При авторизации клиент проходит процедуру аутентификации. Во время транзакции, клиенту доступна информация о состоянии почтового ящика и позволяет загрузить или удалить письмо. Во время обновления, сервер удаляет выбранные сообщения и завершает соединение.

Этот протокол включает в себя следующие команды:

- APOP – команда используется для авторизации;
- USER – команда передает серверу имя пользователя;
- PASS – команда передает серверу пароль пользователя;
- DELE – команда отмечает сообщения к удалению;
- LIST – команда выдачи информации о каком-либо конкретном сообщении, либо обо всех сообщениях в почтовом ящике;
- NOOP – команда no operation, операция-пустышка;
- RETR – команда получения сообщения с указанным номером;
- RSET – команда отката транзакции;
- STAT – команда выдает количество сообщений и размер почтового ящика;
- TOP – команда выдает заголовки указанного сообщения и указанное количество первых строк сообщения;
- QUIT – команда завершения сессии.

Соединение с сервером POP3 происходит примерно по тому же принципу, что и соединение с протоколом SMTP. Настройки представлены ниже:

- порт – 110, 995 (SSL);
- адрес – pop3 [Имя домена];
- для установления соединения обычно используется протокол SSL.

В разработанном в данном исследовании программном приложении запрос писем с сервера реализован с помощью программного пакета OpenPop.

2.2 Стеганографические алгоритмы

Электронное письмо имеет несколько потенциально возможных мест для скрытия данных, например, в теме сообщения или в его технических заголовках. В данной работе сокрытие данных будет реализовано в двух местах:

- в тексте (теле) сообщения;
- во вложениях (в виде изображения .bmp);

Этот выбор обоснован весьма большой вместимостью данных контейнеров и невозможностью обнаружения факта внедрения информации без использования специальных технических средств. Каждый из описанных вариантов требует предусмотреть некоторые нюансы, связанные с работой почтовых сервисов и последующим извлечением скрытого сообщения из контейнера [72].

2.2.1 Соккрытие информации в тексте сообщения

Текст (тело) электронного сообщения – это непосредственный текст письма, который выводится почтовыми программами как содержимое сообщения. Имеется множество вариантов скрытия информации в тексте, многие из которых зависят от содержимого самого письма. Также следует учесть тот факт, что некоторые письма могут содержать в себе HTML-код в качестве текста, что требует особого подхода к его обработке. В целях наибольшей универсальности и вместительности, был продуман следующий *алгоритм внедрения информации* в сообщение [73].

Шаг 1. Исследуем контейнер C на предмет наличия места для внедрения скрытого сообщения M . В качестве подходящих мест выбираются буквенные символы, имеющие одинаковый вид написания как в латинском, так и в кириллическом алфавите, т.е. символы типа: A, C, y, o и т.д.

Шаг 2. Сохраним найденные позиции в массиве P для дальнейшего применения.

Шаг 3. Преобразуем скрываемое сообщение M в последовательность бит M_b .

Шаг 4. Количество (длину) M_b присоединяем непосредственно к началу M_b в виде битовой последовательности фиксированной длины (4 байта).

Шаг 5. Создаем копию контейнера C , обозначив её как C_f . Начинаем процедуру внедрения скрытого сообщения. Выбор символа замены производится по формуле:

$$\begin{cases} C_f[P[i]] = \text{Latin}(C[P[i]]), & \text{если } M_b[i] = 1 \\ C_f[P[i]] = \text{Cyrillic}(C[P[i]]), & \text{если } M_b[i] = 0 \end{cases};$$

где $\text{Latin}(x)$ – символ латиницы, имеющий идентичный внешний вид с символом x , но, возможно, отличающийся кодовым значением; $\text{Cyrillic}(x)$ – символ кириллицы, имеющий идентичный внешний вид с символом x , но, возможно, отличающийся кодовым значением. Параметр i находится в диапазоне $[0, \text{Length}(M_b)]$.

Шаг 6. Сохраняем заполненный контейнер C_f .

Этот метод позволяет внедрить достаточно большое количество информации, так как в список взаимозаменяемых символов входит 11 прописных и 7 строчных букв, что составляет весьма значимую долю обоих алфавитов. Стоит заметить, что для сообщений, написанных буквами латинского алфавита, вместимость будет статистически выше, ввиду меньшей длины самого алфавита.

Извлечение сообщение происходит схожим образом, как и внедрение.

Шаг 1. Исследуем заполненный контейнер C_f на предмет наличия мест для внедрения скрытого сообщения. В качестве подходящих мест выбираются буквенные символы, имеющие одинаковый вид написания как в латинском, так и в кириллическом алфавите, т.е. буквы типа A, C, y, o и т.д.

Шаг 2. Сохраняем найденные позиции в массиве P для дальнейшего применения.

Шаг 3. Извлекаем длину скрытого сообщения (при фиксированной длине в 4 байта – первые 32 бита). Делается это по следующей формуле:

$$\begin{cases} L[i] = 1, \text{ если } C_f[P[i]] \in A_L; \\ L[i] = 0, \text{ если } C_f[P[i]] \in A_C; \end{cases}$$

где A_L – латинский алфавит, а A_C – кириллический алфавит.

После применения этого алгоритма необходимо перевести найденную битовую последовательность в числовое значение L .

Шаг 4. Извлекаем, по той же схеме, оставшиеся биты сообщения, до тех пор, пока их количество не превысит значения L . Затем необходимо преобразовать его в восстановленное электронное сообщение M .

Скрываемое сообщение M может являться как текстом, так и непосредственно последовательностью байтов, представляющую какой-либо файл. Именно поэтому для полноценного восстановления сообщения необходимо присоединять к нему значение его длины, т.к. если ёмкость контейнера превышает длину сообщения, то при восстановлении будут записаны лишние байты, что может привести к его повреждению. Существует различные методы обозначения конца чтения потока информации для данной ситуации, но обозначение его длины является одним из наиболее надёжных вариантов для сохранности данных [74].

2.2.2 Соккрытие информации в изображении

В разработанной программе реализовано соккрытие секретной информации в изображении с помощью метода LSB (Least Significant Bit). В данной реализации метода производится замена только последнего бита каждого из цветов RGB без каких-либо условий отбора, как, например, контрастности, близости к краю и т.п. Сила данного метода заключается в его скрытности – человеческий глаз не способен разглядеть разницу между цветами в 1 бит, тем более, если при этом используется насыщенное различными объектами и цветами изображение, то обнаружить наличие тайно вложенного сообщения невозможно без использования специальных средств стеганографического анализа [75].

Поскольку данный алгоритм производит соккрытие данных в пространственной области, то его показатели *робастности* (свойства статистического метода, характеризующего независимость влияния на результат исследования различного рода выбросов, устойчивость к помехам) оставляют желать лучшего. Это значит, что заполненный контейнер уязвим к сжатию с потерями, что приведет к потере информации после перевода в некоторые другие форматы изображений [76].

Пример пустого и заполненного контейнеров, где применён метод сокрытия информации в изображении, продемонстрирован на рисунках 2.2 и 2.3.



Рисунок 2.2 – Пустой контейнер



Рисунок 2.3 – Заполненный контейнер.

Стеганографическое сообщение – «Здесь никого нет, кроме меня»

Алгоритм внедрения информации состоит из следующих шагов:

Шаг 1. Исследуем контейнер C как байтовую последовательность. Определяем область, содержащую информацию о цветовых значениях пиксела (обычно такая область занимает 54 байта от начала файла).

Шаг 2. Преобразуем скрываемое сообщение M в последовательность бит M_b .

Шаг 3. Количество (длину) M_b присоединяем непосредственно к началу M_b в виде битовой последовательности фиксированной длины (4 байта).

Шаг 4. Начинаем процедуру внедрения скрытого секретного сообщения. Замена байта происходит по следующей формуле:

$$C[i] = C[i] \wedge (2^7 + M_b)$$

где $\wedge$ – операция побитового умножения.

Шаг 5. Сохраняем получившуюся последовательность символов C , с внедрённым скрытым секретным сообщением, как заполненный контейнер C_f .

Алгоритм извлечения информации состоит из следующих шагов.

Шаг 1. Исследуем контейнер C как байтовую последовательность. Определяем область, содержащую информацию о цветовых значениях пиксела (такая область обычно занимает 54 байта от начала файла).

Шаг 2. Извлекаем длину скрытого сообщения (при фиксированной длине в 4 байта – первые 32 бит). Делается это по следующей формуле:

$$L_b[i] = C_f[i] \wedge 1,$$

где L_b – битовая последовательность длины сообщения.

Шаг 3. Извлекаем по той же схеме оставшиеся биты сообщения, до тех пор, пока их количество не превысит L . Затем преобразуем его в восстановленное сообщение M .

2.3 Криптографическое шифрование сообщений

Шифрование сообщения реализовано с помощью алгоритма симметричного блочного шифрования AES (Advanced Encryption Standard) в режиме шифрования CBC (Cypher–Block Chaining). Программная реализация разрабатываемого приложения выполнена с помощью встроенных библиотек Microsoft Visual Studio.

Параметры алгоритма AES:

- размер ключа: 128, 192, 256 бит;
- размер блока: 128 бит;
- число раундов: 10, 12, 14 (зависит от размера ключа).

Ход выполнения алгоритма имеет следующий вид:

- AddRoundKey() над исходным блоком данных;
- SubBytes() => ShiftRows() => MixColumns() => AddRoundKey() для каждого раунда, кроме последнего раунда шифрования алгоритма AES;
- SubBytes() => ShiftRows() => AddRoundKey() для последнего раунда.

Операция SubBytes() обрабатывает каждый байт состояния, независимо производя нелинейную замену байтов, используя таблицу замен. Такая операция обеспечивает нелинейность алгоритма шифрования. Для построения данной таблицы необходимо взять обратного числа в поле Галуа $GF(2^8)$ и применить к каждому байту b таблицы замен следующую операцию:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i,$$

где $0 \leq i \leq 7$, и где b_i является i -м битом b , а c_i является i -й битом константы 99.

Операция ShiftRows() подразумевает под собой сдвиг строк по горизонтали, зависящий от их позиций: нулевая строка остается неизменной, первая сдвигает на одну позицию, вторая на две и так далее.

В операции MixColumns() каждая колонка представляется в виде полинома третьей степени и перемножается на фиксированный полином:

$$3x^3 + x^2 + x + 2 \text{ по модулю } x^4 + 1.$$

Операция AddRoundKey() объединяет каждый ключ раунда с состоянием. Делается это путём побайтового выполнения операции XOR (исключающее ИЛИ) соответствующих байтов позиции и ключа раунда.

Каждый раундовый ключ имеет длину 128 бит или 4 четырехбайтовых слова $w_i, w_{i+1}, w_{i+2}, w_{i+3}$, а длина в битах всех раундовых ключей равна 128 бит. Первые четыре слова в ключевом массиве заполнены ключом шифра, из остальных выработанных 40 слов выбираются по 4 слова для ключа раунда. Каждая четверка ключей является ключом раунда (первые четыре слова совпадают с ключом шифра и являются ключом с номером 0).

Вычисление слов $w_{i+4}, w_{i+5}, w_{i+6}, w_{i+7}$ происходит на основе предыдущих, с помощью следующих уравнений:

$$w_{i+5} = w_{i+4} \oplus w_{i+1}; \quad w_{i+6} = w_{i+5} \oplus w_{i+2}; \quad w_{i+7} = w_{i+6} \oplus w_{i+3}.$$

При этом w_{i+4} вычисляется следующим образом:

$$w_{i+4} = w_i \oplus g(w_{i+3}),$$

где функция g выполняет: циклический сдвиг слова на один байт; замену каждого байта полученного слова в соответствии с таблицей замен; суммирование байтов по модулю 2 с раундовой постоянной $(2^i, 0, 0, 0)$, где i – номер раунда.

Выводы

Для полноценной реализации программного приложения было разработано несколько модулей с собственным функционалом:

- модуль связи с почтовым сервисом, для отправки и приема электронной почты со всеми приложенными вложениями;
- модуль стеганографической обработки информации, состоящий из кодера и декодера, которые взаимодействуют как с текстом, так и с файлами;
- модуль криптографического шифрования информации, обеспечивающий конфиденциальность информации в случае обнаружения и восстановления сообщения сторонними лицами.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Выбор операционной системы и языка программирования играет важную роль при реализации приложения и его дальнейшей поддержке.

На основе оценки компании Net Applications за 2018 г., занимающейся сбором статистических данных об использовании операционных систем, наибольшая доля их использования приходится на Microsoft Windows и составляет 87,27%. На основе этих данных был сделан выбор платформы Windows в качестве целевой платформы для приложения. Наибольшей популярностью, среди современных языков программирования этой ОС, являются Java, Python и C#. При выборе языка программирования необходимо проанализировать достоинства и недостатки каждого из них.

Java – кроссплатформенный язык, транслирующий написанную программы в байт–код, исполняемый виртуальной машиной Java на поддерживающей её платформе. Имеет большой объём стандартных библиотек. Недостатком является низкое быстродействие и повышенные требования к объёму оперативной памяти.

Python является интерпретируемым языком, имеющим большое число модулей, подключаемых к программе, и выполняющий код после предварительной компиляции в байт–код. Низкое быстродействие программ, требовательных к ОС, может сделать его выбор непривлекательным для реализации сложных систем.

В целях разработки нового импортозамещающего программного средства для кодирования и декодирования скрытых сообщений в электронной почте был выбран язык программирования C#. Данный язык обладает собственной средой разработки Visual Studio со встроенным менеджером пакетов NuGet и удобными средствами разработки графического интерфейса, такими как WPF (Windows Presentation Foundation) и Windows Forms. Является полностью объектно–ориентированным языком, включая элементарные типы данных. Этот язык программирования обладает всеми нужными библиотеками для реализации почтового клиента.

Среди них особо важны [77]:

- библиотека System.Net.Mail, используется для отправки, обработки писем и установления связи с сервером по SMTP–протоколу;
- библиотека OpenPop, используется для извлечения писем с почтового сервера, устанавливает связь по протоколу POP3;
- библиотека Microsoft.Win32, позволяет использовать удобные диалоговые окна для взаимодействия с файловой системой компьютера;
- библиотека System.Security.Cryptography, используемая для реализации различных алгоритмов шифрования под любую операционную систему.

Графический интерфейс приложения описан с помощью WPF, использующий язык разметки XAML для описания элементов. В него включено множество готовых элементов управления, которые могут понадобиться в ходе написания программы для обеспечения удобного взаимодействия между пользователем и приложением. При необходимости, этот набор элементов можно пополнить как самостоятельно, так и применив сторонние библиотеки.

3.1 Описание структуры программы

Код программы написан в соответствии с принципами объектно-ориентированного программирования. Разные модули программы разбиты на отдельные классы и вызываются по мере необходимости в ходе выполнения. В программе используются отдельные классы как для отображения окон, так и для выполнения функций по скрытию и шифрованию информации. UML-диаграмма структуры классов представлена на рисунке 3.1.

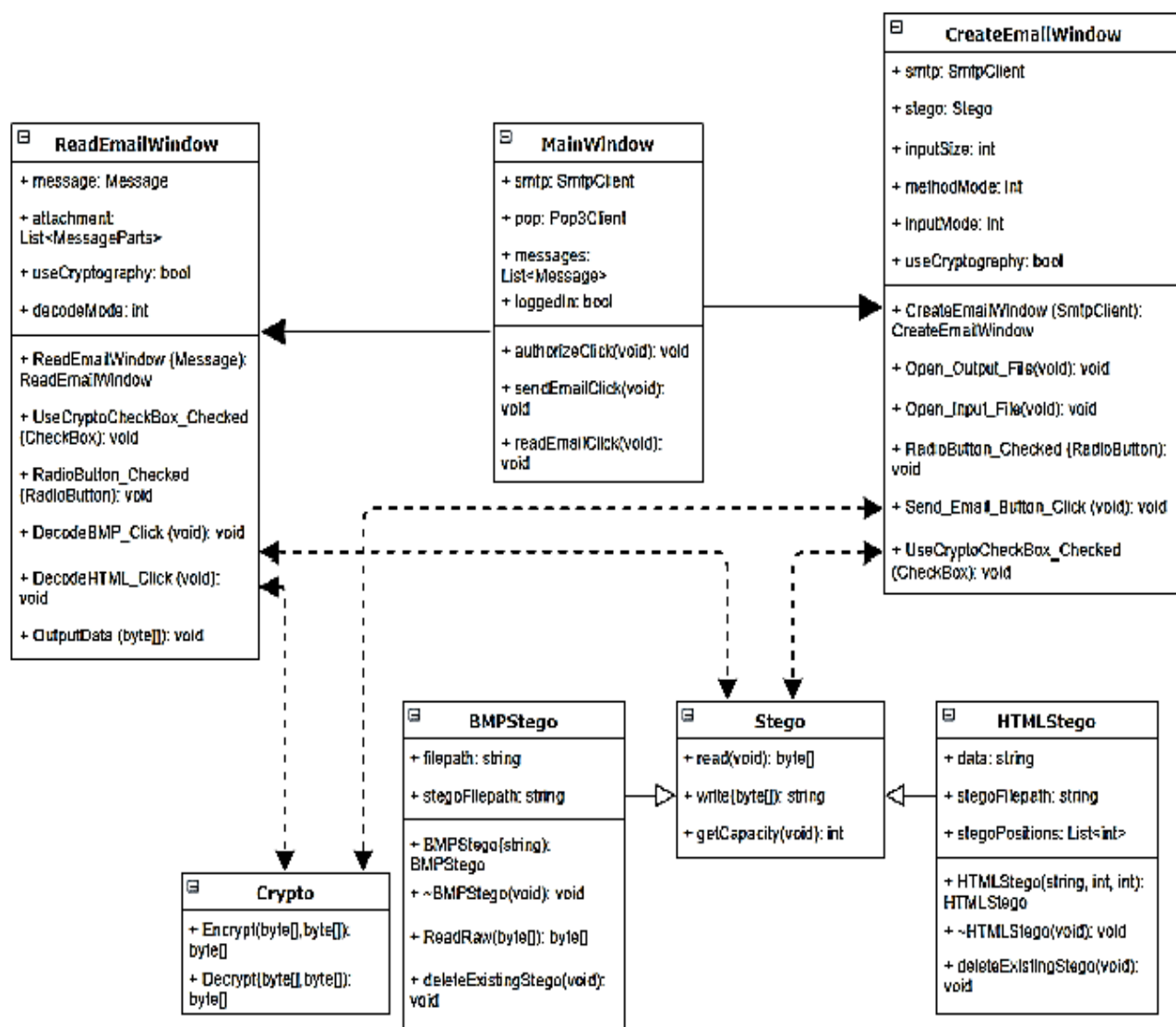


Рисунок 3.1 – UML-диаграмма

3.2 Реализация модулей программы

В главном окне программы осуществляется авторизация пользователя и выбор последующих действий. Оно является открытым в течение всего времени работы программы, что позволяет одновременно производить как отправку, так и чтение электронных писем. Также, в нем происходит первоначальная связь клиента с почтовым сервером и отображение входящих сообщений электронной почты. Алгоритм связи по протоколам IMAP и POP3 представлен на рисунках 3.2 и 3.3.

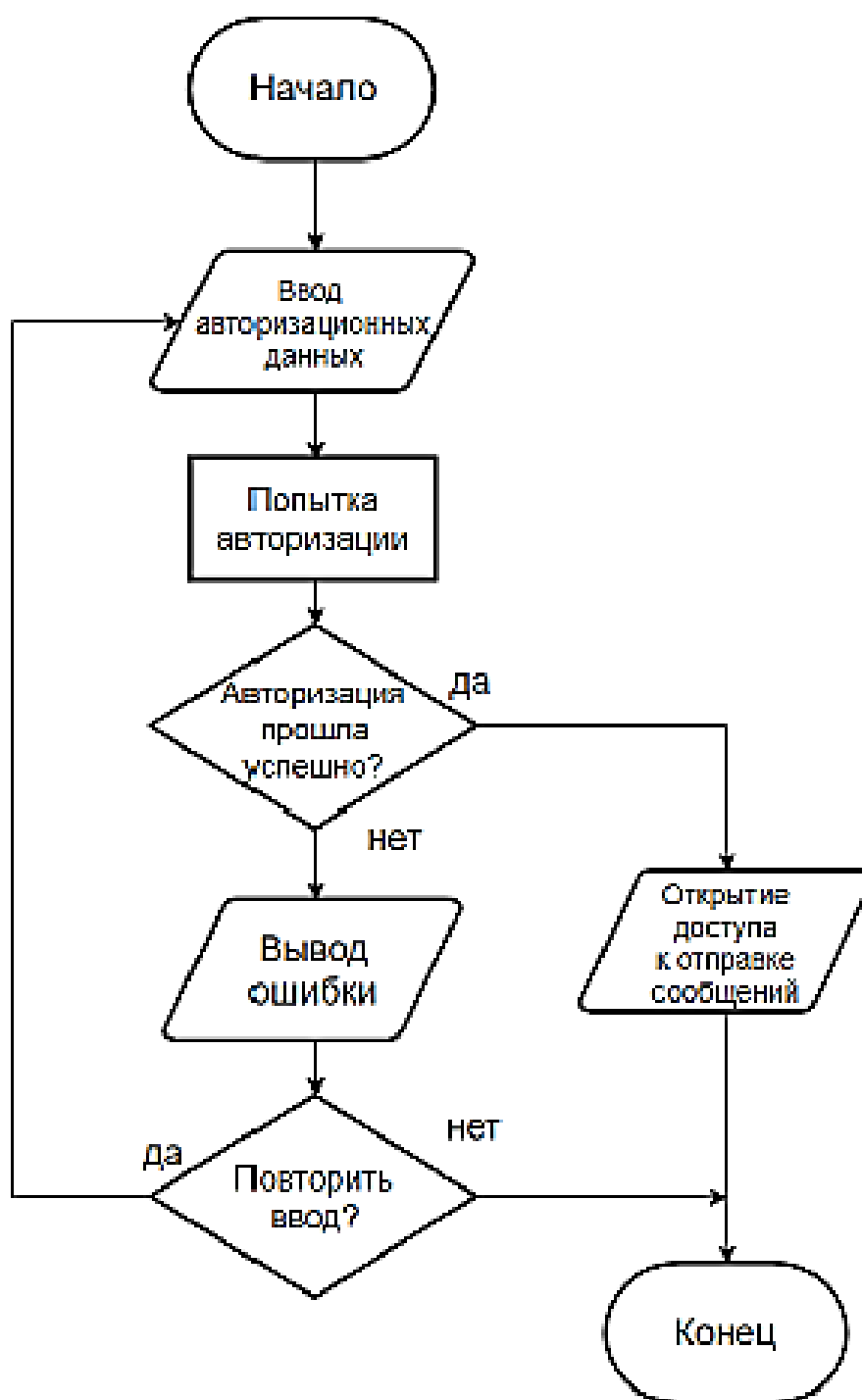


Рисунок 3.2 – Алгоритм соединения с почтовым сервером по протоколу SMTP



Рисунок 3.3 – Алгоритм соединения с почтовым сервером по протоколу POP3

Преобразование, внедрение и извлечение стеганографических сообщений происходит с помощью абстрактного класса Stego. Из него наследуются 2 класса:

- HTMLStego предназначен для контейнеров в виде текстовых или html-файлов;
- BMPStego – предназначен для контейнеров, являющихся bmp-изображениями.

Общий алгоритм работы этих двух классов [78] можно представить в виде блок – схем, продемонстрированных на рисунках 3.4 и 3.5.

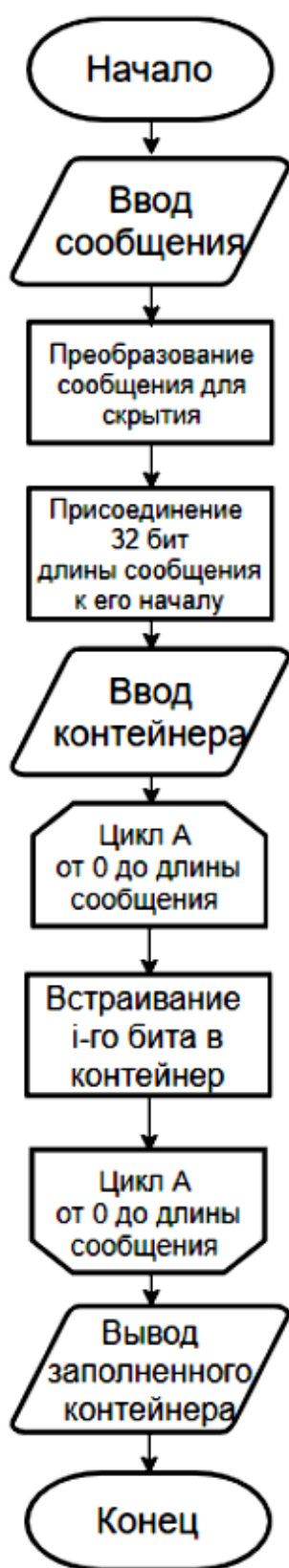


Рисунок 3.4 – Алгоритм внедрения стегосообщения

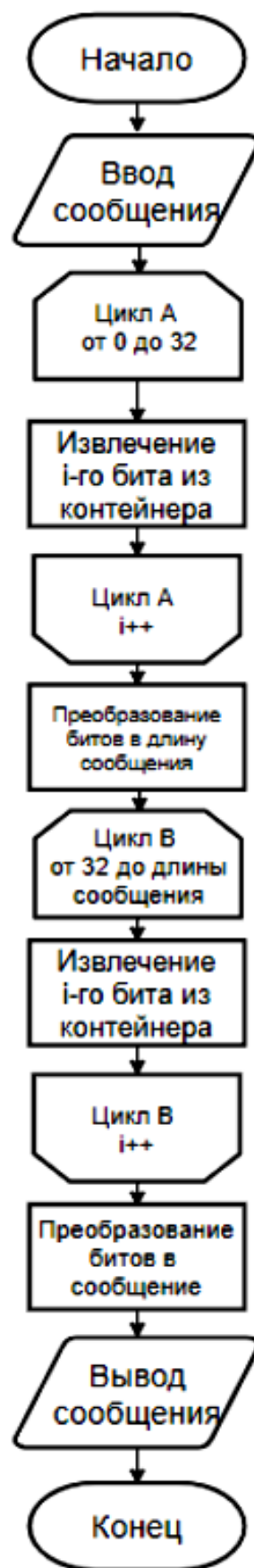


Рисунок 3.5 – Алгоритм извлечения стегосообщения

Классы ReadEmailWindow и CreateEmailWindow занимаются соответственно чтением и передачей электронных писем. В ходе их работы используются описанные выше функции стеганографии, а также, при необходимости, криптографическое шифрование [79]. Алгоритм работы представлен на рисунках 3.6 и 3.7.



Рисунок 3.6 – Алгоритм отправки электронного сообщения

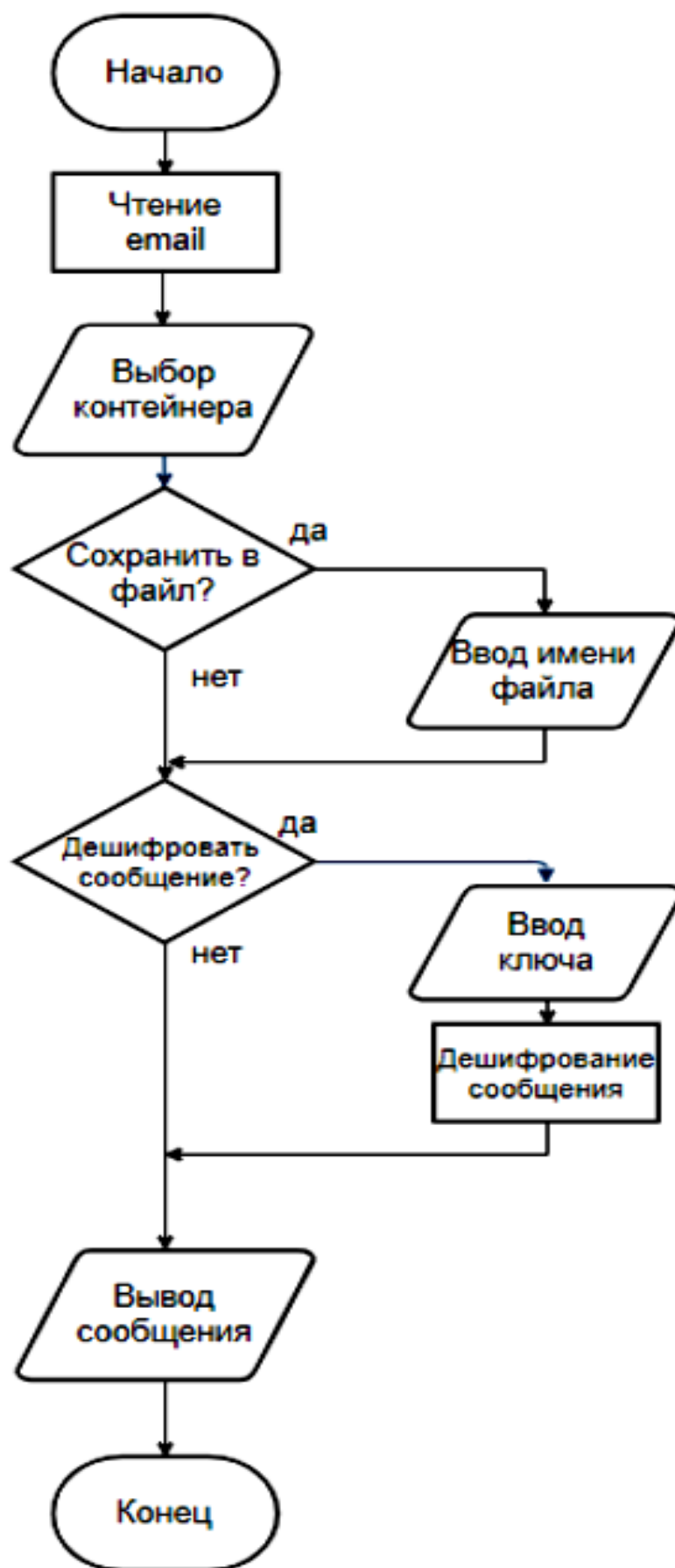


Рисунок 3.7 – Алгоритм чтения электронного сообщения

3.3 Демонстрация работы приложения

Работа пользователя с программой начинается с авторизации. В качестве аутентификационных данных используются адрес электронной почты и пароль, как показано на рисунке 3.8. После нажатия на кнопку «Войти», программа производит попытку подключения к почтовому серверу. Если подключение произошло успешно, то появляется надпись об успешном входе, показываются последние входящие письма и появляется возможность прочитать или отправить письмо, как показано на рисунке 3.9.

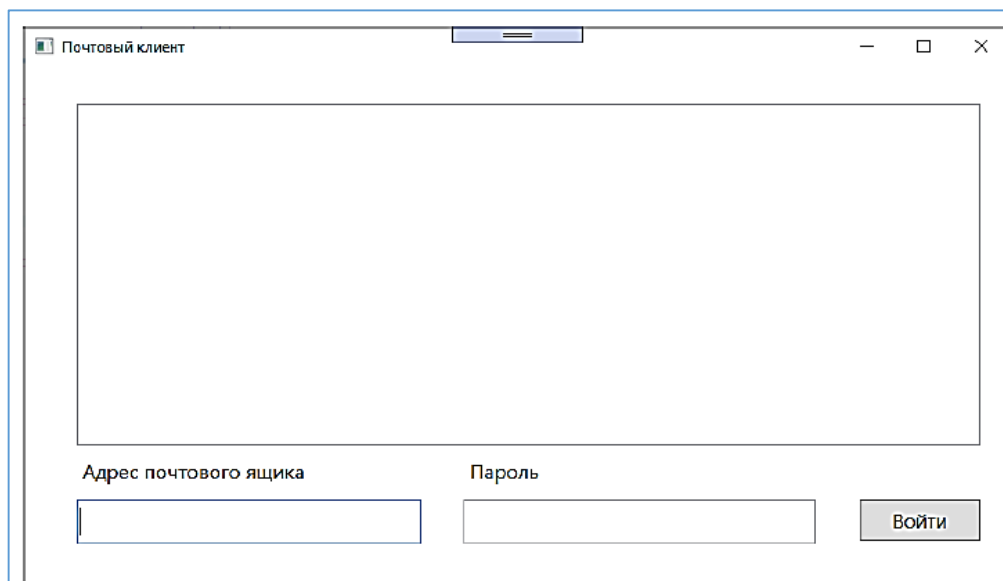


Рисунок 3.8 – Главное окно программы при входе

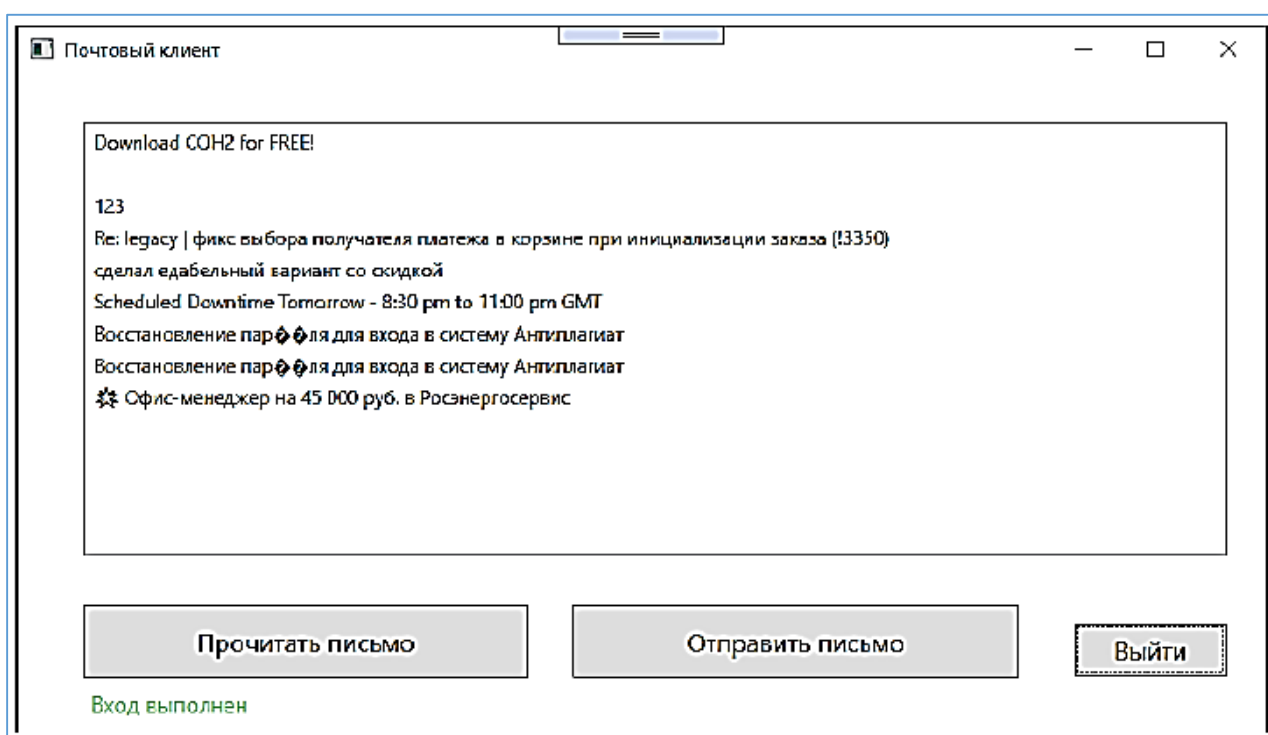


Рисунок 3.9 – Главное окно программы после авторизации

При неудачном входе, программа оповещает об ошибке, и пользователь может снова попробовать ввести свои данные как показано на рисунке 3.10.

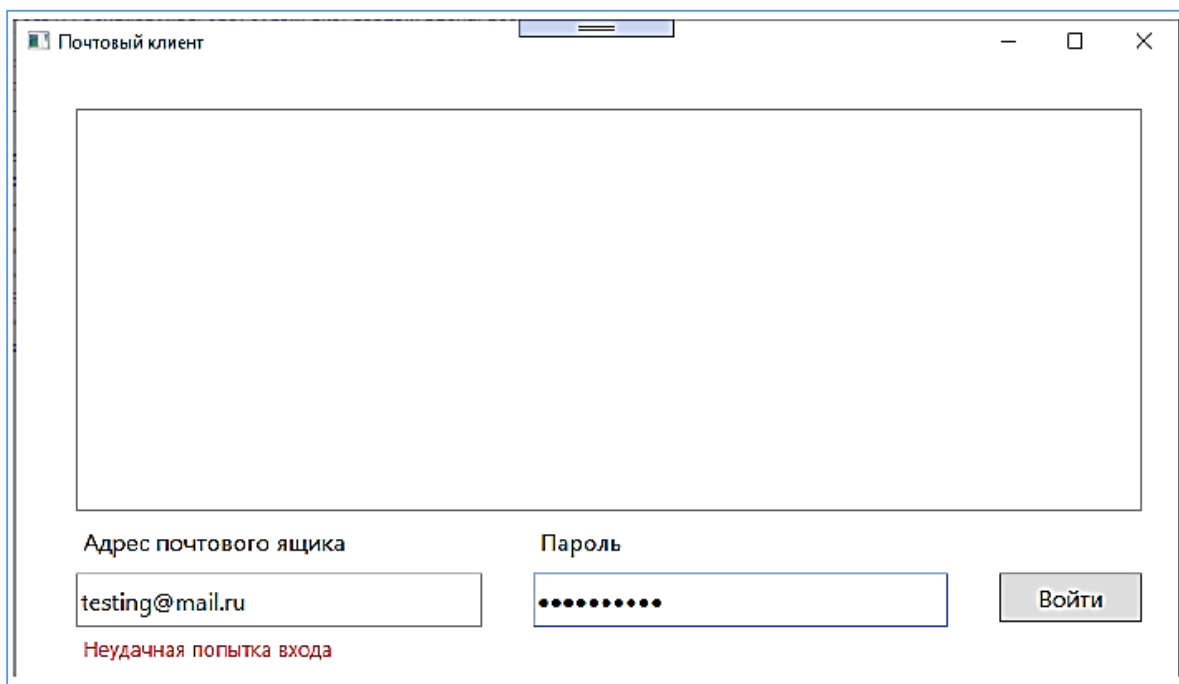


Рисунок 3.10 – Главное окно программы после неудачной попытки входа

После авторизации, в любой момент можно произвести выход из своего почтового ящика. После этого можно снова зайти на тот же ящик, либо ввести какой-нибудь другой, как показано на рисунке 3.11.

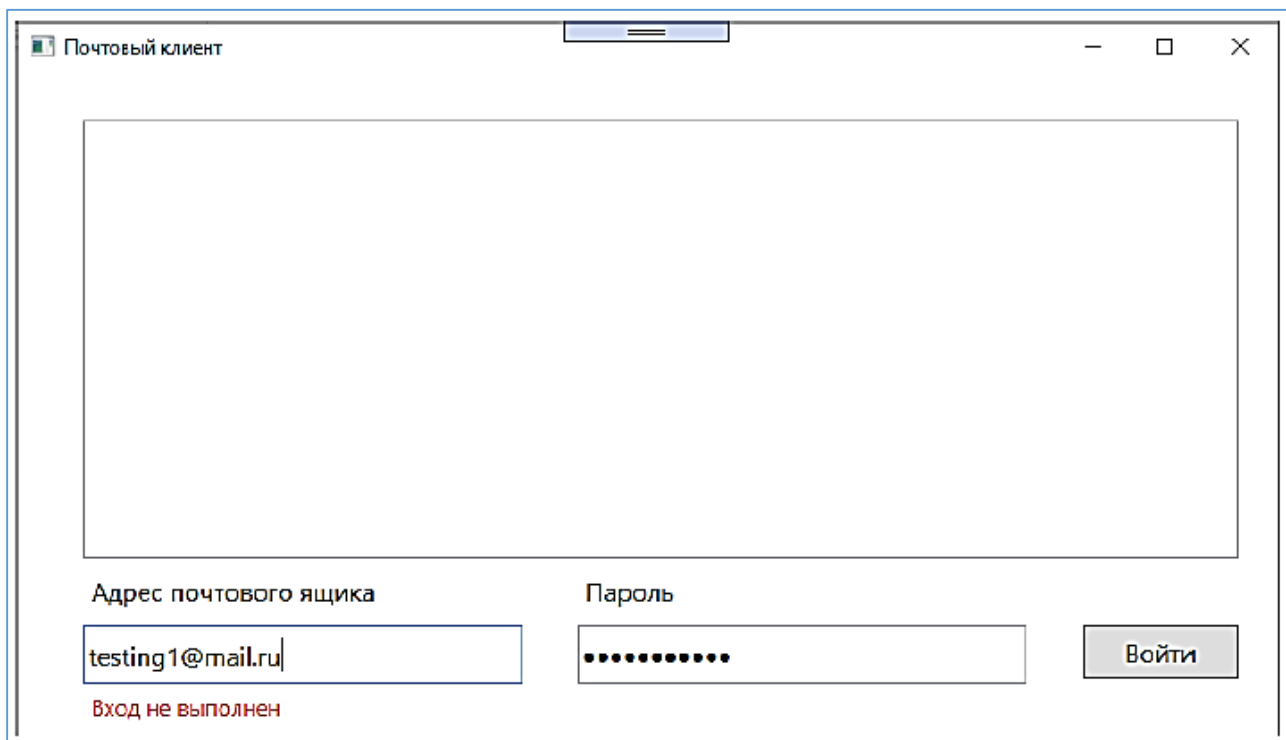


Рисунок 3.11 – Главное окно программы после выхода

После авторизации, можно отправить электронное письмо адресату. Чтобы открыть окно отправки почты, необходимо нажать на кнопку «Отправить письмо». Окно отправки письма показано на рисунке 3.12.

Рисунок 3.13 – Окно отправки письма

В процессе отправки письма пользователь может как заполнить стандартные поля в письме, такие, как тема и непосредственное сообщение, так и указать, в какую часть письма будет внедряться скрытое сообщение (в текстовую или графическую), а также выбрать способ ввода сообщения: вручную или из файла.

При выборе нужного файла открывается диалоговое окно проводника и отображаются допустимые файлы, как показано на рисунке 3.14. Разрешённые форматы файлов зависят от метода внедрения [80]:

- для BMP соответственно только файлы с расширением .bmp;
- для HTML можно использовать как файлы .html, так и .txt.

В случае, если сообщением является файл, то для него допустимы любые расширения, так как в любом случае этот файл с сообщением будет преобразован в последовательность байтов.

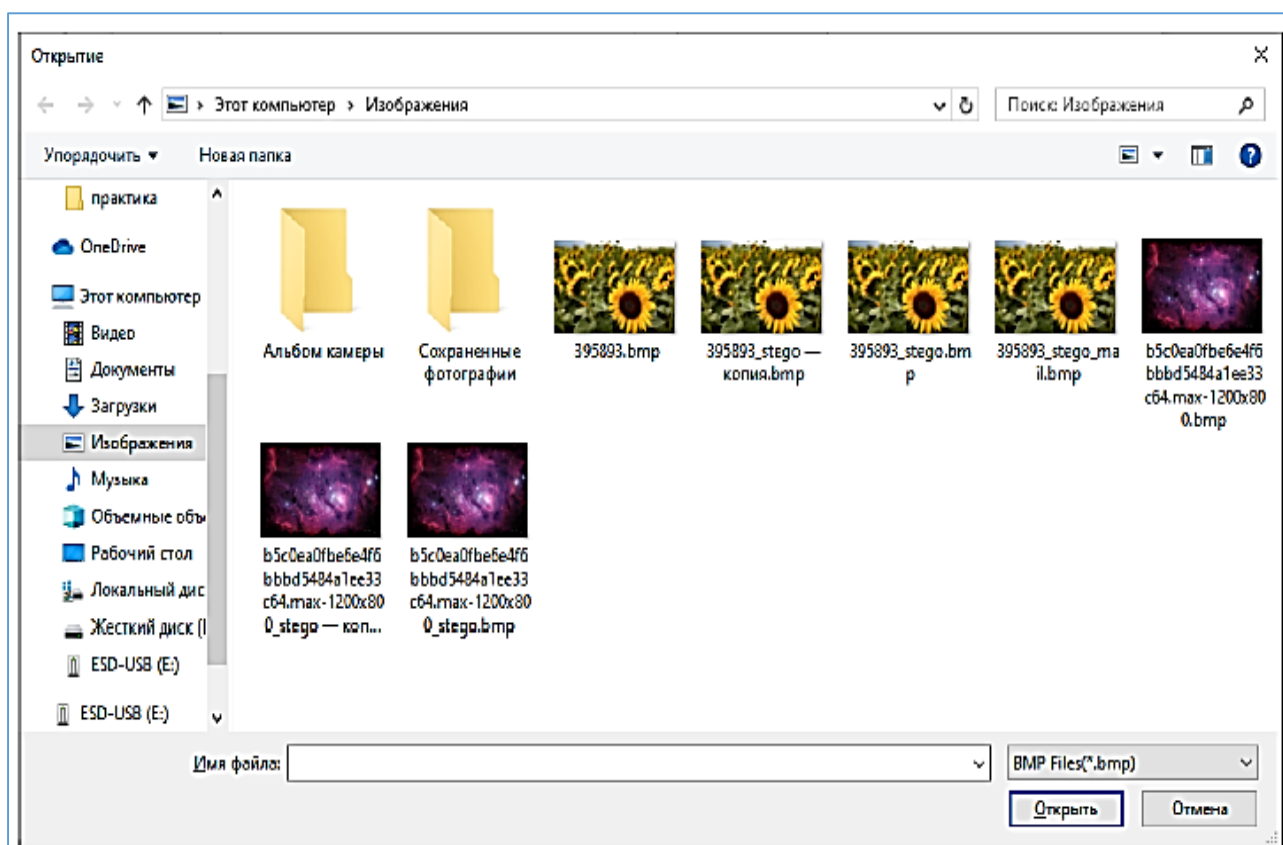


Рисунок 3.14 – Диалоговое окно выбора файла–контейнера

После выбора всех необходимых параметров можно совершить отправку письма. Пример заполненного окна отправки показан на рисунке 3.15.

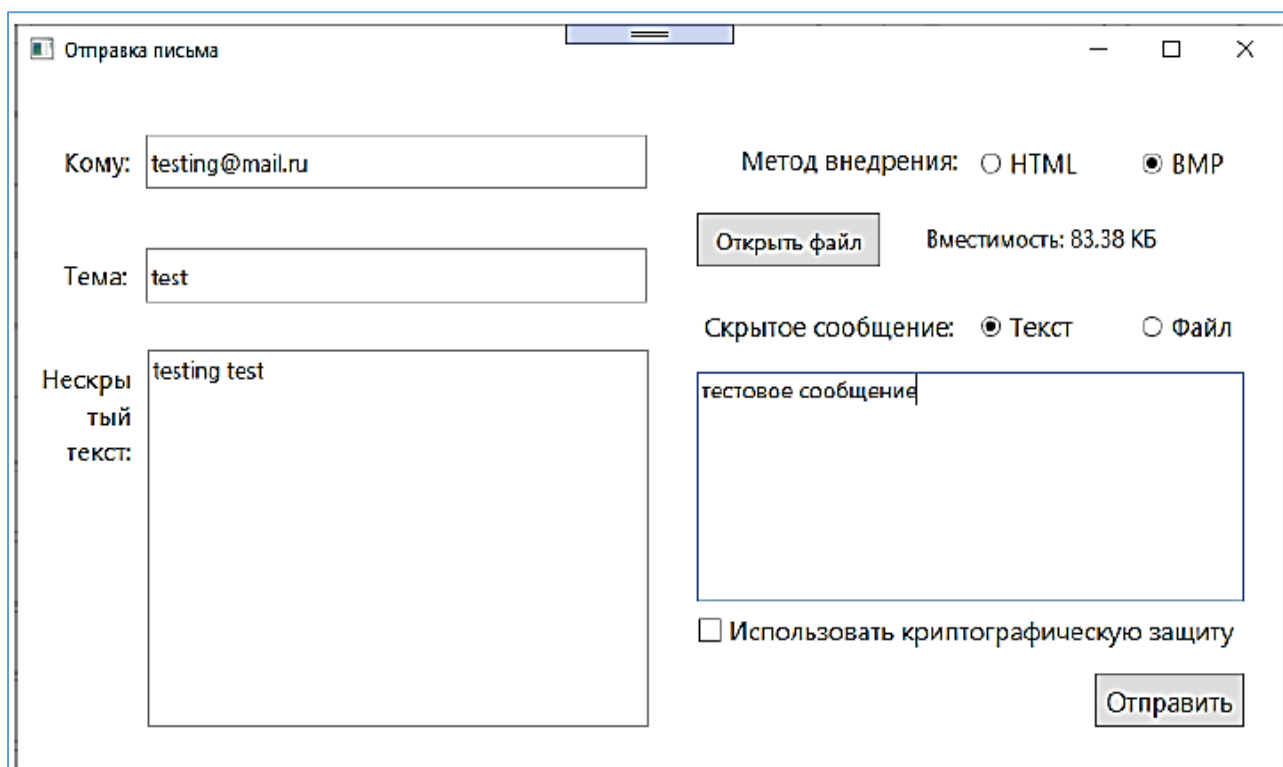


Рисунок 3.15 – Заполненное окно отправки электронного письма

При желании, пользователь может зашифровать сообщение с помощью шифра AES. Для этого необходимо отметить галочку у надписи «Использовать криптографическую защиту» и ввести ключ шифрования. Пример показан на рисунке 3.16. Ключ вводится в виде hex-строки длиной 32, 48 или 64 символа.

При последующем декодировании сообщения понадобится использовать этот же ключ, чтобы выполнить обратный процесс – дешифрование.

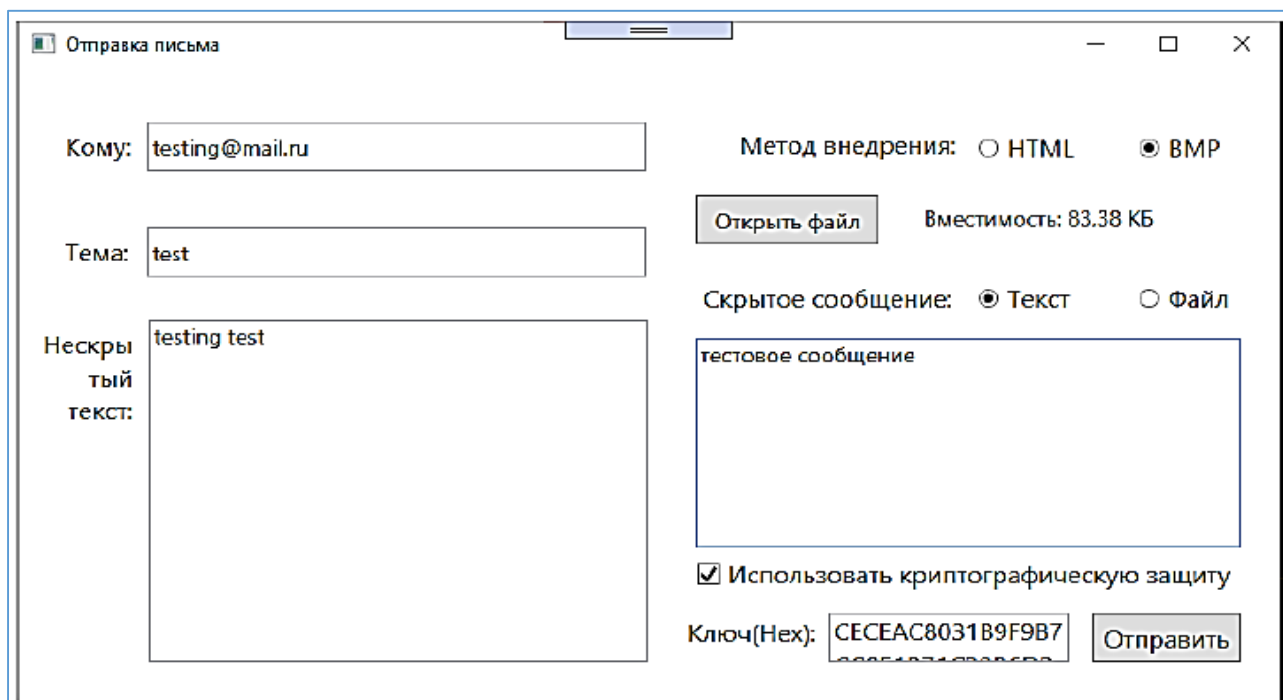


Рисунок 3.16 – Заполненное окно отправки электронного письма с включенной криптографической защитой сообщения

После нажатия на кнопку «Отправить», будет совершена попытка отправить письмо. Если попытка прошла успешно, то появится соответствующее окно с сообщением, показанное на рисунке 3.17. Если же какие-то из данных были введены неверно, то всплывает сообщение об ошибке, как показано на рисунке 3.18.

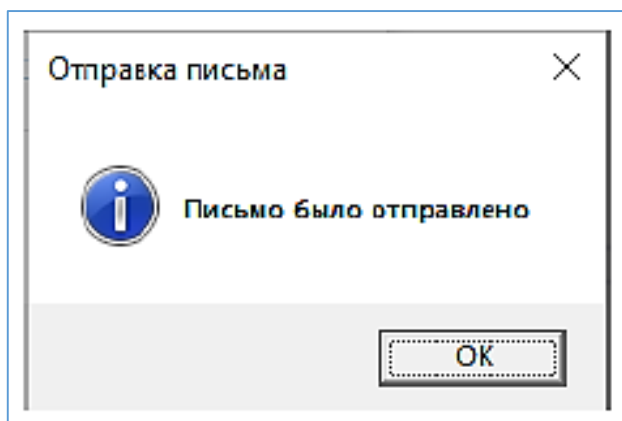


Рисунок 3.17 – Сообщение об успешной отправке письма

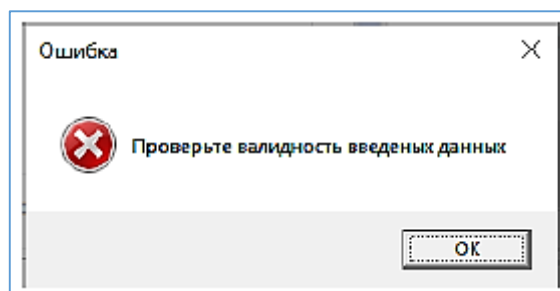


Рисунок 3.18 – Сообщение о неверно введенных данных

Отправленное письмо можно просмотреть в почтовом ящике адресата, как показано на рисунке 3.19. Среди вложений можно найти контейнер, в котором спрятано сообщение. На рисунках 3.20 и 3.21 можно сравнить изображения до и после внедрения стегосообщения. Различить разницу практически невозможно.

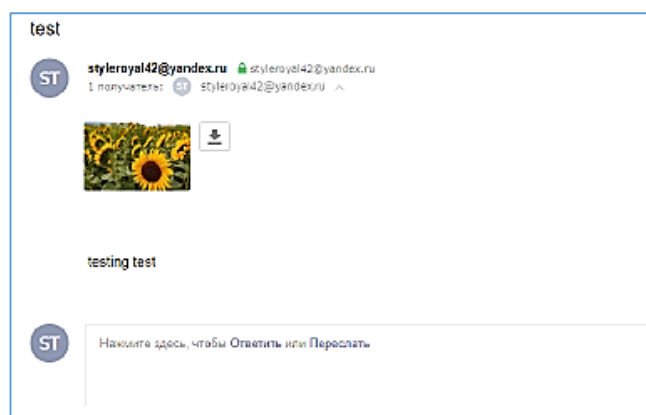


Рисунок 3.19 – Оправленное письмо в почтовом ящике



Рисунок 3.20 – Пустой контейнер



Рисунок 3.21 – Заполненный контейнер

После отправки письма со спрятанным сообщением можно продолжить работу с окном отправки, либо же вернуться к главному окну.

Окно чтения письма становится доступно после авторизации в главном окне. Вызывается оно после выбора письма из списка в верхней части окна и нажатия на кнопку «Прочитать письмо». На рисунке 3.21 представлен внешний вид окна.

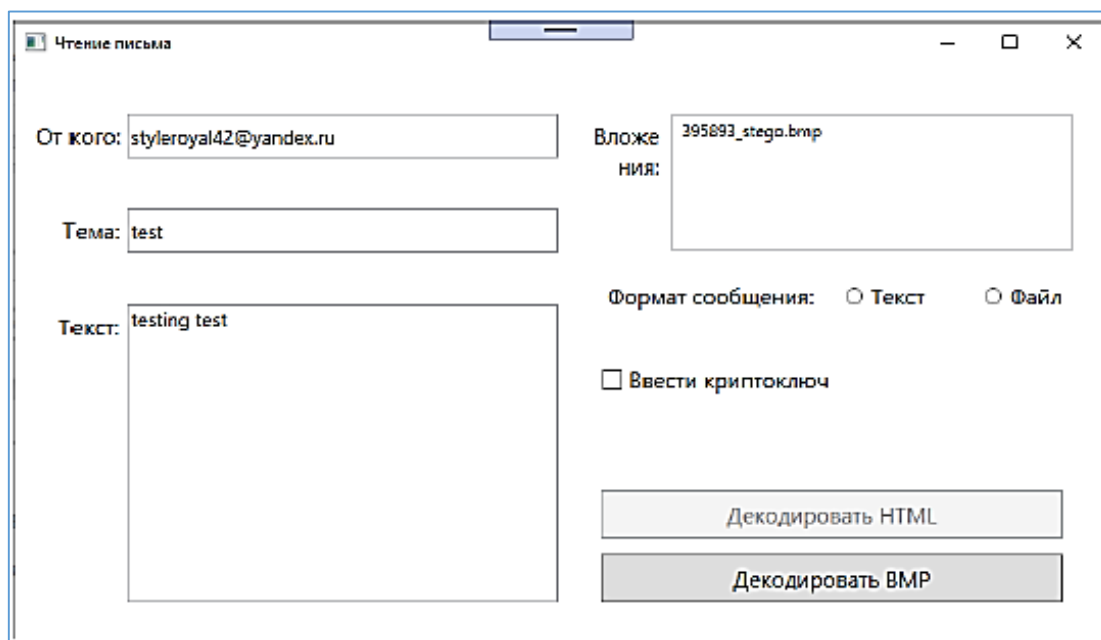


Рисунок 3.21 – Окно чтения письма

В соответствующих полях отображается информация о письме. При возможности декодирования сообщения открывается доступ к кнопкам.

ВМР–декодирование доступно в случае, если среди вложений письма есть файл формата .bmp, а HTML–декодирование доступно если тело письма помечено как HTML–код. Проверки на наличие стеговложения при этом не производится, поэтому эти действия будут доступны и для обычных писем, если удовлетворяются вышеописанные условия. Декодированное сообщение можно выводить как текст, либо записать в файл. Примеры вывода показаны на рисунках 3.22 и 3.23.

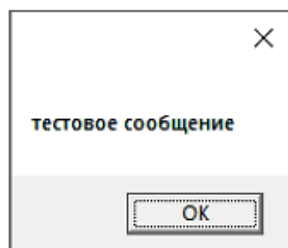


Рисунок 3.22 – Вывод стегосообщения в окно

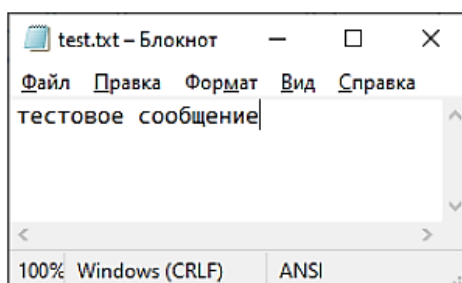


Рисунок 3.23 – Запись стегосообщения в файл

Если сообщение было зашифровано с помощью алгоритма AES, а пользователь не сможет ввести нужный криптографический ключ, то сообщение будет восстановлено неверно, как показано на рисунке 3.24.

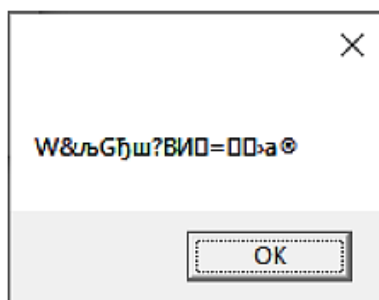


Рисунок 3.24 – Вывод зашифрованного сообщения «тестовое сообщение» без использования криптографического ключа

Выводы

В этом разделе был обоснован выбор языка программирования и целевой платформы для ПО. Была описана структура классов и их взаимоотношения, и проведена демонстрация работы программного приложения на всех этапах работы.

4 ТЕСТИРОВАНИЕ МОДУЛЕЙ ПРОГРАММНОГО СРЕДСТВА

В этом разделе будут рассмотрены контрольные примеры работы программного средства. Всего будет произведено два теста с различными исходными данными. Алгоритм выполнения программы в обоих случаях представлен на рисунке 4.1.

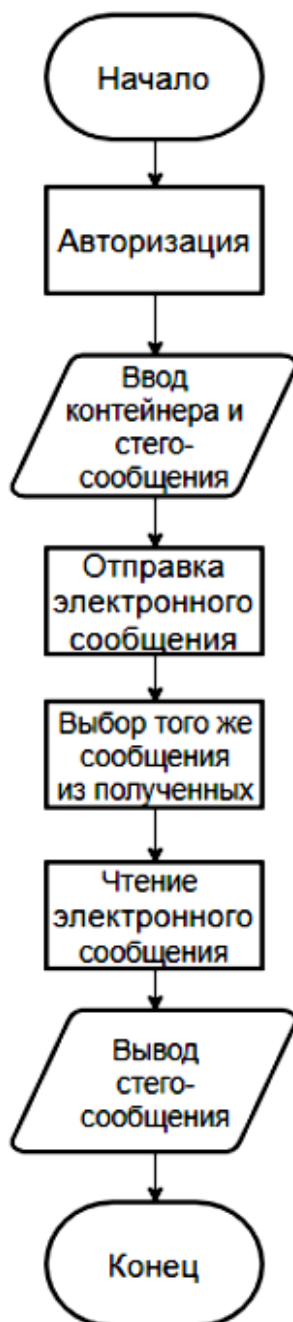


Рисунок 4.1 – Алгоритм работы с программой

Исходные данные для теста №1:

- контейнер: test.html (рисунок 4.2);
- скрываемое сообщение: “Тестирование ПО”;
- криптографическая защита: не используется.

Полезный подарок!

Здравствуй, Александр Викторович!

Сегодня хотим рассказать о необычной книге «[Путь в программисты](#)».



Автор книги — Константин Шереметьев, 7 лет работал программистом в РКК «Энергия», затем 7 лет преподавал программирование в университете МГУЛ, параллельно программируя на заказ.

Десятки благодарных студентов Константина Шереметьева работают программистами и получают зарплату **от 100 000 рублей** и выше.

Ответы на 12 самых важных вопросов, которые возникают у людей о программировании, вы найдете в мини-книге «[Путь в программисты](#)».

Получите книгу на свой емэйл здесь:

<https://sheremetev.aoserver.ru/?r=ac&id=6434&lg=ru>

Рисунок 4.2 – Отрывок test.html

Результаты теста представлены на рисунках 4.3 – 4.6

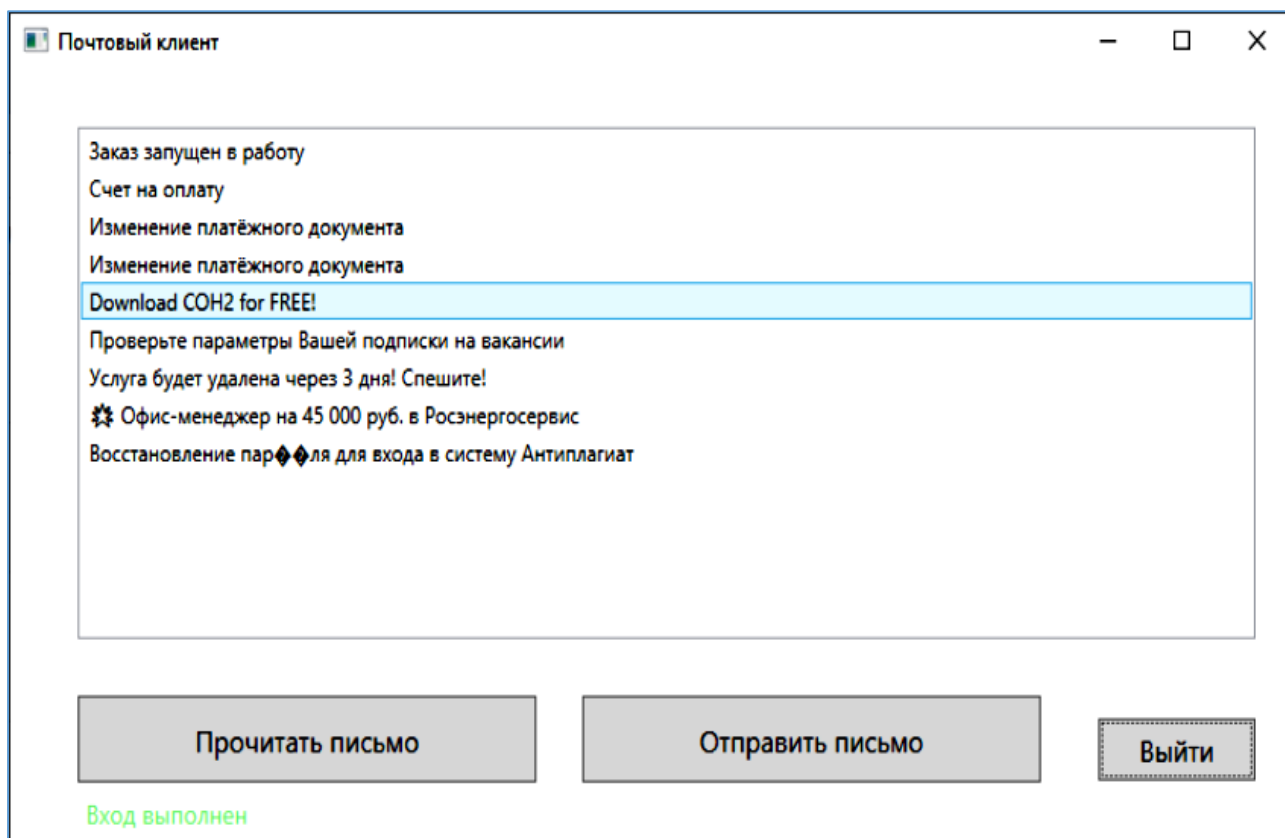


Рисунок 4.3 – Авторизация (Тест №1)

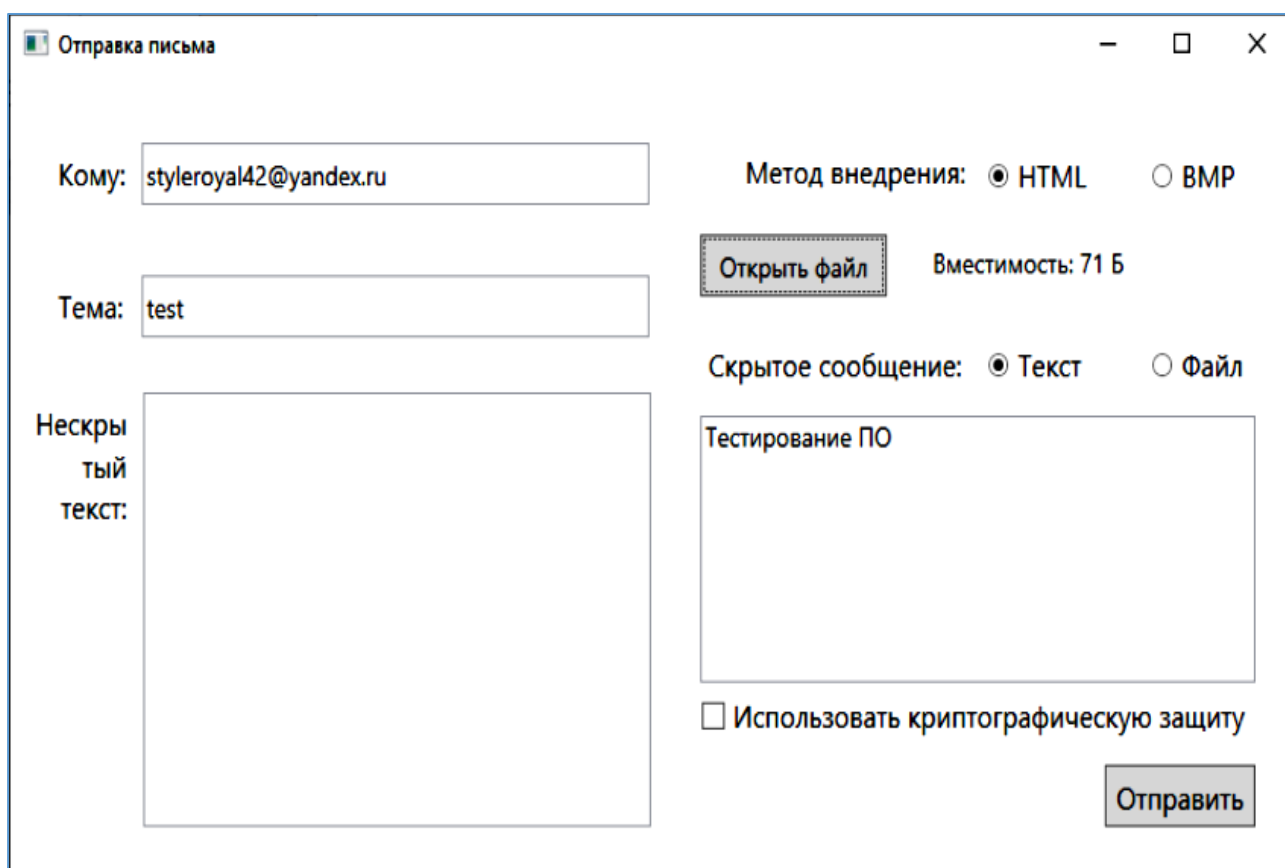


Рисунок 4.4 – Отправка электронного сообщения (Тест №1)

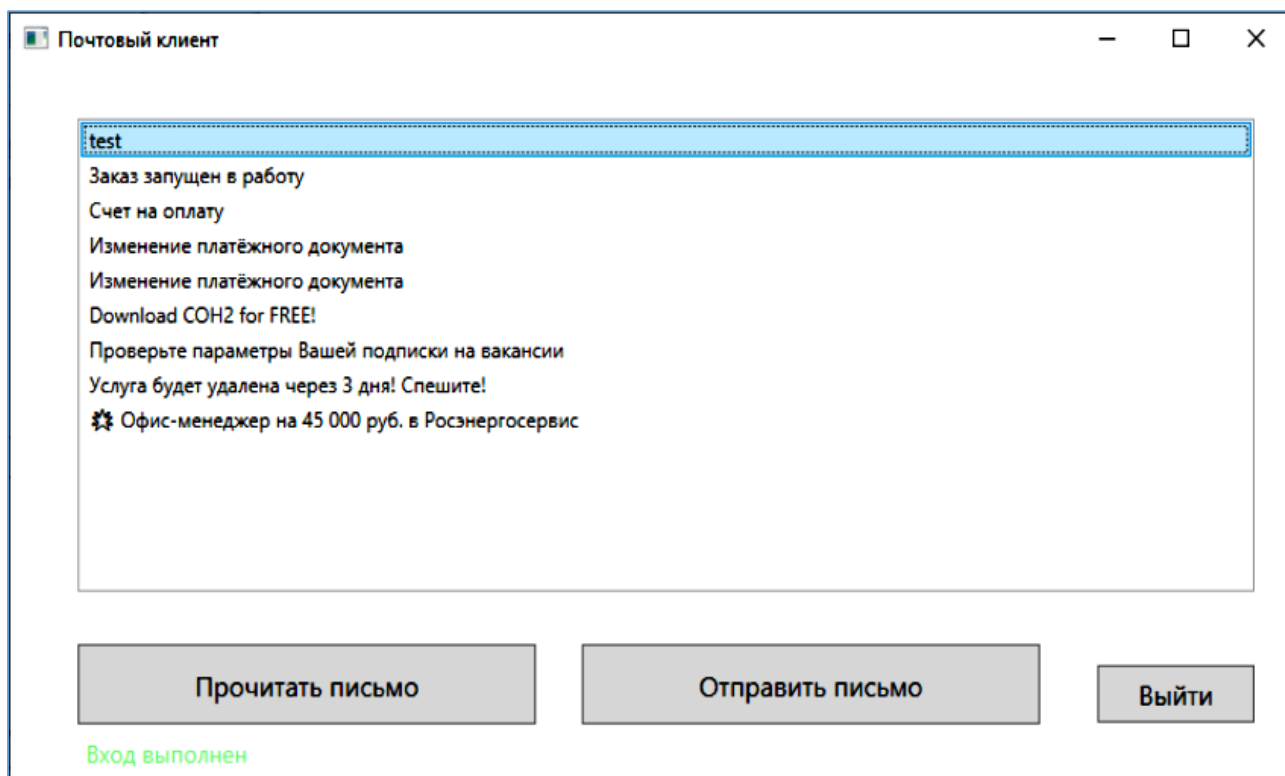


Рисунок 4.5 – Отправленное письмо среди полученных (Тест №1)

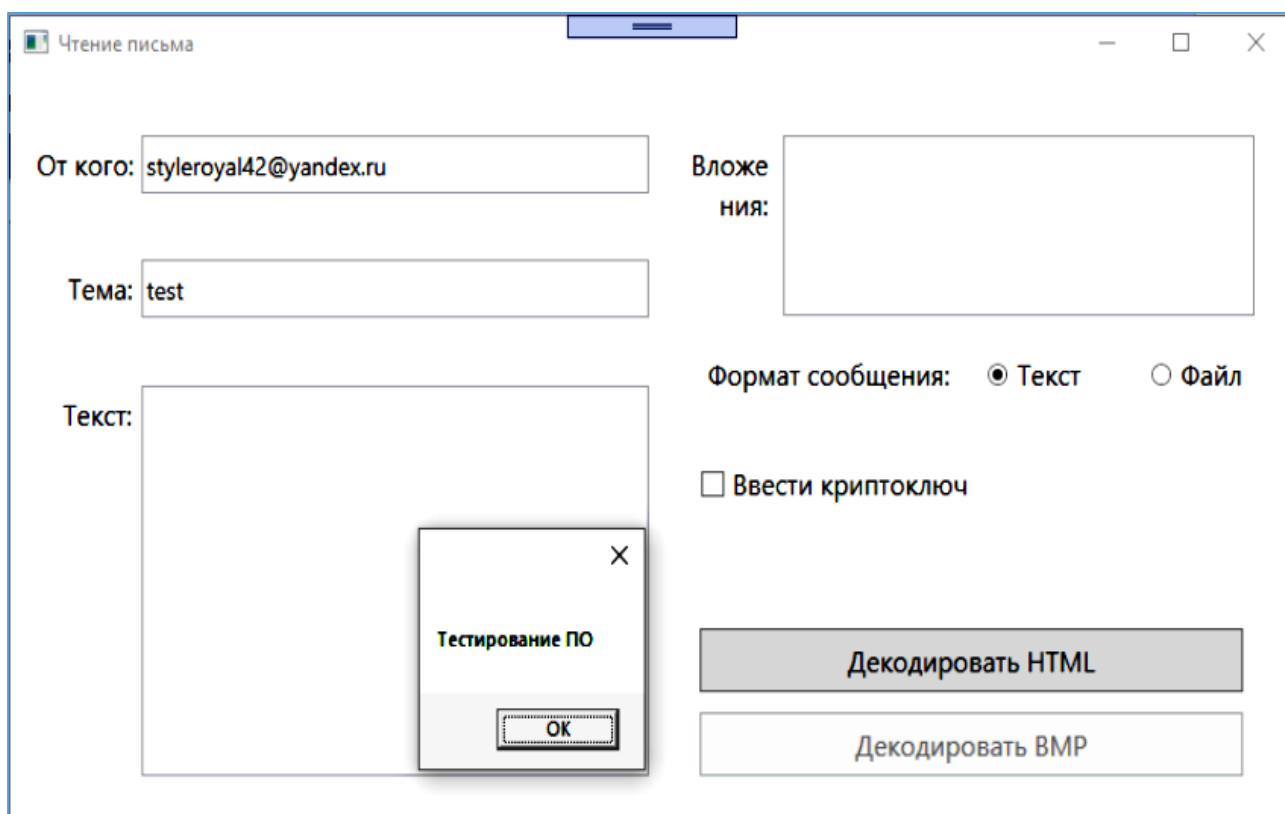


Рисунок 4.6 – Извлечение стегосообщения (Тест №1)

Извлеченное стегосообщение является тем же, что было внедрено в ходе отправки электронного сообщения. Следовательно, тест пройден успешно.

Исходные данные для теста №2:

- контейнер: test.bmp (рисунок 4.7);
- скрываемое сообщение: message.png (рисунок 4.8);
- криптографическая защита: используется,
ключ – 641E69603C55C641B1788E8D443E29B7.



Рисунок 4.7 – test.bmp

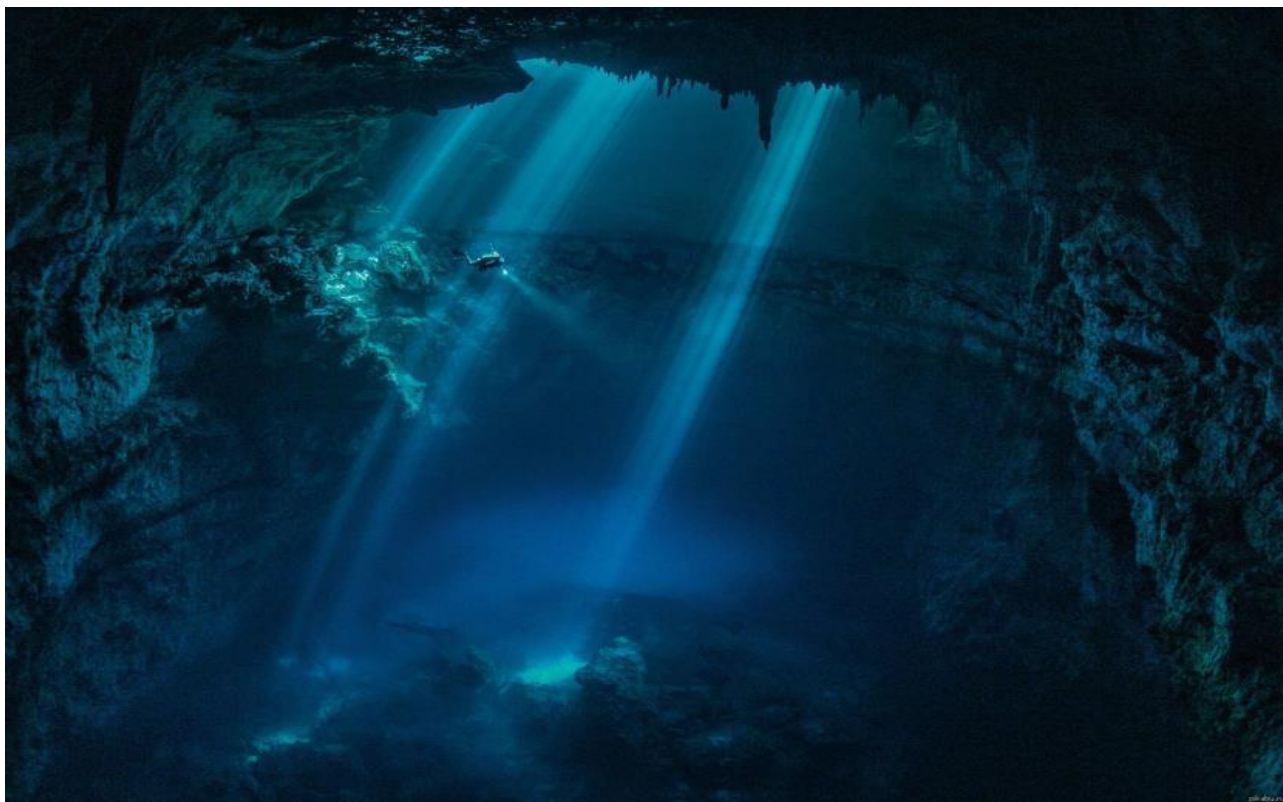


Рисунок 4.8 – message.jpg

Результаты теста представлены на рисунках 4.9 – 4.13.

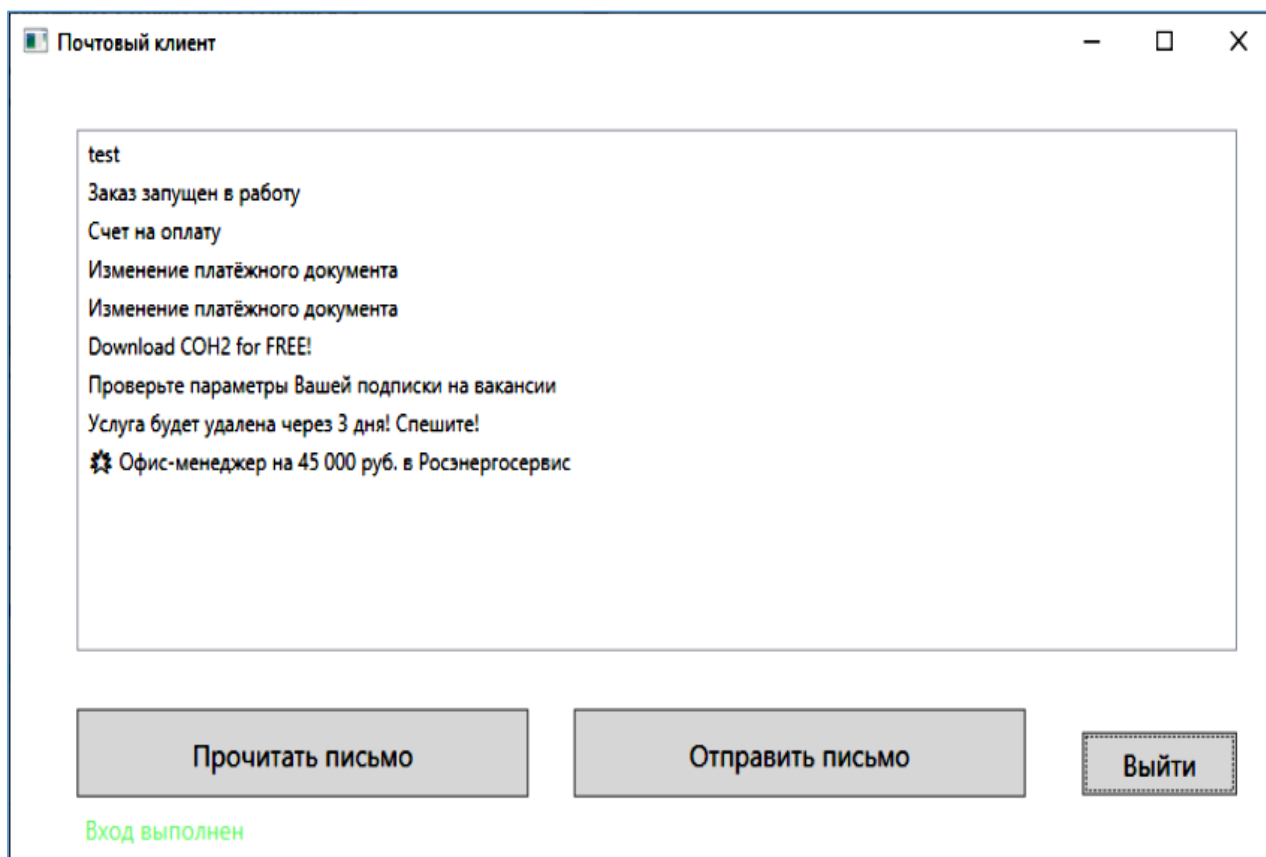


Рисунок 4.9 – Авторизация (Тест №2)

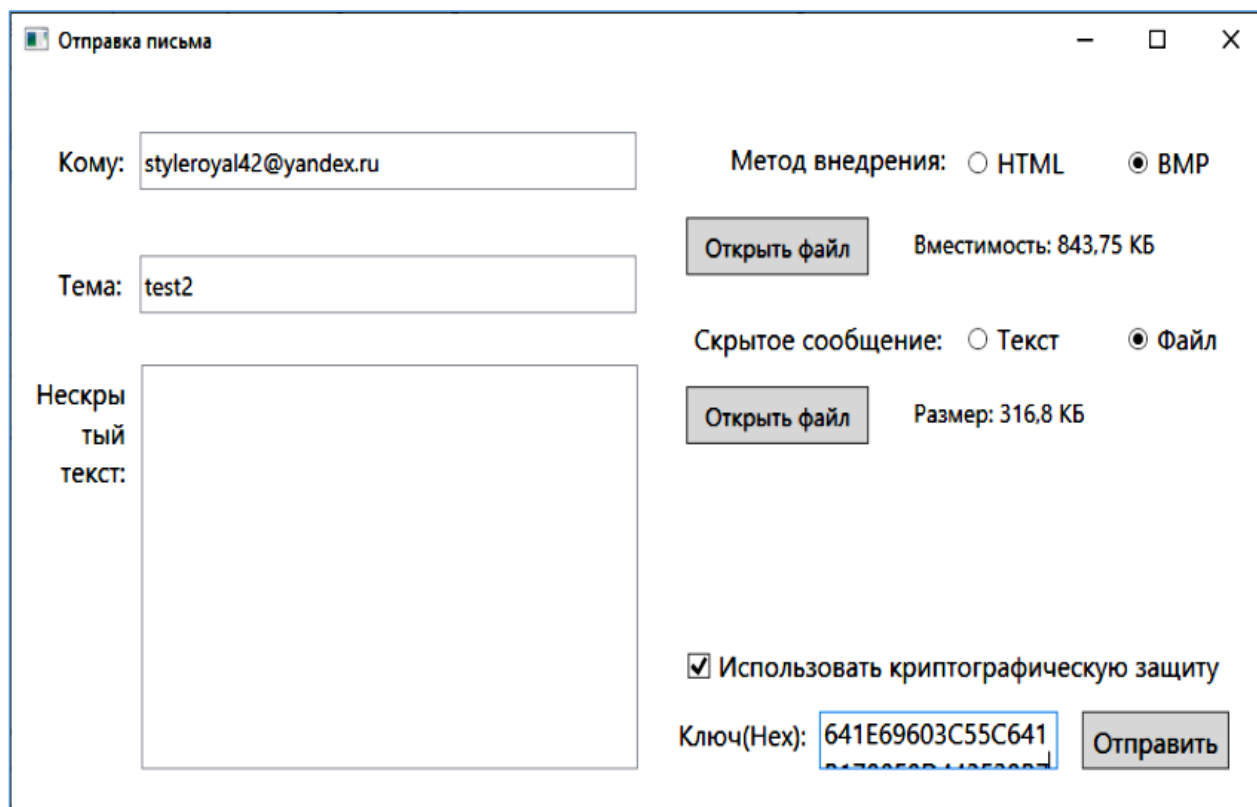


Рисунок 4.10 – Отправка письма (Тест №2)

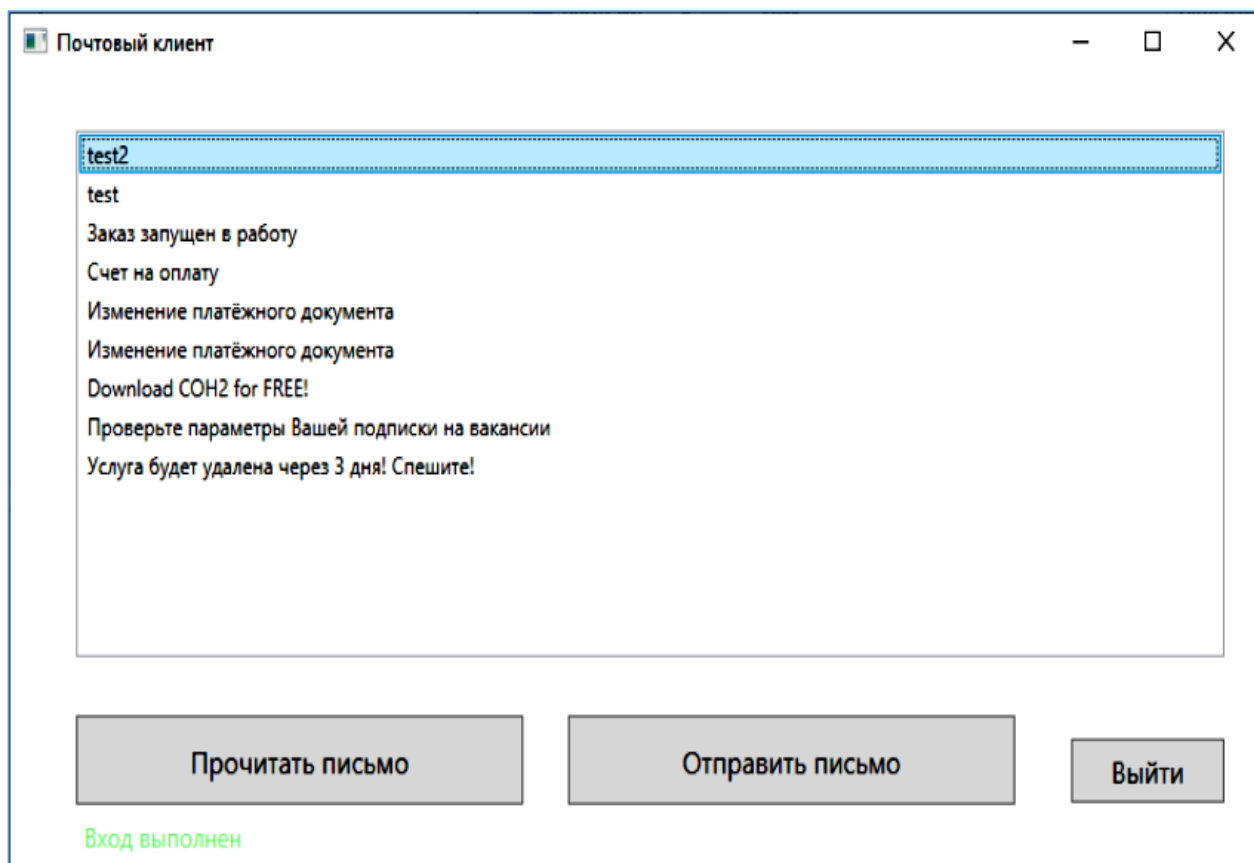


Рисунок 4.11 – Отправленное письмо среди полученных (Тест №2)

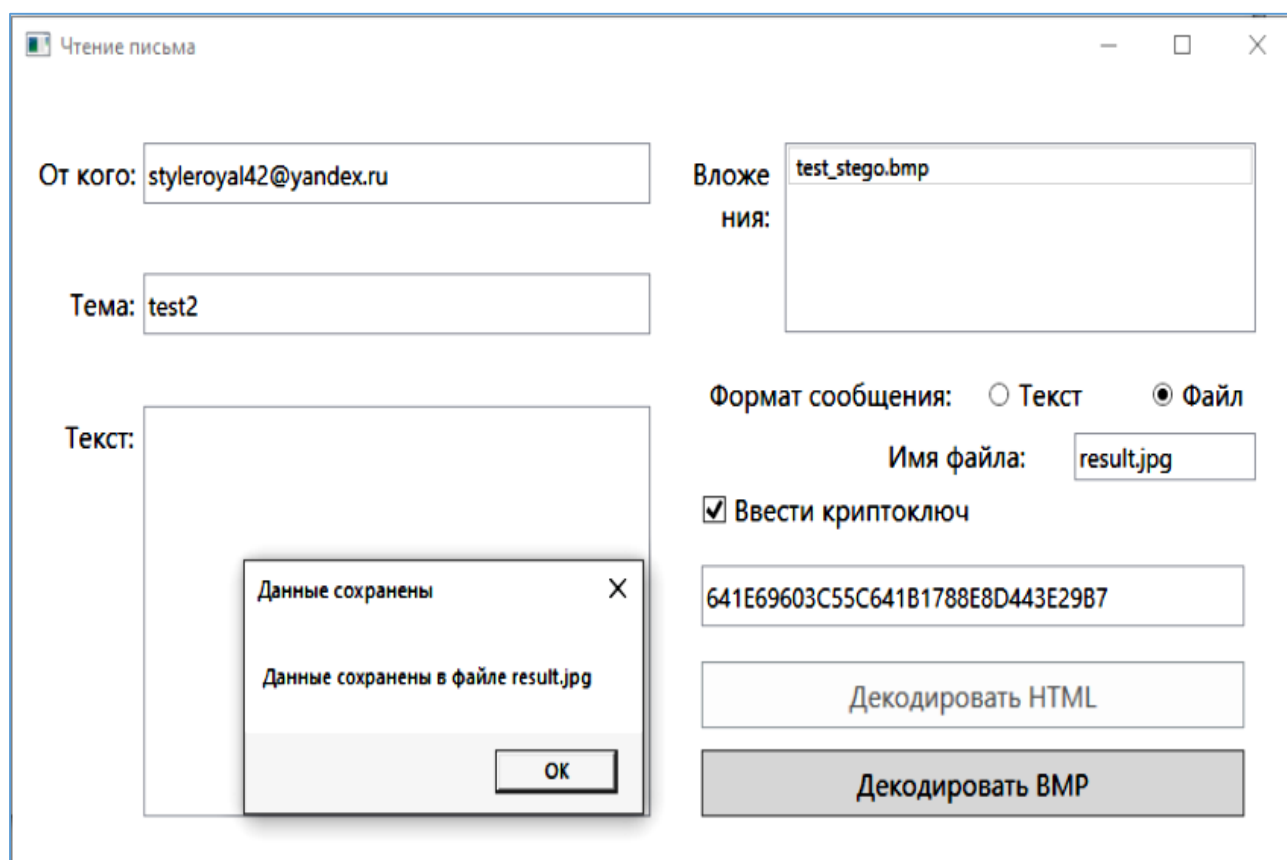


Рисунок 4.12 – Извлечение стегосообщения (Тест №2)

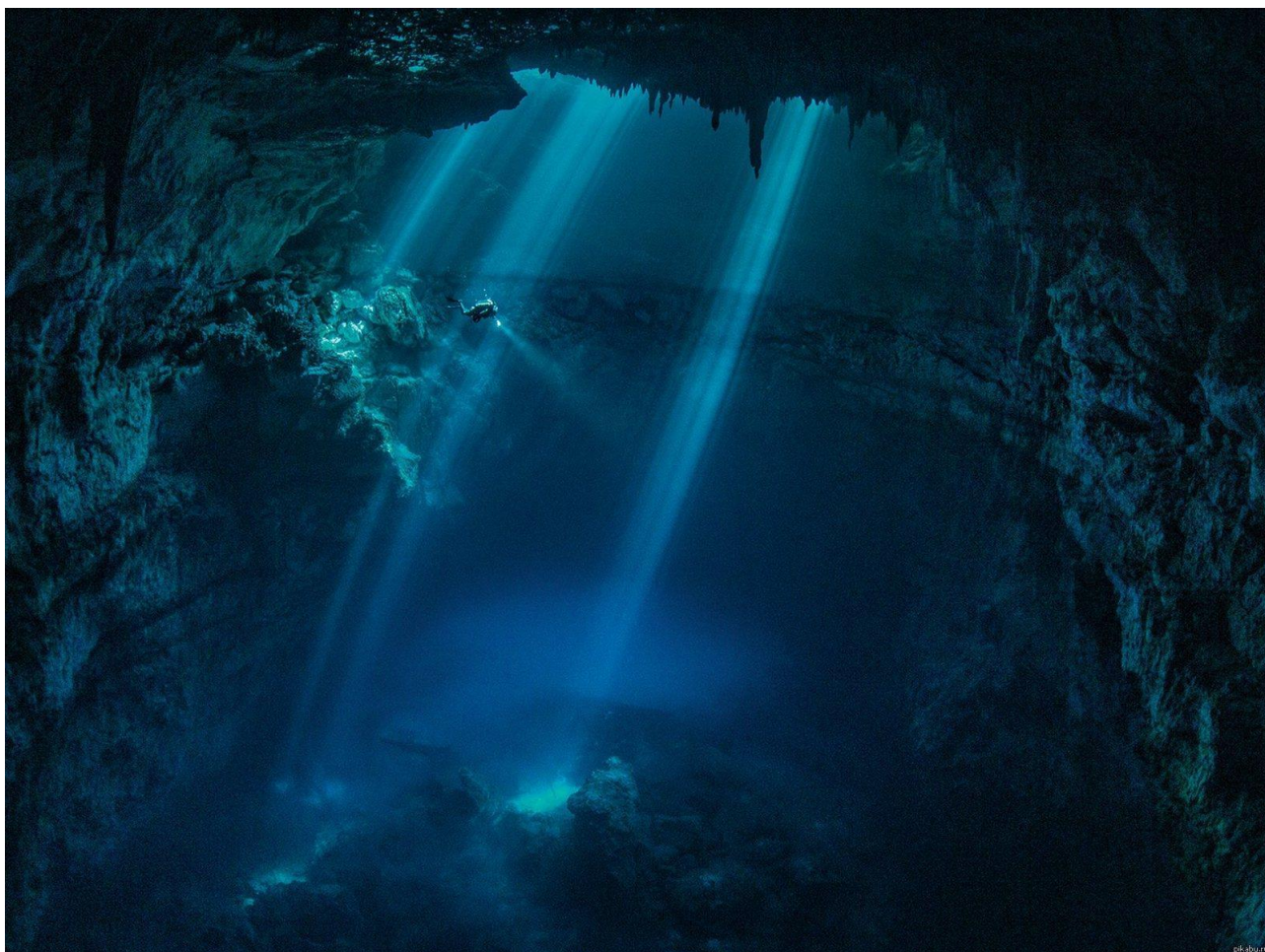


Рисунок 4.13 – result.jpg

Извлеченное изображение соответствует исходному, внедрённому в ходе отправки электронного сообщения, следовательно, тест пройден успешно.

Выводы

В этом разделе были рассмотрены контрольные примеры работы разработанного импортозамещающего программного средства стеганографии, осуществляющую вложение, обнаружение и чтение скрытых сообщений в электронных письмах. Проведено тестирование модулей программного средства с различными исходными данными.

РЕЗУЛЬТАТЫ 10 ГЛАВЫ

В современном мире обеспечение безопасности информации при коммуникации является важным требованием для информационных систем. Чтобы её обеспечить, создаются сложные и комплексные системы безопасной обработки и передачи информации, в которых используются методы как криптографии, так и стеганографии.

Ценность стеганографии проявляется именно в её возможности скрыть факт передачи информации, при этом имеется огромное множество вариантов реализаций стеганографических вложений сообщений со своими критериями и характеристиками, что делает выбор метода сокрытия сообщений очень гибким, в зависимости от необходимых требований.

В ходе работы был создан импортозамещающий почтовый клиент, реализующий скрытие данных в электронных письмах. Дополнительно для него было реализовано криптографическое симметричное шифрование сообщения, с целью обеспечить еще один уровень защищенности информации.

Приложение является объектно–ориентированным, обладает интуитивно–понятным интерфейсом. Обладает возможностью выводить на экран имеющиеся на почте клиента электронные сообщения и производить их чтение. Также позволяет производить отправку писем от имени авторизовавшегося пользователя и внедрять стеговложения в изображения формата .bmp или HTML– документы. Оба варианта позволяют обеспечить скрытность передачи информации при чтении письма без данного приложения или помощи специальных программных средств.

Главным достоинством приложения является простота в использовании и малая нагруженность вычислительных ресурсов.

Стоит отметить, что приложение использует ограниченное количество методов сокрытия информации, но эта проблема решается путем добавления дополнительных классов для реализации других стеганографических алгоритмов. Это несложно осуществить из–за модульной структуры программы.

ЗАКЛЮЧЕНИЕ

Импортозамещение – перспективная тенденция, которая, бесспорно, будет влиять на рынок информационной безопасности еще в течение многих лет. Экономические и политические предпосылки уже сейчас – налицо. Поскольку растёт спрос на импортозамещающие программные продукты, то возрастает готовность производителей их предоставить. И эти продукты становятся все более приближёнными к тем стандартам, которые требовались российскому заказчику давно, но не были доступны ранее в силу ряда причин.

Сегодня российскими продуктами интересуются не только государственные заказчики (которым принятое законодательство не позволяет работать без отечественных импортозамещающих продуктов), но и коммерческий сектор. Успешный переход на отечественные ИБ – продукты является одной из значимых задач в рамках общего тренда по замещению импорта со стороны государства. За последнее время происходит замена зарубежных решений отечественными аналогами. Этот процесс активно проходит в вооруженных силах России, государственных органах, в топливно–энергетическом комплексе, в военно–космической отрасли и других областях критической инфраструктуры.

И если раньше заказчики предпочитали внедрять иностранное программное и аппаратное обеспечение, оборудование и информационные системы, то сегодня на рынке активно возрастает количество инсталляций отечественных программных продуктов, ничем не уступающих зарубежным аналогам. Растёт их популярность у пользователей, чему способствует положительное соотношение цены и качества программных продуктов, а также их лучшая адаптированность к нуждам отечественных заказчиков.

Разработчики программного обеспечения в области информационной безопасности активно регистрируют свои решения в реестре отечественного ПО.

Идея импортонезависимости России приобретает все большее значение. По оценкам аналитиков, импортозамещение стало основным драйвером ИБ – сегмента уже в 2015 г. А в 2016 г. на долю российских компаний уже приходилось около 50% рынка программных продуктов и около 30% продаж в сегменте аппаратных решений. Российские разработчики совершенствуют свои решения, не уступающие по всем основным параметрам известным западным продуктам. На рынке появились новые игроки – наши соотечественники.

Компании различных отраслей реализуют целый ряд проектов на базе отечественных программных продуктов, отвечающих строгим требованиям кибербезопасности информационных систем.

СПИСОК РЕКОМЕНДОВАННОЙ ЛИТЕРАТУРЫ

1. Интерактивная карта киберугроз / [Электронный ресурс]. Kaspersky_URL: <https://cybermap.kaspersky.com/ru/stats/> (дата обращения – 13.09.2019)
2. «Ростелеком» провел новый этап испытаний отечественных решений по квантовой защите передачи данных/портал pro_город_url_ <https://progorod33.ru/news/46882> (дата обращения–13.09.19)
3. IBM создала первый «персональный» квантовый компьютер / CNews URL: https://cnews.ru/news/top/2019-01-08_ibm_predstavila_samyj_kompaktnyj_kvantovyj (дата обращения – 13.09.2019)
4. Импортозамещение в сфере ИТ: вызовы и ответные меры URL: http://www.cioperm.ru/doc_2014/prez_tehnoserv_importzam.pdf (дата обращения – 13.09.2019)
5. Криптографические протоколы. Классификация криптографических протоколов / StudFiles URL: <https://studfile.net/preview/6009392/page:11/> (дата обращения – 15.09.2019)
6. Черемушкин А. В. Криптографические протоколы: основные свойства и уязвимости // ПДМ. Приложение. 2009. №2. URL: <https://cyberleninka.ru/article/n/kriptograficheskie-protokoly-osnovnye-svoystva-i-uyazvimosti> (дата обращения: 21.09.2019).
7. Основные принципы конструирования и анализа криптографических протоколов / Энциклопедия теоретической и прикладной криптографии URL:https://cryptowiki.net/index.php?title=Основные_принципы_конструирования_и_анализа_криптографических_протоколов (дата обращения 21.09.2019).
8. Первые шаги по настройке и использованию SSH // IBM.com URL: <https://www.ibm.com/developerworks/ru/library/au-sshsecurity/> (дата обращения 12.10.2019)
9. Протоколы квантовой криптографии / allbest URL: <https://revolution.allbest.ru/programming/d00330346.html> (дата обращения 4.10.2019)
10. Гриц В.И., Шестаков И.Я., Малышев Д.О. Основные протоколы квантовой криптографии // Решетневские чтения. 2018. №. 1 URL: <https://cyberleninka.ru/article/n/osnovnye-protokoly-kvantovoy-kriptografii> (дата обращения: 29.10.2019).
11. Слепов Н. Квантовая криптография: передача квантового ключа. Проблемы и решения. // Журнал «Электроника»: НТБ. [Электронный ресурс] URL: http://www.electronics.ru/files/article/pdf/0/article_705_722.pdf (дата обращения 4.11.2019).
12. Ashley V., Goppa R. Codes and Their Use in the McEliece Cryptosystems // Syracuse University Honor Program Capstone 2015 URL:http://surface.syr.edu/cgi/viewcontent.cgi?article=1846&context=honors_capstone (дата обращения 11.11.2019)
13. Черемушкин А.В. Криптографические протоколы: основные свойства и уязвимости// ПДМ. Приложение. 2009. №2. URL: <https://cyberleninka.ru/article/n/kriptograficheskie-protokoly-osnovnye-svoystva-i-uyazvimosti> (дата обращения: 11.12.2019).
14. Hoffstein J., Howgrave-Graham N., Pipher J., et all. NTRUEsing: Digital Signature Using the NTRUE Lattice // Burlington 2003 URL:https://web.archive.org/web/20130130160805/https://securityinnovation.com/uploads/Crypto/NTRUSign_RSA.pdf (дата обращения 17.11.2019)
15. Калюжный К. А. Состояние и перспективы импортозамещения в российской ИТ-отрасли // Управление наукой и наукометрия. 2016. №2. URL: <https://cyberleninka.ru/article/n/sostoyanie-i-perspektivy-importozamescheniya-v-rossiyskoy-it-otrasli> (дата обращения: 21.11.2019).

16. Модель угроз и нарушителя безопасности персональных данных, обрабатываемых в специальных информационных системах персональных данных отрасли. Министерство связи и массовых коммуникаций Российской Федерации. Москва, 2019. [Электронный ресурс]. URL: <http://minsvyaz.ru/common/upload/publication/1410084of.pdf> (дата обращения 20.11.2019).
17. Норткат С., Новак Дж. Обнаружение нарушений безопасности в сетях. Третье издание. Перевод с английского: Издательский дом «Вильямс», 2013 – 448 с.
18. Скудис Э. Противостояние хакерам. М.: ДМК Пресс, 2013. — 506 с. — с. 349-370.
19. Терновой О.С. Раннее обнаружение DDOS-атак методами статистического анализа. Перспективы развития информационных технологий. – Новосибирск, изд. «Сибпринт», 2012. 245 с.
20. Ясницкий Д.Л. "Разработка методики раннего обнаружения и отражения распределённых атак типа "отказ в обслуживании"". Магистерская аттестационная работа. Харьков: ХНУРЭ. 2016.
21. Лукацкий А. В. Предотвращение сетевых атак: технологии и решения. – СПб.: Изд-во: «Экспресс Электроника». 2016. – 268 с.
22. Обзор механизмов реализации и обнаружения кибератак [Электронный ресурс]. URL: <http://comp-bez.ru/?p=778>.
23. Wei M., Grupp L., Spada F. “Reliably Erasing Data From Flash-Based Solid State Drives” // FAST'11: 9th USENIX Conference on File and Storage Technologies. 2011. 13 p.
24. Законодательно-правовое и организационно-техническое обеспечение информационной безопасности АС и ИВС // Котенко И.В., Котухов М.М., Марков А.С. и др. СПб.: ВУС, 2000. – 190 с.
25. Gutmann P. “Secure deletion of data from magnetic and solid-state memory”. In SSYM'96: Proceedings of 6th USENIX Security Symposium. Focusing on Applications of Cryptography. 1996. 8 p.
26. Безмалый В.А. Полноценное удаление данных // Windows IT Pro/RE. 2012. [Электронный ресурс] URL: <https://www.osp.ru/winitpro/2012/04/13016769> (дата обращения: 20.10.2019).
27. Лебедеенко Е.Г. Эволюция SSD-накопителей // Железо. 2012. № 11 (104). С. 86—89.
28. Шнайер Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си. — М.: Триумф, 2002. — 816 с.
29. Фергюссон Н., Шнайер Б. Практическая криптография. М.: Диалектика, 2004. 432 с.
30. Вавилов В. Безвозвратное удаление информации // Экспертные библиотеки. 2013. № 8. С. 15–17. [Электрон. ресурс] URL: <http://avicorp.ru/index.php/sample-sites-4/74-erase> (дата обр. 17.10.2019).
31. Малышев. Ю.М. Обзор небольших утилит // iXBT, библиотеки. 2008. № 11. С.4–8. [Электронный ресурс] URL: <https://www.ixbt.com/soft/util1207.shtml> (дата обр. 12.11.2019).
32. Бокиева К. Г. Электронная подпись в России: состояние и перспективы // Молодой ученый. 2018. № 52. С. 185–186. URL: <https://moluch.ru/archive/238/55256/> (дата обращения: 21.11.2019).
33. Diffie W., Hellman M. E. “New Directions in Cryptography” // IEEE Trans. Inf. Theory / F. Kschischang. IEEE, 1976. Vol. 22. Iss. 6. Pp. 644–654. ISSN 0018–9448 doi:10.1109/TIT.1976.1055638.
34. Rivest R., Shamir A., Adleman L. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems” // Commun. ACM. New York City: ACM, 1978. Vol. 21, Iss. 2. P. 120–126. ISSN 0001–0782; 1557–7317. doi:10.1145/359340.359342.
35. Merkle R. Secrecy, authentication, and public key systems: Technical Report No. 1979–1. Citeseer, 1979. doi:10.1.1.637.3952.
36. КриптоПро CSP. // [Сайт ООО «КРИПТО-ПРО»] URL: <https://www.cryptopro.ru/products/cryptopro-csp> (дата обращения: 24.11.2019).

37. КriptoAPM. // [Сайт ООО «КРИПТО–ПРО»] URL: <https://www.cryptopro.ru/products/partner/crypto-arm> (дата обращения: 24.11.2019).
38. Коломыченко М., Парфентьева И. Подпись уходит в облака / М. Коломыченко, И. Парфентьева // РБК. 2019. № 163. 18 октября. URL: <https://www.rbc.ru/newspaper/2019/10/18/5da876a89a79472c0bcd51a3> (дата обращения: 24.11.2019).
39. Черемушкин А.В. Криптографические протоколы: основные свойства и уязвимости. Институт криптографии, связи и информатики. Учебное пособие. М.: Стрингер, 2009. 116 с.
40. Станек У.Р. Microsoft SQL Server 2008. Справочник администратора. М.: Русская Редакция, 2009. 336 с.
41. Accessing the SQL Server Virtual Device Interface via .NET (C#) // [Персональная страница S. Abraham] // CodeProject 2007. URL: <https://www.codeproject.com/Articles/19449/Accessing-the-SQL-Server-Virtual-Device-Interface> (дата обращения: 14.11.2019)
42. Руководство пользователя PackDB // forosoft 2018 URL: https://sparesync.com/files/PackDb_Manual-ru.pdf (дата обращения: 25.11.2019).
43. Родичев Ю.А. Нормативная база и стандарты в области информационной безопасности. Учебное пособие. Санкт–Петербург: Издательство «Питер», 2017. 256 с.
44. Баранова Е.К. Информационная безопасность и защита. Учебное пособие. М: Издательство «РИОР, Инфра –М». 2017. 324 с.
45. Бондарев В.В. Введение в информационную безопасность автоматизированных систем. Учебное пособие. М: Издательство «МГТУ им. Н. Э. Баумана» 2016. 252 с.
46. Нестеров, С.А. Основы информационной безопасности. Учебное пособие. Санкт–Петербург: Издательство «Лань». 2016. 324 с.
47. Комаров А. Современные методы аутентификации: токен // Т–Comm. 2008. С. 25–30.
48. Мушовец К.В., Метусало М.П., Золотарев В.В. Применение протоколов аутентификации пользователей для защиты каналов спутниковой связи // Актуальные проблемы авиации и космонавтики. 2010. № 6. С. 53 – 61.
49. Никешин А.В., Шнитман В.З. Обзор расширяемого протокола аутентификации и его методов. Труды ИСП РАН. том 30 вып. 2. 2018. С. 113 – 148.
50. IETF RFC 3748. В. Aboba, etc. Extensible Authentication Protocol (EAP). June 2004. Доступно по ссылке: <https://tools.ietf.org/html/rfc3748>.
51. Laponina O.R. “Fundamentals of network security. Cryptographic algorithms and protocols of interaction. Lecture course”. Tutorial. Internet University of Information Technologies. 2005. 608 p.
52. Беккер М.Я., Терентьев А.О., Гатчин Ю.А., и др. Использование цифровых сертификатов и протоколов SSL / TLS для шифрования данных при облачных вычислениях // Науч.–техн. вестник информационных технологий, механики и оптики. 2011. № 4 (74). С. 83–91.
53. The Transport Layer Security (TLS) Protocol Version 1.1. The Internet Engineering Task Force [Электронный ресурс]. Режим доступа: <http://datatracker.ietf.org/doc/rfc4346/>.
54. Гапанович Д.А., Чубариков В.Н. Хэш–алгоритм с управляющей древовидной структурой и метод его реализации на параллельных архитектурах // Современные информационные технологии и ИТ–образование. 2017. №1. С. 15 – 23.
55. Порублева Е.С., Гареева Г.А., Григорьева Д.Р. Электронные деньги и электронные платежные сервисы в России в XXI веке // Символ науки. 2018. № 1 – 2.

56. Бобров А.В., Семёнов Д.А. Повышение надёжности сервера // Вестник науки и образования. 2018. № 16 – 2 (52). С. 8 – 21.
57. Мусалаева С.А. Электронные деньги и платежные системы // ПСЭ. 2010. № 4. С. 21–28.
58. Введение в «Цифровую» экономику / А.В. Кешелава [и др.]. – М.: ВНИИ Геосистем, 2017.
59. Гуфан К. Ю., Иванков М. П., Хади Р. А. О требованиях к системам защиты электронных платежей от технического мошенничества // Известия ЮФУ. Технические науки. 2006. № 16.
60. Шапиро Л. Компьютерное зрение: учебное пособие. М.: Бином, 2006. 742 с.
61. Форсайт Д. Компьютерное зрение. Современный подход. М.: Вильямс, 2004. 752 с.
62. Лукьяница А.А. Цифровая обработка видеоизображений. М.: Ай-Эс-Эс Пресс, 2009. 518 с.
63. Желтов С.Ю. Обработка и анализ изображений в задачах машинного зрения: учебное пособие. М.: Физматкнига. 2009. – 518 с.
64. Журавлев Ю.И. «Распознавание». Математические методы. Программная система. Практические применения: учебное пособие. М.: Фазис, 2006. 127 с.
65. Harris R. Corner detection. [Электронный ресурс]: OpenCV Documentation. URL: http://docs.opencv.org/3.0beta/doc/py_tutorials/py_feature2d/py_features_harris/ru_features_harris.html (дата обращения 15.12.2019).
66. Флах П. Машинное обучение: учебное пособие. М.: ДМК Пресс, 2015. 400 с.
67. Чен Ш. Принципы проектирования систем визуальной информации. / М.: Мир. 2014. – 408 с.
68. Грибунин, В. Г. Цифровая стеганография. М.: СОЛОН – Пресс, 2012. 271 с.
69. Конахович Г.Ф. Компьютерная стеганография. Практика. К.: «МК–Пресс», 2016. 288 с.
70. Хорошко В.О. Основы компьютерной стеганографии: уч. пособие для студентов и аспирантов / В.О. Хорошко, О.Д. Азаров, М.Э. Шелест. — М: МИФИ, 2003. — 143 с.
71. Алиев А.Т., Аграновский А. В. Вопросы построения криптостеганографических систем. Модель стеганографического канала передачи данных // Известия ТРТУ. Тематический выпуск. «Информационная безопасность». Таганрог: Изд-во ТРТУ. 2006. № 7 (62). С. 185—192.
72. Пескова О.Ю., Халабурда Г.Ю. Применение сетевой стеганографии для защиты данных, передаваемых по открытым каналам Интернет // ЮФУ. Ростов–на–Дону: Электронные библиотеки и развитие технологий информационного общества, 2012. С. 348 — 354.
73. Pfitzmann B. “Information Hiding Technology”. In R. Anderson, Information Hiding: first international workshop, Proceedings. Vol. 1147, Berlin: Springer, 1996.
74. Рябко Б.Я. Криптографические методы защиты информации: учеб. пособие для вузов / А.Н. Фионов. 2–е изд. М.: Горячая линия. Телеком, 2012. 229с.
75. Голубев Е.А., Емельянов Г.В. Стеганография как одно из направлений обеспечения информационной безопасности // Технологии информационного общества. Спецвыпуск Т – Comm, 2009. С. 185–186.
76. Хорошко В.А. Методы и средства защиты информации. К.: Юниор, 2013. – 504 с.
77. Abdikarim A., Tütüncü K. “New Approach in E-mail Based Text Steganography” // International Journal of Intelligent Systems and Applications in Engineering, 2018.
78. Shukran H., Mohd Z., Omar S., et al. // Detection of Steganographic Messages in Email Attachment. Modern Applied Science. 6.10. 2012.
79. Шарп Д. Microsoft Visual C#. СПб. Питер. Библиотека программиста, 2017. 848 с.
80. Галатенко В.А. Основы информационной безопасности. ИНТУИТ. РУ "Интернет–университет информационных технологий", 2016. 208 с.

ОГЛАВЛЕНИЕ

Введение	3
Глава 1. Исследование современных криптографических протоколов и возможностей их импортозамещения	6
Глава 2. Разработка и анализ эффективного импортозамещающего средства противодействия DDoS–атакам	49
Глава 3. Импортозамещение программного монитора безопасности для отражения кибератак типа «Удалённый доступ»	110
Глава 4. Импортозамещающее программное средство безвозвратного гарантированного уничтожения критически важной информации для пользователя компьютерной системы	143
Глава 5. Разработка импортозамещающего программного средства формирования электронной подписи при организации электронного документооборота отечественного учреждения	170
Глава 6. Реализация импортозамещающего криптографического алгоритма для шифрования данных в СУБД SQL Server	206
Глава 7. Разработка импортозамещающей системы разграничения доступа для работы с программным средством, построенным по микросервисной архитектуре для защиты информации от несанкционированного доступа	243
Глава 8. Разработка импортозамещающей системы безопасной аутентификации пользователей электронных платежных систем Secret Token	274
Глава 9. Создание импортозамещающей системы обнаружения и распознавания лица человека в кадре с целью авторизации пользователей с помощью видеокамеры	305
Глава 10. Разработка импортозамещающего программного приложения почтового клиента со скрытой передачей информации на основе стеганографических методов	345
Заключение	390
Список рекомендованной литературы	391
Оглавление	395

Учебное издание

**Короченцев Денис Александрович
Черкесова Лариса Владимировна
Ревякина Елена Александровна
Болдырихин Николай Вячеславович
Сафарьян Ольга Александровна**

**ИМПОРТОЗАМЕЩАЮЩИЕ ТЕХНОЛОГИИ
ОБЕСПЕЧЕНИЯ ИНФОРМАЦИОННОЙ
БЕЗОПАСНОСТИ И ЗАЩИТЫ ДАННЫХ**

Учебное пособие

Под общей редакцией д.ф.–м.н., проф. кафедры КБИС Черкесовой Л.В.

Редактор:

Компьютерная обработка: **только не И.В. Кикичева !!!**

Тем. План 2020

В печать:

Объём ___ усл. печ. листов. Формат ___

Заказ № ____ . Тираж 100 экз. Цена свободная

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344000, г. Ростов–на–Дону, пл. Гагарина, 1